

EXPERT INSIGHT

Machine Learning for Trading

A disciplined workflow from research to live execution,
with nine case studies and AI agents

Foreword by:

Antonio Gulli,
Sr. Engineering Director
Distinguished Engineer
Office of the CTO, Google

Third Edition



Stefan Jansen

<packt>

Machine Learning for Trading

Third Edition

A disciplined workflow from research to live execution, with nine case studies and AI agents

Stefan Jansen



Machine Learning for Trading

Third Edition

Copyright © 2026 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Portfolio Director: Sunith Shetty

Relationship Lead: Tushar Gupta and Apeksha Shetty

Project Manager: Shashank Desai

Content Engineer: David Sugarman

Technical Editor: Seemanjay Ameriya

Copy Editor: David Sugarman

Indexer: Tejal Soni

Production Designer: Ganesh Bhadwalkar

Growth Lead : Anjitha Murali

First published: December 2018

Second edition: July 2020

Third edition: July 2026

Production reference: 1160726

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-80324-697-0

www.packt.com

*I would like to dedicate this book to Mariana and Bastian, who are my
source of happiness and who put up with me and the many hours I
missed while I wrote this book.*

– Stefan Jansen

Foreword

I've read a lot of quant finance books, and most of them fall into the same trap: pile on the algorithms, sprinkle in some market data, and call it a day. Stefan Jansen's third edition of *Machine Learning for Trading* doesn't do that. It's grounded in a way that's almost refreshing—more of a working practitioner's notebook than a textbook. The whole thing assumes markets are messy, non-stationary, and out to ruin your backtest, and it builds from there.

A few things stuck with me.

The book cares about workflow more than algorithms. Jansen treats ML in trading as a lifecycle, not a model hunt. There's a clear line drawn between building your data infrastructure and the day-to-day grind of researching strategies. That separation sounds obvious until you realize how many people skip it.

The data chapters don't pull punches. The section on data quality is the part I'd hand to anyone starting out. Point-in-time errors, survivorship bias, the headache of corporate actions—Jansen walks through exactly how each one fabricates “alpha” that was never really there.

Microstructure is explained like it matters, because it does. The breakdown of the limit order book and how prices actually form connects the tidy theory to the ugly reality of execution. Spreads, queue position, the friction that quietly eats your returns—it's all here, and it changes how you think about whether a strategy is even tradable.

Alternative data gets sober treatment. Instead of breathless hype, you get a checklist: Is there real signal? Is the data clean? Are there legal

landmines? What's it going to cost to engineer? Useful questions to ask before you sink three months into a dataset.

The synthetic data chapter is the one I keep thinking about. GANs, diffusion models, LLMs—all aimed at the quant's oldest problem: we only get one history to test against. Jansen is genuinely curious about generating more, but he never lets you forget that a generator's biases become your biases.

Three ideas actually shifted how I think:

Process beats the “perfect” model. Markets drift, break, and shift regimes. The edge isn't some pristine model you discover once—it's a research process disciplined enough to notice when your signal is decaying and flexible enough to change course.

Lookahead bias quietly kills everything. Future information sneaks into the past in ways you won't see: restated macro numbers, ignored SEC filing lags, a universe filtered down to the companies that happened to survive. Each one prints fake profits that vanish the moment real money is on the line.

Clock time is the wrong clock. A one-minute bar at the open and a one-minute bar at lunch are not the same animal, because activity isn't constant. Sampling by volume, dollars, or imbalance instead gives you return distributions that are far better behaved—and far better food for a model.

I definitely recommend this book if you want to understand Finance in the modern AI and Agentic world. It's a definitive guide.

Antonio Gulli

*Sr. Engineering Director, Distinguished Engineer, Office of the CTO,
Google*

Contributors

About the author

Stefan Jansen is the founder of Applied AI and the author of *Machine Learning for Trading*. For over a decade, he has built production machine learning systems across finance, insurance, and healthcare—from contract intelligence that turns legacy insurance documents into structured logic; to forecasting at scale; to live trading infrastructure; and beyond, into LLMs and agents. He maintains the open-source `ml4t-*` libraries and the book’s companion repository, which has drawn more than 19,000 GitHub stars. He holds a master’s in economics from Harvard, an MS in computer science from Georgia Tech, and the CFA charter.

Writing a book is never a solo effort. My deepest thanks go to my friends and family, who supported me throughout and put up with the long hours this book demanded. I am grateful, too, to the team at Packt who brought this third edition to life: to David Sugarman, who edited the manuscript, and to Apeksha Shetty, Shashank Desai, and Yash Basil, who guided the project through to publication—along with everyone in editorial, production, and design whose work turned a draft into a finished book. My thanks also to the technical reviewer for feedback that improved the book, and to Antonio Gulli for his generous foreword.

About the reviewer

Haitam Borqane is an AI engineer, open source advocate, and huge coffee enthusiast. He has a vast array of experience in AI application across a variety of fields, from sustainability and agriculture to finance and innovation. Professionally, he develops and deploys robust AI solutions and systems for clients to enhance their processes; on the side, he is an AI fanatic who contributes to open source models for low resource languages, while also raising awareness on the state of AI across different conferences. When not working, you can find him at your nearest cozy specialty coffee spot.

I would like to thank the Packt team and author for this opportunity on this amazing book, and I would like to thank my parents and my sister for their constant support.

Contents

[Preface](#)

[A book and a living repository](#)

[How each chapter maps to the repository](#)

[The companion ecosystem at ml4trading.io](#)

[Who this book is for](#)

[What this book covers](#)

[To get the most out of this book](#)

[Download the example code](#)

[Get in touch](#)

[Free benefits with your book](#)

1. [The Process Is Your Edge](#)

[1.1 Why process discipline matters](#)

[Four shockwaves and the fragility of assumptions](#)

[A vocabulary for change](#)

[Why process trumps models](#)

[Process as a research-to-production system](#)

[1.2 Introducing the ML4T workflow](#)

[Data infrastructure](#)

[Research and evidence framework](#)

[Feature engineering and model development](#)

[Strategy design from signals to trades](#)

[Deployment and monitoring](#)

[The evidence boundary](#)

[1.3 Causal inference and generative AI in the workflow](#)

[Two practical starting points and two deliverable types](#)

[Causal inference as a discipline-enforcing tool](#)

[Expanding capability and scaling risk with generative AI](#)

[1.4 Keeping up with changing market regimes](#)

[Regime detection with labeling, conditioning, and monitoring](#)

[Style regimes with century-long factor data](#)

[What the model estimates](#)

[Macro regimes and volatility environments](#)

[Market validation – Volatility and drawdowns](#)

[Real-time detection and risk management](#)

1.5 Independent versus institutional workflows in the real world

Solo failure modes that institutions partially avoid

Decision discipline through documentation and checkpoints

Scoping check (implementability first)

Data integrity check

Signal check (robustness before cleverness)

Tradability check (fragility audit)

Monitoring check (diagnosis maps to action)

Constraints and asymmetric advantages

Compounding returns with reusable infrastructure

1.6 Summary

2. The Financial Data Universe

2.1 A Modern taxonomy of financial data

Market data – A hierarchy of aggregation

Fundamental data – Drivers of value with release lags

Alternative data – High variety, high validation burden

2.2 The asset-class market data landscape

Equities

What you observe

Failure modes

Engineering decisions

Datasets

Exchange-traded products

What you observe

Failure modes

Engineering decisions

Futures

What you observe

Failure modes

Engineering decision

Datasets

Options

What you observe

Failure modes

Engineering decision

Datasets

Digital assets

What you observe

- [Failure modes](#)
 - [Engineering decision](#)
 - [Datasets](#)
- [Foreign exchange](#)
 - [What you observe](#)
 - [Failure modes](#)
 - [Concrete example](#)
 - [Engineering decision](#)
 - [Datasets](#)
- [Fixed income](#)
 - [What you observe](#)
 - [Failure modes](#)
 - [Engineering decision](#)
- [Swaps and OTC derivatives \(rates, credit\)](#)
 - [What you observe](#)
 - [Failure modes](#)
 - [Engineering decision](#)
- [Commodities](#)
 - [What you observe](#)
 - [Failure modes](#)
 - [Engineering decision](#)
 - [Microstructure tooling applicability](#)
- [2.3 A due diligence framework for data sourcing](#)
 - [General data quality dimensions](#)
 - [Finance-specific failure modes](#)
 - [Point-in-time correctness](#)
 - [Survivorship bias](#)
 - [Corporate actions](#)
 - [Identifier integrity](#)
 - [The vendor ecosystem](#)
 - [Vendor due diligence checklist](#)
 - [Data quality](#)
 - [Legal and compliance](#)
 - [Technical and commercial](#)
 - [Minimum viable data validation checklist](#)
 - [Data governance](#)
- [2.4 Storing data](#)
 - [What we benchmark and why](#)
 - [Benchmark context and caveats](#)
 - [File-based storage](#)

[Embedded analytics databases](#)

[Server time-series databases](#)

[ASOF join performance](#)

[Strategic decision framework](#)

[2.5 Summary](#)

[3. Market Microstructure](#)

[3.1 How microstructure impacts price formation](#)

[The frictions faced by liquidity providers](#)

[How order types express intent in the limit order book](#)

[Market design and intraday regimes](#)

[3.2 The anatomy of modern market data feeds](#)

[The data hierarchy – From best quote to full order book](#)

[Level 1 \(L1\): Top-of-book quotes](#)

[Level 2 \(L2\): Market-by-price depth \(MBP\)](#)

[Level 3 \(L3\): Market-by-order messages \(MBO\)](#)

[Trades and quotes – The standard data package](#)

[Price conventions in this chapter](#)

[Sample datasets – From MBO to enriched bars](#)

[Message protocols – From human-readable to fast binary](#)

[Data integrity challenges](#)

[3.3 From raw messages to the limit order book](#)

[NASDAQ TotalView-ITCH – A case study](#)

[Binary parsing at scale](#)

[The LOB state machine](#)

[Maintaining book integrity](#)

[Single-exchange data – A local view](#)

[Snapshots and analysis](#)

[Empirical findings](#)

[LOB stylized facts – Predictive patterns](#)

[3.4 The art of sampling](#)

[Standard aggregation methods](#)

[Time bars](#)

[Tick, volume, and dollar bars](#)

[Information-driven bars](#)

[Trade classification – Data sources matter](#)

[The Lee-Ready algorithm](#)

[Empirical validation – Classification accuracy](#)

[Volume imbalance bars](#)

[Tick imbalance bars](#)

[Run bars](#)

- [Calibration – Avoiding threshold spiral](#)
 - [Statistical properties comparison](#)
 - [Trade-offs and selection](#)
 - [3.5 Detecting price jumps in intraday returns](#)
 - [Continuous and jump variance](#)
 - [The Lee–Mykland Test](#)
 - [What the data show](#)
 - [Why a local volatility adjustment matters](#)
 - [Jumps as features](#)
 - [3.6 Microstructure data quality and sessionization](#)
 - [3.7 Summary](#)
 - 4. [Fundamental and Alternative Data](#)
 - [4.1 The point-in-time pipeline](#)
 - [Why restatements create leakage](#)
 - [Bitemporal storage and as-of queries](#)
 - [Timestamp authority by modality](#)
 - [The EDGAR sourcing pipeline](#)
 - [XBRL and taxonomy structure](#)
 - [Handling data inconsistency](#)
 - [4.2 Entity resolution and mapping](#)
 - [Why resolution is critical](#)
 - [A hierarchical resolution approach](#)
 - [Stage 1: Deterministic matching](#)
 - [Stage 2: Probabilistic matching](#)
 - [Stage 3: Manual review and QA](#)
 - [Building a master security database](#)
 - [Temporal entity mapping](#)
 - [Mapping quality assurance](#)
 - [4.3 Fundamentals across the asset-class spectrum](#)
 - [Macro and sovereign data \(FX/Rates\)](#)
 - [Key sources](#)
 - [Alignment challenges](#)
 - [Commodities – Physical market data](#)
 - [Energy and agriculture](#)
 - [Mapping to tradable instruments](#)
 - [Cryptocurrency and digital assets – On-chain fundamentals](#)
 - [Core on-chain metrics](#)
 - [Data sources](#)
 - [AMM and DEX data](#)

4.4 Understanding alternative data

The alternative data landscape

The evaluation framework

Build versus buy for alternative data

Data sources and implementation

4.5 Using text data for NLP features

Targeting MD&A and risk factors

The engineering pipeline

Step 1: Filing access and metadata

Step 2: Section extraction

Step 3: Cleaning and normalization

Step 4: PIT-correct storage

Output artifact

4.6 Summary

5. Synthetic Financial Data

5.1 The quant's dilemma

The multiple testing challenge

Synthetic data as simulation infrastructure

5.2 Evaluating synthetic financial data

Does it preserve the relevant data structure?

Does it support the intended use case?

Does it leak training information?

Synthetic-specific failure modes

Practical validation checklist

5.3 Classical simulation baselines

Bootstrap methods

Parametric price and volatility models

Limitations and how to use these baselines

5.4 Generative model taxonomy

5.5 GANs for financial time series

Using TimeGAN for sequential data

Architecture

Training

Data requirements

When to use TimeGAN

Using Tail-GAN for risk-constrained generation

Portfolio focus

When to use Tail-GAN

Using Sig-CWGAN for path signatures for temporal fidelity

[Using GT-GAN for irregular time series](#)
[GAN training challenges](#)

[5.6 Diffusion models for financial time series](#)

[The denoising diffusion framework](#)

[Why diffusion models fit financial returns](#)

[Conditional generation for regime stress testing](#)

[Applying Diffusion-TS with interpretable decomposition](#)

[Choosing between GANs and diffusion models](#)

[5.7 LLMs for structured financial data](#)

[The GReaT framework](#)

[Financial applications](#)

[Limitations and risks](#)

[5.8 Applying the Fidelity–Utility–Privacy framework](#)

[Reading the Diffusion-TS results](#)

[Privacy–utility trade-off with DP-GAN](#)

[5.9 Summary](#)

[6. Strategy Research Framework](#)

[6.1 From idea to evidence with the ML4T workflow](#)

[The live trading loop](#)

[The research loop](#)

[Illustrating the research framework with case studies](#)

[6.2 Mapping strategies and sources of edge](#)

[Strategy families as feasibility filters](#)

[Price-based strategies](#)

[Fundamental and valuation strategies](#)

[Flow and microstructure strategies](#)

[Market mechanics and payoff strategies](#)

[Defining edge, alpha, and capacity](#)

[Sources of edge as durability filters](#)

[Slow adjustment and behavioral responses](#)

[Risk compensation](#)

[Market segmentation and limits to arbitrage](#)

[Mechanical and institutional flows](#)

[Information and measurement advantages](#)

[Using the map](#)

[6.3 Defining the trading setup](#)

[What the trading setup must fix](#)

[Universe and tradability rules](#)

[Decision schedule and admissible information](#)

[Cadence and horizon feasibility](#)

Score-to-trade mapping

Constraints and risk controls in scope

Cost model class and components

When a change requires a new setup version

6.4 Setting objectives and evaluation metrics

Three metric layers, each with a different role

6.5 Evaluation protocol for time series

Data leakage and estimation bias

From k-fold cross-validation to time series evaluation

Walk-forward cross-validation as the baseline

Retraining frequency

Temporal buffers – Label buffer and feature buffer

Holdout test set and rolling retuning

Combinatorial methods and path dependence

Design commitments

6.6 Establishing a baseline checkpoint

Three preflight sanity checks

The first reference run is narrow by design

6.7 Search accounting and run logging

Trial taxonomy

Minimum contents of the run log

6.8 Summary

7. Defining the Learning Task

7.1 Data preprocessing and encodings

Split-aware preprocessing

Outliers and heavy tails

Scaling and representation choices

Handling missing data

7.2 Label engineering

Execution conventions

Fixed-horizon labels

Continuous targets

Discrete targets

Variable-horizon labels

Adaptive horizons via trend scanning

Event-style labels

Maximum favorable and adverse excursion

diagnostics

Overlap, sample dependence, and effective sample size

Dealing with overlap

Label diagnostics before modeling

Meta-labels for confidence-weighted sizing

7.3 Univariate feature–label evaluation

Correctness screens – The non-negotiable first pass

Evaluating continuous labels

The Information Coefficient

Fold-level summarization – ICIR and sign consistency.

IC across horizons and rebalancing frequency

Inference at triage

Kendall’s and mutual information

Discrete labels – Evaluating features as classifiers

Threshold-dependent evaluation

Threshold-free metrics

Nonparametric diagnostics – Quantile and decile analysis

Preliminary feasibility checks

7.4 Search accounting and multiple testing

Define the searched set

Separate exploration from confirmation

Adjustment methods

Report effect sizes, not only significance

Strategy-level selection-bias diagnostics

7.5 From correlation to causality.

Why causal thinking matters for features

What is a directed acyclic graph?

Three structural roles

From DAG vocabulary to falsification checks

Mechanism plausibility checks

Worked example – 12-1 momentum versus short-term reversal

Plausibility scorecard

7.6 Summary.

8. Financial Feature Engineering

8.1 Capturing and configuring the economic drivers

8.2 Price-derived features

Trend and momentum

Reversal and mean reversion to an anchor

Volatility and tail-risk state

Efficient volatility estimation from OHLC data

Tail-risk and regime indicators

Microstructure, volume, and order flow

8.3 Structural and cross-instrument features

Carry, funding, and term structure

Cross-asset structure and relative value

Options-implied features

8.4 Contextual and slow-moving features

Fundamentals and characteristics

Time, calendar, and event encodings

Macro and policy state

8.5 Cross-cutting feature types and the limits of direct aggregation

When direct aggregation is not enough

8.6 Combining features and controlling search

Interaction diagnostics

Common interaction patterns

Worked example – Momentum $\times$ volatility-regime terciles

Event studies

Degrees of freedom discipline

Three implementation choices that change the hypothesis

8.7 Summary

9. Model-Based Feature Extraction

9.1 Diagnostics and stationarity features

Stationarity tests as features

Structural break features

Fractional differencing features

9.2 Transforming signals to uncover hidden structure

Kalman filter features

Spectral features

Wavelet features

Path signature features

9.3 Volatility Features

Autoregressive integrated moving average features

Generalized autoregressive conditional heteroskedasticity features

Asymmetric volatility – Exponential GARCH features

Heterogeneous autoregressive volatility features

Re-estimation cadence and online updates

Rough volatility and the Hurst exponent

9.4 Uncertainty features

- [The uncertainty-as-feature principle](#)
 - [Stochastic volatility features](#)
 - [ARIMA forecast uncertainty features](#)

[9.5 Regime features](#)

- [Observable regime rules](#)
 - [Hidden Markov model features](#)
 - [Markov-switching autoregressive features](#)
 - [Distribution-based regime features](#)
 - [Using regime features downstream](#)

[9.6 Cross-sectional and panel features](#)

- [Cross-sectional transforms of temporal features](#)
 - [Relative and benchmark-adjusted features](#)
 - [Pairwise temporal features](#)
 - [Universe-level aggregation](#)
 - [Point-in-time handling in panels](#)

[9.7 Summary](#)

[10. Text Feature Engineering](#)

[10.1 Lexical and statistical models](#)

- [Lexicon-based sentiment](#)
 - [Statistical representations using bag-of-words](#)
 - [Critical limitations](#)

[10.2 Static embeddings](#)

- [Learning from local context with Word2Vec](#)
 - [Global co-occurrence statistics with GloVe](#)
 - [Asset embeddings from portfolio data](#)
 - [The polysemy problem](#)

[10.3 Sequential models](#)

[10.4 Transformers](#)

- [Query, key, value, and the attention mechanism](#)
 - [Multi-head attention and positional information](#)
 - [Transformer architectures](#)
 - [Bidirectional transformers](#)
 - [Domain-Specific Variants for Finance](#)
 - [Beyond BERT in the LLM Era](#)

[10.5 The modern feature extraction workflow](#)

- [Text-to-signal pipeline contract](#)
 - [The pre-train, adapt, fine-tune cascade](#)
 - [Stage 1: General pre-training](#)
 - [Stage 2: Domain adaptation](#)
 - [Stage 3: Task fine-tuning](#)

- [Fine-tuning in practice](#)
 - [Tokenization pitfalls](#)
 - [From tokens to document embeddings](#)
- [Embedding selection and long documents](#)
 - [Handling long documents](#)
- [Text signal families and factor construction](#)
 - [Embedding-based factor construction](#)
 - [Evaluating text-derived signals](#)
- [Extract-to-schema feature engineering](#)
- [Model interpretability](#)

[10.6 Summary](#)

11. [The ML Pipeline](#)

[11.1 From inference to prediction](#)

- [Two modeling cultures](#)
- [What inference requires](#)
- [Why prediction demands a different approach](#)

[11.2 Regularized regression](#)

- [Ridge regression for distributed signal](#)
- [LASSO regression for sparse signal](#)
- [Combining both priors with elastic net](#)
- [Standardization](#)
- [Hyperparameter optimization with Optuna](#)
- [Loss function choice](#)
- [Evaluating regression models](#)
- [Sample weighting](#)
- [Training loss versus trading objective](#)

[11.3 Predicting direction with logistic regression](#)

- [Binary logistic regression](#)
- [Multi-class extension](#)
- [Regularization](#)
- [Probability calibration](#)
- [From probabilities to trading signals](#)
- [Handling class imbalance](#)
- [Evaluating classification models](#)

[11.4 Interpreting models with SHAP](#)

- [Game-theoretic attribution with SHAP](#)
 - [Efficient computation](#)
- [Practical application in global and local analysis](#)
 - [Global feature importance](#)
 - [Local explanations](#)

- [Feature dependence](#)
 - [Building an economic narrative](#)
 - [Limitations and failure modes](#)
 - [Integration with the walk-forward pipeline](#)
 - [11.5 Quantifying predictive uncertainty](#)
 - [Split-conformal prediction](#)
 - [The exchangeability problem](#)
 - [Adaptive conformal inference](#)
 - [Conformalized quantile regression](#)
 - [Conformal prediction sets for classification](#)
 - [Evaluating calibration quality](#)
 - [11.6 Case study insights](#)
 - [Where the signal is](#)
 - [Choice of regularization](#)
 - [Whether the signal persists](#)
 - [Horizon effects](#)
 - [Direction versus magnitude](#)
 - [From IC to profitability](#)
 - [11.7 Summary](#)
- 12. [Advanced Models for Tabular Data](#)
 - [12.1 From decision trees to ensembles](#)
 - [Random forests – Reducing variance with bagging](#)
 - [The path to boosting](#)
 - [12.2 Gradient boosting machines](#)
 - [High performance with boosting](#)
 - [Why GBMs excel on tabular data](#)
 - [Inside the engines – XGBoost, LightGBM, and CatBoost](#)
 - [XGBoost – Regularization and sparsity](#)
 - [LightGBM – Speed through sampling and bundling](#)
 - [CatBoost – State-of-the-art categorical handling](#)
 - [Practical comparison – Which GBM to choose?](#)
 - [Native feature importance and its limitations](#)
 - [Learning to rank](#)
 - [When LTR outperforms pointwise regression](#)
 - [A worked example – LambdaMART versus Pointwise MSE](#)
 - [Enforcing economic logic with monotonic constraints](#)
 - [12.3 Deep learning alternatives for tabular data](#)
 - [Tabular foundation models](#)
 - [Modern neural baselines – TabM and strong defaults](#)

[Retrieval-augmented models – TabR](#)

[When to look beyond GBMs – A decision framework](#)

[12.4 Advanced hyperparameter tuning with Optuna](#)

[The tree-structured Parzen Estimator](#)

[The define-by-run API](#)

[GBM-specific tuning strategy](#)

[Multi-objective optimization](#)

[Optimizing IC versus turnover in practice](#)

[12.5 Model explainability with SHAP](#)

[Dependence and interaction effects](#)

[SHAP-based drift monitoring](#)

[The Rashomon Effect – When equally good models disagree](#)

[Using SHAP for feature selection and pruning](#)

[From explanation to uncertainty and robustness](#)

[12.6 Case study insights](#)

[Where flexibility adds ranking content](#)

[What the lift rests on](#)

[Choosing the configuration](#)

[Whether the lift persists](#)

[Regression versus classification](#)

[Reading the model](#)

[From IC to profitability](#)

[12.7 Summary](#)

[13. Deep Learning for Time Series](#)

[13.1 Recurrent networks and their limits](#)

[The LSTM gating mechanism](#)

[Two limitations](#)

[13.2 N-BEATS and explicit decomposition](#)

[Blocks, stacks, and doubly residual learning](#)

[Basis expansion](#)

[Making N-BEATS interpretable](#)

[Extensions – N-BEATSx and N-HiTS](#)

[Practical caveats](#)

[13.3 Attention for time series](#)

[Adapting Transformers for temporal data](#)

[The self-attention mechanism](#)

[13.4 Linear baselines versus Transformers](#)

[Permutation-invariance and temporal order](#)

[The LTSF-Linear benchmark](#)

[Results and diagnostic evidence](#)

[Counterpoints and caveats](#)

[13.5 Modern Transformer variants](#)

[Patching as inductive bias with PatchTST](#)

[Inverting the dimensions with iTransformer](#)

[Covariate-rich forecasting with Temporal Fusion Transformers](#)

[13.6 Alternative architectures and foundation models](#)

[State space models](#)

[Time series foundation models](#)

[Adaptation modes](#)

[The limits of scaling](#)

[The finance transfer gap](#)

[13.7 A practical framework](#)

[Step 1: Establish strong baselines](#)

[Step 2: Diagnose the problem](#)

[Step 3: Apply the selection matrix](#)

[Forecasting paths versus predicting horizon labels](#)

[Forecasting libraries](#)

[13.8 Quantifying prediction uncertainty](#)

[Estimating uncertainty with MC dropout and deep ensembles](#)

[Foundation model calibration](#)

[13.9 Case study insights](#)

[Where the sequence models add ranking content](#)

[Which architectures carry the signal](#)

[Whether the predictive uncertainty is reliable](#)

[Direct prediction versus forecasting](#)

[From IC to profitability](#)

[13.10 Summary](#)

[14. Latent Factor Models](#)

[14.1 Making the case for latent factors](#)

[Three axes of the factor zoo debate](#)

[Axis 1 – Statistical validity](#)

[Axis 2 – Identification and measurement](#)

[Axis 3 – Risk versus mispricing](#)

[The recurring core](#)

[From the zoo to latent factors](#)

[14.2 Extracting latent factors with PCA](#)

[The mechanics of eigendecomposition](#)

[Assumptions and design choices](#)
[Covariance or correlation?](#)
[Idiosyncratic volatility normalization](#)

[Covariance estimation quality](#)
[How many PCA factors are real?](#)

[14.3 Eigenportfolios for equity strategies](#)
[Constructing and interpreting eigenportfolios](#)
[Applications in quantitative finance](#)
[Improving interpretability with hierarchical PCA](#)
[Fixing eigenvector instability](#)
[Practitioner recipe for two-stage production PCA](#)

[14.4 Decoding the yield curve](#)
[Level, slope, and curvature](#)
[Why PCA works for yield curves](#)
[Practical application for efficient hedging](#)

[14.5 Bridging economics and statistics with advanced models](#)
[The three-stage latent-factor forecasting adapter](#)
[Dynamic betas with instrumented PCA](#)
[Finding priced factors with risk-premium PCA](#)
[Test assets as a design choice](#)

[14.6 The conditional autoencoder](#)
[Preparing the characteristic panel](#)
[Specifying the dual-network architecture](#)
[Estimating the CAE](#)
[Turning latent factors into forecasts](#)
[Tuning, validation, and failure modes](#)
[Empirical evidence and replication caveats](#)

[14.7 The stochastic discount factor and the supervised autoencoder models](#)
[The stochastic discount factor](#)
[Building the SDF with adversarial deep learning](#)
[The supervised autoencoder](#)

[14.8 Case study insights](#)
[Where latent factors add ranking content](#)
[Which objective extracts the signal](#)
[Dimensionality and stability](#)
[Regression versus classification](#)
[Choosing a method](#)

[14.9 Summary](#)

[15. Causal Machine Learning](#)

15.1 From theory to estimation

The causal estimation challenge

Three estimation problems

15.2 Identification and validation

From outcome and intervention to estimand

Directed acyclic graphs and adjustment sets

Alternative identification designs

Instrumental variables

Difference in differences

Regression discontinuity

Event study counterfactuals

Choosing among designs

15.3 Validation and refutation

Placebo and negative control tests

Sensitivity to omitted confounding

Stability across samples, specifications, and outcomes

15.4 Isolating factor effects with DML

The intuition behind double machine learning

Measuring the causal impact of momentum

Outcome choice shapes causal credibility

Regime-conditional position sizing – A case study

15.5 Measuring event impact with Bayesian structural time-series

The intuition behind BSTS

The impact of a Fed announcement on bond ETFs

Illustrative results

Placebo test validation

Posterior probability versus credible intervals

15.6 Causal discovery from observational data

Time-series discovery with Granger, PCMCI, and VAR-LiNGAM

Cross-sectional DAG learning with NOTEARS

Empirical method comparison

15.7 Case study causal evidence

The two gates

Confounding bias

Variation across horizons

Causal evidence and predictive signal

15.8 Summary

16. Strategy Simulation

- 16.1 Backtesting as falsification
 - Failure modes and diagnostic design
 - Practical workflow and decision standard
- 16.2 Specifying your backtest protocol
 - Signal generation timing
 - Score to signal conversion
 - Rebalancing frequency and holding period
 - Position sizing
 - Order types and fill assumptions
 - Constraints
 - Cost model
 - Testing sensitivity to cost assumptions
- 16.3 Vectorized and event-driven backtesting
 - How vectorized backtesting works
 - How event-driven backtesting works
 - How to select a backtest methodology
 - How simulation semantics control results
 - Where the major libraries fit
 - A practical workflow
- 16.4 An auditable non-ML baseline
 - A simple baseline – Momentum with risk filter
 - Step-by-step baseline construction
 - Documenting baseline specification and performance
- 16.5 Understanding performance metrics
 - Design principles for backtest reporting
 - Return and compounding metrics
 - Risk and drawdown metrics
 - Risk-adjusted performance metrics
 - Trading and exposure metrics
 - Cost and implementation metrics
- 16.6 Diagnosing the economic value
 - From aggregate metrics to diagnostic decomposition
 - Cost sensitivity and turnover fragility
 - Baseline comparison and incremental model value
 - Regime-sliced performance
 - Regime-sliced drawdowns and failure modes
 - Regime snooping and diagnostic discipline
 - Reporting economic diagnostics
- 16.7 Statistical inference and backtest overfitting
 - A backtest statistic is an estimate

[Sharpe inference under serial dependence](#)
[From strategy inference to search adjustment](#)
[Selection bias and White's reality check](#)
[Deflated Sharpe ratio and search haircuts](#)
[False discovery control and effective search complexity](#)
[Search budget, sample length, and trial accounting](#)
[Reporting and decision standards](#)

[16.8 Summary](#)

[17. Portfolio Construction](#)

[17.1 Defining the allocation problem](#)

[The decision problem – Weights, exposure, and timing](#)
[Constraints as first-class modeling choices](#)
[How signal quality becomes portfolio performance](#)

[17.2 A portfolio construction workflow](#)

[The allocator term sheet](#)
[Estimation windows and leakage prevention](#)

[17.3 Portfolio evaluation metrics](#)

[Benchmark-relative performance](#)
[Concentration and diversification](#)
[Stability and implementation metrics](#)
[Evaluating the risk model through the portfolio](#)

[17.4 Defining baseline allocators](#)

[Equal weight – The null model](#)
[Inverse volatility weighting](#)
[Score-weighted portfolios](#)
[Conformal position sizing](#)
[Risk parity – Equal risk contribution](#)
[Volatility targeting](#)
[The Kelly criterion – Optimal sizing from signal quality](#)

[17.5 Mean-variance optimization and the Markowitz curse](#)

[Why MVO fails out of sample](#)
[Making MVO more robust with covariance shrinkage](#)
[Shrinkage hedging as robust optimization](#)
[Factor-mimicking portfolios](#)

[17.6 Optimizing for stability with Hierarchical Risk Parity](#)

[From correlations to weights – The HRP algorithm](#)
[Why HRP can be more stable than naive MVO](#)
[Limitations and extensions](#)
[ETF walk-forward results](#)

[17.7 Comparing allocator performance](#)

[Experimental design](#)

[Empirical results for an ETF momentum strategy](#)

[The broader lesson](#)

[17.8 Deep learning for portfolio construction](#)

[The end-to-end pipeline](#)

[Training a portfolio objective directly](#)

[Architecture under a portfolio objective](#)

[Variable selection – VLSTM as the case study](#)

[DeePM – Structural priors and a regime-robust loss](#)

[Evaluating learned allocators](#)

[17.9 Summary](#)

[18. Transaction Costs](#)

[18.1 Where costs enter the ML4T workflow](#)

[The false positive problem](#)

[18.2 A cost taxonomy for practitioners](#)

[Explicit costs](#)

[Implicit costs](#)

[Capacity costs](#)

[The cost stack](#)

[18.3 The microstructure regime link](#)

[Intraday liquidity patterns](#)

[Volatility and spread dynamics](#)

[Regime transitions](#)

[Implications for cost modeling](#)

[18.4 Baseline backtesting cost models](#)

[A unified view of impact models](#)

[The spread model](#)

[The linear slippage model](#)

[The square-root impact model](#)

[Choosing and stress testing a baseline cost model](#)

[Costs as portfolio regularizer](#)

[18.5 Execution algorithms as controls](#)

[Time-weighted average price](#)

[Volume-weighted average price \(VWAP\)](#)

[Regime-aware participation](#)

[Execution as strategy design input](#)

[18.6 Optimizing execution with Almgren–Chriss as a unifying framework](#)

[Model components](#)

[The optimal trajectory](#)

[Regime-dependent parameters](#)

[What Almgren-Chriss means for research](#)

[18.7 Transaction cost analysis and model validation](#)

[Implementation shortfall](#)

[Cost decomposition](#)

[Regime context in TCA](#)

[Closing the loop](#)

[The hidden costs of risk-model-driven turnover](#)

[TCA report elements](#)

[18.8 Designing practical cost guardrails](#)

[Break-even turnover](#)

[Minimum required edge](#)

[Alpha-to-go – What a signal is worth after costs](#)

[Capacity analysis](#)

[Kill switch criteria](#)

[When to abandon rather than modify](#)

[A practical example – Cost mitigation for options](#)

[Comparing fixed and relative cost regimes](#)

[18.9 Summary](#)

[19. Risk Management](#)

[19.1 Turning your backtest winner into a tradable system](#)

[19.2 A practical risk taxonomy for quant strategies](#)

[Market risk](#)

[Factor risk](#)

[Leverage risk](#)

[Concentration risk](#)

[Liquidity and capacity risks](#)

[Model risk](#)

[Operational risk](#)

[The risk control matrix](#)

[19.3 Measuring the tail – VaR and CVaR](#)

[Value at risk](#)

[Conditional value-at-risk](#)

[Why VaR understates real risk](#)

[Regime-conditional risk metrics](#)

[Incorporating liquidity haircuts](#)

[Evaluating risk model quality](#)

[Practical guidance](#)

[19.4 Drawdowns, path risk, and time to recovery](#)

[Maximum drawdown](#)

The historical record – 150 years of drawdowns

Conditional drawdown analysis

Linking drawdowns to kill switches

19.5 Decomposing factor, sector, and macro exposures

The factor model framework

Intended and unintended exposures

Exposure stability across regimes

Sector and geographic decomposition

Macro factor sensitivities

Attribution and decomposition in practice

Attribution uncertainty and why point estimates are not enough

Detecting hidden dependencies in residuals

Precision matrix quality – The Mahalanobis-distance variance diagnostic

Resolving attribution ambiguity via factor rotation

Trade-level SHAP as a diagnostic tool

19.6 Stress testing and scenario analysis

Historical crises as regime examples

Stressing cost parameters

Scenario matrix construction

Factor-specific stress tests

Reverse stress testing

Connecting stress results to controls

19.7 Adaptive risk controls without leakage

GARCH and EWMA as operational volatility engines

Volatility targeting

Short-term volatility updates for crisis response

Regime-triggered exposure caps

Concentration and turnover limits

Position-level controls

From rule-based controls to learned hedging policies

Anti-patterns to avoid

Walk-forward validation and auditability

19.8 Applying kill switches and risk governance

Escalation and review cadence

The drawdown rule paradox

Drift detection as early warning

Model risk management documentation

Governance as a competitive advantage

19.9 Summary

20. Strategy Synthesis

20.1 First-pass results across nine case studies

Evaluation metrics

Headline outcomes

Cases that hold or decay

Case studies that show little to no edge

From validation to holdout

20.2 Setup and feature evaluation

Feature survival and strategy survival

20.3 Signal quality and prediction uncertainty

The noisy link from model to strategy performance

Stability metrics – A diagnostic bundle

Uncertainty around the best-performing configuration

Label choice as a first-order decision

20.4 From signals to strategies

The fundamental law, empirically

Cadence and effective breadth

Win rate and payoff asymmetry

Family rankings shift across the pipeline

Ensembling the top clusters

When deep learning helps

Complementary Roles

20.5 Portfolio allocation across the case studies

How much the allocator moves the result

Which allocator leads, and why

20.6 Trading realism – Costs, capacity, and execution

The cost survival landscape

Capacity and concentration

20.7 Risk overlays

20.8 Causal credibility and confounding bias

20.9 Next steps after the first research iteration

20.10 Summary

21. Reinforcement Learning

21.1 The sequential decision-making paradigm in finance

The temporal credit assignment problem

End-to-end optimization

Why this chapter focuses on execution rather than alpha

21.2 Financial markets as Markov decision processes

The MDP framework

- [Partial observability in financial markets](#)
 - [State representation](#)
 - [Action space design](#)
 - [Reward engineering](#)
 - [Transition dynamics and discount factors](#)
- [21.3 Core algorithms – From DQN to actor-critic](#)
 - [The Bellman equation](#)
 - [Deep Q-networks](#)
 - [Actor-critic – On-policy](#)
 - [Actor-critic – Off-policy](#)
 - [Risk-aware RL](#)
 - [Model-based and model-free approaches](#)
 - [Practical algorithm selection](#)
- [21.4 Application I – Optimal trade execution](#)
 - [Classical benchmarking with Almgren-Chriss](#)
 - [The RL approach to execution](#)
 - [High-fidelity simulation requirements](#)
- [21.5 Application II – Market making](#)
 - [Classical benchmark – Avellaneda-Stoikov](#)
 - [The RL approach to market making](#)
 - [Learned risk management behaviors](#)
- [21.6 Application III – Deep hedging for derivatives](#)
 - [Overcoming classical hedging limitations](#)
 - [MDP formulation for deep hedging](#)
 - [The no-transaction band](#)
 - [Implementation with pfhedge](#)
 - [Q-Learner in Black-Scholes – A value-based alternative](#)
- [21.7 Inverse reinforcement learning – Learning from observed behavior](#)
 - [The IRL framework](#)
 - [Behavior cloning versus reward inference](#)
 - [Financial applications](#)
 - [Practical considerations](#)
- [21.8 The simulation-to-reality gap](#)
 - [Sources of failure](#)
 - [High-fidelity simulation](#)
 - [Offline reinforcement learning](#)
 - [RL-specific backtesting pitfalls](#)
 - [Deployment checklist](#)
 - [Explainability and regulation](#)

21.9 Summary

22. RAG for Financial Research

22.1 From feature extraction to generation

22.2 Grounding LLMs with retrieval-augmented generation

The index-retrieve-generate pipeline

The orchestration layer

22.3 Intelligent document ingestion

The failure of naïve chunking

Chunk size – The fundamental trade-off

Structure-aware parsing

Multimodal content – Tables, charts, and images

Metadata – The key to verifiable citations

22.4 Domain-specific embeddings

The FinMTEB benchmark

Domain-adapted embedding models

Embedding dimensions and quantization

Evaluating embedding models on the target corpus

22.5 Hybrid retrieval and vector databases

Reciprocal rank fusion

Beyond bi-encoders – The retrieval architecture spectrum

Query enhancement techniques

Vector database selection

Filtered retrieval

22.6 Re-ranking and constraint-based prompting

Cross-encoder reranking

Context window management

Constraint-based prompting

Numeric reliability and tool-verified computation

Programmatic output validation

The prompt as contract

22.7 Diagnosing RAG pipeline bottlenecks

Failure modes

RAGAs and claim-level evaluation

Extending the harness to adversarial inputs

Iterative correction loops

22.8 Applications and strategic choices

The 10-K due diligence assistant

Comparative case study – ESG analysis

Generation model selection

Production lifecycle

- [Deployment in regulated environments](#)
 - [Structured data extraction – Institutional holdings](#)
 - [22.9 Introducing agentic frameworks](#)
 - [The ReAct paradigm](#)
 - [RAG as an agent tool](#)
 - [Industry adoption](#)
 - [22.10 Summary](#)
- 23. [Knowledge Graphs](#)
 - [23.1 When relational structure justifies a graph](#)
 - [When graphs add value](#)
 - [Multi-hop dependency queries](#)
 - [Structural crowding and co-ownership](#)
 - [Temporal relationship evolution](#)
 - [When graphs do not help](#)
 - [Single-entity attribute lookup](#)
 - [Narrative synthesis over broad corpora](#)
 - [Sparse graphs with few relationships](#)
 - [Practical decision framework](#)
 - [23.2 Constructing financial knowledge graphs](#)
 - [From legacy pipelines to directed extraction](#)
 - [Governance before prompt](#)
 - [Identity contract](#)
 - [Schema contract](#)
 - [Provenance contract](#)
 - [A five-stage extraction workflow](#)
 - [Error taxonomy and monitoring](#)
 - [Supply chain schema example](#)
 - [23.3 Deterministic relational reasoning with graph RAG](#)
 - [End-to-end architecture](#)
 - [Example query flow](#)
 - [Comparing explicit KG graph RAG to community-summary GraphRAG](#)
 - [Evaluation and failure modes](#)
 - [23.4 From graphs to machine learning features](#)
 - [Network topology features](#)
 - [Supply chain risk features](#)
 - [Institutional crowding features](#)
 - [Cross-graph integration and temporal dynamics](#)
 - [23.5 Correlations and portfolios in financial networks](#)
 - [The established tradition](#)

[From minimum spanning trees to portfolio construction](#)

[GNN maturity assessment](#)

[The pragmatic recommendation](#)

[23.6 Temporal integrity and leakage-safe evaluation](#)

[The three-timestamp model](#)

[8-K filings as a temporal case study](#)

[13F temporal considerations](#)

[Snapshot construction and dynamic features](#)

[Leakage-safe evaluation protocol](#)

[23.7 Building a KG-ready pipeline](#)

[Engine and query language selection](#)

[Ontology strategy](#)

[Query safety controls](#)

[Schema versioning](#)

[23.8 Summary](#)

[24. Autonomous Agents](#)

[24.1 From prediction functions to agentic workflows](#)

[Where agentic workflows do not help](#)

[Autonomy levels and realistic deployment boundaries](#)

[Interface with reinforcement learning](#)

[24.2 Cognitive architectures – How agents reason](#)

[The ReAct framework](#)

[Tree of thoughts](#)

[Reflexion](#)

[Choosing and composing reasoning patterns](#)

[24.3 Agent memory – State, persistence, and replay](#)

[The memory hierarchy](#)

[Why explicit state is mandatory](#)

[Checkpointing, replay, and diagnosis](#)

[Memory lifecycle and governance](#)

[Memory and evaluation are coupled](#)

[24.4 Tool integration – Contracts, controls, and context engineering](#)

[Tool categories for research and forecasting](#)

[Tool contracts and selection quality](#)

[Context engineering](#)

[Provenance and structured output](#)

[Enforcing structured output](#)

[Source policy](#)

[Error handling and graceful degradation](#)

24.5 The engineering stack – Frameworks and migration

Start from constraints, not preferences

From notebook pattern to production demo

Migration sequence that reduces rework

Comparing frameworks locally

24.6 Designing the research agent at the heart of the pipeline

Architecture and toolset

ReAct loop and rich output extraction

Trace inspection and replay

Walkthrough of a typical run

Evaluation and acceptance criteria

24.7 Multi-agent forecasting systems

Architecture – From agent ensemble to production pipeline

Aggregation and extremization

The debate-supervisor pattern

Probability calibration

Evaluation, ablation, and baselines

24.8 The ML4T research agent

The operator's shape

Skills are the task-specific knowledge layer

Two example runs of the ML4T research loop

What the runs share and where the limits sit

Key Takeaways

24.9 Preparing for production

The reliability gap and observability

Statistical testing and point-in-time integrity

Contamination-resistant evaluation

Evaluation metric stack

Cost, latency, and deployment fit

Release process and monitoring

Collective effects and risk posture

24.10 Security and governance

Threat model – Where failures originate

Prompt injection and retrieval defenses

Least privilege and the warden pattern

Human-in-the-loop controls and governance artifacts

Security testing and metrics

24.11 Summary

25. Live Trading Systems

25.1 The unified research-to-production framework

Technical divergence

The unified framework solution

Architecture for dual-mode operation

From simulation to live

25.2 Integrating with Interactive Brokers

TWS versus IB Gateway

Connection patterns

Order types and execution

Position and account management

Error handling

Historical data access

25.3 Integrating with Alpaca

REST and WebSocket APIs

Paper trading mode

Key differences from IBKR

Direct crypto exchange APIs

Error handling

Idempotency keys

25.4 QuantConnect and managed platforms

The platform approach versus self-hosted

LEAN engine overview

Trade-offs – Convenience versus flexibility versus cost

When platforms make sense

Migration considerations

25.5 Order lifecycle management

State machine for order status

Handling partial fills, rejections, and cancellations

End-of-day reconciliation

Idempotency for crash recovery

25.6 Ensuring technical parity through pipeline verification

The verification methodology

Feature parity testing

Prediction consistency testing

Sizing logic verification

Automated regression tests

Diagnosing divergence points

Unified data sources

Funding rate strategy – Live deployment

Parity versus regime stability

[A contrasting deployment – FX daily signals](#)

[25.7 Operational readiness](#)

[Preflight checklist](#)

[Kill switches and emergency procedures](#)

[Enforced safety in the live runtime](#)

[Transitioning from paper to live trading](#)

[Regulatory and jurisdictional considerations](#)

[United States](#)

[European Union](#)

[India](#)

[Transaction taxes and access constraints](#)

[Monitoring handoff to Chapter 26](#)

[25.8 Summary](#)

[26. MLOps and Governance](#)

[26.1 Two sources of live trading failure](#)

[Technical failure caused by pipeline divergence](#)

[Statistical failure caused by performance decay](#)

[Why the distinction matters](#)

[26.2 Performance monitoring](#)

[Data integrity gates](#)

[The rolling metrics framework](#)

[Rolling Sharpe ratio](#)

[Information coefficient](#)

[Hit rate and win/loss ratio](#)

[Drawdown depth and duration](#)

[Alert thresholds](#)

[Visual dashboards](#)

[Comparison to backtesting expectations](#)

[Execution quality monitoring](#)

[Example monitoring workflow](#)

[26.3 Drift detection](#)

[Data drift caused by input distribution changes](#)

[Feature drift caused by importance shift](#)

[Concept drift caused by relationship changes](#)

[Practical workflow](#)

[From detection to diagnosis](#)

[26.4 Safe model updates](#)

[When to retrain](#)

[Shadow mode evaluation](#)

[Deciding duration](#)

- [Statistical testing](#)
 - [Effect size matters](#)
 - [Phased rollouts](#)
 - [Rollback procedures](#)
 - [26.5 Circuit breakers and safety](#)
 - [Multi-level protection](#)
 - [Loss-based breakers](#)
 - [Position-based breakers](#)
 - [Anomaly-based breakers](#)
 - [Implementation patterns and libraries](#)
 - [Recovery procedures](#)
 - [Manual override considerations](#)
 - [26.6 MLOps infrastructure overview](#)
 - [Preventing training-serving skew with feature stores](#)
 - [Data versioning and lineage](#)
 - [Experiment tracking with model registries](#)
 - [CI/CD for trading models](#)
 - [Right-sizing your stack](#)
 - [26.7 Summary](#)
- [27. The Systematic Edge](#)
 - [27.1 The systematic edge – From techniques to philosophy](#)
 - [27.2 The modern quant career](#)
 - [Five core roles](#)
 - [Institutional ecosystems](#)
 - [Geographic dynamics](#)
 - [27.3 Building a learning practice](#)
 - [The foundational library](#)
 - [Staying current online](#)
 - [Mastering the toolchain](#)
 - [Structured learning pathways](#)
 - [Community and brand building](#)
 - [27.4 Navigating the frontiers – Quantum, DeFi, and ethical AI](#)
 - [Quantum computing – Promise versus reality](#)
 - [The reality check](#)
 - [Practical implications](#)
 - [DeFi – A live market](#)
 - [AI ethics – From philosophy to compliance](#)
 - [Strategic prioritization](#)
 - [27.5 Building your path forward](#)
 - [Conduct a skills assessment](#)

[Design a learning system](#)

[Avoid common pitfalls](#)

[27.6 Summary](#)

[Other Books You May Enjoy](#)

[Index](#)

Preface

Machine learning has changed what is possible in systematic trading, and it has changed how easily you can fool yourself. The same tools that surface a genuine edge will, used carelessly, manufacture one that exists only in your backtest. This book is about the difference between careful and careless tool use: how to take a research idea all the way to a strategy you can run, and keep running, in a live market without mistaking noise for signal along the way.

The third edition is a ground-up rebuild, organized around a single end-to-end workflow. Earlier editions moved technique by technique. This edition runs one process from start to finish: from data infrastructure and strategy research, across an *evidence boundary* that separates tuning from evaluation, to deployment and monitoring, with a feedback loop that retrains, pauses, or retires a strategy as its edge decays. Nine case studies run throughout the book, from raw data through features, models, backtests, costs, and risk to a deployment assessment. ETFs, crypto perpetuals, intraday equities, options, FX, futures, and equity factor panels each go through the *same* pipeline, which lets you see where a disciplined process works, where it breaks down, and why.

Two strands are new and run deliberately *across* the workflow rather than sitting at a single stage. **Generative AI** brings retrieval-augmented generation grounded in filings, knowledge graphs, and multi-agent research systems. **Causal machine learning** brings the tools for separating a real effect from a spurious correlation. Alongside them, the edition adds a full production track, a wider model toolkit ranging from gradient boosting to modern time-series and tabular architectures,

reinforcement learning for execution and hedging, and synthetic data generators for validation when history is short. Throughout, methodological rigor is treated as a first-class subject: walk-forward validation, the multiple-testing and overfitting problems that quietly invalidate most backtests, and the tools that confront them, from the Deflated Sharpe Ratio to conformal prediction.

If there is one claim the book defends from beginning to end, it is the title's: the process is your edge. Markets drift, break, and change regime. A pristine model you discover once is not an edge; a research process disciplined enough to notice when a signal is decaying and flexible enough to change course is.

A book and a living repository

This edition is designed to be read alongside its code. The book contains the concepts, reasoning, and results; the **GitHub repository contains everything you run**. There are no code listings in these pages. That is deliberate. Code printed in a book is frozen the day it goes to press, and trading code ages badly: libraries move, data sources change, and a subtle bug found after publication can only be errata. The repository does not have that problem.

So the complete, runnable implementation of every method, every figure, and all nine case studies lives in the repository at:

<https://github.com/stefan-jansen/machine-learning-for-trading>

The repository is alive. It is version-controlled, continuously tested, and maintained past the print date. When a library changes or a reader finds a problem, the fix lands there, and everyone gets it. It can hold far more than a book can: full pipelines, intermediate artifacts, reproducible Docker environments, and material that grows over time. And it makes the subject what it has to be: hands-on. You are meant to clone it, run it, break it, and change it. The book tells you *what* a method does and *why* it matters; the repository is where you watch it work on real data and make it your own.

Read the two together. Each chapter in the book corresponds to a numbered directory in the repository, and the most productive way through the material is to read a chapter, then open its directory and run the notebooks that implement it.

How each chapter maps to the repository

Every chapter has its own directory, and each directory opens with a `README` that serves as the hub for that chapter's hands-on work. Each one gives you:

- Learning objectives, outlining what you should be able to do after the chapter
- A section-by-section guide to how the chapter's ideas map onto the notebooks
- The notebooks to run, with the exact commands, from the repository root
- References to the primary sources behind the chapter—the foundational papers and books worth reading next if you want to go deeper than the chapter goes

The notebooks themselves are paired Jupyter files: a `.py` source you edit and a generated `.ipynb` you run. Each notebook states in its preamble which environment it needs and any parameters worth changing. Start a chapter from its `README`, and the chapter's structure and its code stay in step.

The companion ecosystem at ml4trading.io

The book and the repository are anchored by a companion site, ml4trading.io, that extends both:

- **Primers** : Focused introductions to the financial, statistical, and machine-learning background the chapters build on. If a chapter

assumes something you would like to firm up, the primer for it is the place to start; new readers should start there before *Chapter 1* .

- **Agent skills** : A library of skills that teach AI coding assistants the ML4T workflow. Each one packages a reviewer-grade check that keeps a coding agent from shipping the field's expensive mistakes—data leakage, lookahead bias, overfit backtests, unrealistic costs—so the agent you build with applies the book's discipline by default.
- **A page for every chapter** : A detailed table of contents, the full bibliography, and write-ups of all nine case studies: the online counterpart to the printed book.
- **Courses** : Guided paths through the material, including a research-to-production course that works the end-to-end workflow and a course on autonomous and multi-agent systems for financial research. (See maven.com/stefan-jansen.)
- **ML4T Insights** : An ongoing newsletter with new research, methods, and developments in machine learning for trading, well beyond the book itself.

The book is the durable spine. The repository, the primers, the coding skills, and the newsletter are how the subject keeps moving after the spine is solidified.

Who this book is for

This book is for anyone who wants to apply machine learning to trading as a disciplined, end-to-end practice rather than a collection of models: quantitative researchers and traders, data scientists and ML engineers moving into finance, advanced students, and technically minded investors who want to build and evaluate strategies seriously.

You should be comfortable reading and writing Python and at ease with the core ideas of probability, statistics, and linear algebra. Prior exposure to machine learning and to financial markets helps but is not assumed. Where a chapter leans on background you would like to shore up, the **primers at ml4trading.io** are there to fill the gap. You do not need a trading background to start; you do need a willingness to run the code.

What this book covers

An introduction and a closing chapter bracket six workflow-aligned parts.

Chapter 1, The Process Is Your Edge , opens the book by making the case that process discipline beats model sophistication and introduces the ML4T workflow and the evidence boundary that separates exploration from confirmation.

Part I — Financial Data (Chapters 2–5) covers the markets, instruments, and infrastructure that the rest of the book builds on.

Chapter 2, The Financial Data Universe , lays out a taxonomy of market, fundamental, and alternative data across eight asset classes, quantifies survivorship bias, and benchmarks storage formats and the data-quality framework used throughout.

Chapter 3, Market Microstructure , goes from raw exchange messages to feature-ready bars: parsing NASDAQ ITCH, reconstructing the limit order book, trade classification, and bar-sampling methods.

Chapter 4, Fundamental and Alternative Data , builds point-in-time pipelines for SEC EDGAR filings, entity resolution, macro and commodity fundamentals, and alternative-data evaluation, including on-chain crypto data and prediction markets.

Chapter 5, Synthetic Financial Data , generates alternative market histories for robust validation with TimeGAN, Tail-GAN, Sig-CWGAN, diffusion models, and LLM-based generation, evaluated through a fidelity–utility–privacy framework.

Part II — Research Design and Feature Engineering (Chapters 6–10) defines the trading problem, then turns data into the labels, features, and evaluation that determine what any model can learn.

Chapter 6, Strategy Research Framework , defines the trading game before any model is built—universe rules, decision schedule, cost model, evaluation protocol, and run logging—and introduces the nine case studies alongside the walk-forward discipline that anchors *Chapters 7–20* .

Chapter 7, Defining the Learning Task , covers label engineering, univariate feature evaluation with information coefficients, multiple-testing control, and causal plausibility checks.

Chapter 8, Financial Feature Engineering , builds five feature families from price data along with structural, cross-instrument, and contextual features, and covers feature selection with robustness testing.

Chapter 9, Model-Based Feature Extraction , derives features from fitted models—Kalman filters, spectral methods, GARCH volatility, and HMM regime probabilities—with point-in-time correctness enforced throughout.

Chapter 10, Text Feature Engineering , moves from bag-of-words through to transformers: TF-IDF, word embeddings, sequence models, FinBERT sentiment, financial NER, and news-return signals.

Part III — Model Development (Chapters 11–15) applies five model families to the same nine case studies, each building on the linear baseline.

Chapter 11, The ML Pipeline , establishes regularized linear models as the baseline every later model must beat, with SHAP interpretability, conformal prediction, and a cross-dataset comparison.

Chapter 12, Advanced Models for Tabular Data , covers gradient boosting (XGBoost, LightGBM, CatBoost) with multi-objective tuning and deep-learning tabular alternatives, with explainability and cross-dataset results.

Chapter 13, Deep Learning for Time Series , surveys LSTM, N-BEATS, Transformers, TSMixer, TCN, and Mamba against the LTSF-Linear debate, with evidence on when deep learning helps.

Chapter 14, Latent Factor Models , covers PCA eigenportfolios, IPCA, conditional and supervised autoencoders, and adversarial SDF estimation, with results on when latent factors add value.

Chapter 15, Causal Machine Learning , applies Double Machine Learning, Bayesian Structural Time Series, and causal discovery to isolate real effects from spurious correlation across the case studies.

Part IV — Strategy Implementation (Chapters 16–20) turns predictions into deployable strategies through backtesting, portfolio construction, costs, risk, and synthesis.

Chapter 16, Strategy Simulation , treats backtesting as falsification: protocol specification, vectorized versus event-driven engines, metric reporting, and strategy-level overfitting control.

Chapter 17, Portfolio Construction , goes from scores to portfolios with mean-variance optimization and its pitfalls, Hierarchical Risk Parity, Kelly sizing, conformal position sizing, and a controlled allocator comparison.

Chapter 18, Transaction Costs , covers the cost taxonomy, spread and market-impact estimation, execution algorithms, transaction-cost analysis, and breakeven costs that vary widely by asset class.

Chapter 19, Risk Management , covers VaR/CVaR, drawdown controls, factor and sector decomposition, stress testing, adaptive overlays, deep hedging, and kill switches.

Chapter 20, Strategy Synthesis , draws together what nine experiments reveal about turning predictions into strategies: IC–Sharpe decorrelation,

the model-family cascade, cost-survival analysis, and a practitioner's decision framework.

Part V — Advanced AI (Chapters 21–24) covers reinforcement learning, large language models, knowledge graphs, and autonomous agents for finance.

Chapter 21, Reinforcement Learning , formulates execution and hedging as Markov decision processes and applies DQN, PPO, and SAC to optimal execution, market making, and deep hedging.

Chapter 22, RAG for Financial Research , builds retrieval-augmented generation grounded in SEC filings, from ingestion and domain embeddings through hybrid retrieval, evaluation, and the transition to agentic workflows.

Chapter 23, Knowledge Graphs , covers KG construction from filings, Graph RAG for multi-hop reasoning, graph features for ML, and temporal-leakage prevention.

Chapter 24, Autonomous Agents , covers agent architectures, memory and tools, the engineering stack, a stateful equity-research agent, and multi-agent forecasting with adversarial debate.

Part VI — Production (Chapters 25–26) takes strategies live, with the systems and operational infrastructure that this requires.

Chapter 25, Live Trading Systems , presents a unified framework bridging research and production with Interactive Brokers, Alpaca, and managed platforms, order lifecycle management, and operational readiness.

Chapter 26, MLOps and Governance , covers an ML failure taxonomy, drift detection, safe rollout, circuit breakers, feature stores, and experiment tracking.

Conclusion brackets the opening chapter: having run the full workflow, the book returns to where it began.

Chapter 27, The Systematic Edge , closes with the systematic philosophy, career paths, learning resources, research frontiers, and how to build your own edge.

To get the most out of this book

Read each chapter with its repository directory open beside you, and run the notebooks as you go. The workflow is cumulative: the case studies introduced in *Chapter 6* are carried through every chapter that follows, so the value compounds when you run them rather than only read about them. You should:

- **Set up the environment once** . Clone the repository and use the **Docker images** to ensure results are reproducible across machines, or set up a local environment with `uv` . Run everything **from the repository root** , never from a chapter directory.
- **Start small with data** . Most notebooks need datasets; begin with the free tier (no API keys), documented in the repository's `data/` directory.
- **Start with the ETF case study** . It uses free data, requires no API keys, and runs fast, so it is the quickest way to see the whole pipeline end-to-end before you commit to a larger dataset.
- **Fill gaps from the companion site** . If a chapter assumes background you would like to firm up, use the **primers at `ml4trading.io`** .

You will get the most from the book if you treat the code as the main focus rather than an appendix. The repository `README` documents installation for Linux, Windows (WSL2), and macOS, including GPU setup.

Download the example code

All of the book's code lives in the GitHub repository, which is the primary and continuously maintained source:

<https://github.com/stefan-jansen/machine-learning-for-trading>

```
git clone https://github.com/stefan-jansen/machine-learning-for-trading
cd machine-learning-for-trading
```

Packt also distributes a downloadable code bundle for the book. The GitHub repository is the authoritative, up-to-date version; prefer it, and use the `README`’s quick-start instructions to set up and run the notebooks.

Conventions used

`CodeInText` indicates code words in text: module, function, and variable names, file and directory names, and dataset and notebook names. For example: “run the `01_process_is_edge/factor_regimes.py` notebook from the repository root.”

Bold indicates a new term or an important word.

A data convention worth knowing up front: across every dataset and chapter, the code uses a single canonical schema, with `symbol` identifying an instrument and `timestamp` identifying time, for both daily and intraday data. Notebooks and case studies share that vocabulary so results carry cleanly from one chapter to the next.

Note/Tip/Warning callouts appear throughout, flagging respectively something to be aware of, a useful shortcut, and a step where it is easy to go wrong.

Get in touch

Feedback from our readers is always welcome.

General feedback : Email feedback@packtpub.com and mention the book's title in the subject of your message. If you have questions about any aspect of this book, please email us at questions@packtpub.com .

Errata : Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit <http://www.packtpub.com/submit-errata> , click **Submit Errata** , and fill in the form.

Piracy : If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packtpub.com with a link to the material.

If you are interested in becoming an author : If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packtpub.com/> .

Free benefits with your book

This book includes free benefits designed to support your learning and help you apply what you learn effectively. Activate them now for instant access (see the *How to unlock* section for instructions).

Here's a quick overview of what you can instantly unlock with your purchase:



Complete Code Bundle

Download the full source code for this book's examples and projects.



DRM-Free PDF Version

Download DRM-free PDF and ePub copies of this book.



7-Day Packt Library Access

Get 7-day unlimited access to 8,000+ books and videos. No credit card required.

Available for first-time Packt+ trial users only.



Next-Gen Reader Access

Read this book on Packt Reader with progress sync, dark mode and note-taking.

How to unlock

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

Share your thoughts

Once you've read *Machine Learning for Trading, Third Edition* , we'd love to hear your thoughts! Please [click here to go straight to the Amazon review page](#) for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

1

The Process Is Your Edge

Machine learning (ML) now reaches every stage of systematic trading, from feature engineering and portfolio construction to execution and monitoring. However, the fundamental challenge remains unchanged: markets are dynamic, competitive, and unforgiving of undisciplined research.

The ML4T Workflow aims to help you meet that challenge as a systematic process for generating, testing, and deploying strategies that adapt to evolving markets. The workflow draws from quantitative finance practice, industrial ML deployment frameworks, and lessons repeatedly learned through market regime shifts.

By the end of this chapter, you will be able to:

- Distinguish structural breaks, regimes, and drift, and explain why static trading models degrade.
- Understand the ML4T Workflow, with its foundation layer (data infrastructure) and iterative research modules (scoping, features, modeling, strategy, deployment), and how each module's artifacts flow into the next.
- Explain the evidence boundary that separates exploration from confirmation, and how logging trials and selection-adjusted inference maintain research integrity.
- Recognize where modern tools like *causal inference* and *generative AI* fit within a disciplined workflow.

- Apply *regime thinking* to diagnose strategy vulnerabilities across different market states.

The chapter is grounded in process discipline, followed by a vocabulary for market change, the workflow's structure, the place of causal inference and generative AI within it, regime change as an engineering constraint, and the contrast between institutional and independent contexts, before closing with takeaways.



Your purchase includes a free PDF copy + code bundle

Your purchase includes a DRM-free PDF copy of this book, the code bundle, and additional exclusive extras. See the *Free benefits with your book* section in the *Preface* to unlock them instantly and maximize your learning.

1.1 Why process discipline matters

Financial markets exact a high price for undisciplined inference. They also expose every weakness in a research workflow: an unquestioning reliance on unstable relationships or small, noisy effects, and insufficient consideration of real-world frictions such as liquidity, trading costs, execution timing, and capacity constraints. These challenges are not new, but three developments have increased the value of process discipline relative to model sophistication:

- **Market behavior changes** —sometimes abruptly and sometimes gradually—so assumptions that are held in one period can fail in the next

- **Modeling capacity has grown** , increasing the degrees of freedom available to the researcher and making overfitting easier to produce and harder to diagnose
- **Tooling has accelerated research** , amplifying both good practices (faster iteration, better monitoring) and bad ones (faster data mining, faster deployment of brittle systems)

Recent disruptions did not create these dynamics; they made their consequences more visible. Our core claim is that **durable performance depends at least as much on a disciplined, adaptive workflow steeped in trading reality** as on choosing an especially sophisticated model.

Four shockwaves and the fragility of assumptions

Between 2020 and 2025, markets experienced a sequence of disruptions that stressed assumptions calibrated to the 2010s. The catalysts differed, but the shared effect was the same: relationships learned from recent data proved unreliable when the environment changed. The major catalysts include:

- **The 2020 pandemic (liquidity shock):** Treasury intermediation strained (Duffie, 2020), and correlations shifted abruptly, weakening diversification assumptions and revealing that cross-asset relationships are regime-dependent rather than structural.
- **The 2021 meme-stock episode (sentiment shock):** Sentiment- and positioning-driven flows dominated fundamentals long enough to break short-horizon strategies—especially mean-reversion systems whose holding periods and risk limits assumed faster normalization.
- **The 2021–2023 inflation surge (macro regime shift):** The return of high inflation coincided with higher volatility and altered equity–bond co-movement, stressing models calibrated to a low-inflation

decade and destabilizing common portfolio hedging heuristics (Marshall, 2023)

- **An equity concentration episode (crowding shock):** Index returns became unusually concentrated in a small set of mega-cap names, degrading strategies sensitive to breadth, diversification, and factor balance—and exposing how “market” exposure can become crowded exposure.

These are recent examples, not special cases. Liquidity crises, sentiment cascades, policy shifts, and crowding recur with different triggers. A workflow built for stable conditions is incomplete unless it includes explicit checks for instability and decay.

A vocabulary for change

Adaptive systems require precise language about how the data-generating process changes. Four concepts are commonly conflated but play different roles in research and production:

- **Structural break:** An abrupt change point where the process generating the data shifts. This concept is about *when* the system changed (for example, the onset of COVID-19).
- **Regime:** A persistent state in which market properties (for example, volatility or correlations) are relatively stable within the state but materially different across states (for example, a low-inflation regime versus a high-inflation regime).
- **Drift:** A model-centric description of change. Operationally, drift is not a diagnosis; it is a flag that triggers an investigation into data integrity, feature validity, execution conditions, and regime exposure:
 - **Data drift** occurs when the distribution of input features shifts.

- **Concept drift** occurs when the relationship between features and the target changes. Concept drift is often a model-side manifestation of the same underlying change captured by a structural break; the distinction is a matter of perspective rather than substance (a statistical process versus model error).
- **Online detection** is the operational task of identifying change in real time, using only information available at decision time. This differs from ex post labeling, which is easier but not actionable for trading.

This vocabulary clarifies what a monitoring system must do. A research backtest may tolerate ex post regime labeling; a live strategy cannot.

Why process trumps models

The shocks of the 2020s and many earlier episodes point to a practical constraint: **no single model remains optimal across regimes indefinitely**. The differentiator is not the ability to forecast regime transitions (which is often infeasible), but the ability to rigorously validate ideas, systematically monitor performance, and respond to deterioration without improvisation. As Fabozzi and Stenholm (2025) argue in the context of modern asset management, disciplined preparation and adaptation dominate impulse and overconfidence.

This claim is consistent with Andrew Lo's **Adaptive Markets Hypothesis** (2004), which reframes market efficiency as an evolutionary outcome rather than a permanent condition. In this view, markets consist of heterogeneous participants who learn, compete, and adapt. The degree of efficiency varies with “market ecology,” including the mix of participants, capital, technology, regulation, and available opportunities. Under stable conditions, behavior can resemble a steady state; under

changing conditions, temporary anomalies can emerge, persist, and disappear as the environment evolves.

For practitioners, the implication is operational rather than philosophical.

Key relationships are not structurally stable : risk premia, correlations, and execution costs shift with the participant mix and constraints; profit opportunities decay as they are exploited; and strategies often wax and wane, sometimes reviving when conditions become favorable again. A durable **edge** (a repeatable source of profitable trades), therefore, depends less on selecting a “right model” and more on maintaining a research-to-production loop that detects decay early, distinguishes noise from regime change, and reallocates capital toward what remains robust.

Process as a research-to-production system

We treat machine learning for trading as a managed lifecycle rather than a sequence of disconnected experiments. López de Prado (2018) describes this industrialization as an “ **alpha factory** ”: a repeatable pipeline that generates, evaluates, deploys, and monitors strategies under explicit constraints.

The underlying idea is broader than trading. In applied ML and quality engineering, **models are part of a lifecycle** that includes monitoring, diagnosis, and controlled interventions when performance degrades. Industry evidence indicates that **process failures predominate** over algorithmic failures. Gartner (2018) and RAND’s more recent review of AI implementations (Ryseff et al., 2024) warned that many initiatives would deliver erroneous outcomes due to issues across data, modeling, and organizational practices.

Trading increases these risks because non-stationarity and transaction costs can turn small data or modeling errors into large financial losses. Trading also further complicates modeling “success” because the objective is not predictive accuracy but risk-adjusted performance after costs, under changing conditions.

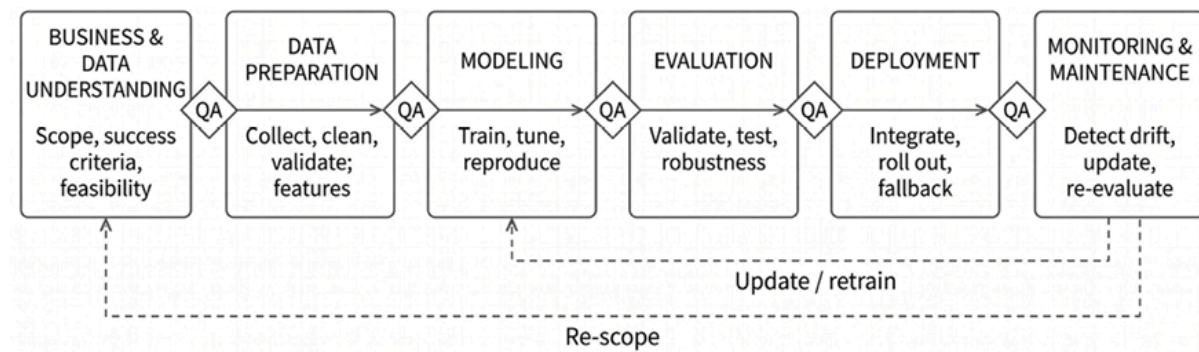


Figure 1.1: Cross-Industry Standard Process for ML with Quality Assurance CRISP-ML(Q)

The workflow introduced in the next section aligns with established engineering frameworks for addressing these failure modes. **CRISP-DM** (**Cross-Industry Standard Process for Data Mining** , 2000) provides a structured process from business understanding through deployment. **CRISP-ML(Q)** (Studer et al., 2021) adapts this logic to modern ML systems. It integrates the understanding of business and data realities, emphasizing explicit handoff checks and post-deployment monitoring (*Figure 1.1*).

We adopt this lifecycle discipline and adapt it to trading constraints: limited effective sample size under non-stationarity, pervasive opportunities for leakage and point-in-time errors, and objectives driven by transaction costs, market impact, and risk constraints rather than predictive accuracy. A disciplined workflow functions as a practical guardrail against common failure modes in quant research:

1. **Data mining and narrative overfitting** : Generating hypotheses after observing outcomes rather than pre-committing to what would

falsify the idea.

2. **Leakage and non–point-in-time data** : Using information that was not available at decision time (revisions, survivorship effects, corporate actions, timestamp errors).
3. **Multiple testing and selection bias** : Searching across many variants until one “works,” then treating noise as signal (Harvey, Liu, and Zhu, 2016).
4. **Ignoring implementability** : Confusing predictability with tradable edge after costs, constraints, and execution frictions.
5. **Sunk costs and delayed exits** : Maintaining a decaying strategy because it once worked, rather than because it remains supported by current evidence.

Undisciplined research tends to cycle through vague hypotheses, rapid backtests, and premature deployment, followed by post hoc explanations when performance breaks down. Disciplined research makes assumptions explicit, defines checks before optimization, and builds a feedback loop from hypothesis to validation to monitoring. Over time, both approaches require comparable effort. Only one produces a workflow that can compound learning and sustain performance in the face of change. Let’s take a closer look at a workflow that leads to disciplined research.

1.2 Introducing the ML4T workflow

The instability described in *Section 1.1* makes any fixed model fragile.

The practical response is workflow discipline: a lifecycle that separates data infrastructure from strategy research, and separates exploration from confirmation.

Figure 1.2 summarizes the ML4T workflow. It adapts standard process frameworks to systematic trading, where dominant constraints include limited effective history, point-in-time data integrity, time-aware evaluation, implementability after costs, risk limits, and monitoring that distinguishes signal decay from execution or risk-control failures.

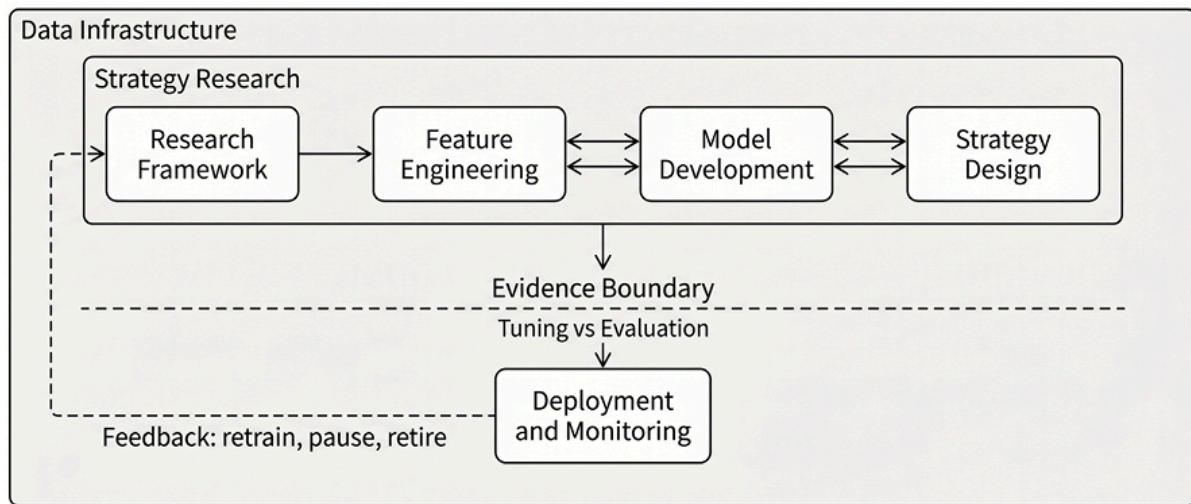


Figure 1.2: The ML for Trading workflow separates a foundational data-infrastructure layer from an iterative strategy-research loop, gated by an evidence boundary that reserves a holdout for confirmation

The workflow has two layers:

1. **Data infrastructure** (*Chapters 2–5*) provides reproducible, auditable, point-in-time-correct data that supports all research and live-trading.
2. **Strategy research** (*Chapters 6–21*) is an iterative loop that refines ideas into tradable systems.

The strategy-research boxes in *Figure 1.2* map to the subsections below: Research Framework, the combined Feature Engineering and Model Development pair, Strategy Design, and Deployment and Monitoring.

The workflow is cyclical and iterative rather than linear: each module produces artifacts used downstream, and live results feed back into revised hypotheses, data checks, and implementation assumptions.

The workflow does not require machine learning. Signals may come from discretionary research, rule-based systems, or learned models; the same structure applies.

Data infrastructure

Data infrastructure is an ongoing investment, not a one-time stage. In trading, the most damaging errors rarely crash code. They inflate backtests by leaking future information, mishandling revisions, or embedding unrealistic assumptions about execution. This layer establishes shared semantics and invariants so results are comparable across projects:

- **Data sourcing and coverage** : Select vendors and feeds, verify identifiers, address missing data, and document sampling and revision policies.
- **Time and availability semantics** : Define event time versus publish time, align features to what was knowable at decision time, and preserve ordering when events share timestamps.
- **Asset-class mechanics** : Apply rules that make data economically meaningful, including corporate actions and fundamental revisions for equities, rolling and stitching conventions for futures, and venue and funding rules for digital assets.
- **Quality invariants and auditability** : Implement checks for survivorship, stale quotes, broken adjustments, and identifier-mapping errors, and ensure pipelines are reproducible so that datasets can be regenerated exactly.

Chapters 2–5 develop these foundations, including microstructure-aware data (how trades and quotes become bars), point-in-time handling for fundamentals and alternative data, and storage and performance choices that affect research velocity. Synthetic data is introduced as an optional

tool, with an emphasis on when it helps (for example, stress testing, privacy, rare-event augmentation) and when it can mislead (for example, distributional mismatch, unrealistic execution conditions).

Research and evidence framework

ML-driven strategy research rarely tests one clean statement like “momentum is positive.” It evaluates a **research pipeline** : a set of data choices, feature families, model classes, and selection rules that together produce trades. That pipeline can surface relationships we did not anticipate, including interactions, regime dependence, or sign reversals. The practical question is not “did this one coefficient differ from zero?” but “will this pipeline produce tradable value when evaluated the way it would be run live?”

Many finance strategies are motivated by a **mechanism** —a plausible economic reason an effect might exist (for example, underreaction, liquidity provision, risk premia, or institutional flows). Mechanisms provide orientation and, later, help interpret performance and diagnose decay. In ML research, however, a mechanism is not a substitute for an explicit **trading specification** .

Credible evidence requires stating, up front, the decision point, the tradable universe, the target horizon, the cost assumptions, and the evaluation protocol that define what would count as success or failure.

Within those constraints, ML can be used as a flexible search tool. **Priors** guide where to look—candidate sources of edge and the measurable proxies that might capture them—while model training discovers the functional form. The core discipline is to fix the choices that determine what evidence means, then iterate downstream without revising the evaluation rules:

- **Decision-time correctness** : Specify the information available at each decision point and align each feature with its availability.
- **Tradability and universe rules** : Specify what can be traded and the constraints under which it can be traded. Fix liquidity and capacity screens and shorting rules up front.
- **Label and horizon definitions** : Define the target and the holding period. Labels define what the model optimizes and what counts as success.
- **Cost-model class** : Specify which frictions apply (such as spreads, slippage, financing, market impact). The class is fixed even if parameters are estimated from data.
- **Evaluation protocol** : Commit to how exploration is separated from confirmation, including the walk-forward structure, holdout design, and trial logging.

Iteration occurs in *feature engineering* , *model development* , and *strategy design* : diagnostics routinely prompt revisions to proxies, targets, model classes, and trading rules. The framework keeps that iteration honest by maintaining measurement conventions and evaluation rules, so improvements reflect stronger signals and better decisions rather than changes in definitions.

Feature engineering and model development

Signal research converts engineered datasets into decision inputs: forecasts, scores, rankings, or state estimates that can translate into trades. Feature engineering and modeling are best treated as a nested loop because diagnostics in each step should update the other:

- *Feature and label engineering* (*Chapters 7–10*) encodes market structure into measurable inputs and targets (such as forward returns,

volatility, drawdown risk, and execution quality). Constraints include low signal-to-noise ratios, misalignment between model objectives and trading objectives, and false discoveries from extensive search. Lightweight evaluation methods such as information coefficients, rolling stability checks, and regime slicing can screen candidates before higher-capacity modeling.

- *Model design and evaluation* (*Chapters 11–15*) begins with strong baselines and transparent diagnostics, then increases capacity as warranted. As flexibility increases, the validation burden increases. Leakage checks, stability across regimes, and sensitivity to perturbations become requirements. Model diagnostics should also be treated as feature diagnostics, as failure modes often reflect data alignment problems, target leakage, unstable measurements, or a mismatch between the target definition and the trading objective. *Chapter 21* extends model development to reinforcement-learning agents that learn execution and allocation policies directly, placing RL alongside the supervised methods of *Chapters 11–15* rather than treating it as a deployment topic.

Time-aware validation is the core discipline. Evaluation must preserve temporal order through walk-forward splits and handle overlapping labels using cross-validation that prevents information leakage. Hyperparameter tuning must respect the same constraint. The objective is not only to estimate performance, but to characterize when a signal fails, why it appears to work, and whether it survives trading frictions.

Strategy design from signals to trades

A prediction is not a trade. Strategy design turns signals into an executable system by specifying how forecasts become positions, how positions become orders, and how the system behaves under constraints. The following must be considered:

- **Signal-to-trade translation** : Define the mapping from model outputs to actions: thresholds, ranks, position scaling, and entry and exit logic, including explicit “no trade” states.
- **Portfolio construction and constraints** : Convert decisions into target exposures while respecting leverage, concentration, liquidity, and risk limits. Portfolio design determines capacity and drawdown behavior and often dominates implementation feasibility.
- **Backtesting as falsification** : Use realistic assumptions about prices, fills, slippage, borrowing, fees, and turnover, and require stress tests across regimes and plausible operational failures.
- **Execution and risk controls** : Incorporate transaction-cost and market-impact models and define risk controls consistent with the scoping constraints position limits, kill switches, hedges, and drawdown-based de-risking.

Backtesting may show that statistically strong signals do not yield a tradable edge, prompting a return to modeling or feature engineering. *Chapters 16–20* expand strategy design across simulation, portfolio construction, transaction costs, risk management, and synthesis.

Deployment and monitoring

The final module connects research to live execution through deployment, monitoring, and feedback loops that enable adaptation. The transition from simulation to production introduces risks that backtests do not fully represent, including execution delays, queue effects, and liquidity shifts. The module covers:

- **Phased deployment** : Use a graduation process: shadow deployment (log orders but do not execute), canary deployment (trade with minimal capital to test the end-to-end pipeline), and full scale only after stability is demonstrated.

- **Monitoring and diagnosis** : Monitoring is a diagnostic system. It should distinguish data health (pipeline breaks, missing fields, timestamp misalignment), signal health (feature drift, label drift, calibration breakdown), execution health (slippage, fill rates, impact), and risk behavior (constraint breaches, exposure creep, drawdowns).
- **Decay detection and lifecycle rules** : Monitoring must detect systematic degradation and specify what actions follow. This includes both model performance (data drift and concept drift, as defined in *Section 1.1*) and strategy performance (profitability, risk profile, execution quality). Implement explicit retrain, pause, and retire triggers instead of ad hoc reactions, and treat these rules as part of the system specification rather than an afterthought.

These triggers make the workflow adaptive rather than linear. *Chapters 25–26* provide frameworks for live trading operations, market-adapted MLOps, and strategy lifecycle governance.

Beyond the iterative workflow, *Chapters 22–24* introduce research-tooling techniques—RAG, knowledge graphs, and autonomous agents—that augment the entire pipeline rather than fitting any single layer.

The evidence boundary

The central discipline mechanism in the workflow is the separation of exploration from confirmation. The goal is not to pre-commit to a fixed number of trials. The goal is to preserve the integrity of evidence despite iterative search. There are two modes:

- **Exploration mode** is where ideas develop. Use available data for diagnostics and iteration, and log trials in a research ledger so the search can be counted and characterized.

- **Confirmation mode** is where evidence is generated. Use a sealed holdout set that is not touched during exploration, evaluate a frozen specification, use predefined metrics, and apply selection-adjusted inference that accounts for the number of trials.

The evidence boundary is a transition, not a single event. Credibility comes from being able to count and characterize the search and from reserving untouched data for final evaluation. The workflow responds to a single fact: the environment changes. Later chapters treat change as an engineering constraint expressed through data availability, leakage risk, costs, and governance.

The next section locates two modern tools within the workflow and makes change tangible with a simple regime example used for risk framing and monitoring, not for regime timing.

1.3 Causal inference and generative AI in the workflow

The ML4T workflow is deliberately tool-agnostic: it is a lifecycle for turning ideas into tradable systems amid uncertainty, costs, and non-stationarity. What *has* changed is the toolkit. We add two families of methods that matter for modern practice, but for different reasons:

- **Causal inference** makes researchers more explicit about both the hypothesis they are advancing and the empirical evidence that would undermine it. It provides a framework for disciplined hypothesis formation, variable selection, and diagnosis when may be confounded by other variables.
- **Generative AI** broadens the range of data researchers can use, especially unstructured text and documents, and accelerates iteration across feature design, coding, analysis, and documentation. Without

guardrails around data provenance, point-in-time availability, leakage, validation, and factual accuracy, it can also amplify errors by producing more unsupported signals, flawed code, and convincing but false explanations at greater speed.

These tools do not replace process discipline. They amplify the consequences of the workflow that employs them: used well, they reduce false discoveries and sharpen diagnosis; used poorly, they accelerate the production of plausible but unsupported results.

We also broaden the scope of “ML for trading.” Beyond return prediction, we address forecasting and decision-making under uncertainty, end-to-end trading systems that run in real time, and richer market structures across asset classes. Concretely, we add:

- **Uncertainty quantification** using conformal prediction to turn model outputs into calibrated decisions
- **Stronger statistical hygiene** control for false discoveries in modeling and backtesting due to repeated experiments
- **Regime-aware modeling and monitoring** , including regime detection and HMMs
- **Modern deep learning for time series**, where it is empirically justified
- **Agentic automation** for research and operations

The workflow is the spine that places these additions into specific modules and shared safeguards, rather than treating them as a grab bag.

Two practical starting points and two deliverable types

Figure 1.3 describes two entry points into the research loop and two types of deliverables that can emerge. These are not mutually exclusive

“modes”—most real projects blend elements of both, and some produce both types of deliverables. The distinction is practical: it clarifies *what artifact you make testable first* and *what “being wrong” would mean*.

The two options are:

- **Prediction-first (complexity-tolerant)** : Start with a forecast target and broad feature set; allow high-capacity models when the objective is economic value (risk-adjusted returns, loss limitation, turnover, capacity). Recent work shows that high model complexity (many features) can coexist with strong out-of-sample performance through regularization and ensemble diversification (Kelly and Malamud, 2025; see *Chapter 9*). This entry point legitimizes high-dimensional forecasting, but only when leakage-resistant protocols, walk-forward validation, and statistical budgeting prevent “lucky” models from being promoted.
- **Mechanism-first (structure-imposing)** : Start with an economic rationale that bounds the search space and defines what should break in the presence of drift. This is not anti-ML; it is a response to limited effective sample sizes and winner’s curse dynamics that make even cross-validated backtests vulnerable to false positives (Arnott et al., 2018). The mechanism imposes discipline: it filters nonsensical strategies, raises the evidentiary bar when the rationale is weak, and makes monitoring actionable when conditions change. Crucially, the “economic story” functions as a *search constraint and an aid to interpretability*, not necessarily as a claim of causal identification.

Both entry points can produce a *tradable signal*—a forecast that drives positions. mechanism-first work can also produce a *measurement deliverable* : a premia estimate, risk attribution, or exposure measurement used to constrain portfolios, construct hedges, or allocate risk budgets. When the deliverable is a measurement, specification correctness matters

differently: misspecification (for example, missing variables) can produce “factor mirages”—estimates that appear precise but are causally wrong and fail in live markets (López de Prado and Zoonekynd, 2025).

A simple litmus test: if being wrong means “it doesn’t make money after costs,” you are in signal territory. If being wrong means “the estimate is biased or misleading,” you are in measurement territory. Use this framing to decide what to freeze first (objective, constraints, and evaluation), not to label projects.

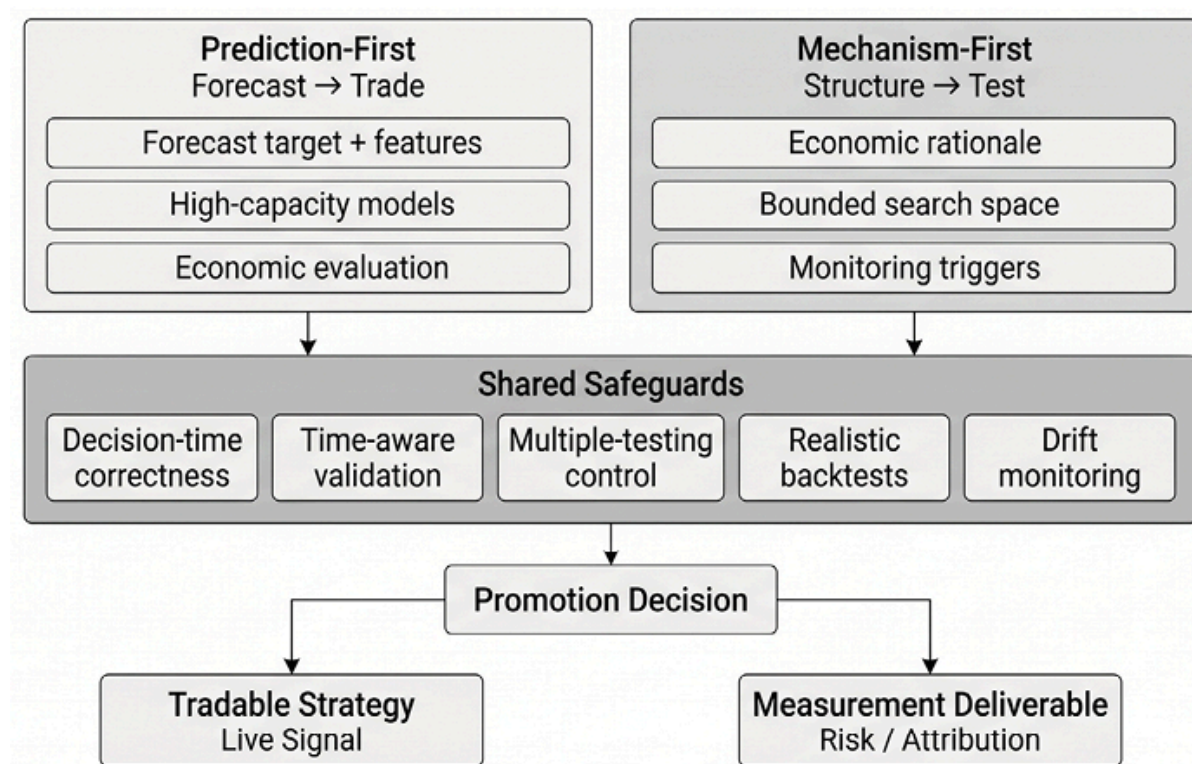


Figure 1.3: Two entry points into one trading research loop

Figure 1.3 shows that both entry points share the same safeguards: decision-time correctness, time-aware validation, multiple-testing control, robust backtests, and drift monitoring. Signal deliverables receive the larger share of treatment in subsequent chapters, which focus on forecasting-driven trading systems. Measurement deliverables—factor premia, risk attribution, hedging—appear primarily in *Chapter 19* and are not a prerequisite for tradable edge.

Causal inference as a discipline-enforcing tool

Causal inference matters here because it strengthens mechanism-first work—both signal and measurement deliverables—and improves diagnosis even when full identification is not feasible.

In trading, prediction alone is an unstable foundation: regimes shift, correlations invert, and the same feature can change meaning when market microstructure, policy, or participant composition changes. A mechanism-based view does not guarantee profits, but it makes research *testable* and monitoring *actionable* : you can state why the strategy should work, which assumptions must hold, and what evidence would invalidate the mechanism.

Stating *why* a strategy should work has always separated durable approaches from data-mined artifacts. In finance, that “why” is hard to establish because we rarely get clean experiments; we work with observational data shaped by confounding, selection, and market feedback. Modern causal inference provides a framework for reasoning under these constraints—using tools such as graphical models, instrumental variables, and causal machine learning—to move from “what predicts returns?” toward “what mechanism could plausibly generate them?”, subject to explicit identification assumptions that must be defended and stress-tested (see *Chapters 9 and 15* ; Pearl, 2019; Schölkopf et al., 2021).

Causal methods have become increasingly common in empirical finance and systematic investing—recognized by the 2021 Nobel Prize in Economics—but their conclusions remain conditional on identification assumptions that can fail under regime change (Fabozzi et al., 2024; López de Prado et al., 2025).

In the ML4T workflow, causal inference is a **discipline-enforcing tool** , not a universal requirement. It helps constrain scoping, avoid “bad controls” and spurious predictors in feature engineering and modeling, and sharpen diagnosis in monitoring by linking performance decay to mechanism-relevant conditions. A mechanism-grounded strategy is easier to monitor under regime change than one built on statistical coincidence—but robust out-of-sample evidence and implementability remain the final arbiters.

In many trading problems, causal identification is hard to justify and easy to overstate; when identification is weak, it is often better treated as a diagnostic lens (what to control for, what not to control for, what to monitor) than as a requirement.

Expanding capability and scaling risk with generative AI

Generative AI matters because it accelerates every entry point, but it also scales leakage, complexity, and confabulation unless the same safeguards are enforced.

Large language models (LLMs) can parse unstructured data, such as earnings calls, filings, and news sentiment at scale, assist with feature engineering, and accelerate research tasks (Luk, 2023). Within the workflow, generative AI can help with reasoning about trading strategies, improve productivity in data processing and feature generation, and support documentation throughout the process (see *Chapters 23 and 24*).

Automation introduces new failure modes. When models generate text, code, features, or even strategy variants, three risks become acute:

- **Hallucination and confabulation:** Plausible outputs that are wrong, especially in summaries of complex market events or in

“explanations” of backtest results.

- **Leakage by construction:** Features, labels, or prompts that inadvertently incorporate future information, not only through subtle timestamp or revision errors but through inclusion in the LLM’s training data.
- **Complexity inflation:** Strategy and pipeline bloat—more moving parts, more degrees of freedom, and more ways to overfit—without commensurate evidence of robustness.

The workflow response is the same: enforce decision-time correctness, pre-commit evaluation protocols, and treat automated outputs as candidates that must pass the same evaluation safeguards as any other research artifact. As automation increases, the practitioner’s role shifts from “manual implementer” to **supervisor** and **validator**, with explicit governance in deployment (see *Chapter 26*).

Workflow stage	Machine learning	Generative AI
Strategy hypothesis	Anomaly detection and data-quality scoring	Synthesis and hypothesis checklists
Data and QA	Representation learning and embedding features	Document parsing and entity mapping
Feature and label design	Forecasting models and uncertainty calibration	Event extraction and feature ideation
Modeling and evaluation	Risk models and constrained optimization	Coding assistance and experiment summaries
Portfolio and constraints	Impact/cost models and execution policy tuning	Scenario enumeration and constraint encoding
Execution and costs	Drift detection and performance attribution	Venue-rule summaries and runbook assistance
Monitoring and operations	Shared with adjacent stages	Alert triage and incident summaries

Figure 1.4: ML/AI-augmented trading workflow

Figure 1.4 maps machine learning and generative AI contributions across the seven workflow stages: strategy hypothesis, data and QA, feature and label design, modeling and evaluation, portfolio and constraints, execution and costs, and monitoring and operations. At each stage,

generative AI accelerates synthesis, documentation, and code production, while machine learning handles measurement: anomaly detection, forecasting, calibration, risk modeling, and drift attribution.

ML and generative AI can assist throughout, but automated outputs remain candidates that must pass the same evaluation safeguards as any other research artifact; monitoring closes the loop by triggering retraining or strategy revision when assumptions fail. We next turn to the challenges of constant change that any live strategy will inevitably face.

1.4 Keeping up with changing market regimes

The disruptions described earlier share a common feature: they reflect shifts in the market environment. A strategy should not try to predict specific events. It should answer an operational question: which market states are adverse for the strategy, and how can those states be detected in time to reduce exposure?

As defined in *Section 1.1*, a **regime** is a persistent market state in which the joint behavior of returns changes materially relative to other periods. Relevant dimensions include expected returns, volatility, cross-asset correlations, and liquidity. A strategy can be correct about its economic mechanism yet fail when its operating assumptions are violated. For example, a strategy calibrated to low volatility can break down when volatility rises, while correlations increase and liquidity deteriorates. Regime awareness is primarily a risk lens, not a return-timing tool.

Regime detection with labeling, conditioning, and monitoring

Regime methods support three tasks that share techniques but require different evaluation standards:

- **Ex post labeling (explanation)** : Cluster historical data to build a vocabulary for when a strategy struggled and which market variables changed. This is descriptive and may use the full sample.
- **Backtest conditioning (robustness)** : Evaluate performance conditional on labeled states. This is a stress test and must prevent leakage of regime labels into decision-time features or trading rules.
- **Live monitoring (operations)** : Detect state changes using only information available at decision time, and evaluate performance using a walk-forward approach. The objective is early warning and risk control, not “regime-timing” alpha.

This separation matters because the most common misuse is to treat ex post labels as if they were known in real time. In this chapter, we use regimes to illustrate *diagnostic overlays* (what tends to break, and what risk action follows), not to propose regime-timing strategies. Throughout the book, regimes are used as *context* for stress testing and monitoring, not as signals that reliably forecast returns.

A practitioner literature applies unsupervised learning to construct regime maps of market history. For example, Two Sigma (Botte and Bao, 2021) applies **Gaussian mixture models (GMMs)** to investment-style returns. Horváth et al. (2021) use Wasserstein k-means (demonstrated in *Chapter II*). Uysal and Mulvey (2021) use supervised learning to characterize regime-dependent behavior in risk-parity portfolios. The examples below follow this general approach.

Style regimes with century-long factor data

The notebook `factor_regimes.ipynb` uses the **Century of Factor Premia** dataset provided by AQR, one of the largest hedge funds managing over USD 100bn, which reports monthly returns for a broad equity market proxy and several factor portfolios (Ilmanen et al., 2021). These series represent **standardized systematic exposures**—that is, returns to broad, rule-based *styles* (such as value, momentum, carry, or defensive) that can be held across many securities and are designed to reflect common risk/return drivers in markets—rather than returns of individual stocks. The notebook standardizes the series and clusters their joint behavior to identify recurring states in how market and factor returns co-move across decades.

What the model estimates

The notebook fits GMMs with 2–6 components; a two-state specification is the most stable and interpretable in this dataset. AIC prefers $K=6$, but the silhouette at that choice is near zero and the partition fragments after 1950—a useful reminder that AIC alone is a poor model-selection criterion when the goal is an interpretable regime map. The clusters are then labeled ex post as “Risk-On” and “Risk-Off” based on which cluster coincides with worse equity-market outcomes. This labeling is a post hoc interpretation, not a tradable real-time signal.

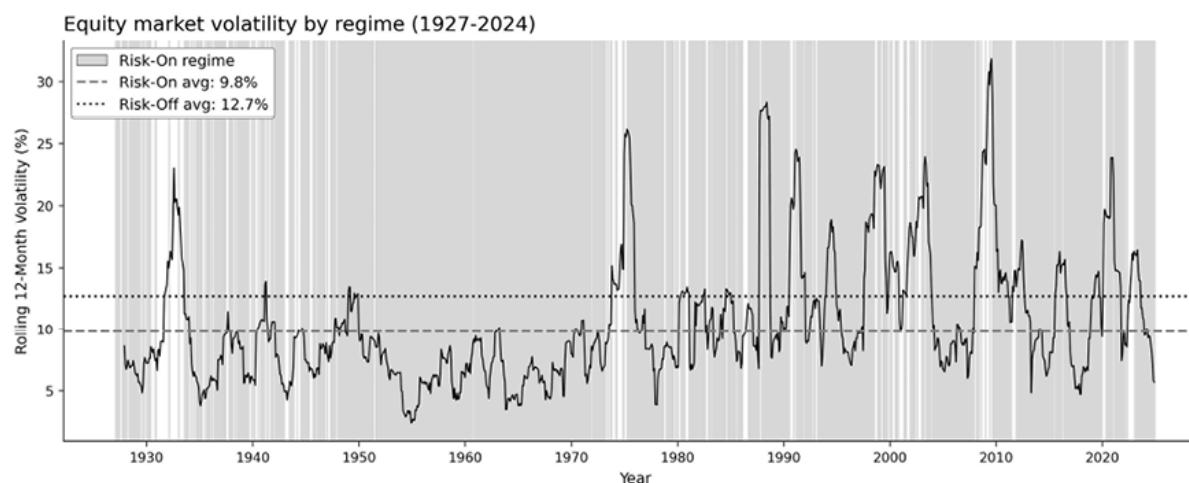


Figure 1.5: Equity market volatility by regime, 1927–2024

Figure 1.5 overlays regime membership with rolling 12-month equity volatility (annualized) and summarizes volatility by regime:

- Risk-Off volatility is 19.1% versus 8.6% in Risk-On (a $2.2\times$ ratio), and the equity market Sharpe ratio drops from 1.11 to 0.12.
- Risk-Off also concentrates drawdown risk, with a maximum drawdown of -77% versus -23% in Risk-On.
- Factor behavior diverges across regimes as well: value is countercyclical ($+5.3\%$ annualized in Risk-Off versus $+1.6\%$ in Risk-On), while carry (-0.6%) and defensive (-0.5%) turn negative—precisely when diversification is most needed.

A coarse two-state map can separate risk environments in a way that supports concrete controls. The correct approach is not “buy when Risk-On,” but to adjust the risk posture when the environment shifts. With 267 regime transitions over 98 years (roughly every 4 months on average), the model switches frequently enough to be noisy as a tactical signal—which reinforces the point that regimes are for risk management, not timing.

In workflow terms, this example illustrates what a regime overlay should provide:

- A compact vocabulary for “calm” versus “stressed” conditions
- A measurable link to risk (here, realized volatility, Sharpe ratio, and drawdown)
- Evidence that factor exposures behave differently across states
- A mapping to risk actions (position sizing, exposure caps, and buffers)

Macro regimes and volatility environments

The factor regime map is a “style” lens: it clusters systematic exposures by how they co-move. A macro regime map clusters *economic conditions* and then evaluates how market risk outcomes vary across those clusters.

The notebook `macro_regimes.ipynb` uses four monthly indicators from FRED:

- **UNRATE:** Unemployment rate
- **DFF:** Federal funds rate
- **T10Y2Y:** Term spread (10-year minus 2-year yield)
- **CPIAUCSL:** Consumer price inflation (year-over-year change)

The CPI is converted to a year-over-year rate because the raw price level is non-stationary and would dominate the clustering. The series is resampled to monthly observations, standardized, and clustered using a four-component GMM. The notebook reports a silhouette score as a separation diagnostic and summarizes cluster means. A complementary hierarchical clustering pass on the broader macroeconomic state vector yields a cophenetic correlation of 0.710, indicating that the dendrogram preserves the pairwise distance structure reasonably well—Ward, GMM, and K-Means give comparable regime structure on this panel.

Because unsupervised clusters are unlabeled, the notebook assigns short interpretive names based on cluster means (for example, high unemployment with near-zero rates suggests a “crisis” configuration; rising rates with a flatter curve and elevated inflation suggest “tightening”). These names are interpretations of the macro configuration, not claims about subsequent market performance.

Market validation - Volatility and drawdowns

To connect macroeconomic regimes to investable outcomes, the notebook uses monthly S&P 500 data to evaluate regime-conditional realized volatility and maximum drawdown, with major episodes annotated (for example, the global financial crisis, COVID-19, and the inflation shock). In many applied settings, macro regimes separate *risk conditions* (volatility and drawdown) more cleanly than they separate average returns. Therefore, the notebook treats macro regimes as inputs to **risk management** rather than as return forecasts.

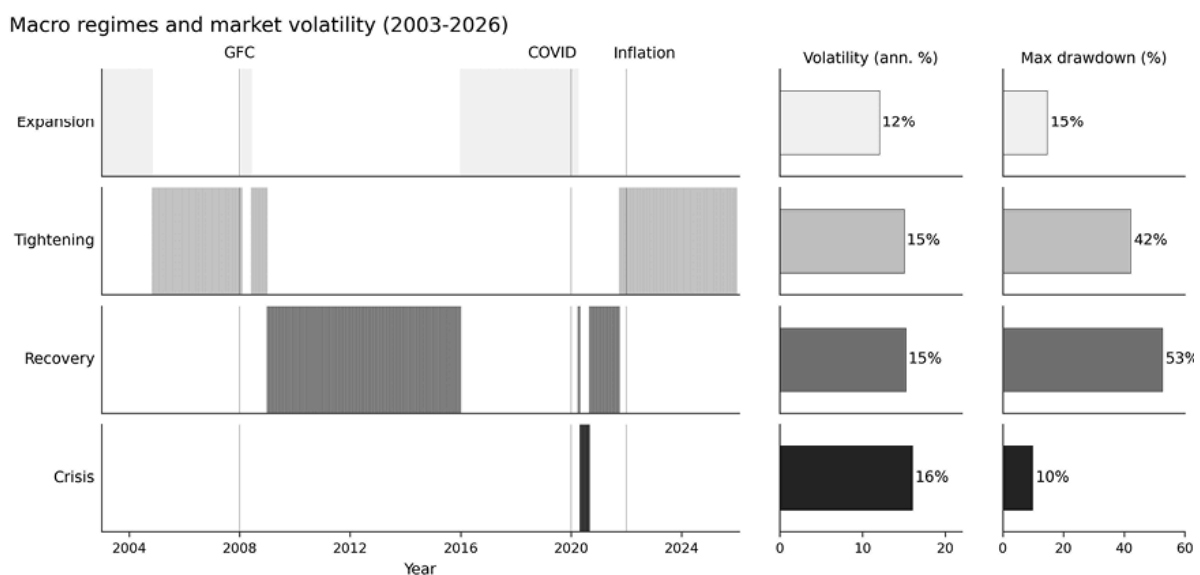


Figure 1.6: Macro regimes and market volatility, 2003–2026

Figure 1.6 shows regime membership over time and summarizes per-regime outcomes (annualized volatility and maximum drawdown), sorted by volatility. The goal is not to claim that macro regimes forecast returns, but to show that macro conditions provide an interpretable overlay for when risk tends to be higher and drawdowns deeper—conditions under which exposure caps, hedging triggers, or de-risking rules are most valuable.

The notebook also includes an extended analysis that expands the feature set to a broader collection of FRED series (for example, breakeven inflation, credit spreads, monetary aggregates, housing indicators, and

volatility indices), subject to coverage filters. This yields a richer view of economic conditions at the cost of more noise and more modeling choices.

Real-time detection and risk management

Detecting regime changes as they occur is harder than labeling them after the fact. Sequential change-point tests and regime-switching models (*Chapter 11*) can help, but they must be evaluated strictly in a walk-forward design with decision-time features and out-of-sample monitoring metrics.

For the workflow in this chapter, the practical recommendation is narrower: monitoring should include indicators aligned with a strategy's known vulnerabilities, such as volatility spikes, correlation breaks, liquidity deterioration, or macro-policy pivots. The objective is not to perfectly classify a regime, but to trigger predefined risk actions when conditions that historically harmed the strategy reappear.

Regime models can create false confidence. Common pitfalls include:

- **Look-ahead bias** : Fitting or labeling on the full history and then using those labels in trading logic.
- **Degrees of freedom** : Results are sensitive to features, preprocessing, window lengths, and the number of states.
- **Over-interpretation** : Regimes summarize recurring patterns; they are not explanations.

Use regimes as a **risk lens** : identify which environments degrade the strategy and specify how risk controls will respond to them. The question is not “what regime is the market in?” but “how does this strategy behave when these conditions arise?” For example:

- **Momentum** : Does performance degrade when correlations spike or reversals cluster?
- **Carry** : What happens during funding stress and policy pivots?
- **Mean reversion** : how do drawdowns change when volatility rises and liquidity thins?

Process discipline requires that this environmental awareness be explicit. Pre-commit to regime-sensitive failure criteria, define the monitoring statistics that detect those conditions, and specify the risk actions that follow. We will discuss how to apply process discipline in different settings.

1.5 Independent versus institutional workflows in the real world

The workflow is universal, but the dominant failure modes differ by environment. In large institutions, specialization and independent review create natural friction: execution, risk, and data constraints are enforced by people whose incentives are not aligned with making the backtest look favorable. In a small team or solo practice, that friction is weaker. The primary risk is not insufficient sophistication—it is *unconstrained interpretation* : iterating until a result looks convincing, then treating it as evidence rather than as a candidate that emerged from search.

In the independent setting, we run the same lifecycle (see *Section 1.2*) without external gatekeepers. We must therefore supply our own governance through decision-making discipline: documentation, explicit assumptions, and checkpoints that make it easy to stop, revise, or retire ideas based on the evidence.

Solo failure modes that institutions partially avoid

Institutional infrastructure does not guarantee success, but it mitigates several predictable self-inflicted errors we must address deliberately:

- **Goalpost drift** : After seeing results, it becomes easy to redefine success—accepting lower Sharpe for smaller drawdown. The failure mode is not bad math; it is changing the question after observing outcomes.
- **Assumption stacking** : Strategies often appear profitable only because multiple small assumptions each lean favorably: optimistic fills, end-of-bar execution, understated costs, ignored capacity, or a favorable sample period. Each may seem harmless in isolation; together, they can manufacture performance.
- **Flexibility without accounting** : Large feature sets, extensive tuning, and flexible models increase the odds of finding a lucky winner unless the result is treated as conditional on the search that produced it. Without a research record, it becomes impossible to distinguish learning from selection effects.
- **Late tradability discovery** : Without a separate execution function, weeks of signal refinement can produce a result that cannot survive realistic spreads, slippage, latency, liquidity limits, or risk constraints.

These are not philosophical problems but workflow problems. We address them by making assumptions explicit early, recording the search, and requiring basic checks for implementability and robustness before investing in refinement.

Decision discipline through documentation and checkpoints

A practical solo process requires a small set of documented decisions and checks—not to discourage iteration, but to keep evidence interpretable after we iterate.

Scoping check (implementability first)

Write down the decision point and information set: what arrives when, why it might be slow to price, and what frictions plausibly prevent immediate arbitrage. Record a small number of “stop criteria” that justify ending the project early—turnover that makes costs dominant, capacity that is incompatible with the intended deployment size, or instability across obvious regime slices (see *Section 1.4*).

Data integrity check

Prefer definitions where “what the model knew at time t ” is unambiguous. Messy point-in-time semantics, corporate actions, revisions, timestamps, or session alignment add uncertainty that can invalidate downstream results.

Signal check (robustness before cleverness)

As flexibility increases, the burden of validation increases. The key question is not “does it look significant,” but “does it survive variation”: different walk-forward splits, plausible regime slices, modest changes in preprocessing, and conservative cost assumptions. Signals that only work in a narrow configuration should remain in the exploration phase.

Tradability check (fragility audit)

For independent setups, sensitivity tests carry more information than point estimates:

- Does the edge persist if costs are doubled?
- If execution is delayed by seconds or minutes?
- If position sizes or participation are capped?
- If entry and exit timing shifts within the bar?

A strategy that degrades gracefully is more valuable than one that relies on a precise set of optimistic assumptions.

Monitoring check (diagnosis maps to action)

The minimum viable monitoring requirement is the ability to separate signal decay from operational failure. When performance drops, the diagnosis must distinguish a weakened signal, a degraded data pipeline, and rising execution costs, because each implies a different response. Without this separation, intervention reacts to noise, and intervention is itself a source of overfitting.

When failure rates are high, fast, well-documented rejection and clean post-mortems accelerate throughput more than any single refinement.

Constraints and asymmetric advantages

Independent researchers generally should not compete where institutional advantages are structural:

- **Speed and microstructure** : If the strategy depends on being first, it is unlikely to be viable. Favor strategies whose economics tolerate seconds-to-minutes of delay and still survive after costs.

- **High fixed-cost data as prerequisite** : If success requires expensive institutional datasets as a starting point, the capital and tooling gradient is steep. Prefer problems where information is accessible but interpretation and discipline are the differentiators.

Where independents can be advantaged:

- **Capacity-constrained opportunities** : Many effects do not scale. A large fund may ignore them because it cannot deploy size without excessive impact. Independents can operate below the capacity ceiling, provided capacity is modeled honestly and small-size viability is treated as a strategy class rather than a flaw.
- **Tighter iteration loops** : Faster movement from result to diagnosis to revision compounds. The advantage comes not from trying more configurations but from cleaner diagnostics and reusable tooling that lower the marginal cost of the next experiment.

Compounding returns with reusable infrastructure

For independents, the highest-leverage investment is reusable infrastructure that reduces the marginal cost of running disciplined experiments:

- Dataset versioning and point-in-time pipelines
- Standardized backtest and evaluation harnesses
- Shared cost and slippage models with sensitivity tests
- Monitoring templates that map failure modes to actions

Backtest harnesses (*Chapter 16*), cost models (*Chapter 18*), risk and monitoring (*Chapter 19*), and live-operations machinery (*Chapter 25*) develop these in depth. Every strategy we run correctly makes the next

one faster, not through a learned trick but through accumulated machinery that makes discipline cheap and repeatable.

1.6 Summary

This chapter's core claim is that durable trading performance depends less on finding a "right model" and more on maintaining a disciplined research-to-production workflow that withstands non-stationarity, costs, and operational frictions. Market shocks are routine occurrences in which relationships shift, and assumptions fail; without disciplined iteration, backtests document narratives rather than supply evidence.

The ML4T Workflow provides that discipline: a persistent data-infrastructure foundation and an iterative research loop that moves from scoping invariants (decision-time correctness, tradability, labels, cost model, evaluation protocol) to features, models, strategy design, deployment, and monitoring. The evidence boundary formalizes the separation of exploration from confirmation through trial logging, sealed holdouts, and selection-aware inference. Causal inference and generative AI can strengthen this process, but in the absence of guardrails, they also amplify failure modes, especially for independent researchers who lack institutional friction.

Chapter 2 , The Financial Data Universe , lays the foundation by introducing the asset classes, data structures, and storage choices that the workflow operates on.

2

The Financial Data Universe

The financial data landscape is expanding, and the surface area for analytical error grows in parallel. Alternative data providers have proliferated over the past decade alongside the broader expansion of large-scale data infrastructure, adding satellite imagery, credit card panels, and social sentiment feeds. Exchanges publish once-proprietary microstructure feeds. New providers are broadening access to institutional-grade market data. Prediction markets such as Polymarket and Kalshi add a new class of event-contract data. This expansion creates both opportunity and complexity.

Rigorous data management determines whether that complexity becomes insight or noise. Point-in-time errors, survivorship bias, and mishandled corporate actions have always invalidated backtests; each new source expands the failure surface. The value alternative data adds depends on the strategy, but it increases the number of ways a pipeline can fail. Vendor evaluation requires discipline across quality, legal, and technical dimensions. Storage decisions between parquet files, time-series databases, and lakehouse formats shape research velocity and production feasibility.

Chapter 1 established the ML4T workflow and emphasized process discipline. Before specifying strategies, we need to understand the data landscape—its possibilities, constraints, and failure modes. This chapter maps that universe, focusing on *three questions* : what kinds of data exist, which specific data challenges arise across asset classes, and which decisions matter along the dimensions of quality, sourcing, and storage. The following chapters deepen this foundation: market microstructure (*Chapter*

3), fundamental and alternative data (*Chapter 4*), and synthetic data generation (*Chapter 5*). Strategy specification (*Chapter 6*) builds on these data foundations. At the end of this chapter, you will be able to:

- Distinguish between Market, Fundamental, and Alternative data and their quality challenges.
- Evaluate the trade-offs between data availability and transparency across major asset classes.
- Apply a five-dimensional data-quality framework and identify specific failure modes in financial data.
- Conduct systematic vendor due diligence across technical, legal, and commercial factors.
- Select storage technologies based on access patterns and operational requirements.

This chapter and the next three focus on **dataset characteristics** —what data exists, how it behaves, and what can go wrong. They cover the raw material for strategy development, a process that begins in *Chapter 6* .

2.1 A Modern taxonomy of financial data

A dataset is a set of measurements, along with the rules that make those measurements comparable over time. It is a particular way of measuring real-world activity. That measurement comes with built-in definitions: timestamp conventions, trading calendars, adjustment policies, identifier schemes, and rules for revisions or restatements. These are not optional details; understanding them is necessary to understanding what the data *means* .

Because every dataset embeds definitions, the first question is what real-world process it measures: trading, fundamentals, or proxies. Financial data falls into three categories:

- **Market data** : Records what the market did—quotes, trades, and derived aggregates
- **Fundamental data** : Captures economic drivers of value (definitions vary by asset class)
- **Alternative data** : Signals not in standard market or fundamental feeds, often proxies for fundamentals

This taxonomy helps you reason about the kinds of real-world activity a dataset represents, what it omits, and the errors it can introduce. When you download “daily prices,” you are adopting the provider’s definitions. Define “close”: last trade, auction close, or a timestamped snapshot (and in which timezone)? Are splits and dividends reflected point-in-time or back-adjusted? If you get any of these wrong, you may not see an obvious failure. Instead, you get silent misalignment and biased results that propagate through feature engineering, labels, and backtests.

Before you model anything, lock down four items: timestamp semantics, corporate-action adjustment methodology, identifier stability, and the handling of revisions and restatements. Make these choices explicit in your pipeline configuration and carry them forward as metadata, so downstream code cannot accidentally reinterpret the data. These issues are the focus of this chapter and the next two.

Market data - A hierarchy of aggregation

Market data reflects trading activity in financial markets. It embodies the institutional setting that shapes this activity, from centralized exchanges to over-the-counter environments. It originates in real time with message-level events related to individual orders and executions. Vendors typically aggregate this data in real time over a given interval to produce the familiar open, high, low, close, and volume summaries. Moving up the aggregation

spectrum makes data easier to store and model, but discards micro-level information about supply and demand dynamics.

At the base, raw event streams reflect the mechanics of the trading venue: how orders are posted and matched, when trades are reported, and what trading rules apply. In modern markets, trading is split across multiple venues, including off-exchange venues such as dark pools, internalizing broker-dealers, and other alternative trading systems. As a result, “the best price” you see depends on which venues your feed covers and how it merges them.

Exchanges sell high-quality data products with strong completeness guarantees and validation fields; free feeds are often delayed, filtered, or missing metadata needed to verify correctness. Sourcing and validation discipline applies even to “basic” price data, regardless of cost.

Chapter 3 covers working with market data across levels of aggregation, including order-book reconstruction from tick data and the construction of analytical bars at multiple frequencies.

Fundamental data - Drivers of value with release lags

Fundamentals are economic variables that justify valuations. What counts as “fundamental” depends on asset class and institutional framework:

- **Equities** : Financial statements, corporate actions, guidance
- **Commodities** : Inventories, production, shipping, weather
- **Foreign Exchange (FX)** : Interest rates, inflation, growth, policy
- **Crypto** : Issuance schedules, fees, on-chain activity, liquidity structure

Two properties drive most engineering errors:

- **Release-time data, not event-time data.** Fundamentals arrive with lags, schedules, embargoes, and revisions. The relevant question is not

“what is the value,” but “what was knowable at decision time.”

- **Shaped by rules.** Accounting standards determine what equities disclose. Government agencies define commodity reporting. Statistical agencies revise macro series and publish vintages. Protocol rules govern which crypto variables are observable. Interpreting fundamentals requires understanding institutional constraints, not just downloading a series.

Chapter 4 works with three primary sources of fundamental data:

- For **equities** , we use regulatory filings to extract balance sheet and income statement data, using point-in-time filing dates to support bitemporal queries that respect announcement timing.
- For **macro** , we use the Federal Reserve’s FRED data service to retrieve Treasury yields, unemployment claims, inflation measures, and volatility indices, with explicit handling of release schedules and vintage corrections.
- For **commodities** , we use CFTC Commitment of Traders reports to track institutional and speculator positioning in futures markets.

Alternative data - High variety, high validation burden

Alternative data extends observations beyond standardized price feeds and structured public information, such as regulatory filings. The rise of alternative data over the last decade reflects the broader trend toward digitalization (Ekster and Kolm, 2020).

Many categories of alternative data give timelier or more granular views of economic activity, which makes them useful complements to fundamental data:

- **Geospatial/mobility** : Satellite imagery, foot traffic, shipping AIS
- **Consumer analytics** : Credit/debit panels, app usage, web traffic

- **Corporate exhaust** : Job postings, procurement, supply-chain signals
- **Text/events** : News, earnings calls, filings, social media
- **ESG/disclosures** : Emissions, workforce metrics, governance ratings

While alternative data may provide novel measurements, it comes with its own interpretation challenges. For each category, the validation questions are: How stable is coverage over time? What selection effects exist in the sample? Can you reproduce the methodology? Do usage rights permit your intended deployment?

Chapter 4 applies the alternative data evaluation framework to three categories:

- **Prediction markets** —both CFTC-regulated Kalshi (economic events: Fed decisions, CPI, unemployment) and crypto-based Polymarket— show how to extract probability time series and engineer features from event contracts.
- **On-chain crypto data** from DeFi Llama provides Total Value Locked (TVL) metrics for protocol activity, evaluated as a forward-return predictor across major chains (see *Chapter 4* for the panel construction and IC analysis).
- **Text data** from SEC EDGAR filings demonstrate extraction pipelines that feed into NLP features, covered in more detail in *Chapter 10* .

The next section surveys market data across asset classes, lists the datasets used throughout the book, and points to the notebooks that explore them.

2.2 The asset-class market data landscape

Data characteristics vary across asset classes because market structures and economic determinants of value differ. The organization of trading— exchanges versus over-the-counter (OTC), electronic versus voice, consolidated versus fragmented—determines what market data you can

obtain, how informative this data is, and where engineering choices affect results.

Table 2.1 summarizes observability, key failure modes, and engineering priorities across asset classes:

Asset Class	Observability	Key Failure Modes	Engineering Priority
Equities	High (consolidated)	Corp actions, fragmentation, identifiers	Adjustment methodology
Futures	High (exchange)	Roll rules, continuity method	Continuous series construction
Options	High (but sparse tails)	Illiquidity noise, surface construction	Surface representation
Digital Assets	Medium (venue-specific)	Volume integrity, 24/7 sessionization	Venue screening
FX	Low (OTC)	No consolidated tape, close conventions	Aggregation rule definition
Fixed Income	Low (OTC, sparse)	Matrix pricing, indicative versus firm	Liquidity inference
Swaps/OTC	Low (reported)	Curve construction, convention mismatches	Curve-based representation

Asset Class	Observability	Key Failure Modes	Engineering Priority
Commodities	Medium (futures high)	Spot ambiguity, delivery specs	Contract spec in metadata

Table 2.1: Asset class overview

The subsections move from exchange-traded instruments (equities, futures, options, digital assets), where data is richer and more standardized, to OTC markets (foreign exchange, fixed income, swaps, commodities), where data is sparser and conventions more fragmented.

Equities

Global equity market capitalization was approximately \$126.7T (WFE, 2024); U.S. equity market cap was about \$67.7T (SIFMA, Q3 2025). Listed equities trade on multiple lit exchanges and off-exchange venues (ATS/dark pools and retail wholesalers/internalizers). In the U.S., consolidated reporting defines a market-wide best bid and offer (NBBO) and publishes consolidated quotes and trades; direct venue feeds can be faster and expose richer detail, but cost more.

What you observe

Equity market structure varies along three axes that affect what you can observe and how you should define “the price”:

- **Consolidation versus fragmentation.** Some jurisdictions provide consolidated reporting, while others have historically been fragmented across venues. The EU’s consolidated tape for equities launches in 2026.
- **Auction intensity.** Many markets rely heavily on opening and closing auctions, which can dominate end-of-day liquidity and benchmark pricing. “Close” often means “auction close,” not “last trade.”

- **Trading constraints and conventions.** Tick sizes, short-selling constraints, price limits/volatility interruptions, and settlement conventions differ across markets and can alter the meaning of intraday liquidity measures and transaction cost estimates.

Quotes, trades, and derived bars are widely available for liquid names, but “best price” and “available liquidity” depend on feed scope and aggregation rules. Treat each venue feed as a partial view of the market; any consolidated view is a constructed aggregate with its own timestamps, inclusion rules, and latency profile.

Failure modes

Fragmentation creates ambiguity about timestamps and aggregation (especially when stitching venues or mixing consolidated and direct feeds). Corporate actions (splits, dividends, spinoffs) create discontinuities that require consistent adjustment and return definitions. International datasets add additional pitfalls: currency conversion and FX timestamps, withholding taxes for dividend-inclusive series, and cross-listing/ADR mapping that can break identifier joins.

Engineering decisions

Before research, clarify and document:

- Your definition of “close” (auction versus last trade; local close convention)
- Whether you use consolidated versus venue feeds for quotes
- Your corporate-action and total-return methodology
- Identifier policy for cross-listed names

These are dataset design choices with material impact on results.

Datasets

We use free daily data on 3,199 US equities (1962–2018) sourced from NASDAQ (the original Quandl WIKI price data, no longer updated after the Quandl acquisition). We also use two commercial datasets generously provided by Algoseek: daily OHLCV bars for S&P 500 constituents (2017-21) and minute bars for NASDAQ 100 constituents (2020-21), enriched with several dozen ML-focused market microstructure features.



Notebooks : `01_us_equities_eda` explores US equity characteristics; `02_corporate_actions` validates adjustment factors and compares unadjusted versus adjusted returns. We will explore the Algoseek NASDAQ 100 data in *Chapter 3* on Market Microstructure.

Exchange-traded products

Exchange-traded products (ETPs) are *exchange-listed wrappers that package exposures* to one or more underlying markets. They trade like equities—with quotes, trades, and order books—but holdings, index rules, and replication mechanics. ETPs may wrap bond, commodity, FX, volatility, or multi-asset exposures, so their trading and risk characteristics reflect both the wrapper and the underlying assets. ETPs also differ across common structures:

- **Exchange-traded funds (ETFs)** use a **creation/redemption** mechanism with authorized participants that links secondary-market prices to underlying asset values.
- **Exchange-traded notes (ETNs)** are unsecured debt instruments; they embed **issuer credit risk** and may track an index via an issuer promise rather than a fund structure.
- **Closed-end funds (CEFs)** generally lack continuous creation/redemption and can trade at **persistent premiums/discounts** to NAV.

What you observe

Consolidated quotes and trades, bars, and (from direct feeds) deeper order book information. Unlike single-name equities, ETP research requires a second layer of data:

- **NAV** (typically end-of-day) and, for many ETFs, an **intraday indicative value** .
- **Holdings and weights** (or index constituents) are often updated daily, but sometimes disclosed with or as representative baskets.
- **Fund actions** such as distributions, splits, and (for leveraged/inverse products) periodic rebalancing that alter exposure through time.

Failure modes

ETPs bundle a range of different products; therefore, the failure modes multiply:

- **Return definition.** Distributions like dividends or coupons are often material. Price-only series can understate returns and distort volatility; the “adjusted close” embeds vendor-specific adjustment methodology (see *Section 2.3* and *Chapter 4*). Be also explicit about (i) total versus price return, (ii) reinvested versus cash distributions, and (iii) taxes and currency treatment where relevant.
- **Premium/discount.** Traded prices can deviate from NAV, especially when the underlying market is closed, illiquid, or stressed. Avoid treating NAV returns as tradable.
- **Liquidity mismatch.** A liquid ETF can wrap illiquid underlyings. Secondary-market liquidity can overstate true capacity when flows are transmitted to constituents through creation/redemption or dealer hedging, particularly in stress regimes.
- **Look-through leakage.** Holdings and baskets change through rebalances and at the manager’s discretion, and public holdings data

may not be point-in-time. Using today's holdings to compute yesterday's exposures creates subtle forward-looking bias.

- **Replication complexity.** Many ETPs hold complex products: commodity products may be **futures-based**, leveraged/inverse products are **path-dependent** due to daily rebalancing, and options-overlay products embed *volatility surface* dynamics.

Engineering decisions

Treat ETPs as a distinct dataset type. At minimum, lockdown:

- **Return methodology:** Price versus total return; distribution handling; currency conversion.
- **Valuation anchor:** Price, NAV, or both, and which is used for signals versus evaluation.
- **Exposure mapping:** Holdings versus index constituents versus factor proxies; point-in-time and disclosure-lag policy.
- **Liquidity model:** ETF-level costs versus look-through capacity constraints for large orders/stress regimes.
- **Replication metadata:** Physical versus synthetic; futures-based; leveraged/inverse; options overlay.



Datasets and notebooks : We use daily data on 100 ETFs (2006–2025) sourced from Yahoo Finance, with 50 categorized by asset class for the rotation case study. Liquidity spans a $23\times$ range between SPY (126M shares/day) and the Commodities bucket (5.5M); 41 of the 100 ETFs launched after 2006, constraining lookback for features and evaluation. The notebook `03_etfs_eda` introduces this dataset.

Futures

Global exchange-traded futures volume in 2024 was about ~28B contracts (FIA). Futures trade on exchanges with transparent contract-level data. The complication is identity over time: a market is a sequence of expiring contracts with shifting liquidity.

What you observe

High-quality prices, volume, and open interest per contract. No perpetual ticker exists—you must construct one.

Failure modes

Roll rules (calendar, volume, open-interest) and continuity methods (ratio adjustment, difference adjustment) are backtest-defining choices. Different strategies need different roll logic.

Back-adjusted continuous series typically preserve futures price PnL but do not embed collateral return (margin interest). Back-adjusted continuous series often behave like **excess-return** proxies unless you explicitly model collateral return. When comparing to spot assets or total-return indices, be explicit about whether you model futures PnL, collateral return, or their sum.

Engineering decision

Store raw contract histories, along with one or more continuous variants so you can test and reconstruct roll decisions.

Datasets

We use hourly data on 30 CME futures for a diverse set of underlyings covering 2011-25, sourced from Databento.



Notebooks : `04_cme_futures_eda` explores the futures data; `05_futures_session_aggregation` shows how to align the data with CME trading sessions; and `06_futures_continuous` demonstrates volume-based roll detection and the two mainstream back-adjustment methods (ratio and difference), validated against the vendor's pre-built continuous series.

Options

Global exchange-traded options volume in 2024 was about ~177B contracts (FIA). Listed options are electronically quoted, but the instrument space explodes across strikes, expiries, and types. Liquidity is highly uneven—concentrated in near-money, near-dated contracts.

What you observe

Rich quote and trade data for liquid strikes; sparse and noisy data elsewhere.

Failure modes

Storing and modeling the whole chain is expensive and mostly illiquid noise. Treating the chain as a single “dataset” conflates liquid and illiquid instruments.

Engineering decision

Represent options markets through derived surfaces—implied volatility surfaces, ATM vol, skew, term structure—rather than raw chains. This representation is a dataset design decision: the surface is the dataset, not a summary of it. Store volume and open interest for liquid contracts; ignore or aggregate the illiquid tail.

Datasets

S&P 500 daily options prices and analytics covering 2017-2021 provided by AlgoSeek. The notebook `07_sp500_options_eda` explores this dataset; the notebook `08_options_greeks_computation` derives Black-Scholes pricing and Greeks from first principles—Delta, Gamma, Vega, Theta, and Rho—and validates Delta, Gamma, Vega, and Theta against vendor-computed values; `09_options_continuous` demonstrates roll contamination in constant-maturity option series and constructs backtestable continuous returns.

Digital assets

Total crypto market capitalization was about ~\$3T on 18 January 2026 (CoinMarketCap). Digital assets combine centralized exchanges (CEX) with familiar order books and decentralized exchanges (DEX) with automated market makers. The on-chain state is publicly observable, while exchange microstructure is venue-specific.

What you observe

CEX data resembles equities: spreads, depth, order flow—the microstructure concepts we will discuss in *Chapter 3* apply directly, and order book reconstruction works as expected. DEX data differs fundamentally: AMMs replace order books with reserve-based pricing (for example, constant-product AMMs; concentrated-liquidity designs generalize this), where price emerges from reserve ratios rather than bid-ask matching. There is no order book to reconstruct; “depth” becomes slippage at a given trade size, computable directly from pool reserves.

Failure modes

24/7 trading complicates sessionization. Reported volume on some venues is unreliable. **Maximal extractable value (MEV)** refers to the profit that

validators or other actors can capture by reordering, inserting, or censoring transactions within a block; together with impermanent loss and on-chain latency, it materially affects DEX execution. Venue fragmentation makes cross-exchange comparison difficult.

Engineering decision

Screen venues for integrity before including them. Handle 24/7 timestamps explicitly. For DEXs, derive slippage and liquidity from reserves rather than treating AMM quotes as order-book depth.

Datasets

We use Perpetual Futures prices at hourly frequency and Premium Index at 8-hour intervals for 19 symbols covering 2020-25, sourced from Binance. The notebooks `10_crypto_perps_eda` and `11_crypto_premium_analysis` explore this data; the funding-rate premium analysis is relevant to funding arbitrage strategies.

Foreign exchange

Global daily average FX turnover was about ~\$7.5T/day (April 2022, Fed NYC). FX is predominantly OTC with no single consolidated tape. Even “daily close” (for example, 5 pm New York) is a convention, not an exchange-defined event. Different platforms display different liquidity and spreads.

What you observe

Venue-specific quotes without centrally reported volume. In FX, “the price” is a convention: you choose a venue and an aggregation rule (mid, best-of, VW-mid).

Failure modes

Assuming a single authoritative price when none exists. Stale quotes, crossed markets, and outlier filtering all require explicit rules.

Concrete example

A “4 pm London close” and a “5 pm New York close” for EUR/USD often differ by several pips on volatile days. If your strategy mixes close conventions across currencies, you may be comparing non-comparable timestamps.

Engineering decision

Specify which price you can realistically trade: best-of aggregation, volume-weighted mid, or a specific venue. Document the close convention in your dataset metadata.

Datasets

We use 20 currency pairs at 4-hour intervals from 2011–2025 via OANDA and explore this data in the notebook `12_fx_pairs_eda`.

Fixed income

The outstanding amount in the global fixed-income market was approximately ~\$145T (2024, SIFMA). Fixed income is predominantly OTC, with a vast instrument universe—thousands of corporate bonds and tens of thousands of municipal bonds. Many instruments trade infrequently; quoted markets may be indicative rather than firm.

What you observe

Sparse transaction prints, indicative quotes, and model-derived valuations. When the last trade is stale, research pipelines operate on yields, curves, and inferred prices rather than clean price series.

Failure modes

Liquidity estimation is an inference problem, not a measurement. Matrix pricing—mapping illiquid bonds to similar liquid instruments—introduces model assumptions. Still, any approach requires distinguishing between observed prints and indicative quotes, and between model outputs and model inputs.

Engineering decision

Decide how to handle illiquid instruments: exclude them, impute prices via matrix pricing, or model them separately. Document the methodology because “price” means different things for liquid treasuries versus thin municipals.

Swaps and OTC derivatives (rates, credit)

OTC derivatives notional amounts are enormous (~\$846T in mid-2025, BIS), but **notional is not exposure** ; it mainly reflects contract scaling. **Interest rate derivatives** accounted for ~79% of OTC derivatives notional (~\$670T). Swaps markets sit “behind” much of modern fixed income and macro trading. Interest rate swaps, OIS, cross-currency swaps, and CDS are primarily OTC (with varying degrees of central clearing), and their data is shaped by reporting regimes and market conventions rather than a single consolidated tape.

What you observe

Post-trade reporting (where available), indicative dealer runs, cleared pricing where accessible, and curve-building inputs rather than a single authoritative “price series.” Even when transactions are reported,

timestamps, block-size masking, and venue fragmentation can affect what is observable at research granularity.

Failure modes

Treating OTC quotes as firm executable prices; mixing venues/reporting feeds with incompatible conventions; and building curves without consistent day-count, calendars, collateral/discounting conventions, or roll rules. Many downstream “returns” are artifacts of choices in curve construction.

Engineering decision

Decide whether your dataset is (1) transaction-based, (2) quote-based, or (3) curve/surface-based—and treat that choice as fundamental. For rates, store curve snapshots with explicit conventions; for credit, separate index products from single-name CDS; and keep a clear line between observed inputs and model-derived marks.

Commodities

Exchange-traded commodity derivatives volume was about ~8B contracts in 2024 (options and futures, FIA). Commodity datasets sit at the intersection of financial markets and physical constraints. “Spot” is often not a single tradable object (and can be reported, assessed, or location-specific), while futures curves are standardized, liquid, and exchange-observable for major commodities.

What you observe

Futures prices and term structures are typically the most reliable market data. Physical-market indicators (inventories, shipping, weather, refinery runs) often drive fundamentals but are subject to reporting lags and coverage bias.

Failure modes

Treating assessed spot series as if they were exchange-traded prices; ignoring contract specs (delivery location/grade) that determine basis; and mixing curve snapshots from different timestamps when computing roll yield or calendar spreads.

Engineering decision

Decide whether your “commodity price” is (1) a specific futures contract, (2) a continuous futures series (with explicit roll/adjustment), or (3) a curve representation (front-to-back term structure). Store the contract spec metadata in the dataset, not as documentation.

Microstructure tooling applicability

Chapter 3 discusses order-book reconstruction and bar-sampling methods. They apply directly to exchange-traded instruments (equities, futures, options, CEX crypto). They do not apply to OTC markets (much of fixed income many swaps) where quotes are indicative and fragmented across dealers, nor to DEX AMMs where no order book exists. For these markets, alternative approaches—such as quote aggregation, curve construction, matrix pricing, and reserve-based liquidity computation—are required.

Next, we discuss how to evaluate data sources and providers.

2.3 A due diligence framework for data sourcing

Data sourcing discipline catches defects before they become “alpha” (risk-adjusted trading gains) in a backtest and turn into production losses. Errors are systematic, not random: microstructure artifacts, reporting lags, revisions, corporate actions, and vendor backfills create error patterns that can inflate or deflate apparent performance. A single undetected defect can

invalidate months of research. Luo et al. (2014) catalog systematic errors that inflate backtest returns; López de Prado (2018) emphasizes that many ML strategy failures trace to data issues rather than modeling.

General data quality dimensions

Data quality assessment typically spans five dimensions:

- **Timeliness** : Lag/latency from release/event to availability
- **Completeness** : Coverage gaps across instruments, dates, and fields
- **Accuracy** : Values match ground truth
- **Consistency** : Stable schemas and definitions over time
- **Validity** : Constraints satisfied (timestamps ordered; values within plausible bounds; field-level invariants hold)



Implementation : `13_data_quality_framework` implements an automated validation pipeline using the `ml4t-data` library to check these dimensions on OHLCV data. The 100-ETF panel from `03_etfs_eda` illustrates the scale to expect: 473 OHLC invariant violations (~0.1% of rows) trace to independent split and dividend adjustment of the four price fields, an artifact small enough to ignore for most uses but worth flagging in a vendor-comparison report. Prioritize checks that create lookahead bias (PIT, backfills, revisions) and universe leakage (survivorship).



This section covers data quality issues applicable to *all* financial data. For evaluation criteria specific to alternative data, see *Section 4.4* .

Finance-specific failure modes

Four failure modes commonly result in manufactured alpha:

Point-in-time correctness

Point-in-time (PIT) correctness means that every value used in historical research reflects what was *knowable at the time of the decision* , under the definitions and reporting conventions in force . PIT violations occur when datasets are revised after the fact, and you unknowingly train or backtest on the revised version.

Common culprits include revisions and restatements (fundamentals, macro), corporate action backfills, delayed or amended filings, and vendor database updates that overwrite history.

Example failure. A company reports Q4 2019 revenue as \$91.8B in early 2020. By 2024, after a restatement, a vendor's database shows \$92.3B for the same quarter. If you query "Q4 2019 revenue" today and use it in a 2020 backtest, you have introduced lookahead bias: the model is trained on information that did not exist at the time of the decision.

Fixing PIT requires **bitemporal storage** and **as-of queries** . A PIT-correct system stores values along with the period they refer to and when each value became publicly observable, so you can reconstruct the historical information set exactly as it existed at time t . We use the following terms to describe PIT correctness:

- **Event time** : When the economic event occurred (for example, end of Q4 2019)
- **Knowledge time** : When the information became publicly available (for example, filing acceptance time)
- **As-of time** : The historical moment being reconstructed in your analysis (the decision time)

- **Reference period** : What the value refers to (for example, “Q4 2019 revenue”)

Figure 2.1 illustrates PIT correctness for fundamental data.

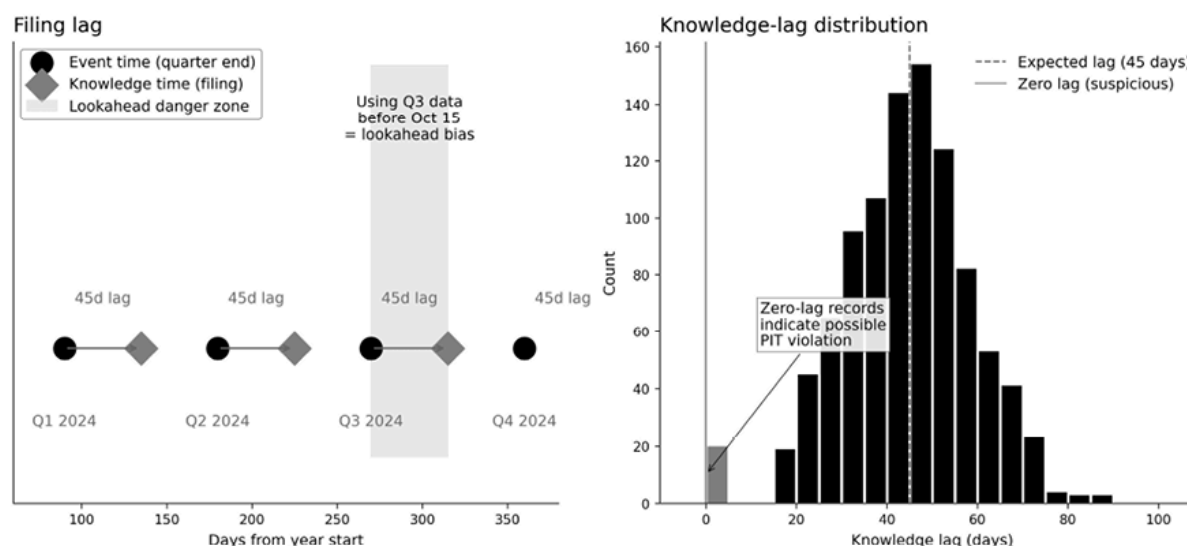


Figure 2.1: Point-in-time correctness for fundamental data

The left panel shows the “lookahead danger zone” between event time and knowledge time; the right panel highlights zero-lag records as a red flag. The companion notebook `14_point_in_time_validation` demonstrates bitemporal query patterns and produces *Figure 2.1*.

For any fundamental or alternative metric, store the *valid time interval* the value describes and the *availability* timestamp when that specific value became observable. Each record should also include a stable `source_id` (for example, SEC accession number or agency release identifier), an `entity_id` at the intended layer (issuer versus security), and a `version` identifier. A downstream feature at decision time t may use a record only if `available_at <= t`.

All joins across data sources must be constrained by `available_at <= decision_time`. For identifier mappings (for example, ticker→CIK, CUSIP→ISIN), also require that `effective_date <= decision_time < end_date`, so the mapping was valid at the decision point. Violating either

constraint introduces lookahead bias—often silently—through “correct” joins that were impossible at the time.

Before using any dataset with fundamental-like inputs, verify PIT correctness across three dimensions:

Dimension	Question	Failure mode
Values	Does each data point have both a valid time and an availability (knowledge) time?	Using restated values for historical decisions
Universe	Is index/universe membership tracked historically?	Survivorship bias from current-day selection
Identifiers	Are entity mappings time-valid (ticker changes, mergers)?	Joining data across corporate events incorrectly

Table 2.2: PIT checklist

A dataset that passes value-level PIT but fails universe-level PIT can still produce biased backtests, especially for cross-sectional strategies and historical index universes.

Survivorship bias

Survivorship bias occurs when the historical universe is filtered using information from the future (for example, “still exists today”). Its effect can be positive or negative depending on what you exclude: excluding *bankruptcies* inflates returns (you miss losses), while excluding merger and acquisition (M&A) targets can deflate returns (because you may miss acquisition premiums). The missing names are not random: they include bankruptcies, delistings, acquisitions, and value traps—precisely the types

of situations many strategies are exposed to. Therefore, the entities that “survive” are biased towards cases that did not experience such situations.

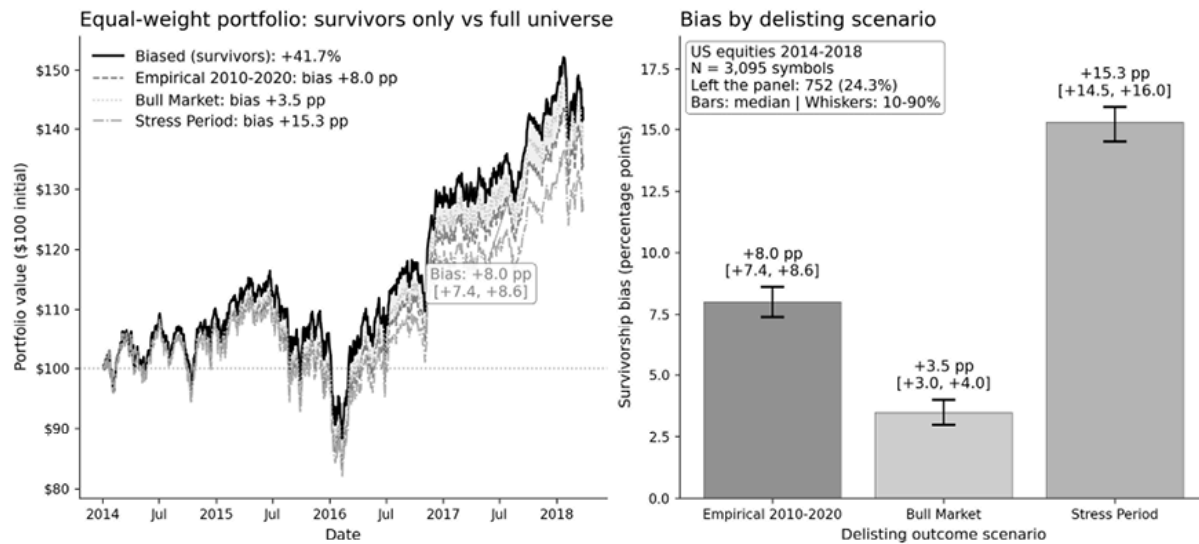


Figure 2.2: Survivors overstate the full-universe return (2014–2018 US equities panel)

The *US equities panel* contains 3,199 stocks, 777 of which (24.3%) were delisted before the dataset’s end date—all after 2014, the only window in which the legacy Quandl WIKI feed captured delistings. Quantifying survivorship bias requires simulating terminal returns for delisted stocks by exit reason: distressed delistings, M&A, or voluntary exits, each with its own probability and terminal return. Eckbo and Lithell (2025) tabulate CRSP 2010–2020 outcome shares; the empirical scenario applies those tabulations, assuming 26% compliance failures at -60% , 65% acquisitions at $+25\%$, and 9% other exits at 0% . Terminal returns are conservatively applied on the delisting date itself. In practice, acquisition premiums often materialize gradually as deal rumors circulate and the price converges toward the offer, so by delisting day, much of the $+25\%$ may already be priced in; bankruptcy losses, by contrast, tend to be more sudden. Therefore, this calibration overstates the magnitude of bias on M&A-heavy panels.

Figure 2.2 shows the results across Monte Carlo simulations with 1,000 draws per scenario. Over 2014–2018, survivorship bias inflated perceived

returns by 3.5–15.3 percentage points across the three scenarios (+8.0 in the empirical scenario): dropping the delisted names removes the weaker half of the distribution, since the leavers returned a median +13.5% against the survivors' +30.8%. The sign of the bias depends on the dominant delisting cause: bankruptcy-heavy periods inflate survivor returns, while M&A-heavy periods can deflate them. Shumway (1997) and Beaver, McNichols, and Price (2007) document the underlying patterns of delisting returns.

The sign of the bias depends on the dominant delisting cause: bankruptcy-heavy periods inflate survivor returns, while M&A-heavy periods, such as the 2010–2017 sample, deflate them. Shumway (1997) and Beaver, McNichols, and Price (2007) document the underlying patterns of delisting returns.

Another source of bias is using today's S&P 500 constituents as the “universe” for a historical backtest. Even if those companies survive, the selection criterion is outcome-dependent: current members are firms that have grown large and successful enough to be included in the index. Backtesting on them retroactively bakes that success filter into the experiment.

The remedy : point-in-time universe construction. The 2010 backtest uses 2010 membership, with correct handling of delisting returns and corporate actions. CRSP is widely used as a benchmark for survivorship-aware U.S. equity data because it tracks delistings over long horizons, but many institutional vendors provide comparable histories.



Implementation : `15_survivorship_bias_detection`

quantifies delisting rates and shows how the composition of delisted names (M&A versus distress) affects the magnitude of the bias.

Corporate actions

Corporate actions—splits, dividends, spinoffs, mergers—create discontinuities in price series that require adjustment. The failure mode is the mixing of adjustment methodologies or the use of end-of-sample factors (covered later in this chapter) in historical research.

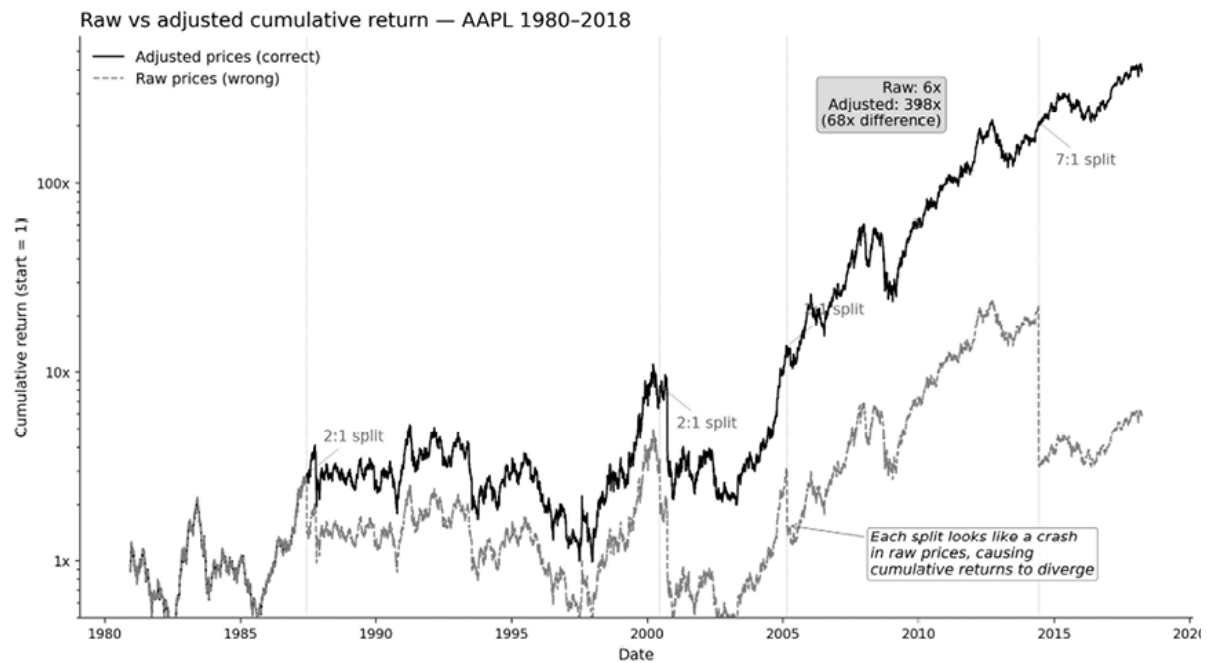


Figure 2.3: A graph comparing raw and adjusted prices impacted by corporate actions

A 7:1 stock split makes raw prices appear to have crashed by ~86%. Computing returns from unadjusted prices understates cumulative performance by a catastrophic margin. *Figure 2.3* illustrates this for AAPL: over its multi-decade price history (four splits plus dividends within the Wiki/Quandl panel through 2018), raw prices show a 5.9× cumulative return, while adjusted prices show a 398× return, a roughly 68× understatement.

Adjustment methodologies differ. Some vendors provide point-in-time adjustments (as-known-then factors); others offer back-adjusted series (end-of-sample factors applied retroactively). Mixing them introduces lookahead. Vendors also differ in how they handle dividends (total return versus price return), spin-offs, and rights issues.

The remedy : document and validate your adjustment methodology.

`02_corporate_actions` validates adjustment factors against Apple’s known historical splits. *Chapter 3* covers the mechanics in detail.

Identifier integrity

Identifier joins can align the wrong instrument. Ticker symbols get reused (when an old company delists, a new company takes the symbol). CUSIP changes through mergers. Vendor IDs have edge cases.

This creates fake alpha. Suppose your signal table has ticker “XYZ” from 2015, and your price table has “XYZ” from 2020, but these are different companies. The join succeeds; the signal now predicts the returns of a different company. If the second company happened to perform well, you have manufactured correlation from a data error.

The remedy : use permanent IDs with effective date ranges (vendor permanent IDs; FIGI/ISIN/CUSIP via crosswalks).

`16_provider_comparison` demonstrates multi-source stitching with careful handling of identifiers; `17_complete_pipeline` shows Hive partitioning patterns that preserve identifier integrity across incremental writes.

The vendor ecosystem

Vendors span a wide range of cost and capability; key attributes that vary with cost include depth (intraday versus daily), history length, handling of corporate actions, PIT support, latency, redistribution rights, and methodology auditability.

Tier	Typical Profile	PIT Support	Survivorship-aware	Best For
Free/public	Easy access, limited	Rare (macro	Rare	Learning, prototypes,

Tier	Typical Profile	PIT Support	Survivorship-aware	Best For
	guarantees	vintages via ALFRED)		sanity checks
Prosumer (API-first)	Good coverage for liquid markets	Partial	Sometimes partial	Individual research
Institutional	Deeper history, documented methodology	Often available	Often available	Production, regulated environments

Table 2.3: Data provider tiers

When deciding what to buy and what to build, separate differentiating work (strategy-specific transforms) from commoditized work (clean histories, identifier crosswalks):

- *Build* the strategy-specific parts: ingestion pipelines, storage and versioning, validation checks, canonical identifiers, and the transformations that encode your assumptions (such as sessionization, adjustment policies, roll rules, PIT-aware features).
- *Buy* when the problem is non-differentiating or prohibitively expensive to reproduce: long-horizon cleaned histories, corporate action adjustments at scale, survivorship-aware universes, and standard identifier crosswalks (for example, FIGI mappings, Bloomberg identifiers).

Entity and identifier mapping is often a months-long effort: tracking an instrument across ticker changes, mergers, and cross-vendor conventions.

Whether you buy vendor crosswalks or build your own, treat identifier integrity as core risk control.

Vendor due diligence checklist

Vendor selection is risk management. In practice, diligence comes down to three questions: Can you trust the numbers? Are you allowed to use them as intended? Will the vendor behave like a reliable dependency?

Data quality

The first task is to determine whether the dataset supports historically correct analysis rather than just convenient access to current values. That means checking not only what fields are available, but also how the vendor handles time, revisions, identifiers, and corporate events.

Key questions include:

- **PIT support** : Can the vendor provide snapshots or as-of queries?
How are revisions handled?
- **Survivorship awareness** : Are delistings included with the correct final prices?
- **Adjustment methodology** : How are splits, dividends, and other corporate actions handled? Is the methodology documented and stable?
- **Coverage and definitions** : What is the true start date, instrument coverage, update cadence, and field definition?
- **Identifier integrity** : Which identifier scheme is used, and how are mappings handled through ticker reuse, mergers, and other entity changes?

These questions establish whether the dataset is analytically credible; the next step is to confirm that it is also permissible and defensible to use.

Legal and compliance

Even high-quality data can create problems if the rights and provenance are unclear. This review should establish that the data is lawfully sourced, that usage is permitted for the intended workflow, and that the vendor can document its methods if needed. Focus on four issues:

- **MNPI screening** : Does the vendor have documented provenance controls?
- **Privacy and consent** : For consumer data, is the collection compliant with GDPR, CCPA, and similar rules?
- **Usage rights** : What is permitted for backtests, live trading, redistribution, and model training?
- **Auditability** : Can the vendor provide documentation of its methodology for regulators, clients, or internal control functions?

Once the legal footing is clear, the practical question is whether the vendor can operate as a stable part of your research and production stack.

Technical and commercial

A vendor also needs to function as a dependable production dependency. Strong content is not enough if the delivery model is fragile, opaque, or difficult to integrate into research and live systems. In particular, assess the following:

- **Reliability** : Is there a status page, uptime history, or SLA?
- **Change management** : Are schema changes and methodology revisions versioned and communicated clearly?
- **Access patterns** : Are rate limits, bulk exports, and backfill mechanisms compatible with your pipelines?
- **Portability** : Can you export the data in a usable format, and are the exit terms reasonable?

If the vendor looks sound in principle, the next step is a short hands-on validation to confirm that the data behaves as expected in practice.

Minimum viable data validation checklist

Once a vendor passes initial due diligence, a brief validation pass should confirm that the dataset is usable for research. At a minimum, verify the following:

- **PIT** : No zero-lag records for delayed data
- **Survivorship** : Historical universe not filtered by future information
- **Corporate actions** : Adjustment methodology documented
- **Identifiers** : Join on permanent IDs + effective dates
- **Timestamps** : Timezone explicit, session assignment clear
- **Outliers** : Stale quote and anomaly detection in place
- **Versioning** : Reproducible as-of queries possible

Vendor diligence, however, is only part of the control framework; internal processes are also required to make data issues visible, traceable, and reversible.

Data governance

Good vendors do not replace internal governance, which makes problems discoverable and reversible across five areas:

- **Lineage** : Track where each dataset came from and how it was transformed.
- **Versioning** : Version data rather than overwriting. When you fix an error, preserve the ability to reproduce prior results.
- **Logging** : Log cleaning decisions so results are reproducible.
- **Validation automation** : Use tools like Great Expectations or Pandera; fail loudly on violations rather than silently dropping records.
`13_data_quality_framework` provides the `AnomalyManager` toolkit, which produces JSON audit trails for each validation run.
- **Parallel runs** : When switching vendors or methodologies, run sources in parallel long enough to quantify differences. Preserve rollback

options.

Treat sourcing as risk management: PIT and survivorship checks prevent the most expensive errors. The next section turns to efficient data storage.

2.4 Storing data

Storage is the foundation of research velocity and production reliability. There is no best solution independent of access patterns and operational constraints. The right choice depends on data volume, access patterns (scans, point lookups, range queries, joins), concurrency requirements, and operational maturity.

What we benchmark and why

These benchmarks address two distinct questions:

- **File formats** : If your data lives in files (the most common research pattern), which format gives the best trade-off between size, read speed, and write speed?
- **Database engines** : When you need SQL queries, concurrent access, or time-series operations like ASOF joins (see *ASOF join performance* in this chapter), which engines perform well for financial data workflows?

We separate these because most research workflows start with files (Parquet) and add databases only when the use case demands it. The benchmarks help you decide if and when to make that transition.

Benchmark context and caveats

The tables and figures in this section summarize benchmarks from the companion code at the L scale (approximately 1 million OHLCV rows, 64 MB of in-memory storage in Polars). The repository includes scripts to run

at larger scales (XL: 10 million rows, XXL: 100 million rows) on your hardware.

Keep the following in mind when interpreting benchmark results:

- **Operation definitions matter.** “Read time” can mean opening a file handle, scanning a subset of columns, or fully scanning all data. Arrow IPC supports memory mapping, so opening the file does not necessarily load the entire file into RAM.
- **Warm versus cold cache.** A warm-up run often primes the OS page cache, making subsequent reads far faster than true cold reads.
- **Relative rankings are more portable than absolute times.** Treat the numbers as illustrative for your hardware and workload; use the companion scripts to generate benchmarks on your own systems.

File-based storage

We benchmarked file formats on a 1-million-row OHLCV dataset (100 symbols $\times$ 10,000 bars). *Figure 2.4* traces read-time scaling across formats as the row count grows; columnar formats hold their advantage as size increases, while CSV read time degrades linearly.

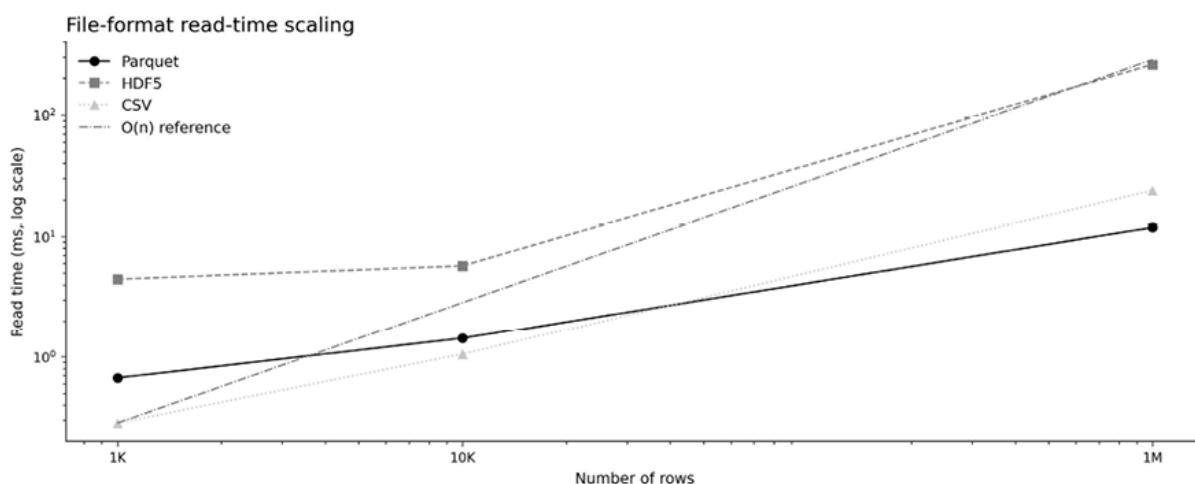


Figure 2.4: File format read-time scaling with dataset size

Table 2.4 summarizes file size, write time, and scan time at L scale for the three formats suitable for persistent storage. Rankings are often more stable than absolute timings, but ingestion paths and caching can materially affect results.

Format	File Size	Write Time	Full Scan	Notes
CSV	136 MB	0.13s	0.05s	Baseline: engine-dependent overhead
Parquet	40 MB	0.07s	0.02s	Columnar, compressed, selective reads
HDF5	71 MB	0.13s	0.40s	Performance depends on chunking

Table 2.4: File storage benchmark results

When choosing a format:

- **Parquet** : Recommended default for research and production: strong compression (3.4× versus CSV), efficient columnar reads, universal tool support across Python, R, Spark, and cloud platforms.
- **HDF5** : Still common in scientific and legacy finance stacks. It can work well for specific append-and-chunked access patterns, but read performance lags behind columnar formats.
- **CSV** : Use for exports and interoperability, not as a primary analytical store.

Note on Arrow IPC (Feather)

Arrow IPC is a fast serialization format optimized for interchange. It excels at moving data between notebooks and processes (including memory-mapped reads), but lacks key “data lake” features such as





predicate pushdown, partition-aware layouts, and multi-file governance. Use it for short-lived interchange; use Parquet for persistent, queryable storage.

Lakehouse formats (such as **Delta Lake** , **Iceberg** , and **Hudi**) store data in Parquet and include a transaction log that provides ACID semantics, schema evolution, and multi-writer governance. Start with plain, partitioned Parquet; adopt a lakehouse format when you need update-heavy workflows, schema evolution with strong guarantees, or concurrent writes across multiple teams.



Implementation : `20_storage_benchmark_file` reproduces these benchmarks and extends them to larger scales.

Embedded analytics databases

Embedded databases provide SQL analytics without running a server:

- **DuckDB** : An in-process analytical database optimized for OLAP workloads. It can query Parquet directly and supports ASOF joins in SQL, making it a strong default for research workflows that want SQL semantics over a Parquet data lake.
- **SQLite** : A reliable embedded relational database for metadata, configuration, and small relational tables. Not competitive with DuckDB for analytical scans or time-series workloads.
- **ArcticDB** : A DataFrame-oriented store with versioning and time-travel. It is licensed under **BSL 1.1** (with time-based conversion to Apache 2.0 per release); verify licensing and operational constraints before adopting.

A useful default architecture:

1. Store raw and curated data as partitioned Parquet.
2. Use DuckDB for SQL analytics and quick joins over those files.
3. Use a DataFrame engine, such as Polars, for feature engineering. See [22_pandas_polars_benchmark](#) for an empirical pandas-versus-Polars comparison across read, groupby, join, and lazy-evaluation operations.

Server time-series databases

Server databases are valuable when you need concurrent access, centralized governance, strict durability, and production-service levels.

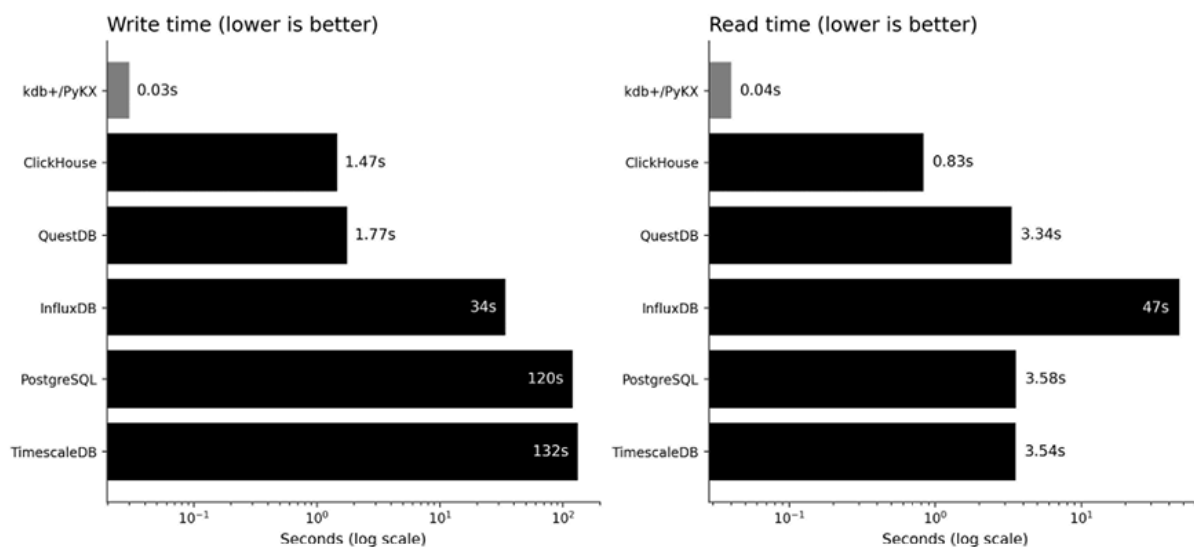


Figure 2.5: Server database comparison (1 million rows)

Figure 2.5 compares write and read performance across six server databases at L scale (kdb+/PyKX, ClickHouse, QuestDB, PostgreSQL, TimescaleDB, and InfluxDB). These results are directional; write time is sensitive to ingestion method, schema design, and durability settings. Rerun benchmarks using your intended ingestion path before making decisions.

Server time-series databases include:

- **kdb+/KDB X** : A highly optimized time-series and analytical engine widely used in market data contexts. KX offers free editions with

material limits (often non-commercial); verify current license terms before committing.

- **ClickHouse** : A general-purpose OLAP database optimized for high-throughput columnar analytics, with strong SQL support and a large ecosystem.
- **QuestDB** : A time-series database optimized for ingestion and time-series SQL, including ASOF-style queries. Supports tiered storage patterns with Parquet for colder data.
- **PostgreSQL/TimescaleDB** : PostgreSQL is the default general-purpose relational database; TimescaleDB extends it with hypertables (automatic time partitioning) and continuous aggregates for time-bucketed rollups.
- **InfluxDB** : Designed for metrics and observability workloads. Flux supports equi-joins, but as-of style joins are not a native primitive; treat it as a monitoring database rather than a market microstructure engine.



Implementation : `21_storage_benchmark_database` provides reproducible server benchmarks. For production-grade orchestration, `18_data_management` composes the DataManager, Universe, and HiveStorage primitives into a working pipeline, while `19_incremental_updates` walks the daily-update lifecycle with weekend-aware gap detection and a health dashboard.

ASOF join performance

ASOF joins align events that are not perfectly time-synchronized—matching each trade to the most recent prevailing quote, for example. They are typically the dominant cost in panel-data preparation that mixes tick

streams with slower-moving reference series. Both pandas and Polars require sorted inputs.

In-memory engines (Polars `join_asof` and pandas `merge_asof`) consistently outperform embedded-SQL engines (DuckDB `ASOF JOIN`) on tick-scale workloads at the sizes we use in this book, with Polars typically the fastest of the three. The absolute gap depends on hardware, sort state, and whether the benchmark counts file I/O, so we report ordering rather than headline times.

A few measurement caveats matter when comparing implementations:

- In-memory engines measure join execution when data is already loaded; SQL engines may include file scans and conversion overhead.
- For fair comparisons, decide whether sorting is part of the benchmark. If the pipeline maintains sorted data, exclude sorting consistently across engines.
- Tick-versus-quote ASOF performance is sensitive to the ratio of left- to right-side rows; benchmark at the ratio that matches the intended workload.

Strategic decision framework

The following matrix summarizes standard defaults by objective.

Objective	Strong default	Also consider
Research velocity	Parquet + Polars	Parquet + DuckDB
SQL analytics on files	DuckDB + Parquet	Polars lazy scans
Production reliability	PostgreSQL	TimescaleDB

Objective	Strong default	Also consider
Extreme time-series throughput	kdb+/KDB X	ClickHouse
High-throughput ingestion	QuestDB	ClickHouse
ASOF joins in memory	Polars	pandas
ASOF joins with SQL	DuckDB	QuestDB
Lowest operational burden	Parquet + DuckDB	Managed Postgres

Table 2.5: Recommended default and strong alternative by storage objective

Start with partitioned Parquet plus Polars or DuckDB. Add server databases when concurrency and governance requirements justify the operational overhead.

2.5 Summary

This chapter mapped the financial data universe and the decisions that make research reproducible. Use the taxonomy—market, fundamental, and alternative data—to match datasets to strategy horizons and to surface the assumptions embedded in timestamps, adjustments, identifiers, and revision rules.

Most alpha failures in production trace to a small set of data defects: point-in-time violations, survivorship bias, corporate action mistakes, and identifier join errors. Treat these as default risks rather than edge cases, and design validation checks to catch them early.

Market structure constrains what you can observe across asset classes: exchange-traded markets provide richer event data but require careful sessionization and adjustments; OTC markets often rely on indicative quotes and inferred prices; derivatives require explicit contract identity and

roll logic; digital assets add 24/7 trading and venue-integrity concerns. Vendor selection is therefore a risk management process that involves evaluating quality, legal rights, and operational reliability.

Finally, storage choices determine iteration speed. A sensible default is partitioned Parquet queried DuckDB or Polars for most research; add specialized databases only when concurrency, governance, or latency requirements justify the operational cost.

Chapter 3 develops the details of market data—both as a key input to labels and features for modeling and as the basis for simulating historic strategy performance—and examines the institutional environment of exchanges that shapes how market data should be interpreted.

3

Market Microstructure

Market prices result from an engine that matches supply and demand under explicit rules. These matching engine rules determine what the data means (what counts as a trade, a quote, a “close”), and what a strategy can realistically earn once spreads, depth, and execution constraints come into play. We lay the foundation for working with modern market data, from parsing raw exchange messages to reconstructing order books and sampling tick-level data into bars whose statistical properties match how information arrives.

Consider a simple momentum rule: buy when the price rises above its 20-day moving average. A naïve backtest might use daily closes and assume immediate fills at the bar’s end price. In live markets, execution is shaped by the order book:

- A market buy typically executes at the **ask** and a market sell at the **bid** , so traders pay the spread rather than transacting at the last price or a bar reference such as the “close.”
- Execution quality depends on *available depth and queue position* . Larger or poorly timed orders can walk the book, fill partially, and move prices.
- In volatile periods, spreads widen and displayed depth becomes less reliable as cancellations accelerate, making “filled at the close” a fragile assumption even at daily horizons.

When backtests ignore spreads, depth, and queueing, the error is not just a “cost” but a mis-modeled price-formation process. Our goal is to make the price-formation process explicit so readers can (i) interpret market data correctly, (ii) choose sampling methods that respect how information arrives, and (iii) model execution and transaction costs in a way that survives contact with real markets. After completing this chapter, you will be able to:

- Explain how liquidity, order types, and price discovery shape market data (*Section 3.1*).
- Describe the main market data feed types from market-by-order to aggregated bars (*Section 3.2*).
- Parse exchange messages and reconstruct a limit order book (LOB) from an event stream (*Section 3.3*).
- Build time- and information-driven bars to reduce noise and improve statistical properties (*Section 3.4*).
- Detect price jumps in intraday returns and separate continuous from jump variance (*Section 3.5*).
- Apply data-quality and sessionization controls for intraday data (*Section 3.6*).

Section 3.2 introduces the five tick-level sources used in this chapter, positions each within the data hierarchy, and explains what analyses each enables.

3.1 How microstructure impacts price formation

Market data is the observable output of a matching engine: rules plus order flow produce quotes and trades. As O’Hara (1995) emphasizes, market structure choices have first-order effects on trading costs and price

discovery. Without these mechanics, the data is easy to misread, strategies are mis-specified, and backtests do not survive contact with real execution.

At the core, trading coordinates latent supply and demand among anonymous counterparties. The **bid** is the highest price a buyer is willing to pay; the **ask** (or offer) is the lowest price a seller is willing to accept. Their difference, the **bid–ask spread**, is both a fundamental transaction cost and a primary measure of liquidity. For trading decisions, liquidity has three separable dimensions:

- **Spread** : The cost of immediate execution. Tighter spreads generally indicate more competitive markets.
- **Depth** : The quantity available at and away from the best prices. Depth determines how much can be traded before prices move.
- **Resiliency** : How quickly liquidity replenishes after trades consume it.

Together, these dimensions determine the feasibility of strategy: small signals cannot overcome wide spreads, and large positions require depth and resilience.

The frictions faced by liquidity providers

Participants either supply liquidity with resting orders or demand it with marketable orders:

- **Liquidity providers** (including market makers) post limit orders that supply quotes. They earn the spread *in expectation*, but only if they control selection risk and manage inventory.
- **Liquidity takers** demand immediacy by submitting marketable orders (market orders or aggressively priced limits). They incur

spread costs and execution costs that depend on depth, volatility, and routing.

Two frictions drive quote formation and realized spreads:

1. **Adverse selection:** Liquidity providers quote under information asymmetry: if counterparties trade on information not yet reflected in prices, passive fills become losses. Providers respond by widening spreads, shading quotes, or reducing displayed depth to compensate for expected losses.

Kyle (1985) formalizes this: price impact is proportional to net order flow, with **Kyle's lambda** measuring the strength of price responses to trade imbalances—a key input for the execution cost models we develop in *Chapter 18*. Glosten and Milgrom (1985) show that bid-ask spreads arise purely from adverse selection, even in the absence of transaction costs.

2. **Inventory risk:** Providing liquidity requires accumulating positions. A provider that becomes too long into a falling market, or too short into a rising one, can lose more on inventory than it earns from spread capture. Therefore, inventory constraints influence both quote placement and supply depth.

The spread compensates for **order processing** (fees, technology), **inventory holding**, and **adverse selection**. Madhavan (2002) shows that price impact has both temporary and permanent components, and is nonlinear in order size. In fast markets, adverse selection dominates; in calmer conditions, fee schedules and venue incentives matter more.

How order types express intent in the limit order book

Orders encode trading intent. Their details determine how a trade executes, what is paid in spreads and slippage, and what can be inferred from quotes and prints.

A **resting order** adds displayed depth; a **marketable order** removes it by crossing the spread.

- **Marketable orders** (market orders and marketable limit orders) execute immediately against the best available resting liquidity. Trade prints typically appear at the current best bid/ask, alongside a corresponding reduction in displayed size at that level (often followed by a price change if the level is exhausted).
- **Limit orders** specify a worst acceptable price. If priced to rest, they add liquidity at their price level (increasing displayed depth, and sometimes improving the best bid/ask). If priced to cross, they behave like marketable orders but with a price bound.
- **Stop and stop-limit orders** activate when a reference price crosses a threshold. Stops are often broker-managed and therefore absent from exchange-visible book data until triggered. Once triggered, a **stop-market** becomes a marketable order (often visible as a burst of executions and rapid top-of-book depletion near the trigger). In contrast, a **stop-limit** becomes a limit order that may not fill if the market moves past it quickly.

Modern venues also offer order types that reduce information leakage or automate repricing:

- **Hidden and iceberg orders** trade without fully displaying size, disguising intent. A common signature is repeated same-price prints

with the displayed quote size refreshing rather than depleting.

- **Pegged orders** automatically repriced to a reference (often the midpoint or best quote). A common signature is prints inside the spread (for example, at the midpoint) without a corresponding displayed quote at that exact price.
- **Post-only orders** are designed to *rest* (maker behavior): they add liquidity and are rejected or repriced rather than executed immediately if they would cross the spread.

These signatures are useful diagnostics but only suggestive: hidden liquidity and routing can generate similar prints. Repeated fills with quote refresh suggest non-displayed supply; inside-the-spread prints suggest reference-priced liquidity that may never appear in the visible book.

Market design and intraday regimes

Markets match orders through different mechanisms, and microstructure varies predictably over the trading day; both dimensions shape what the data means and when signals are actionable.

Electronic limit order books dominate modern price formation, with orders executing under price–time priority across multiple venues. Quote-driven markets remain important in OTC, FX, and fixed income, where dealer inventory and relationship pricing shape execution. Periodic auctions—especially opening and closing—concentrate liquidity and set benchmark reference prices.

Intraday seasonality is a first-order confounder: the same “signal” can mean different things at 9:35 AM and 2:00 PM. Three regimes dominate:

- **Opening** (first 30–60 minutes): High volatility, wide spreads, heavy trading as overnight information is incorporated. Madhavan, Richardson, and Roomans (1997) show that information asymmetry

—measured via the MRR model—starts high at the open and declines as prices are discovered, while inventory costs rise toward the close. Schwartz, Ross, and Ozenbas (2022) confirm that variance ratios remain elevated in the opening half-hour, indicating microstructure noise rather than fundamental information.

- **Midday** : Lower activity, thinner liquidity. Spreads may tighten, but depth is shallow—the same order size has greater price impact than during busier periods.
- **Closing** (final hour): Elevated volume as institutions complete programs and index funds rebalance. The closing auction concentrates substantial liquidity and sets reference prices for benchmarks and settlement.

These U-shaped patterns in volume, volatility, and spreads shape execution timing and signal interpretation. Across the 13-day ITCH sample, the first and final 30 minutes each account for roughly 15–17% of daily volume, compared with 2.6% at midday—about a 6× open-to-midday ratio (see `06_itch_intraday_patterns`).

O’Hara (2015) emphasizes that algorithmic trading has altered flow interpretation: institutional parent orders now break into many passive child orders so that a large buy may appear as a stream of sell-side prints (passive fills on the bid). This can confound naive trade-signing heuristics. The next section turns to the content of market data feeds.

3.2 The anatomy of modern market data feeds

Market data arrives through a hierarchy of feeds that differ in *latency*, *cost*, *coverage*, and *information content* . Reck (2022) offers a practitioner’s view of how market design choices propagate into data

products and, ultimately, what strategies are feasible. In practice, data decisions are part of strategy design: the feed you can afford and process determines what you can observe (and which information is not visible), how quickly you can observe it, and which research and backtesting assumptions are defensible.

The data hierarchy - From best quote to full order book

Vendor terminology varies; here, we define the distinctions among market data feeds by information content: top-of-book, price-level depth, and order-level events. Examples use U.S. equities, but the hierarchy generalizes across venues and asset classes:

Level 1 (L1): Top-of-book quotes

L1 provides the *best displayed prices* currently available.

- **What it is:** The highest bid and lowest ask at the top of the book.
- **BBO** is venue-specific (in other words, the best bid/offer for a given venue).
- **NBBO** is the national best bid/offer across protected exchanges via the SIP (top of book). It reflects protected quotations on exchanges (not dark pools/internalizers), and its construction depends on the prevailing lot/quotation rules.
- **What you can do with it:** Compute spreads and midprice, build basic **Trades and Quotes (TAQ)** signals, and define immediately actionable reference prices.
- **What it hides:** Depth beyond the top level and any queue dynamics that determine fill probability and slippage.

Level 2 (L2): Market-by-price depth (MBP)

L2 adds *displayed depth (number of resting limit orders) by price level* on each venue (often top N levels).

- **What it is:** Aggregated size at each price level (orders are combined by price).

- **What you can do with it:** Estimate displayed liquidity, depth imbalance, and execution difficulty beyond top-of-book.
- **What it hides:** Order count, queue position, and time priority within a price level.

Level 3 (L3): Market-by-order messages (MBO)

L3 provides *order-level events* that allow deterministic replay of the book state.

- **What it is:** Submissions, modifications, cancellations, and executions at the individual order level.
- **What you can do with it:** Reconstruct venue-level book state and measure order flow (cancellations, replenishment, event-by-event dynamics).
- **What it hides:** Nothing about displayed order flow on that venue—but it is more demanding operationally and typically more expensive than L1/L2.

Most commercial “tick” products are TAQ; deeper products add depth or order messages.

Trades and quotes - The standard data package

Most medium-horizon strategies can be built from TAQ alone (prices, spreads, volume, and basic liquidity proxies).

- **Trades** are completed transactions with price and size.
- **Quotes** summarize the displayed bid/offer (top-of-book); depth is provided separately in L2/MBP products.

In U.S. equities, consolidated infrastructure produces a market-wide view of top-of-book quotes and trades (including the official NBBO), while exchanges sell proprietary direct feeds with faster delivery and richer fields. Consolidated feeds prioritize coverage; direct feeds prioritize speed and venue-specific detail.

TAQ-style datasets are sufficient for many medium-horizon strategies, but they are limited for microstructure work. Without order-level messages, you cannot reconstruct the evolving book, estimate queue position, or directly measure liquidity replenishment and cancellation dynamics.

Enriched TAQ products (for example, minute bars with precomputed microstructure fields) can help bridge part of this gap by incorporating trade-direction buckets, quote-pressure proxies, or off-exchange activity. They are useful, but treat them as *derived features* with embedded assumptions. Understanding what they measure (see `13_algoseek_minute_bars_eda.py` for the 61-column schema) prepares the way for feature engineering in *Chapter 8*.

Price conventions in this chapter

Different data products expose different “prices,” and they are not interchangeable. We use the following conventions and label them explicitly in figures and captions:

- **Midprice** : $(\text{bid} + \text{ask}) / 2$, computed from the relevant L1 quote (venue BBO or consolidated NBBO, depending on the feed). We use midprice changes as a default proxy for price discovery because it reduces bid–ask bounce relative to trade prices.
- **Last trade (print)** : The most recent transaction price. It reflects a completed trade but includes microstructure noise from trade direction and execution venue.
- **Close** : In bar construction, typically the last trade within the bar interval; for daily bars, often the official closing price from the exchange’s closing auction.

Rule of thumb: Use midprice for measuring short-horizon price response (signals); use executable prices (bid for sells, ask for buys) for PnL markouts; use last trade/close for standard OHLC bars unless stated otherwise.

These distinctions shape strategy feasibility. L1 signals are widely available and therefore more crowded. Strategies that depend on depth, queue dynamics, or very short-horizon order flow usually require L2 or L3—and that shift increases engineering requirements and data costs.

The feeds, books, and bars in this chapter are outputs of market microstructure mechanisms, not neutral measurements. That perspective helps you choose fit-for-purpose data, encode realistic backtest assumptions, and make sampling decisions that reduce bias rather than create it.

Sample datasets - From MBO to enriched bars

This chapter uses five tick-level sources spanning the data hierarchy:

Market-by-order (L3):

- **NASDAQ ITCH** provides order-level events for LOB reconstruction and order-flow analysis (notebooks 01 – 07 , with bar sampling on the same source in notebooks 14 – 16). Raw binary messages (>10GB/day uncompressed for our sample) require parsing but offer complete visibility into venue-level book dynamics.
- **DataBento MBO** offers similar granularity in a user-friendly format on a commercial basis (notebooks 08 , 09 , and 17).

Market-by-price depth (L2):

- **IEX HIST** provides free, publicly accessible depth data aggregated by price level (L2). You can build a price-level book (aggregate size at each price level), but you can't track individual orders or queue positions. Simpler than MBO but sufficient for spread analysis and depth-based signals (notebook 10).

Trades and quotes:

- **AlgoSeek TAQ** delivers nanosecond-precision trades and NBBO quotes without depth. Notebooks 11 – 12 analyze tick-level patterns during the March 2020 crash.
- **AlgoSeek minute bars** provide 61 precomputed microstructure columns derived from TAQ data—trade buckets, pressure indicators, and off-exchange volume—serving as input to *Chapter 7* feature engineering (notebook 13).

These sources illustrate a practical reality: “tick data” is a family of products, not a single standard.

Message protocols - From human-readable to fast binary

The hierarchy maps directly to distribution protocols: TAQ-style consolidation vs. venue-direct binary feeds; crypto often via APIs:

- The **FIX** protocol remains central to *order entry* and many *post-trade workflows*, but it is not designed for high-rate public-market data distribution.
- Real-time exchange feeds typically use **binary protocols and multicast** for high throughput. Examples include NASDAQ ITCH (notebook `01_itch_parser`), NYSE XDP, and Cboe PITCH.
- Crypto markets often expose data through API-style access: **REST** for historical downloads and **WebSocket** streams for real-time updates. The barrier to entry is lower, but stability, rate limits, schema drift, and cross-venue timestamp synchronization become first-order data-quality problems.



Further reading : Novocin and Weber (2022) examine how blockchain, decentralized autonomous organizations, and generative AI are reshaping exchange platforms and the data products built on them.

Data integrity challenges

At high frequencies, infrastructure details become data-quality constraints:

- **Latency and co-location:** For sub-second strategies, *where* you ingest data, and *which feed you use* can change what you observe.

SEC market structure work and SIP latency analyses (Holden et al., 2023) document that consolidation and distribution introduce delays, and that direct feeds can arrive earlier—differences that can range from hundreds of microseconds to single-digit milliseconds depending on network design and proximity. Aquilina et al. (2021) assert that latency arbitrage races occur roughly once per minute per liquid symbol (FTSE 100), with the modal race lasting just 5–10 microseconds, and impose an aggregate tax of approximately 0.5 basis points on trading—billions of dollars annually across global equity markets.

- **Timestamp semantics:** Vendors may normalize, truncate, or re-stamp timestamps. Verify whether a timestamp reflects the exchange event, SIP publication, vendor processing, or local arrival.
- **Exchange time versus decision time:** Treating exchange time as decision time creates look-ahead bias when observation and processing delays are ignored. Use arrival timestamps when available, or apply conservative lags when they are not—especially for event studies and sub-second strategies—depending on the estimated event travel time. Holden et al. (2023) emphasize that latency affects trade–quote alignment and can bias standard microstructure measures if handled incorrectly.
- **Corrections and cancellations:** Real feeds include corrections, cancellations, and late reports. For historical research, prefer vendor-cleaned end-of-day datasets over archived real-time captures, but confirm the provider’s correction policy.
- **Fragmentation and normalization:** Modern equity markets span exchanges and off-exchange venues (ATs, dark pools, internalizers). Off-exchange volume has grown steadily—from roughly 37% in 2019 (SEC 2020) to over 50% by the early 2020s (SIFMA US Equities Volume Summary)—which shapes data availability and price discovery. The burden is not just “more

venues,” but schema normalization, symbol handling, clock alignment, and deterministic replay across heterogeneous feeds.

Subscription, parsing, timestamping, and precision set up everything that follows. The next section turns to the most information-rich case, message-level events. The goal is not parsing for its own sake but making implicit market state explicit—a prerequisite for understanding what microstructure features measure and what backtests assume.

3.3 From raw messages to the limit order book

The **limit order book (LOB)** contains all active limit orders at any given point in time. Reconstructing the LOB turns message traffic into state: a time-indexed view of best prices, depth by level, and the event history that produced them. This is the core data-engineering step behind microstructure-aware research and execution modeling. Throughout this section, “LOB” refers to the *displayed, venue-local* book implied by the feed (not a consolidated market-wide book).

Practically, reconstruction follows a simple pipeline: parse messages, normalize fields, apply updates, enforce invariants, snapshot the state, and validate outputs against independent references (for example, BBO/NBBO time series). Gould et al. (2013) survey LOB modeling and empirical regularities, providing useful conceptual foundations. Zhang, Zohren, and Roberts (2019) apply convolutional neural networks (DeepLOB) and Zhang et al. (2025) apply clustering (ClusterLOB) to generate trading signals from LOB data.

NASDAQ TotalView-ITCH - A case study

NASDAQ’s TotalView-ITCH data feed exemplifies modern, message-by-order exchange data. It publishes order-level events—adds, cancels, replaces, and executions—that let you reconstruct the visible book throughout the trading day and study how liquidity evolves at the venue.

NASDAQ also provides downloadable historical samples, making ITCH a practical learning resource for readers who want hands-on exposure to

real exchange protocols. We use a single day of data (January 30, 2020) for the headline pipeline and detailed empirical work. The raw file contains 423 million messages and expands to roughly 13 GB uncompressed, which demands streaming I/O, memory discipline, and reproducible parsing.

Binary parsing at scale

Each message follows a precisely defined binary structure with nanosecond timestamps and fixed-point prices (four implied decimals). A single trading day contains hundreds of millions of messages, requiring streaming processing—parse sequentially and write to disk incrementally.

Processing hundreds of millions of binary messages exposes the performance gap between research and production systems. The companion repo provides two parsers: a **Python implementation** prioritizing readability (about 23 minutes for a full day via `01_itch_parser.py`) and a **Rust implementation** that uses memory-mapped I/O for roughly an order-of-magnitude faster parsing (well under five minutes).

Metric	Python	Rust	Notes
Wall-clock time	~23 min	< 5 min	~5–8× speedup
Memory footprint	~8 GB peak	< 500 MB	~16× smaller

Table 3.1: 13 GB ITCH file (423M messages). Timings are hardware-dependent and approximate; precise Rust benchmarks are pending a multi-machine benchmark notebook

Both parsers emit the same schema and are validated via checksum sanity checks, round-trip decoding tests, and consistency checks on downstream outputs (for example, identical BBO time series).

The parsed output—Parquet files partitioned by message type—serves as the stable input to all downstream notebooks. Whether the parse runs in Python, Rust, or a commercial service, the reconstruction logic below is unchanged.

The LOB state machine

A reconstructed order book is a state machine updated by each event. The implementation must maintain two related views of state:

- An **order registry** keyed by order reference number to track remaining size, price, and side. Later messages (cancel/execute/delete) refer to the original order ID.
- A **price-level view** for fast queries and snapshot generation.

The ITCH 5.0 specification defines 20+ message types; for LOB reconstruction, the essential ones are:

- **Add Order** : **A** (anonymous) and **F** (with MPID attribution)
- **Cancel/Delete/Replace** :
 - **X** (partial cancel)
 - **D** (delete)
 - **U** (replace: old order terminated, new order ID created)
- **Executions** :
 - **E** (executed at the order price)
 - **C** (executed with an explicit execution price)

System context messages (for example, **S** system events, **R** stock directory, **H** trading action) are required to correctly interpret the trading day. Trade-reporting messages such as **P** (trade) and **Q** (cross trade) are important for activity analysis and auctions, but they are not book-update events in the same way as adds, cancels, replaces, and executions.

The registry answers “what remains of order 123?”, the price view answers “how many shares are bid at \$101.23?” The core ITCH mechanics are:

- **Add (A, F)** creates a visible limit order: assert that the reference is new, insert it into the registry, and increase the depth at that price. **F** is “add with attribution” (MPID), not a different economic event.
- **Cancel/Delete/Replace** modifies or terminates liquidity: **X** reduces shares (partial cancel), **D** removes the order, **U** terminates the original and creates a new reference—correct handling of replace chains is essential.
- **Executions** reduce the remaining shares on a resting order: **E** executes at the order price, while **C** supplies the execution price when it differs from the order price. Trade-reporting messages such as **P** (trade) and **Q** (cross-trade) are useful for activity analysis and auctions, but they are not book-update events like adds, cancels, replaces, and executions.
- **Trade and cross messages (P, Q)** contribute to execution analytics but do not substitute for order-level book updates.

Figure 3.1 maps each ITCH message type to a state transition in the order lifecycle.

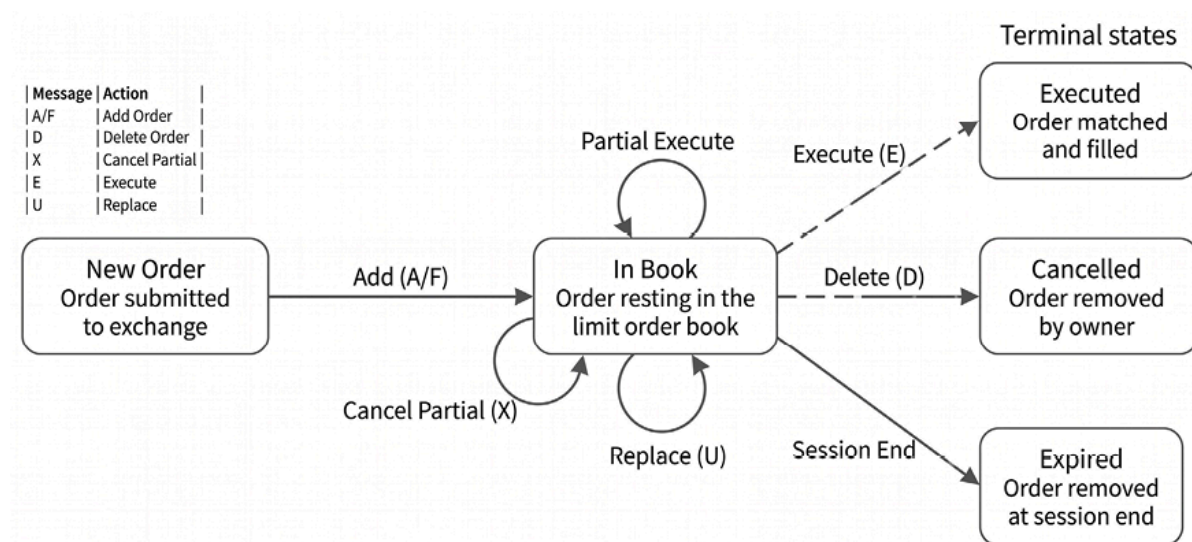


Figure 3.1: Order lifecycle state machine showing how ITCH message types drive transitions

On our sample day, roughly 96% of submitted orders are ultimately deleted; many never execute, while some execute partially and are later deleted (see `04_itch_order_lifecycle_analysis.py`).

Maintaining book integrity

LOB reconstruction is an accounting problem: every message updates the state, and small inconsistencies compound quickly. Cont, Kukanov, and Stoikov (2014) analyze how order book events translate into price movements, making accurate reconstruction a prerequisite for meaningful measurement of impact and liquidity. In practice, treat reconstruction as a state machine, as depicted in *Figure 3.1*, with explicit invariants. Our reference implementation fails fast when these invariants are violated; production pipelines often quarantine and count violations instead, then decide whether to drop symbol/time ranges or apply feed-specific repair rules. At a minimum, validation checks should enforce that:

- Order references exist before any modification, execution, or cancellation
- The remaining quantity is never negative (per order and per price level)
- Prices are valid for the instrument (tick size, auction bands where applicable) and remain within sanity bounds
- Aggregate depth by price level equals the sum of constituent orders at that price (within the reconstructed scope)

In addition, maintain *price-level totals* as a first-class state. Price-level cleanup is then a consequence of the accounting: decrement the size when trades execute, or orders are canceled; remove a level when its aggregate size reaches zero; and flag an error if a level would become negative.

When the pipeline persists periodic snapshots, reconcile them against the event-driven state (for example, level totals, best bid/ask) to catch drift early.

Single-exchange data - A local view

A key limitation of direct feeds, such as Nasdaq TotalView-ITCH, is that they describe the state of a **single venue's order book** (Nasdaq's "single book") rather than a consolidated market-wide book across venues. In fragmented equity markets, the best price and deepest liquidity may reside on other exchanges or off-exchange at the same moment, so a single-venue book is an incomplete view of executable liquidity.

This matters in two ways:

1. *Venue-local best quotes* can differ materially from the NBBO, and the depth that looks "available" on one venue may be irrelevant if an order routes elsewhere (or if routing constraints prevent access).
2. Comparing a venue-local reconstruction with consolidated data can produce *apparent* locks or crosses at the market level (for example, a local best bid exceeding the consolidated best ask) due to timestamp differences, feed latency, and the fact that "market-wide" best quotes are assembled from multiple asynchronous sources.

Treat these as diagnostic signals: they can indicate real fragmentation and timing issues, but they can also surface clock alignment issues or message-handling bugs.

Market-wide best prices require consolidated top-of-book data (SIP/CTA/UTP) and, for depth, a **multi-venue aggregation pipeline** that normalizes timestamps and quote semantics across feeds. Single-venue books remain valuable for venue-level microstructure, queue dynamics, and building intuition about how order events translate into observed prices—provided the analysis is explicit about what the book does and does not represent.

Snapshots and analysis

Once reconstruction passes invariant checks, capture order-book snapshots either at a fixed clock interval (for example, every 100ms) or in event time (for example, every N book updates). The choice is part of the measurement design: clock-time snapshots align naturally with latency and execution constraints; event-time sampling aligns with information arrival but can over-represent active periods.

From snapshots, compute a small set of metrics that summarize liquidity and state:

- **Bid-ask spread** measures the cost of immediate execution in the displayed book. Track both the absolute spread and a relative measure (for example, spread divided by midprice) to compare instruments.
- **Displayed depth** measures available size by level. Top-of-book depth matters for short-horizon execution; deeper levels for capacity and impact.
- **Top-of-book imbalance** refers to the asymmetry in displayed depth at the best prices. This snapshot descriptor differs from event-based **order flow imbalance (OFI)** (Cont, Kukanov, and Stoikov, 2014). Hautsch and Huang (2012) show that limit orders, not just marketable orders, carry meaningful information for price discovery—challenging the view that only market orders drive prices.

NVDA reconstructed limit order book depth

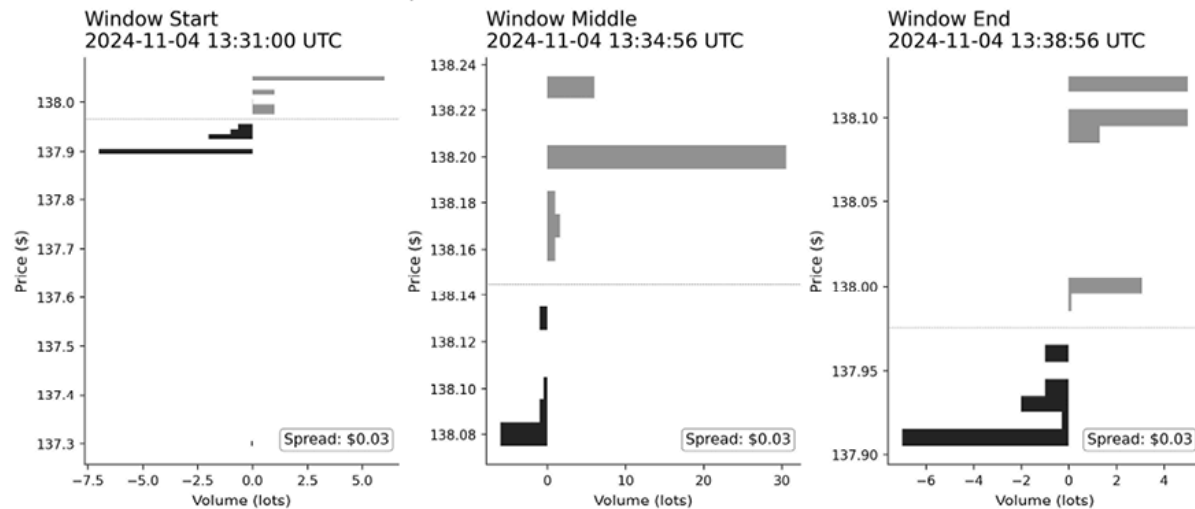


Figure 3.2: A reconstructed LOB showing bid (green, left) and ask (red, right) depth across price levels at three times during the trading day. Data: DataBento MBO for NVDA, regular session

In Figure 3.2, the Nasdaq-visible spread varies from \$0.01 at midday (when liquidity is deepest) to \$0.06 near close. This intraday pattern—tighter spreads during calm periods, wider at the open and close—reflects the changing cost of providing liquidity. These are venue-local measures from Nasdaq’s single book and may differ from NBBO-based measures in fragmented markets (see `08_databento_lob_reconstruction.py`).

Empirical findings

Modern electronic markets are dominated by order updates and cancellations. Hasbrouck and Saar (2013) document that the vast majority of limit orders are canceled before execution and that median lifetimes are well under a second—“fleeting orders” probing for hidden liquidity. Our single-day NASDAQ ITCH snapshot (January 30, 2020) of 423 million messages includes 186.6 million add-order events and the following lifecycle counts (computed by `04_itch_order_lifecycle_analysis.py`):

Metric	Count
Add-order events (A + F)	186,610,705
Delete events (D)	180,285,101
Partial-cancel events (X)	4,990,972
Replace events (u)	36,777,372
Execute events (E + C)	8,555,084
Unique orders with a delete event	172,764,815
Unique orders with at least one execution	6,258,508
Cancellation rate (delete events/add events)	96.6%
Execution rate (orders with execution/add events)	3.4%

Table 3.2: Order Lifecycle Statistics (NASDAQ, January 30, 2020)

Rates are not complements: an order can be partially executed and later deleted. Cancellation rate is the fraction of submitted orders that receive at least one delete event in their lifecycle; execution rate is the fraction with at least one **E** or **C** execution. Counts of **D** / **X** / **U** events exceed unique-order counts because the same order can be partially canceled before being deleted, and replace messages create a new order reference whose lifecycle is then tracked separately.

Time to cancellation:

- 41% of cancellations occur within 500 milliseconds
- 50% within one second (median: 0.99 seconds)
- 80% within 10 seconds

Time to execution:

- 10% of executions occur within 1 millisecond
- Median execution time is 6.1 seconds

- 1% of orders wait over 40 minutes for execution

These patterns are consistent with algorithmic market making and automated order management. High cancellation and replacement activity is not, by itself, evidence of misconduct; it is a mechanical consequence of competitive quoting, inventory control, and rapid information incorporation—while still implying that displayed liquidity can be fleeting and must be modeled carefully. See

`05_itch_trading_activity.py` for market-wide statistics.



Important caveat : Reconstruction systems show the *visible* order book. Hidden orders (icebergs) and internalized order flow remain invisible—a reconstructed LOB represents only displayed liquidity, not total available liquidity.

LOB stylized facts - Predictive patterns

Beyond reconstruction mechanics, LOB data reveals empirical patterns with predictive power. The companion notebooks demonstrate three well-documented regularities:

- **Top-of-book depth imbalance** measures the asymmetry between bid-side and ask-side volume at the best prices: $(\text{bid_size} - \text{ask_size}) / (\text{bid_size} + \text{ask_size})$. When bids exceed asks, buying pressure often leads to short-term price increases—but the signal is weaker than often assumed.

Our analysis of Nasdaq-book OFI across 50 stocks shows a *weak, noisy correlation* (cross-stock mean $\rho \approx 0.01$, $\sigma \approx 0.10$, range -0.35 to 0.16) with subsequent returns (*Figure 3.3*). Each point in

Figure 3.3 represents one stock; the x-axis shows activity level, and the y-axis shows the imbalance-return correlation; see `03_itch_lob_analysis` for details.

The finding that naive imbalance metrics are not directly tradeable is consistent with the literature: raw microstructure signals require conditioning on context (spread dynamics, time of day, extreme events) to extract actionable edge. *Chapter 6*’s order flow reversal strategy demonstrates how conditional signals can work where unconditional ones fail.



Terminology note : This is a *state-based* measure from LOB snapshots. *Chapter 8* covers additional imbalance metrics, including *trade imbalance* (from signed trade volume) and *event-based OFI* (Cont, Kukanov, and Stoikov, 2014), which measure queue changes rather than queue levels.

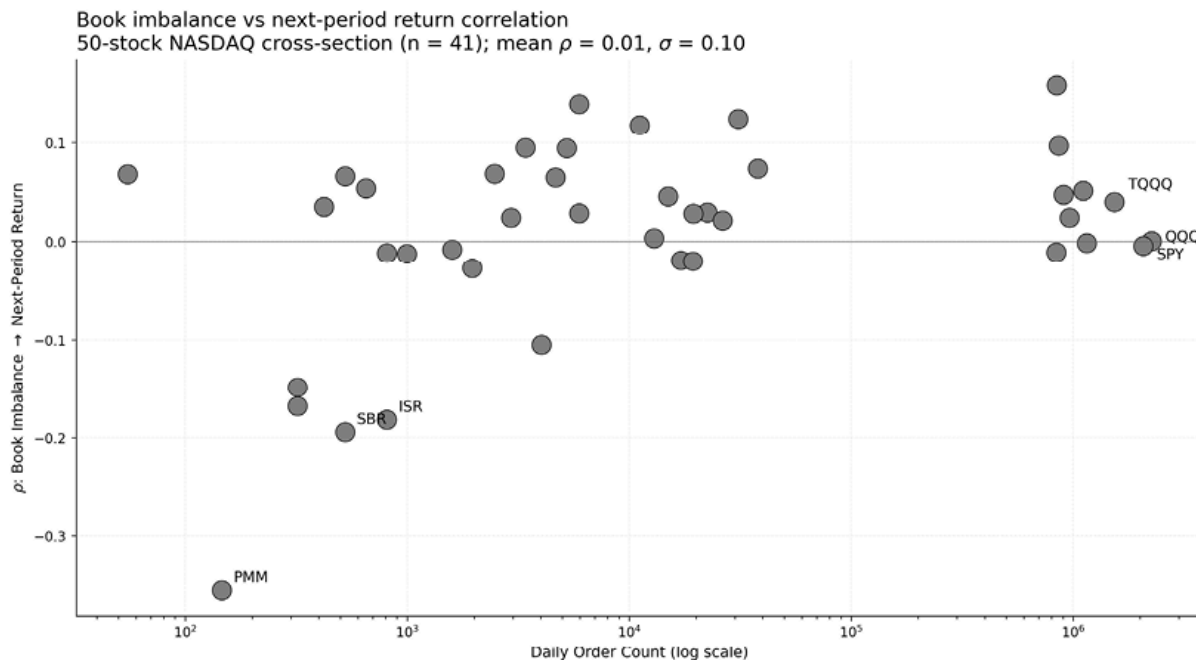


Figure 3.3: Naive imbalance shows weak predictive power. Data: NASDAQ ITCH, multiple symbols, January 30, 2020



Tradability caveat : A positive association at a one-second horizon is not a trading rule. At this horizon, spread and queue dynamics dominate PnL. The notebook `09_databento_mbo_analysis.py` reports both midprice and executable markouts (using bid/ask at decision time) to illustrate this gap.

- **Intraday U-shaped patterns** in volume and volatility are well established: trading activity peaks at the market open and close, with a lull in the middle of the day. This pattern persists across both liquid and illiquid stocks, though the magnitude varies. See `06_itch_intraday_patterns.py` for the aggregate cross-section.
- **Bid-ask bounce** manifests as negative first-order autocorrelation in tick-level returns. When trades alternate between hitting the bid and lifting the ask, sequential returns exhibit apparent mechanical mean reversion. Understanding this effect is essential for constructing return targets—naïve tick-to-tick returns contain significant microstructure noise. See `07_itch_stylized_facts.py` for autocorrelation analysis.

The liquidity spectrum spans orders of magnitude. AAPL trades 50+ million shares daily; obscure ETFs like UGA may trade fewer than 100,000 shares. This dispersion in displayed depth is associated with differences in spread, volatility, and price impact—patterns we exploit when engineering features in *Chapters 8 and 9*, and modeling execution costs in *Chapter 18*. The next section addresses how to sample this high-frequency data into analytical bars.

Implementation : For LOB reconstruction across the four data sources surveyed in *Section 3.2* :



- `02_itch_lob_reconstruction.py` (ITCH MBO)
- `08_databento_lob_reconstruction.py` (DataBento MBO)
- `10_iex_lob_reconstruction.py` (IEX MBP)
- `12_algoseek_taq_lob_reconstruction.py` (AlgoSeek TAQ NBBO)

3.4 The art of sampling

Raw tick data is voluminous and noisy. The **bid-ask bounce** alone causes price oscillations as trade initiation alternates between buyers and sellers—a buy market order executes near the ask, a sell near the bid, creating apparent volatility that reflects microstructure mechanics rather than information.

Sampling ticks into analytical **bars** reduces noise, compresses data, aligns observations with a chosen notion of ‘time’ (clock time or activity time), and produces time series with statistical properties better suited to machine learning models. The choice of sampling method significantly affects model performance.

Standard aggregation methods

All bar types compute similar summary statistics over their respective intervals:

- **Open, High, Low, Close (OHLC)** : First, maximum, minimum, and last prices
- **Volume** : Total shares or contracts traded
- **VWAP** : Volume-weighted average price
- **Trade count** : Number of individual transactions

The methods differ in how they define interval boundaries.

Time bars

Time bars aggregate trades within fixed calendar intervals (1-minute, 5-minute, daily). They are intuitive and ubiquitous, but they suffer from **unequal information content** : a minute during the opening rush can see hundreds of trades; a minute during lunch might see only a handful.

Tick, volume, and dollar bars

Several bar types aim to hold some proxy of trading activity, rather than time, constant:

- **Tick (or trade) bars** aggregate a fixed number of transactions regardless of size—equal transaction count, but ignoring trade magnitude.
- **Volume bars** aggregate until the total volume exceeds a threshold, thereby better capturing market activity because large trades carry more weight.
- **Dollar bars** aggregate by traded value ($\text{price} \times \text{volume}$), providing more robust handling of price level changes than volume bars—the same shares at doubled price represent twice the economic activity.



Note : Dollar bars help capture organic price changes but still require attention to corporate actions; a 2-for-1 split doubles the share count while halving the price, so the dollar threshold must be rescaled or prices must be split-adjusted.

Easley, López de Prado, and O'Hara (2012) develop the “ **volume clock** ” concept, showing that sampling based on volume rather than calendar

time yields more stable statistical properties—a key motivation for activity-time sampling methods such as volume and dollar bars. High-frequency trading is defined not by speed per se but by operating in “**event-based time**,” which yields returns that are closer to a normal distribution and exploits traders who think in clock time.

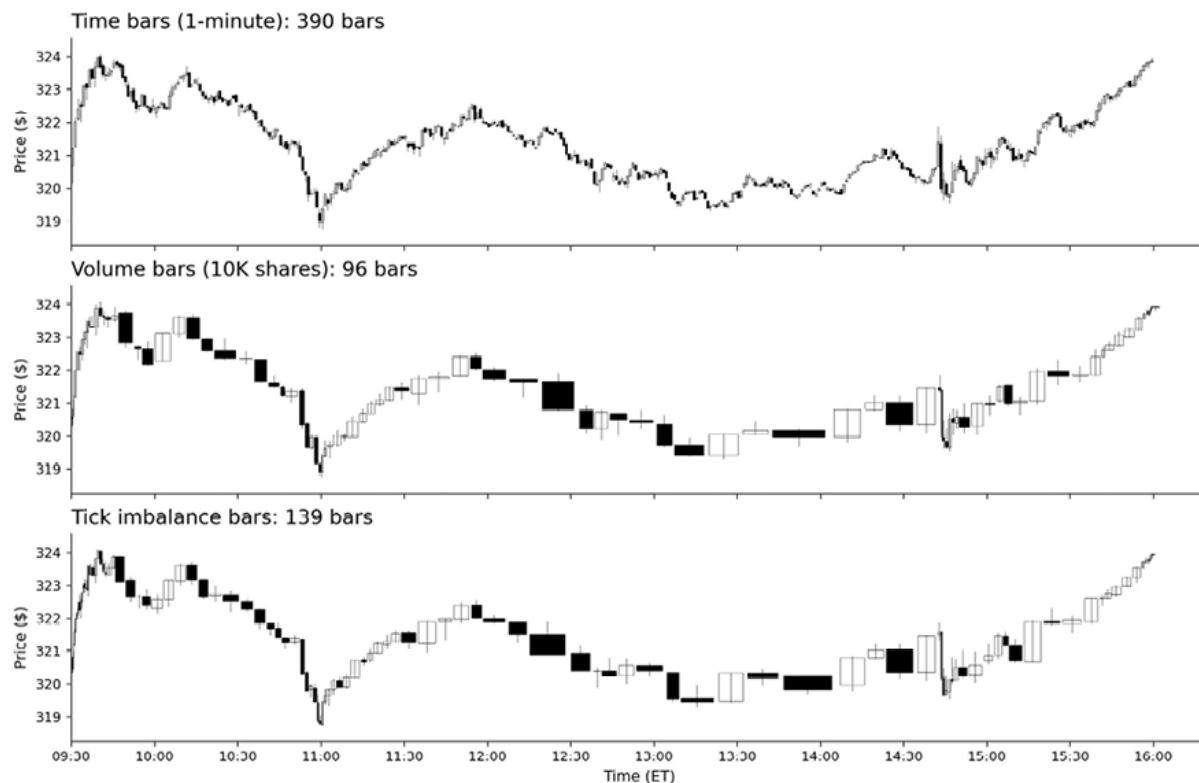


Figure 3.4: Comparison of bar sampling methods on the same underlying tick data

As shown in *Figure 3.4*, time bars produce a fixed sequence of intervals. In contrast, volume bars adapt to market activity, concentrating samples during high-activity periods and spacing them out during quiet periods. See `14_itch_bar_sampling.py` for the comparison code.

Information-driven bars

Standard time and volume bars summarize trading activity but don’t explicitly target when new information arrives. *Information-driven bars* aim to identify when new information enters the market, drawing on

concepts from market microstructure theory. Marcos López de Prado introduced these methods in *Advances in Financial Machine Learning* (López de Prado, 2018), building on research showing that sequences of trades consistently initiated by one side—buy or sell streaks—often signal informed trading.

Trade classification - Data sources matter

Information-driven bars require knowing the **aggressor side** of each trade—whether the buyer or seller initiated the transaction to identify when a buy or sell streak starts. López de Prado lists “Aggressor side” as standard market data in his taxonomy, and CME provides it directly via FIX tag 5797. The challenge is that not all data feeds include this field:

- **Some institutional MBO feeds** (for example, DataBento) provide the aggressor side directly. Most trades in our DataBento NVDA sample include aggressor-side labels (B=buy, A=sell), with a minority marked as neutral/unknown—enabling direct computation of imbalance bars without estimation. Other vendors (Refinitiv, Bloomberg) vary by product and entitlement.
- **ITCH Trade (P) messages**, while excellent for learning the protocol, do not include aggressor direction—each trade reports the same indicator regardless of which side initiated it. For the ITCH sample data, we must estimate direction.

The Lee-Ready algorithm

When the aggressor side is unavailable, the **Lee-Ready algorithm** (Lee and Ready, 1991) estimates it by combining quote and tick information:

1. **Quote (midpoint) test** : Compare trade price to the midpoint of the prevailing bid-ask spread. Trades above the midpoint are classified as buys; trades below as sells.

2. **Tick test fallback** : When trades occur exactly at (or extremely near) the midpoint, use the price change from the previous trade. An uptick (a price increase) indicates a buy; a downtick (a price decrease) indicates a sell. If unchanged, carry forward the last non-zero direction.



Practical note : Lee-Ready is only as good as the **quote alignment** . If quotes and trades are not time-synchronized, define an “as-of” rule (or apply a small lag) to avoid using quotes that were not yet visible at the time of the trade.

Empirical validation - Classification accuracy

We validated classification methods using DataBento NVDA data, in which direct aggressor labels serve as the ground truth. The results reveal important differences between approaches:

Method	Coverage	Accuracy	Notes
Lee-Ready (quote plus tick)	100%	94.7%	Requires the best bid/ask quotes aligned with each trade
Tick test (continuous)	100%	80.0%	Carries forward the last direction
Tick test (non-zero)	19%	90.0%	Only classifies when the price changes

Table 3.3: Trade classification accuracy (NVDA, 742,527 trades across 5 DataBento days)

Table 3.3 shows validation results for NVDA trades using DataBento aggressor labels as the ground truth over five trading days (2024-11-04 to

2024-11-08). Accuracy is computed on trades with non-unknown ground-truth side; unknown/neutral trades are excluded, and quotes are aligned using an as-of rule with a small lag. Lee-Ready accuracy was stable across days (94.4–95.0%); the tick test averaged 80%. “Coverage” indicates what percentage of trades receive a classification. See `15_itch_lee_ready.py` for the full multi-day analysis.

The tick test faces a fundamental challenge: **most consecutive trades occur at the same price** (zero-tick)—across the five NVDA days, only about 19% of trades carry a non-zero price change. The continuous tick test handles this by carrying forward the last non-zero direction, reaching 80% accuracy—roughly 15 percentage points below Lee-Ready. When only non-zero ticks are classified, accuracy rises to 90%, but coverage drops to that same 19% of trades.

Lee-Ready achieves 95% accuracy by using the order book state (best bid/ask at trade time) for most classifications, falling back to the tick test only for trades at the midpoint. This exceeds the 70–90% range typically cited in academic literature, likely because modern markets with tighter spreads allow more precise quote-based classification.

The practical implication of the ~16-point improvement from the tick test (78%) to Lee-Ready (95%) matters for information bars. With ITCH data, the order book must be reconstructed to compute quote midpoints for Lee-Ready classification—worthwhile complexity for accurate trade-direction estimation. See `14_itch_bar_sampling.py` for a complete implementation.

With only trade prices and no order book reconstruction, the tick test provides reasonable (78%) but not optimal classification. For information bars where direction accuracy directly impacts signal quality, prefer Lee-Ready.

Volume imbalance bars

Volume imbalance bars (VIBs) close when cumulative signed volume exceeds an expected threshold:

- **Label each trade** : +1 if buyer-initiated, -1 if seller-initiated (via Lee-Ready or similar)
- **Compute signed volume** : Label $\times$ trade volume
- **Accumulate imbalance** : Running sum of signed volumes
- **Close bar** : When $|\text{accumulated imbalance}|$ exceeds expected value

The expected imbalance is derived from historical bar characteristics, updated using an exponential moving average. The algorithm adapts: periods of balanced trading produce fewer, longer bars; periods of sustained buying or selling pressure produce more frequent bars.

Tick imbalance bars

Tick imbalance bars (TIB) accumulate signed trade direction rather than signed volume:

$$\theta_T = \sum_{t=1}^T b_t, \quad E[|\theta_T|] \approx E[T] \cdot |2Pr(b_t = +1) - 1|$$

where $b_t \in +1, -1$ is the trade direction. TIBs count all trades equally regardless of size, while VIBs (above) weight by volume.

This means that when the order flow is close to balanced ($Pr(b_t = +1) \approx 0.5$), the expected imbalance $|2Pr(b_t = +1) - 1|$ becomes small and TIBs produce *many short bars* . When one side dominates, thresholds grow, and bars become longer. TIBs are therefore most interpretable as a *directional pressure sampler* , not as a fixed “bars-per-day” construction. Choose TIBs when trade count matters (detecting

informed trading frequency); choose VIBs when volume flow matters (detecting institutional activity).

Run bars

Run bars form when one side of the market dominates, measured by cumulative trade counts within a potential bar:

$$\theta_T = \max(N_T^{buy}, N_T^{sell}), \quad E[\theta_T] = E[T] \cdot \max(Pr(b_t = +1), 1 - Pr(b_t = +1))$$

Run bars count *cumulative* trades on each side, not consecutive same-direction trades. The statistic is the maximum of cumulative buy versus sell counts within the bar, not the length of the longest consecutive streak. Direction changes within a bar do not reset the counts. This captures sustained directional pressure even when individual trades interleave.

Calibration - Avoiding threshold spiral

The adaptive **exponential weighted moving average (EWMA)** algorithm places greater weight on recent data points and is sensitive to persistent order-flow imbalances. When stocks show systematic buy/sell bias, the standard $\alpha = 0.1$ causes a **threshold spiral** , a positive feedback loop where bars grow progressively larger:

Symbol	Buy Fraction	E[T] Drift ($\alpha = 0.1$)	E[T] Drift ($\alpha = 0.001$)
NVDA	47%	17.5×	0.98× ✓

Table 3.4: Threshold behavior for NVDA

Even at a 47% buy fraction, $\alpha = 0.1$ inflates the expected bar size by 17.5× over the trading day—the runaway feedback loop that collapses the day into a handful of oversized bars—while $\alpha = 0.001$ holds it essentially flat (0.98×), avoiding the spiral.

Recommended settings: $\alpha = 0.1$ can be too reactive at the tick level for liquid equities; start with a much smaller α (for example, 0.001) and verify stability. Use warmup=100+ bars and monitor the `expected_imbalance` column for drift detection. For production use, consider fixed-threshold imbalance bars, which avoid adaptive feedback entirely. See `16_itch_information_bars.py` for formula verification and `17_databento_bar_sampling.py` for multi-day calibration analysis.

Information-driven bars pose several practical difficulties:

- **Trade classification** : The tick test (~78%) is roughly 16 points below Lee-Ready (~95%). For optimal quality, reconstruct the order book.
- **Circular initialization** : Expected bar length requires existing bars; initialize with reasonable estimates and update via EMA.
- **Parameter sensitivity** : Single trading days rarely suffice; multi-day data produces more robust thresholds.
- **Flow balance** : When $P[\text{buy}] \approx 0.5$, the expected imbalance approaches zero, causing rapid bar formation.

Statistical properties comparison

Before the formal statistics, it helps to compare the bar constructions on three axes: the sampling clock, the quantity each method keeps approximately constant, and whether it depends on trade-direction classification (the first four don't, the last three do).

Bar Type	Clock	Target	Typical use
Time	Calendar	Time interval	Clock-time forecasting

Bar Type	Clock	Target	Typical use
Tick	Activity	Trade count	Equal trade frequency
Volume	Activity	Shares/contracts	Size-based activity
Dollar	Activity	Traded value	General ML default
Tick imbalance	Information	Signed trade count	Directional pressure
Volume imbalance	Information	Signed volume	Informed flow by size
Run	Information	Side dominance	Persistent one-sided flow

Table 3.5: Bar characteristics

Empirical testing reveals meaningful differences between bar types. Using NASDAQ ITCH trade data for AAPL (January 30, 2020), we compare standard bars and information-driven bars (with tick-test trade classification). Bar thresholds are noted in parentheses:

Bar Type	N Bars	Jarque-Bera Stat	Lag-1 Autocorrelation
Time (1 min)	390	609.0	-0.023
Volume (10K shares)	96	1.9	0.026
Dollar (\$3M)	102	2.2	-0.027
Tick imbalance	139	22.6	0.057
Tick Run	455	12.5	-0.007

Table 3.6: Bar Type Statistical Comparison (AAPL, 14,184 trades); higher Jarque-Bera values make a normal distribution less likely

Table 3.6 shows trade direction, classified using the tick test (~78% accuracy, per *Table 3.3*). Volume and dollar bar thresholds (10K shares, \$3M) produce roughly 100 bars per day for AAPL; information-driven bars adapt to flow dynamics, with tick imbalance bars (139) concentrating during directional episodes and tick run bars (455) capturing persistent one-sided flow. The **Jarque-Bera (JB)** statistic measures how far a return distribution departs from normality by combining two features: **skewness** and **kurtosis** . A low JB suggests returns are closer to normal, while a high JB indicates greater asymmetry and/or fatter tails, making normality less plausible.

Using DataBento NVDA data with **direct aggressor labels** (100% accuracy), activity-driven bars achieve modestly better normality than time bars—Tick (500 trades/bar) JB=58.8 and Volume (50K shares/bar) JB=2.3 versus 132.8 for one-minute bars on the same day. The flow on this day was essentially balanced, marginally buy-tilted ($P[\text{buy}]=0.426$), so volume imbalance bars (calibrated to ~500 bars/day) produced 691 bars; dollar bars at \$5M/bar yielded 519 bars. Autocorrelation remains small for all types—a desirable property for ML.

Bar Type	N Bars	Jarque-Bera Stat	Lag-1 Autocorrelation
Time (1 min)	390	132.8	0.014
Tick (500 ticks)	390	58.8	-0.061
Volume (50k shares)	380	2.3	-0.017
Dollar (\$5M)	519	33.0	0.039

Bar Type	N Bars	Jarque-Bera Stat	Lag-1 Autocorrelation
Vol Imbalance	691	51.0	0.035

Table 3.7: Bar type comparison with Direct Aggressor Labels (NVDA, 2024-11-04, 195,420 trades)

Table 3.7 shows trade direction from DataBento’s side field. Activity-driven bars (tick and volume) achieve materially lower Jarque-Bera than time bars, while dollar and volume-imbalance bars remain far closer to normal than the one-minute clock.

Easley et al. (2021) revisit these dynamics in light of machine learning, arguing that information-based sampling remains relevant as algorithmic and ML-driven strategies have grown more prevalent.

Trade-offs and selection

Several trade-offs and selection decisions matter in practice:

- The default choice (most ML workflows) is to **use dollar bars** . They are simple to implement, require only trades, and, in practice, tend to produce more stable return distributions than time bars. They also scale naturally with the price level (define thresholds in notional terms). Adjust consistently for corporate actions (and contract multipliers/roll rules for futures).
- **Use time bars when the decision and label horizon are in clock time** (for example, “next 5 minutes,” intraday seasonality features, session-aligned execution constraints). Time bars are intuitive but have uneven information per sample across the day; expect heteroskedasticity and regime-dependent sample quality.
- **Use volume bars when “activity” is naturally measured in shares/contracts** (and price level is relatively stable), but prefer

dollar bars where robustness to price-level changes matters.

- **Use information-driven bars (imbalance/run) when order-flow dynamics are the object of measurement** (toxicity, informed trading, execution risk), not when the primary goal is “more normal” returns. These methods require reliable trade direction: use a direct aggressor field when available; otherwise, use Lee–Ready with aligned quotes. Tick-test-only classification is usable but materially noisier. Expect higher implementation and tuning overhead (for example quote alignment or book reconstruction, initialization, and stability monitoring).
- **With only trades and no quotes, prefer dollar (or volume) bars** and avoid imbalance/run bars unless the limitations of tick-test direction are acceptable and sensitivity has been validated.

For most applications, **dollar bars** are the best starting point: they exhibit good statistical behavior in many settings, handle price-level changes gracefully, and are not dependent on trade classification. Next, we turn to the role of price jumps.



Implementation : `14_itch_bar_sampling.py`

demonstrates how to construct bar types and compares them.

3.5 Detecting price jumps in intraday returns

Intraday returns mix two statistically distinct processes. A continuous component reflects the steady arrival of small order-flow innovations and is well approximated by a diffusion. A discrete component reflects scheduled news, unexpected announcements, and large block trades, and

manifests as price moves that are too large to attribute to diffusion. Separating the two matters because realized variance, autocorrelation diagnostics, and label distributions all change shape once jumps are removed.



Implementation : `18_algoseek_jump_detection` builds the full pipeline on AlgoSeek minute bars for AMD, AMZN, and FB across 2020. The results below are computed end-to-end in that notebook.

Continuous and jump variance

For a trading day with n intraday log returns $r_1, \dots, r_n$, the realized variance:

$$RV_d = \sum_{i=1}^n r_i^2$$

is a consistent estimator of total quadratic variation: continuous plus squared jumps. The bipower variation of Barndorff-Nielsen and Shephard (2004),

$$BV_d = \mu_1^{-2} \sum_{i=2}^n |r_i| |r_{i-1}|, \quad \mu_1 = \sqrt{2/\pi}$$

is jump-robust: a single large $|r_i|$ is multiplied by its smaller neighbor and contributes negligibly to the sum.

Therefore, the non-negative gap $\max(RV_d - BV_d, 0)$ estimates the jump-attributable variance on day d . In the notebook, the three names show annualized continuous volatility (from $\sqrt{BV}$) of 30% for AMZN, 34% for FB, and 45% for AMD; the jump component contributes roughly 6–10%

of the annual realized variance (6.8% for AMD, 10.4% for AMZN, 6.2% for FB). The 2020 sample spans the March dislocation, and the jump share reflects that episode rather than a steady-state value.

The Lee-Mykland Test

To time individual jumps to the bar, Lee and Mykland (2008) form a standardized statistic

$$L_i = \frac{r_i}{\hat{\sigma}_i}, \quad \hat{\sigma}_i^2 = \frac{1}{K-2} \sum_{j=i-K+2}^{i-1} |r_j| |r_{j-1}|$$

where $\hat{\sigma}_i$ is a local bipower volatility built from a trailing window of K prior returns. The window adapts to slow-moving volatility regimes and inherits the jump-robustness of bipower variation. Under the no-jump-at- i null and a continuous-diffusion alternative, the maximum of $|L_i|$ over n intraday bars converges to a Gumbel distribution.

The rejection rule at family-wise level α over n bars per day is:

$$|L_i| > S_n \beta^*(\alpha) + C_n, \quad \beta^*(\alpha) = -\log(-\log(1 - \alpha))$$

with $C_n = (2\log n)^{1/2}/\mu_1 - (\log \pi + \log \log n)/(2\mu_1(2\log n)^{1/2})$ and $S_n = 1/(\mu_1(2\log n)^{1/2})$.

The multiple-testing burden is borne by the n -dependent constants rather than by an ad hoc *Bonferroni adjustment* (see *Chapter 7* for more detail).

In the notebook, the test runs at $\alpha = 0.01$ with $K = 12$ (about an hour at the five-minute frequency, near Lee–Mykland’s $\sqrt{n}$ recommendation for 78 bars per day), yielding a critical value of 5.10 and 0.37 jumps per symbol-day on average. The window is built strictly within session, so the first $K - 1$ bars of each day are not testable—the trade for removing overnight-gap contamination from the local-volatility denominator.

Longer K is slower-adapting and burns more bars on warm-up; shorter K tracks the intraday shape more closely but is noisier.

What the data show

The daily jump-count distribution is right-skewed. Roughly a quarter to a third of symbol-days contain at least one jump (24% for AMD, 31% for AMZN, 34% for FB); the median day with any jumps has exactly one, and the maximum is three jumps in a single symbol-day.

The corresponding Q–Q plot in the notebook makes the diagnostic visible: standardized returns from all bars curve well above the normal-quantile line in the upper tail with a heavier lower-tail outlier, while standardized returns from jump-filtered bars track the normal line much more closely. Jumps are the source of most of the tail heaviness, not diffusion heterogeneity.

The time-of-day distribution shows a clear close-of-day cluster within the testable window: 27% of flagged jumps occur in the last thirty minutes of the session, where end-of-day rebalancing flows and the close auction produce sharp moves. The first thirty minutes fall within the $K - 1$ warm-up window, so the open-time rejection rate is zero by construction rather than an empirical finding; probing it at this frequency would require either finer-grained bars or a complementary estimator that pools cross-session information without overnight-gap contamination.

The year-2020 variance decomposition in the notebook (illustrated for AMD) stacks BV_d and the jump residual $RV_d - BV_d$ over time and shows that jump variance is episodic—concentrated around the March 2020 dislocation, a handful of earnings dates, and several macro-release days—rather than uniformly distributed.

Why a local volatility adjustment matters

A common shortcut flags any bar with $|r_i/\hat{\sigma}_d| > 4$, where $\hat{\sigma}_d$ is a single daily standard deviation. The notebook compares this rule with Lee–Mykland on the same data. The two disagree on most bars, and the disagreement is directional. The counts are 70 Lee–Mykland versus 55 naive jumps with 2 overlapping for AMD; 103 versus 48 with 6 overlapping for AMZN; 109 versus 49 with 6 overlapping for FB.

On stressed days, $\hat{\sigma}_d$ is dragged up by the jumps themselves so genuinely large bars no longer clear $4\hat{\sigma}_d$ —the naive rule under-detects. On calm days, $\hat{\sigma}_d$ is small, and the same threshold is easy to clear, so the rule fires on borderline returns—over-detection. The bipower local estimator avoids both failure modes by ignoring its own large neighbors and tracking the intraday volatility shape.

Jumps as features

The notebook persists a per-symbol, per-day feature panel with the four columns that downstream chapters consume: `jump_count`, `signed_jump_var`, `jump_variance`, and `jump_share` (jump variance as a fraction of RV). The panel feeds two specific uses:

- In *Chapter 7*, jump indicators support *label conditioning*—skipping the first thirty minutes after a flagged jump removes a class of returns whose statistical properties differ sharply from the diffusive labels the model is trained on.
- In *Chapter 8*, the daily continuous and jump variances are features in their own right: separate volatility regimes (continuous-only days, jump-heavy days, blow-up days) are estimable rather than implicit.

The decomposition operates on the same bar grid as *Section 3.4*’s sampling pipeline; it adds an event-aware feature layer on top of it. The next section turns to issues in the quality of microstructure data.

3.6 Microstructure data quality and sessionization

Microstructure research depends on two forms of correctness: data-quality checks that keep corrupted records out of analysis, and session logic that prevents time-boundary mistakes from creating artificial returns and invalid state transitions. *Chapter 2* provides the general framework; here we focus on intraday-specific failure modes.

Many microstructure “errors” are not extreme values but inconsistencies: out-of-sequence events, invalid book transitions, stale quotes, or trades that qualifiers mark for exclusion. Prefer invariant checks and cross-field consistency over purely statistical filters.

Start with event sequencing, because everything downstream assumes a correct event order:

- **Enforce ordering per symbol and venue** : Check that timestamps are monotone within each stream, and use feed sequence numbers to break ties. If the provider offers both, treat the sequence as authoritative ordering and the timestamp as event time.
- **Be explicit about time semantics** : Confirm whether a timestamp reflects exchange event time, SIP publication time, vendor processing time, or local arrival time. Holden et al. (2014) show that misaligned trade and quote times can introduce systematic bias into standard microstructure measures.

Once ordering is trustworthy, double-check the two primary observables—trades and quotes:

- Validate basic fields (positive price and size, valid symbols, expected tick size where available), and flag impossible spreads and other violations of instrument constraints.
- Then, honor qualifiers and condition codes: many feeds mark late reports, corrected prints, auctions, and other non-standard trades. Default to excluding records that are explicitly out-of-sequence, corrected, or non-eligible for “last sale” style analytics unless your analysis intentionally models them.
- Finally, treat stale quotes and feed drops as first-class issues. Distinguish “no trades occurred” from “data is missing”: long gaps are normal in illiquid names, but for liquid symbols, a long gap in quote updates often indicates a feed interruption.

If you reconstruct a LOB, quality control becomes an accounting problem: treat the book as a stateful system with explicit invariants—no negative depth at the order or price level; best bid $\leq$ best ask, with locks or crosses either forbidden or allowed only during explicitly modeled auction or transition states (open, close, halts) under documented rules; and order lifecycle consistency.

After aggregation, validate again at the derived data product level:

- Reconcile bars against the underlying events: bar volume matches summed trade size, VWAP equals the size-weighted mean of constituent trades, and OHLC matches first/max/min/last by the bar definition.
- When you have both venue-local and consolidated sources, expect differences in timing and conventions; treat large discrepancies as triggers for investigation rather than automatic errors. They may reflect real fragmentation, but they can also indicate clock misalignment, filtering differences, or a reconstruction bug.

Two AlgoSeek explorations make these failure modes concrete.

1. On AAPL during the March 16, 2020, COVID crash, median spread runs at 2.4 bps against a baseline closer to 1 bps, with a 9.9% intraday range—stress-day numbers that distinguish a real liquidity event from a feed glitch (see `11_algoseek_taq_eda`) .
2. AlgoSeek’s NASDAQ-100 minute panel runs continuously from 04:00 to 20:00 ET, so trade fields are null in 20–23% of bars (extended-hours intervals with no executions) while quote fields are never null; treating those nulls as missing data would silently bias any aggregate (see `13_algoseek_minute_bars_eda`) .

Statistical outlier filters (MAD, rolling z-scores) remain useful mainly as triage to surface candidates for inspection. For ticks and quotes, the default treatment is flagging and exclusion rather than interpolation: interpolation fabricates market activity, blurs discontinuities, and breaks consistency with the event stream. If you winsorize (replace extremes with a fixed value, for example, the 5th/95th percentile), drop, or otherwise modify records, log the rule, and keep an audit trail so results remain reproducible.

Apply the same rigor to sessionization. Misclassifying weekends, holidays, early closes, or daylight-saving transitions creates artificial returns (for example, “overnight” returns inside the regular session) and invalid book states. A robust workflow looks as follows:

1. Build the schedule from the current exchange calendar, including early closes and special sessions.
2. Tag observations as pre-, regular-, or post-session using local exchange time.
3. Validate that closed dates contain no regular-session events, and that early closes end on schedule.
4. Define the session date explicitly (the trading day a timestamp belongs to) and apply it consistently across trades, quotes, and

reconstructed book states.

5. Handle halts and auctions deliberately: exclude them from intraday modeling or treat them as distinct states with separate rules.

Run these checks after parsing, after reconstruction, and after bar aggregation, and use the data-quality and lineage framework from *Chapter 2* to keep QA rules and calendars configurable while preserving a consistent audit trail.

3.7 Summary

We treated market data as the output of a trading mechanism, not as a neutral time series. We connected market structure and feed design to the observables a researcher models: how quotes and trades encode liquidity provision, how order types and execution rules shape intraday patterns, and why message-level data enables analyses that are impossible with top-of-book snapshots. We also showed how this context matters operationally: reconstructing a limit order book requires enforcing book invariants and lifecycle consistency; sampling ticks into bars changes the statistical and economic meaning of each observation depending on the boundary rule.

The next step is to turn these clean, correctly sessionized observables into features. *Chapter 2* provided the general data-quality framework; *Chapters 8* and *9* use the bar- and book-derived outputs from this chapter to engineer predictive signals, including microstructure features that depend on order flow and depth dynamics. Disciplined parsing, validation, and sampling are not preliminaries; they determine what models can learn and what conclusions research can legitimately support.

The next chapter covers fundamental and alternative data across asset classes.

4

Fundamental and Alternative Data

While market data captures *what* happened in the market—prices, volume, and the mechanics of trading—fundamental and alternative data help explain *why*. Turning these sources into model-ready signals is less about clever algorithms than about disciplined pipelines: ingesting messy inputs, tracking revisions, validating identifiers, and producing time-consistent features that hold up in backtests and production.

Fundamental data is asset-specific information about the underlying economic drivers of value, rather than trading activity or market microstructure. In practice, fundamentals capture variables that influence expected cash flows, discount rates, or key constraints (for example, supply and demand), and they typically arrive through reporting or measurement rather than through markets. Examples include financial statements and filings for equities, issuer and covenant information for credit, supply-demand and inventory measures for commodities, and protocol and on-chain metrics for crypto. Alongside these sources, **alternative data** has expanded to include digital exhaust, such as web traffic, transactions, geolocation data, satellite imagery, and text, reflecting its growing use in active management (Green and Zhang, 2024).

In practice, two failures dominate the real-world use of fundamentals and alternative data: time consistency and entity consistency.

- **Point-in-time (PIT) accuracy** ensures that research uses only information *available* at the decision date. *Chapter 2* introduces the PIT vocabulary and the bitemporal data model; this chapter applies them to SEC filings, macroeconomic releases, and other revision-prone sources.
- **Entity resolution** ensures that records from disparate sources map to the same real-world issuer and tradable instrument despite naming variants, identifier changes, corporate actions, and mergers, and that the correct security identifier remains aligned over time.

This chapter provides the engineering playbook for both challenges; it will enable you to:

- Implement a bitemporal data model to distinguish valid time from knowledge time (*Section 4.1*).

- Parse SEC EDGAR filings and construct point-in-time corporate fundamental databases (*Section 4.1*).
- Apply various hierarchical entity resolution approaches (*Section 4.2*).
- Characterize fundamental data across asset classes (*Section 4.3*).
- Evaluate alternative data sources along signal, data quality, legal, and commercial/engineering dimensions (*Section 4.4*).
- Extract structured data from EDGAR text filings for NLP analysis (*Section 4.5*).

We'll start with the practical challenge of creating a data pipeline that is point-in-time correct.

4.1 The point-in-time pipeline

Chapter 2 introduced PIT correctness and the bitemporal vocabulary (valid time versus knowledge time). The question here is operational: given EDGAR filing histories, how do we build a dataset that returns the value eligible at decision time t , not the value that exists in a vendor database today? The core complications are amended filings, taxonomy changes, and corporate actions that create multiple “versions” of the same reporting period.

Why restatements create leakage

Consider a simple **P/E ratio strategy**. A company might report Q3 revenue of \$95M in October, then restate it to \$98M in the next 10-Q and again to \$100M in the next 10-K. A backtest that uses \$100M for October decisions has applied the *later* accounting view to an *earlier* decision—this is lookahead bias.

Luo et al. (2014) identify it as one of the “seven sins” of quantitative investing, showing how errors in data handling can completely reverse a strategy’s performance. The others are survivorship bias, storytelling bias, overfitting and data snooping bias, turnover and transaction cost, outliers, and shorting cost.

Bitemporal storage and as-of queries

The implementation follows the *Chapter 2* PIT contract: each fact must carry both the period it describes and the timestamp when that specific version became observable. A single reporting period may therefore have multiple versions over time: the original filing, an amendment, or a later restatement.

For earnings, markets often react first to the earnings release, typically via Form 8-K Item 2.02, and only later to the detailed Form 10-Q or Form 10-K. Using the 10-Q or 10-K

acceptance time as the availability timestamp is conservative for PIT backtests. If event-time precision matters, track both timestamps explicitly.

The **as-of query pattern** is straightforward:

1. Select the latest reporting period eligible at historical time t .
2. Within that period, select the latest version with an availability timestamp no later than t .

A practical caveat is the **SEC XBRL Frames API**. Frames returns the “last filed” value that best matches the requested period, which makes it useful for cross-sectional snapshots and prototyping, but not PIT-safe on its own. For PIT research, reconstruct the history from the filing-level records and timestamps.

The *Chapter 2* PIT checklist is a precondition here: values must be time-valid, the historical universe must be reconstructible, and identifier mappings must be valid at the decision date. The remainder of this section shows how to satisfy those requirements for large-scale fundamental sourcing.

Timestamp authority by modality

The authoritative availability timestamp varies by data source. The conventions below apply uniformly across pipelines:

Modality	Authoritative Timestamp	Notes
SEC	accepted_at (filing acceptance)	Optionally track earlier earnings releases if event-time matters
Macro	Official release timestamp (timezone-normalized)	Use ALFRED vintage data for historical accuracy
Commodities	Agency release timestamp + contract roll mapping	EIA, USDA, and CFTC each have specific release schedules
Crypto	Block time + confirmation/finality policy + vendor ingestion time	Treat observation as available only after N confirmations to avoid reorg lookahead

Table 4.1: Timestamp authority by source

The EDGAR sourcing pipeline

The SEC’s **Electronic Data Gathering, Analysis, and Retrieval (EDGAR)** system provides free, comprehensive access to U.S. corporate filings. EDGAR’s phase-in began in

1993 and became mandatory for domestic issuers by May 1996. The system now offers multiple access channels:

Bulk Downloads : The Financial Statement and Notes (FSN) datasets provide parsed XBRL data in tabular format. These cover all 10-K, 10-Q, and 8-K filings back to 2009, organized into eight related tables:

File	Description
SUB	Submission metadata—CIK, form type, filing date, accession number
NUM	Numeric values extracted from XBRL tags
TAG	Taxonomy tag definitions and descriptions
DIM	Dimensional breakdowns (by segment, geography, and so on)

Table 4.2: Selected FSN tables

The FSN dataset includes four additional files (TXT, PRE, CAL, REN) for text content and display formatting.

REST APIs : Real-time access to company filings, specific concepts, and submission histories. No authentication required—callers identify themselves via the user-agent header. The company-concept API returns all historical values for a specific metric (for example, `EarningsPerShareDiluted`) in JSON format.

Python Libraries : Packages such as `edgartools` , `sec-edgar` , and `sec-downloader` wrap the API complexity with convenient interfaces. `edgartools` provides direct access to parsed financial statements and text sections. For high-throughput applications, services like `datamule` offer bulk downloads via hosted archives, often with higher throughput than direct EDGAR access (check current rate limits and terms).

For systematic research, the bulk approach is often most efficient: download the compressed FSN archives, extract to columnar Parquet format for fast queries, then build the bitemporal database from the parsed content.

XBRL and taxonomy structure

The **eXtensible Business Reporting Language** (**XBRL**) standardizes electronic business reporting. Each numeric value in a filing is tagged with:

- **Concept** : What the number represents (for example, `us-gaap:RevenueFromContractWithCustomerExcludingAssessedTax`)
- **Period** : The time span covered (point-in-time for balance sheet items, duration for income statement)

- **Unit** : Measurement unit (USD, shares, USD/share)
- **Dimensions** : Optional breakdowns by segment, geography, or other categories

The U.S. GAAP taxonomy defines thousands of standardized concepts, but companies can extend it with custom tags. This flexibility enables precise reporting but complicates cross-company analysis—the same economic fact might appear under different tags.

For PIT reconstruction, the filing date (`knowledge_date`) is derived from the submission metadata rather than the financial data itself. For each filing, the valid period is derived from the XBRL content and paired with the filing date from the submission record. This separation makes PIT reconstruction possible.

Implementation :

- `@1_academic_characteristics` introduces the academic-fundamentals panel and the long-run cross-section that motivates careful PIT construction.
- `@2_sec_filing_explorer` introduces EdgarTools for interactive EDGAR exploration.
- `@3_sec_form4_insider_transactions` demonstrates how to retrieve and parse Form 4 insider-transaction filings as an example of event-timestamped fundamental data.
- `@4_sec_xbrl_fundamentals` consumes the materialized XBRL Frames output and demonstrates bitemporal query patterns.

Handling data inconsistency

Even with proper PIT tracking, fundamental data presents consistency challenges:

- **Accounting standard evolution** : U.S. GAAP changes over time. A tag like `RevenueFromContractWithCustomerExcludingAssessedTax` might replace `SalesRevenueNet` mid-series, requiring mapping between taxonomy versions.
- **Company-specific choices** : Within GAAP, companies have discretion in categorization. One firm’s “Cost of Revenue” might include items another firm reports separately. Cross-company comparisons require careful normalization.
- **Restatements and amendments** : When companies file amended filings (10-K/A, 10-Q/A) or restate prior periods, multiple versions of the same reporting period coexist. A bitemporal model stores each version with its own availability timestamp; analysis code must be explicit about which version is eligible at each decision date.

- **Corporate actions** : Stock splits require retroactive adjustment of per-share metrics. A 3:1 split means historical EPS must be divided by 3 for comparability. Mergers and spin-offs create discontinuities that simple adjustment factors cannot resolve.

Leakage becomes visible when the same simple strategy is run twice: once using an as-of eligible (“as-known”) fundamentals view and once using a naïve “latest available today” view. The second backtest often appears better—not because the signal is stronger, but because the dataset has silently incorporated information that arrived later.

In practice, even small amounts of leakage can swamp the true signal-to-noise ratio in fundamentals-based strategies. The lesson is methodological rather than a fixed percentage: if a vendor cannot explain how it handles revisions, amendments, and identifier history, presume the dataset is *not* PIT-correct until proven otherwise.

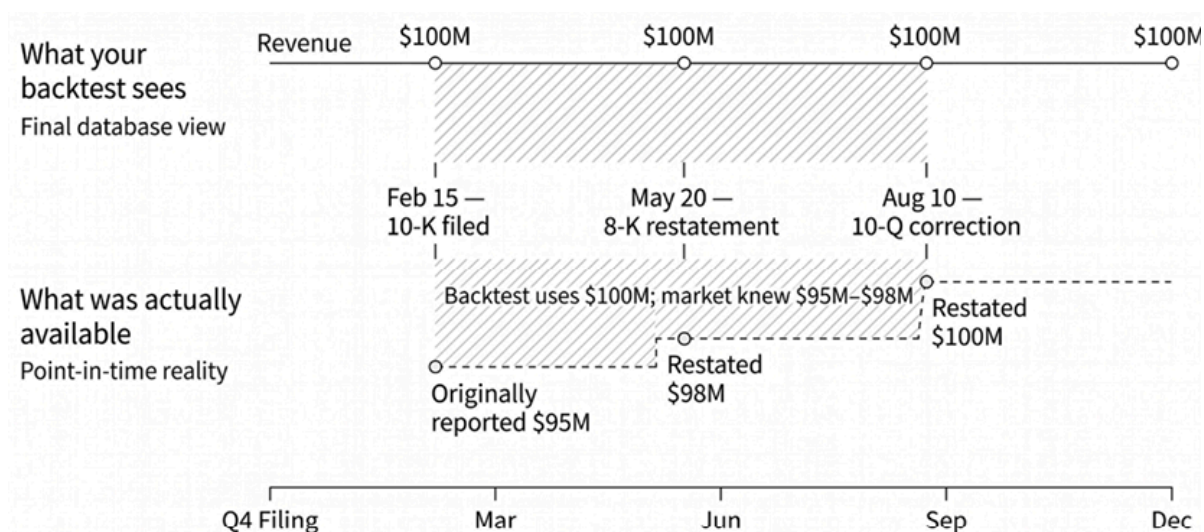


Figure 4.1: Lookahead bias from restated fundamentals

Figure 4.1 shows a naïve “latest value” fundamentals dataset that assigns the restated Q3 revenue (\$100M) to dates before its public release. A point-in-time dataset keeps the original value (\$95M) until the restatement date, preventing leakage at the strategy decision date. The next section discusses how to address entity resolution.

4.2 Entity resolution and mapping

Before we can combine any data from disparate sources, we must address the universal problem of **entity resolution**. Different data providers, regulatory systems, and internal databases all identify the same company differently. Ekster and Kolm (2020) identify entity mapping as one of the primary technical challenges in alternative data integration. Without correct mapping, any multi-source analysis is compromised from the start.

Why resolution is critical

Consider a strategy that combines SEC fundamental data with alternative data on government contracts. The SEC identifies companies by Central Index Key (CIK). Federal award data commonly includes the Unique Entity Identifier (UEI) (which replaced DUNS in April 2022), as well as legal names and address fields. Credit card transaction data might use proprietary merchant codes. None of these maps directly to stock tickers without crosswalks and a clear definition of the target layer (issuer, subsidiary, security).

A single false-positive match can poison every downstream join; prioritize precision and escalate uncertain matches. If “ZOOM VIDEO COMMUNICATIONS” (ticker ZM) is confused with “ZOOM TECHNOLOGIES INC” (formerly ticker ZOOM, now ZTNO), fundamental data from one company gets paired with alternative signals from another. The resulting “alpha” is noise.

The problem multiplies with corporate complexity. IBM’s government contracts might be filed under “INTERNATIONAL BUSINESS MACHINES CORPORATION,” “IBM CORP,” “IBM GLOBAL SERVICES,” or dozens of subsidiary names. These must all map to ticker IBM, but contracts from spun-off, now-independent units like Kyndryl should not.

A hierarchical resolution approach

Practical entity resolution proceeds in stages, from deterministic to probabilistic to model-based.

Stage 1: Deterministic matching

When a data source provides **standard identifiers**, start with deterministic joins. The main pitfall is resolving to the wrong *layer*, because “company identifiers” can refer to different things. More specifically, identifiers can point to:

- A **legal entity** (the corporate parent or filing entity)
- A **security** (a specific tradable instrument, such as a share class, bond, or ADR)
- A **derivatives contract** (a specific listed contract/expiration on an exchange)

Production systems typically maintain three linked master tables— **entities**, **securities**, and (when relevant) **contracts** —with **time-valid mappings** between them.

For instance, “Apple” can legitimately resolve to different objects depending on the question:

- A legal entity (via a **Legal Entity Identifier** or **Central Index Key**)

- A security (for example, the common stock versus a specific bond issuance, each with its own identifier)
- An exchange-listed contract (for futures/options)

So “identifier match succeeded” is not the same as “resolved correctly” unless the correct layer (entity, security, or contract) has been selected.

Common standardized identifiers (and what they identify):

- **Legal Entity Identifier (LEI)** : A 20-character identifier for *legal entities* used in financial transactions (ISO 17442).
- **Central Index Key (CIK)** : The SEC identifier for *filing entities* in EDGAR; mapping to traded instruments typically requires additional reference tables (for example, subsidiaries versus holding companies).
- **Financial Instrument Global Identifier (FIGI)** : An *instrument-level* identifier (security-level, not entity-level); OpenFIGI provides lookup and mapping APIs.
- **CUSIP/ISIN/SEDOL** : *Security-level* identifiers used in trading and settlement (CUSIP: U.S.; ISIN: global; SEDOL: U.K.).

When a source includes any of these, resolution is usually a straightforward join against the relevant crosswalk. The harder cases come from sources that omit institutional identifiers—or sources that include one but only at a different layer than the target.

A wrong-join example

Alternative data often arrives at the *issuer* level (for example, web traffic for “Amazon.com Inc”), while returns are measured at the *security* level (AMZN common stock). Without an effective-date security master, this join can fail silently: (1) joining issuer-level data to the wrong share class (Class A versus Class B with different voting rights and prices); (2) joining to an ADR instead of the ordinary share; (3) using a ticker that belonged to a different entity before a corporate action. The fix: maintain explicit issuer→security mappings with effective dates, and constrain all joins by `effective_date <= decision_time < end_date`.

Stage 2: Probabilistic matching

When identifiers are absent, text matching becomes necessary. String similarity algorithms handle variations in company names:

- **Levenshtein Distance** : Counts character edits (insertions, deletions, substitutions) to transform one string into another. “TESLA INC” and “TESLA, INC.” are one edit

apart.

- **Jaro-Winkler Similarity** : Weighted similarity favors matching prefixes. Handles abbreviations well—“INTERNATIONAL BUSINESS MACHINES” scores high against “INTL BUSINESS MACH”.
- **Token-based methods** : Jaccard similarity on word tokens. “APPLE INC” and “APPLE INCORPORATED” share the important token despite different suffixes.

Libraries like `rapidfuzz` implement these algorithms efficiently, enabling fuzzy matching against reference lists with configurable similarity thresholds. A typical pattern extracts the best matches above a confidence threshold (often 85-90%), and flags lower-confidence matches for manual review.

Probabilistic matching benefits from human-in-the-loop QA because the failure mode is asymmetric: a missed match costs coverage, but a false match can corrupt every downstream join and backtest. The right objective is usually high precision first, then improved recall via escalation stages and better candidate generation.

The complicated cases tend to fall into repeatable buckets:

- Subsidiaries whose legal names are unrelated to the parent brand
- DBAs and trade names
- Joint ventures and consortia
- Generic names that collide across unrelated entities

For example, “Amazon Web Services” may not fuzzy-match to “Amazon.com Inc” despite the clear relationship, requiring either subsidiary lookup tables or embedding-based semantic matching. Name-only fuzzy matching is rarely sufficient for these cases.

For threshold selection, a tiered scheme is typical: auto-accept only at very high confidence (practitioner thresholds often sit around 95%); route a middle band (commonly 85-95%) to manual review; escalate the tail (below roughly 85%) to richer features such as addresses, websites, or descriptions. Calibrate these cutoffs against the empirical score distribution in a labeled sample rather than treating the numbers as universal constants. Track match quality over time—if post-hoc review reveals incorrect matches, adjust thresholds or add disambiguation features.

Stage 3: Manual review and QA

Records in the middle-confidence band—high enough that deterministic and probabilistic matching produced a credible candidate, but not high enough for auto-acceptance—escalate to a human-review queue. Effective workflows separate the queue by failure mode (ambiguous fuzzy match, missing identifier, candidate-set tie) so reviewers can apply the

right disambiguation procedure rather than triaging case by case. Each decision is written back to the master entity table as a new alias or a rejection, with the reviewer, timestamp, and rationale. Hence, the audit trail supports both downstream debugging and periodic threshold recalibration.

Embedding-based semantic matching, useful when fuzzy and identifier matching both fail to disambiguate (for example, “Amazon Web Services” versus “Amazon.com Inc”), is covered in *Chapter 10* (text embeddings) and *Chapter 23* (knowledge graphs), where Chen et al. (2023) describe FinKG. This knowledge graph integrates SEC filings and market data for entity linking across U.S. companies.

Building a master security database

Production systems maintain a master entity table that serves as the canonical reference:

id	name	ticker	cik	lei	figi
1001	Tesla, Inc.	TSLA	1318605	54930043XZGB27CTOV49	BBG000N9MNX3
1002	Apple Inc.	AAPL	320193	HWUPKR0MPOU8FGXBT394	BBG000B9XRY4

Table 4.3: Master entity example

All incoming data—regardless of source identifier—resolves to this master `entity_id` before joining with other datasets. The master table accumulates name variants and identifier mappings over time, improving resolution accuracy as more data is processed.

Temporal entity mapping

Static mapping is a hidden source of lookahead and survivorship bias. Entity relationships are dynamic: tickers are reused (ZOOM belonged to Zoom Technologies before Zoom Video took the ticker ZM), and identities evolve through rebranding (FB → META) or mergers. A production system must be temporally aware, tracking when each identifier was valid:

id	identifier_type	identifier_value	effective_date	end_date
1095	ticker	FB	2012-05-18	2022-06-08
1095	ticker	META	2022-06-09	NULL

Table 4.4: Temporal mapping example

Without `effective_date` constraints, a backtest might join 2015 alternative data to 2024 ticker symbols—creating a dataset that could never have existed. Every cross-source join must be constrained by observation timestamp to ensure the mapping was valid at that historical moment.

Mapping quality assurance

Production entity resolution requires ongoing validation:

Check	Method	Frequency
Confidence Tiers	Segment mappings by confidence using calibrated cutoffs (commonly around 95% auto-accept, 85-95% review, below 85% use another method)	Per batch
Sampling Audit	Manually verify 1–5% of auto-accepted mappings for false positives	Monthly
Drift Monitoring	Track match rate and confidence distribution over time; investigate sudden changes	Weekly

Table 4.5: Quality assurance example

A declining match rate often signals changes in the data source (for example, new naming conventions or added subsidiaries). Rising false-positive rates indicate threshold drift or coverage gaps in the reference data.

Key QA metrics to track:

- **Match rate by tier** : Percentage of records resolved at each stage (deterministic, fuzzy, embedding, or unresolved)
- **Confidence score distribution** : Histogram of match scores; watch for bimodal patterns indicating threshold issues
- **False positive estimate** : From sampling audit—fraction of auto-accepted matches that are incorrect
- **Temporal drift** : Week-over-week change in match rate and mean confidence; sudden shifts warrant investigation

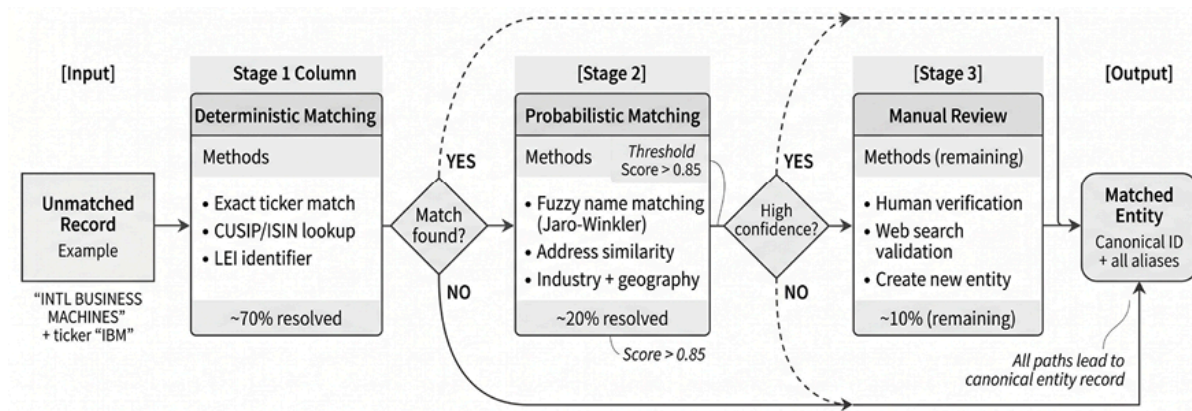


Figure 4.2: Hierarchical entity resolution pipeline

Figure 4.2 shows the entity-resolution pipeline. Deterministic identifier matching resolves the bulk of records; probabilistic record linkage absorbs name variants above a confidence threshold; the residual middle-confidence band escalates to manual review, with all decisions written back to a master entity table under time-valid mappings. The next section surveys the fundamental data landscape across asset classes.

Implementation : `05_entity_resolution` demonstrates deterministic joins, fuzzy string matching (with normalization), and a simple master database pattern in an end-to-end pipeline.

4.3 Fundamentals across the asset-class spectrum

Sections 4.1–4.2 established the PIT contract and entity mapping for equities. This section generalizes those concepts: every asset class has revision-prone fundamentals, but the *timestamp authority* and *instrument mapping* differ across asset classes. Getting these details wrong introduces the same lookahead bias—the failure mode is universal, even if the mechanics vary.

The idea of “fundamentals” extends well beyond corporate equities, but the meaning changes by asset class. Each domain has distinct economic drivers, publication mechanics, and revision behavior, so the engineering problem is not “collect more data”—it is building time-consistent inputs that respect release calendars, vintages, and instrument conventions.

Macro and sovereign data (FX/Rates)

Rates and FX are driven by macro fundamentals such as growth, inflation, labor conditions, monetary policy, and fiscal sustainability. The engineering challenge is that these series

come from multiple agencies and are published on *release schedules* with **lags** and **revisions**, so what was known at each decision time must be modeled explicitly.

Key sources

Federal Reserve Economic Data (FRED) aggregates hundreds of thousands of time series from many sources and provides APIs for retrieval and vintage-aware workflows. For non-U.S. macro, primary sources include Eurostat, IMF/World Bank datasets, national statistics agencies, and central banks. (See *Chapter 2* for a complete data source inventory; this section focuses on timing and revisions.) FRED data show the U.S. 10y-2y yield curve has been inverted on 15.5% of days since 2000, including 36% of days since 2020—a regime-shift anchor for macroeconomic state variables.

Alignment challenges

Macro data presents several engineering challenges:

- **Release cadence variation** : Many key macro series are measured over a period (month/quarter) but published later on scheduled dates. GDP is published quarterly, with multiple monthly vintages (advance/second/third estimates), while claims are weekly and CPI is monthly. Aligning these to a daily decision grid requires a clear policy: treat releases as **event data** (timestamped) and carry forward only after they become available, or treat macro as a slowly evolving **state** updated at release times. Avoid interpolation methods that blend future values into the past.
- **Release time matters** : For daily or intraday models, the release *timestamp* (date, time, and time zone) determines eligibility; “same day” does not mean “pre-trade.” Many U.S. releases are scheduled at 8:30 a.m. ET, but schedules can shift due to holidays or disruptions, so the pipeline should ingest and store the published release timestamp rather than hard-coding assumptions.
- **Time zones and calendars** : A global macro dataset must normalize timestamps (typically to UTC) and track the local release calendar. “Quarterly” series from different jurisdictions can be released on different local dates/times and revised on different schedules, so PIT correctness requires storing both the *period covered* and the *availability timestamp*.
- **Revision histories** : Many macro series are revised after initial release. Backtesting on today’s “final” series gives the model information that was not available at the time. The correct input for realistic backtests is vintage data (“what was known when”), not the latest revision.

FRED supports **vintage-aware retrieval** through **ALFRED** (Archival FRED): observations have real-time validity windows (in other words, the value that was current between two vintage dates), and the API exposes vintage dates and vintage-specific downloads. The Philadelphia Fed Real-Time Data Set provides the canonical reference for vintage-aware macroeconomic research; Croushore (2008) surveys the methods and pitfalls. In practice, the two production patterns are (i) querying specific vintage dates to reconstruct what a model would have seen historically, or (ii) storing the revision ladder explicitly when the downstream strategy is revision-sensitive.

PIT requirements include storing the release timestamp and calendar; maintaining the vintage/revision ladder; normalizing time zones; and distinguishing the period covered from the availability time.



Implementation : `@6_fred_macro_ed` introduces the macro data; `@7_macro_data_alignment` illustrates how to load native-frequency series, model publication lag, and how vintage-aware inputs differ from “latest series” inputs.

Commodities - Physical market data

Commodity prices are anchored to physical supply-and-demand constraints. However, the data is often **domain-specific** (inventories, shipments, crop conditions, refinery utilization), and the tradable instruments are **contractual** (futures by delivery month). That creates a two-part engineering problem:

1. Ingest scheduled fundamental releases with correct timestamps.
2. Map them to the contract you could actually trade.

Energy and agriculture

U.S. government agencies anchor the calendar for energy and agricultural fundamentals. The U.S. Energy Information Administration (EIA), the U.S. Department of Agriculture’s World Agricultural Supply and Demand Estimates (WASDE) report, and the U.S. Commodity Futures Trading Commission’s (CFTC) Commitments of Traders (COT) report are timestamped events; joins should occur only after their published release times. The EIA’s Weekly Petroleum Status Report follows a standard schedule (Wednesdays at 10:30 a.m. ET, with holiday exceptions); the release timestamp must be part of the dataset.

USDA’s WASDE report is released at 12:00 p.m. ET on scheduled dates and is widely viewed as a risk event for grains and oilseeds. Use the release timestamp; avoid any

workflow that implicitly treats pre-release expectations as published facts.

Meteorological data (for example, from NOAA) often enter as high-frequency state variables rather than as scheduled releases; treat it differently from “lock-up” economic reports.

Mapping to tradable instruments

Physical market data must be mapped **to the instrument you can trade**. When an inventory number is released, the relevant price response may concentrate in the actively traded front-month contract (or the most liquid point on the curve), not in an abstract “continuous series” that blends multiple delivery months.

This is where the *futures roll logic* from *Chapter 3* becomes an input dependency: a consistent mapping from calendar time to the active contract. Fundamental event timestamps should be joined to the active contract at their timestamp, then carried into any continuous-series representation using the same roll rules as prices.

Beyond the physical fundamentals, CFTC’s COT reports provide a public snapshot of trader positioning by category. COT is a PIT dataset: positions are as of Tuesday. They are generally released on Friday at 3:30 p.m. ET (with holiday delays), so backtests must treat the report as **available only after the release time**. Positioning extremes and rapid changes are often used as sentiment-style features (contrarian or trend-confirming); any claim about predictability should be evaluated out-of-sample with the release lag enforced.

PIT requirements for this include tracking scheduled release timestamps; mapping fundamental events to the correct contract and curve point at event time; applying roll rules consistent with prices; and enforcing reporting lags.



Implementation : `08_futures_positioning` shows COT retrieval, PIT availability timestamps, and positioning z-scores.

Cryptocurrency and digital assets - On-chain fundamentals

Crypto markets introduce a new category of fundamentals: on-chain data is publicly observable and programmatically verifiable, often at high frequency—the engineering challenge shifts from access to interpretation. Different chains differ in how quickly transactions become final, how vulnerable recent blocks are to reorganization, which token standards they use, and how protocol events are represented in the data. As a result, even the same metric may need to be defined differently across ecosystems. Harvey et al. (2022)

provide a practitioner-oriented investor guide that emphasizes that crypto is much broader than just spot bitcoin.

Core on-chain metrics

Core on-chain metrics include:

- **Network activity** : Active addresses, transaction counts, and transfer volumes provide a first view of protocol usage. These measures can signal adoption and engagement, but they require careful interpretation because bots, internal transfers, or address-reuse patterns may inflate activity.
- **Security metrics** : In proof-of-work networks, hash rate measures the total computational power securing the chain; in proof-of-stake networks, staked value measures the capital validators have committed and put at risk. In both cases, these are rough proxies for the economic cost of attacking the network, though their interpretation depends on the protocol design and market structure.
- **Economics** : Token supply schedules, inflation mechanics, issuance rules, and fee dynamics shape incentives for users, validators, and investors. These variables also influence valuation narratives by affecting scarcity, dilution, and the distribution of economic rewards. Separately, futures positioning data, including COT-style classifications, are often used to study who trades crypto futures and whether specific trader categories exhibit timing ability (Baur and Smales, 2022).
- **DeFi metrics** : Total value locked, or TVL, is widely used as a proxy for the capital committed to a protocol. It can be informative, but should be interpreted cautiously because it may reflect double-counting, leverage, or token price swings rather than net new economic activity. Protocol fees and revenue offer additional measures of usage and value capture, but definitions vary across aggregators—record the exact definition used.

Data sources

When it comes to cryptocurrency data sources, direct access (running nodes) provides maximum control but entails high operational costs. Indexing protocols and analytics platforms (for example, The Graph, Dune) make decoded blockchain data queryable without requiring full infrastructure maintenance.

Commercial aggregators provide curated metrics via API, but they should be treated like any vendor feed: document transformations, definitions, and backfills. Alexander and Dakos (2020) examine which cryptocurrency data sources scholars should use, finding that major

aggregators provide statistically equivalent data for liquid assets but can diverge for illiquid tokens.

AMM and DEX data

Decentralized exchanges using AMMs generate transparent on-chain records of liquidity provision, swaps, and fees. Lehar and Parlour's (2021) empirical work on Uniswap illustrates how this transparency enables measurement of market structure that is hard to obtain in traditional venues.

PIT requirements include understanding finality and reorg policies; documenting metric definitions and provider transforms; tracking backfills and recomputations; and accounting for differences in chain and token standards. The next section explores alternative data and discusses how to work with this increasingly relevant source.



Implementation : `09_onchain_fundamentals` illustrates how to query on-chain metrics and align them with traditional market data.

4.4 Understanding alternative data

Alternative data can be a genuine edge, but it is also a high-variance investment: noisy signals, unstable coverage, hidden backfills, and real legal and reputational risk. A disciplined evaluation process is what separates interesting datasets from production-worthy inputs.

In *Chapter 2*, we covered data quality issues such as PIT correctness, survivorship bias, and identifier integrity, which apply to *all* financial data. This section focuses on evaluation criteria for alternative data acquisition decisions—specifically, whether to invest in a new dataset.

The alternative data landscape

Alternative data encompasses information beyond traditional market and fundamental sources. Green and Zhang (2024) provide a useful taxonomy:

- **Text and attention data** : News articles, social media, earnings call transcripts, and search activity. Classic evidence shows that media tone can predict short-horizon market behavior, though effects are sensitive to leakage and model specification (Tetlock, 2007). The *engineering* of text data (extraction, cleaning, PIT storage) is covered in *Section 4.5*; the *featurization* of text (sentiment, embeddings, transformers) is covered in *Chapter 10*.

- **Business process data** : Credit card transactions, point-of-sale records, job postings, and hiring patterns. LinkedIn entry/exit data and Glassdoor ratings can predict operational health, though coverage is uneven and firm-reported numbers in 10-Ks are often too lagged to compete (Green and Zhang, 2024).
- **Sensor and geospatial data** : Satellite imagery of parking lots or construction sites, geolocation tracking, shipping vessel positions, and industrial sensor readings.
- **Prediction markets** : Platforms trading binary event contracts whose prices produce continuously updated probability proxies. Ng et al. (2025) document meaningful price discovery in modern prediction markets, finding that higher-liquidity venues incorporate information more quickly. Limitations include thin liquidity outside headline contracts and regulatory fragmentation.

On the demand side, evidence suggests sell-side analysts increasingly reference non-traditional datasets in their research workflows, and that this adoption is associated with improved forecast accuracy (Chi, Hwang, and Zheng, 2024).

The evaluation framework

Alternative datasets usually fail for predictable reasons: they are not incremental once existing features are controlled for, they cannot be reconstructed at decision time, or they introduce legal and operational risks that overwhelm their expected value. A practical due diligence focuses on signal content, data quality, legal risks, and commercial/engineering costs with an explicit bias toward stopping early on hard constraints rather than debating marginal score differences.

Signal content evaluation asks whether the dataset can plausibly deliver incremental, tradable information:

- **Uniqueness** : Specify the baseline feature set and test incremental value rather than raw correlations (for example, baseline-versus-baseline and dataset-versus-dataset-only). Improvements that vanish under reasonable controls are typically redundant proxies.
- **Decay and durability** : Treat half-life as a design constraint. McLean and Pontiff (2016) show that published stock return predictors lose ~58% of their performance post-publication—a useful baseline for how quickly signals erode. Whether a specific alternative dataset “accelerates diffusion” is an empirical question to test under realistic timestamps and costs (Hong et al., 2000).
- **Common failure modes** : (i) Real signal content but not tradable after costs/capacity, (ii) apparent performance created by revisions or look-ahead.

Data quality checks verify that we can reproduce results under production constraints, especially those related to point-in-time availability.

- **Quality and stability** : Verify versioning, revision exposure, and detectability of changes to the methodology. Backfills and vendor process changes can affect performance; ESG ratings illustrate how methodological differences drive disagreement, making “stability of measurement” central (Berg et al., 2022).
- **Coverage** : Evaluate representativeness across the tradable universe (names, regions, sectors) and across regimes; diagnose systematic bias or missingness (for example, low coverage of small-cap or non-U.S.).
- **Latency** : Align publication lag, cadence, and timestamp conventions with the decision process. If “first observable time” cannot be stated precisely, point-in-time backtests are not defensible.

Legal risks are a hard-fail gate: risk cannot be “optimized away” by model performance.

- **Compliance and rights** : Establish provenance, permitted uses, contractual restrictions, audit/deletion obligations, and downstream redistribution limits.
- **Red flags** : (i) Material Nonpublic Information (MNPI) by narrowness (small panels, constrained geographies, entity-specific “channel checks”), (ii) privacy risk through linkage even without explicit identifiers (device/ad IDs, granular location/event data) with GDPR/CCPA exposure.

Commercial and engineering gates ensure the dataset fits deployable capacity and does not impose disproportionate operational drag.

- **Technical friction and economics** : Evaluate total cost of ownership (integration, entity resolution, storage, monitoring, schema-change handling, incident load) and compare expected value to roadmap capacity.
- **Operationalization test** : Estimate one-time engineering lift, ongoing maintenance burden, and a conservative value range after realistic latency, costs, and capacity constraints.
- **Decision rule** : Privilege hard constraints (legal, point-in-time integrity) over marginal differences elsewhere; only compare “scores” among datasets that clear the hard-fail gates.

Build versus buy for alternative data

Chapter 2 discussed general build-versus-buy tradeoffs for financial data. For alternative data specifically, additional factors apply:

- **Alpha crowding** : Built pipelines using proprietary internal data remain differentiated; purchased datasets erode as adoption spreads.
- **Legal risk asymmetry** : Web scraping and data collection operate in legal gray areas; vendors may provide representations, but the buyer still owns compliance and the

interpretation of what use is permitted.

- **Maintenance burden** : Alternative data sources change frequently (API breaks, coverage shifts, and methodology updates); vendors absorb these operational costs.

For most organizations, a hybrid approach works best: buy cleaned, validated data for mainstream alternative sources; build capabilities only for truly proprietary signals where internal data or relationships provide unique access.

Data sources and implementation

The ecosystem spans terminal platforms (Bloomberg, LSEG Workspace), vendors specializing in different modalities, such as payments, web data, or geospatial data, and marketplaces such as Nasdaq Data Link, Snowflake Marketplace, and AWS Data Exchange. Marketplaces can simplify procurement, but do not eliminate the need for PIT validation and methodology audits.

Several sources provide meaningful alternative data for experimentation (the chapter [README](#) lists additional sources):

- **GDELT Project** : Large-scale global news event data for geopolitical risk and event detection; just the 2015 dataset weighs 2.5TB; requires NLP pipelines to extract signals (see next section and *Chapter 10*).
- **SEC 13F filings** : Institutional holdings disclosed quarterly; useful for “smart money” positioning and crowded trade detection.
- **On-chain DeFi metrics** : Total Value Locked (TVL) and protocol revenue as crypto-native fundamentals.
- **Prediction markets** : Kalshi (CFTC-regulated) and Polymarket (crypto-based) provide time series of event probabilities.

Text data , including corporate filings, news, and earnings calls, represents the largest category of accessible alternative data—but requires substantial engineering before it becomes model-ready. The next section covers the extraction and storage pipeline, and *Chapter 10* covers featurization (sentiment dictionaries, embeddings, and fine-tuned transformers such as FinBERT).

Implementation :

- `10_institutional_holdings_13f` builds the 2024Q3 manager-stock graph from SEC 13F filings, computes manager-level concentration metrics and co-ownership edges, and surfaces the most widely held issuers.



- `11_defi_tvl_evaluation` walks the DeFi TVL panel (4 chains, 2017-2026) and tests TVL growth as a forward-return predictor (peak IC = 0.12; weak signal).
- `12_kalshi_prediction_markets` extracts event-prior probabilities from Kalshi markets and demonstrates the API pattern for prediction-market consumption.
- `13_polymarket_prediction_markets` covers the Polymarket equivalent, with notes on liquidity and contract-design differences.

4.5 Using text data for NLP features

SEC filings are *fundamental* data by source, but the unstructured text they contain, such as Management Discussion and Analysis, Risk Factors, and earnings call transcripts, is, by nature, *alternative* data: it requires NLP pipelines to extract signals that cannot be reduced to accounting line items. This section covers the engineering work that makes text usable: source selection, section extraction, cleaning, and PIT-correct storage.

Targeting MD&A and risk factors

SEC 10-K annual reports contain standardized narrative sections that provide rich signals about a company's outlook:

- **Management's Discussion and Analysis (MD&A, Item 7)** : Narrative discussion intended to provide material information for assessing financial condition, results of operations, liquidity, and cash flows (Regulation S-K Item 303).
- **Risk Factors (Item 1A)** : Required disclosure of *material* factors that make an investment speculative or risky, organized under relevant headings; the SEC discourages generic risk factors (Regulation S-K Item 105). Often lengthy and standardized, but *changes* in structure, emphasis, or newly introduced themes are frequently more informative than the level of boilerplate.

These sections contain information that cannot be reduced to numbers: qualitative context, forward-looking language, and explanations of changes that may not appear in line items. Tetlock (2014) surveys how textual information transmits to financial markets, establishing the theoretical foundation for extracting signals from corporate narratives. Modern NLP approaches (including finance-adapted BERT variants such as FinBERT) can extract more context-sensitive signals than dictionary methods, but only if extraction and timestamps are correct.

The engineering pipeline

Building a filing-based text dataset requires four engineering steps:

1. **Document selection** : Pick the primary filing document, not an exhibit.
2. **Section extraction** : Identify item boundaries robustly.
3. **Cleaning and normalization** : Remove markup and non-prose artifacts while preserving paragraph structure.
4. **Point-in-time storage** : Store availability timestamps and audit keys.

Step 1: Filing access and metadata

EDGAR provides filing packages plus submission metadata. Libraries such as `edgartools` can simplify retrieval, but the pipeline must still capture the filing's stable identifier (accession number), form type (10-K/10-Q, amendments), period end, and the **SEC acceptance timestamp** (when the filing became public). That metadata becomes part of the PIT contract for downstream NLP features. The accession number is the natural primary key because multiple filings can share the same date, and amendments require separate tracking.

Step 2: Section extraction

Extraction via document parsers should be layered (structured access when available; HTML-anchored boundaries otherwise) and instrumented with measurable quality checks (missing sections, collisions, length outliers). Item numbers label sections, but the same filing can contain multiple “Item 7” mentions (for example, in the Table of Contents and cross-references), and formatting varies across filers and over time. Amendments (10-K/A) and older filings can break naive boundary logic—treat extraction quality as a measurable output and sample failures for iterative rule refinement.

Step 3: Cleaning and normalization

Cleaning should remove markup and artifacts of repeated filing without destroying linguistic structure. Common issues include:

- **HTML remnants** : Unclosed tags, entity codes (` `, `&`), style attributes.
- **Table structures** : Financial tables embedded in narrative sections. May need separate handling or removal.
- **Boilerplate** : Safe-harbor statements, standard risk language, repeated cross-references. In many corpora, boilerplate accounts for a large fraction of the extracted text. The right approach is to *measure* it (for example, by comparing year-over-year text overlap)

and decide whether to remove it (for novelty/change features) or keep it (for level-based sentiment features).

Cleaning should preserve paragraph boundaries (important for downstream chunking and change detection) while removing non-prose artifacts and normalizing whitespace.

Step 4: PIT-correct storage

Store each extracted section as a row keyed by a stable filing identifier (accession number) and include both the reporting period and the availability timestamp. The minimal schema includes:

- **Identifiers** : `cik` (company), `accession` (filing), and `section` (Item 7, Item 1A, and so on) together form the primary key—this triple uniquely identifies each extracted section, including amendments.
- **Timestamps** : `filing_date` and `accepted_at` (the SEC acceptance timestamp, to the second) determine *availability* —when this text became public. `period_end` indicates the fiscal period covered.
- **Content** : The `text` column stores the extracted section; optionally, store both raw and cleaned versions.
- **Metadata** : Form type (10-K, 10-Q, amendments), extraction method, pattern version, and character offsets enable auditing and re-extraction when rules change.

Following the bitemporal data model introduced above:

- `filing_date` / `accepted_at` (knowledge time) determines *availability* —when this text became public. Use `accepted_at` for intraday PIT correctness.
- `period_end` (valid time) indicates the reporting period covered.

Store both **the raw extracted text** and **the cleaned text**, along with extraction metadata (method, pattern version, offsets), so the extraction can be audited and rerun when rules change. Persist to Parquet for efficient filtering by section and time.

Output artifact

The pipeline produces a **text corpus** with the schema described above. Key fields:

- `cik + accession + section` uniquely identify an extracted section version (including amendments).
- `filing_date` / `accepted_at` enable PIT-correct feature construction.
- `section` enables targeted modeling (MD&A versus Risk Factors versus Business).

Text is most useful when aligned with numeric outcomes. The engineering requirement is that joins must be PIT-safe and audit-safe. The accession number is the natural primary key; keeping both `accepted_at` and `period_end` allows the same filing to be queried as of any historical date without leaking later revisions.

This corpus is the input for *Chapter 10*, which covers the full NLP feature engineering pipeline—dictionary-based sentiment, static word embeddings, and fine-tuned transformer models—and demonstrates a ‘news surprise’ alpha factor that measures semantic deviation of today’s news from recent coverage, depending directly on the PIT-correct text storage established here.



Implementation : `14_text_data_extraction` shows the complete pipeline from EDGAR filings to a structured Parquet dataset. It demonstrates the year-over-year change-analysis primitive on Apple’s 10-K: the 2025 filing’s Risk Factors section runs 9,792 words with 92.9% word overlap relative to the prior year (Jaccard 0.31), surfacing 28 new risk-factor terms.

4.6 Summary

We have laid the data engineering foundations for working with fundamental and alternative data. The central theme is point-in-time correctness: every data source—whether SEC filings, macroeconomic releases, futures positioning, or blockchain metrics—must be timestamped with both the date it became available and the period it covers. The bitemporal model introduced here applies uniformly across asset classes and data types, enabling defensible backtests that avoid look-ahead bias.

The companion notebooks demonstrate these principles in practice: extracting XBRL fundamentals with proper vintage handling, aligning macro data by release schedule rather than reference period, building text corpora from SEC filings with robust section extraction, and evaluating alternative data sources through a systematic framework.

These PIT-correct artifacts become the inputs to *Chapter 8*’s feature engineering workflow. The EDGAR text corpus from *Section 4.5* feeds *Chapter 10*, where we build NLP-based features (sentiment, topics, and embeddings) on the storage pipeline established here.

Chapter 5 turns to synthetic financial data: generative models that augment scarce histories and stress-test backtests against unobserved scenarios.

5

Synthetic Financial Data

Every quantitative strategy depends on historical data, but history provides only one realized path through a large space of market outcomes. A backtest on a 2010–2025 sample can indicate a stable relationship, but it can also look strong simply because the realized path was favorable to the strategy’s assumptions and parameter choices. One response is to augment the single realized history with additional, statistically consistent market paths sampled from a fitted generative model.

Those sampled paths are **synthetic data**, drawn from a model fitted to the empirical joint distribution of observed data, designed to preserve marginal behavior and dependence structure across time, assets, and regimes. Related terms include **simulated data** (typically parametric Monte Carlo under an assumed process) and **generated scenarios** (samples conditioned on a specified regime or state). All three supplement a single realized history with additional samples to assess robustness.

The objective is *robustness assessment*, not prediction. If generated paths reproduce key characteristics of financial time series, then strategy performance can be evaluated as a distribution under the learned model rather than as a single point estimate. Synthetic data can also support *model development* by augmenting real training samples. In these settings, the performance must still be assessed on real out-of-sample data, and generators must be treated as potential sources of bias.

Throughout this chapter, synthetic data is treated as a *decision-support tool* rather than as a substitute for historical evidence. Its value depends on whether it improves a downstream task. In finance, this standard is demanding: general distributional similarity is not enough if the generated data fails to preserve tail behavior, dependence, volatility clustering, or regime dynamics. By the end of this chapter, you will be able to:

- Diagnose when a historical backtest is underpowered and when synthetic generation is a reasonable supplement.
- Select a generative architecture based on data type and sampling frequency.
- Apply stylized-fact diagnostics to generated returns.
- Evaluate synthetic data using **Fidelity–Utility–Privacy criteria** and Train-on-Synthetic-Test-on-Real validation.
- Identify generator-specific risks, including bias amplification, generator overfitting, and missing novel scenarios.

The chapter first motivates the use of synthetic financial data, describes the stylized facts it needs to reproduce, and explains a practical evaluation framework. We then turn to classical simulation methods, before proceeding to learned generators, including GANs, diffusion models, and LLM-based generators for tabular financial data. The chapter closes by applying the evaluation framework across these methods.

5.1 The quant's dilemma

A quantitative strategy is developed and validated on a single realized history. This is true for discretionary rule-based systems and for machine-learning pipelines that fit predictive models and then backtest the implied trading rules. The historical record contains only a limited number of crises, regime shifts, and correlation breakdowns, so inference can be

fragile because evidence is **path-limited** : apparent performance may be dominated by episodes that need not repeat.

The multiple testing challenge

This path limitation manifests as inflated performance when strategy research adapts as results emerge: strategy development is an iterative search over many coupled choices, including the universe and filters; signal and feature definitions; label horizons and sampling schemes; model class and regularization; cross-validation design; hyperparameters; risk controls; execution assumptions; and portfolio construction details. Each additional decision expands the search space and increases the chance of selecting an in-sample winner that reflects noise, data quirks, or favorable market episodes rather than a persistent edge.

Adaptive research creates a selection problem. As the number of tested variants grows, selection inflation becomes material: the best observed in-sample performance improves mechanically with the number of trials, even when true performance is zero. Bailey et al. (2015) formalize this risk and argue that the probability of backtest overfitting can exceed 50%: under independence and normal-error approximations, after testing 10 configurations, the expected maximum in-sample Sharpe, a measure of risk-adjusted performance, is 1.57, even when all configurations have a true Sharpe of 0; with 100 trials, the expected maximum exceeds 2.5.

Interpreted broadly, this is a statement about the end-to-end research process: the reported result is the output of a selection procedure operating on limited data. The implication is not that backtests, or predictive models, are useless, but that **performance estimates should be treated as conditional on the research path taken** . Bailey and Lopez de Prado (2014) propose the **Deflated Sharpe Ratio** , which adjusts performance inference for non-normality and the number of trials

(see *Chapter 17*). We will encounter additional multiple-testing corrections throughout the book. However, selection-aware statistical corrections can only adjust inference to account for this effect; they do not create additional market histories to assess robustness.

Synthetic data as simulation infrastructure

Synthetic generation addresses path-limited evidence by turning a single realized history into a distribution of plausible histories. Cetingoz and Lehalle (2025) describe this as sampling additional trajectories from an estimated distribution to support robustness checks beyond a single realized path. In practice, the workflow fits a generative model to observed data, aiming to capture key **stylized facts of financial time series** , such as:

- **Heavy tails:** Extreme returns occur more frequently than under a Gaussian model.
- **Volatility clustering:** Large absolute returns cluster in time; the autocorrelation of squared or absolute returns decays slowly.
- **Leverage effect:** Negative returns are associated with higher subsequent volatility, especially in equity markets.
- **Weak return autocorrelation:** Linear autocorrelation in returns is typically small at daily horizons.

At a minimum, the resulting model should generate data that capture marginal distributions, temporal dependence, and cross-asset relationships, enabling it to produce alternative trajectories consistent with the learned structure.

However, a synthetic return generator is useful only if it reproduces empirical regularities that matter for downstream decisions (Takahashi

and Mizuno, 2024). This limitation is especially acute in finance because the features that drive decisions are concentrated in the extremes. Risk management, leverage, and portfolio construction depend on tail outcomes, correlation breakdowns, and regime transitions that are rare and unstable. A method that produces samples that “look realistic” in the bulk of the distribution can still be useless if it understates drawdowns, misses volatility clustering, or fails to reproduce dependence in stress.

In this chapter, we therefore treat the generator as part of the modeling stack and evaluate it primarily by whether it supports downstream decisions, not by whether its samples are visually or marginally similar to historical returns.

Used carefully, synthetic data supports three practical capabilities:

- **Robust parameter selection:** Evaluate candidate parameters across many sampled trajectories rather than relying on a single realized history.
- **Regime stress testing:** Generate additional stress-like trajectories consistent with observed volatility and dependence to probe drawdowns, correlation breakdowns, and risk-control stability.
- **Privacy-preserving development:** When transaction-level data cannot be shared, synthetic data can support development and collaboration, provided it is subject to explicit privacy evaluation (*Section 5.8*).

Synthetic data does not remove selection bias. If strategies are tuned on synthetic trajectories, they can overfit to the generator’s inductive biases and failure modes. The generator must therefore be treated as part of the modeling stack: fix it before strategy selection when possible, validate it on held-out real data, and use an outer validation loop when synthetic data affects model selection.

Before introducing specific generators, we first define the validation framework used throughout the chapter. The key question is not whether generated samples look realistic in isolation, but whether they preserve the structure required for the intended use case while controlling privacy and leakage risk.

5.2 Evaluating synthetic financial data

Synthetic data can appear plausible yet still be unsuitable for its intended use. In finance, this risk is acute because the economically relevant structure often lies in rare events, shifts in dependence, and conditional dynamics rather than at the center of the distribution. A generator that matches marginal returns but understates drawdowns, weakens volatility clustering, or misses stress-period correlations can produce misleading conclusions.

For this reason, the validation protocol often matters more than the generative architecture. Throughout this chapter, we evaluate synthetic data using three criteria: **fidelity**, **utility**, and **privacy**. These objectives typically trade off against one another. Improving privacy can reduce fidelity; improving distributional fidelity does not necessarily improve downstream task performance; and a generator that performs well for one use case may fail for another.

Does it preserve the relevant data structure?

Fidelity asks whether synthetic samples reproduce the empirical structure of the real data at the resolution required by the application. This includes marginal behavior, dependence, and time-series dynamics.

For tabular or cross-sectional data, start with feature-level distributions. Compare real and synthetic samples using statistics such as the Kolmogorov–Smirnov statistic or Wasserstein distance, and complement these with histograms, empirical CDFs, and QQ plots. Summary statistics alone can hide artifacts, especially in the tails.

Dependence must be tested separately. A generator can match each feature's marginal distribution while failing to preserve cross-feature relationships. For multivariate financial data, compare correlation or covariance matrices, rank correlations, sector- or regime-level relationships, and other dependence measures appropriate to the use case.

For financial time series, fidelity also requires stylized-fact diagnostics. At a minimum, test whether the generated series preserves heavy tails, weak raw-return autocorrelation, volatility clustering, leverage effects where relevant, and cross-asset dependence. The autocorrelation function of squared or absolute returns is a useful compact diagnostic because it summarizes volatility persistence, but it should be complemented with tail and dependence checks.

Does it support the intended use case?

Utility evaluates whether synthetic data improves or preserves performance on the downstream task. Fidelity is usually necessary but not sufficient: synthetic data can match broad distributional properties while missing the specific signal needed for forecasting, classification, risk estimation, or strategy evaluation.

A common benchmark is **train-on-synthetic, test-on-real (TSTR)**:

1. Train a model using synthetic data only.
2. Evaluate it on held-out real data.

3. Compare the result with **train-on-real, test-on-real (TRTR)** using the same model class, features, target, and evaluation period.

For error metrics such as MSE or MAE, a useful summary is:

$$TSTR \text{ error ratio} = \frac{Error_{TSTR}}{Error_{TRTR}}$$

Values near 1 indicate that synthetic data preserves task-relevant information. Values above 1 indicate utility loss. Values below 1 can occur when synthetic data suppresses idiosyncratic noise. Still, they require careful interpretation, as they may also signal leakage, target simplification, or an evaluation design that is too narrow.

For score metrics where higher is better, such as AUC or accuracy, report the TSTR and TRTR scores directly, or use a score ratio with the direction clearly stated. Do not mix error ratios and score ratios without explaining the convention.

Utility is always use-case-specific . Tail-GAN should be evaluated on tail-risk metrics such as VaR and Expected Shortfall; Sig-CWGAN should be evaluated on path-wise criteria; diffusion models require temporal and dependence diagnostics; and tabular LLM generators require schema validity, distributional checks, and downstream predictive tests.

Does it leak training information?

Privacy validation tests whether synthetic samples reveal information about the records used to train the generator. This is especially important for transaction-level, customer-level, and proprietary institutional data.

At a minimum, check for exact or near duplicates between synthetic and training records. More informative diagnostics compare nearest-neighbor distances between synthetic records and well-trained and held-out real

records. Membership-inference tests provide a stronger adversarial check by asking whether an attacker can infer whether a specific record was included in the training set.

When formal privacy guarantees are required, train the generator with a **differentially private mechanism**. Methods such as **Differentially Private Stochastic Gradient Descent** (DP-SGD, Abadi et al., 2016) clip per-sample gradients and add calibrated noise during training, with privacy loss summarized by a budget (ϵ, δ) . Smaller values of ϵ imply stronger privacy but usually reduce fidelity and utility. Privacy should therefore be treated as a design constraint, not as an after-the-fact label attached to generated data.

Synthetic-specific failure modes

The fidelity–utility–privacy framework is necessary but incomplete. Synthetic data introduces failure modes that are specific to generated samples.

1. Generators can **amplify biases in the training data**. If the historical sample overrepresents one regime, one sector, or one market state, the generator may reproduce that imbalance more strongly.
2. Strategies can **overfit to the generator**. Selecting a strategy because it performs well across many synthetic paths can produce a solution tailored to the generator’s assumptions rather than to the market. Finalists should always be evaluated on held-out real data that was not used to train the generator.
3. Synthetic data has **limited novelty**. Most generators are better at interpolating within the support of the training distribution than at producing genuinely unprecedented scenarios.

Synthetic data is therefore useful for robustness checks, stress testing, and model development. Still, it should not be treated as evidence about events outside the training regime unless the generator was explicitly designed and validated for that purpose.

Practical validation checklist

At a minimum, every synthetic-data experiment in finance should report:

- A **fidelity diagnostic** , such as marginal distribution error, correlation-matrix distance, tail statistics, or volatility-persistence error
- A **utility benchmark** , such as TSTR versus TRTR or a task-specific risk metric
- A **privacy check** , such as duplicate detection, nearest-neighbor analysis, membership inference, or a formal DP budget
- A **real-data holdout result** , especially when synthetic data is used for model selection or strategy development

Thresholds should be calibrated to the data frequency, sample size, asset class, and downstream objective. There is no universal synthetic-data score. The relevant question is whether the generated data preserves the structure on which the decision depends.

5.3 Classical simulation baselines

Classical simulation methods provide the first test of the evaluation framework. Before using learned generators, practitioners should compare them with transparent baselines that are easier to calibrate, diagnose, and govern.

These baseline models can be grouped into **bootstrap models** , which resample historical data, and **parametric models of stochastic processes**

that generate new data based on model assumptions. Both families are limited, but their limitations are visible, which makes them valuable reference points for evaluating more flexible learned generators.

Bootstrap methods

Bootstrap methods generate new paths by resampling observed returns:

- Their main advantage is fidelity to the empirical marginal distribution: heavy tails and other non-Gaussian features present in the data are preserved by construction.
- The key limitation is equally direct: resampling cannot create fundamentally new events outside the historical record. If no crisis-scale move or correlation breakdown appears in the sample, bootstrapping cannot invent it.

Key bootstrap techniques include:

- **IID bootstrap** (IID means *independently and identically distributed*) draws individual returns with replacement. It preserves the marginal distribution but destroys temporal dependence, including volatility clustering. As a result, it is typically unsuitable for risk or sizing studies that require time-varying volatility.
- **Block bootstrap** resamples contiguous blocks of fixed length, preserving dependence within blocks. This recovers the short-horizon autocorrelation structure but can introduce artificial boundaries between blocks and tends to weaken long-range persistence when blocks are too short. Block length is therefore a bias–variance tradeoff: longer blocks preserve dependence better but reduce sample diversity. A common starting point is one month (about 22 trading days), then adjust based on diagnostics.
- The **stationary bootstrap** (Politis and Romano, 1994) uses random block lengths (geometric), thereby reducing boundary artifacts while

preserving dependence in expectation. In practice, it is often a better default than fixed blocks when you want volatility clustering without imposing a parametric volatility model.

A high-value diagnostic for bootstrap methods is the **autocorrelation function** (**ACF**) of squared (or absolute) returns: the IID bootstrap collapses this structure toward zero, whereas block and stationary bootstrap retain a decaying pattern that is at least directionally consistent with volatility clustering.

Parametric price and volatility models

Parametric models simulate market data by specifying a stochastic process for prices or returns and sampling paths from that process. In continuous time, these models are written as **stochastic differential equations** (**SDEs**), where dt denotes a small time step, dW denotes a Brownian (Wiener) increment (random noise that scales with $\sqrt{dt}$), and dN denotes a Poisson jump-count increment.

Parametric models can generate paths “beyond history,” which is useful for stress testing and scenario expansion, but the results are only as good as the assumptions; misspecification tends to show up in the tails and during regime transitions.

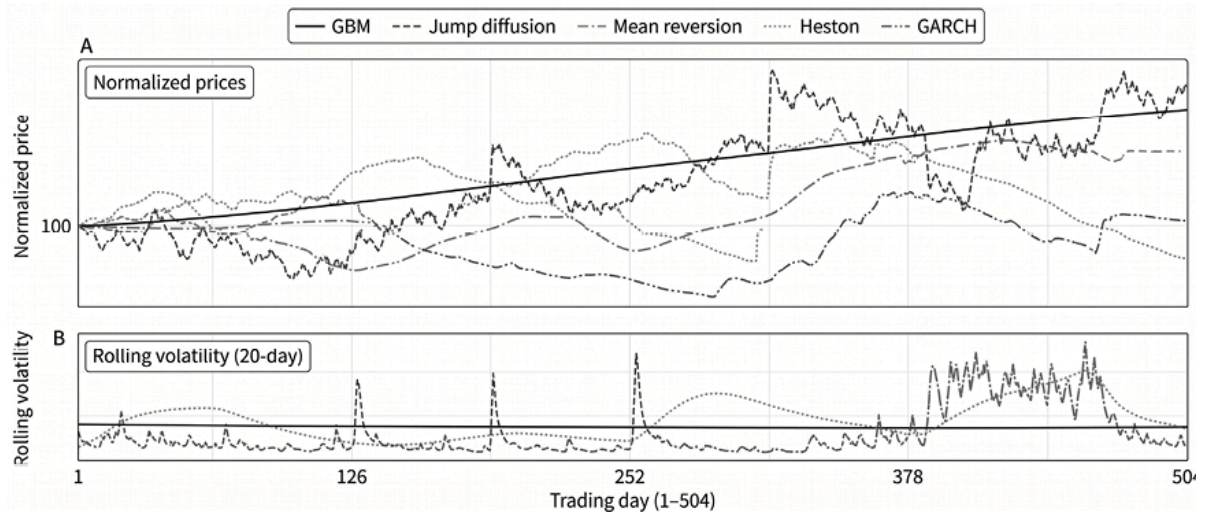


Figure 5.1: Parametric price and volatility models

Figure 5.1 compares stylized simulations from five stochastic time-series models over two years, with all prices normalized to 100 at the start. The top panel shows different price trajectories produced by each model; the bottom panel characterizes the distinct volatility patterns.

Geometric Brownian Motion (GBM) defines a foundational baseline. Price S follows a trajectory described as follows:

$$dS = \mu S, dt + \sigma S, dW$$

where μ is drift and σ is volatility. Log-price increments are Gaussian, and volatility is constant. GBM is analytically tractable (Black-Scholes) but fails core stylized facts: it produces no volatility clustering and tails that are too thin (no excess kurtosis).

Jump-Diffusion (Merton model) adds discontinuous moves via a compound Poisson process:

$$dS = \mu S, dt + \sigma S, dW + S, (e^Y - 1), dN$$

where dN is a Poisson increment with intensity λ and $Y \sim N(\mu_J, \sigma_J^2)$ is the log jump size. In practice, the drift is typically adjusted to account for the expected jump contribution; otherwise, the unconditional mean can be

misstated. Jump diffusion generates fat tails and explicit crash-like moves, but in its basic form, jump arrivals are independent of the volatility state, unlike real markets, where jumps cluster during episodes of market stress.

Mean-Reversion (Ornstein-Uhlenbeck) models log-prices (or spreads) gravitating toward equilibrium:

$$d(\log S) = \kappa(\theta - \log S), dt + \sigma, dW$$

where κ is the mean-reversion speed and θ is the long-run level. The half-life is $\ln(2)/\kappa$. This is appropriate for spreads and some rates/commodity settings, but it is not a general return model for trend-capable assets.

Stochastic volatility (Heston) introduces a separate variance process v :

$$dS = \mu S, dt + \sqrt{v}, S, dW_S$$

$$dv = \kappa(\theta - v), dt + \xi\sqrt{v}, dW_v$$

$$\text{Corr}(dW_S, dW_v) = \rho$$

Negative ρ produces a leverage effect (price drops increase volatility), and the model can generate fat tails and volatility clustering. Practical use depends on careful discretization (for example, full-truncation Euler or Andersen's QE) to maintain non-negative variance.

Generalized Autoregressive Conditional Heteroskedasticity , GARCH(p,q), models the variance of returns in discrete time, with parameters p and q denoting the numbers of lagged squared shocks and lagged conditional variances, respectively (see also *Chapter 9*). The most common specification is GARCH(1,1), which uses one lag of each:

$$r_t = \mu + \sigma_t \varepsilon_t, \quad \varepsilon_t \sim N(0,1)$$

$$\sigma_t^2 = \omega + \alpha(r_{t-1} - \mu)^2 + \beta\sigma_{t-1}^2$$

where α captures the reaction to recent return shocks and β captures volatility persistence; a common stationarity condition is $\alpha + \beta < 1$. GARCH is typically calibrated by maximum likelihood, making it a pragmatic bridge between theory and data. Basic GARCH often reproduces volatility clustering well in practice.

Limitations and how to use these baselines

Classical baselines make strong tradeoffs explicit. Bootstrap methods preserve the empirical distribution but are constrained to the historical record: they cannot generate truly novel extremes or structural breaks. Parametric models can generate paths “beyond history,” but only along the dimensions implied by their dynamics; misspecification often shows up exactly where finance cares most (tails, stress dependence, regime transitions).

In this chapter, we use these methods as reference points. They provide sanity checks for learned generators (a deep model should at least outperform an appropriate baseline on the diagnostics that matter), and they remain useful when interpretability, small-sample robustness, or governance requirements dominate. The next section provides an overview of different generative models.



Implementation : `00_classical_simulation.py`
implements all methods with calibration diagnostics.

5.4 Generative model taxonomy

This section covers **learned generators** : models that approximate the joint distribution of the variables of interest (for example, financial returns of multiple assets) by optimizing a training objective rather than by specifying a parametric process or resampling scheme. The goal is not to produce “realistic-looking” samples, but to preserve the temporal structure, dependencies, and tail behavior required for downstream robustness analysis or model development.

A useful distinction is between **discriminative** models, which learn $p(y | X)$, and **generative** models, which learn $p(X)$ or $p(X, y)$ and can therefore produce new samples. In finance, learned generators are attractive because they can represent complex dependence structures that are difficult to write down parametrically. Still, they can also fail silently by smoothing tails, collapsing modes, or learning spurious regime structure.

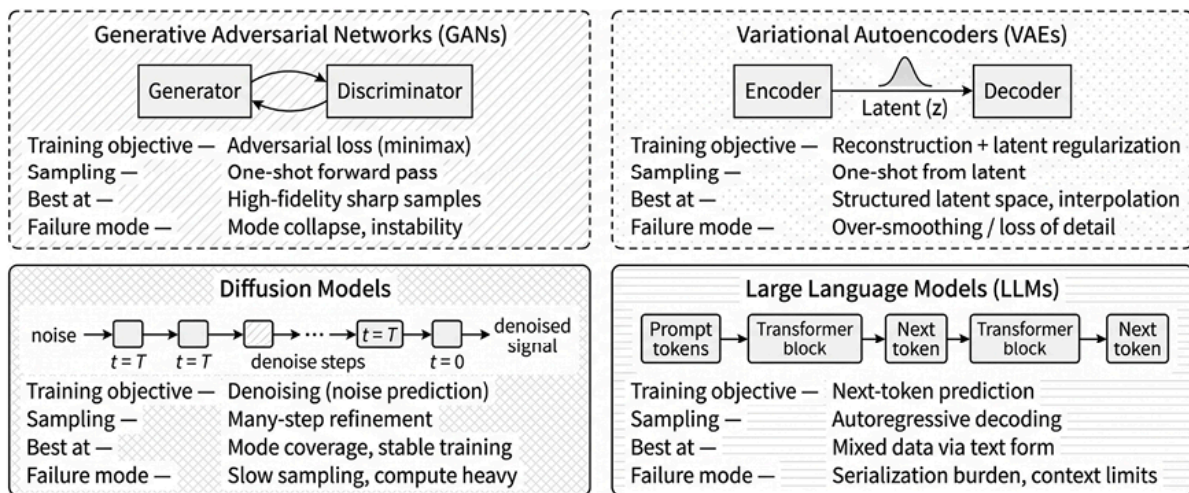


Figure 5.2: Comparing various deep generative models

The learned generators used in this chapter fall into four families:

- **Variational Autoencoders (VAE)** learn a latent-variable generative model that aims to both reconstruct the input and conform to certain distributional assumptions. VAEs often train more stably than

adversarial models and can serve as strong baselines for mixed-type structured data. Still, they may oversmooth and require calibration or more expressive likelihoods to avoid underrepresenting rare categories and tail behavior.

- **Generative Adversarial Networks (GANs)** learn through adversarial training. They can produce sharp samples and capture complex dependence, but training can be unstable. *Section 5.5* focuses on GAN variants designed to reduce collapse and improve path-wise and tail fidelity.
- **Diffusion models** learn to generate by iteratively denoising. Training is typically stable, and the framework supports conditional generation. *Section 5.6* presents a diffusion model for regime-conditioned time-series generation.
- **LLM-based generators for tabular data** serialize records as text and generate rows autoregressively. This can work well for heterogeneous, mixed-type datasets, especially when conditional generation is important. *Section 5.7* covers this approach, along with faster specialized alternatives where appropriate.

The following sections focus on specific, finance-oriented instantiations of these families. These learned generators should be treated as part of the modeling stack: they can expand the space of plausible trajectories, but they can also introduce their own inductive biases.

Each model family below should be evaluated against the framework introduced in *Section 5.2* . GAN variants are useful when their objectives align with the desired notion of fidelity, such as temporal structure, tail risk, or path-wise similarity. Diffusion models often provide more stable training and stronger conditional generation, at the cost of slower sampling. LLM-based tabular generators can handle heterogeneous structured data, but they require strict schema validation and privacy checks.

The practical question is therefore not which generator is universally best, but which generator preserves the structure required by the downstream decision, performs competitively against classical baselines, and satisfies the applicable privacy constraints.

5.5 GANs for financial time series

GANs generate synthetic samples via an adversarial game between a **generator** and a **discriminator** (*Figure 5.3*). The generator maps random input to candidate samples, while the discriminator scores both real and generated samples as real or fake.

Training alternates between strengthening the discriminator and updating the generator to fool it. This setup provides a useful learning signal even when the data likelihood is hard to specify.

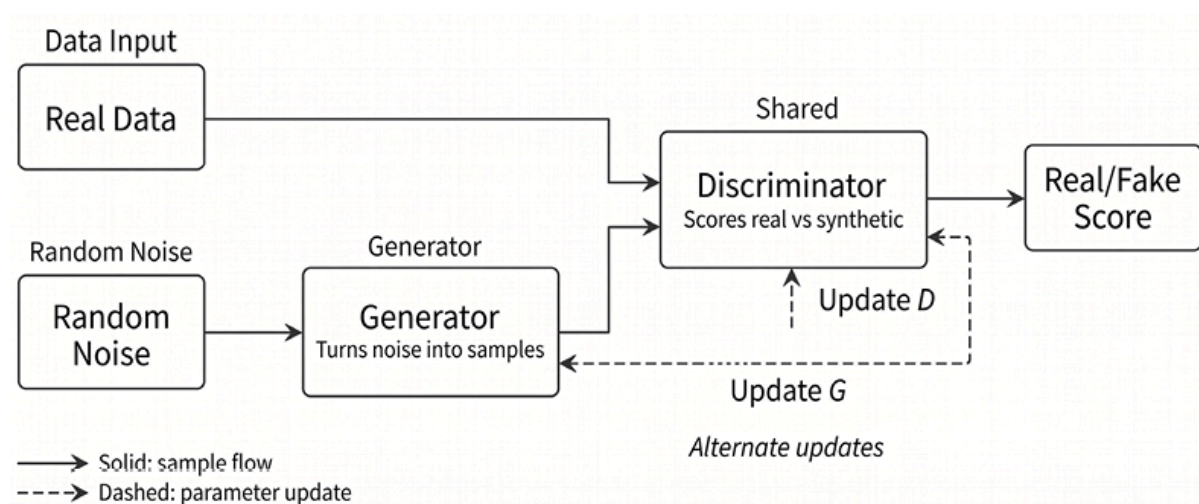


Figure 5.3: The GAN training workflow

Figure 5.3 illustrates the core loop without committing to any particular sequence model, loss function, or stabilization technique. In finance, however, vanilla GANs tend to prioritize the high-density “center” of the distribution and underrepresent rare but economically important tail events. The models that follow keep the adversarial loop but adapt it to

what vanilla GANs miss: temporal dependence (**TimeGAN**), tail behavior and risk measures (**Tail-GAN**), path-wise similarity (**Sig-CWGAN**), and irregularly sampled continuous-time dynamics (**GT-GAN**).

Using TimeGAN for sequential data

TimeGAN (Yoon, Jarrett, and van der Schaar, 2019) adapts adversarial training to time series by adding supervised temporal objectives and by learning in an embedding space rather than directly on raw sequences.

Architecture

TimeGAN comprises five networks trained in stages:

1. **Embedding network** (e) : maps a raw sequence x to a latent representation h .
2. **Recovery network** (r) : reconstructs x from h ; together, (e, r) form an autoencoder trained to minimize reconstruction error.
3. **Supervisor network** (s) : predicts the next latent state h_{t+1} from h_t , adding a supervised loss that encourages temporal coherence in latent space.
4. **Generator** (g) : produces synthetic latent paths from noise.
5. **Discriminator** (d) : distinguishes real from synthetic latent trajectories.

Training

Training usually proceeds in three phases:

1. Pretrain the embedding and recovery networks as an autoencoder.
2. Train the supervisor on real embeddings.

3. Jointly optimize the full system under reconstruction, supervised, and adversarial losses.

Data requirements

Many reference implementations use sigmoid activations in the recovery network, constraining outputs to $([0,1])$. This is convenient for normalized OHLCV inputs (see *Chapter 3*) but incompatible with unbounded return series unless returns are mapped to a bounded range or the output layer is modified. For returns, a linear output layer preserves the full real-valued support of the target.

When to use TimeGAN

TimeGAN is a strong baseline for multivariate sequence generation: it is well documented, has reference implementations, and provides a benchmark for comparing newer architectures. It does not explicitly target tail fidelity and assumes a regular sampling grid.



Implementation : The notebook `01_timegan.py` trains TimeGAN on six stocks (BA, CAT, DIS, GE, IBM, KO) using 24-step windows. Discriminative accuracy is 68%, with ROC AUC 0.87, indicating residual separability between real and synthetic sequences.

The **TSTR** ratio for one-step forecasting is 1.76: training on TimeGAN-generated multi-stock adjusted-close data produces 76% higher one-step forecast error than training on real data, suggesting that synthetic data is informative but imperfect for downstream prediction.

Under the validation framework from *Section 5.2*, these results show partial utility but incomplete fidelity: the generated data contains enough

structure to support a weak downstream forecasting task, yet the discriminator and dependence diagnostics indicate that real and synthetic sequences remain materially separable.

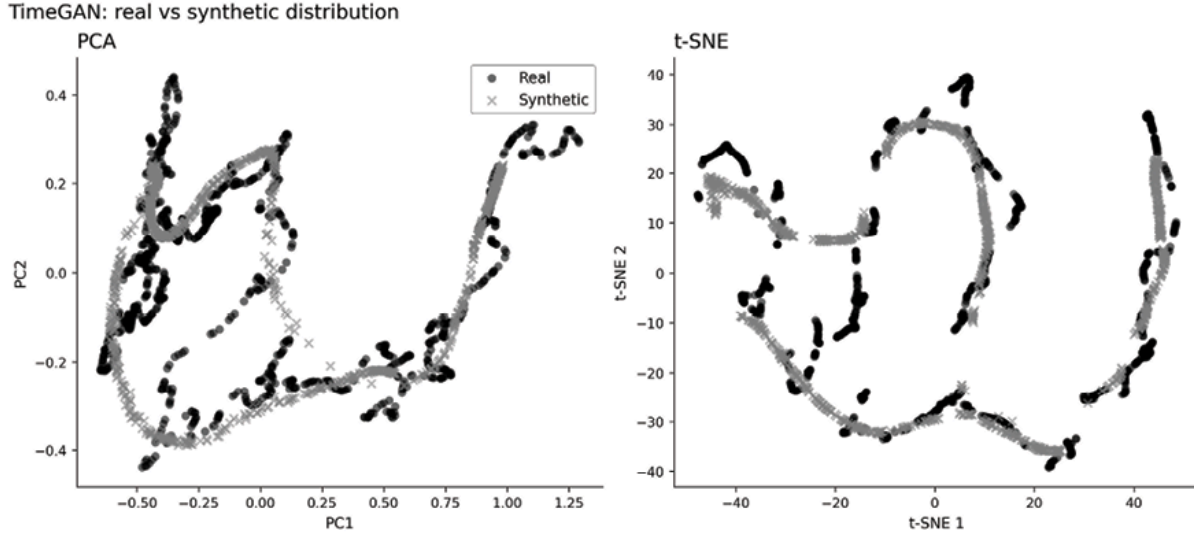


Figure 5.4: TimeGAN fidelity, showing how PCA and t-SNE overlap

In our analysis, synthetic sequences underrepresent temporal and cross-asset dependence relative to real data, consistent with TimeGAN’s focus on overall distribution rather than fine-grained dynamics. On similar inputs, Diffusion-TS (*Section 5.6*) performs better in our experiments (TSTR ratio: 1.00 compared to 1.76).

Using Tail-GAN for risk-constrained generation

Tail-GAN (Cont et al., 2025) variants address a limitation of standard GAN objectives: optimizing for average distributional fidelity can underweight rare but consequential tail events. For risk management, matching tail risk measures such as **Value-at-Risk (VaR)** and **Expected Shortfall (ES)** is often more important than matching the distribution center.

Tail-GAN augments the generator loss with penalties that discourage discrepancies between real and synthetic tail risk measures:

$$L_{tail} = \lambda_1 |VaR_{\alpha}^{real} - VaR_{\alpha}^{synth}| + \lambda_2 |ES_{\alpha}^{real} - ES_{\alpha}^{synth}|$$

where α is the tail probability (often 1% or 5%), and (λ_1, λ_2) trade off tail matching against overall fit.

Portfolio focus

Tail constraints are typically computed on portfolio returns rather than on individual assets. This aligns the objective with risk management practice, where joint dependence drives drawdowns and limits are set at the portfolio level.

When to use Tail-GAN

Use Tail-GAN when the downstream task is risk measurement and tail accuracy dominates other fidelity notions. The method may sacrifice fit in the distribution center to improve VaR/ES matching.



Implementation : `02_tailgan_tail_risk.py` trains Tail-GAN on 5 ETFs using 32 random long-short portfolio strategies. At $\alpha = 5$, VaR relative error is 13% and ES error is 11%, illustrating that explicit tail penalties produce usable risk estimates. In this 32-strategy configuration, synthetic VaR is slightly more negative than real (synthetic ≈ -10.33 versus real ≈ -9.13), so the trained Tail-GAN modestly overstates loss magnitude rather than understating it.

Tail-GAN improves selected tail statistics, but the constraints are usually defined on specific benchmark portfolios and risk levels. Matching VaR

and ES for those portfolios does not guarantee accurate tail dependence for other portfolios, individual assets, or path-dependent risk measures.

More generally, VaR and ES capture only part of the tail: two models can match these quantities yet differ in tail shape, clustering, and temporal dynamics. Performance, therefore, depends materially on the choice of tail probability α , the benchmark portfolios used during training, and the regimes represented in the data.

Using Sig-CWGAN for path signatures for temporal fidelity

Sig-CWGAN (Ni et al., 2020) replaces the learned discriminator with an analytic criterion based on **path signatures** from rough path theory. The objective reduces reliance on an adaptive discriminator and provides a structured notion of distance between path distributions.

A path signature is a **sequence of iterated integrals** that summarizes the shape of a path: not just where it starts and ends, but how it moves through its coordinates, in what order, and with what directional interactions.

Intuitively, the first level records the path's net increments, the second level records pairwise interaction effects between coordinates and the order in which moves occur, and higher levels capture progressively richer geometric structure.

A key mathematical property is that the signature depends on the path's traced-out shape rather than the speed at which the path is traversed, so it is **invariant to time reparameterization**. In financial applications, this is often useful but not always desirable, because elapsed time and observation spacing can themselves contain information. Therefore,

implementation often augments X with time as an additional channel. For a path $X: [0, T] \rightarrow R^d$, a level- k term has the form:

$$S(X)^{i_1, \dots, i_k} = \int_{0 < t_1 < \dots < t_k < T} dX_{t_1}^{i_1} \dots dX_{t_k}^{i_k}$$

and the depth- n signature collects all terms up to order (n). In practice, one can think of the truncated signature as a finite feature map that converts a sequential path into a set of statistics describing its geometry at increasing levels of detail. Implementations often augment (X) with time as an additional channel and use truncated signatures at modest depth.

Sig-CWGAN minimizes a signature-kernel **maximum mean discrepancy (MMD)** between real and synthetic path distributions, yielding a stable training signal without adversarial classification.

The number of signature terms grows as $O(d^n)$ in the path dimension d and truncation depth n . With two assets, time $d = 3$, and depth 3, the truncated signature has $1 + 3 + 9 + 27 = 40$ terms. By contrast, 50 assets at depth 3 would require over 125,000 terms, making straightforward scaling impractical without dimensionality reduction.

Use Sig-CWGAN when **path-wise fidelity** matters (for example, in path-dependent pricing or sequential decision problems) and the dimension is small enough to compute signatures reliably. The analytic objective can reduce failure modes driven by discriminator overfitting, but scalability is the binding constraint.

Implementation : `03_sigcwgan_signatures.py`

reproduces the paper-exact setup: a single asset (S&P 500 index log returns, 2005–2020) at signature depth 4 over 16-day windows, trained for 2,500 generator steps.



- The Sig-W1 distance (RMSE of expected signatures) descends to 0.054 in training and 0.42 on the held-out window, and the train-on-synthetic, test-on-real ratio reaches 0.954—close to parity with the real data.
- Stylized facts are only partially captured: synthetic skewness and excess kurtosis fall far short of the empirical values, and the squared-return autocorrelation collapses to 0.05 (versus 0.49 in the data), so volatility clustering is not preserved.

Sig-CWGAN is most attractive in low-dimensional settings. Truncating the signature discards higher-order path information, while increasing depth quickly becomes computationally expensive and statistically harder to estimate. In practice, **the method is best viewed as a structured low-dimensional generator** rather than a general-purpose solution for broad multivariate market panels.

Using GT-GAN for irregular time series

GT-GAN (Jeon et al., 2022) extends adversarial generation to **irregularly sampled** time series, in which observations occur at non-uniform intervals (for example, tick data, event-driven bars, consolidated feeds).

GT-GAN models continuous-time dynamics with a neural **ordinary differential equation (ODE)** that parameterizes:

$$\frac{dx}{dt} = f_{\theta}(x, t)$$

and integrates the dynamics to generate values at arbitrary timestamps.

The discriminator evaluates both sampled values and their associated timestamps, and a supervised component encourages temporal coherence, analogous to TimeGAN's supervisor.

GT-GAN is designed for naturally irregular data, where observation times carry information (for example, trade arrivals or information events). Applying it to regularly sampled series with artificial irregularity (random subsampling) is unlikely to outperform simpler discrete-time generators.

Use GT-GAN when observation times carry information. For regularly sampled daily or minute bars, discrete-time generators are usually simpler and more efficient.



Implementation : `04_gtgan_irregular.py` trains GT-GAN on NVDA dollar bars from *Chapter 3*.

Reconstruction MSE is 0.026, and the interpolation/real smoothness ratio is 0.02—the ODE-driven paths are considerably smoother than real bars. Because the experiment uses only 446 bars from a single trading day, these results should be read primarily as an illustration of the continuous-time training procedure rather than as a competitive benchmark of distributional fidelity.

GT-GAN introduces substantially more modeling and optimization complexity than discrete-time generators. With limited data, a continuous-time generator may produce visually smooth paths without accurately matching the true event-time distribution, dependence structure, or regime behavior. The method is most compelling when irregular timing is economically meaningful; if irregularity mainly reflects sampling choices or microstructure noise, the additional

continuous-time machinery may add complexity without improving fidelity.

GAN training challenges

GAN variants share recurring implementation risks:

- **Mode collapse:** The generator covers only a subset of regimes, underrepresenting rare states that matter for risk.
- **Training instability:** Optimization can oscillate or diverge; stabilization methods include spectral normalization, gradient penalties (WGAN-GP), and phased training (TimeGAN).
- **Hyperparameter sensitivity:** Learning rates, architectures, and regularization weights often require dataset-specific tuning.
- **Evaluation ambiguity:** There is no ground-truth label; evaluation relies on held-out distributional tests, downstream utility (TSTR), and diagnostics for temporal structure and cross-sectional dependence.

Kwon and Lee (2024) report that GANs can approximate marginal return distributions but often struggle with finer temporal structure and multivariate dependence; reported performance varies across architectures, and multivariate generation can collapse to a low-dimensional dependence structure. Passing marginal tests is therefore necessary but not sufficient; temporal dynamics and cross-sectional relationships require separate validation. *Section 5.6* turns to diffusion models.

5.6 Diffusion models for financial time series

Diffusion models have shown strong empirical performance on synthetic data, from images to financial time series.

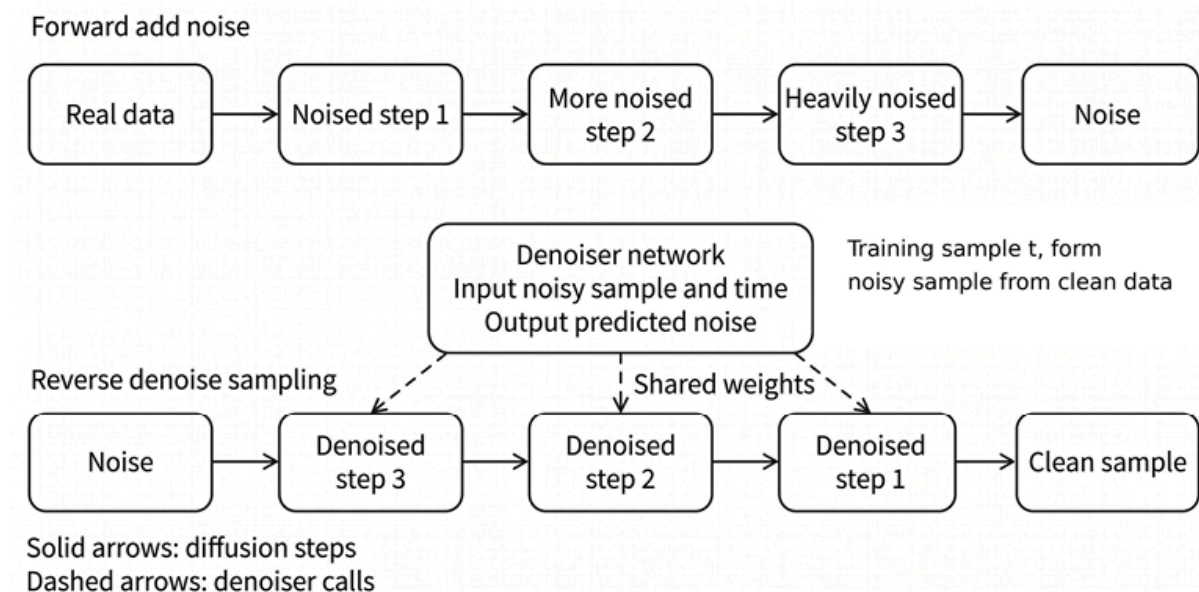


Figure 5.5: DDPM forward/reverse schematic

In *Figure 5.5*, the forward process progressively corrupts the real data into near-pure noise. During training, a timestep t is sampled, noise is added to the real data, and a single denoiser network is optimized to predict the added noise. At generation time, sampling starts from near-pure noise and iteratively applies the same denoiser to produce real data; the reverse chain is stochastic, injecting fresh noise at intermediate steps (except the final step), so each run can yield a different sample.

Their iterative denoising mechanism can reproduce non-Gaussian features of market data that simpler generators often miss, which makes them useful when sample fidelity is the primary objective (Takahashi and Mizuno, 2024). The main trade-off is computational cost: training and generation are typically slower than with one-shot generators.

The denoising diffusion framework

Diffusion models define a **forward process** that gradually corrupts an observed sequence with noise, and a **reverse process** that learns to remove it. Sampling starts from noise and applies learned reverse transitions to generate a synthetic sequence.

The forward process is a Markov chain that adds Gaussian noise over diffusion steps $t = 1, \dots, T$. Given original data x_0 , define:

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)I)$$

where $\alpha_t = 1 - \beta_t$ and β_t is a noise schedule. With sufficiently many steps, x_T approaches an isotropic Gaussian. The schedule determines the signal-to-noise ratio across timesteps and shapes the difficulty of denoising.

The reverse process learns a conditional transition from x_t to x_{t-1} :

$$p_\theta(x_{t-1} | x_t) = N(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

The denoiser takes x_t and an embedding of t as inputs and outputs either parameters $\mu_\theta, \Sigma_\theta$ or an equivalent quantity such as the noise ε . Training samples diffusion steps and minimizes an expectation over timesteps, so the model learns to invert the corruption process at multiple noise levels.

The **Denoising Diffusion Probabilistic Model (DDPM)** (Ho et al., 2020) popularized a mean squared error loss between the realized forward-process noise ε and the network prediction $\varepsilon_\theta(x_t, t)$, avoiding direct reconstruction of x_0 during training. This objective is widely used because it is stable and maps directly to an iterative sampling procedure.

Why diffusion models fit financial returns

The stylized facts in *Section 5.1* —heavy tails, volatility clustering, leverage effects, and weak return autocorrelation—constrain generators for returns or return-like series. A useful generator must match both the marginal distribution and conditional dynamics, such as persistent volatility and time-varying tail risk. Diffusion models can capture such structure because the denoiser is trained across noise levels rather than being optimized to match only low-order moments.

The three families have distinct failure profiles. VAEs can produce overly smooth samples during decoding, as they average over uncertainty. GAN training can be unstable and may underrepresent tails or minority modes. Diffusion training is typically more stable, but sampling can be slow because it performs many reverse steps per generated sequence.

Conditional generation for regime stress testing

Conditional generation produces samples consistent with specified attributes. Two common approaches are:

- **Classifier guidance** , which trains a separate classifier on noised data and uses its gradient to steer sampling
- **Classifier-free guidance** , which embeds conditioning during training

Both can direct generation toward states such as “low volatility” or “crisis,” enabling regime-specific scenario generation and stress testing.

A workflow that integrates with regime detection methods (discussed in more detail in *Chapter 9*) is:

1. Fit a regime model (for example, an HMM) to label historical periods.

2. Train a classifier to predict regime labels from noisy sequences across diffusion timesteps.
3. At generation time, compute $\nabla_{x_t} \log p(\text{regime} | x_t)$, the gradient of the log-probability of the target regime with respect to the current noisy sample x_t , and use it to adjust the denoising update.

Classifier guidance (Dhariwal and Nichol, 2021) requires tuning. Strong guidance can reduce diversity and exaggerate rare regimes, producing unrealistic extremes. Temperature scaling and stochastic sampling (for example, Denoising Diffusion Implicit Models with $\eta > 0$) can help balance targeting and diversity. *Figure 5.6* shows the result: low- and high-volatility regime samples produce visibly separated paths, with a $2.1\times$ regime-volatility ratio.

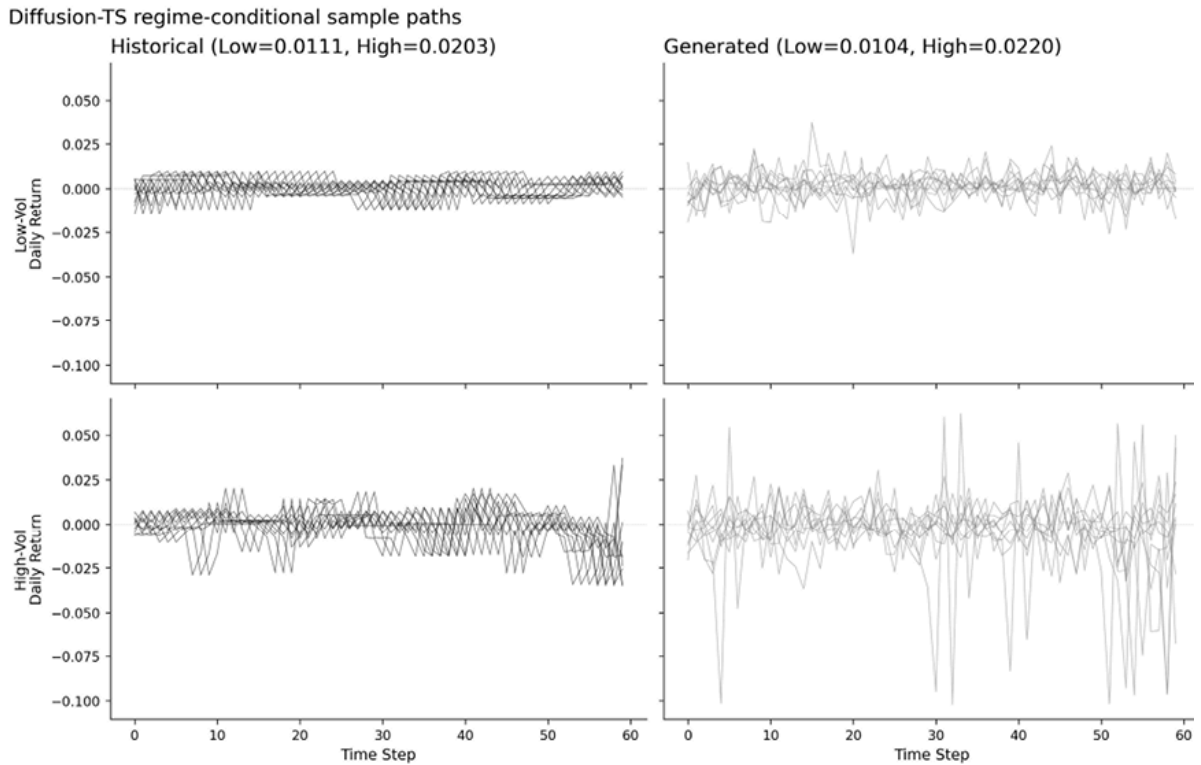


Figure 5.6: Diffusion-TS regime-conditional sample paths

Because regimes are model-derived, the conditional generator inherits errors from the regime detector. Misclassified periods become mislabeled

training data, which can distort conditional samples. Use regime-conditioned sequences as model-based scenarios and validate them with regime-specific diagnostics before downstream use.

Applying Diffusion-TS with interpretable decomposition

Time-series generators must represent dependencies across the sequence, not just marginal distributions. Diffusion-TS (Yuan and Qiao, 2024) adapts the diffusion denoiser to sequential data by using an **encoder-decoder transformer** and explicitly decomposing the reconstructed sequence into low- and mid-frequency components.

The denoiser takes a noised sequence x_t and a timestep embedding t . An encoder produces a context representation of x_t , and a decoder uses cross-attention to predict the clean sequence $\hat{x}_0$.

Rather than predicting $\hat{x}_0$ directly, Diffusion-TS constrains the output to be a sum of two components:

$$\hat{x}_0 = \text{trend} + \text{season}$$

The **trend** component is parameterized as a low-order polynomial fit, intended to capture slow-moving drift over the generation horizon. The **seasonal** component is represented using a truncated Fourier basis (top- k modes), which targets periodic or quasi-periodic structure.

A **Fourier-domain loss** complements the time-domain loss by penalizing mismatches in the frequency representation, encouraging spectral fidelity, and helping preserve autocorrelation-related patterns.

This decomposition is not a guarantee of **interpretability** in the economic sense, but it provides an operational handle: when samples fail diagnostics, the error can often be localized to the slow component

(trend), the periodic component (seasonality), or the residual high-frequency variation that neither component represents well.

In practice, Diffusion-TS shares training and evaluation challenges with other DDPM models:

- **Variance calibration:** A trend-plus-seasonal output can understate high-frequency variation and reduce realized return variance. In the reference implementation, raw unconditional samples understate variance by approximately one-third in the normalized space before post hoc rescaling. Downstream tasks that depend on volatility matching may require explicit variance-calibration terms during training.
- **Compute and latency:** DDPM sampling can be slow because it requires many reverse steps per sample. DDIM and distillation-style methods reduce the number of steps, typically at the expense of speed.
- **Temporal coherence:** Alternative approaches impose sequential structure via path summaries such as signatures, which are stable under reparameterization. For example, Sig-CWGAN uses a signature-based discrepancy in its training objective (*Section 5.5*). For diffusion models, include temporal diagnostics (ACF, volatility persistence, leverage proxies) in addition to marginal distribution tests.
- **Validation:** Matching stylized facts is necessary but not sufficient: a generator can satisfy distributional diagnostics yet fail to preserve task-relevant structure. *Section 5.8* presents validation aligned with the intended application.



Implementation : `05_diffusion_ts.py` implements Diffusion-TS end-to-end with interpretable



decomposition, Fourier loss, and classifier-guided regime-conditional generation.

For the 20 ETF daily return series (2018–2025), the implementation reports a KS statistic of 0.06, a correlation error of 0.04, and an ACF error of 0.05. Regime-conditional generation produces low-volatility samples at $0.93\times$ historical volatility and high-volatility samples at $1.08\times$ historical volatility.

Choosing between GANs and diffusion models

Diffusion models trade generation speed for training stability. GANs produce samples in a single forward pass; diffusion models require iterative denoising, though DDIMs reduce this cost to 50–100 steps. The GAN variants in *Section 5.5* each address a specific limitation: Tail-GAN targets risk measures, Sig-CWGAN targets path fidelity, and GT-GAN handles irregular timestamps. Diffusion-TS offers a more general-purpose alternative with built-in conditional generation, but specialized GANs remain appropriate when their design objective aligns with the evaluation target.

Direct comparison of these toy implementations would be misleading—each generator was designed for a different task with different data requirements. TimeGAN requires bounded data; Tail-GAN optimizes for portfolio-level tail metrics; Sig-CWGAN works only at low dimensionality; GT-GAN needs naturally irregular timestamps. The practical question is not “which generator is best?” but “which generator matches my use case?” For tabular data, large language models can be a good candidate, as we will see next.

5.7 LLMs for structured financial data

Large language models (LLMs) can generate realistic synthetic samples from tabular datasets when the table is represented as text. Although LLMs are trained on natural language, they can learn and reproduce complex cross-column dependencies in structured data, which is useful for financial tables that mix numerical, categorical, and occasional free-text fields.

Serialization converts each table row into a text record that explicitly encodes column–value pairs. A serialized table becomes a corpus of short records that an autoregressive model can learn to reproduce. Consider a credit application record:

income	debt_ratio	emp_length	home_status	approved
85000	0.32	7	MORTGAGE	1

Table 5.1: A table showing a record for a credit application

One serialized representation is:

```
income is 85000, debt_ratio is 0.32, emp_length is 7, home_st
```

Training on many such records encourages the model to capture conditional structure across fields (for example, how income and debt ratio relate to approval, conditional on employment and housing status).

The GReaT framework

Generate Realistic Tabular Data (GReaT) formalizes a simple workflow (Borisov et al., 2023):

1. **Serialize the training data** . Convert each row into a text record of column–value pairs. To reduce sensitivity to a fixed presentation order, training often randomizes the column order.
2. **Fine-tune an autoregressive LLM** . Fine-tune a pre-trained language model (for example, GPT-2-scale) on the serialized corpus so it learns to predict the next token given the preceding tokens, thereby implicitly modeling a joint distribution over fields.
3. **Generate new samples** . Sample new text records and parse them back into rows. Filter invalid outputs (malformed records, missing fields, out-of-range values).

The framework can be useful for financial applications subject to some limitations and risks.

Financial applications

LLM-based tabular generation is well-suited to datasets with mixed field types and complex conditional dependencies:

- **Credit and loan data:** Application records with demographic variables, financial ratios, and categorical fields are natural candidates. Synthetic datasets can support model development while reducing exposure of sensitive customer data.
- **Customer profiles:** Retail banking and wealth datasets often combine transaction aggregates, account attributes, and occasional text fields. Serialization provides a uniform interface to this heterogeneity.
- **Corporate fundamentals:** Financial statements and derived ratios, together with industry codes and accounting flags, can be augmented or anonymized using LLM-based generators, subject to strict point-in-time and accounting-convention checks.

Limitations and risks

LLM-based generation introduces failure modes that require explicit mitigation:

- **Invalid records:** Autoregressive generation can produce internally inconsistent combinations (for example, an employment history that is incompatible with age). Post-generation validation is required.
- **Numerical fidelity:** LLMs optimize for token likelihood, not distributional accuracy. High-precision numeric strings are difficult to represent reliably; practitioners often discretize, bin, scale, or normalize numeric features before serialization. Even with preprocessing, marginal distributions and tail behavior can drift relative to the training data.
- **Compute cost:** Fine-tuning and inference add computational overhead. In many tabular settings, moderate-sized models can be sufficient, but cost should be evaluated relative to simpler tabular generators and classical baselines.
- **Privacy leakage:** Synthetic samples can memorize and reproduce rare training examples. Privacy evaluation (*Section 5.8*) is required before using generated data outside a controlled environment.

Serialized generation often violates domain constraints. To manage this risk, implement a **constraint layer** that validates each parsed row:

- **Type constraints:** Age is an integer; ratios are non-negative
- **Range constraints:** Employment years `< age - 16`
- **Logical constraints:** If `home_status = "RENT"`, then `mortgage_balance = 0`

Monitor rejection rates as a diagnostic. Sustained rejection rates above 10–15% suggest that the serialization format, preprocessing, or training protocol needs adjustment.



Implementation : `06_llm_tabular_great.py` fine-tunes `distilgpt2` on ETF-derived tabular features (returns, volatility, volume ratios, and categorical regime labels). Fifty epochs of fine-tuning on an RTX 3090 take roughly thirteen minutes—modest compared with training a dedicated tabular generator from scratch.

- On a held-out test set of 600 samples (extreme-move classification, $|fwd_{ret_{5d}}|$ above the 90th percentile), TSTR achieves an AUC-ROC of 0.70, close to the 0.74 TRTR baseline; the TSTR ratio is 92%, indicating that synthetic training data largely preserves downstream model utility for this task.
- Distributional fidelity is weaker than utility. Among numerical columns, only `volume_ratio` matches well (KS = 0.10); `volatility` and all four return features (1-, 5-, 20-day, and forward 5-day) sit in the 0.30–0.59 range with p-values below 0.001, so synthetic and real marginals are statistically distinguishable. The categorical distribution is heavily mode-collapsed: the model overgenerates the negative-return regime (`direction = down` 94% synthetic versus 45% real) and the no-trend regime (`momentum = flat` 69% versus 36%), and undergenerates strong-trend states (`momentum = strong` 10% versus 28%; `vol_regime = high` 1% versus 7%). This gap between high task utility and modest distributional fidelity is the practical limit of short LLM fine-tuning on small tabular datasets—longer fine-tuning, larger base models, or rejection sampling on schema constraints close it at the cost of compute, which is the trade-off *Section 5.2* formalized.

This result illustrates why utility and fidelity must be evaluated separately. The synthetic data preserves enough task-relevant structure to support the extreme-move classifier, but the marginal and categorical

diagnostics reveal substantial distributional distortion. The next section presents the comprehensive evaluation framework recommended for practical use cases.

5.8 Applying the Fidelity-Utility-Privacy framework

The preceding sections show that the quality of synthetic data depends on the intended use case. No generator dominates across fidelity, utility, privacy, dimensionality, and computational cost. The Fidelity–Utility–Privacy framework from *Section 5.2* provides a common language for comparing methods without reducing them to a single score.

Reading the Diffusion-TS results

The Diffusion-TS reference run on 20 daily return series of ETFs illustrates how to interpret the framework in practice. The model reports a KS statistic of 0.06 for marginals, a correlation error of 0.04 for cross-asset dependence, and an ACF error of 0.05 for volatility persistence. These are **fidelity** diagnostics: they suggest that the generated samples preserve broad distributional and temporal structure in this configuration.

Utility is evaluated separately. For the extreme-move classification task, the TSTR result is approximately at parity with the TRTR benchmark, indicating that synthetic training data preserves the task-relevant signal for this specific prediction problem.

Privacy is not established by these results. Unless the generator is trained with a formal privacy mechanism or subjected to leakage tests, strong fidelity and utility do not imply privacy. A model can reproduce useful structure while still memorizing rare records or leaking proprietary information.

The interpretation is therefore conditional: Diffusion-TS is a strong, general-purpose generator in this experiment, especially in terms of broad fidelity and task utility. Tail-focused risk applications may still require explicit tail penalties, as in Tail-GAN. Low-dimensional path-dependent applications may benefit from signature-based objectives, as in Sig-CWGAN. Privacy-sensitive applications require a separate privacy mechanism and leakage evaluation.

Privacy-utility trade-off with DP-GAN

When formal privacy guarantees are required, differential privacy provides a mathematical bound on the influence of individual training records.



Implementation : The notebook `07_dp_gan.py` demonstrates this trade-off by training a GAN with DP-SGD using Opacus and sweeping across privacy budgets.

Privacy Budget	Mean Abs Diff	Correlation Distance	Interpretation
1.0	8.02	0.12	Strong privacy, poor utility
5.0	2.40	0.15	Good balance
10.0	2.67	0.06	Good utility
50.0	1.68	0.11	Best utility, weak privacy

Table 5.2: Privacy-utility tradeoff example

At $\epsilon = 1$, the noise required for stronger privacy substantially degrades distributional fidelity. As the privacy budget increases, utility improves, but the formal privacy guarantee weakens. In this example, the middle range around $\epsilon \in [5,10]$ offers a more practical trade-off, but the appropriate threshold depends on the data sensitivity, regulatory context, and intended use.

The important lesson is that **privacy cannot be inferred from realism**. Synthetic data may look different from the training data and still leak information; it may also satisfy privacy constraints while becoming too distorted for downstream modeling. Privacy-sensitive synthetic-data workflows therefore require explicit leakage tests or formal DP accounting in addition to fidelity and utility evaluation.

5.9 Summary

Synthetic data generation addresses a core limitation in quantitative finance: we only observe one realized market path, yet strategies must remain robust across many plausible alternatives. Modern generative models can produce “alternative histories” that preserve key statistical structure, enabling more rigorous robustness checks, stress testing, and parameter selection than a single backtest trajectory can support.

Effective use hinges on disciplined benchmarking and validation. Generators should reproduce core stylized facts, and classical models remain strong, interpretable baselines when transparency or data constraints matter.

Deep generative approaches (GAN variants, diffusion models, and LLMs for serialized tabular data) can target objectives such as tail risk, path-wise fidelity, irregular time sampling, and heterogeneous schema generation. Each must be evaluated against the Fidelity–Utility–Privacy triad and stress-tested for synthetic-specific failure modes such as bias

amplification, limited novelty, and overfitting to the generator, with conclusions confirmed on held-out real data.

In *Chapter 6* , we present the strategy research framework that guides the ML4T workflow and our nine case studies.

6

Strategy Research Framework

Every trading strategy starts with an idea. The hard part is turning that idea into trustworthy evidence: results that survive realistic costs and constraints, remain interpretable after iterative search, and translate into a repeatable process that performs out-of-sample rather than a one-off in-sample backtest. We introduce the strategy research framework that anchors the ML4T workflow: a research loop we develop in *Chapters 7-20* , from feature engineering and modeling to strategy backtesting, before taking a strategy live in *Chapter 25* .

The framework requires us to define the trading setup early: what is traded, when decisions are made, how signals become positions, and how positions produce net outcomes after costs, constraints, and risk controls. Some parameters are known from the outset (for example, the universe of investable assets, or explicit fees); others require further research (for example, market impact or risk management). The goal is to make dependencies visible, state conservative assumptions for early feasibility screens, and record open issues for later chapters.

By the end of the chapter, you will be able to:

- **Place an idea on the strategy map**, connecting it to a strategy family, a plausible source of recurring profits, and the dominant feasibility constraints.
- **Specify the trading setup** in decision-time terms: what is tradable, when decisions are made, what is admissible, and how scores become positions.
- **Define “better” economically** , and separate metric roles so diagnostics guide iteration without substituting for strategy evaluation.
- **Adopt a time-series evaluation protocol** that separates selection from performance estimation.
- **Define a baseline checkpoint** that is narrow by design and supports early feasibility screens before broad search.
- **Keep search countable and recoverable** using automatic run logging and a simple trial taxonomy.

In practice, these choices live in code: a small set of versioned configuration objects and a pipeline that consumes them. Every run automatically emits a record (config, splits, metrics, outputs) so that results are reproducible and iterations are countable.

6.1 From idea to evidence with the ML4T workflow

A trading strategy is an executable process. Once live, it runs on a schedule, uses the information available at decision time to produce model inputs, converts model outputs into positions, and realizes outcomes after accounting for costs and constraints. This workflow is shown in *Figure 6.1*.

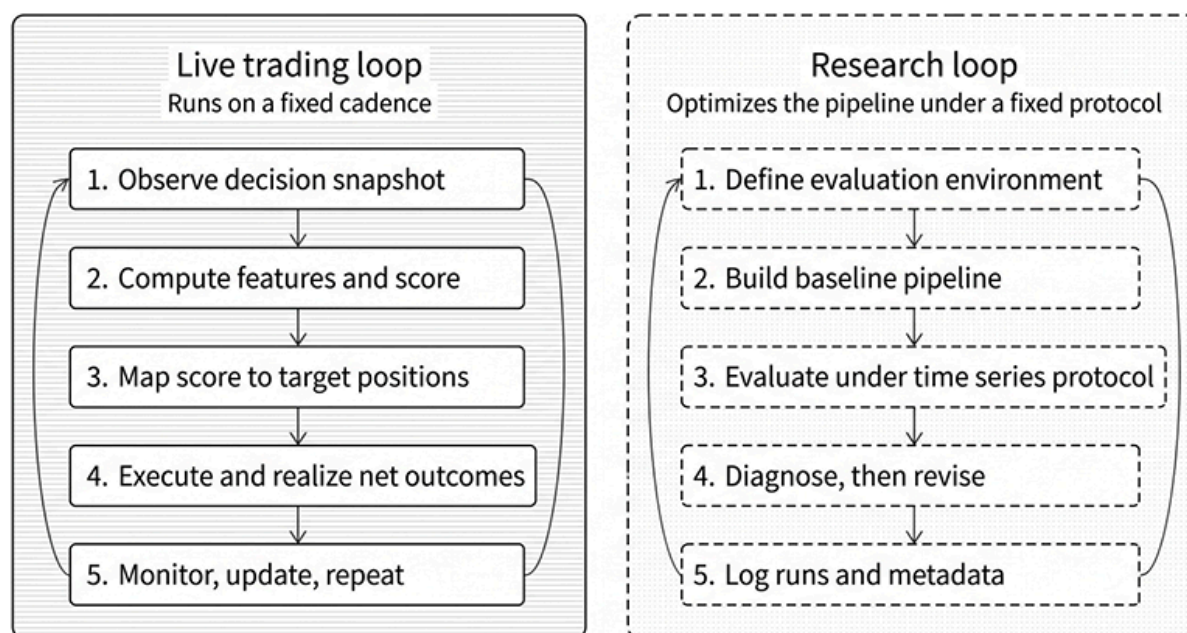


Figure 6.1: Live trading and the research loop

During research, we simulate this process using historical data. The goal is to optimize the process by iteratively refining each component while keeping comparisons meaningful and interpretable, so that historical simulation matches the live system's behavior at decision time.

The live trading loop

Every systematic strategy—whether it uses a hand-built signal or an ML model—can be described as a sequence of steps that repeat on a fixed cadence:

1. **Observe a decision snapshot** . At each decision time, the strategy takes a snapshot of the available information that defines what it “knows” when it commits to trades.
2. **Compute features and produce a score** . This data is transformed into a set of features for a model (or a rule-based signal) to predict rankings, probabilities, return forecasts, or other scores useful for trading decisions.
3. **Translate the score into positions** . A trading policy converts these predictions into target positions, subject to risk limits and constraints, and places orders accordingly.
4. **Execute and realize net outcomes** . Orders become fills (completed trades) with slippage, spreads, fees, financing, funding, roll costs, and other market-specific mechanics. Positions generate profit and loss (PnL) and exposures that are measured and monitored.
5. **Monitor, update, and repeat** . Performance, risk, and data quality are tracked; operational exceptions are handled; the loop repeats at the next decision time.

This loop is only meaningful when its timing is well defined. It is impossible to use information that is not available when trading live, but deceptively easy to do so during research on historical data. We must therefore clearly define the timing of trading decisions and keep these conventions stable, so that improvements can be attributed to deliberate changes rather than to shifting assumptions.

The research loop

Research builds evidence about how the live loop will behave without “moving the goalposts” as we iterate. We start with a baseline version of the live loop, evaluate it using a time-series protocol, diagnose weaknesses, and change one component at a time, while accounting for measurement noise. A practical research loop looks like this:

1. **Define the evaluation environment** . Declare the tradable universe, the decision cadence, basic constraints, and the cost components treated as material. This setup makes results comparable across iterations and avoids surprises during deployment.
2. **Build a baseline pipeline** . Implement the simplest end-to-end version that is consistent with the setup: minimal features, a baseline label, a baseline model class, and a simple mapping from scores to positions.
3. **Evaluate under a time-series protocol** . Compare candidates using time-aware validation and reserve a holdout test set for confirmation once the pipeline is

stable. This is where we prevent leakage and selection bias from dominating conclusions.

4. **Diagnose, then revise** . Use diagnostics to decide what to change next: data definitions, feature families, model class, hyperparameters, mapping class parameters, constraints, or cost assumptions. Make changes deliberately and keep the rest fixed.
5. **Log what happened** . Record enough metadata to reproduce each run and to count the alternatives tried. Without this, iteration becomes untraceable, and selection bias becomes invisible.

The loop does not assume a single crisp hypothesis at the outset. In ML-driven research, we typically explore a **hypothesis class**: a family of measurements that might, in combination, yield better trading decisions. The discipline is not “never explore” but to explore within a stable setup and robust protocol so that results remain interpretable.

Illustrating the research framework with case studies

Nine case studies illustrate the strategy research workflow throughout our remaining chapters. Each is a scaffold for timing, feasibility, and evaluation, not a recommended trading system. They span multiple asset classes and data frequencies, drawing on the datasets from *Chapters 2 and 3* . For each case study, we highlight the asset class, data frequency, and coverage in terms of the number of symbols and the time period.

- **etfs**: *Multi-asset · Daily · ~100 symbols over 20 years*. Cross-asset rotation strategies that highlight turnover economics and regime dependence across equities, fixed income, and commodities.
- **us_equities_panel**: *Equities · Daily · ~3,000 symbols over 50 years*. The workhorse cross-sectional dataset for signal construction, universe filtering, and capacity analysis at scale.
- **us_firm_characteristics**: *Equities · Monthly · ~2,500 stocks · 1996–2016*. Factor models built on ~57 firm-level characteristics, emphasizing point-in-time discipline to avoid survivorship and lookahead bias.
- **fx_pairs**: *FX · 4-hour bars / daily NY 5PM decisions · 20 pairs · 2011–2025*. Currency strategies that confront close definitions, bar aggregation choices, and tradable return construction across time zones.

- `cme_futures`: *Futures · Daily · 30 contracts over 15 years*. Trend and carry strategies that require careful session alignment, roll mechanics, and term-structure modeling.
- `crypto_perps_funding`: *Crypto · 8-hour · ~20 symbols over 5 years*. Perpetual futures, where funding schedules, exchange-specific market mechanics, and transaction costs dominate signal economics.
- `nasdaq100_microstructure`: *Equities · Minute (downsampled to 15 Minutes) · ~114 symbols over 2 years*. High-frequency features that expose timing conventions, execution realism, and the gap between theoretical and realized fills.
- `sp500_equity_option_analytics`: *Equities + Options · Daily · S&P 500 over 5 years*. Equity strategies enriched with option-derived features such as implied volatility, skew, and put-call ratios.
- `sp500_options`: *Options · Daily · S&P 500 over 5 years*. Options strategies that foreground payoff accounting, hedging turnover, and the financing assumptions behind net returns.

The next section presents strategy families and sources of edge, using the case studies to make the strategy map concrete.

6.2 Mapping strategies and sources of edge

Most strategy ideas start as narratives: “prices underreact,” “carry is a premium for bearing risk,” or “flows create predictable pressure.” A map turns those narratives into a usable design tool. Before building models or running large backtests, the map should answer two questions in operational terms:

- **What is the strategy’s structure?** What is traded, how often are decisions made, how long is risk held, and which frictions dominate at that cadence?
- **Why should this earn net returns, and why should it persist?** What economic force could plausibly generate returns after costs and constraints, and what would have to remain true for the effect to survive competition and regime change?

The map uses two complementary lenses. **Strategy families** classify ideas by cadence, information source, and the frictions that become first order at that cadence. They identify what must be fixed early. **Sources of edge** are economic narratives about why a premium might exist and what persistence depends on. They guide stress tests and

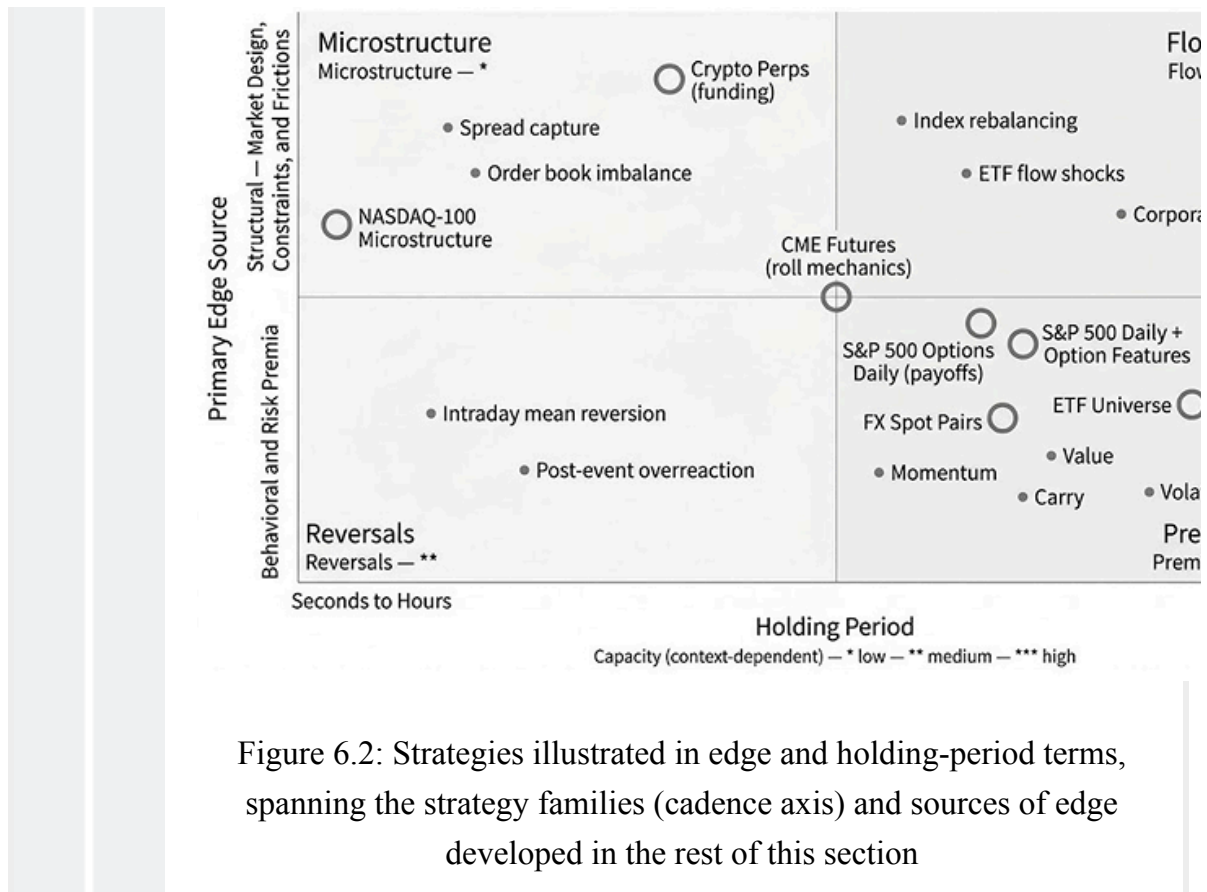
failure-mode hypotheses. A credible story with an infeasible implementation is not a strategy. A feasible implementation without a credible story is a backtest whose failure modes remain uninterpretable.

Box 6.1: Time notions to keep separate

The recurring activity of a trading strategy implies several periods and points in time that are important to define:

- **Decision cadence:** How often the strategy is allowed to change its position. This is the schedule for decisions and rebalancing. A strategy might enter only once per day, for example, but monitor exit signals intraday.
- **Holding period:** How long risk is typically held once a position is entered. It can be fixed (every 5 days, for example) or variable (exit on a signal or risk control).
- **Forecast horizon:** The future interval the model is trained to predict. It does not need to match the holding period when predictions span a given horizon and positions are held until a reason to exit manifests. A strategy can also use multiple models for entry and exit signals, each with a different horizon.
- **Lookback window:** How far back features look when formed at decision time. It determines the historical span available to features. Different model inputs can use different windows, irrespective of holding and forecast periods.





Strategy families as feasibility filters

Families overlap. The same label (for example, “momentum”) can refer to both monthly cross-sectional sorting and intraday trend-following. The family does not uniquely determine the cadence; it identifies what will break first once one is chosen. Use the family whose dominant constraints match the intended cadence, and treat the family as a feasibility checklist: what must be fixed *early enough* to make results interpretable.

Price-based strategies

These strategies generate signals from past prices, returns, or realized volatility, including time-series trend-following, cross-sectional momentum, and short-horizon mean reversion (Moskowitz et al., 2012; Hurst et al., 2017; Jegadeesh and Titman, 1993; Asness, Moskowitz, and Pedersen, 2013). At short cadences, turnover and execution realism become first order. At longer cadences, risk concentrates in episodic drawdowns and regime shifts (momentum crashes are a known stress case; Daniel and Moskowitz, 2016).

To make a price-based idea testable, fix the holding horizon and rebalance cadence, lookback window, and score-to-position mapping. First screens: does performance hold up to modest changes in lookback/rebalance choices, and does it remain plausible under conservative, turnover-aware costs?

Case studies : `etfs` , `us_equities_panel` , and `fx_pairs` illustrate turnover economics, exposure control, and regime sensitivity at daily-to-monthly horizons.

Fundamental and valuation strategies

These strategies trade on cross-sectional differences in valuation, quality, growth, or profitability, typically at monthly to quarterly cadences. Their defining constraint is not speed but the **decision-time meaning** of the inputs. The most common failure mode is accidental use of information unavailable at decision time (see *Chapters 2 and 4* on **point-in-time conventions** and **coverage rules**) . When signals move slowly, the **effective sample size** is much smaller than the row count suggests.

Case studies : `us_firm_characteristics` is the primary anchor for point-in-time discipline, coverage rules, and the interaction between signal definitions and universe construction.

Flow and microstructure strategies

These strategies target small edges per trade using order-flow proxies, quote dynamics, or microstructure-motivated features. As cadence speeds up, the problem shifts from “forecast returns” to “capture microstructure effects under realistic trading rules.” Feasibility depends on timing discipline, execution assumptions, and market access. Precision matters about what is observable at decision time, what latency is assumed, and how fills scale with size relative to depth and volume.

Case studies : `nasdaq100_microstructure` prioritizes timing conventions and execution realism as first-order concerns.

Market mechanics and payoff strategies

These strategies depend on instrument mechanics that form part of the economic payoff: futures roll and term structure, perpetual funding, margin and financing, option payoffs and hedging turnover, settlement conventions, and contract eligibility rules. Testability requires an explicit **return decomposition** and explicit **lifecycle rules**

(contract selection, roll schedule). First screens: component attribution, rule stability, and stress behavior under binding constraints (margin, funding, forced deleveraging).

Case studies : `cme_futures` and `crypto_perps_funding` are mechanics-first examples. `sp500_equity_option_analytics` and `sp500_options` anchor options research, where payoff accounting, surface timing/coverage, hedging turnover, and financing assumptions dominate interpretation.



A note on “machine learning strategies.”

Machine learning is not a family of strategies. It is a modeling and decision-support layer that can be applied to any family. Because ML expands the effective degrees of freedom (more features, more model classes, more hyperparameters), it increases, rather than reduces, the value of a bounded strategy definition and disciplined evaluation.

Defining edge, alpha, and capacity

An **edge** is a repeatable advantage that makes the distribution of **net** trade outcomes favorable under a given strategy, reflecting both the likelihood of gains versus losses and their magnitudes after costs. Two related terms matter:

- **Alpha** is performance relative to a benchmark (an index or factor model) representing alternative opportunities or compensated risk exposures. A strategy that cannot outperform a passive baseline with similar exposures and constraints may not be worth pursuing.
- **Capacity** is the capital that can be deployed before the edge degrades from impact, fees, financing constraints, or opportunity-set limits. Capacity does not determine whether an edge exists, but often determines whether it is economically usable at scale.

Sources of edge as durability filters

Feasibility addresses whether the idea can be tested credibly. Durability addresses what must remain true for the edge to persist. McLean and Pontiff (2016) found that published predictors lose roughly half their predictive power post-publication, partly due to overfitting and partly to arbitrage. The implication: edges that depend on information alone tend to erode; edges that depend on risk tolerance, constraints, or

capacity limits can persist even when widely known. Real strategies often combine multiple sources of edge; the goal is to be explicit about the dominant ones and to test their corresponding failure modes.

A useful organizing framework distinguishes six sources of excess returns based on *why* the opportunity persists (Paleologo, 2025):

Source	Mechanism	Durability	ML4T examples
Risk compensation	Bearing exposures others avoid (tail, volatility, illiquidity risk)	High — persists as long as investors are risk averse	cme_futures carry, sp500_options volatility risk premium
Liquidity provision	Earning a premium for supplying immediacy or absorbing inventory	High — structural demand for immediacy	nasdaq100_microstructure intraday reversals
Funding constraints	Exploiting dislocations when capital is scarce or leverage is restricted	Moderate — episodic, strongest during stress	Short-horizon mean reversion after forced liquidations
Flow predictability	Front-positioning around mandated or rules-based demand (index rebalancing, hedging programs)	Moderate — erodes with crowding but recurs with each event	Index reconstitution trades, ETF creation/redemption flows
Informational advantage	Better inference from public data: cleaner features,	Low to moderate — erodes as competitors	ML-based feature engineering across case studies

Source	Mechanism	Durability	ML4T examples
	faster processing, superior models	replicate methods	
Pure arbitrage	Exploiting identical-asset mispricing across venues or structures	Very low — fleeting, capacity- constrained, operationally intensive	Not represented in our case studies

Table 6.1: Comparing six sources of excess

The following subsections expand on several of these sources and related mechanisms. Most real strategies blend two or more; the table helps identify which ones anchor a thesis and which failure modes to test first.

Box 6.2: Why most published anomalies fail in practice

The academic literature documents hundreds of cross-sectional predictors, yet very few survive as tradable strategies. Three gaps account for most of the attrition:

- **Post-publication decay.** McLean and Pontiff (2016) show that anomaly alphas decline roughly 50% after publication, partly due to overfitting corrections, partly due to arbitrage capital flowing in.
- **Implementation gap.** Academic studies typically use simplified portfolio construction (equal-weight long-short deciles, monthly rebalancing, no costs), which overstates live performance. Turnover, bid-ask spreads, and borrow costs often consume the reported alpha.
- **Detail sensitivity.** Small definitional choices (universe filters, rebalancing timing, winsorization thresholds) can flip the sign of a backtest result. A backtested anomaly is a hypothesis, not evidence; the evidence comes from out-of-sample evaluation under a fixed protocol (Sections 6.5 and 6.6).

The defense is not to avoid published ideas but to treat them as starting points that require independent validation within a researcher's own



Slow adjustment and behavioral responses

Prices may incorporate information gradually due to limited attention, delegated decision-making, or slow portfolio rebalancing, motivating underreaction narratives behind momentum and intermediate-horizon trend effects (Jegadeesh and Titman, 1993).

Early failure-mode tests : Examine drawdown and rebound episodes (momentum crashes; Daniel and Moskowitz, 2016) and check whether performance concentrates under benign conditions but reverses when volatility spikes or liquidity dries up, a pattern that often reveals that alpha partly compensates for state-contingent risk.

Risk compensation

Some returns compensate for bearing exposures others avoid: liquidity risk, tail risk, volatility risk, or balance-sheet-intensive exposures. Persistence can coexist with widespread awareness because the constraint is risk tolerance and leverage capacity, not information.

Early failure-mode tests : Performance during tail episodes when constraints bind (margin/funding shocks, forced deleveraging) and performance conditional on volatility and liquidity proxies.

Market segmentation and limits to arbitrage

Segmented markets and institutional constraints create persistent wedges: funding differences, capital charges, short-selling constraints, inventory limits, and client-flow internalization. The edge persists because the arbitrage is balance-sheet intensive or operationally constrained.

Early tests: Stability under stressed financing and fee regimes, realism of access assumptions, and capacity/crowding behavior.

Box 6.3: Why prediction alone is insufficient

A correct forecast does not guarantee a profitable trade. When 3Com announced the IPO of its Palm subsidiary in 2000, Palm's implied valuation quickly exceeded that of the parent—a textbook mispricing



visible to every market participant. Yet the trade was nearly impossible to execute: Palm shares available to borrow were scarce, borrowing fees spiked to extreme levels, and uncertainty about the spin-off timeline created open-ended funding risk. The mispricing persisted for months despite being obvious. For ML-driven research, the lesson is concrete: before engineering features for a mispricing hypothesis, verify that the short side is accessible, that borrowing and funding costs do not consume the spread, and that the holding-period risk is bounded. A model that identifies unexploitable opportunities is operationally useless regardless of its predictive accuracy.

Mechanical and institutional flows

Predictable demand from mandates, hedging, index rebalancing, roll schedules, or systematic execution programs creates transient or recurrent price pressure, driven by rules and constraints rather than beliefs.

Early tests: Calendar dependence and event alignment, decay under crowding, and execution sensitivity (flow edges often live near the spread).

Index reconstitution is the standard example. When an index provider announces an addition or deletion, passive funds tracking that index must trade by the effective date, creating a demand shock whose direction, approximate magnitude, and timing are publicly known in advance. The edge is not in having private information but in positioning ahead of the mandated flow: predicting *which* securities will be added or removed, and *when* the demand will materialize. The risk is real: holding-period exposure to company-specific losses, potential event cancellations, and crowding as more participants exploit the same signal. As passive investment continues to grow (estimated at 35-40% of U.S. equity market capitalization), these flow events create larger dislocations and attract more competition, illustrating the tension between opportunity size and crowding that recurs across flow-based strategies. In our case studies, futures roll effects in `cme_futures` and ETF creation/redemption dynamics in `etfs` create analogous flow-driven patterns at different cadences.

Information and measurement advantages

Net returns can come from better inference: cleaner definitions, better timing discipline, more reliable labeling, and a more faithful mapping from observables to actions. This is

rarely “free money”; it is often what turns a fragile backtest into a robust implementation within another category.

Early tests emphasize robustness to definition choices and conservative timing, sensitivity to coverage/missingness, and the extent to which incremental signal quality survives realistic costs and constraints.

Using the map

A strategy is well posed when the following questions can be answered:

- **Family and cadence:** Which frictions dominate?
- **PnL decomposition:** Which payoffs generate returns, and which costs are first order?
- **Edge narrative:** What is the primary economic source, and why should it persist?
- **Top failure modes:** Two or three ways the strategy could fail.

Answering these before expanding the search helps focus the next design decision: what to fix in the trading setup, which diagnostics to run first, and which stress tests are non-negotiable. We discuss the trading setup components next.

6.3 Defining the trading setup

Comparability breaks when the evaluation environment drifts without notice: the instrument set changes, the rebalance snapshot shifts, execution timing moves, or costs are treated differently. The **trading setup** is the fixed part of that environment: the invariants the code treats as constant within a research line, and every run automatically records. We can explore aggressively inside those invariants, but changing them defines a new setup and requires bumping the version.

What the trading setup must fix

The trading setup fixes the decision-time snapshot, mapping form, and friction regime, rather than enumerating every lookback or threshold. It should cover the following topics:

Universe and tradability rules

Define eligibility and membership mechanics that make pricing and execution economically meaningful. Specify filters (liquidity, listing venue, contract type), and how listings, delistings, and stale or missing prices are handled. For example, `etfs` need explicit investability filters and rules for ETF launches and closures; `crypto_perps_funding` needs venue and contract eligibility and a rule for how convention changes (such as fees, funding caps, margining) are treated.

Decision schedule and admissible information

Specify the trading snapshot and what is knowable at that moment. This is the guardrail against leakage, so it must be explicit: the timestamp convention, the bar frequency (if any), and any mechanical execution delay. `etfs` might use a weekly or month-end snapshot; `crypto_perps_funding` aligns decisions with the funding timestamp; `nasdaq100_microstructure` requires a fixed bar schedule and a stated observation-to-execution delay because a one-bar shift can change results.

Cadence and horizon feasibility

The cadence choice is intimately linked to costs: at each horizon, what fraction of typical price moves exceeds round-trip transaction costs? This principle determines feasibility before considering any signal.

Each case study includes a setup notebook with a horizon feasibility analysis that shows return distributions across multiple cadences and a cost reference line. These identify three regimes:

- A **hard floor** where costs clearly dominate (for example, `nasdaq100_microstructure` at 15-minute bars)
- A **gray zone** where feasibility depends on signal strength and turnover (for example, `etfs` for daily trading)
- A **comfortable zone** where costs are not the binding constraint (for example, `etfs` with monthly rebalancing)

The cadence decision is made in the following chapters, when signal diagnostics reveal whether features are sufficiently predictive, either on a standalone basis or in a model context, to justify trading at a given frequency.

Score-to-trade mapping

Define how a model's score translates into orders, positions, and exits. This mapping determines trade frequency, holding-period distribution, and where costs and constraints bite. At a minimum, determine the following design parameters to ensure results are comparable:

- **Position state space:** For example, long-only, long/short, or multi-leg (spread/hedge).
- **Entry logic:** For example, rank selection or threshold gating.
- **Exit logic:** For example, time-, signal-, or risk-based exit.
- **Position sizing:** How scores translate into exposures (for example, equal weight, volatility targeting).
- **Order timing:** When orders are placed (at open, intraday window, execution delay).

The specifics will likely evolve, but when they do, the reference for comparison will as well.

Constraints and risk controls in scope

Specify the constraints treated as binding: exposure and leverage limits, concentration rules, and any liquidity or capacity limits enforced. Risk controls that alter trading actions (for example, a de-risking overlay or a kill switch) must be explicit and versioned, as they alter the effective strategy.

Cost model class and components

Early research does not need a perfect simulator, but it does require consistency about which frictions are treated as material. State the cost components included (fees, spreads, slippage/impact, financing or funding, borrow, roll) and how conservative the assumptions are. Later chapters expand the cost model; early screens use conservative, coarse assumptions.

Example (perpetual futures) : in `crypto_perps_funding`, funding is a position-dependent cash flow that is part of the total return stream: a cost when paid and a benefit when received. Because it accrues only when a position is held at the funding timestamp and is settled under venue-specific conventions, the trading setup must specify (i) how trades and position changes are aligned to the funding snapshot, (ii) which pricing and funding formula (including any caps/floors) and fee schedule are

assumed, and (iii) how convention changes are versioned. Otherwise, the same backtest can imply different effective exposures and produce materially different PnL.

Example : The **ETF cross-asset momentum setup (v1)** defines the following invariants:

- **Universe:** 100 cross-asset ETFs with point-in-time eligibility based on trailing annual ADV $\geq$ \$10M. The lenient threshold preserves cross-sectional breadth in early sample years (70–95 eligible ETFs per year). Note that the universe composition has survivorship bias that cannot be fully resolved without historical constituent data, while within-universe eligibility is point-in-time correct.
- **Decision schedule:** Monthly month-end close snapshot with execution at the next-day bar open. This aligns with momentum literature for benchmark comparability. Weekly (5-day) cadence is tested as a variant.
- **Mapping:** Long-only, rank-select top-N, equal-weight, rebalanced on cadence. Long-only because ETFs are expensive to short; equal-weight because it isolates the ranking signal from sizing optimization that would confound evaluation.
- **Constraints:** Fully invested long-only with no leverage, a minimal constraint set appropriate for a pedagogical baseline.
- **Cost model:** Material is a per-share commission (\$0.0035/share, IBKR Pro Tiered) plus tiered half-spread slippage (0.5¢ for the most liquid mega-ETFs, 1¢ for sector ETFs, 2¢ default for thematic and regional ETFs). The horizon feasibility analysis uses a 5–15 bps-per-leg reference cost line to screen cadences and confirms that monthly moves exceed 30 bps round-trip costs in over 90% of observations; daily trading operates in the gray zone where feasibility depends on signal strength and turnover.

The setup notebook writes these choices to versioned YAML artifacts (`setup.yaml`) that downstream chapters consume programmatically. Changing the mapping from long-only to long/short would require `setup_version: v2` and a new baseline checkpoint; parameter changes, such as adjusting top-N from 10 to 20, remain within `v1` .

When a change requires a new setup version

Treat the trading setup as versioned. Bump the setup version on any change that alters the strategy's decision-time meaning or shifts its dominant frictions and constraints.

Parameter tuning stays within a setup; changes to mechanics define a new setup. A change is a new setup version if it changes at least one of:

- **Universe or tradability rules** , for example, eligibility, membership mechanics, instrument scope.
- **Decision schedule or admissible information** , for example, rebalance snapshot, cadence, execution delay, and input availability rules.
- **Score-to-trade mapping** at the level of state space, entry or exit form, sizing normalization, or order timing.
- **Constraint regime** , such as binding exposure/leverage limits, concentration rules, capacity or liquidity constraints, action-changing overlays.
- **Cost model class or included components** , for example, adding funding or roll mechanics, or changing the slippage model in a way that affects feasibility.

To keep this boundary unambiguous, separate **mechanics** from **parameters** :

Same setup version (parameter tuning)	New setup version (mechanics changed)
Threshold 1.8 → 2.0	Rank selection → threshold gating
Top 10% → top 20%	Long-only → long/short
20-day → 60-day normalization window	Adding a stop rule when none existed
Different lookback horizon	Changing rebalance cadence or execution delay
Cost assumption 10 bps → 15 bps per leg	Adding funding/roll as a cost component

Table 6.2: Comparing mechanics and parameters

Results across setup versions can still be informative, but they are not direct competitors within a single research program. The next section defines what counts as improvement within a fixed setup, separating development diagnostics from strategy-level evaluation.



Implementation : 02_case_study_overview inventories the trading setup for each of the nine case studies (universe, decision schedule,



mapping, constraints, cost model) and is the source of the worked examples above.

6.4 Setting objectives and evaluation metrics

A research program needs an explicit definition of “better” before running large experiment grids. Here, “better” means **economic value under a fixed trading setup** : portfolio returns produced by mapping model outputs into positions and applying constraints and costs. This section specifies three important rules:

- Evaluate strategy candidates in **economic terms** , based on a consistent PnL definition
- **Keep metric roles separate** for model and strategy evaluation
- Choose **one primary selection metric for development** and a **separate reporting set** for strategy outcomes

Specific metrics appear in later chapters on features, models, and backtests; here, we establish the framework for how these metric layers interact.

Three metric layers, each with a different role

Confusion in ML-driven strategy research often arises from relying on a single metric to address multiple questions. Treat metrics as three distinct layers, each tied to a specific decision:

- **Model diagnostics** answer a narrow question: can the model learn the label in a way that generalizes across time splits? Typical checks include loss or error, calibration, and stability across folds. If these fail, treat downstream trading results as difficult to interpret because the predictor is unstable. These checks do not require a cost model, position sizing, or portfolio simulation; they serve as an early gate that prevents overinterpreting noise.
- **Signal diagnostics** test whether the model output behaves like a tradable signal under the chosen mapping class. A common choice is the **information coefficient (IC)**, the Spearman rank correlation between predicted scores and subsequent realized returns (see *Chapter 7*). Signal diagnostics treat the output **as a score to**

be traded ; model diagnostics treat it **as a predictor to be trusted** . *Chapter 7* covers both in detail.

- **Strategy outcomes** evaluate the end-to-end process under costs and constraints, in terms of risk and return. Typical reporting includes risk-adjusted returns, volatility and drawdowns, exposure and concentration behavior, and realized turnover (see *Chapter 17*). The failure mode is to use these outcomes as the objective for every small design choice during development, which encourages overfitting to the simulator’s degrees of freedom and will likely disappoint out-of-sample.

A complementary diagnostic, **conformal prediction intervals**, provides calibrated uncertainty estimates without distributional assumptions. *Chapter 11* introduces conformal methods for time-series prediction; here, note that well-calibrated prediction intervals can flag when a model’s confidence is misaligned with realized outcomes, serving as an early warning alongside the three layers mentioned earlier.

Use model diagnostics to assess learnability, signal diagnostics to assess tradability under the mapping, and strategy outcomes to determine whether the declared setup produces economic value. Keep strategy outcomes “late-stage”: when they drive every micro-decision, the risk of overfitting to implementation details rises, masking a robust edge. We now turn to evaluating our target metrics in a time-series context.

6.5 Evaluation protocol for time series

Evaluation answers a simple question: how will a procedure learned from historical data behave on data it has not seen? We estimate this by testing decisions on data unavailable when those decisions were made. The notebook `01_cv_foundations` illustrates these concepts using the `ml4t-diagnostic` library. The core discipline: separate selection from performance estimation, and enforce a fixed decision time so no future information leaks into training or simulation.

Data leakage and estimation bias

Look-ahead bias occurs when any component of the pipeline uses information that would not have been available at the time of the trading decision. It is the most dangerous form of data leakage because it produces spectacular backtest results that cannot be replicated in live trading. Look-ahead bias manifests in several forms in the context of time-series evaluation (in addition to source data problems like survivorship bias or point-in-time errors, as discussed in *Chapters 2 and 4*):

- **Label leakage:** Using future returns in the same observation's features (for example, computing 5-day-forward returns and accidentally including next-day returns in a feature).
- **Standardization leakage:** For derived features, computing inputs on the full dataset, instead of the training set alone. For example, using the mean and standard deviation to compute z-scores from the full data may reveal a distribution shift during the validation period.
- **Threshold leakage:** Setting thresholds using statistics computed from validation or test data. For example, choosing “enter when the z-score exceeds 2.0” after observing that 2.0 maximizes returns on the test period embeds future information in the trading rule.

Every component of the ML pipeline must respect point-in-time constraints: labels (*Chapter 7*), features (*Chapters 8-10*), validation splits, and signal calibration. A backtest that assumes access to information before it was knowable produces fictional performance.

From k-fold cross-validation to time series evaluation

Standard *k-fold cross-validation* trains on, say, 80% of randomly selected observations and validates on the remaining 20%, repeating k times (Kohavi, 1995). This works when observations are IID, which implies they are exchangeable draws from a stable distribution. Financial time series break that assumption in two ways:

- **The live procedure is directional.** In production, at time t , only information available at or before t can be used. A random split almost always trains on timestamps after the validation point, which means it estimates a procedure that cannot be executed in real time. Nearby observations are also dependent due to autocorrelation and slow-moving regimes, so random shuffling inflates apparent performance.
- **Rows are coupled through windows.** Features and labels are computed from lookback and forward windows (rolling volatility, trailing returns, h -day forward returns). Even with chronological splits, overlapping windows across the boundary can leak information.

Time-series evaluation requires split rules that preserve chronology and prevent window overlap, matching a decision-time procedure that could actually be run.

Bergmeir, Hyndman, and Koo (2018) show that standard k-fold CV can produce unbiased error estimates for autoregressive models when residuals are uncorrelated, a condition that often fails when model misspecification leaves exploitable structure in the errors.

Walk-forward cross-validation as the baseline

Walk-forward CV mirrors deployment: fit on a past window, predict on a future window, score, roll forward, and repeat. *Figure 6.3* compares k-fold with walk-forward CV. A minimal CV design:

1. Choose a **training window**.
2. Choose a **validation window** (a future block for prediction).
3. Enforce **decision-time admissibility** at the boundary: training data must end early enough that every training label is already resolved before validation begins.
4. Fit on the admissible training window.
5. Predict on the validation window.
6. Score with the selection metric.
7. Advance by a fixed step and repeat.

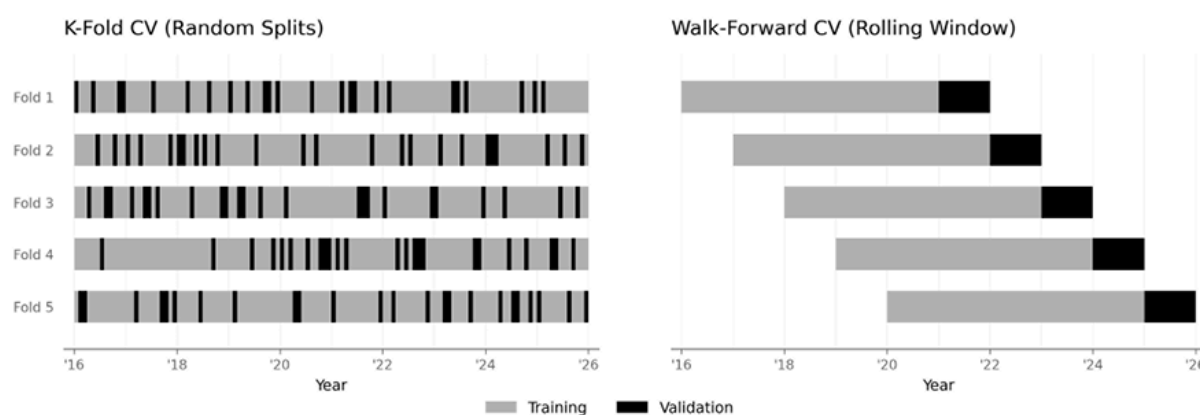


Figure 6.3: Comparing k-fold and walk-forward cross-validation

The protocol can be checked with one question: *At decision time t , which labeled samples were actually available?* A sample indexed at time τ is admissible for training at decision time t only if its:

- **Features** are computable using information available at or before t
- The **label** is already known by t

Example (FX 4-hour bars) : For an FX strategy that decides on 4-hour bars, “the close” depends on the venue and bar construction. If the decision snapshot is the 4-hour bar close, features must be computable from prices available at that close, and the execution assumption must specify whether the trade fires at the same close, the next bar open, or with a fixed delay. A one-bar shift can turn a realistic rule into leakage.

The second condition is a common failure mode. If the label is an h -day forward return, then the label for τ is not known until $\tau + h$. So, training “as of t ” must exclude the most recent h timestamps. This is the **label buffer** (called “purging” by López de Prado, 2018). For example, with 5-trading-day forward-return labels and a five-day trading week, suppose decision time is Tuesday, Feb 10. Then, the latest eligible training timestamp is Tuesday, Feb 3. Interim timestamps are ineligible because their labels have not yet resolved as of Feb 10.

Walk-forward evaluation uses either an **expanding window** (all admissible history, which may have lower variance but may introduce bias from stale data) or a **rolling window** (the most recent L admissible observations, which offers faster adaptation but may produce higher variance).

Retraining frequency

Retrain when new data represents at least 1–5% of the training window, or when out-of-sample performance degrades systematically. With a 10-year window, monthly or quarterly retraining suffices; with 20-day windows, daily retraining captures meaningful change. These design choices materially affect results and must be logged.

Temporal buffers - Label buffer and feature buffer

Walk-forward validation eliminates the grossest form of leakage—training on the future—but leaves a subtler problem. Training observations can “see” validation data through two channels:

- **Labels look forward** : A 20-day return label period begins on the observation date. If a training observation occurs 15 days before validation starts, its label overlaps with five validation days.
- **Features look backward** : A 20-day momentum feature uses data from the past 20 days. If training *follows* validation (as in k -fold or combinatorial schemes; see

Combinatorial methods and path dependence below), a training observation immediately after the validation period computes features using prices from the validation period.

The solution is **temporal buffers**: gaps that prevent labels and features from reaching across the boundary. In practice, always count buffer gaps in **trading days**, not calendar days. The difference is material. For example, a 21-trading-day label buffer in January 2024 spans 30–32 calendar days; a naive 21- *calendar* -day purge covers only 15 trading days, leaving 6 days of label leakage (see `01_cv_foundations`):

- The **label buffer** removes training observations whose labels overlap with those in the validation set. Its size equals the label horizon: for 20-day forward returns, exclude training observations within 20 (trading) days of the validation boundary. For one-day-ahead forecasts on daily data, no buffer is needed. López de Prado (2018) calls this **purging**.
- The **feature buffer** removes training observations whose feature lookback windows overlap with the validation period. This buffer is only needed when training data appear after the validation period, which does not occur in standard walk-forward CV but does occur in k-fold and combinatorial schemes. López de Prado (2018) calls this an **embargo** and proposes 1% of the sample length as a rule of thumb; the appropriate size depends on the strategy’s horizon and the strength of temporal dependence.

Holdout test set and rolling retuning

Walk-forward splits reveal how choices behave when the “fit on the past, predict the future” pattern is repeated. But they do not, by themselves, answer the governance question: *when do we stop selecting and start measuring?*

To make that boundary operational, reserve a **sealed holdout test set**: a final time block that plays no role in any development decision (signal definition, feature selection, hyperparameter tuning, model selection, or reporting choices). The holdout may be inspected only after the pipeline is frozen. During development, we select using the primary development metric computed on walk-forward validation; we report strategy outcomes after freezing the pipeline; and we use the sealed holdout only once for confirmation.

With that rule in place, two distinct evaluation modes are useful:

- **Fixed holdout (tune once, test once)** : Use walk-forward validation on the development period to select a single configuration, freeze the full pipeline (including the score-to-trade mapping and cost/constraint assumptions), and evaluate it once on the sealed holdout to estimate out-of-sample performance.
- **Rolling retuning** : To estimate the performance of a procedure that regularly retunes as new data arrives, split the overall test period into smaller windows and repeat the “fixed holdout” procedure for each window. At each test step, tune using only data strictly prior to the next test window, refit on admissible history, and then score on the next test window. Each test window is used for measurement only. This approach is called **nested walk-forward** because it uses an outer test loop and an inner model-selection loop. Bates et al. (2021) show that confidence intervals computed from nested CV have superior statistical properties. *Figure 6.4* illustrates this procedure with expanding and rolling windows.

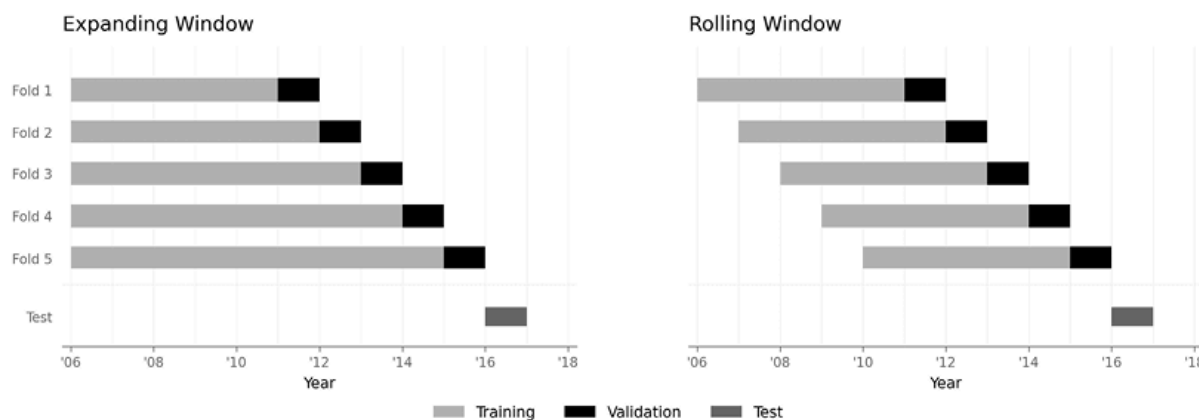


Figure 6.4: Nested walk-forward CV with rolling windows

Nested walk-forward can capture when model configurations change and can be more realistic. For example, with a two-year test period and a five-year lookback, a live strategy would likely retune the model more than once as assumed by the fixed-holdout approach. Nested walk-forward with many feature sets is computationally demanding; parallelized evaluation, incremental retraining, and feature caching are standard optimizations. The key invariant remains: tuning never uses future data relative to the outer test window.

Combinatorial methods and path dependence

A standard walk-forward yields a single sequence of out-of-sample windows: a single historical path. **Combinatorial methods** partition time into blocks and evaluate many train–test combinations, producing a distribution of out-of-sample outcomes rather than a single trajectory. The motivation is **path dependence** : a strategy may appear unusually strong or weak depending on which regimes fall along the single test path.

López de Prado (2018) proposes **Combinatorial Purged Cross-Validation (CPCV)**, which selects k groups of contiguous observations from n total groups, creating $\binom{n}{k}$ distinct train/test configurations. Each configuration applies both label and feature buffers—necessary because validation groups can appear anywhere relative to training blocks.

Combinatorial evaluation is most useful when comparing many variants and seeking a more stable view of selection risk. It does not replace decision-time discipline: overlap leakage must still be prevented using the buffers described above, and each test block must be treated as “future” relative to its paired training blocks. Combinatorial evaluation is computationally expensive— $\binom{n}{k}$ configurations grow quickly—and offers diminishing returns when the time series is short or when most blocks share the same regime. We will discuss CPCV in more detail in *Chapter 17* .

Design commitments

Before running experiments, specify and log the evaluation design:

- **Window lengths:** Training, validation (if used), and test
- **Step size:** How far the splits roll forward each iteration
- **Retraining cadence:** How often the model is refit
- **Label horizon:** The forward window used to define labels, which determines the label buffer size
- **Longest feature lookback:** The maximum lookback across all features, which determines the feature buffer size
- **Test period:** The dates reserved for final evaluation, not used for any development decision

These are design commitments, not tuning parameters. Changing them mid-research defines a different evaluation procedure rather than an “improved” result under the same evaluation. Throughout, we encode these commitments in the `setup.yaml` configuration that each setup notebook writes; downstream chapters convert it to a

`WalkForwardConfig` object for the `ml4t`-diagnostic splitters. This makes the evaluation design both explicit and executable. Logging these choices before experimentation enables reproducibility and makes “the same strategy” operationally well-defined.

Figure 6.5 shows the prediction coverage across the case studies. Coverage varies substantially; this heterogeneity motivates strategy-specific protocol choices, and the figure serves as a reference for understanding the statistical depth available for each case study.

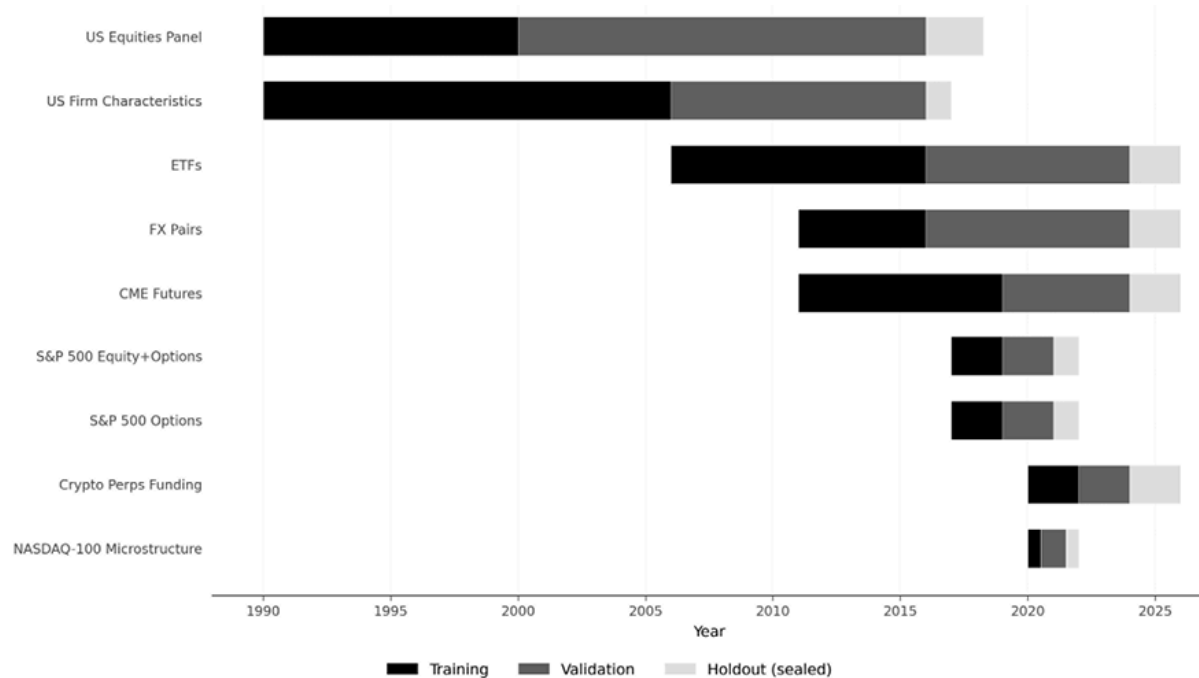


Figure 6.5: Training, validation, and holdout periods for the nine case studies

The notebook `02_case_study_overview.py` shows the trading setups and time-series validation configurations for all case studies; the underlying analysis is in the `01_setup.py` notebooks in each case study directory. Next, we discuss how to start research with a focused baseline.

6.6 Establishing a baseline checkpoint

Before running large experiment grids, we need a **baseline checkpoint**: the smallest runnable specification that answers an early governance question: under the trading setup and evaluation protocol, is there enough stable structure to justify deeper work, or should the setup itself be revised?

Three preflight sanity checks

Before investing in feature engineering and model search, run three checks that catch common failures early. These are not evidence of an edge. They are sanity checks that the checkpoint is well-posed. Here are the three checks:

1. **Timing sanity.** Confirm that the baseline measurement and label can be computed using only information available at the declared decision snapshot, respecting data and model pipeline timelines, and execution delays.
2. **Coverage sanity.** Report how much of the intended universe is actually tradable at each decision time under the eligibility rules, and whether that coverage changes materially across history.
3. **Trading-intensity sanity.** Use a coarse proxy for how often the mapping would trade. The goal is to detect definitions that implicitly demand unrealistic turnover before building a detailed cost model.

If these checks fail, the right response is usually to revise the trading setup conventions (snapshot, eligibility, mapping class) rather than to “optimize around” a brittle definition.

The first reference run is narrow by design

The first reference run produces comparable, interpretable output under the declared setup and protocol. Its inputs (labels, features, and a baseline model) arrive across the next five chapters. Keep it narrow:

- **One label and horizon** . Choose a single forward horizon aligned with the strategy cadence and mapping class. This horizon determines the label buffer required by *Section 6.5* .
- **Compact feature families** . Use a small set of feature families that represent the narrative at the correct cadence. The goal is representative coverage, not exhaustiveness.
- **One simple baseline model class** . Use a stable baseline with minimal tuning. At this stage, the objective is a comparable reference point, not peak performance.
- **One walk-forward design** . Specify training and test window lengths, step size, and the purging rule implied by the label horizon. Reserve the sealed holdout for later confirmation.

The reference run answers a narrow question: Given the declared setup and evaluation protocol, does a simple baseline produce stable, non-pathological behavior without relying on fragile timing conventions or extreme implied turnover?

If the answer is “no,” the next step is often to revisit the setup (cadence, mapping class, tradability filters, or the measurement/label alignment). If the answer is “yes,” the information set and the model class can be expanded in later chapters, while keeping the baseline comparable. The next section addresses the bookkeeping that makes this comparison trustworthy: how to log every run so that search breadth is countable and selection bias is visible.

6.7 Search accounting and run logging

A **run log** is automatic metadata, along with pointers to the artifacts produced by each execution. When the same configuration can be rerun and the same inputs, splits, metrics, and outputs recovered, results can be compared reliably. Otherwise, the discussion of outcomes proceeds without verifying what produced them. The multiplication of research opportunities, amplified by LLM-assisted hypothesis generation (*Chapters 22–24*), further strengthens the case for countable search. A run log supports three practical needs:

- **Comparability** : Performance differences can be attributed to a change in the pipeline only when everything else is held constant and recorded, including the trading setup version, data snapshot, split design, and cost model class.
- **Selection transparency** : Once variants are compared, selection bias enters. We must know how many alternatives were tested, which metric was used to choose among them, and whether any results were used solely for confirmation. Bailey and López de Prado (2014) introduced the Deflated Sharpe Ratio, which adjusts performance metrics to account for the probability of overfitting, based on the number of trials. Applying these corrections requires knowing the trial count, which, in turn, requires logging.
- **Recovery** : A figure, a backtest, or a table can be reproduced from a concrete run identifier, without guessing which notebook cell or intermediate file produced it.

Trial taxonomy

Use a small, consistent taxonomy so the search is countable without being bureaucratic. A workable four-level structure is:

1. **Strategy** . A named trading setup and objective with fixed invariants (universe, decision cadence, position mapping class, and cost model class), for example, `etfs` .
2. **Trial family** . A group of comparable alternatives that share the same setup and evaluation design but differ along one intended axis, for example, baseline signal definitions for the same horizon, or alternative feature sets for the same model class.
3. **Trial** . One fully specified pipeline configuration within a family, including the exact signal definition, feature specification, model class, hyperparameters (if any), and position-sizing rules.
4. **Run** . One execution of a trial tied to a concrete code version, data snapshot, random seed, and compute environment, producing metrics and artifacts.

This taxonomy answers “how many signal definitions did we compare?” and “which run produced *Figure 6.3* ?” without turning research into a document workflow.

Minimum contents of the run log

The goal of a run log is to make results **reproducible, comparable, and gateable** .

There is no single way to organize research. However, the following five categories of records are non-negotiable, while the specific representations may vary:

- **Provenance (what ran):** Strategy setup version, trial IDs, code revision (git hash), timestamp, random seeds.
- **Data and evaluation (what was tested):** Dataset and point-in-time conventions, dev and holdout ranges, split scheme (windowing, step, label, and feature buffers), label horizon, and admissibility constraints.
- **Configuration (how it was produced):** Baseline definition and timing, feature and preprocessing spec, model class and hyperparameters, position mapping, plus risk and cost model parameters.
- **Artifacts (where outputs live):** References to features and labels, predictions, and backtest outputs (positions, trades, PnL, diagnostics).
- **Decision and gates (why it was kept):** Selection metric and rule, reported acceptance metrics, and the holdout gate outcome.

This list is intentionally small. If a field does not affect reproducibility, comparability, or the holdout rule, do not add it by default. Tools such as MLflow and Weights &

Biases automate much of this logging; *Chapter 26* discusses MLOps governance in detail (*Section 26.6*).

One structural way to reduce the effective number of strategies tested is to pre-register the hypothesis: specify the economic thesis, signal definition, and evaluation criteria before running the backtest. Pre-registration does not preclude exploration, but it draws a clear line between confirmatory tests (which count toward the deflation adjustment) and exploratory analyses (which inform future hypotheses). *Chapter 16* formalizes this idea via the Rademacher Anti-Serum, which quantifies the penalty for searching over multiple strategies (*Section 16.7*). The summary draws these commitments together into the research contract that *Chapters 7-20* will carry forward.

6.8 Summary

We have turned a strategy idea into a research program that produces trustworthy evidence under iterative search. We distinguished the live trading loop from the research loop, then used a strategy map, built from families (dominant frictions by cadence) and sources of edge (durability requirements and failure modes), to force early clarity on the strategy's structure and what must remain true for an effect to persist.

Discipline rests on four commitments: a versioned trading setup that fixes the evaluation environment; explicit economic objectives that separate development metrics from reporting outcomes; a time-series evaluation protocol that enforces chronology and decision-time admissibility with a sealed holdout; and a compact baseline checkpoint with run logging that keeps the search auditable, reproducible, and countable.

The next chapter is devoted to defining our learning task.

7

Defining the Learning Task

A machine learning model can only be as good as the learning problem you define. Before you select an algorithm, you need a label that captures an outcome relevant to your strategy, preprocessing conventions that keep inputs comparable across trials, and an evaluation framework that can distinguish genuine signal from artifacts of timing, overlap, or search. This chapter builds that scaffolding.

Chapters 8–10 provide the candidate features (financial, model-based, and text-derived) that this framework evaluates. The definitions, diagnostics, and decision gates built here govern every candidate in that sequence.

After completing this chapter, you will be able to:

- Build a split-aware preprocessing pipeline that produces stable, auditable inputs for label and feature computation.
- Define execution-consistent labels and diagnose their overlap and implied trading intensity.
- Evaluate feature–label bundles using fold-aware diagnostics for continuous and binary targets.
- Apply feasibility screens to filter out candidates who cannot survive implementation costs.
- Account for selection bias by defining searched sets, separating exploration from confirmation, and adjusting fold-level summaries.

- Apply mechanism plausibility checks to distinguish stable mechanisms from confounded proxies.

Throughout, diagnostics are computed within each walk-forward fold and aggregated across folds to surface instability that pooled statistics hide.

7.1 Data preprocessing and encodings

Data preprocessing turns validated datasets into stable inputs for label and feature computation. Earlier chapters handled point-in-time correctness, corporate actions, and calendar alignment; *Chapter 6* defined tradeable universes. This section covers the steps that are new at the feature-engineering stage: split-aware fitting, outlier treatment, representation choices, and missing-data protocols.

Box 7.1: Protocol-safe definitions and computation

This chapter treats labels, features, and preprocessing transforms as data products. To keep results comparable across iterations and to prevent silent leakage, every run must satisfy five invariants:

- **Observability** : Every raw input must be observable under the trading setup at decision time. Reporting lags, update times, eligibility rules, and calendar conventions are part of the definition.
- **Train-only fitting** : Transforms that estimate parameters from data must be fit using only the training portion of each walk-forward split.
- **Overlap-aware evaluation** : Overlapping label horizons induce dependence. Handle this using a



split design (*Chapter 6*) and report overlap diagnostics alongside the metrics.

- **Versioned metadata** : A “feature” or “label” is a fully specified recipe. Log the definition with the run record; changes create new trial families.
- **Auditable masks** : Eligibility masks are part of the learning problem. Store the mask definition (not just the resulting rows) and report effective sample sizes by fold and asset group.

Split-aware preprocessing

Any preprocessing step that estimates parameters from data (means, standard deviations, quantiles, imputed values) must be fit only to the training split. Fitting on the full dataset before splitting leaks future information into the training representation, inflating performance.

In walk-forward evaluation (*Chapter 6*), this means refitting the preprocessor at each fold boundary using only data available up to that point. Steps that are **stateless** (dropping rows with invalid values, applying a fixed unit conversion, computing a boolean calendar flag) do not require split-aware fitting, but anything that computes a summary statistic from a population of observations does.

When in doubt, treat the step as fitted . Record which preprocessing steps are fitted and which are stateless in the pipeline configuration.

Ensure that the same pipeline object handles both training and inference, so that no transform is accidentally omitted or double-applied (*Box 7.1* , invariant 2). See `@2_preprocessing_pipeline` for a `SplitAwarePreprocessor` that enforces train-only fitting and shows the consequences of leakage: test mean 0.110 under train-only fitting versus 0.101 with full-panel fit.

Outliers and heavy tails

The first distinction is between **invalid** and **rare** . Invalid observations violate domain constraints (negative volume, impossible timestamps, or crossed bid-ask quotes) and should be removed. Single-interval spikes that revert immediately are often data errors; field-aware spike checks that compare the magnitude of the change and reversion pattern against the field's distribution help separate errors from genuine moves.

Valid observations that fall in the tails require a different treatment. Aggressive truncation discards real information; the goal is to limit the influence of extreme values on downstream transforms without eliminating them. **Winsorization** or clipping at a fixed percentile prevents single-point blowups during scaling and normalization. **Robust scaling** (centering on the median and dividing by the interquartile range) is generally more stable than mean-variance standardization when the distribution is skewed or leptokurtic. `02_preprocessing_pipeline` applies winsorization and domain filters to the US Equities panel, with before-and-after diagnostics.

The important exception: when the label itself targets extreme moves or barrier events, clipping returns removes precisely the outcomes the model is meant to learn. Preserve tails when tails are the prediction target.

Scaling and representation choices

Representation is part of the definition of a learning task. For level-like series (prices, yields, spreads), models learn more reliably from changes (returns or differences) than from raw levels. Stationarity diagnostics and the transformations that enforce it are covered in *Chapter 9* ; return calculations for labels follow in *Section 7.2* .

Risk scaling (dividing by realized volatility) stabilizes features and labels across regimes. Treat the volatility estimate (its definition and lookback) as a fixed part of the representation rather than a parameter tuned between trials.

Ranks and percentile ranks are often the most robust encoding for cross-sectional features because they reduce sensitivity to outliers and vendor-specific units. Specify whether you rank across assets at each timestamp (cross-sectional) or within an asset's history (time-series percentile), and keep that choice stable.

For **categorical encodings**, use one-hot encoding for low-cardinality fields, ordinal encoding when ordering is meaningful (for example, credit rating tiers), and hashing or learned embeddings for high-cardinality identifiers. Fit the encoder on the training split.

Handling missing data

Missing values are not one problem but three. Your response depends on *why* the data is missing and what your model can accept:

- **Missing values as noise**. Values are absent for reasons unrelated to the asset, the time, or the value itself (random gaps). If your model requires complete inputs, fill in a robust default (for example, the cross-sectional median). If your model can handle missing values, you can often leave them missing.
- **Missing values due to observed coverage rules**. Missingness is explained by fields you do observe (for example, vendor coverage depends on size, exchange, region, or listing age). Either impute from the observed fields and add a “was missing” indicator, or keep the missing values and rely on a model that can handle missingness and the indicator.

- **Missing values that are themselves informative** . Absence is linked to the unobserved value or an unobserved driver (for example, non-reporting when metrics are poor; sparse prints precisely when liquidity is low). Here, “missing” can be signal or bias. Preserve it explicitly (using an indicator or a separate “not reported” category), and treat imputation as a modeling assumption, not a default.

Two practical defaults include

- If the model requires complete inputs, impute robustly and add a missingness indicator
- Use models that handle missingness natively (for example, tree-based methods, see *Chapter 12*)

Avoid silent imputations that make missingness disappear without a trace.

Table 7.1 lists some dataset-specific considerations:

Dataset	Key preprocessing concerns
ETF Universe	Adjustment artifacts, ticker changes, coverage gaps
Crypto Premium/Perps	Funding rate alignment, liquidation outliers, delistings mid-panel
CME Futures	Roll-adjusted continuity, vacation calendar alignment
FX Pairs	Weekend gaps, rollover liquidity, timezone conventions
US equities	Survivorship bias, split/dividend adjustments, penny-stock filtering
Firm Characteristics	Pre-cleaned panel; missingness patterns and structural breaks

Dataset	Key preprocessing concerns
NASDAQ-100 Bars	Session boundaries, auction prints (pre-cleaned by AlgoSeek)
S&P 500 Bars	Index rebalancing effects, consistency with ETF universe
S&P 500 Options	Moneyness/expiry filtering, Greeks checks, alignment with underlying

Table 7.1: Case study preprocessing concerns



Implementation :

- `01_data_quality_diagnostics` summarizes distribution characteristics across all datasets, including tail statistics that inform the choice between scaling methods.
- `02_preprocessing_pipeline` reports coverage by column, asset, and time period, and shows cross-dataset alignment.

The next section turns to engineering labels.

7.2 Label engineering

Labels define the learning task: what outcome you want to predict, over what horizon, and when that outcome becomes known. A label that is even slightly misaligned (starting or ending at an untradeable price) can create “predictability” that disappears in live trading. Label definitions must be **execution-consistent** and evaluated under the time-series protocol from *Chapter 6* ; see also *Figure 7.1* .

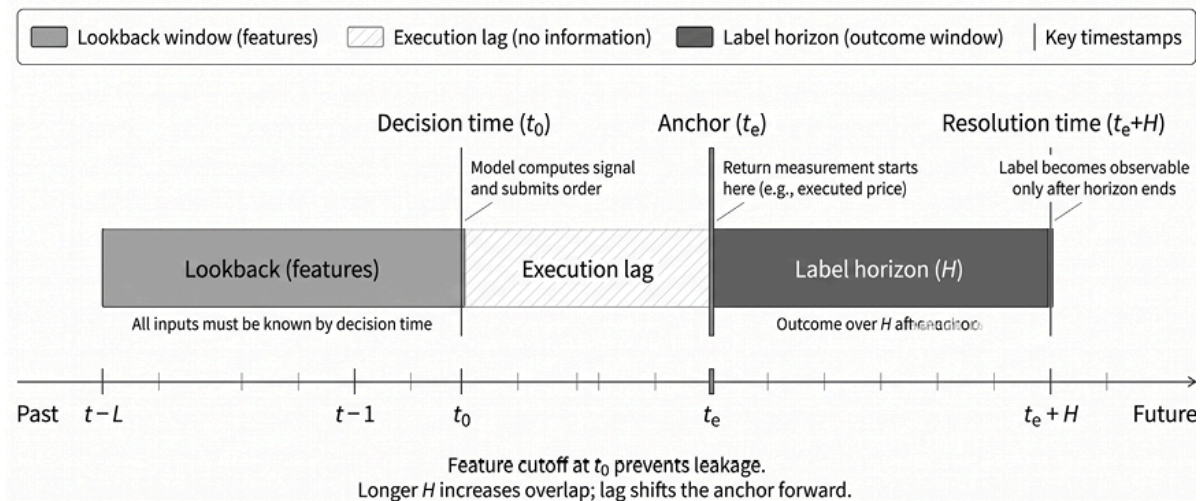


Figure 7.1: Label and Feature Horizons

Let t denote the decision time, t_e the anchor at which the position is established, a the asset, and $p_{t,a}$ the tradable price at t ; every price in a label formula must be tradable under the chosen execution assumption. The horizon H counts forward bars from t_e to the resolution time $t_e + H$, in time or volume-related units (*Chapter 3*). `03_label_methods` demonstrates the methods in this section.

Execution conventions

The execution convention determines which prices are tradable at the start and end of the label window. For bar data, two conventions dominate:

- **Close-to-close** : The signal is generated at the close of bar t ; the return is measured from $close_t$ to $close_{t+H}$. This is the simplest convention and the default in most academic work.
- **Next-open-to-open** : The signal is generated at the close of bar t , but the position is entered at the open of bar $t + 1$ and exited at the open of bar $t + H + 1$. This reflects the fact that most end-of-bar signals cannot be executed at the closing price.

The difference is not cosmetic. On daily equity data, the close-to-close label can differ from the next-open-to-open label by 50–100 basis points per trade in either direction, depending on overnight moves and opening gaps. The effect is large: over the past 30 years, nearly all U.S. equity market returns have accrued overnight rather than intraday (Glasserman et al., 2025).

Choose the convention that matches your execution assumption, document it, and keep it consistent between label computation and backtest PnL. `03_label_methods` quantifies the close-to-close versus next-open gap on SPY for a 21-day horizon.

Fixed-horizon labels

The default label is a forward return computed over a fixed number of bars H . Two common return conventions are:

- *Simple (arithmetic) return* :
$$r_{t,t+H,a}^s = \frac{p_{t+H,a}}{p_{t,a}} - 1$$
- *Log return* :
$$r_{t,t+H,a}^{log} = \log\left(\frac{p_{t+H,a}}{p_{t,a}}\right)$$

For small moves, the two are nearly identical. Log returns are additive across periods; simple returns align with PnL arithmetic. Choose one, use it consistently, and record the choice.

Fixed-horizon labels align best with strategies that act on a stable cadence with a roughly fixed holding period, whether you are predicting a continuous value or a discrete direction. The horizon need not be measured in calendar time: you can define H in bar counts over volume bars or information bars (*Chapter 3*), so the horizon adapts to activity rather than the clock. The outcome is always known at $t + H$, which simplifies overlap analysis and fold construction.

The key decision is whether to train on a continuous metric or discrete categories. Many long/flat/short strategies model continuous returns and discretize model scores using a threshold to generate entries.

Continuous targets

The most common continuous target is the raw forward return $r_{t,t+H,a}$ itself. Alternative representations include **transformations** that are robust to outliers, such as (percentile) ranks relative to the cross-section or an asset's own history, **excess returns** relative to a peer group, a benchmark, or a financing series, and risk measures such as realized volatility. The choice depends on the task: directional prediction, ranking, or risk estimation.

Discrete targets

Discrete labels are useful when small returns are economically irrelevant relative to costs, when you want to learn a directional or event-like outcome, or when your evaluation and model selection benefit from classification metrics:

- **Binary labels** : The simplest discretization flags a directional outcome: set $y_{t,a} = 1$ if $r_{t,t+H,a} > \tau_t$, else 0 . Common variants include flagging extreme returns (top decile, meaning values above the top 10 percent of historical returns), volatility spikes, or drawdown events.
- **Multi-class labels** : A common three-class construction assigns $+1$ (long), 0 (neutral), and -1 (short) based on upper and lower thresholds:

$$y_{t,a} = \{+1 \mid r_{t,t+H,a} > \tau_t \mid 0 \mid r_{t,t+H,a} \leq \tau_t \mid -1 \mid r_{t,t+H,a} < -\tau_t\}$$

The threshold τ_t controls two things simultaneously: the **magnitude** of the event being labeled and the **base rate** (fraction of positives). A threshold that is too tight produces a near-50/50 split and labels noise; one that is too wide produces a rare-event problem and implies a trading frequency that may be infeasible. Choose thresholds that are economically meaningful and consistent with your turnover and cost budget. Three rules cover most cases:

1. **Fixed absolute threshold** : Set τ to a constant (for example, 1%). Simple and interpretable, but the implied base rate drifts with volatility.
2. **Volatility-scaled threshold** : Set $\tau_t = k \hat{\sigma}_{t,a} \sqrt{H}$ or $\tau_t = k \widehat{ATR}_{t,a}$, where the volatility estimate uses past-only inputs. This stabilizes the base rate across regimes.
3. **Percentile thresholds** : Time-series percentiles set τ_t to be the rolling percentile of the asset's recent return distribution (for example, the 75th percentile of $|r|$ over the past 252 bars), adapting to asset-specific volatility. Cross-sectional percentiles rank all assets by forward return at each t and assign labels by quantile membership (for example, top quintile = $+1$, bottom = -1). Cross-sectional labels have the useful property that *class proportions are stable by construction*. Time-series percentiles must be estimated within the training fold; global percentiles leak information about the future distribution.

Most multi-asset strategies use cross-sectional percentiles; single-asset strategies favor volatility-scaled or time-series percentile thresholds. Treat threshold selection as part of the label definition.

Variable-horizon labels

Fixed horizons are simple and well-behaved, but many real trading problems do not have a single natural horizon.

Adaptive horizons via trend scanning

When the appropriate horizon is uncertain, **trend scanning** provides an adaptive alternative: for each observation, it evaluates candidate horizons (typically 5–60 bars) and selects the one with the highest t-statistic for a linear trend fit (López de Prado, 2018).

The adaptive horizon introduces **selection bias** : by picking the “best” horizon for each observation, trend scanning systematically selects extreme outcomes that may not persist outside the sample. You can mitigate this by applying **Bonferroni correction** to the selected t-statistic (dividing by the number of candidate horizons) or by constraining the horizon range to match strategy turnover constraints. *Section 7.4* generalizes this per-observation correction to the search-level problem of evaluating many candidates.

Because the prediction horizon varies across observations, you cannot compute a single IC. Report the distribution of selected horizons alongside performance metrics, and verify that the distribution is stable across folds.



Implementation : `03_label_methods` demonstrates trend scanning on SPY using 5–20-bar windows and compares raw versus Bonferroni-corrected t-statistics.

Event-style labels

Event-style labels let the price **path** determine when the outcome resolves. The broader risk-management context—stop-loss design,

position sizing, execution—is in *Chapter 20* ; here, the goal is a precise, auditable label definition.

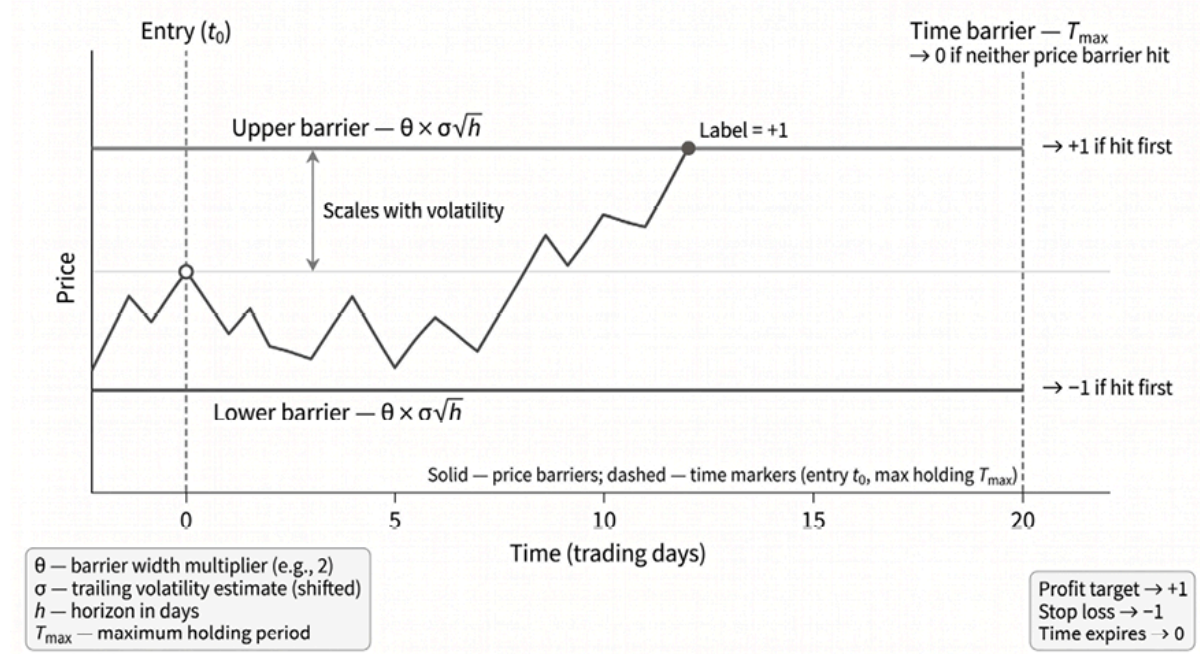


Figure 7.2: Triple barrier labels

The standard construction is the **triple-barrier label** (López de Prado, 2018). Starting from the tradable price at t , define an upper barrier $\pi_t > 0$, a lower barrier $\ell_t > 0$, and a vertical barrier at $t + H_{\max}$. The cumulative return from the entry at t to any subsequent bar u is $r_{t,u,a}$. Assign the event label as follows:

- $+1$ if $r_{t,u,a}$ first reaches $+\pi_t$
- -1 if $r_{t,u,a}$ first reaches $-\ell_t$
- 0 if neither barrier is hit before $t + H_{\max}$

Unlike fixed-horizon labels, where the outcome is always known at $t + H$, event labels resolve at a variable **resolution time**, namely the moment a price barrier is hit, or $t + H_{\max}$ otherwise. We denote this as t^{res} when we need to refer to it explicitly—it becomes the key quantity in the overlap analysis later in this chapter.

With bar data, when both barriers are crossed in the same bar, specify a resolution rule and log it: for example, consider the event a loss (conservative) or exclude ambiguous bars.

Barrier widths can be set using the same volatility-scaling logic as threshold design, with asymmetric multipliers when the strategy has directional conviction.

Maximum favorable and adverse excursion diagnostics

Barrier widths are not arbitrary. They shape class balance, time to resolution, and the amount of label overlap. A practical way to calibrate them is to examine **the maximum favorable and adverse excursions (MFE / MAE)** over the candidate holding period:

- MFE measures the best return reached before the event resolves
- MAE measures the worst drawdown over the same window

Together, they show how far prices typically move for and against a position before a barrier is hit or the horizon expires. These diagnostics help set profit-taking and stop-loss barriers at levels that are economically sensible rather than arbitrary:

- Narrow barriers tend to resolve quickly but can be dominated by noise
- Wide barriers increase time to resolution, reduce the number of usable observations, and create more overlap across labels

In practice, the goal is to choose widths that yield a workable class balance, plausible upper- and lower-barrier hit rates, and reasonable resolution-time distributions.



Implementation :

`04_maximum_favorable_adverse_excursion` demonstrates MFE/MAE visualizations, barrier calibration, and validation using hit fractions and resolution-time distributions.

Overlap, sample dependence, and effective sample size

Overlap is structural: adjacent H -bar labels share most of the same price increments, and event-style labels overlap irregularly because resolution times vary. Any price shock affects every label alive at that moment, creating mechanical dependence.

A label generated at time t is alive from t until its resolution time t^{res} — the interval $L_{t,a} = [t, t^{res}]$. The **concurrency** at bar u , denoted $c(u)$, counts how many labels are alive at u . The **average uniqueness** of a label summarizes its overlap:

$$w_{t,a} = \frac{1}{|L_{t,a}|} \sum_{u \in L_{t,a}} \frac{1}{c(u)}$$

where $|L_{t,a}|$ is the number of bars in the label's lifetime. A uniqueness of 1.0 means no overlap; values near $1/H$ indicate near-maximal overlap.

The **effective sample size** is approximately $N_{eff} = \sum_{t,a} w_{t,a}$. For fixed-horizon labels sampled at every bar, $N_{eff} \approx N/H$ (ignoring cross-section and time-series correlation that may further reduce the effective sample size).

As a concrete example, `03_label_methods` measures uniqueness for SPY with $H = 21$ over roughly 10 years: 248,436 nominal observations

collapse to an effective sample size of about 11,830, so confidence intervals based on the nominal count are roughly $4.6 \times$ too narrow.

Standard errors should use N_{eff} , not N .

Overlap creates three distinct problems:

1. Training and test labels that share price increments leak information across the split.
2. Gradient updates and evaluation metrics overweight high-concurrency periods.
3. Serial dependence in labels inflates apparent statistical significance.

The **time-series protocol** (*Chapter 6*) addresses problem 1 through purge gaps; sample weighting addresses problem 2; and reporting N_{eff} (or block-bootstrap standard errors) addresses problem 3.

Dealing with overlap

Four complementary strategies address overlap:

- **Protocol-correct splits** are non-negotiable: leave at least H_{max} bars between training and validation/test windows (*Chapter 6*).
- **Sample weighting** assigns a uniqueness weight $w_{t,a}$ to each observation, so that high-concurrency periods do not dominate loss or metrics.
- **Subsampling** reduces overlap by sampling every H bars or starting a new label only after the previous one resolves.
- A more data-efficient alternative is the **sequential bootstrap** (López de Prado, 2018). Instead of drawing each index with equal probability, the sequential bootstrap updates selection probabilities after every draw so that observations with higher expected uniqueness, that is, with less overlap with the labels already selected, become more likely to enter the sample. Intuitively, labels that

overlap heavily with the current sample contribute less new information and are less likely to be drawn, while labels from less crowded periods are more likely to be selected. The result is a resampled dataset with lower effective redundancy, even though the original labels may overlap substantially.

Weighting retains all data; subsampling is simpler but discards information. Most setups combine at least two of these four.

Label diagnostics before modeling

Before training, verify labels are correctly defined and stable. Inspect distribution and tail behavior by time and asset group. Outliers often signal broken adjustments. Track base rates over time (class fractions should be stable across regimes), examine resolution-time distributions for event labels, and compute implied trading intensity. If trading intensity is infeasible, revise the label before modeling.

Close each label definition with a short audit record: anchor convention, horizon definition, resolution-time rule (including bar-data tie-breaking), overlap summary, and base-rate summary. These fields keep trials comparable under the *Chapter 6* research ledger (see `03_label_methods` and `04_maximum_favorable_adverse_excursion` for reusable diagnostics).

Meta-labels for confidence-weighted sizing

A different way to structure the labeling problem is to **decompose direction and confidence** into two stages (López de Prado, 2018). A primary model (often a simple rule or an existing strategy) generates entry signals; the secondary label records only whether each signal was profitable after costs. A separate **meta-model** then learns *when* the

primary signal is trustworthy, and its calibrated probability is mapped to a position size, for example through a sigmoid that scales conviction into a fraction of the maximum bet.

`03_label_methods` includes a self-contained illustration: a 20-day-momentum signal on SPY is meta-labeled by its forward-return outcome, and a sigmoid bet-size mapping converts the confidence score into a position. The case studies in this book do not adopt this two-stage structure: each case study trains a single model on a single label horizon and sizes positions via an allocator (*Chapter 17*). The technique is most useful when a directional model is already in place and the goal is to filter weak signals rather than re-engineer the entry rule.

Label diagnostics here focus on *the quality of definitions* ; the next section shows how to evaluate the relationship between a label and a candidate feature.

7.3 Univariate feature-label evaluation

The first layer of evaluation is univariate feature–label screening: each candidate is assessed against the label one at a time. Preprocessed inputs and execution-consistent labels are described in *Sections 7.1–7.2* ; candidate features appear in *Chapters 8–10* . The screens apply in rough order:

1. **Correctness** : Can you trust the definition at decision time?
2. **Association** : Does the feature carry information about the label?
3. **Shape** : Is the relationship compatible with a plausible mapping from signal to positions?
4. **Feasibility** : Could anything implementable survive the trading setup's cost and capacity constraints?

A feature must clear each screen to advance to the next. All diagnostics are computed within each fold and aggregated across folds: medians, interquartile ranges, and worst-fold views rather than a single pooled statistic.

Univariate triage is necessary but not sufficient. A feature that looks promising in isolation may duplicate another candidate, and a feature with weak marginal association may become valuable once conditioned on the right companion. Multivariate model selection (*Chapters 11–12*) makes the decisive judgments. *Section 7.4* addresses the multiple-testing problem that arises when many candidates pass screening.

Correctness screens - The non-negotiable first pass

Before evaluating predictive power, verify that the feature and label are usable under the stated protocol. Run these checks on each feature–label bundle:

1. **Coverage** : What fraction of eligible (asset, decision-time) pairs have non-null feature values? Sparse coverage changes the effective sample size and may indicate stale or lagged inputs.
2. **Timing and lag consistency** : Verify that the feature uses only information available at the time of decision. Check reporting lags for fundamental data, publication times for third-party signals, and update schedules for derived quantities.
3. **Mask alignment** : Confirm that the eligibility mask is applied identically to feature computation, label computation, and evaluation. Misaligned masks are a common source of leakage.
4. **Staleness** : For features that update infrequently (for example, quarterly fundamentals), check that the feature value changes at

appropriate intervals. A feature that never updates may be dominated by asset-specific intercepts rather than a time-varying signal.

These checks do not establish economic value; they prevent the costliest research failure: promoting a definition you cannot trust. See `05_signal_evaluation` for reusable diagnostics of correctness and coverage applied to the case-study ETF universe.

Evaluating continuous labels

For continuous labels (forward returns, excess returns, realized risk targets), the workhorse diagnostic at triage is the **information coefficient** (**IC**): the cross-sectional association between a feature observed at decision time and a label that resolves later.

The Information Coefficient

The default is **rank IC** (Spearman), which equals the Pearson correlation of cross-sectional ranks:

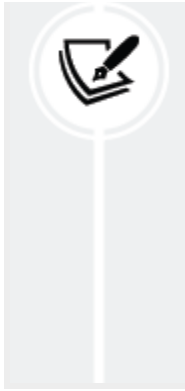
$$IC_t := \text{corr}_a(\text{rank}(x_{t,a}), \text{rank}(y_{t,a}))$$

where $x_{t,a}$ is the feature value for asset a at time t , $y_{t,a}$ is the label anchored at t , and $\text{corr}_a(\cdot)$ denotes correlation across assets at time t . Computing IC_t for each decision time yields an IC time series $\{IC_t\}_t$.

IC is a learnability diagnostic for the definition bundle (feature, label, masks) under the evaluation protocol. It is not, by itself, a claim about tradable performance.

Rank IC versus Pearson IC

Rank IC is the conservative default: it aligns with ranking-based portfolio construction and is less sensitive



to heavy tails and nonlinear monotone relationships. Pearson IC is appropriate when you care about linear association on raw values—for example, a signal that will enter a linear forecast where scale matters. Keep the representation consistent: if you store a feature as ranks, evaluate it with rank IC.

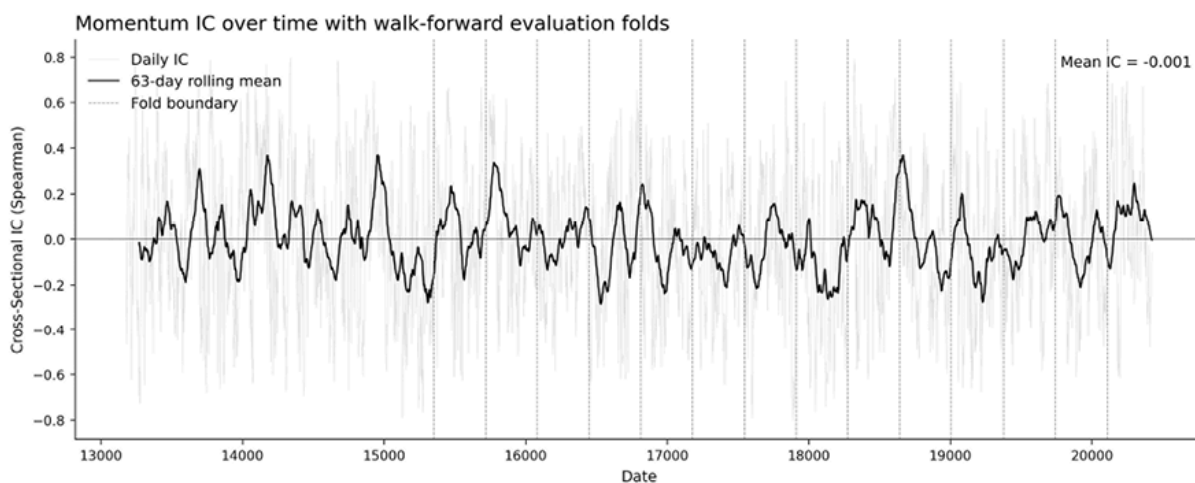


Figure 7.3: IC time series with fold boundaries

Figure 7.3 shows IC_t over the full sample period. Vertical dashed lines mark fold boundaries. A centered rolling average highlights the trend. Two patterns to watch for: (i) IC that concentrates in a single episode—the feature may be exploiting a transient shock or protocol artifact; (ii) IC that flips the sign across folds—the feature’s usefulness depends on state variables not yet modeled.

Fold-level summarization - ICIR and sign consistency

Because IC_t observations are not IID, overlapping horizons induce serial dependence, and cross-sectional universes share common shocks—treat the fold as the unit of summary. Within each fold k , compute a fold mean $IC^{(k)}$ and a dispersion estimate $\sigma^{(k)}$. Summarize across folds using the

median and interquartile range (IQR) of $IC^{-(k)}$ plus a worst-fold view (see also *Figure 7.3*).

If you want a compact stability ratio, compute a fold-level IC information ratio:

$$ICIR^{(k)} := \frac{IC^{-(k)}}{\sigma^{(k)}}$$

and report it by fold rather than as a single pooled statistic. **Sign consistency** —the fraction of folds where $IC^{-(k)}$ shares the sign of the overall median—is the key stability gate. A momentum feature might show a mean IC of 0.04 across five folds but achieve 0.08 in two trending folds and -0.02 in three mean-reverting folds—the aggregate masks a regime switch that would devastate a static allocation. In the ETF universe, 21-day momentum has a pooled IC of 0.001 (near zero) but a fold-level mean IC of 0.064 with ICIR of 0.79 and 75% of folds positive—the pooled statistic hides a signal that is real in most regimes but cancelled by a few adverse episodes (see `05_signal_evaluation`).

IC across horizons and rebalancing frequency

Computing IC across a grid of forward-return horizons reveals the signal’s useful life. A signal that peaks at 5 days and halves by 10 should not be held for a month; one flat from 5 to 21 days offers rebalancing flexibility. Report the IC decay curve alongside fold-level statistics (see `05_signal_evaluation` for the ETF momentum signal).

Inference at triage

If you use inference, apply it to fold summaries, not daily IC_t : (1) **block bootstrap** within each fold, with block lengths reflecting horizon overlap;

(2) **within-time permutation** that shuffles the asset–label assignment within each cross-section, preserving cross-sectional dependence while breaking feature–label pairing. Treat p-values as descriptive at this stage; *Section 7.4* covers multiplicity adjustment.

`05_signal_evaluation` demonstrates walk-forward fold construction and reports per-fold IC, spread, and monotonicity. See `06_ic_inference` for HAC-adjusted inference and block bootstrap confidence intervals applied to IC time series. For the 21-day ETF momentum IC, HAC inflates the naive standard error by $2.54\times$ and effective sample size drops from 4,989 to 773, an 85% efficiency loss that anchors minimum-track-record planning.

Box 7.3: IC and portfolio performance

Under simplifying assumptions, Grinold’s Fundamental Law of Active Management (1989) suggests an approximate relation between forecast skill and portfolio performance:

$$IR \approx IC \times \sqrt{B}$$

where IC measures forecast skill and B is the effective number of independent bets (“breadth”). The approximation is not a performance forecast—bets are rarely independent, and a universe of 100 ETFs with shared sector exposure may offer effective breadth of 20–30. The triage lesson: small but persistent ICs matter when breadth is genuinely large; peak IC is less informative than stable IC across folds.

Kendall’s τ and mutual information

Kendall's τ and **mutual information (MI)** measure different kinds of dependence. Kendall's τ is a **rank-based measure of association**: it assesses whether pairs of assets are consistently ordered by the signal and the realized outcome. More formally, it compares the number of *concordant* and *discordant* pairs, so it is interpretable as a measure of **monotone association** . Like Spearman's rank correlation, it is insensitive to scale, but it is often more stable in very small cross-sections ($n < 30$) because it is built directly from pairwise orderings rather than rank moments. In practice, τ is usually smaller in magnitude than Spearman for the same relationship, so the two should not be compared numerically as if they were interchangeable.

Mutual information (MI) is different: it is an **information-theoretic measure of dependence** , not a rank correlation. MI asks how much knowing the signal reduces uncertainty about the outcome. As a result, it can detect **nonlinear** and **non-monotonic** relationships that Kendall's τ , Spearman's, or Pearson's may miss. However, MI is harder to estimate reliably in finite samples, typically requires discretization or other density-estimation choices, and is less directly interpretable than a rank IC. It is also nonnegative, so unlike Kendall's τ , it does not indicate direction.

For routine triage of large cross-sections with a monotone prior, rank IC remains the default. Use **Kendall's τ** when cross-sections are small, and you want a robust monotone dependence check based on pairwise ordering. Use MI only when quantile plots or other diagnostics suggest a clear nonlinear or non-monotone pattern that rank-based measures miss.

`05_signal_evaluation` computes cross-sectional IC at multiple horizons and visualizes the resulting time series.

Discrete labels - Evaluating features as classifiers

For binary labels with $y_{t,a} \in \{0,1\}$, evaluate whether a feature can act as a score that separates positives from negatives. As with continuous labels, all metrics should be computed fold-by-fold, respecting masks.

Threshold-dependent evaluation

Given a score threshold or top- k action set, report precision, recall, and specificity. In trading terms, **false positives** are costly entries (round-trip costs plus adverse move); **false negatives** are foregone gains.

Threshold-free metrics

Start with the **Receiver Operating Characteristic** (ROC) area under the curve (AUC), which measures how well the feature ranks positives above negatives across decision thresholds, and the **Precision-Recall** (PR) AUC. PR AUC is often more informative when the positive class is rare because precision directly reflects how many predicted positives are true positives.

Under strong **class imbalance**, a feature can have a high ROC AUC while still producing many false positives relative to true positives at practically relevant thresholds, which the PR curve makes more visible. Compute both metrics fold-by-fold and report the median and inter-quartile range (IQR).

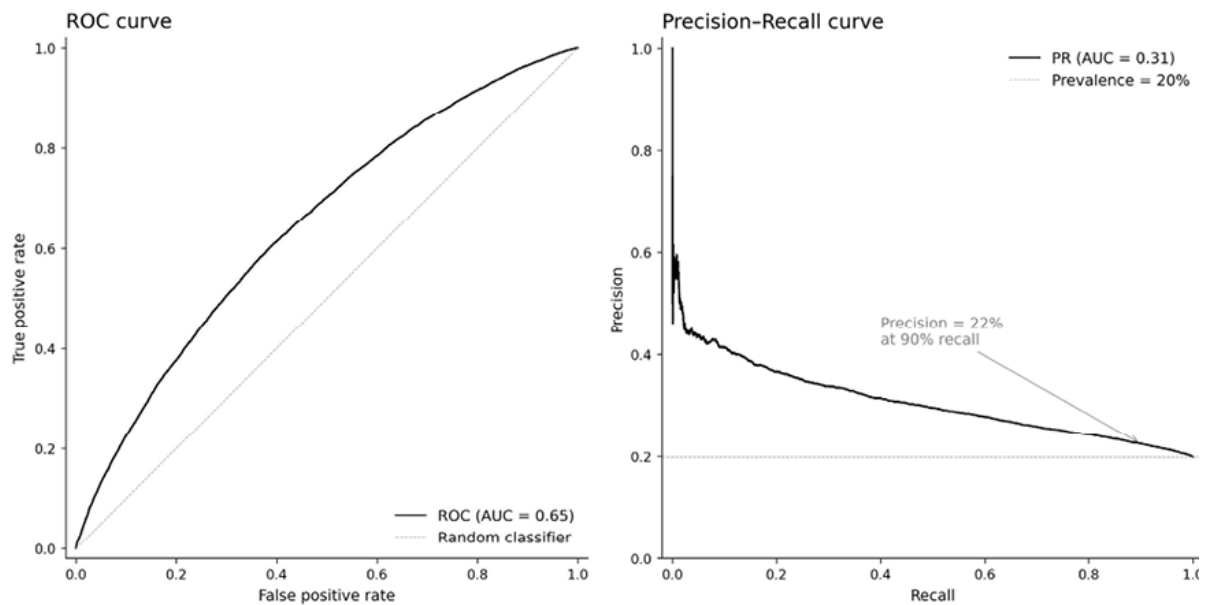


Figure 7.4: ROC and PR curves, side by side. Left: ROC curve for a feature with moderate discrimination ($AUC \approx 0.65$), with the diagonal reference. Right: PR curve for the same feature, baseline at the prevalence rate.

As shown in *Figure 7.4*, the PR curve reveals that at high-recall operating points, precision drops sharply, a pattern hidden by the ROC curve when the base rate is low.

For multi-class event labels (profit, stop, time-out), report base rates by class and fold, which shift across regimes. If you collapse multi-class labels into binary actions, explicitly record the collapse rule and verify that it aligns with your intended action set.



Implementation : `05_signal_evaluation` computes ROC AUC and PR AUC with fold-level breakdowns and Wilson confidence intervals for precision and recall.

Nonparametric diagnostics - Quantile and decile analysis

Correlation is compact, but it can hide non-monotone structure. At each decision time t , sort assets by $x_{t,a}$ within the eligibility mask, split into Q bins, and compute the mean label per bin $\mu_{t,q}$. Track the spread $S_t := \mu_{t,Q} - \mu_{t,1}$ and summarize within folds.

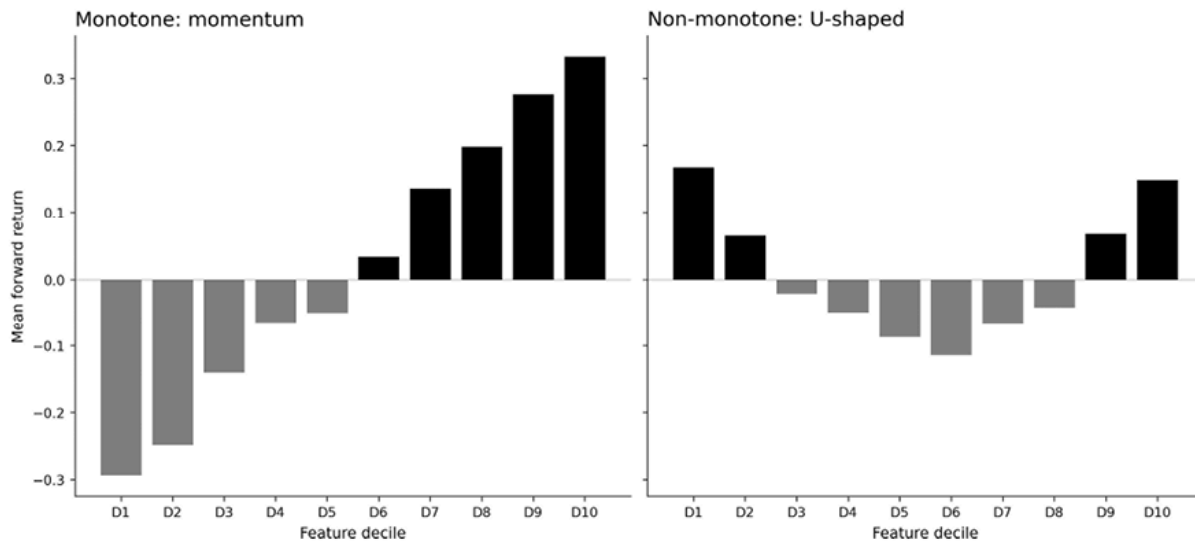


Figure 7.5: Quantile bar charts: monotone versus non-monotone. Left: mean forward return by feature decile increases monotonically, consistent with a ranking-based mapping. Right: a U-shaped pattern where both extreme deciles show elevated returns.

Figure 7.5 shows that a monotone mapping will miss the signal in the bottom decile; this implies either a nonlinear mapping or a confounder.

Monotone increasing patterns are compatible with ranking-based approaches (top- k selection, long-short spreads). Non-monotone patterns may require nonlinear mappings or suggest an interaction with market state. At this stage, you are validating shape, not designing the mapping.

Preliminary feasibility checks

Feasibility screens ask whether any plausible mapping could survive implementation. These use the simplest possible mapping—rank assets by

score and select the top k —as a stress test. Three checks cover most early failures:

1. **Turnover proxies** : Measure entry and exit rates in the top- k set across decision times within each fold. A feature in which the top- k set turns over completely at each rebalance is a warning; one in which positions persist has a structural cost advantage.
2. **Break-even cost sanity checks** : Compare typical fold spreads to conservative cost estimates. If the median fold spread is 20 basis points (0.2 %, bps) and round-trip costs are 15 bps, the net spread is 5 bps—barely above noise. A signal whose gross spread does not clear $2 \times$ estimated costs warrants caution; one that fails to clear the cost hurdle is a stop.
3. **Capacity warnings** : Recompute IC or spreads by liquidity bucket. A signal confined to the least liquid bucket is a stop for the current setup; smooth degradation from liquid to illiquid suggests the signal is broad-based.

Standardize triage decisions so later stages can trust the results:

Decision	Condition	Action
Proceed	Correctness screens pass; primary diagnostic directionally consistent across folds; quantile/confusion-matrix supports plausible mapping	Hand off to the modeling chapters
Revise	Idea plausible, but definition flawed (anchor mismatch, horizon mismatch, unstable base rates)	Change definition, re-run triage, record delta
Stop	Sign unstable across folds; spread too small to survive costs; signal confined	Archive and document the

Decision	Condition	Action
	to untradeable liquidity	failure mode

Table 7.2: Triage decisions standards

Read triage output in order: correctness first, then association, then shape and feasibility, then search accounting. Record the decision, diagnostic summaries, and searched-set metadata in the research ledger alongside the definition bundle (see *Box 7.1*).

Feature evaluation is not model selection, hyperparameter search, or a backtest. Its output is an auditable decision (proceed, revise, or stop) supported by fold-level summaries and plots that can be regenerated from the run record. The next section addresses the risks associated with an extensive search for predictive features.



Implementation : `05_signal_evaluation` covers correctness screens, fold-level IC and ICIR, IC decay, turnover, and break-even costs for the ETF momentum signal. `06_ic_inference` covers HAC inference, stationary block bootstrap, and minimum-track-record planning.

7.4 Search accounting and multiple testing

Trend scanning (*Section 7.2*) applied Bonferroni correction to a single observation's t-statistic when the best horizon is selected from many candidates. The same logic (correcting for the number of comparisons made) applies at the search level. When triage screens many candidate

features, labels, or variants, the best-observed IC, spread, or AUC come with a positive bias: some candidates will look good by chance.

Define the searched set

The starting point is to record what you actually evaluated. Define the **search set** as all candidates tested under the same decision rules. This typically includes

- Label-definition variants (anchor, horizon, threshold, or barrier rules)
- Feature-family variants (lookbacks, transforms, reference frames)
- Any conditioning templates you have tried

Record the searched-set size and the generation rules (grid, deduplication, exclusions). Without this, “significance” claims are uninterpretable.

For example, evaluating 5 label horizons $\times$ 3 threshold rules $\times$ 10 feature variants gives 150 bundles. Even if each bundle has only a 5% chance of passing a naive significance test under the null, we would expect roughly 7—8 false positives. The searched-set size is 150, and that number must accompany any reported p-value. See `07_multiple_testing` for a simulation showing how selection bias inflates the best IC when all factors are noise.

Separate exploration from confirmation

There are two phases to evaluation:

1. **Exploration pass** : Evaluate many candidates, but promote primarily on fold stability rather than peak performance (sign

consistency, lack of single-episode dominance, robustness to timing and coverage checks).

2. **Confirmation pass** : Freeze promoted candidates and re-run triage with a minimized comparison set to produce the summaries you will cite later.

The exploration pass is where we learn; the confirmation pass is where we commit. Mixing the two is the most common cause of failure in multiple testing: promoting on peak performance, then “confirming” on the same data.

Adjustment methods

If you report p-values or confidence intervals, compute them on fold summaries and adjust over the searched set you actually evaluated. Two families of correction dominate:

Family-wise error rate (FWER) controls the probability of *at least one* false positive across all tests. Use FWER when promoting a small number of candidates from a large pool: the cost of a single false positive (a strategy built around a spurious feature) is high.

The standard procedure is **Holm–Bonferroni** : sort the m p-values in ascending order, $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$, and reject $p_{(i)}$ only if $p_{(i)} \leq \alpha / (m - i + 1)$ for all $j \leq i$. This is uniformly more powerful than the original Bonferroni ($p_{(i)} \leq \alpha / m$) while controlling the same error rate.

False discovery rate (FDR) controls the expected fraction of false positives among all rejections. FDR is appropriate for large screens where you expect many true positives and can tolerate some false discoveries: for example, when screening 200 feature variants and expecting to advance 20–30.

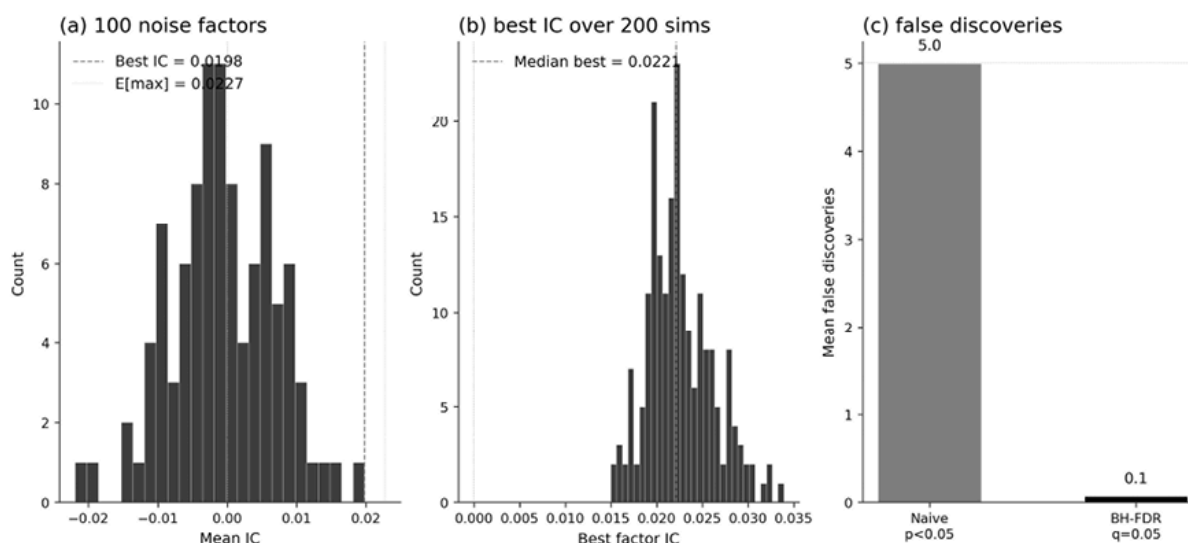


Figure 7.6: Selection bias under the null

Figure 7.6 shows three graphs. (a) Among 100 noise factors (50 assets, 252 days), the best achieves an IC of 0.020, close to the theoretical maximum of 0.023. (b) Across 200 simulations, the median best-factor IC is 0.022 — a spurious signal that would pass most screens. (c) Naive testing ($p < 0.05$) yields 5 false discoveries per simulation; BH-FDR control reduces this to near zero.

The standard procedure is **Benjamini–Hochberg** : sort p-values and reject $p_{(i)}$ if $p_{(i)} \leq (i/m) \cdot q$, where q is the target FDR (for example, 0.05). The notebook `07_multiple_testing` applies both BH and Holm–Bonferroni to a simulated factor zoo and compares discovery rates, false-positive counts, and statistical power. Applied to 13 ETF features, zero survive BH-FDR or Holm-Bonferroni at $\alpha = 0.05$: RVol 10d leads with HAC $t = 2.36$ but is rejected after correction.

The choice depends on the **downstream cost structure** :

- If each promoted feature incurs expensive modeling and backtesting costs, prefer FWER
- If you are building a feature pool where a few false positives are tolerable because model selection will weed them out, FDR is more

practical

Harvey, Liu, and Zhu (2016) surveyed over 300 published factors and recommended raising the significance threshold to $t > 3.0$ (from 2.0) for new factor discovery — a practical shortcut that accounts for the accumulated search across the literature.

When candidates are correlated (lookback variants or parameter sweeps around a single factor), independence-based corrections overstate the penalty. **Rademacher complexity** provides sharper bounds by measuring the effective richness of the hypothesis class (see *Chapter 16*).

Report effect sizes, not only significance

Adjustment methods control error rates but say nothing about economic magnitude. For any candidate carried forward, report:

- Searched-set size and generation rules
- Fold-level summaries (median and worst fold) for the primary diagnostic
- A stability indicator, such as sign consistency and time concentration
- Whether the numbers come from the exploration or confirmation pass



Box 7.4: Adaptive choices are additional trials

If a threshold, lookback grid, missingness rule, or interaction was chosen or modified after reviewing results, treat it as a new trial family and include it in the search-set accounting. These are legitimate research moves, but they increase the multiple-comparison burden. This is the single most common

accounting failure: optimizing a parameter after peeking at results and not counting it as a trial.

Strategy-level selection-bias diagnostics

When the outcome is a Sharpe ratio rather than an IC, three analogous diagnostics apply:

- The **Deflated Sharpe Ratio** (Bailey and López de Prado, 2014) estimates the probability that the best Sharpe exceeds chance
- the **Probability of Backtest Overfitting** (Bailey et al., 2015) measures how often the in-sample best-ranked configuration becomes the out-of-sample worst
- **Minimum Track Record Length** quantifies the amount of data required before promoting a strategy



Implementation : `07_multiple_testing`

implements the full pipeline: HAC-adjusted p-values, BH-FDR, and Holm–Bonferroni. It also previews Rademacher adjustments and the Deflated Sharpe Ratio (for more detail, see *Chapter 16*).

The next section provides a first introduction to causality.

7.5 From correlation to causality

The association diagnostics in *Section 7.3* establish that a feature carries information about the label. They do not explain why. A feature can appear predictive because it proxies for a market state characterized by volatility, liquidity, or risk appetite that also drives returns, rather than because it captures a stable, exploitable relationship.

This section introduces causal thinking as a **falsification filter** : state a mechanism, encode it as a graph, and then check whether the data are consistent with the assumptions implied by the mechanism.

The point is not to prove causality but to rule out stories that fail to meet their own basic implications before committing to more complex modeling.

In practice, this means asking whether the observed association survives simple checks suggested by the assumed mechanism, or whether it disappears once plausible confounders, timing effects, or selection effects are taken into account. These are robustness diagnostics guided by **mechanism reasoning** ; they test one feature at a time and cannot detect multivariate confounding. *Chapter 15* supplies that more comprehensive toolkit.

Why causal thinking matters for features

Most trading data is observational. Nobody runs randomized experiments on asset prices. A feature that correlates with future returns may do so for reasons that will not persist. Causal thinking surfaces three common traps:

- **Confounding** : A feature and a label share a common driver; the correlation disappears when that driver shifts. For example, both momentum and forward returns may respond to the volatility regime: the observed correlation reflects a shared exposure, not a direct link.
- **Conditioning traps** : Adding the wrong “control” can create spurious relationships (collider bias) or remove the effect you care about (conditioning on a mediator). Whether a variable should be

included as a control depends on the causal structure, not on its statistical significance.

- **Construction artifacts** : The pipeline inadvertently encodes information about selection or timing rather than about the market: for example, a rolling window that overlaps the label horizon, creating spurious correlation through shared data points.

What is a directed acyclic graph?

A **directed acyclic graph (DAG)** is a diagram that encodes causal assumptions. Each node represents a variable (a feature, a label, or a third quantity), and each arrow represents a hypothesized direction of influence, one variable affecting another. “Acyclic” means no path through the arrows leads back to its starting point; there are no feedback loops. *Figure 7.7* illustrates the basic anatomy.

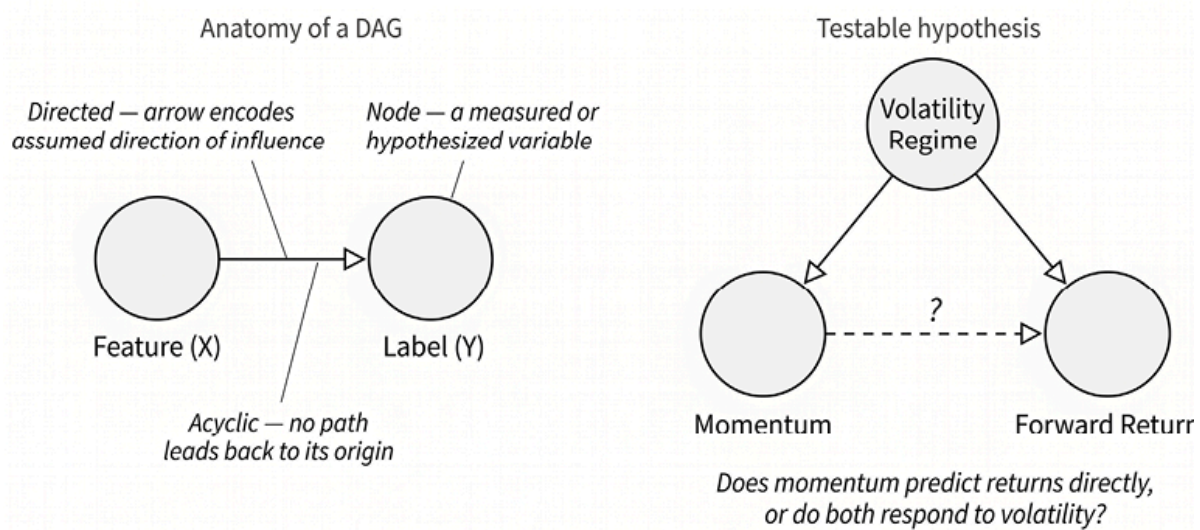


Figure 7.7: Directed Acyclic Graph

Why draw a DAG? Because it forces you to state your assumptions explicitly. A claim that “momentum predicts returns” is vague; a DAG that places an arrow from momentum to returns—and specifies that volatility regime affects both—is a testable structure. The arrows commit

you to specific implications: which variables should be correlated, which should become independent once you condition on a third, and which conditioning choices are safe. If the data contradict those implications, the assumed mechanism is wrong, and you have saved yourself from modeling a spurious relationship.

In financial applications, DAGs are typically small (three to five nodes) and encode a single hypothesis about one feature. The goal is not a comprehensive model of the economy but a minimal structure sufficient to identify what could go wrong with a specific feature-label association. Three recurring patterns—confounders, mediators, and colliders—determine what you should and should not condition on.

Three structural roles

Every variable adjacent to a feature-label pair plays one of three structural roles. Getting this right determines whether conditioning on a variable clarifies or distorts the relationship you are trying to evaluate. *Figure 7.8* illustrates each pattern with a minimal DAG:

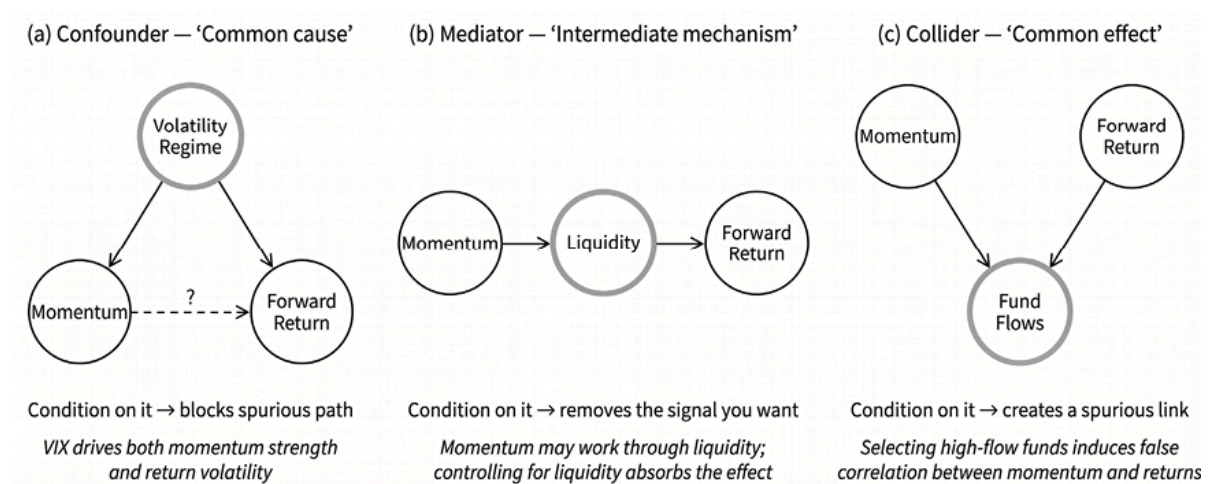


Figure 7.8: Fundamental causal relationships

The three components shown in this diagram are:

- Confounder: A common cause** . A confounder is a variable that influences both the feature and the label. In the DAG, it sits above both, with arrows pointing down to each. For example, the volatility regime (measured by the VIX) affects both the strength of the momentum signal and return dispersion. If you do not account for it, the feature-label correlation is inflated by the shared exposure. *Conditioning on a confounder blocks the spurious path* and isolates whatever direct relationship may exist. In the worked example below, the regime heterogeneity check partitions by VIX precisely to assess this.
- Mediator: An intermediate mechanism** . A mediator lies on the causal chain between feature and label: the feature affects the mediator, which in turn affects the label. For example, momentum may partly operate through liquidity: past winners attract flows, improving liquidity and supporting further price appreciation. If you condition on liquidity, you remove the indirect channel and may eliminate the very signal you are measuring. *Do not condition on a mediator* unless your goal is specifically to test whether the feature has any effect beyond the mediated path.
- Collider: A common effect** . A collider is a variable caused by both the feature and the label. In the DAG, arrows from both the feature and the label converge on it. For example, both momentum and forward returns drive fund flows: strong-momentum funds with high returns attract the most capital. Unconditionally, there is no distortion. But if you *condition on the collider* (say, by restricting your analysis to high-flow funds) you induce a spurious negative correlation between momentum and returns that does not exist in the full population. The notebook `08_causal_sanity_checks` includes a synthetic simulation showing this effect: conditioning on fund flows

(the collider) creates an approximately -0.25 correlation between two independent variables.

Before controlling for any variable, decide which role it plays. If you cannot decide, flag the feature for the multivariate toolkit in *Chapter 15* .

From DAG vocabulary to falsification checks

Drawing a DAG is the first step; testing its implications is the payoff. Each arrow in a DAG implies specific, testable patterns in the data: for example, that conditioning on a confounder should reduce a correlation, or that shifting a feature in time should cause its predictive power to decay. The four checks below probe these implications cheaply, using tools you already have from *Section 7.3* .

The diagnostics produce one of three outcomes, matching the triage gates introduced in *Section 7.3* :

- **PROCEED** (the mechanism survives all checks)
- **REVISE** (partial survival: the feature has a signal, but with caveats that inform downstream modeling)
- **STOP** (the mechanism fails: redirect effort to other features)

In efficient markets, cross-sectional ICs are small, and few single features will cleanly pass every check. REVISE is the expected and most informative outcome: it tells you *where* and *when* a feature works, which is precisely what downstream model design needs to know.

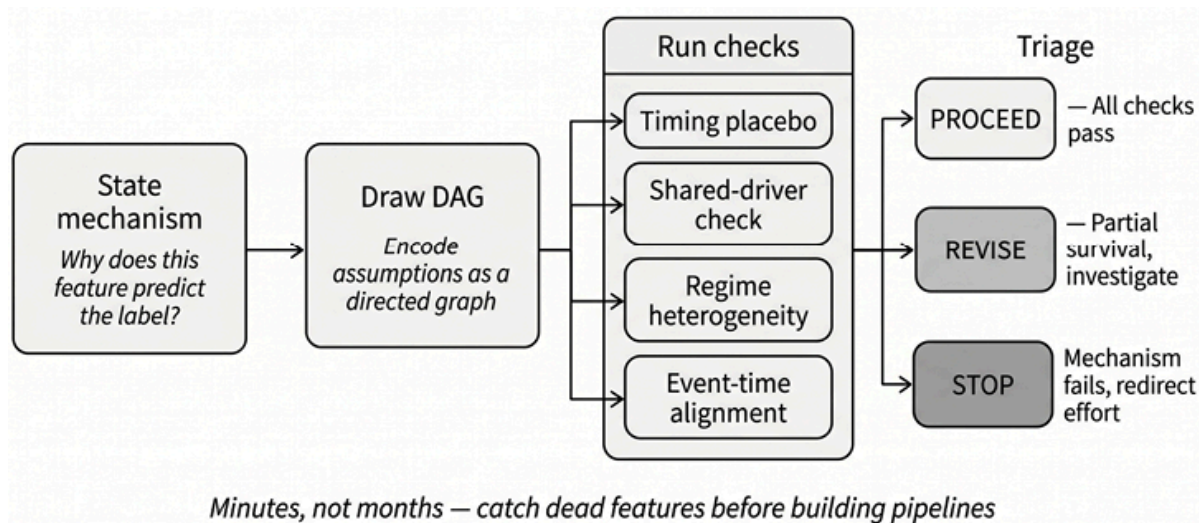


Figure 7.9: The mechanism plausibility workflow from hypothesis to triage decision

Mechanism plausibility checks

Each check has a clear protocol: what you compute, what you expect if the mechanism is real, and what constitutes a red flag. The notebook `08_causal_sanity_checks` first runs a feature $\times$ horizon scan—expanding on the null result from *Section 7.4*—and then applies the first three checks to two selected features.

Try a **timing placebo**. Shift the feature backward by Δ bars (for example, 5, 21, 63, 126, 252 days) and recompute IC at each lag. If the feature contains timely information, IC should be strongest near lag 0 and decay as the feature becomes stale. For a rolling-window feature with lookback L , a Δ -shifted version shares roughly $(L - \Delta)/L$ of its inputs with the original, creating a mechanical floor on IC persistence; focus on decay at lags beyond the lookback window. A red flag: IC that remains flat or *increases* at distant lags suggests the feature proxies a slow-moving state variable rather than timely information.

A **shared-driver check** replaces the label with an outcome that has no plausible link to the feature’s mechanism: for example, Treasury returns for a microstructure-driven equity feature, or a shuffled cross-sectional

assignment. IC should be indistinguishable from zero. This tests whether the feature's information is specific to the hypothesized channel or leaks into unrelated outcomes via a shared driver. A second control uses permutation: shuffling forward returns cross-sectional data at each date, producing a null distribution; the observed IC should lie far outside it.

Regime heterogeneity is important. Partition the sample by a candidate state variable (for example, VIX terciles) and recompute IC within each partition. IC may attenuate in some regimes, but should maintain sign and rough magnitude. If IC flips sign across partitions—with a significant opposite-sign partition ($HAC|t| > 2$) and a near-zero unconditional IC—the unconditional association is a pure aggregation artifact. This check cannot distinguish confounding from genuine effect modification; a feature whose IC varies by regime may be confounded or may have a mechanism that operates differently across states.

For features motivated by a discrete event (earnings, FOMC, rebalancing), compute IC in event-time windows. IC should concentrate where the mechanism predicts an effect. Symmetric pre- and post-event ICs, or ICs peaking before the event, usually indicate anticipation, leakage, or a confound. The example below omits this check because neither feature is event-driven.

Worked example - 12-1 momentum versus short-term reversal

The multiple-testing scan in *Section 7.4* found that no short-lookback feature survived BH-FDR at a 5-day horizon on ETFs. The notebook expands the search: a scan of 10 features across three horizons (5d, 21d, 63d) reveals that long-lookback momentum features (126d+) carry significant cross-sectional information at monthly horizons, confirming academic findings on cross-asset momentum (Asness et al., 2013). Two

features are selected for the mechanism checks, using a 21-day forward return label.

Feature A: 12-1 Momentum (REVISE)

Check	Result	Interpretation
Timing placebo	IC peaks at lag 0 (0.053); persists to lag 252d (0.034), HAC $t_0 = 3.5$	IC strongest at lag 0—partly mechanical for 231d lookback, but timely
Shared-driver check	Treasury HAC $t = 0.2$	
perm $p < 0.001$	No shared driver; IC well above permutation null	
Regime heterogeneity	Low-VIX IC = +0.086 (HAC $t = 3.9$); high-VIX IC = +0.017 ($t = 0.6$)	Sign stable; magnitude varies 5× across VIX regimes

Table 7.3: Momentum analysis

Momentum earns REVISE: only the shared-driver check passes cleanly. It carries genuine cross-sectional information (IC = 0.053, HAC $t = 3.5$) that does not predict Treasury returns, but the timing placebo and regime heterogeneity checks both flag caution—IC persists without clear decay, and the effect concentrates in low-volatility markets, attenuating during high-VIX episodes, consistent with the well-documented “momentum crash” phenomenon (Daniel and Moskowitz, 2016). This is an actionable finding that motivates regime-conditional modeling in later chapters.

Feature B: 1-day Reversal (STOP)

Check	Result	Interpretation
Timing placebo	No timely signal (IC $\alpha = 0.002$, HAC $t_0 = 0.4$)	Near-zero IC at all lags — no information to decay
Shared-driver check	Treasury HAC $t = 0.1$; perm $p = 0.38$	IC indistinguishable from shuffled noise
Regime heterogeneity	Low-VIX IC $= -0.019$ (HAC $t = -2.6$); high-VIX IC $= +0.028$ ($t = 2.5$)	Sign flips (HAC sig.) with near-zero unconditional IC — aggregation artifact

Table 7.4: Reversal analysis

Reversal earns STOP: all three checks fail. The unconditional IC is near zero (0.002 , HAC $t = 0.4$) with no timely signal at any lag, and the regime heterogeneity check reveals a textbook aggregation artifact—the feature encodes opposite information in low-VIX and high-VIX markets, producing a near-zero unconditional IC.

Plausibility scorecard

The following table summarizes the decision criteria for each check:

Check	Pass	Caution	Stop
Timing placebo	IC significant (HAC) with	IC persists without clear decay	No timely signal; IC increases at distant lags

Check	Pass	Caution	Stop
	meaningful decay		
Shared-driver check	IC ≈ 0 on control outcome (HAC)	IC is small but nonzero	IC is significant on control (HAC)
Regime heterogeneity	IC is stable across partitions	IC varies; sign flip not sig. (HAC)	Sign flip sig. (HAC) AND unconditional IC ≈ 0
Event-time alignment	IC concentrates post-event	IC spread across windows	IC peaks pre-event

Table 7.5: Plausibility scorecard

This is visualized in the following figure:

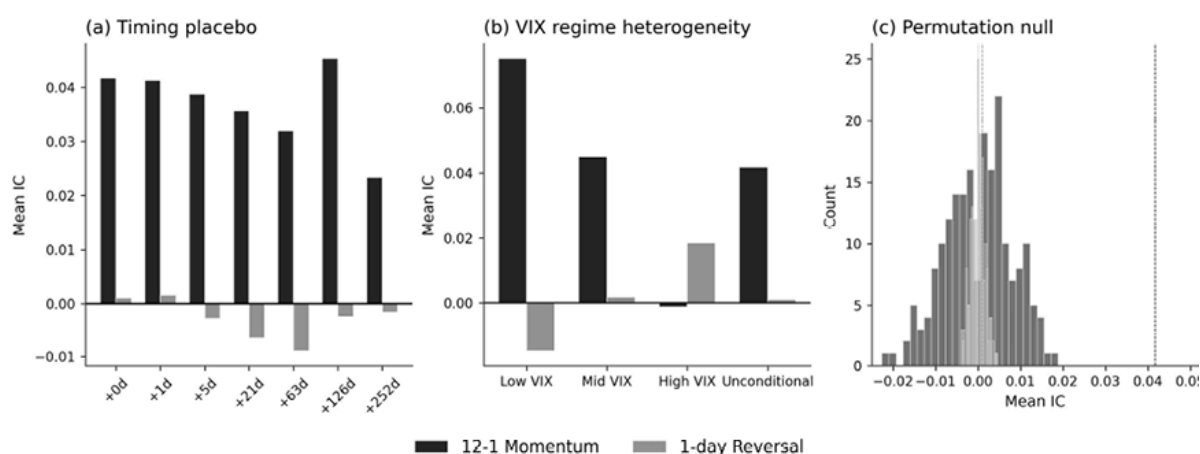


Figure 7.10: Mechanism plausibility scorecard

Figure 7.10 shows the timing placebo decay curves, regime-conditional IC, and permutation null for the two features. Momentum IC decays gradually from 0.053 to 0.034 over 252 days, consistent with a slow-

moving signal that is still strongest when most current. Momentum IC concentrates in low-volatility markets (IC = 0.086 in low-VIX versus 0.017 in high-VIX). Momentum IC of 0.042 (permutation $p < 0.001$) lies far outside the permutation null; reversal IC of 0.002 (permutation $p = 0.535$) is indistinguishable from noise.

All t-statistics use **Newey–West (HAC)** standard errors to account for serial dependence in overlapping IC series. Features that fail multiple checks should be investigated before proceeding; features that pass all checks have survived a first filter but have not been shown to be causal: these are bivariate diagnostics that cannot detect multivariate confounding.

These checks can reject proposed mechanisms but cannot confirm them. The natural next step is to move from single-feature diagnostics to the multivariate toolkit: structural models, double- or debiased machine learning, and sensitivity analysis. That machinery is reserved for the survivors and lives in *Chapter 15*.



Implementation : `@8_causal_sanity_checks` runs the collider simulation, the feature $\times$ horizon scan, and the timing-placebo, shared-driver, and regime-heterogeneity checks on ETF returns.

7.6 Summary

Defining the learning task well is one of the highest-leverage steps in the quantitative workflow. This chapter developed five interlocking disciplines that aim to separate reliable features from research noise. Clean, auditable preprocessing (*Section 7.1*) ensures that every input is comparable across trials and free of encoding surprises. Execution-

consistent labels (*Section 7.2*) anchor predictions to tradeable prices with explicit timing, lag, and horizon conventions; a mismatch of even one bar can fabricate or erase a possible edge. Fold-aware evaluation (*Section 7.3*) surfaces instability that pooled statistics hide: a feature whose IC flips sign across walk-forward folds is a lot less useful, regardless of its aggregate statistic. Search accounting (*Section 7.4*) records the number of candidates tested and applies multiple-testing corrections so that the best-observed IC is not simply the luckiest. Finally, mechanism plausibility checks (*Section 7.5*) encode causal assumptions as directed acyclic graphs and test their implications cheaply—catching confounded proxies and aggregation artifacts before they consume modeling effort.

Together, these disciplines feed a single decision framework. The triage gates (proceed, revise, or stop) convert diagnostics into auditable decisions at every stage. A candidate that advances carries a definition bundle (label specification, feature configuration, preprocessing choices), a fold-level diagnostics (IC, ICIR, sign consistency, quantile spreads), a searched-set record (how many variants were tested, which correction was applied), and a mechanism assessment (which plausibility checks it passed and where its signal concentrates). This documentation also serves as the minimum audit trail to make results reproducible and failures diagnosable.

The shared `ml4t.*` ecosystem wires this framework together under the canonical `timestamp + symbol` schema: `ml4t.data` for loaders, `ml4t.engineer` for feature recipes, `ml4t.diagnostic` for evaluation utilities, and `ml4t.backtest` for execution; see `10_ml4t_library_ecosystem` for the API tour.

With the framework set, *Chapter 8* translates strategy hypotheses into concrete feature specifications and applies the evaluation and triage gates built here to each family.

8

Financial Feature Engineering

Feature engineering is where our trading setup becomes learnable. A model can only exploit relationships that the dataset makes visible at decision time, at the relevant horizon, and in a representation consistent with how we trade.

Starting from the *Chapter 7* label, we propose economic drivers and express them as feature families with design choices and failure modes.

After completing this chapter, you will be able to:

- Translate a strategy hypothesis into a feature specification using the three-step filter.
- Select appropriate design choices for each feature family and distinguish meaning-changing from noise-reduction ones.
- Distinguish signal features from state features and understand when each applies.
- Construct interaction features and evaluate whether they add incremental information.
- Apply degrees-of-freedom discipline to prevent spurious discovery from search.
- Connect feature specifications to the triage protocol from *Chapter 7*

The chapter is organized into four stages:

1. *Section 8.1* introduces the three-step filter (horizon alignment, driver hypothesis, role separation) that turns a strategy narrative into a feature specification, as well as three design choices that configure each family.
2. *Section 8.2* develops price-derived families that apply to all nine case studies, and *Section 8.3* adds families that require multi-instrument data.
3. *Section 8.4* covers slow-moving contextual features (fundamentals, calendars, macro state) where point-in-time correctness is the binding constraint.
4. *Section 8.5* surveys cross-cutting feature types and the limits of direct aggregation, and *Section 8.6* addresses how to combine signals with state variables and control the resulting degrees of freedom.

We begin in *Section 8.1* by formalizing the hypothesis □ driver □ measurement filter that turns an idea into a tested feature.

8.1 Capturing and configuring the economic drivers

A **feature** encodes a claim about why an observable variable should shift the conditional distribution of the predictive target (a return, a volatility measure, or an event). The claim should be consistent with the strategy narrative from *Chapter 6* : the strategy family indicates which information is relevant, and the edge hypothesis explains why. A feature is the *computable expression* of that reasoning: a point-in-time transformation of data, aggregated over a lookback window and expressed in a chosen reference frame, intended to add incremental predictive content given the label definition from *Chapter 7* .

Move from labels to features using three filters (depicted in *Figure 8.1*):

1. **Horizon alignment:** Match lookback, sampling cadence, and aggregation to the label horizon and execution lag. Strategy families provide useful defaults: very short horizons motivate microstructure and order-flow features; medium horizons motivate trend, reversal, volatility state, and event drift; slow horizons motivate carry, term structure, and fundamentals. This is not a rigid partition; mismatches between feature and label horizons are a common source of unstable estimates. Document the implied update frequency and turnover so the design remains compatible with the trading setup.
2. **Driver hypothesis :** When predicting returns, most feature hypotheses map to a small set of economic mechanisms:
 - **Persistence (trend/momentum) :** Information is incorporated gradually due to attention limits or slow institutional flows.
 - **Reversion (microstructural or behavioral):** Predictable flows (hedging, rebalancing, inventory management) create transient pressure and subsequent reversal.
 - **Risk compensation (carry and state variables) :** Returns compensate for exposures others avoid, such as liquidity, tail, or funding risk.
 - **Predictable clocks :** Calendar effects, roll schedules, funding resets, and scheduled events can generate rule-driven patterns tied to market structure. If we cannot name a plausible mechanism, the result is a data pattern rather than a feature hypothesis (*Box 8.1*).
3. **Role separation:** Classify each feature as a **signal** (intended to predict the conditional mean or direction) or a **state variable** (intended to condition when signals work, how aggressively to trade, and how costly or risky trading will be). State variables such as volatility, liquidity, and funding conditions often matter through

interactions (gating or scaling signals) rather than strong marginal association.

The workflow is shown in *Figure 8.1* :

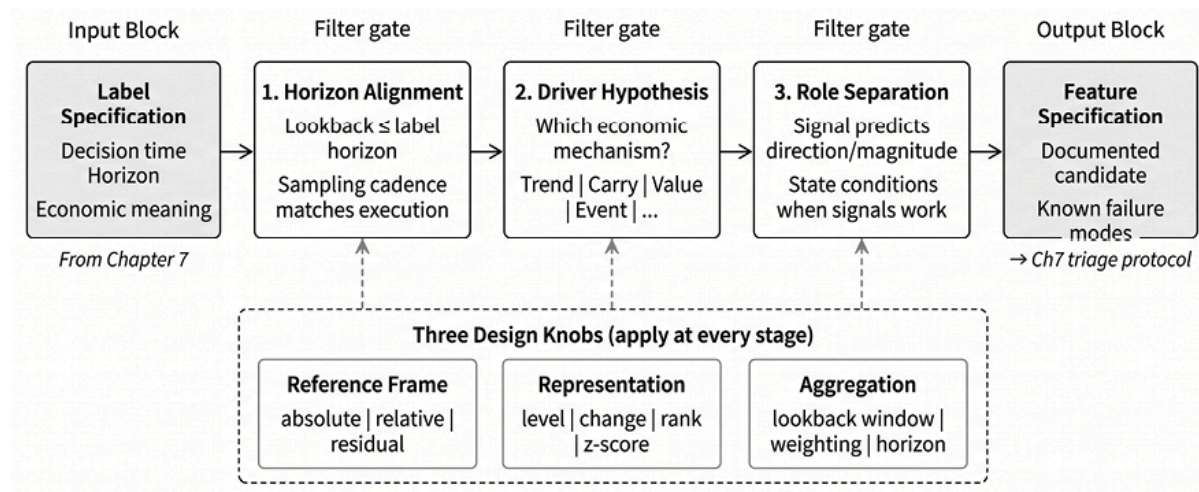


Figure 8.1: The three-step filter

Box 8.1: Patterns without a mechanism

Regularities without an economic explanation require a higher evidentiary standard because they provide no guidance on failure modes or regime dependence. Treat them as provisional:

- Apply stricter multiple-testing discipline and reserve untouched out-of-sample holdouts beyond cross-validation folds.
- Check robustness across related instruments, regimes, and small variants of the definition.
- Monitor decay after deployment and retire the feature when evidence weakens.

Across these steps, most feature designs can be configured using three choices:

- **Reference frame:** Absolute time-series measurements per asset versus relative cross-sectional ranks or peer-set comparisons, depending on whether the hypothesis concerns the time series' own dynamics or relative positioning.
- **Representation:** Raw values versus transformed values such as ranks, volatility scaling, winsorization, or residualization; choose transforms that match the hypothesis.
- **Aggregation:** How raw inputs are summarized to trade responsiveness against variance reduction. Aggregation sets the feature's effective bandwidth and should be guided by horizon alignment.

Treat the resulting choices as part of a **feature specification** . At minimum, we record the name, family, signal-versus-state role, driver hypothesis, and edge source; inputs and observability constraints; lookback and aggregation; reference frame; representation; and expected failure modes.

If we cannot fill in these fields, we have an input awaiting a hypothesis. *Table 8.1* provides three examples. In production systems, this specification corresponds to a feature-registry entry that supports versioning, auditability, and consistent reuse across trials.

Field	21-Day Momentum	Realized Volatility	Carry (Roll Yield)
Name	mom_21d	rvol_20d	carry_roll
Family	Trend/momentum	Volatility	Term structure
Role	Signal	State	Signal
Driver	Persistence	Risk state	Risk compensation

Field	21-Day Momentum	Realized Volatility	Carry (Roll Yield)
Inputs	Close prices	Close prices	Near/far futures
Lookback	21 days	20 days	Current contracts
Reference frame	Time series	Time series	Cross-section
Representation	Volatility-scaled return	Annualized standard deviation	Annualized percent
Failure modes	Crowding, regime shift	Stale in low-vol regimes	Roll-date artifacts

Table 8.1: Feature specification template

A feature is well posed when it operationalizes one element of the strategy narrative as a computable, testable input, with explicit timing and failure modes. The remainder of this chapter develops the main feature families, organized by the economic mechanisms they target.

Each feature specification becomes a trial entry in the *Chapter 7* workflow. Evaluate candidates at the label horizon using fold-aware diagnostics (cross-sectional IC, decile spreads). Conditional diagnostics (for example, momentum IC stratified by volatility terciles) test whether conditioning adds information or merely fragments the sample.

Sections 8.2-8.4 construct features by aggregating observable inputs using rolling windows, cross-sectional ranks, and simple formulas. *Chapter 9* extends the approach to features extracted from fitted objects. We start by looking at price-derived features.

8.2 Price-derived features

This section covers families computed from an asset's own price, volume, and trade data, the minimum dataset available for any traded instrument:

Family	Economic claim	Primary role	Typical horizons
Trend/momentum	Recent performance persists	Signal	Hours to months
Reversal	Prices revert to an anchor	Signal	Seconds to days
Volatility/tail risk	Risk conditions sizing and gating	State	All horizons
Liquidity/tradability	Costs and capacity gate signals	State	All horizons
Microstructure/order flow	Informed flow moves prices	Signal/state	Seconds to hours

Table 8.2: Overview of common features

Other families (carry/funding, cross-asset relative value, options-implied state, fundamentals, calendar effects, and macro shocks) appear later in this chapter sequence.

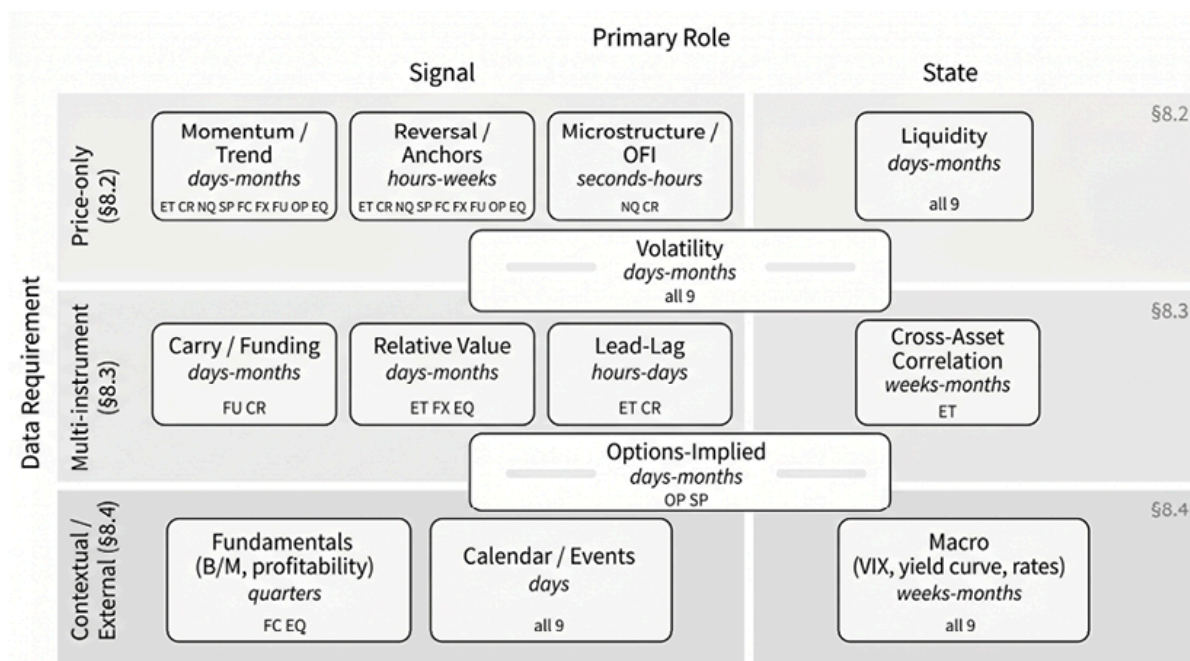


Figure 8.2: Feature families

For each family, use the same four questions:

1. What is the economic claim, and is the feature used primarily as a **signal** or a **state** variable?
2. At what horizons can the claim plausibly operate under your execution assumptions?
3. Which design choices change the hypothesis (not just the noise level)?
4. How does the feature typically fail in backtests and live trading?

Two constraints apply across families:

- **Horizon alignment** : lookahead, sampling cadence, and smoothing must be commensurate with the label horizon and execution lag.
- **Distributional stability** : prefer representations whose distributions drift slowly; use level-based features only with an explicit rationale and diagnostics.

Most price-derived features can be reduced to a few primitives: changes over a window, distances to an anchor, gain–loss asymmetries, and cross-

sectional comparisons. The practical discipline is to separate **meaning-changing choices** (reference frame, anchor definition, peer set, estimator choice) from **noise-reduction choices** (smoothing, shrinkage, winsorization). The former create new hypotheses; the latter trade variance for bias. All nine case studies support the families in this section; differences come from cadence, market structure, and whether overnight gaps matter.



Implementation :

- `@1_price_volume_features` creates trend, reversal, and volatility features.
- `@2_microstructure_features` implements trade-based liquidity proxies and order-flow imbalance.

Trend and momentum

Trend and momentum encode a **persistence** claim: recent performance predicts future returns at related horizons. Two distinctions materially change the hypothesis:

- **Time-series momentum** predicts an asset's future from its own past
- **Cross-sectional momentum** predicts relative performance within a peer set

Let's consider measurement type. *Return-based momentum* uses cumulative returns over a window to measure net displacement. *Moving-average momentum* uses the slope or crossover of a smoothed price series to measure path persistence. These are not interchangeable: they respond differently in choppy markets and during trend acceleration or exhaustion.

A standard implementation is cumulative return over a lookback window, scaled by realized volatility to make magnitudes comparable across assets and regimes. The lookback should match the label horizon: making your lookback too short risks capturing noise that decays before the label resolves; making it too long can smooth away the variation the label is meant to capture. Cross-sectional ranking at each date converts the score into a relative signal and reduces non-stationarity from level effects. In time-series use, volatility scaling carries most of the normalization.

Key characteristics for this feature family:

- **Role and horizons:** Primarily signal; hours to months.
- **Meaning-changing choices:** Time-series versus cross-sectional framing; lookback and holding horizon; decay profile; benchmark/sector residualization; volatility scaling and risk targeting.
- **Typical implementations:** Cumulative-return momentum, rank momentum, moving-average slope/crossover, residual momentum (persistence after factor adjustment).
- **Failure modes:** Regime concentration (trend versus mean-reverting regimes), misalignment between the lookback and label horizons, and search inflation from scanning many windows without selection correction.



Implementation : The same templates appear with cadence-appropriate windows in `etfs` (cross-sectional and daily), `crypto_perps_funding` (time-series and 8-hour), `cme_futures` (time-series across products), and `fx_pairs` (cross-sectional currency momentum).

Reversal and mean reversion to an anchor

Reversion features claim that prices return **to an anchor** , and the anchor defines the economic content.

At short horizons, reversal is often microstructure-driven: bid–ask bounce, inventory effects, and short-lived overreaction. The implicit anchor is a “fair” mid-price or a short-run equilibrium of the quote process. Longer-horizon mean reversion relies on more economically grounded anchors, such as **volume-weighted average price (VWAP)**, moving averages, peer means, or spread equilibria.

A standard template is a normalized distance-to-anchor statistic, often a z-score:

- An **anchor** determines **where the price should revert** : a moving average, VWAP, cross-sectional peer mean, or a fundamental parity level encodes different hypotheses.
- **Normalization** determines **what “extreme” means** : z-scoring by the asset’s own recent volatility measures abnormality relative to its history; cross-sectional ranking measures abnormality relative to peers. (Keep the anchor window and deviation window separate: they control different aspects of the hypothesis.)

Key characteristics for this feature family:

- **Role and horizons:** Signal; seconds to days (longer when the anchor is economically grounded).
- **Meaning-changing choices:** Anchor definition (statistical versus economic); extremeness definition (z-score versus percentile); clock-time versus event-time sampling; conditioning on spread and liquidity.

- **Typical implementations:** Last- k return reversal, bounded-oscillator templates (for example, RSI), explicit distance-to-anchor with a stated normalization.
- **Failure modes:** Edges that vanish after costs and execution delays, instability due to poorly estimated anchors, and “reversion” signals that are actually liquidity proxies.



Implementation :

- `nasdaq100_microstructure` implements bid–ask bounce at minute frequency.
- `fx_pairs` shows parity or moving-average equilibria.
- `sp500_equity_option_analytics` shows implied–realized spread reversion.

Volatility and tail-risk state

Volatility features are usually **state** variables: they summarize dispersion, regime, and tail risk that determine whether directional signals should be trusted and how aggressively to trade. If the label is itself a volatility target, the same measurements become **signals** , and you should evaluate them against a volatility label rather than against returns.

A standard volatility measure is **realized volatility** , which is the square root of the sum of squared returns over a time window (often annualized). The lookback controls the responsiveness–stability tradeoff: short windows react quickly but are noisy; long windows are smooth but can miss transitions. Converting volatility levels to percentiles of the asset’s historical distribution (or cross-sectional ranks) often improves stability,

as downstream models typically require a relative “high/low volatility” state rather than a drifting level.

Key characteristics for this feature family:

- **Role and horizons:** Primarily state (often via signal interaction); all horizons.
- **Meaning-changing choices:** Estimator; lookback and smoothing; downside emphasis (semivariance, drawdown depth); level versus change versus rank.
- **Typical implementations:** Realized volatility, range-based estimators, volatility percentiles, and drawdown summaries.
- **Failure modes:** Single shocks dominate short windows, smoothing that masks spikes, and estimator–cadence mismatch.

Efficient volatility estimation from OHLC data

Close-to-close volatility uses only closing prices. When open, high, and low prices are available, range-based estimators extract more information per bar and can deliver comparable precision with shorter windows.

Estimator	Inputs	Efficiency versus close-to-close	Best when
Close-to-close	Close	1×	Only closes are available
Parkinson	High, Low	~5×	Daily bars with negligible overnight effects
Garman-Klass	OHLC	~7×	Full OHLC with small gaps

Estimator	Inputs	Efficiency versus close-to-close	Best when
Yang-Zhang	OHLC	$\sim 8-14\times$	Overnight gaps are material

Table 8.3: Volatility estimators compared to close-to-close standard deviation

Here, “relative efficiency” refers to the **precision gain per observation** , commonly reported as the ratio of the close-to-close variance to the estimator’s variance under the model assumptions. A $5\times$ efficiency factor means that, for the same target variance, you need roughly one-fifth as many observations:

- The **Parkinson** estimator uses only the range:

$$\hat{\sigma}_P^2 = \frac{1}{4\ln 2} (\ln H_t - \ln L_t)^2.$$

- **Garman-Klass** adds open and close for higher efficiency:

$$\hat{\sigma}_{GK}^2 = 0.5(\ln H_t - \ln L_t)^2 - (2\ln 2 - 1)(\ln C_t - \ln O_t)^2.$$

- The **Yang-Zhang** estimator decomposes volatility into overnight (close-to-open) variation, intraday variation, and a Rogers-Satchell term. This is valuable when opening gaps are nontrivial, since a meaningful share of total variance may occur outside regular trading hours.

These formulas produce single-bar variance estimates. In practice, use a rolling mean over n bars, for example:

$$\hat{\sigma}_P(n) = \sqrt{\frac{1}{n} \sum_{i=0}^{n-1} \hat{\sigma}_{P,t-i}^2}$$

This is visualized in the following chart:

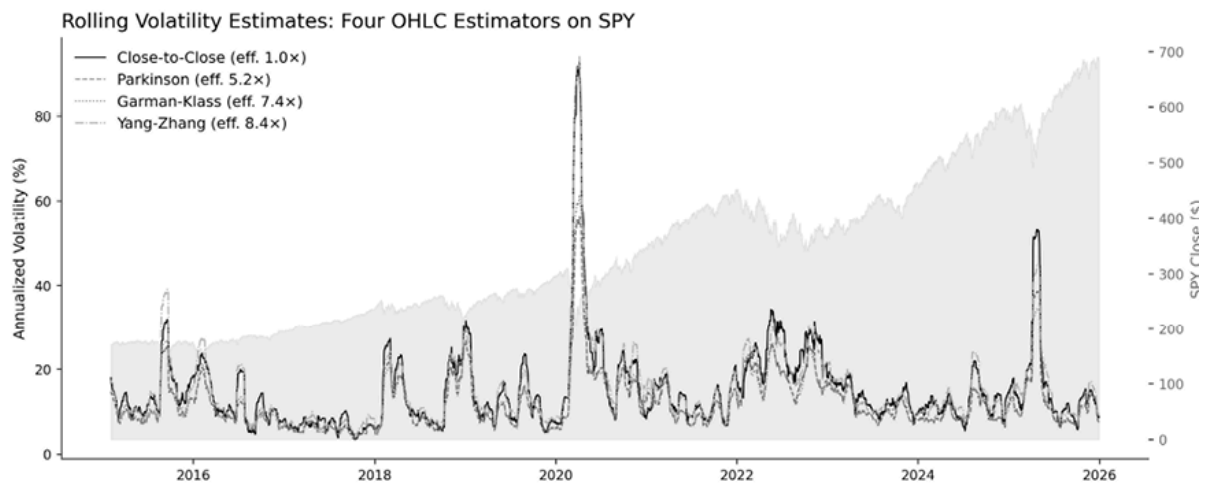


Figure 8.3: Volatility estimators

Implementation :

- `crypto_perps_funding` uses close-to-close on 8-hourly returns; 24/7 trading makes the overnight decomposition unnecessary.
- `etfs` and `us_equities_panel` use close-to-close as the variance estimator and add NATR as a price-denominated range indicator for gating and sizing rather than for variance measurement.
- `fx_pairs` and `sp500_equity_option_analytics` compute Garman-Klass alongside close-to-close, exploiting the full OHLC bar when overnight gaps are small at the chosen cadence.
- `cme_futures` compute Yang-Zhang alongside close-to-close; overnight session moves around the U.S. open are material for the bulk of the contract set.
- `nasdaq100_microstructure` computes realized volatility at minute frequency from intraday returns rather than applying daily OHLC estimators.

Tail-risk and regime indicators

Tail-risk measures extend the volatility state beyond dispersion. **Value-at-Risk (VaR)** and **Conditional VaR (CVaR , expected shortfall)** quantify losses beyond a given percentile (for example, the 95th percentile). Downside deviation (volatility on negative returns only) and tail ratios summarize asymmetry.

Regime indicators summarize path character. The **variance ratio** compares long-horizon variance to the scaled short-horizon variance: values below 1 suggest mean reversion, values above 1 suggest persistence. **Fractal efficiency** (net displacement divided by path length) makes a related “trending versus choppy” distinction. Parametric regime models are discussed in *Section 9.5* .

Average True Range (ATR) measures the typical price range. It is useful for execution rules (stop distances, dollar risk scaling, breakout filters) but is not a statistical estimator of return variance. Use range-based variance estimators to measure volatility and ATR for price-denominated sizing and gating.

Liquidity features operationalize expected costs and capacity. They are usually used as masks (excluding wide spreads or thin depth) or as interaction terms (downweighting signals when liquidity is scarce). Proxies include bid-ask spread, order book depth, turnover, and price impact per unit volume.

Microstructure, volume, and order flow

At short horizons, supply-and-demand mechanics are observable in signed volume, spread, and depth conditions, as well as in event-time behavior around trades and quote updates. Many “price+volume”

indicators reduce to two operations: aggregate signed flow over a window and normalize by a liquidity scale. The binding constraint is delay sensitivity: test whether the effect survives a realistic execution latency before optimizing the details.

A standard feature is **order-flow imbalance** (**OFI** ; see *Chapter 3*): the signed trade volume accumulated over a window, with the sign indicating whether trades are buyer- or seller-initiated:

- The first meaning-changing choice is the trade-signing rule (tick rule, quote-midpoint comparisons such as Lee–Ready-style methods, venue aggressor flags).
- The second is the aggregation structure. Simple OFI sums signed trades; order-book OFI (Cont, Kukanov, and Stoikov, 2014) uses changes in best-bid/ask quantities and is cleaner when full depth is available, but it requires limit order book reconstruction (*Chapter 3*).

Normalization determines portability across assets and regimes: divide OFI by total volume (fraction), by depth (liquidity-scaled), or by a z-score relative to the asset’s own history (rarity). Volume-normalized and depth-normalized OFI can diverge sharply in thin markets, and the divergence can itself be informative.

Key characteristics for this feature family:

- **Role and horizons:** Signal or state; seconds to hours.
- **Meaning-changing choices:** Clock-time versus event-time sampling; trade-signing rule; trade-based OFI versus order-book OFI; normalization (volume, depth, volatility); conditioning on spread/depth regimes.
- **Typical implementations:** Signed volume imbalance, order-book OFI, spread/depth proxies, queue imbalance when available, Kyle’s λ (price impact per unit of flow).

- **Failure modes:** Latency sensitivity (predictive at $t + \varepsilon$ but not at $t + 5min$), venue and data artifacts (mis-signed trades, crossed quotes, reporting delays), and execution assumptions that do not hold up in real order routing.

Venue structure constrains implementation. Central limit order books (for example, CME and major crypto venues) expose depth and queue state; RFQ/dealer markets typically expose flow only through transactions; AMMs expose pool state but not a traditional order book. The claim “informed flow moves prices” must be expressed in the venue-provided data structure.

Implementation :

- `@2_microstructure_features` constructs trade-sign OFI, Kyle’s λ , Amihud illiquidity, and Roll-spread features, and tests delay sensitivity by comparing the contemporaneous and lagged OFI relationship with returns. TSLA shows the highest Kyle lambda (0.003), indicating the greatest price impact per unit of volume. However, contemporaneous OFI-return correlation (+0.053) substantially exceeds lagged OFI-return correlation (-0.011).
- The `nasdaq100_microstructure` pipeline combines quote-based liquidity, signed-volume order flow, price impact, and dark-pool proxies.

8.3 Structural and cross-instrument features

The three families in this section require data beyond a single asset's price series: term structures, cross-instrument relationships, and derivatives-implied quantities. They encode information that is invisible in any individual price history: the cost of carrying a position, an asset's relative position within its peer group, and the market's probabilistic assessment of future outcomes.

Carry, funding, and term structure

Carry features encode **risk compensation** : the return from holding a position in the absence of a price change, measured by roll yield, basis, funding rates, and curve shape.

The standard feature is **roll yield** : the annualized price difference between two futures contracts of different maturities, measuring the return earned or paid to maintain exposure over time. The choice of maturity pair determines what part of the curve the signal represents:

- Comparing the **front-month** with the **second-month** contract captures conditions at the very front of the curve and is most sensitive to short-term supply-and-demand imbalances
- Comparing the **front** contract with a more deferred **back** contract measures the broader slope of the term structure and often reflects more persistent market conditions

Annualizing by the number of days between expirations makes roll yields comparable across contracts with different maturities. Cross-sectional ranking within a commodity sector then converts the raw level into a relative carry signal, separating assets whose curves reward holders from those whose curves impose a cost.

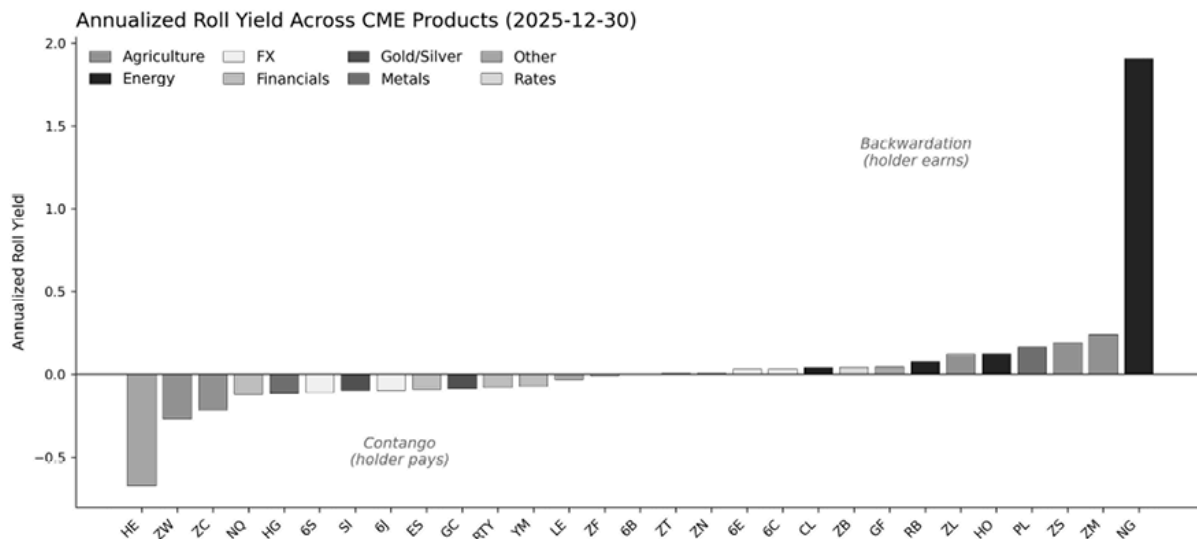


Figure 8.4: Annualized roll yield across 29 CME products on 2025-12-30, sorted by sector. Negative bars (Contango) indicate carry cost; positive bars (Backwardation) indicate carry earned

The curve itself contains more than a single slope. Level, slope, and curvature (the butterfly spread between short, mid, and long maturities) capture parallel shifts, tilt, and non-parallel movements, three independent dimensions driven by different economic forces.

Box 8.2: Crypto perpetuals — A distinct carry regime

Perpetual futures exchange funding at fixed intervals, meaning periodic cash payments between longs and shorts that keep the perpetual price anchored to spot, rather than rolling on monthly or quarterly expiry dates like standard futures. This poses a distinct feature-engineering problem. Most major exchanges (such as Binance, OKX, and Bybit) use 8-hour funding intervals (00:00, 08:00, 16:00 UTC), but some venues use 4-hour or 1-hour funding intervals. The interval should be verified per venue; using 8-hour windows on 4-hour funding data mixes settlements. Three design choices matter:



- **Clock alignment:** Funding rates reset at fixed UTC times (typically 00:00, 08:00, 16:00). Features built on non-aligned windows mix pre- and post-settlement observations. Use 8-hour windows that respect the funding clock, or explicitly model the intra-window dynamics.
- **Basis-funding decomposition:** The perpetual's deviation from spot has two components: the *basis* (the price difference) and the funding rate (the mechanism that anchors the perpetual to spot). These encode different information. The basis reflects the immediate supply-demand imbalance; the funding rate reflects the cost of maintaining positions. Treat them as separate features with distinct horizons and failure modes.



- **Signal versus state:** The funding rate plays a dual role. As a *signal*, extreme funding predicts mean reversion: high positive funding tends to precede perpetual underperformance as longs pay shorts. As a *state*, funding proxies crowding: when funding is persistently elevated, directional signals become less reliable because one-sided positioning creates fragility. Log both roles explicitly.

A common mistake is treating the funding rate as equivalent to traditional roll yield. Roll yield is a known mechanical cost; funding rate is market-determined and can swing by 100 bps within hours during periods of

volatility. Features that assume stable funding regimes fail during the regime transitions that matter most.

Key characteristics for this feature family:

- **Role and horizons:** Signal or state; days to months.
- **Meaning-changing choices:** Roll and funding clock alignment; curve representation; maturity alignment to the label horizon; levels versus changes versus ranks.
- **Typical implementations:** Futures roll yield, spot–future basis, funding-rate level and change (perpetuals), term-structure slopes and butterfly spreads.
- **Failure modes:** Roll discontinuities when contract definitions change, calendar misalignment between roll dates and feature windows, and hidden assumptions about which contract month is “front.”



Implementation : The `cme_futures` case study computes carry features across 30 products; `crypto_perps_funding` uses funding rates as the carry analog for perpetual futures.

Cross-asset structure and relative value

Cross-asset features exploit relationships across instruments rather than relying only on each asset’s own history. They ask three related but distinct questions: what exposure should be treated as common and removed, whether one market leads another, and how the relationship between assets changes over time.

The first class is **relative-value features** . Neutralization is not cleanup; it defines the hypothesis. These features remove what should be common, such as market, sector, or factor exposure, and focus on what remains: spreads, deviations, and residuals. Peer-set stability and refresh cadence are first-order design choices because a shifting peer set changes the hypothesis mid-backtest.

The standard feature is deviation from a peer mean, z-scored by the asset's own volatility: how far this asset has moved relative to its reference group, measured in units of its own noise. The peer-set definition is the meaning-changing choice. Sector peers, factor-model residuals, and statistical clusters encode different theories of what “common” means, and each produces a different residual.

Factor-neutral residual returns deserve particular attention. **Residual momentum** , defined as the persistence of factor-model residuals after neutralizing market and sector exposures, aims to capture stock-specific information that common factors miss. By construction, it is less exposed to broad market direction than raw momentum.

A second cross-asset hypothesis is that some markets lead others. Credit spreads may lead equity drawdowns, FX volatility may anticipate rate volatility, and in crypto markets, BTC moves may lead altcoin positioning at intraday frequencies. Unlike relative-value features, which usually assume contemporaneous relationships, **lead-lag features** model temporal cross-asset structure. They require careful treatment of time zones, trading hours, settlement conventions, and stale prices. Their main failure mode is mistaking delayed price discovery, low liquidity, or nonsynchronous trading for genuine information flow.

A third and often more robust class of cross-instrument features captures **how the relationship between assets changes over time** . These features are usually better interpreted as state variables than as direct trading

signals. For example, the 63-day rolling correlation between SPY and TLT returns measures the equity-bond relationship. When this correlation remains negative, diversification is working, and flight-to-safety dynamics are operative. When it moves toward zero or turns positive, the usual stock-bond hedge weakens, as in 2022, and other signals may behave differently.

This type of cross-asset state variable can have substantial conditioning power. The `etfs` case study computes the 63-day rolling SPY-TLT correlation as `corr_spy_tlt_63d` and supplies it to the LightGBM momentum model alongside other regime features. Aggregated across the $120\text{-fold} \times \text{config}$ observations on the 21-day momentum label, the feature has the highest stable (non-NaN-aggregating) fold-normalized importance among the 27 inputs, with a mean of 0.73, ahead of `yield_curve_zscore` and the momentum-slope features. While this does not isolate the feature's stand-alone predictive content, it confirms that the tree-based model is repeatedly choosing the equity-bond correlation as a node-split criterion, which is consistent with the state-variable interpretation: when the usual stock-bond hedge weakens, the relative attractiveness of momentum versus reversal shifts.

Key characteristics for this feature family:

- **Role and horizons:** Signal or state; hours to months.
- **Meaning-changing choices:** Peer-set definition; spreads versus residualization; neutralization strength; reference-set stability and refresh cadence.
- **Typical implementations:** Pair spreads, deviation from peer mean, z-scored peer deviations, factor-neutral residual returns, within-peer momentum, residual momentum.
- **Failure modes:** Universe drift, moving targets due to frequent peer-set redefinitions, and estimation noise introduced by the

neutralization step itself.

Implementation :

- `03_structural_cross_instrument_features` implements rolling beta and relative-value features and tests SPY-to-sector ETF lead-lag relationships. It finds uniformly negative lag-1 correlations, ranging from approximately -0.06 to -0.13, indicating short-term reversal rather than continuation. At daily frequency, highly liquid instruments trading in the same venue show no exploitable lead-lag effect in this test; the negative correlations likely reflect microstructure effects such as bid-ask bounce, ETF rebalancing, or short-horizon flow reversal. This negative result is useful because it illustrates why lead-lag claims require careful validation before they can be promoted to features.
- The `etfs` case study contains the production SPY-TLT correlation computation; `13_model_analysis` reports the feature-importance evidence cited above, while `06_robustness_sensitivity` evaluates regime-conditioning of momentum IC using SPY realized-volatility terciles (see *Section 8.6*).

Options-implied features

Option prices are useful because they not only reflect what the underlying asset has done in the past but also what the market is willing to pay for protection and upside exposure going forward. Inverting an option price through an option-pricing model yields **implied volatility (IV)**: the volatility input that makes the model match the observed market price. Repeating this across strikes and maturities produces the **implied volatility surface** , which summarizes how the market prices expected volatility, downside risk, and event uncertainty.

These features are often more forward-looking than realized volatility or return-based measures, but they only make sense if the surface is constructed consistently. In practice, this requires a stable **surface policy** : how moneyness is defined (whether an option's strike price is equal or very close to the underlying's price), which quotes are accepted, whether mid or last prices are used, how to interpolate across strikes and maturities, and how to map observed maturities to standardized horizons such as 30 days. If this policy changes over time, the feature may start tracking construction artifacts rather than changes in market beliefs.

Two definitions matter throughout:

- **ATM (at the money)** means an option whose strike is near the current spot or, more precisely, near the current forward price for the maturity in question. ATM options are the standard reference for the overall level of implied volatility because they are usually among the most liquid contracts.
- **OTM (out of the money)** means an option with no intrinsic value at the current price. For equity or index options, an **OTM put** has a strike below the current price, and an **OTM call** has a strike above the current price. OTM options, especially puts, are often used to measure tail-risk pricing because they become valuable only after sufficiently large moves.

Five features capture most of the economically relevant information in the volatility surface:

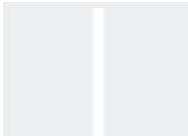
- **ATM implied volatility** (often mapped to a constant 30-day maturity) measures the market's baseline expectation of future dispersion. It is the primary state variable for the volatility regime: high ATM IV usually indicates elevated uncertainty, while low ATM IV is characteristic of calmer periods.
- **Implied-realized spread** compares today's implied volatility to the **subsequent realized volatility** over a matching horizon. This difference is a practical measure of the **variance risk premium** : investors often pay more for option protection than the volatility that is later realized. Historically, for index options such as the S&P 500, implied volatility has, on average, exceeded realized volatility, though the gap varies substantially across regimes. The spread can serve as a direct signal for volatility-selling strategies and as a conditioning variable for other strategies.
- **Risk reversal** is commonly defined as the implied volatility of a 25-delta put minus that of a 25-delta call at the same maturity. It measures the asymmetry of downside versus upside pricing. In equity index markets, puts usually trade at higher implied volatility than calls, so a larger put-minus-call spread indicates stronger demand for downside protection and greater perceived fragility.
- **Volatility term-structure slope** compares implied volatility at a short maturity with implied volatility at a longer maturity, for example, front-month versus 3-month IV. This feature distinguishes near-term stress from more persistent uncertainty. When short-dated IV rises above long-dated IV, the term structure inverts, often signaling an imminent event or market stress.
- **Put-call skew** compares the implied volatility of **OTM puts** with that of **OTM calls** , holding maturity and distance from the money

fixed as closely as possible. It isolates how expensively the market prices downside tails relative to upside tails, rather than the overall ATM volatility level.

These features define the economic content. In practice, however, their meaning still depends on several construction choices, so the implementation needs to be specified just as carefully as the feature definition itself.

Key characteristics for this feature family:

- **Role and horizons:** Usually regime or state variables; sometimes direct signals around earnings, macro announcements, or crisis periods; typical holding horizons range from days to months.
- **Meaning-changing choices:** ATM convention (spot-based, forward-based, or delta-based), delta choice for skew measures, quote filter rules, mid versus last prices, interpolation and extrapolation method, constant-maturity mapping, and whether the feature is used in levels, changes, z-scores, or cross-sectional ranks.
- **Typical implementations:** 30-day ATM IV, IV minus matched-horizon future realized volatility, 25-delta risk reversals, front-versus-back term-structure slopes, and OTM put-minus-call skew measures.
- **Failure modes:** Stale or wide quotes on illiquid strikes distorting the surface; inconsistent moneyness conventions across data vendors; survivorship bias when options expire or series are delisted; look-ahead bias when the “realized” leg of IV-minus-RV uses future data that overlaps the label horizon; and regime-dependent feature meaning—a risk reversal of -5 vol points carries different information in a VIX-15 environment than in a VIX-35 environment.



Implementation :



- `03_structural_cross_instrument_features` illustrates key calculations.
- `sp500_equity_option_analytics` constructs the full feature set (ATM IV, risk reversals, IV-RV spread at multiple horizons, and cross-sectional ranks) from S&P 500 constituent options.
- `sp500_options` works at the contract level, computing VRP features directly from straddle-level implied and realized volatility.

We turn next to features that capture changing state, which conditions how the signals built so far should be interpreted.

8.4 Contextual and slow-moving features

This section covers three feature families that are external to market prices: financial statements, macroeconomic releases, and deterministic calendars. They share a low update frequency (monthly, quarterly, or scheduled), strict *point-in-time correctness* requirements (*Chapter 4*), and a primary role as state variables that condition faster signals.

In practice, these features enter as controls, regime indicators, or conditioning variables in models that also use faster market-derived signals. The limiting factor is data integrity: each observation must reflect only what was available at the time of the trading decision. The `us_firm_characteristics` and `us_equities_panel` case studies are the main testbeds; calendar features apply to both.

Fundamentals and characteristics

Fundamental features are primarily designed to capture **long-horizon premia**. Because accounting variables such as earnings, book equity, leverage, or profitability update slowly, they are usually informative about return drivers that play out over months or quarters rather than days. They can also condition short-horizon signals: a tactical signal like momentum, reversal, or volatility may have different reliability depending on valuation, profitability, or balance-sheet strength.

The central risk is point-in-time error: using backfilled revisions or treating slowly updated fields as if they were new information every day. Repeating a quarterly value across daily rows also inflates the nominal sample size without adding information and creates strong within-period dependence. For example, quarterly book equity, repeated over roughly 63 trading days, multiplies the nominal N by about 63. When evaluating such features, apply the effective-sample-size and uniqueness-weighting logic from *Section 7.2* so repeated values do not dominate fold-level diagnostics.

Per *Section 8.1*'s measurement filter, the feature definition must specify reporting lags, revision policy, and availability rules. A standard example is the **book-to-market ratio**: book equity from the most recent reported quarter (lagged to account for reporting delays) divided by current market capitalization. The numerator updates quarterly and must use the vintage available at the time of the decision; the denominator updates daily.

Cross-sectional ranking (often expressed as percentiles and often sector- or industry-neutral) converts the raw ratio into a relative-value signal comparable across firms. The assumed lag is meaning-changing: a 60-day versus 90-day lag defines a different feature with different lookahead risk.

Standard characteristic families map directly to the `us_firm_characteristics` dataset. They fall into four broad categories: value (book-to-market, earnings yield), profitability (operating profitability, gross margins), investment (asset growth, capital

expenditure), and quality (accruals, leverage), which match the factor-zoo literature developed in depth in *Chapter 14* .

Key characteristics for this feature family:

- **Role and horizon:** Signal or state; weeks to years.
- **Meaning-changing choices:** Reporting lag; revision and restatement policy; update cadence; cross-sectional ranking/normalization; sector/industry neutralization; “surprise” definitions that depend on what was known when.
- **Typical implementations:** Valuation ratios, profitability and quality proxies, balance-sheet growth, and point-in-time surprises, where supported.
- **Failure modes:** Lookahead via revised fundamentals, inflated nominal sample size due to repeated values, and stale fundamentals treated as fresh daily observations.

Time, calendar, and event encodings

Calendar features encode **predictable clocks** : sessions, auctions, roll and funding windows, and scheduled events such as earnings or macro releases. Encode **phase and proximity** , not outcomes. Use time-to-event and post-event decay indicators, not realized post-event quantities in pre-event windows.

A standard feature is **time-to-event** : the number of periods until a scheduled event, capped at a maximum horizon. Optional binning (pre, event, post) turns a countdown into a regime indicator. Time-to-event should be computed in the same time zone and trading calendar as the strategy, using the same session boundaries and holiday rules used for bar construction. For earnings, use confirmed announcement dates from a point-in-time calendar; treating estimated dates as confirmed can create

lookahead. Post-event decay features represent a separate hypothesis about information absorption after the event.

For periodic patterns (time-of-day, day-of-week, month-of-year), cyclical encodings using *sin* and *cos* terms, or low-order Fourier terms, preserve circular structure. The number of harmonics is a noise-control parameter; the choice of which clocks to encode is meaning-changing.

Key characteristics for this feature family:

- **Role and horizon:** Typically state; sometimes a direct signal for known clocks; minutes to months.
- **Meaning-changing choices:** Event windows (pre/post boundaries); proximity caps and binning; decay shape; release-time alignment; encoding choice and number of harmonics.
- **Typical implementations:** Cyclical time encodings, Fourier seasonality terms, time-to-event proximity features, post-event decay indicators.
- **Failure modes:** Time zone and holiday misalignment, non-vintage calendar data, leakage from post-event realized quantities used in pre-event windows.

Calendar features are central in `crypto_perps_funding` (8-hour funding resets) and `cme_futures` (roll schedules that shift basis, volume, and liquidity).

Macro and policy state

In most trading setups, macro data is best treated as a **state** : it conditions when trend, carry, and relative-value signals work and how correlations and drawdowns behave. The feature definition must specify vintage series, release timestamping, and how values persist between releases (*Chapter 4*).

A standard feature is the **yield-curve slope**, such as the spread between long-term and short-term government yields (often 10-year minus 2-year), optionally smoothed and standardized against its own recent history. Standardization turns the level into a statement about how unusual the current regime is relative to a recent baseline. The maturity choice is meaning-changing (10Y-2Y versus 10Y-3M, or a principal-component representation) because it selects different aspects of the term structure. Smoothing and standardization horizons are noise-control parameters.

Related state variables include credit spreads, funding-stress proxies (for example, TED spread, cross-currency basis), and volatility regime indicators such as VIX level or percentile. **Macro surprise** features (realized value minus a consensus proxy) capture news flow, but the expectations series is itself a modeling choice and can be unstable across time and vendors.

Key characteristics for this feature family:

- **Role and horizon:** Primarily state; days to years, with discontinuities around releases and policy events.
- **Meaning-changing choices:** Release-time alignment and revision policy; level versus change versus surprise; smoothing and decay; global versus local panels; normalization (z-scores or ranks).
- **Typical implementations:** Yield-curve slope/curvature, policy-rate summaries, credit and funding-stress spreads, volatility regime indicators, nowcasts, macro surprise indices.
- **Failure modes:** Lookahead via revised data (use ALFRED or equivalent vintage series), incorrect release timestamping, stale values treated as daily-fresh observations, unstable expectations proxies.



Implementation : `04_fundamentals_macro_calendar`

shows representative constructions across these families, including book-to-market with simulated reporting lags, cyclical time encodings, and time-to-event features, and macro-state features such as yield-curve slope and VIX-based regimes.

8.5 Cross-cutting feature types and the limits of direct aggregation

Two cross-cutting feature types deserve explicit treatment because they do not fit neatly into a single family:

- **Learned representations:** Latent factors from correlated inputs, such as principal components, autoencoders, and time-series foundation models (TimesFM, Chronos, Moirai). These representations are fitted objects: they are trained inside folds, persist versioned parameters, and log training windows alongside downstream trials. *Chapter 9* covers model-based feature extraction; *Chapter 13* develops the transformer architectures behind foundation-model embeddings.
- **Flows and positioning:** Proxies for crowding and constraints (for example, the Commitment of Traders report; *Chapter 4*), typically used as a **state** that conditions trend, carry, and mean reversion. Positioning proxies can become unstable when definitions, reporting conventions, or sources change; treat them as first-class hypotheses and apply the same discipline to timestamps, reference sets, and failure modes.

For crypto assets, blockchain data provides direct observability of exchange reserve flows, whale wallet movements, stablecoin supply shifts, and protocol-level metrics (TVL, liquidity pool composition). *Chapter 4* covers the data infrastructure and sourcing for these signals. From a feature-engineering perspective, on-chain data represents a positioning family with novel failure modes: addresses can be relabeled, protocols can fork, and definitions (what counts as a “whale”) are researcher-imposed. We log the classification logic and version it alongside the features.

The cross-case diagnostic in `case_study_feature_summary` makes the breadth-versus-skill tension concrete: across the nine case studies, feature counts range from 39 to 66, with deliberate per-asset-class specialization, and past-return windows are the one family universal to every case study. Plotting universe size against best in-sample $|IC|$, with bubble area proportional to the Fundamental Law’s IR estimate ($IC \times \sqrt{N}$), shows `us_firm_characteristics` as the dominant outlier with an estimated IR near 4 on its 2,483-stock universe—breadth, not a marginally higher IC, is what separates a thin signal from an investable one.

When direct aggregation is not enough

Every feature in *Section 8.2–8.4* is a deterministic formula applied to a trailing window of observable data. This works when the structure we care about is visible in the raw series: trend direction, deviation from an anchor, relative rank, range. But some structure is hidden:

- **Conditional dynamics.** How quickly does a volatility shock decay? A rolling standard deviation shows the current level; it cannot indicate persistence. A fitted model (GARCH) is required to estimate that.

- **Latent states.** Is the market in a trending or mean-reverting regime? No observable variable answers this directly. A regime model, such as an HMM or Markov-switching model, is required to infer the state probabilities.
- **Cyclical structure.** Is there a weekly pattern in this volume series, or is it noise? A spectral decomposition (for example, an FFT) is required to measure cycle strength.
- **Path geometry.** Two 20-day windows with identical returns but different paths (one smooth, one jagged) may predict different outcomes. A sequential encoding (path signatures) is required to distinguish them.

Chapter 9 discusses model-based features designed to capture such hidden structure.

8.6 Combining features and controlling search

Much of the practical improvement in feature-based strategies does not come from individual features but from combining a signal feature with a state feature. The signal carries the directional view (momentum, carry, mean reversion); the state describes the environment in which the signal operates (volatility regime, liquidity conditions, market breadth).

Three **interaction templates** cover most combinations:

Template	What it does	Example	Effect on strategy
Gating	Trade only when the state is favorable;	Trade momentum only when volatility is	Changes the active sample—and therefore

Template	What it does	Example	Effect on strategy
	otherwise, go flat	below its 67th percentile	turnover and capacity
Scaling	Adjust signal strength continuously by state	Divide momentum by realized volatility to get a risk-adjusted signal	Changes position size conditionally on state
Conditional variant	Log “signal in regime” as a separate, versioned feature	Momentum-in-low-vol and momentum-in-high-vol as distinct candidates	Turns the interaction into separately testable hypotheses

Table 8.4: Common interaction patterns

Start with one signal and one state, and vary one template at a time. Record each combination as an explicit trial (see *Section 7.3*) so every interaction is traceable in searched-set accounting.

Not all combinations deserve testing. Prioritize interactions where:

- **The state is economically relevant:** Volatility can condition trend capture; liquidity conditions effective costs.
- **The interaction is asymmetric:** Conditional IC differs meaningfully across state values, rather than a constant rescaling of the average.
- **The state is observable at decision time:** Avoid definitions that depend on contemporaneous information unavailable at execution,

or on revised data.

Interaction diagnostics

Because interactions multiply the searched set, evaluate each with three diagnostics (computed within each walk-forward fold, summarized using medians and worst-fold views):

- **Structure:** Conditional IC varies systematically with the state; irregular patterns suggest overfitting or unstable binning.
- **Fold stability:** Conditional effects do not repeatedly flip sign across folds.
- **Real-time robustness:** Noisy, delayed, or revised state estimates reduce transfer from backtest to live, especially for hard gates.

Common interaction patterns

Dividing a signal by a positive state variable yields risk-adjusted variants:

Ratio feature	Formula	Interpretation
Risk-adjusted momentum	momentum/volatility	Risk scaling
Carry-to-vol	carry/implied_vol	Risk-adjusted carry
Momentum-to-spread	momentum/bid-ask spread	Cost-adjusted signal

Table 8.5: Feature interaction examples

Ratio definitions implicitly assume the denominator is strictly positive and meaningfully estimated at decision time. Handle edge cases

explicitly: clip near-zero volatility, and set the ratio to zero (or missing) when the denominator is undefined.

Discretize the state into bins (for example, terciles), compute the IC within each bin, and compare ICs across bins. This tests whether information content depends on the state and identifies the direction of the dependence.

Worked example - Momentum × volatility-regime terciles

The `etfs` case study illustrates conditional IC analysis using 126-day momentum as the signal and 42-day realized volatility as the state. Volatility is proxied by SPY to define a market-wide regime.

At each decision time t :

1. Compute cross-sectional 126-day momentum ranks for all ETFs.
2. Compute 42-day realized volatility for SPY and assign t to a volatility tercile using expanding-window percentiles (low: <33rd; medium: 33rd–67th; high: >67th). Expanding-window thresholds avoid look-ahead bias.
3. Compute the IC between momentum rank and 20-day forward return, separately within each volatility tercile.

Results (full sample; HAC-adjusted for serial correlation):

Volatility tercile	Momentum IC	HAC t-statistic	p-value
Low	+0.071	2.8	0.006
Medium	+0.025	1.3	0.21
High	−0.034	−1.1	0.26

Table 8.6: Feature interaction example

Momentum is strongest in low volatility and weakens as volatility rises. The monotonic decay across terciles is the relevant pattern. This motivates a **gating** rule: take the momentum signal only when volatility is not in the top tercile.



Implementation : `06_robustness_sensitivity` partitions SPY's 42-day realized volatility into terciles (expanding-window thresholds) and reports HAC-adjusted IC by regime. The 10.5pp IC swing between low-vol (+0.071) and high-vol (−0.034) regimes is the empirical evidence for the regime-conditioning hypothesis tested in this section. The notebook also runs parameter sweeps over lookback windows and interaction features to test signal stability.

Each tested interaction increases the search set. For example, 5 signals $\times$ 3 states $\times$ 3 templates yields 45 interactions. Apply the controls from *Section 7.4* : report the number of interactions searched, use Benjamini–Hochberg false discovery rate (FDR) control, and treat “best-in-search” results as exploratory unless confirmed on held-out folds.

Event studies

A complementary diagnostic asks whether a feature-defined event is followed by abnormal returns in the direction implied by the signal. Following MacKinlay (1997), define an event date $\tau = 0$, estimate normal returns over a pre-event estimation window, compute abnormal returns during the event window, and aggregate them into cumulative abnormal returns. In feature research, the “event” is usually not a corporate announcement but a rule-based signal: for example, an ETF momentum breakout, a transition into the top momentum decile, or a

volatility regime change. The event definition must use only information available at the time of the decision; otherwise, the event study inherits look-ahead bias.

Formally, estimate a normal-return model on a window that ends before the event window begins, such as a market-adjusted return, market model, or factor model. The abnormal return is:

$$AR_{i,\tau} = R_{i,\tau} - \hat{E}(R_{i,\tau} | X_{\tau})$$

where $R_{i,\tau}$ is the realized return for asset i at event time τ , and the conditional expectation is the benchmark return implied by the fitted normal-return model. The cumulative abnormal return over event window $[\tau_1, \tau_2]$ is:

$$CAR_i(\tau_1, \tau_2) = \sum_{\tau=\tau_1}^{\tau_2} AR_{i,\tau}$$

and averaging across events gives the average abnormal return (AAR) or cumulative average abnormal return (CAAR). The key diagnostic is whether the abnormal-return path continues after the event rather than merely reflecting the price movement used to define the breakout. Because feature-generated events often overlap, and because assets share common risk exposures, inference should use HAC or clustered standard errors rather than assuming independent event returns.

Implementation : `07_event_studies` applies this framework to ETF momentum breakouts. The event date is the breakout signal, abnormal returns are measured relative to an explicit benchmark model, and the event window runs from day -3 to day +10. The mean CAR of 1.11% is statistically significant ($t=3.28$, $p=0.001$), and CAAR builds from day -3 to day +10 without a pre-event



reversal. This pattern supports the hypothesis that the breakout captures continuation rather than a mechanical rebound.

Signal-conditioned analysis reveals an important asymmetry: long momentum signals are validated (CAAR=0.87%, $t=3.30$), while short signals are not (CAAR=-0.25%, $t=-1.31$). This asymmetry is consistent with the broader empirical pattern that long and short momentum signals need not be equally reliable after costs, constraints, and market frictions. More importantly, it illustrates the role of event studies as a falsification tool: a candidate feature should not merely produce a significant average effect; it should exhibit the correct event-time shape, withstand reasonable benchmark choices, and behave consistently in the signal direction it claims to capture.

Degrees of freedom discipline

Dense grids over lookbacks, transforms, and interactions create spurious winners and reduce interpretability. Three controls keep feature search disciplined:

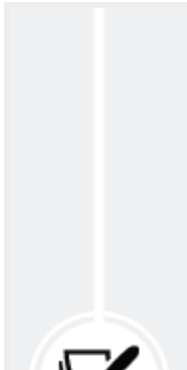
- **Budget by family and by role:** Allocate variants within a hypothesis class (for example, momentum signals) separately from state features (for example, volatility, liquidity).
- **One choice at a time by default:** Fix the driver and representation, vary one choice (often the lookback), keep a small set of survivors, then consider a second.
- **Deduplicate within families first:** Cluster by correlation (or rank correlation) computed within folds to remove near-duplicates before

modeling, preserving diversity across families.

When search produces many near-equivalent variants, deduplication reduces redundancy before the model stage. Here, “near-equivalent” means features that move so similarly that they contribute little distinct information, while “stability” refers to how consistently a feature performs across validation folds rather than how well it does in a single split:

- **Pairwise dependence:** Within each fold, compute rank correlations ρ over the training window, producing a $p \times p$ dependence matrix for p features.
- **Clustering:** Apply hierarchical clustering using distance $1 - |\rho|$, which groups features that are highly similar up to sign. The cut level is a tuning choice rather than a rule; thresholds in the range $|\rho| \approx 0.6$ to 0.8 are common starting points.
- **Representative selection:** Choose one feature per cluster using a transparent rule, such as the highest median IC with low fold-to-fold IC dispersion, simplest construction, or lowest implementation burden. When these criteria conflict, favor stability across folds over a small gain in a single summary metric.

Deduplication reduces redundancy within a family but does not replace model-stage selection or importance analysis. MDI and permutation feature importance can disagree substantially, so the choice of importance method materially affects feature selection outcomes.



Implementation : `05_feature_selection` runs the full pipeline for the `etfs` case study: starting from 57 candidate features, correlation filtering (threshold 0.9) removes 14, IC ranking retains 18 above the significance threshold, and final selection produces 10 features, an



82.5% reduction. The strongest single feature is distance-to-52-week-low ($IC=0.076$). After BH-FDR correction, four of the 34 features significant at the raw 5% level no longer survive the multiple-testing adjustment. MDI and permutation feature importance agree on only 49% of rankings.

Three implementation choices that change the hypothesis

Three implementation choices change the economic claim, not just the noise level:

- **Residualization:** Removing market or sector components defines a different question and introduces a fitted model that must be handled inside the walk-forward protocol.
- **Winsorization:** Clipping at the 1st/99th versus 5th/95th percentiles changes which observations drive the signal. Fit clipping bounds on training data and apply them to the corresponding validation window.
- **Stationarity–memory trade-off:** First-differencing ($d = 1$) removes long-range dependence; fractional differencing with $d \in (0,1)$ preserves more memory while improving stationarity. A feature with $d = 0.4$ encodes a different hypothesis than one with $d = 1.0$ (see *Section 9.1*). The stationarity–memory trade-off applies across all families in *Section 8.2–8.4*; fractional differencing with a data-driven d is developed in *Section 9.1*.

Operationally, fit any transform with learned parameters on walk-forward training splits and apply it to the corresponding validation splits. When a feature depends on a fitted object (for example, a neutralization model,

scaling estimator, or learned representation), log the fitted parameters and the training window to support reproducibility.

8.7 Summary

We turned trading hypotheses into a disciplined feature design process. The three-step filter (horizon alignment, driver hypothesis, role separation) ensures every feature encodes a testable economic claim. The feature families in *Section 8.2* are reusable hypothesis classes, not ad hoc indicators: many named technical indicators are parameterizations of the same underlying claim, and recognizing this prevents indicator sprawl. Within each family, the distinction between meaning-changing choices (reference frame, anchor definition, peer set) and noise-reduction choices (smoothing, shrinkage) determines whether we are testing a new hypothesis or merely smoothing the same one.

Our second theme is combination and control. Signal $\times$ state interactions (gating, scaling, conditional variants) are where much of the practical improvement comes from, but they multiply the search space geometrically. The deduplication workflow and one-choice-at-a-time discipline from *Section 8.6* keep the search disciplined, and every interaction enters the trial taxonomy from *Chapter 7* as a distinct hypothesis with its own multiple-testing burden.

The deliverable is a documented, reproducible candidate feature set with clear economic meaning and known failure modes, ready for the triage protocol in *Chapter 7* and the modeling pipeline in *Chapters 11-16*.

Direct aggregation has limits. When the structure we need is hidden (conditional dynamics, latent states, cyclical patterns, path geometry), we must fit a model and extract features from the fitted object. That is the subject of *Chapter 9*, where the point-in-time obligation tightens: a GARCH parameter or HMM state probability depends on an estimation

window that must be confined to each walk-forward training fold, versioned, and monitored for drift.

9

Model-Based Feature Extraction

In *Chapter 8* , we built financial features by aggregating and applying deterministic transformations to observed data. This chapter turns to **model-based features** , produced by estimated procedures. Some take the form of structural quantities, such as coefficients, persistence measures, or latent states. Many others are operational outputs, including filtered estimates, innovations, forecasts, conditional variances, regime probabilities, and posterior uncertainty summaries. The common idea is simple: fit a procedure to the training data and use its outputs as features.

These procedures add value in two related ways. Some forecast quantities that matter for downstream prediction, especially volatility and other state variables that condition expected returns. Others do not forecast the target directly, but instead infer hidden structure in the data, such as trend, persistence, break risk, or regime membership. In both cases, the fitted procedure can summarize historical information more effectively with respect to the predictive target than a direct backward-looking formula, yielding features that can be more informative, more stable, or more interpretable than raw transformations alone.

The chapter is organized by extraction method rather than by economic family because a single fitted procedure can generate several distinct feature types. A Kalman filter yields level, trend, innovation, and uncertainty features; a GARCH model yields both conditional-volatility

outputs and persistence parameters; a regime model yields probabilities, expected durations, and transition structure. Whatever form the feature takes, the point-in-time requirement remains unchanged. Every estimate, state, forecast, and probability must be generated using only information available at the time of decision, re-estimated within the walk-forward protocol, and versioned alongside the features it produces. After completing this chapter, you will be able to:

- Distinguish direct features from model-based features and judge when a fitted procedure adds useful information.
- Use fitted procedures to extract forecasts, filtered states, residuals, conditional volatility, regime probabilities, and uncertainty summaries as downstream features.
- Design a compact, interpretable set of model-based features from diagnostics, signal transforms, volatility models, uncertainty summaries, and regime models.
- Enforce point-in-time correctness by fitting and selecting models within training windows, using filtered rather than future-informed outputs, and aligning refit cadence and online updates with the walk-forward protocol.
- Transform asset-level temporal outputs into cross-sectional, benchmark-adjusted, pairwise, and universe-level features for multi-asset prediction tasks.
- Distinguish between exploratory time-series methods that are useful for research diagnosis and deployable features that are safe for live, point-in-time use.
- Use uncertainty and regime outputs primarily as conditioning features, and recognize when they should not be treated as stand-alone trading signals.

We begin with diagnostics and stationarity features. *Section 9.2* turns to signal transforms, including Kalman filters, spectral methods, wavelets,

and signatures, which expose temporal structure that rolling statistics miss. *Section 9.3* focuses on volatility features, and *Section 9.4* treats uncertainty itself as a feature. *Section 9.5* develops regime features from threshold rules, HMMs, Markov-switching models, and distribution-based clustering. *Section 9.6* shows how to convert these temporal outputs into cross-sectional and panel features.

9.1 Diagnostics and stationarity features

Model-based features often begin as diagnostics. Before fitting a forecasting or state-space model, inspect the series to understand what kind of temporal structure it contains, what transformations it may require, and which outputs may themselves be useful downstream features. In this section, diagnostics play both roles. They guide preprocessing decisions and produce quantities that can be used directly as features. A rolling stationarity statistic can serve as a regime indicator; a detected break date can become a conditioning variable; a persistence estimate can summarize how quickly shocks decay.

A practical workflow begins with a small set of **visual diagnostics**. A plot of the series itself shows whether the level drifts over time, whether volatility changes across subsamples, and whether large moves cluster. Autocorrelation plots summarize lag dependence:

- The **autocorrelation function (ACF)** shows how strongly current values move with past values at different lag lengths
- The **partial autocorrelation function (PACF)** shows the incremental contribution of each lag after controlling for shorter lags
- A **Q-Q plot** compares the empirical distribution with a reference distribution, typically the normal, and quickly reveals skewness, fat

tails, and outliers

These plots do not prove stationarity or non-stationarity, but they indicate which questions to ask next and which formal tests are worth running.

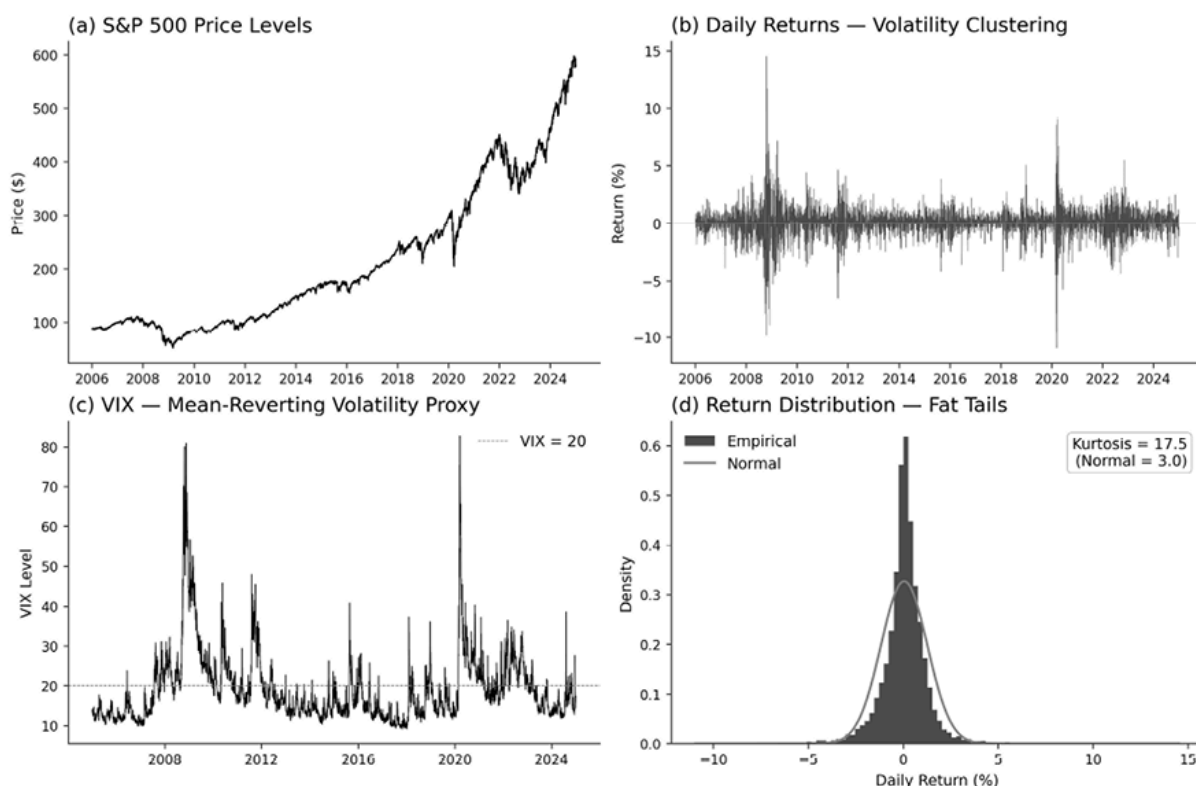


Figure 9.1: A set of four graphs, demonstrating different ways of looking at the data distribution

Figure 9.1 illustrates this first pass for S&P 500 data. The level series trends and is therefore an obvious candidate for non-stationarity. Returns fluctuate around a more stable mean but show volatility clustering. VIX behaves more like a persistent, mean-reverting state variable than a drifting price series. The return distribution departs visibly from normality, especially in the tails. Together, these views suggest three follow-up questions. Is the series stationary? Has it changed regime? If it is non-stationary, what is the least destructive transformation that makes it usable?



Implementation : See `01_visual_diagnostics` for the full workflow, including visual inspection, stationarity tests, autocorrelation diagnostics, distribution diagnostics, and rolling stationarity features.

Stationarity tests as features

Many time series methods assume that the object being modeled has reasonably stable statistical properties over time. In the simplest case, this means that the mean, variance, and dependence structure do not drift in ways the model cannot absorb. Prices often violate this requirement because shocks to the level can persist for long periods. Returns, spreads, volatility measures, and some alternative data series are more likely to exhibit approximate stationarity, though this varies across assets, horizons, and regimes.

Two complementary tests are especially useful because they begin from opposite null hypotheses:

1. The **Augmented Dickey-Fuller (ADF)** test takes a **unit root** as its null hypothesis. In practical terms, a unit root means that shocks to the series have persistent effects rather than fading quickly, which is one common form of non-stationarity. Rejection of the ADF null is therefore evidence against a unit root and, under the chosen deterministic specification, evidence in favor of stationarity or trend-stationarity.
2. The **Kwiatkowski-Phillips-Schmidt-Shin (KPSS)** test reverses the null. Depending on the specification, it tests whether the series is stationary around a constant level or around a deterministic trend. Rejecting the KPSS null is therefore evidence against that form of

stationarity. Running both tests reduces the asymmetry that arises when inference depends on a single null hypothesis.

The **joint interpretation** is useful but should not be treated mechanically:

- If ADF rejects the unit-root null and KPSS does not reject the stationarity null, the evidence favors stationarity under the chosen specifications
- If ADF does not reject the null and KPSS rejects the null, the evidence favors non-stationarity
- If both reject, the tests are sending conflicting signals: the series may contain a deterministic trend, structural breaks, nonlinear mean reversion, changing volatility, or other forms of instability not captured cleanly by either test
- If neither rejects, the result is best treated as inconclusive, often because the window is short, the process is highly persistent, or the tests have limited power

For feature engineering, the test outputs matter as much as the final label. The ADF and KPSS statistics, along with their p-values, can be tracked over rolling windows:

- An ADF statistic moving upward toward zero, or an ADF p-value rising, suggests weaker evidence against a unit root
- A rising KPSS statistic or a declining p-value suggests weaker evidence for stationarity

These diagnostics are therefore not just gatekeepers for preprocessing. They are model-based summaries of persistence and regime stability that can be used as lagged inputs to downstream models.

In **cross-sectional settings**, these tests are often most informative when applied to spreads, valuation ratios, volatility proxies, residualized returns, and slowly evolving firm characteristics rather than to raw daily

equity returns. Returns are usually closer to stationary than prices because differencing removes much of the persistence in levels, but volatility clustering, changing liquidity, and regime shifts can still violate strict stationarity. The feature is therefore not simply “stationary versus non-stationary” but a time-varying measure of how stable the underlying process appears at the time of decision.



Implementation : `@1_visual_diagnostics` shows rolling ADF and KPSS computation, joint interpretation, and examples of time-varying stationarity features.

Structural break features

A series can fail a stationarity test for different reasons. It may drift continuously, as prices often do, or it may be mostly stable but interrupted by one or more regime changes. Break diagnostics focus on the second case.

The **Zivot-Andrews** test (Zivot and Andrews, 1992) extends the unit-root framework by allowing one break to be chosen endogenously from the data. It is useful when a single major discontinuity is plausible, and a genuine unit root must be distinguished from a broken but otherwise stable process. For longer samples, multiple breaks are often more realistic. In that setting, **Bai-Perron** (Bai and Perron, 1998) segmentation is more appropriate because it identifies several candidate break dates rather than forcing the sample into a single break or none at all.

For online monitoring, the problem is different. The goal is not to explain the full sample retrospectively, but to detect instability as it develops. **CUSUM** statistics accumulate deviations from a target level and trigger when sustained drift becomes too large to ignore. This makes them useful

monitoring features even when the exact break date will only be clear in hindsight.

A complementary approach treats break detection as a supervised classification problem. Instead of relying on a single statistic, combine weak signals from several sources, such as shifts in location, scale, dependence, or distributional shape, and let the classifier aggregate them. This approach is often more effective when labeled break examples are available and when no single classical test is decisive across all regimes.

As with stationarity tests, the outputs become features. Break dates, time since the most recent break, pre- and post-break means, CUSUM statistics, and classifier probabilities can all be used as conditioning information for later models. A break detector is therefore both a diagnostic tool and a feature generator.



Implementation : `@2_structural_breaks` shows classical break tests, online monitoring, and the classification-based break-detection pipeline.

Fractional differencing features

When diagnostics indicate non-stationarity, the standard response is **differencing**. Ordinary first differencing applies $(1 - L)x_t = x_t - x_{t-1}$, which is effective but blunt because it removes the low-frequency component in one step. Fractional differencing generalizes this operator by allowing the differencing order to be real:

$$(1 - L)^d x_t = \sum_{k=0}^{\infty} \omega_k x_{t-k}, \quad \omega_0 = 1, \quad \omega_k = -\omega_{k-1} \frac{d - k + 1}{k} = (-1)^k \binom{d}{k}.$$

For $0 < d < 1$, the filter applies a slowly decaying sequence of lag weights, so persistence is attenuated more gradually than under ordinary

first differencing. The transformed series is therefore not simply “between levels and returns” in an informal sense; it occupies the continuum between no differencing and first differencing defined by the fractional integration order.

In the **fractional ARIMA** (**ARFIMA**) literature, this same operator allows the integration parameter to be non-integer. For the stationary long-memory case, typically $0 < d < 0.5$, autocorrelations decay hyperbolically rather than at the exponential rate associated with short-memory ARMA models (Granger and Joyeux, 1980; Hosking, 1981).

In practice, fractional differencing is implemented with a finite lookback rather than the full infinite sequence of weights. The transform combines the current observation with past values whose weights decay with lag; weights beyond a truncation threshold or fixed maximum lookback are dropped. Two boundary conventions then differ on what to do at the start of the sample:

- A **full-window** implementation treats the first observations as unavailable until the required lookback period has accumulated, resulting in a warmup period and a corresponding validity mask. This is the convention used by fixed-width fractional differencing implementations inspired by López de Prado.
- A **boundary-partial** implementation, such as the truncated-weights approach used in `ml4t.engineer.features.fdiff.ffdiff` , applies whatever weights are available near the start of the sample. This preserves the row count but means that early observations are produced by a shorter effective filter than later observations.

Either convention is defensible, but the choice is part of the feature definition. Downstream models should know which observations rely on a partial filter and how many observations would be lost under the corresponding full-window convention.

That mask, and the associated loss of observations, are part of the feature definition. Changing d changes how slowly the weights decay, and changing the truncation rule changes how much history must be retained. Both, therefore, affect two things at once: how much persistence remains in the transformed series, and how many observations are usable for modeling.

This also clarifies what it means to choose or estimate d . The question is not whether fractional differencing is mathematically elegant, but how much persistence must be removed to obtain a usable representation of the series.

A practical workflow :

1. Evaluates a bounded grid of d values over the training window.
2. Computes a stationarity diagnostic, such as the ADF statistic, for each transformed series.
3. Assesses how much correlation with the original series is retained.

The target is usually the smallest d that yields an acceptable stationarity diagnostic while preserving as much memory as possible, rather than a globally “optimal” value computed on the full sample.



Implementation : 03_fractional_differencing

illustrates fixed-width fractional differencing, bounded d -grids, walk-forward-safe d -selection, ADF-based diagnostics, and validity-mask handling.

When it works well, fractional differencing yields a more stable representation than raw levels without paying the full information cost of ordinary first differencing. The next section moves from preprocessing transforms to a richer class of signal decompositions—Kalman filters,

spectral methods, wavelets, and path signatures—that expose latent structure rolling statistics cannot reach.

9.2 Transforming signals to uncover hidden structure

Signal transforms can be grouped by the kind of **hidden structure** they recover: latent state, recurring frequency, time scale, and path shape. Rolling statistics summarize what happened over a window, but they compress away much of the structure that may matter for prediction. A moving average captures the average level, and rolling volatility captures dispersion, but neither tells us whether the path was smooth or jagged, whether fluctuations occurred early or late in the window, or whether a regular cycle was present beneath the noise. Signal transforms address this limitation by mapping the raw series into a new representation and then extracting features from it.

The methods in this section serve different purposes, but they share the same logic:

- **Kalman filters** summarize the hidden state that may underlie noisy observations
- **Spectral methods** summarize how variation is distributed across frequencies
- **Wavelets separate** the series into components at different scales
- **Path signatures** summarize the ordered geometry of a path rather than just its endpoints or moments

In each case, the goal is to convert raw sequential data into summaries that may be more informative than simple rolling statistics.

Kalman filter features

A simple moving average uses a fixed window and fixed weights. That makes it easy to interpret, but also rigid. In a strong trend, it may react too slowly, while in a noisy sideways market, it may react too quickly. A Kalman filter takes a different approach. Instead of mechanically averaging past observations, it maintains a running estimate of an underlying latent state and updates it each time a new observation arrives. Unlike an exponential moving average or a rolling linear trend regression, a Kalman filter does not commit to a fixed lookback period; its gain **adapts to the estimated signal-to-noise ratio** .

The key idea is to distinguish between what we observe and what we are trying to infer. The observed series may be a price, return, spread, or other market variable. The latent state is the smoother or more persistent object we believe generated those observations, such as an underlying level and trend. In a **state-space model** , the hidden state evolves over time, and the observed series is treated as a noisy measurement of that state.

In a local linear trend model, the state contains both a level and a slope. The filter alternates between two steps:

1. In the **predict** step, it projects the state forward using the model's transition rule.
2. In the **update** step, it revises that projection using the new observation.

The result is a feature extraction procedure that balances persistence against surprise. When the series behaves smoothly relative to the assumed noise level, the estimated state moves steadily. When observations become erratic, the filter becomes more cautious.

Originally developed for navigation and control (Kalman, 1960), the model can be written as:

$$x_t = Fx_{t-1} + w_t, \quad y_t = Hx_t + v_t$$

where x_t is the hidden state, y_t is the observed value, F governs how the state evolves, and w_t and v_t represent process and observation noise.

This representation yields several useful features:

- The **Kalman level** is the filtered estimate of the underlying level and serves as an adaptive trend anchor
- The **Kalman trend** is the slope component of the state and serves as a momentum feature without requiring a fixed lookback
- The **innovation** is the prediction error, that is, the gap between what the filter expected and what it observed; it is a natural surprise variable
- The **state uncertainty** is the estimated variance of the state and measures how confident the filter currently is in its own estimate

These outputs often carry more information than a conventional moving-average crossover because they distinguish direction, surprise, and confidence rather than collapsing everything into a single smoothed line. The same state-space logic also extends naturally from single-series trend extraction to dynamic hedge-ratio estimation, where coefficients such as β_t are allowed to evolve over time rather than being treated as fixed constants.

For the `etfs` case study, applying a local-linear-trend Kalman filter to daily returns produces a trend feature that is more stable than 20-day momentum during choppy markets (fewer whipsaws) and comparably responsive during sustained trends.

On SPY, the Kalman slope achieves an information coefficient of -0.16 with 5-day forward returns, roughly nine times stronger than a 20-day rolling OLS slope (-0.02); both are negative, consistent with short-term mean reversion, but the adaptive filter separates signal from noise far

more effectively. The innovation feature shows elevated values around earnings seasons and macro releases—periods when the filter’s smooth model is repeatedly surprised, which can condition the triage of other signals.

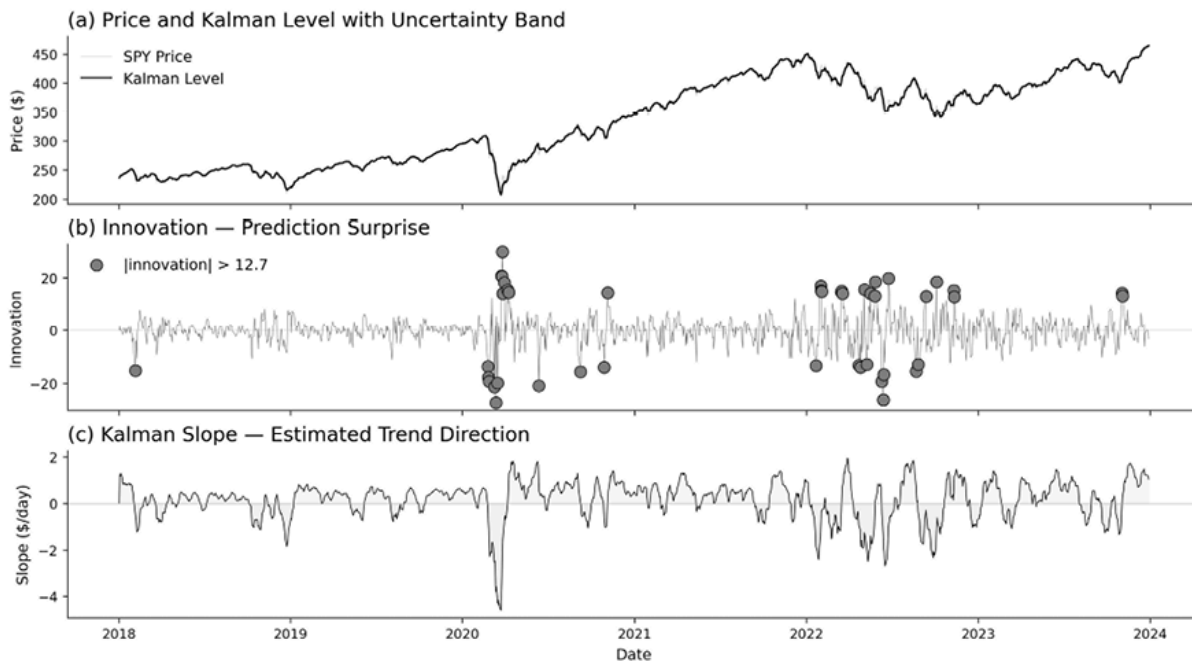


Figure 9.2: Kalman filter smoothing



Implementation : `04_kalman_filter` shows Kalman filter feature extraction, including MLE-based noise covariance estimation, walk-forward refitting, and a dynamic hedge-ratio application for pairs trading.

Spectral features

Kalman filters work in the time domain: they update estimates one observation at a time. Spectral methods answer a different question. Instead of asking how the series evolved day by day, they ask whether part of the variation is organized around **recurring cycles**. A weekly trading rhythm, a monthly rebalance effect, or a quarterly earnings pattern

may be difficult to see directly in the raw series but may become more visible when the series is decomposed by frequency.

A **frequency** is simply a rate of repetition, measured in cycles per unit time. Its reciprocal is the **period**. A weekly cycle in daily data has a period of 5 and a frequency of 1/5; a monthly cycle has a period of about 21 and a frequency of 1/21. The **discrete Fourier transform** converts a finite time series into a set of sinusoidal components at different frequencies. The **fast Fourier transform (FFT)** is the standard algorithm used to compute that transform efficiently. The resulting **power spectrum** shows how much of the series' variance is associated with each frequency band.

The basic transform is:

$$X_k = \sum_{t=0}^{N-1} x_t e^{-2\pi i k t / N}$$

but for feature engineering, the interpretation matters more than the formula. A large value of $|X_k|^2$ at a weekly frequency means that the series contains a strong weekly component; a diffuse spectrum means that no single frequency dominates. The spectrum, therefore, summarizes how structured or noisy the series appears when viewed through the lens of periodic repetition.

Several features follow directly from this representation:

- **Spectral energy** measures total power, or power in a target band, and captures the strength of cyclical structure.
- The **dominant period** is the period associated with the largest peak in the rolling spectrum.
- **Spectral entropy** measures how concentrated or diffuse the normalized power spectrum is: low entropy implies a small number

of strong periodic components, while high entropy implies a flatter, more noise-like distribution of energy.

- The **low-frequency ratio** measures the fraction of energy below a chosen cutoff, such as $(1/21)$, and provides a compact indicator of whether variation is concentrated in slower trend-like components or faster noisy ones.

These features must be computed causally. At time t , the transform should use only a trailing window such as $[t - W, t]$, not the full sample. Window length matters because frequency resolution depends on it: a quarterly cycle cannot be resolved well in a very short window. In practice, these features are often built on returns or other approximately stationary inputs, and several window lengths are useful; for example, 21-, 63-, and 126-day windows capture different cycles in daily data.

Spectral features complement the deterministic calendar features from *Chapter 8*. Calendar encodings impose known cycles through sine and cosine pairs. Spectral features ask whether a cycle is actually present in the data and whether its strength is rising, stable, or fading. That distinction matters whenever cyclical structure is itself time-varying.



Implementation : See `05_spectral_features` for rolling FFT feature extraction, Welch PSD estimation, and a time-frequency heatmap that tracks spectral regime shifts.

Wavelet features

Fourier methods identify which frequencies are present, but they are much less effective at telling us when those frequencies were active. Wavelets address that limitation by providing a **multi-resolution** representation of the series. A wavelet decomposition separates the series

into coarse components that capture slower variation and finer components that capture short-lived detail, while retaining temporal localization. The detail bands provide a rough horizon map, from very short 2- to 4-day fluctuations through 32- to 64-day components, while the approximation term captures the slower trend.

This makes wavelets useful for structures that are localized in time, such as a volatility burst that lasts only a few days or a cycle that appears only during a particular regime. In contrast to the Fourier transform, which is naturally global, the wavelet transform can preserve both scale and timing.

In feature engineering, wavelets are especially helpful for **horizon discovery**. They can show whether the informative structure in a series lives mainly at very short horizons, intermediate horizons, or slower trend-like scales. That information can then guide the design of simpler causal features built on trailing windows, fitted filters, or band-limited transforms.

A **practical caution** is important here. Standard wavelet decompositions are often not causal in their usual offline form because coefficients at time t may depend on observations on both sides of t . That makes them well-suited to exploratory analysis and offline diagnostics, but not automatically safe for live production pipelines. When wavelets reveal that a particular scale is informative, the usual next step is to build a causal proxy at that horizon rather than to deploy the raw offline coefficients directly.

Wavelets are therefore best viewed as a bridge between raw data and deployable features. They help answer the question, “At what horizons does the relevant structure live?”



Implementation : See `05_spectral_features` for wavelet multi-resolution decomposition as a research diagnostic, including scale-variance analysis and the translation of wavelet insights into causal rolling-window proxies.

Path signature features

Spectral methods summarize cycles, and wavelets summarize variation by scale. Path signatures address a different problem: they summarize **path shape**. Two windows can have the same start, end, and realized volatility, yet differ materially in how the path got there. A steady rise followed by a sharp reversal is not the same pattern as an early selloff followed by a gradual recovery. Traditional lag features often treat those two paths as more similar than they really are because they compress the window into endpoint and dispersion summaries. Signatures are designed to preserve more of the ordered geometry.

A path signature, rooted in **rough path theory** (see Chevyrev et al., 2016, for an accessible introduction), is a collection of ordered integrals that summarize how a path evolves through time:

- At *depth-1*, the signature records total displacement, which is essentially net change
- At *depth-2*, it begins to capture interactions between coordinates, including lead-lag structure
- Higher depths capture increasingly fine geometry, but the number of terms grows rapidly

For feature engineering, the main intuition is that signatures preserve sequence information that ordinary lag stacks often discard.

For financial series, **time augmentation** is especially important. Without an explicit time coordinate, two paths with the same geometric trace but different ordering may look too similar at low depth. By adding a monotone time coordinate, the signature can distinguish “rose early, then faded” from “fell early, then recovered.” This is one reason signatures can be useful when the order itself contains a signal.

A second practical distinction is between full signatures and **log-signatures**. The full signature contains algebraic redundancy. The log-signature is a more compact representation that removes much of that redundancy while preserving the expressive information needed for prediction tasks. In practice, low-depth log-signatures are often the most sensible starting point because they balance expressiveness against dimensionality.

Path signatures are **not a general replacement for simpler features**. They are most useful when path shape genuinely matters, when ordering contains signal beyond endpoint summaries, or when we want a compact representation of a multi-dimensional path. In those settings, they complement standard return, volatility, and momentum features rather than displacing them. For most daily applications, depth-2 log-signatures are the sensible starting point; higher depths quickly increase dimensionality and are most defensible when path shape is central to the problem.

The same general principle will reappear in later chapters in learned form. Temporal convolutional networks, recurrent models, and transformer encoders also transform raw sequences into richer internal representations and then expose hidden states, embeddings, or forecasts as features.



Implementation : See `06_path_signatures` for the mathematical formulation (iterated integrals, log-signatures, time augmentation), computation using the



`esig` library, and a head-to-head comparison against traditional lag features.

With latent state, frequency, scale, and path structure now available as features, the next section turns to volatility—the single most predictable property of financial time series—and shows how fitted models such as GARCH, EGARCH, HAR, and rough-volatility estimators convert that predictability into production features.

9.3 Volatility Features

Chapter 8 introduced direct volatility measures built from observed prices, including close-to-close realized volatility and range-based estimators such as Parkinson (1980), Garman and Klass (1980), and Yang and Zhang (2000). Those measures efficiently summarize price variation, but they remain backward-looking summaries. This section turns to fitted volatility models and treats their outputs as features. The aim is not to replace the direct measures from *Chapter 8*, but to model how volatility evolves through time and to extract point-in-time safe summaries from that fitted dynamics.

Fitted volatility models provide three kinds of information that direct aggregation does not:

- They produce **conditional volatility** estimates that update as new shocks arrive
- Some models separate short-, medium-, and long-horizon contributions to current volatility
- They expose parameters that describe persistence and asymmetry, which are often informative in their own right

Throughout this section, the model is the feature extractor. Conditional variance paths, persistence measures, leverage parameters, and horizon-specific components are incorporated into later machine learning pipelines alongside the direct features from *Chapter 8* .

Autoregressive integrated moving average features

Autoregressive integrated moving average (ARIMA) plays a supporting role in this section. Its main use is to remove simple linear structure from a series before volatility modeling or to generate fitted features for ancillary series that are more predictable than daily returns. In practice, residuals are often the most useful output because they provide a de-meaned or de-trended input for later volatility models. For series such as realized volatility, bid-ask spreads, funding rates, or futures basis, the one-step forecast can also be used directly as a feature. For liquid daily returns, however, low-order ARIMA models usually add little as standalone predictors, so ARIMA belongs here mainly as a preprocessing and auxiliary-feature tool.

This distinction matters. For many liquid return series, the direct predictive value of a low-order ARIMA mean equation is modest at best. In those cases, the residual is usually more useful than the forecast because it removes whatever linear dependence is present before the conditional variance model is fit. For series with a clearer autoregressive structure, the fitted ARIMA model can generate two practical features:

- The residual, which measures deviation from the model's expected path
- The one-step forecast, which provides a model-based estimate of the next period's level

Order selection should remain conservative. For stationary inputs, low-order models such as ARIMA(1,0,1) or ARIMA(2,0,1) are usually sufficient; for trending inputs, differencing may be required. Automated selection by information criteria can be useful, but any search over orders must be repeated inside each walk-forward training window. Otherwise, the lag structure itself is chosen with knowledge of later data. In this section, the key outputs are therefore `arima_residual` and, where the series supports it, `arima_forecast`.



Implementation : See `07_arima_features` for ARIMA order selection, residual extraction, and walk-forward evaluation.

Generalized autoregressive conditional heteroskedasticity features

The standard workhorse is the **generalized autoregressive conditional heteroskedasticity** (**GARCH**) model, introduced by Engle (1982) for the ARCH case and generalized by Bollerslev (1986). In a GARCH(1,1) specification, today's conditional variance depends on a constant term, yesterday's squared innovation, and yesterday's conditional variance:

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

For feature engineering, the model matters less as a forecasting object than as a structured decomposition of volatility dynamics. The fitted conditional standard deviation, often stored as `cond_vol`, is the central output. It is a model-based estimate of the volatility state at time t and is often more responsive than a fixed-window realized-volatility estimate because it updates recursively as new shocks arrive.

- The coefficient α summarizes how strongly volatility reacts to recent shocks; this is the natural `shock_impact` feature
- The coefficient β summarizes persistence in the volatility process and becomes a `vol_persistence` feature

Their sum, $\alpha + \beta$, is especially useful because it captures how slowly a volatility shock decays. When $\alpha + \beta$ is close to one, volatility is highly persistent.

That persistence can be translated into a more intuitive quantity. In a stationary GARCH model, the approximate half-life of a volatility shock is $\log(0.5)/\log(\alpha + \beta)$. For SPY, a GARCH(1,1) fit yields $\alpha \approx 0.17$ and $\beta \approx 0.80$, giving $\alpha + \beta = 0.97$ and an implied half-life of about 23 trading days. This helps explain why volatility models often contribute useful features even when mean-return models do not: volatility clusters, and that clustering can be summarized parsimoniously.

The model also implies a long-run variance level:

$$\frac{\omega}{1 - \alpha - \beta}$$

which can be used as a `long_run_vol` feature when the fit is covariance-stationary. That restriction matters. If $\alpha + \beta \geq 1$, the model behaves like an **integrated GARCH (IGARCH)** process and does not imply a finite unconditional variance. In that case, the long-run variance should not be interpreted as a valid feature. In practice, it is safer either to constrain estimation to stationary fits or to explicitly flag nonstationary estimates.



Implementation : See `08_garch_volatility` for GARCH estimation, ARCH-effect testing, and conditional volatility extraction.

Asymmetric volatility - Exponential GARCH features

Standard GARCH treats positive and negative shocks symmetrically because it depends on squared innovations. Financial markets often do not behave that way. In many equity markets, negative returns are followed by larger increases in volatility than positive returns of the same magnitude.

The **exponential GARCH (EGARCH)** model captures this asymmetry by modeling log variance:

$$\log(\sigma_t^2) = \omega + \alpha \frac{|\epsilon_{t-1}|}{\sigma_{t-1}} + \gamma \frac{\epsilon_{t-1}}{\sigma_{t-1}} + \beta \log(\sigma_{t-1}^2)$$

The asymmetry enters through γ . When γ is negative, negative shocks raise future volatility more than positive shocks do (Nelson, 1991). That parameter is the natural `leverage_effect` feature. Its sign and magnitude summarize whether the asset exhibits the familiar equity-style leverage pattern and how strong that pattern is in the current estimation window.

This is often more useful as a conditioning variable than as a forecast on its own. A strongly negative leverage parameter indicates that downside shocks have disproportionate consequences for the volatility state. That can matter for risk controls, for feature interactions with momentum or carry signals, and for interpreting whether a recent increase in volatility reflects generic turbulence or specifically downside stress.

An alternative is the **Glosten-Jagannathan-Runkle GARCH (GJR-GARCH)** model (Glosten et al., 1993), which introduces asymmetry through an indicator for negative shocks rather than through the EGARCH log-variance form. For feature extraction, the two models serve a similar purpose: they add a parameter that distinguishes

symmetric persistence from downside-sensitive persistence. The choice between them is usually empirical.



Implementation : See `08_garch_volatility` for GARCH, EGARCH, and GJR-GARCH estimation and comparison.

Heterogeneous autoregressive volatility features

The **Heterogeneous Autoregressive (HAR) model** (Corsi, 2009) captures daily volatility persistence across different horizons. A standard specification is:

$$RV_{t+1}^{(d)} = c + \beta_d RV_t^{(d)} + \beta_w RV_t^{(w)} + \beta_m RV_t^{(m)} + \epsilon_{t+1}$$

where $RV_t^{(d)}$, $RV_t^{(w)}$, and $RV_t^{(m)}$ denote daily, weekly, and monthly averages of **realized volatility (RV)**.

The horizon-specific volatility measures are inputs of the kind introduced in *Chapter 8*. What HAR adds is a fitted layer on top of them. The coefficients β_d , β_w , and β_m measure how strongly short-, medium-, and longer-horizon volatility contribute to the next day's volatility forecast. A larger monthly coefficient indicates more persistent volatility conditions, whereas a larger daily coefficient indicates greater sensitivity to recent shocks.

HAR is naturally specified on a daily realized-volatility series, ideally constructed from intraday returns. When only daily open-high-low-close-volume data are available, range-based estimators such as Garman-Klass or Yang-Zhang can serve as practical proxies. The model remains the same; only the volatility input changes.

On SPY, the weekly component dominates ($\beta_w \approx 0.43$), consistent with institutional-frequency dynamics driving index volatility. In an out-of-sample comparison, HAR achieves roughly half the RMSE of GARCH (0.061 compared to 0.111), confirming that the multi-horizon decomposition captures persistence structure that a single-lag recursion misses.



Implementation : See `09_har_rough_volatility` for HAR estimation, rolling coefficient tracking, and comparison with GARCH.

Re-estimation cadence and online updates

Volatility features are only useful if they reflect the current volatility environment. That requires two separate decisions: how often to re-estimate model parameters, and how to update features between refits.

The **refit cadence** should match the stability of the parameters and the model's cost:

- GARCH parameters often move slowly enough that weekly or monthly re-estimation is sufficient.
- EGARCH asymmetry parameters usually do not need to be refitted daily either.
- HAR coefficients can be somewhat more regime-sensitive because they summarize changing horizon contributions, but they are still typically re-estimated on a rolling schedule rather than every day.

By contrast, **stochastic volatility** (**SV**) (see *Chapter 5*) models estimated via **Markov chain Monte Carlo** (**MCMC**) are usually

computationally expensive enough that monthly, or even less frequent, full re-estimation is more realistic.

Between refits, however, many features can still be updated online. In GARCH, the conditional variance recursion updates daily using fixed parameter estimates. In HAR, the daily, weekly, and monthly realized-volatility inputs update as new realized-volatility observations arrive. In stochastic-volatility settings, filtering updates can refresh latent-state estimates without rerunning the full posterior.

For large panels, the practical workflow matters as much as the choice of model. A common approach is to re-estimate parameters overnight across the universe and update the recursive features during the next trading day. Where even that is too expensive, an exponential-smoothing approximation can be a reasonable operational substitute, though at the cost of a less interpretable parameterization. Marra (2023) compares these estimators in practice, and Moreira and Muir (2017) show that even simple volatility-managed strategies improve Sharpe ratios across asset classes.

Rough volatility and the Hurst exponent

GARCH and HAR describe persistence well, but they impose relatively smooth volatility dynamics. Rough-volatility methods start from a different empirical observation: volatility is persistent in level, yet the changes in volatility can be much more irregular than standard diffusion models imply. In that sense, volatility can be persistent and rough at the same time.

A compact way to summarize this behavior is the **Hurst exponent**, H . In broad terms:

- $H = 0.5$ corresponds to Brownian scaling
- $H > 0.5$ indicates persistence in increments
- $H < 0.5$ indicates anti-persistence in increments

For log-volatility, Gatheral et al. (2014) documented Hurst exponents around $H \approx 0.1$ for equity indices—well below 0.5—suggesting that volatility shocks are bursty and mean-revert in their increments more quickly than a smooth Brownian specification would imply. The SPY data confirm this pattern: returns show $H \approx 0.5$ (consistent with a random walk), while log-volatility increments give H between 0.12 and 0.23 depending on the estimator—squarely in the rough regime.

For feature engineering, the Hurst exponent is best treated as a **slow-regime descriptor** rather than as a high-frequency signal. A rolling estimate based on log-volatility can indicate whether the current environment is smoother or rougher than usual. When the estimate falls well below 0.5, standard volatility recursions may understate how abruptly volatility can spike and then decay. This does not invalidate GARCH-style features, but it does change how they should be interpreted.

These estimates are noisy, so they should be updated over relatively long windows and interpreted with caution. A 252-day rolling window, refreshed weekly, is often more useful than daily re-estimation. Large one-day moves in the estimate usually reflect estimation noise rather than a genuine structural break. If the Hurst exponent is used on return series rather than on log-volatility, it should be interpreted even more cautiously.



Implementation : See `09_har_rough_volatility` for rescaled-range and detrended-fluctuation-analysis



implementations, rolling Hurst estimation, and cross-asset roughness analysis.

This section treated volatility models as feature generators: ARIMA residuals for prewhitening, GARCH/EGARCH estimates for conditional variance and asymmetry, HAR components for horizon-specific structure, and Hurst-based measures for roughness. The next section turns from volatility itself to uncertainty about fitted states and forecasts.

9.4 Uncertainty features

Model-based features need not stop at point estimates. A fitted procedure also tells us how certain it is about the current state or the next forecast. That uncertainty can be informative in its own right. A narrow posterior or prediction interval indicates a stable estimate; a wide one indicates that the model sees several plausible outcomes. In trading applications, these uncertainty summaries can help condition signal strength, exposure, risk buffers, and the interpretation of other features.

Two distinctions matter:

- First, **uncertainty about a latent state is not the same as uncertainty about a forecast**. A stochastic-volatility model can be uncertain about the current volatility state even before we ask it to predict the next period. An ARIMA model can be relatively certain about the current level of a series while remaining uncertain about its next-step forecast.
- Second, **uncertainty can come from different estimation traditions**. **Bayesian methods** produce posterior distributions directly, whereas **frequentist models** often provide forecast standard errors and prediction intervals.

For feature engineering, both can be useful.

The uncertainty-as-feature principle

Once a fitted model produces a distribution rather than a single value, several additional features become available. Common examples include:

- Posterior standard deviations
- Credible interval widths (the Bayesian version of the frequentist confidence interval)
- Tail probabilities
- Forecast standard errors
- Prediction interval widths

These quantities do not replace the underlying point estimate. They complement it by describing how reliable that estimate appears at decision time.

This perspective is especially useful when two observations share the same point forecast but differ in uncertainty. A volatility forecast of 20 % with a narrow interval carries a different operational meaning from the same 20% forecast with a wide interval. The first suggests a relatively well-identified state; the second suggests that the model sees materially more ambiguity. The uncertainty summary can therefore serve as a conditioning feature even when the central estimate itself is already included elsewhere in the model.



Implementation : See `10_uncertainty_features` for posterior uncertainty from a stochastic-volatility model and forecast uncertainty from ARIMA applied to log realized volatility.

Stochastic volatility features

GARCH models treat conditional volatility as a deterministic recursion once the parameters and past data are given. Stochastic-volatility (SV) models (Taylor, 1982) take a different view. They treat volatility itself as a latent process that evolves with its own shock term, so the current **volatility state is not observed directly** but inferred from the data. Estimation, therefore, yields a distribution over the latent state rather than only a single filtered path.

That distribution produces several useful features:

- The posterior mean of current volatility is the central state estimate
- The posterior standard deviation and the credible interval width quantify the uncertainty of the estimate
- A parameter governing the innovation variance of the log-volatility process summarizes volatility of volatility, and the persistence parameter summarizes how slowly volatility states decay

Together, these quantities distinguish three cases that a single volatility estimate would conflate: volatility is low and well identified; volatility is high and well identified; or volatility is high but poorly identified.

For downstream models, uncertainty about volatility often matters most when it rises independently of the volatility level itself. A large posterior standard deviation means the model is not only detecting elevated risk but is also less certain about where that risk state actually sits. That can be a stronger warning signal than high volatility alone.

Point-in-time handling is critical. For feature extraction, use filtered posteriors rather than smoothed full-sample state estimates that borrow information from the future. In practice, stochastic-volatility models are usually re-estimated less frequently than simpler volatility models because posterior sampling is expensive.



Implementation : `10_uncertainty_features` fits a Student-t stochastic-volatility model in a walk-forward setting, extracts the filtered final-state posterior at each refit, and carries those features forward between refits.

ARIMA forecast uncertainty features

ARIMA models produce more than fitted values and residuals. They also produce forecast distributions that can be used as features to capture the model's confidence in the next step. For volatility-like series, this is often more useful than applying ARIMA directly to returns, because realized volatility, spreads, funding rates, and similar ancillary series tend to have clearer serial structure.

When the target is strictly positive, such as realized volatility, it is often preferable to model log volatility rather than the raw level. The log transform stabilizes scale and avoids negative back-transformed forecasts. It also yields prediction intervals on the original scale that are asymmetric, which is often more realistic for volatility because the downside is bounded near zero, while upside spikes can be large.

Three uncertainty features are especially practical:

- The forecast standard error summarizes uncertainty on the modeled scale
- The prediction-interval width summarizes uncertainty on the original scale
- Their ratio, such as the interval width divided by the forecast level, provides a relative measure of how uncertain the forecast is relative to its central value

High relative uncertainty indicates that the model's forecast is weakly identified even if the point forecast itself looks large.

These features are often better used as conditioning variables than as stand-alone signals. A directional or allocation model may respond differently when its volatility input is estimated precisely than when the same input is surrounded by a wide interval.



Implementation : `10_uncertainty_features` notebook fits rolling ARIMA models to log Garman-Klass realized volatility, back-transforms the interval to the original scale, and tracks forecast-standard-error and interval-width features over time.

As in earlier sections, the fitted-object discipline remains unchanged. Any order selection, interval estimation, and refitting must remain inside the walk-forward protocol. Uncertainty features are only useful when the uncertainty itself is measured without look-ahead bias.

Regime models generate analogous summaries, including uncertainty about the current state assignment and the persistence of transitions. We turn to these next.

9.5 Regime features

Many financial series are better described as moving among recurring market environments than as fluctuating around one stable distribution. Volatility can remain low for extended periods and then shift abruptly into a stressed state. Trend-following signals can work well in persistent markets and fail in choppy ones. Correlations that appear stable in one macro environment can reverse in another (Ang and Bekaert, 2002; see Ang and Timmermann, 2011, for a comprehensive survey of regime-change models in finance). A regime feature attempts to summarize the changing environment.

This differs from a structural break in one important respect:

- A **structural break** marks a discrete change in the data-generating process
- A **regime model** instead assumes that the series can revisit earlier states

A calm market can give way to stress and later return to calm; a trend regime can alternate with a range-bound regime several times within the same sample.

Observable regime rules

The simplest regime features are **deterministic rules** applied to observed data. These remain useful because they are transparent, require no latent-state estimation, and are easy to audit in production. For equities, a volatility threshold, such as the VIX above a fixed level, can indicate a stressed market. A trend rule, such as price above or below a long moving average, can separate upward-trending from downward-trending or range-bound conditions. Other deterministic indicators, such as the choppiness index or a trend-efficiency measure, further refine this classification.

These rules are often dismissed as crude, but they **solve a real problem** . A threshold feature fails transparently. If VIX ceases to be informative for a particular universe or horizon, the failure is easy to diagnose. A more flexible latent-state model can represent richer dynamics, but it also introduces more assumptions and more failure modes. For this reason, deterministic regime indicators are often good **baseline features** even when more elaborate models are added later.

A useful way to think about these features is that they **encode a market context** rather than a forecast. A volatility threshold does not predict returns by itself. It tells the downstream model whether other features

should be interpreted in a calm or stressed environment. In practice, this **conditioning role** is often the most important function of regime features more broadly.

Hidden Markov model features

The standard latent-state approach is the **Hidden Markov Model** (**HMM** ; Rabiner, 1989, provides a standard tutorial). An HMM assumes that an unobserved state variable follows a **Markov chain** , so the probability of tomorrow's state depends only on today's state. Conditional on that hidden state, the observed data are generated from a state-specific distribution. In finance, the observations are often returns, realized volatility, or a small vector of market variables, and the hidden states are interpreted as regimes such as calm and stressed volatility or low- and high-dispersion return environments (Uysal and Mulvey, 2021).

The most important output is the **filtered state probability** , $P(S_t = k \mid y_{1:t})$, which measures the probability that the system is in state k given only information available through time t . This is the central object for feature engineering. By contrast, the smoothed probability, $P(S_t = k \mid y_{1:T})$, uses future observations and is therefore suitable only for retrospective analysis. *Figure 9.3* illustrates the HMM architecture: hidden states generate observed returns through regime-specific emissions.

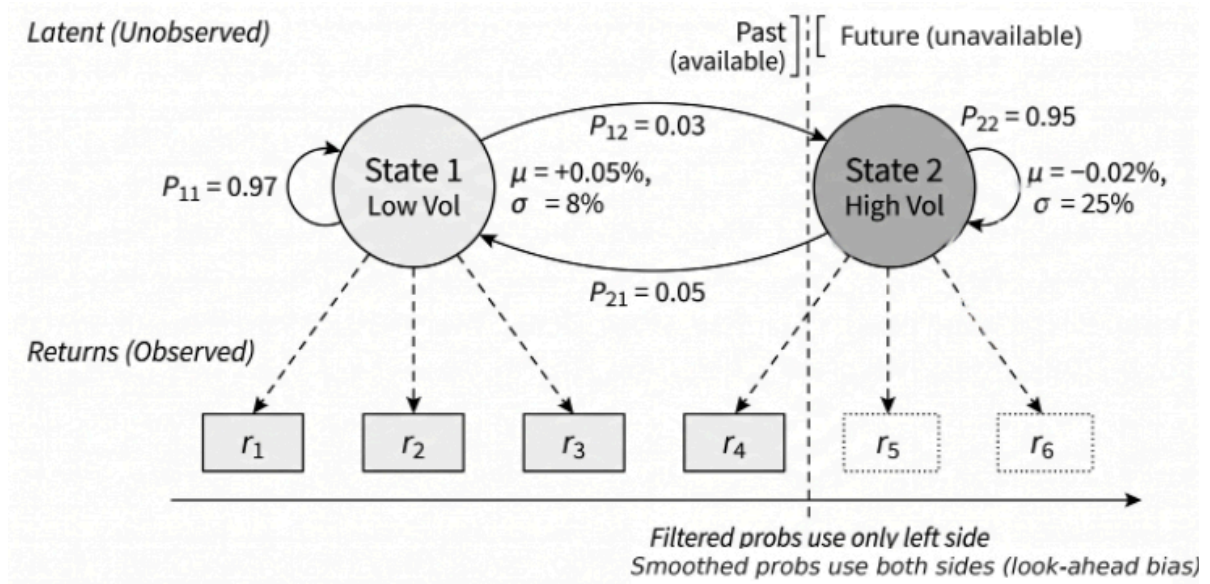


Figure 9.3: Hidden Markov Model architecture

Several other features follow naturally from the fitted model:

- The filtered probability of each state preserves uncertainty and is usually more useful than a hard state label.
- The **transition matrix** provides quantities such as the probability of moving from a low-volatility state to a high-volatility state in the next step.
- The **expected duration** of state k , $\frac{1}{1-p_k}$, summarizes regime persistence, where p_k is the probability of remaining in that state.
- The **entropy** of the filtered probability vector quantifies classification uncertainty: it is low when a single state dominates and high when the model is uncertain among states.

Two implementation issues deserve explicit attention:

- The states are unlabeled. What one estimation window calls “state 0” may correspond to “state 1” in the next window. To keep features comparable over time, relabel the fitted states using a stable characteristic, such as variance, if the states primarily differ in volatility.

- The number of states should remain modest unless there is a strong reason to do otherwise. Two-state specifications are often easier to interpret and more stable out-of-sample than richer models that fit historical noise.



Implementation : `11_hmm_regimes` walks through the forward-filtering logic, contrasts filtered and smoothed probabilities, and extracts transition, duration, and entropy features from HMM fits.

Markov-switching autoregressive features

An HMM with simple state-specific emissions is useful when the main difference across regimes lies in the unconditional distribution of the observations. Sometimes the dynamics themselves also change across states. Hamilton's (1989) **Markov-switching autoregressive (MS-AR)** model addresses this by allowing the autoregressive process to vary by regime.

This produces a different class of features. In addition to regime probabilities and regime-specific variances, the fitted model yields **regime-specific autoregressive coefficients** . These coefficients summarize whether persistence, mean reversion, or short-horizon momentum differs across states. A market can, for example, display stronger continuation in one regime and weaker or negative serial dependence in another. In that case, the regime-specific autoregressive coefficient becomes a natural conditioning feature for downstream return models.

MS-AR models are most useful when the question is not only which regime the market is in but also how the series' behavior changes across regimes. If the regimes primarily differ in volatility, a simpler HMM may be sufficient. If they also differ in persistence or directional dynamics, the autoregressive extension can add useful structure.



Implementation : `11_hmm_regimes` includes a Markov-switching autoregression alongside the HMM, so the differences in the extracted features are visible within the same workflow.

Distribution-based regime features

Moment-based regime models, including many HMM specifications, distinguish states through quantities such as the mean and variance. That is often appropriate, but it can miss cases where the main difference lies in distributional shape, especially in skewness and tail behavior. Two return windows can have similar means and volatilities while differing materially in downside concentration or upside convexity.

A distribution-based approach addresses this by treating each rolling window as an empirical distribution and clustering those windows directly. The **Wasserstein distance** is a natural metric for this task because it measures the cost of reshaping one distribution into another. In one dimension, it can be interpreted as the distance between quantile functions, making it sensitive to differences across the entire distribution rather than only at a few summary moments.

This changes both the regime definition and the resulting features. The basic output is the **cluster assignment**, which acts as a regime label. The distance from the current window to its assigned cluster center measures how typical or atypical the current regime instance is. A window that lies

far from every centroid may indicate an unusual environment, even if it is still assigned to the nearest cluster.

A further feature can be obtained by comparing distribution-based and moment-based classifications. When a Wasserstein-based assignment differs from a moment-based regime label, the disagreement itself is informative, suggesting that tails or skewness matter more than the mean and variance alone.

This type of feature is especially relevant when the trading problem is sensitive to downside asymmetry, jump risk, or changing tail exposure rather than just to volatility level.



Implementation : On a synthetic two-regime benchmark, Wasserstein K-means achieves an adjusted Rand index of 0.87, compared with 0.31 for moment-based K-means—a large gap that confirms the distributional metric captures regime structure that summary statistics miss.

`12_wasserstein_regimes` applies this to S&P 500 returns using the stream-lift methodology of Horvath et al. (2021).

Using regime features downstream

Regime features are usually more effective as **conditioning variables** than as hard switches between separate predictive models. The central reason is that regime inference is uncertain precisely when regime information matters most. Near transitions, the model may assign meaningful probabilities to several states simultaneously. A hard-switching system must still commit to one regime-specific model, which creates instability when small changes in regime probability lead to large changes in the active predictor.

A single predictive model that receives regime features as inputs handles this more gracefully. Filtered state probabilities, regime entropy, duration measures, and regime-conditioned interaction terms enable the model to learn that a signal behaves differently in calm and stressed environments without forcing an abrupt change in the model. As regime certainty falls, the regime inputs naturally become less decisive. The system degrades smoothly instead of switching discontinuously at the boundary between two estimated states. Shu and Mulvey (2025) show a similar approach to factor allocation, confirming that regime probabilities, treated as continuous features, outperform hard regime-switching rules.

This does not make separate regime-specific models useless. They can still be informative when there is a strong economic reason to believe that distinct mechanisms operate in different states and when each regime contains enough data to support separate estimation. But they impose a cost in sample size, increase operational complexity, and are often fragile near the boundary cases that matter most in live trading.



Implementation : `13_regime_as_feature` constructs both unified and regime-conditioned designs and shows how regime probabilities and entropy can be incorporated directly into a downstream gradient-boosting workflow.

All features developed so far—diagnostics, signal transforms, volatility models, uncertainty measures, and regime indicators—are computed asset-by-asset from each series' own history. The next section shows how to convert these temporal outputs into cross-sectional and panel features that place each asset in the context of its peers and the broader universe, completing the bridge to the multi-asset modeling workflow used in later chapters.

9.6 Cross-sectional and panel features

A conditional volatility estimate of 25 % annualized means something different for a utility stock than for a biotech. The same is true for a Kalman trend, a regime probability, or a spectral-energy measure.

Temporal features are first computed asset by asset from each series' own history. They become more useful in multi-asset settings when placed in a cross-sectional context or combined across related assets. The result is a second layer of feature engineering: transform temporal outputs into **relative, pairwise, and universe-level signals** .

Cross-sectional transforms of temporal features

Cross-sectional ranking is the simplest and often the most useful transform. At each date, take a temporal feature already finalized for each asset and replace its raw value with a rank, percentile, or z-score relative to the eligible universe. A raw Kalman trend then becomes a universe-relative momentum signal, a conditional-volatility estimate becomes a volatility percentile, and a regime probability becomes a measure of whether the asset looks more stressed or calmer than its peers.

These transforms solve two problems. They reduce the importance of absolute scale and align the feature with cross-sectional prediction tasks, where the objective is to sort assets rather than forecast their standalone levels. Ranking is often the most robust choice because it is less sensitive to outliers than z-scores and remains comparable across heterogeneous assets. Z-scores retain more information about cross-sectional distance and are often preferable when dispersion itself matters. Percentiles provide a bounded compromise that is easy to interpret.

The temporal model still does the primary extraction. The cross-sectional transform does not create the signal from scratch; it re-expresses an already meaningful temporal quantity in a form suited to multi-asset decisions.



Implementation : See `14_panel_features` for cross-sectional ranking of 60-day momentum and volatility across an ETF universe and how those relative positions evolve over time.

Relative and benchmark-adjusted features

Some temporal features become more informative when expressed relative to a benchmark rather than the full cross-section. Subtracting market or sector momentum from an asset's momentum isolates the idiosyncratic component. Dividing an asset's conditional volatility by market volatility distinguishes asset-specific risk from market-wide stress. Subtracting an asset's stress probability from the universe average shows whether it is unusually defensive or unusually exposed relative to the current environment.

This distinction matters because many temporal features mix systematic and idiosyncratic variation. An oil-sector ETF can show strong absolute momentum simply because the entire equity market is rising. Once broad-market momentum is removed, the residual measures whether the asset is outperforming or lagging that common trend. The same logic applies to volatility, drawdown risk, and regime indicators.

Benchmark choice should follow the economic question. For broad equity universes, a market index is often the natural first benchmark. Sector,

country, duration-bucket, or commodity-sleeve benchmarks can be more appropriate when the goal is to separate local structure from a narrower common factor.



Implementation : `14_panel_features` constructs market-relative momentum and relative volatility for ETFs by subtracting or scaling with SPY-based benchmarks.

Pairwise temporal features

Multi-asset feature engineering also includes relationships between assets. Some of the most useful features are defined in terms of spreads, hedge ratios, or rolling dependence measures rather than in terms of a single series. Cointegration diagnostics summarize whether two prices share a stable long-run relationship. A time-varying hedge ratio from a Kalman filter summarizes how that relationship changes through time. The spread built from that hedge ratio can then produce standardized deviation measures, such as z-scores, and mean-reversion summaries, such as an estimated half-life.

These pairwise features are most natural in relative-value and statistical-arbitrage settings, but their usefulness is broader. A rolling beta to a benchmark, a rolling correlation regime indicator, or a cointegration score can all serve as conditioning features in a larger predictive model. They describe whether an asset is moving with, away from, or back toward a related instrument or common factor.

Interpret these quantities as state summaries, not trading rules. A short half-life does not by itself justify a trade, and a cointegration test does not guarantee a robust spread after costs. The important point is that the pairwise model produces temporally structured features that are unavailable from single-asset transformations alone.



Implementation : `14_panel_features` screens ETF pairs with Engle-Granger and Johansen tests, compares static and Kalman-filtered hedge ratios, and derives spread z-scores and half-life estimates from the resulting spreads.

Universe-level aggregation

Once asset-level temporal features are available, they can be aggregated across the universe to summarize market-wide conditions. The cross-sectional mean of conditional volatility measures the overall volatility state. Cross-sectional dispersion shows whether assets are behaving similarly or differentiating sharply. The fraction of assets with elevated stress probability becomes a breadth measure of instability. An asset's deviation from the universe average then measures whether it is moving with the market-wide regime or against it.

These aggregates are especially useful for regime features. A high average stress probability suggests that pressure is broad rather than idiosyncratic. High breadth indicates that many assets are simultaneously under stress. High dispersion, by contrast, indicates disagreement across the universe and often accompanies rotation, segmentation, or selective risk-taking. The combination is often more informative than any single asset-level regime label.

The same aggregation logic applies beyond regime models. Cross-sectional averages of Kalman trends, volatility forecasts, or forecast-interval widths can summarize the market's prevailing direction, risk state, or level of uncertainty.



Implementation : `14_panel_features` constructs `universe_crisis_prob` , `crisis_breadth` , and



`regime_dispersion` to show how such aggregates behave through time.

Point-in-time handling in panels

The **order of operations** is non-negotiable. First, compute each asset's temporal feature from its own trailing history. Then align those asset-level outputs at the decision date. Only after that step should the cross-sectional rank, benchmark adjustment, pairwise transform, or universe aggregation be applied. Reversing the order leaks information across assets or across time.

The **universe definition** matters as well. When assets enter or leave the tradable set, cross-sectional ranks shift even if no underlying signal changes. Fixed-composition checks, eligibility filters defined in advance, and explicit handling of missing values help separate genuine signal movement from composition effects. Benchmark mappings require the same discipline: sector, country, or market assignments should be those known at the time of decision, not later classifications.

Cross-sectional and panel transforms add a final layer of structure to model-based features. They turn asset-level temporal estimates into relative signals, pairwise states, and market-wide summaries that can be consumed directly by downstream models.

9.7 Summary

This chapter expanded feature engineering from direct transformations to features produced by fitted procedures. Diagnostics contribute rolling measures of stability and break risk. Signal transforms recover latent state, frequency structure, scale, and path geometry. Volatility models add conditional variance, persistence, asymmetry, and horizon-specific

components. Uncertainty models turn interval width and posterior dispersion into usable conditioning variables. Regime models summarize market state through thresholds, filtered probabilities, durations, and distribution-based clusters. In multi-asset settings, these temporal outputs become more useful once ranked, benchmarked, paired, or aggregated across the universe.

The practical challenge is not to include every feature a method can generate, but to keep the set interpretable and nonredundant. Many families overlap, so a compact baseline often works better than a maximal catalog: one or two volatility features, one regime feature, one signal-transform feature, and a limited set of cross-sectional transforms are usually enough to establish what the fitted methods add. From there, feature selection can decide whether additional structure improves the model or only increases complexity.

Chapter 10 applies the same fitted-object discipline to text features, in which timing, decay, and embedding-model dependencies serve as the binding constraints.

10

Text Feature Engineering

Quantitative investors have always sought informational edges. Yet the richest source of financial insight—text—has historically resisted systematic analysis. Earnings calls, analyst reports, regulatory filings, news articles, and social media posts contain forward-looking statements, management sentiment, and market-moving revelations that no price chart can capture.

The challenge is scale and structure. Most enterprise information is not stored in clean relational tables; it appears in text, audio, images, presentations, logs, filings, transcripts, and messages. Common industry estimates put the unstructured share of enterprise data around 80-90%, but the exact number matters less than the practical implication: much of the information relevant to investors arrives in forms that require representation, extraction, and timestamp discipline before it can become a tradable feature.

An earnings call transcript arrives at 4:30 PM. We extract a “guidance confidence” feature by comparing current language to the prior quarter. The feature becomes tradable only after we verify (a) when we received it, (b) that our embedding model was trained before the call date, and (c) that no revisions contaminate the historical series. These constraints—not the NLP model—often determine whether a text signal survives backtesting.

Investor attention to corporate filings is now measurable at internet scale, and the volume of text long ago outgrew manual analysis.

By the end of this chapter, you will be able to:

- Distinguish lexical features, static embeddings, sequential models, and transformers in terms of the information each representation preserves and loses.
- Explain how transformer self-attention produces contextual embeddings and why this mitigates key limitations of earlier NLP methods, including polysemy, weak context handling, and long-range dependence.
- Apply a practical financial NLP workflow that combines pre-trained checkpoints, domain adaptation when needed, and task fine-tuning for classification or extraction tasks.
- Design text-derived features such as sentiment, narrative surprise, or structured event signals using point-in-time-safe timestamps, model cutoffs, and aggregation rules.
- Evaluate text-derived signals using horizon-aware diagnostics, coverage-aware analysis, and event-time alignment rather than benchmark accuracy alone.
- Use token-level attribution and related diagnostics to audit, debug, and stress-test NLP features before deployment.

We begin by covering lexical and statistical baselines to establish what simple methods can and cannot capture. *Section 10.2* introduces static embeddings that learn semantic relationships from co-occurrence patterns. *Section 10.3* traces the development of sequential models to motivate the transformer revolution. *Section 10.4* covers the transformer architecture, self-attention, and domain-specific variants like FinBERT, with empirical comparisons across paradigms. *Section 10.5* assembles the

production workflow: pipeline contracts, fine-tuning, embedding-based factor construction, structured extraction, and signal evaluation.

Figure 10.1 previews this progression from lexical methods to contextual transformers. The structure traces an increasingly sophisticated search for *context*: lexical methods treat documents as unordered collections of terms; static embeddings capture semantic similarity but assign a single vector per word; sequential models introduce memory but process words one at a time; and transformers enable parallel processing with contextual embeddings that dynamically adjust to surrounding text.

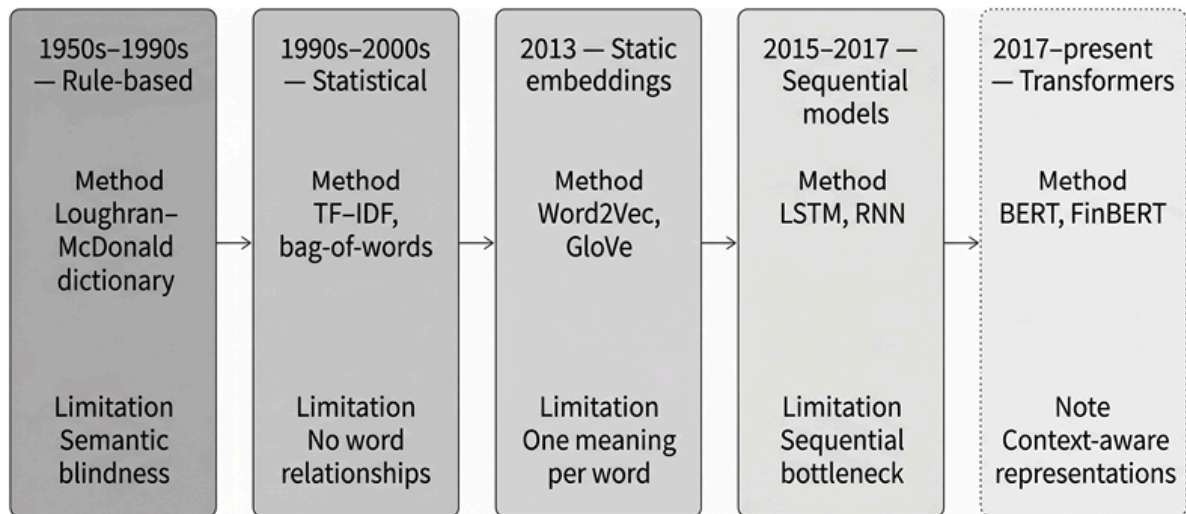


Figure 10.1: From rule-based to data-driven text understanding

The first step in this progression—lexical and statistical models—provides the baselines against which all subsequent methods are measured.

10.1 Lexical and statistical models

The earliest text analysis methods treat documents as collections of terms, extracting features by counting and weighting them. These approaches remain useful baselines, but their semantic blindness motivates the embedding methods that follow.

Financial text analysis progressed through several iterations. Tetlock (2007) established that the tone of news predicts market outcomes under event-time framing. Loughran and McDonald (2011) showed that general sentiment dictionaries misclassify financial language, motivating the development of domain-specific lexicons. More recent work uses unsupervised representations (topics, embeddings) to build “narrative” signals; reported performance varies with corpus choice, coverage bias, and evaluation protocol. This chapter focuses on representations and leakage-safe pipelines for turning text into features that can be evaluated like any alpha factor.

Lexicon-based sentiment

Sentiment measures have a long history in markets, but large-scale systematic extraction became practical only with digitized corpora and modern NLP (Loughran and McDonald, 2020). The intuition is straightforward: count positive and negative words and compute a polarity score. But whose word lists?

General-purpose dictionaries designed for psychology or product reviews fail in finance. Loughran and McDonald (2011) found that nearly three-quarters of “negative” words in the Harvard General Inquirer are *not* negative in financial contexts. “Liability”—unambiguously negative in everyday language—is a neutral accounting term on a balance sheet. Similarly, “tax,” “vice,” and “capital” carry negative connotations in general dictionaries yet appear neutrally in corporate filings.

The Loughran-McDonald Master Dictionary addresses this mismatch. Built for 10-Ks and related filings, it provides six curated word lists:

Category	Example Terms	Financial Interpretation
Negative	loss, impairment, termination	Adverse events
Positive	gain, favorable, improved	Favorable developments
Uncertainty	may, possible, approximate	Hedging, risk disclosure
Litigious	litigation, court, lawsuit	Legal exposure
Strong modal	will, must, always	Commitment
Weak modal	could, might, may	Hedging intent

Table 10.1: Loughran-McDonald word list examples

The Strong/weak modal distinction is particularly valuable: executives who consistently use “will” versus “might” reveal different levels of confidence in forward-looking statements.

Statistical representations using bag-of-words

Beyond sentiment dictionaries, machine learning requires general-purpose document representations. The **bag-of-words** (**BoW**) model represents each document as a vector of term counts, ignoring word order entirely.

The process involves several steps: **tokenization** splits text into terms; **normalization** converts text to lowercase and removes punctuation; stop-word removal filters out uninformative terms (“the,” “is,” “and”); and a **vocabulary** of unique terms maps each document to a count vector.

Raw counts have an obvious problem: common words dominate regardless of informativeness. **Term Frequency–Inverse Document Frequency (TF-IDF)** weights each term’s frequency by its rarity across documents:

$$TF - IDF(t, d) = TF(t, d) \times \log \frac{N}{DF(t)}$$

where $TF(t, d)$ is term frequency in document d , N is total documents, and $DF(t)$ is documents containing term t . Implementations typically use smoothed IDF variants and L2-normalize vectors. BM25 (Robertson and Zaragoza, 2009), a probabilistic extension used in search engines, incorporates document-length normalization and term saturation.

Critical limitations

These lexical methods establish useful baselines—simple, interpretable, and computationally cheap. But their limitations are severe:

- **Semantic blindness** : Bag-of-words cannot distinguish “assets exceed liabilities” from “liabilities exceed assets.” Word order carries meaning that counting destroys.
- **No synonym awareness** : “Buyout,” “acquisition,” and “takeover” are synonyms, but TF-IDF treats them as distinct features.
- **Polysemy ignored** : “Interest” means something different in “interest rate” versus “interest in the company.” Context determines meaning; lexical models lack it.
- **Negation handling** : “Not profitable” contains a positive word but conveys negative sentiment. Simple counting fails.
- **Sparsity** : With 50,000+ term vocabularies, most documents contain tiny fractions of all words. The resulting high-dimensional, sparse

vectors pose challenges for many algorithms—the curse of dimensionality.

- **Boilerplate dilution** (finance-specific): 10-K risk factor sections contain boilerplate that dominates term frequencies, drowning out incremental disclosure.
- **Amendment leakage** (finance-specific): Document revisions (amended filings, updated articles) contaminate historical features without careful timestamp handling.

These limitations do not make lexical baselines obsolete. In small samples, narrow domains, and highly formulaic document types, well-regularized lexical models often remain difficult to beat. They are also valuable diagnostics: if a complex encoder fails to outperform a strong TF-IDF baseline, the problem may lie in label quality, temporal alignment, or task definition rather than model capacity.

But lexical models memorize fixed vocabulary and scoring rules. They cannot *learn* that semantically related words should produce similar features, *generalize* to new terminology, or resolve context-dependent meanings. This problem has motivated a shift from counting words to learning **representations** —dense, low-dimensional vectors that capture semantic relationships through distributional patterns.

Lexical models with the right dictionary (Loughran-McDonald) are interpretable, fast, and domain-specific, but they remain semantically blind: no synonym awareness, no context, no negation handling. A lexical model cannot learn that “shortfall” and “miss” mean the same thing.

The next section takes the first step beyond counting: learning dense vectors that encode semantic similarity from distributional patterns in text.



Implementation : `03_sentiment_evolution.py` compares TF-IDF baselines against embedding approaches.

10.2 Static embeddings

Bag-of-words reached its limits: treating words as atomic symbols with no relationships. The **distributional hypothesis** offered a path forward: words that appear in similar contexts tend to have similar meanings. “You shall know a word by the company it keeps” (Firth, 1957). This principle enabled models that *learn* semantic representations directly from text.

Instead of sparse, high-dimensional count vectors, embedding models produce dense, low-dimensional representations—typically 100-300 real values per word. These vectors occupy a semantic space where geometry captures meaning: “earnings” near “profit,” “equity” near “stock,” “volatility” near “risk.”

Embeddings encode analogical relationships through vector arithmetic:

$$v_{king} - v_{man} + v_{woman} \approx v_{queen}$$

This reflects consistent directional patterns in the appearance of gendered and royal terms in context. In finance, embeddings place “equity” near “stock” and “volatility” near “risk,” reflecting co-occurrence structure, though analogies are less stable than the standard “king/queen” example.

Learning from local context with Word2Vec

Word2Vec (Mikolov et al., 2013) trains a shallow neural network on a simple task: predicting a word from its context or predicting context from a word. Two architectures accomplish this:

- **Skip-Gram** : Given a target word, predict surrounding context words within a window (typically 5-10 words). Better for rare words and smaller datasets.
- **CBOW (Continuous Bag-of-Words)** : Given the surrounding words, predict the central target word. Faster to train, works well for frequent terms.

Words that can substitute for each other in similar contexts develop similar vectors. Training on billions of words from news, Wikipedia, or corporate filings produces vectors capturing subtle semantic relationships.

For financial applications, domain-specific training matters. Word2Vec trained on 10-K filings captures relationships invisible to general-purpose embeddings—for example, “risk factors” associate differently when trained on SEC filings versus Wikipedia.

Global co-occurrence statistics with GloVe

Global Vectors for Word Representation (GloVe) (Pennington et al., 2014) takes a different approach: rather than predicting context word by word, GloVe factorizes a global co-occurrence matrix that counts how often each word pair appears together across the corpus.

The model learns word and context vectors whose inner products, together with bias terms, fit the logarithm of global co-occurrence counts. The resulting geometry captures ratios of co-occurrence probabilities, which helps distinguish terms that appear in systematically different contexts. These ratios are embedded in the vector space.

Both methods produce high-quality embeddings with similar downstream performance. The choice often depends on implementation convenience and the availability of pre-trained models.

Asset embeddings from portfolio data

Embedding techniques generalize to any domain with meaningful co-occurrence patterns—including financial data. Word2Vec’s core assumption—items in similar contexts share similar properties—applies wherever meaningful co-occurrence exists. Portfolio holdings offer a compelling application: stocks that institutions consistently hold together are likely to share characteristics relevant to asset pricing.

We train Word2Vec on data from SEC Form 13F filings, which disclose the long U.S. equity positions of large institutional investment managers each quarter. We treat each portfolio as a “sentence” and each stock as a “word.” When funds repeatedly hold NVIDIA alongside other semiconductor stocks, the model learns that these stocks occupy similar positions in the institutional investment space. Portfolios encode implicit views about stock relationships that no fundamental database captures directly.

Gabaix et al. (2025) show practical value on 500 institutional portfolios (6,343 stocks): on the **masked asset prediction** benchmark—predicting which stocks belong in a portfolio given its other holdings—Word2Vec embeddings substantially outperform random baselines. The resulting embeddings complement traditional factor exposures. Two stocks may have similar market betas yet occupy different positions in institutional portfolio space—information useful for portfolio construction, risk decomposition, or similarity-based analysis.

Implementation : `02_asset_embeddings.py` replicates this approach on the largest 500 institutions in the 2024 Q3 13F bulk filing (1.32 million holdings, 9,167 stocks after a min-count filter of 5):



- Trained Word2Vec embeddings place Apple nearest to Microsoft (0.914), Amazon (0.913), and NVIDIA (0.906) — the institutional-ownership analog of “you shall know a stock by the company it keeps.”
- On the masked-asset benchmark, embeddings reach 30.6 % Hits@5 on top-decile portfolio positions versus a 0.055 % analytical random baseline (a 229× lift, 95 % CI 217–241×, bootstrap over 1,000 iterations); the lift attenuates to ~6 % on positions 11–50 and below 1 % on positions 51–200, where individual fund preference dominates the co-occurrence signal.

The polysemy problem

Static embeddings advance beyond bag-of-words—they capture similarity, analogy, and semantic relationships. But they share a critical limitation: **each word gets exactly one vector, regardless of context** .

Consider the word “bear” in the following sentences:

- “Bear market conditions persist.”
- “The bear wandered into the campsite.”

In each case, the word “bear” has a different meaning, but Word2Vec assigns identical vectors to both. For financial NLP, this polysemy problem—words with multiple context-dependent meanings—pervades investor communication:

- **“Prime”** : prime broker versus subprime mortgage (opposite connotations)
- **“Interest”** : interest rate versus interest in acquiring (different concepts)

- **“Guidance”** : earnings guidance versus regulatory guidance (different domains)
- **“Margin”** : operating margin versus margin loan (different financial concepts)

Static embeddings conflate these usages into a single representation, typically dominated by the most frequent sense. They cannot dynamically adjust to surrounding words.

This is not merely academic. A sentiment model that cannot distinguish “growth slowed” (negative) from “tumor growth” (irrelevant) produces unreliable signals. Context determines meaning; static embeddings ignore it at inference time.

Resolving polysemy requires models that read the surrounding words before committing to a representation, which motivates contextual embeddings and transformers.

Static embeddings deliver semantic similarity, dense representations, and vector arithmetic; synonyms cluster together and generalize to new vocabulary. They remain weak on polysemy, however, because a single vector per word ignores context: “margin” in “operating margin expanded” and “margin call triggered” is represented identically. The next section traces the brief era of sequential models and the limitations that led directly to the transformer architecture.



Implementation : For Word2Vec training, see `01_word2vec_training.py` , and `03_sentiment_evolution.py` for GloVe, TF-IDF, and transformers.

10.3 Sequential models

Static embeddings ignore the surrounding context. **Recurrent Neural Networks (RNNs)** addressed this by processing text word by word, maintaining a hidden state that acts as a compressed memory of the sequence so far. In principle, an RNN's representation of “growth” differs depending on whether it follows “revenue” or “tumor”—a step towards resolving the polysemy problem.

In practice, RNNs suffer from vanishing gradients: error signals can shrink rapidly over long sequences, making it difficult for vanilla RNNs to learn dependencies that span many tokens. **Long Short-Term Memory (LSTM)** networks (Hochreiter and Schmidhuber, 1997) mitigate this by using gated cell states that selectively preserve and discard information, enabling the modeling of longer-range dependencies. Three learned gates—forget, input, and output—regulate what the cell state retains, incorporates, and exposes at each step. LSTMs were the dominant sequence models in NLP through approximately 2018 and were used for sentiment analysis, named entity recognition, and time-series state extraction.

Three limitations drove the transition to transformers:

- The **sequential bottleneck** : processing the 100th word requires completing words 1–99, preventing parallelization on GPU hardware designed for massively parallel computation.
- **Imperfect long-range memory** : despite gating, information still decays over hundreds of words, making full 10-K processing (50,000+ words) impractical.
- **Unidirectional bias** : standard LSTMs read left-to-right only; bidirectional variants double computation without enabling parallelism.

The transformer architecture removes the recurrent bottleneck and gives tokens direct access to other eligible tokens through self-attention. This

substantially improves contextual modeling, although sequence length remains constrained by the quadratic cost of standard attention.

10.4 Transformers

In 2017, Vaswani et al. introduced the **transformer**, a sequence model built around self-attention rather than recurrence or convolution. This design enabled processing all positions in sequence in parallel during training, while allowing each token representation to depend directly on other relevant tokens in the same sequence. The result was a simpler and more scalable architecture that quickly became the foundation for modern language models.

For feature engineering, the key idea is not that the model reads text “word by word,” but that it learns a contextual representation for each token. In the phrase “revenue growth slowed,” the representation of “growth” can incorporate information from “revenue” and “slowed” in the same layer. In “operating margin expanded” versus “margin call triggered,” the token “margin” receives different contextual representations because the surrounding tokens change the representation the model computes.

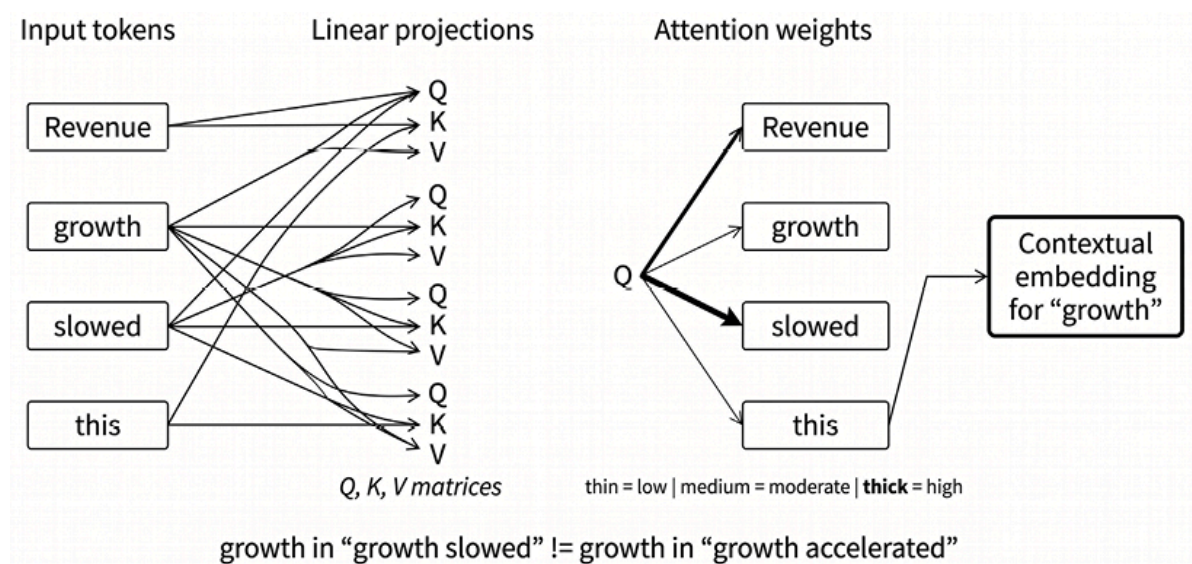


Figure 10.2: The self-attention mechanism

Attention, therefore, helps with lexical ambiguity, but it does not by itself solve meaning or provide an explanation of the model's decision.

Attention weights are internal routing weights; they can be informative, but they should not be treated as faithful explanations without additional validation.

Query, key, value, and the attention mechanism

Self-attention begins by projecting each token representation into three learned vectors:

- **Query (Q):** the representation used to search for relevant context
- **Key (K):** the representation used by other tokens to decide whether this token is relevant
- **Value (V):** the information this token contributes when it is attended to

For each token, the model compares its query with the keys of all eligible tokens in the sequence. The resulting similarity scores are normalized to attention weights, which determine the weighted average of the value vectors. The output is a new contextual representation for the token.

For example, when processing “growth” in “Revenue growth slowed,” the query for “growth” may assign higher weights to “revenue” and “slowed” than to less informative tokens. The resulting representation combines what the token is, what it modifies, and what happened to it. In deeper layers, repeated attention and feed-forward transformations allow the model to build more abstract syntactic, semantic, and discourse-level representations.

Multi-head attention and positional information

A single attention operation gives each token one learned way to aggregate context. **Multi-head attention** runs several attention operations in parallel, each with its own learned projections. Different heads can learn different relationship patterns, such as local syntax, longer-range dependencies, or repeated references, although head specialization is an empirical tendency rather than a design guarantee. The head outputs are concatenated and projected back into the model dimension.

Self-attention alone is permutation-invariant: without positional information, the model would not know whether “assets exceed liabilities” or “liabilities exceed assets” came first. Transformers, therefore, add positional information to token representations or to the attention computation itself. Original transformers used sinusoidal positional encodings, BERT used learned absolute position embeddings, and many modern architectures use relative position methods or rotary position embeddings to improve length generalization.

Box 10.1 Scaled dot-product attention

The query-key-value description above can be written compactly as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V$$

Here, Q , K , and V are the query, key, and value matrices for a sequence of length T . Each row corresponds to one token. For each query token, the model computes dot products against all eligible key tokens, divides by $\sqrt{d_k}$ to stabilize the scale of the scores, and applies a row-wise



softmax to obtain attention weights that sum to one. These weights are then used to average the value vectors.

In encoder self-attention, K , and V are derived from the same input sequence, so each token can attend to the rest of the sequence, subject to padding masks. In decoder-only language models, a causal mask prevents each token from attending to future tokens. In cross-attention, the queries come from one sequence while the keys and values come from another, as in an encoder-decoder model.

The attention matrix has size $T \times T$, so standard self-attention scales quadratically with sequence length. This is acceptable for short sequences but becomes the main bottleneck for long financial documents, such as earnings call transcripts, 10-K filings, and research reports. Modern implementations reduce this cost through better kernels, sparse or local attention patterns, longer-context positional schemes, and document chunking strategies. These engineering choices do not change the basic attention equation, but they determine which documents can be processed efficiently in practice.

Transformer architectures

Transformer models appear in three common configurations:

- **Encoder-only models** such as BERT, FinBERT, DeBERTa, and ModernBERT use bidirectional self-attention. They are well-suited for embeddings, classification, retrieval, entity extraction, and regression features built from text.

- **Decoder-only models** such as GPT-style and LLaMA-style models use causal self-attention, so each position attends only to earlier positions. They are the dominant architecture for open-ended generation and increasingly also support extraction, classification, and reasoning workflows through prompting or fine-tuning.
- **Encoder-decoder models** such as T5 and BART combine a bidirectional encoder with an autoregressive decoder. They remain natural choices for translation, conditional generation, and some summarization tasks.

For financial feature engineering in this chapter, encoder-only models remain the most direct choice because they produce stable document-, sentence-, or token-level representations that can feed downstream models. *Chapters 22-24* return to decoder-only LLMs for retrieval-augmented generation, knowledge-graph workflows, and agentic research systems.

Bidirectional transformers

Bidirectional Encoder Representations from Transformers (BERT) stacks transformer encoder layers with a powerful pre-training objective: **Masked Language Modeling (MLM)**. Random words are masked; the model predicts them from *both* directions (Devlin et al., 2019).

Bidirectional training produces representations that capture richer meaning than unidirectional approaches. Pre-training on massive corpora (Wikipedia, BooksCorpus) gives BERT a general understanding of language that transfers to specific tasks through fine-tuning.

Domain-Specific Variants for Finance

General-purpose encoders learn broad linguistic patterns, but financial text has domain-specific vocabulary, genres, and label conventions.

Words such as “liability,” “guidance,” “impairment,” “beat,” “miss,” and “margin” can carry meanings that differ from everyday usage. **Domain adaptation** addresses this gap by continuing pre-training on financial corpora before task-specific fine-tuning.

Model families, pre-training corpora, and task heads are distinct.

FinBERT refers to several checkpoints, not a single model:

- `ProsusAI/finbert` , based on Araci (2019), adapts BERT to Reuters TRC2 financial text and fine-tunes it for Financial PhraseBank sentiment.
- `yyianghkust/finbert-tone` , associated with Huang, Wang, and Yang (2023), targets analyst-report tone using different training data and labels.

These models should not be treated as interchangeable: the correct checkpoint depends on the document genre and label definition.

More recent general-purpose encoders can also be strong starting points when finance-specific labels are available:

- **DeBERTa-v3** improves on the BERT family with stronger pre-training and disentangled attention mechanisms, making it a competitive supervised baseline.
- **ModernBERT** (Warner et al., 2024) updates the encoder architecture for contemporary training and inference, including an 8,192-token native context window. This makes it better suited than classic BERT for longer financial passages, although full 10-K filings and many complete earnings-call transcripts still require chunking or hierarchical aggregation.

The practical rule is simple: use domain-specific checkpoints when their corpus and labels match the target task; otherwise, start from a strong modern encoder and fine-tune on task-specific financial labels. For long

documents, context length and aggregation strategy are part of the modeling design, not implementation details.

Beyond BERT in the LLM Era

Encoder-only transformers remain the default for high-throughput text feature extraction, but **decoder-only LLMs** have changed how financial NLP systems are designed. Their advantage is not only generation. They can perform tasks as diverse as:

- Extraction-by-prompt
- Document question answering
- Table-to-text reasoning
- Multi-document synthesis
- Workflow orchestration

These capabilities matter when the task is ambiguous, rare, or schema-rich enough that building a separate supervised model is uneconomical.

The trade-off is reliability. LLM outputs can be sensitive to prompts, retrieval context, decoding parameters, and model version. They are also more expensive to run than compact encoders and harder to audit at scale. For systematic strategies, this means LLMs are often best used as controlled components in a larger pipeline: generating labels for supervised distillation, extracting rare events under schema constraints, summarizing evidence for human review, or supporting research workflows. High-volume daily features should still prefer encoders or distilled models unless the LLM adds measurable incremental value after realistic latency, cost, and validation constraints.

BloombergGPT (Wu et al., 2023) was an early finance-specific LLM milestone: a 50-billion-parameter decoder-only model trained on a large mixed corpus that combined Bloomberg financial data with general-purpose text. BloombergGPT no longer defines the default for financial

NLP; its importance is that it showed the value of combining domain-specific financial corpora with general language modeling at LLM scale.

Benchmarks should match the use case:

- **FinBen** evaluates LLMs across a broad range of financial tasks, including information extraction, textual analysis, question answering, generation, risk management, forecasting, and decision-making.
- **FinMTEB** is more relevant when the problem is embedding quality, retrieval, or semantic similarity in financial corpora.
- General retrieval benchmarks such as MTEB and RTEB are useful reference points, but they should not substitute for domain-specific evaluation on the documents, labels, and retrieval tasks that define the investment use case.

To measure the progression concretely, we classify sentences from **Financial PhraseBank** into three sentiment categories—positive, neutral, and negative. The full Financial PhraseBank contains 4,846 English-language sentences drawn from financial news on LexisNexis, each labeled by 5–8 finance professionals. We evaluate on the `sentences_allagree` subset (2,264 sentences; 3-class label distribution: 1,391 neutral, 570 positive, 303 negative; majority-class baseline: 61.4%), which retains only sentences in which all annotators agreed. This is a common high-precision benchmark for financial sentiment models.

Method	Accuracy	F1 (macro)	Key Characteristic
TF-IDF + Logistic Regression	83.2%	0.742	Lexical features only
GloVe + Logistic Regression	80.6%	0.709	Static document averages

Method	Accuracy	F1 (macro)	Key Characteristic
FinBERT-tone (zero-shot, no PhraseBank training)	93.2%	0.917	Pre-trained classification head
FinBERT (fine-tuned on PhraseBank)	97.1%	0.955	Domain-specific fine-tuning
DeBERTa-v3 (fine-tuned on PhraseBank)	94.4%	0.932	Disentangled attention
ModernBERT (fine-tuned on PhraseBank)	96.5%	0.962	General-purpose, long context

Table 10.2: Financial Phrasebank sentiment classification results

Three patterns emerge:

- On the high-precision `all_agree` subset, GloVe averages give up about 2.6% points to TF-IDF (80.6 % versus 83.2 %)—averaging GloVe vectors discards order and negation, which the bigram TF-IDF representation partially recovers, so the gain from semantic similarity is more than offset by the loss of word order.
- Zero-shot FinBERT-tone leads both lexical baselines by roughly 10% (93.2% compared to 83.2%)—the analyst-report $\square$ financial-news distribution shift is mild enough on `all_agree` that the pre-trained classification head transfers cleanly.
- Task-specific fine-tuning further lifts accuracy to 94–97%: FinBERT 97.1%, ModernBERT 96.5%, DeBERTa-v3 94.4%, all on roughly 1,600 fine-tuning examples.

The order is informative—domain-adapted FinBERT and long-context ModernBERT are within a percentage point, suggesting that on a clean,

short-sentence sentiment benchmark, the gain from domain pre-training is small once a strong base encoder has been fine-tuned.

Transformers deliver context-dependent representations that address polysemy, pre-trained checkpoints that transfer to specific tasks, and strong accuracy with modest fine-tuning data. The costs are higher compute per document, attention patterns that are harder to audit than dictionaries, and sequence limits of 512 to 8,192 tokens that remain inadequate for full 10-Ks.

Implementation :

- See `04_bert_finetuning.py` for fine-tuned results on a 70/15/15 train/val/test split. “FinBERT-tone (zero-shot)” applies `yyianghkust/finbert-tone`, already fine-tuned on analyst-report sentiment, to PhraseBank without further training; the 93.2 % accuracy shows that the distribution shift from analyst-report to news is small enough on this dataset that the zero-shot head transfers cleanly.
- See `06_finbert_cross_dataset.py` for cross-dataset evaluation on FinMarBa market-labeled headlines: FinBERT, fine-tuned on Financial PhraseBank in its release form (ProsusAI’s published in-domain accuracy 87.0%), scores only 49.4% on FinMarBa—a 37.6% drop confirming that the same labels across different text distributions produce near-random performance.

With representation methods established, the next section turns to the harder problem: converting these models into reliable, backtestable trading signals without introducing look-ahead bias.

10.5 The modern feature extraction workflow

The preceding sections traced a progression in how models represent text—from term counts to contextual embeddings. Representation quality, however, is only half the problem. A FinBERT embedding of an earnings call is not yet a tradable signal. It becomes one only after you define when the embedding was available, how it maps to a tradable entity, and what temporal constraints prevent leakage. This section covers the conversion from a pre-trained model to a backtestable alpha factor.

Text-to-signal pipeline contract

Before selecting a model or writing any extraction code, define the data contract that determines whether a text feature is tradable. In *Chapter 6* terms, this means specifying the observation timestamp, the entity mapping, and the admissible information set for each text-derived feature.

The following items form a text-feature checklist:

- **Timestamp definition** : Publication time, scrape time, and vendor timestamp are three different quantities—only one defines when you could have acted. Choose it explicitly and use it to index backtests.
- **Publication lag** : SEC filings depend on the form type and the filer’s status (large accelerated filers have shorter deadlines). News arrives in seconds to hours; transcripts are same-day to next-day. Measure the lag empirically for your data source.
- **Entity resolution** : Map every mention to a tradable identifier—ticker, CIK, FIGI. Handle ambiguous names (“Apple” the company rather than the word), subsidiaries, and M&A transitions where identifiers change mid-series (see *Chapter 4*).

- **Deduplication** : Syndicated news, wire updates, and editorial revisions count information multiple times. TF-IDF similarity within (ticker, date) groups removes near-duplicates; see `07_news_return_signals.py` .
- **Sampling unit** : Sentence, paragraph, or full document? Define per-firm-day aggregation rules before modeling, not after.
- **Universe construction** : News coverage is endogenous—heavily covered firms differ systematically from lightly covered ones. Coverage itself may predict returns.
- **Pre-training horizon** : Verify that every model’s training cutoff predates your backtest period. A model trained on 2024 web crawls has “seen” news you are using to predict 2023 returns.

A text feature is valid only if every upstream step respects an availability timestamp. That timestamp—not the document’s date label—indexes backtests.

Walk-forward testing for text signals requires four additional safeguards beyond the standard cross-validation protocol defined in *Chapter 6* :

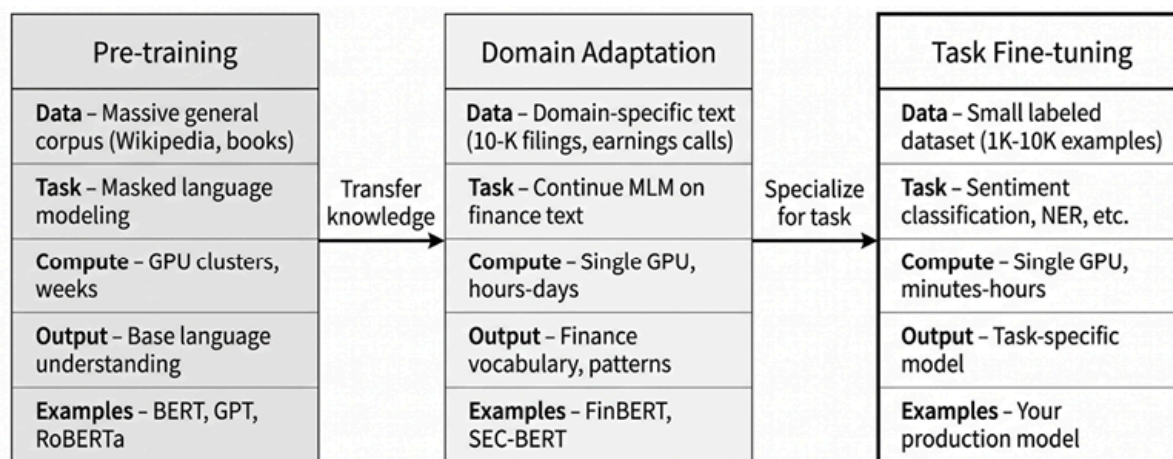
- **Purged cross-validation** : Exclude documents within k days of the prediction boundary to prevent information leaking across folds. The gap k depends on the signal’s decay horizon—a 1-day sentiment signal needs $k \geq 1$; a topic-drift signal operating at monthly frequency may need $k \geq 20$.
- **Rolling fine-tuning windows** : When fine-tuning on recent labeled data, ensure no test-period documents appear in any training window. Track both document dates and model training dates—a model retrained on January data cannot score December documents in a backtest.
- **Revision leakage** : Amended filings, corrected transcripts, and updated articles can silently alter historical features. Snapshot

documents at first availability; flag and exclude revisions. The `amendment_leakage` field in your data contract should default to “reject” unless you explicitly verify the revision timeline.

- **Event-study alignment** : A document arriving at time t predicts returns over $[t + \Delta, t + H]$. Verify that Δ accounts for actual availability, not publication date. For earnings calls, Δ may be minutes; for 10-K filings, hours to days depending on the vendor’s processing pipeline.

The pre-train, adapt, fine-tune cascade

Training a transformer from scratch requires thousands of GPU-hours and billions of tokens. The three-stage cascade lets practitioners build powerful text features without that cost. *Figure 10.3* visualizes the workflow.



Cost and data requirements decrease left to right. Knowledge transfers downstream.

Figure 10.3: From pre-training to fine-tuning

Stage 1: General pre-training

Models like BERT train on massive corpora—Wikipedia, web crawls, books—learning syntax, semantics, and world knowledge through masked language modeling. You never run this stage yourself. You download a checkpoint.

Stage 2: Domain adaptation

Resume masked language modeling on domain-specific text: 10-K filings, earnings call transcripts, analyst reports, and financial news. The model learns vocabulary (“EBITDA,” “covenant”), phrasing patterns (“guidance raised to”), and domain-specific co-occurrence relationships (“margin expansion” near “operating leverage”). This stage requires hours of GPU time, not days, and produces checkpoints like FinBERT.

Stage 3: Task fine-tuning

Add a task-specific output head—classification layer, regression head, token-level tagger—and train on labeled examples. For sentiment: sentences labeled positive, neutral, or negative. Fine-tuning requires thousands of examples, not millions, because the model already understands both language (*Stage 1*) and financial vocabulary (*Stage 2*).

This cascade explains why FinBERT outperforms general BERT on financial sentiment: broad language understanding, plus domain knowledge, plus task specialization.

The complementary failure mode is **distribution shift** . A model fine-tuned on one text source may fail when applied to another source with a different language, label conventions, or market context.

Implementation : `06_finbert_cross_dataset.py`

illustrates this failure mode. FinBERT—with ProsusAI’s



published in-domain accuracy of 87.0% on PhraseBank—scores only 49.4% on the FinMarBa market-labeled headline benchmark, a 37.6% drop with the same model, the same label names, and a different text distribution. Domain adaptation does not rescue a model when the fine-tuning and inference corpora disagree on what counts as positive or negative.

Fine-tuning in practice

The Hugging Face Trainer API handles the mechanical boilerplate. The workflow: load a pre-trained model and its tokenizer, prepare a dataset mapping text to integer labels, configure training arguments (learning rate around 2×10^{-5} , 3–5 epochs, batch size 16–32), call `trainer.train()`, and evaluate on held-out data. For BERT-style classifiers, the final hidden state of the ([CLS]) token is commonly passed to the classification head. During fine-tuning, this representation is adapted to summarize the information needed for the supervised task. For retrieval and similarity tasks, mean pooling over token embeddings often performs better than raw ([CLS]) pooling unless the model has been trained specifically for sentence embeddings.

For limited GPU memory, LoRA (Low-Rank Adaptation; Hu et al., 2022) updates only a small fraction of model weights while freezing the rest. This reduces memory by 60–80% with modest quality loss—often acceptable for production pipelines where hundreds of task-specific models must coexist.

The following table offers some example scenarios of when to fine-tune:

Scenario	Recommendation	Rationale
A few hundred labeled examples	Start with frozen embeddings, prompt-based labels, or parameter-efficient tuning	Full fine-tuning may overfit without careful validation
1,000–10,000 labeled examples	Fine-tune with early stopping	Sweet spot for adaptation
Domain-specific vocabulary	Prefer domain adaptation or supervised fine-tuning	Depends on whether the gap is vocabulary, document genre, or label definition
Real-time latency-critical	Fine-tune a smaller model	FinBERT-base over LLMs
Nuanced reasoning required	Consider LLM + prompting	Fine-tuning may not capture complex logic

Table 10.3: Fine-tuning scenarios

The “few hundred examples” limitation has a practical workaround: use an LLM (GPT-4, Claude) to synthetically label thousands of examples, then fine-tune a smaller encoder on the resulting labels. This converts the first row into the second—see *Chapter 5* for synthetic data generation pipelines and quality validation.

Practitioner warning: Look-ahead bias in pre-trained models



A model trained on 2024 web crawls may have “seen” the news you are using to predict 2023 returns. Topic models like batch LDA pose the same risk: they estimate topics over the entire corpus, incorporating future documents. Online LDA (Bybee et al., 2023) processes documents sequentially using only past data. Verify every pipeline component—not just the final model—uses only information available at prediction time.

Tokenization pitfalls

Financial text contains terms unseen during pre-training: tickers (NVDA, TSLA), acronyms (EBITDA, YoY), and composite expressions (\$94.8B). Subword tokenizers (WordPiece, BPE) break unknown words into known pieces, increasing sequence length and potentially reducing numeric precision. “Non-GAAP” becomes three tokens; “\$94.8B” may split into five or more, fragmenting the number across subwords.

There are two consequences for pipeline design. First, documents that approach the 512-token limit after tokenization may lose content at truncation—switch to ModernBERT’s 8,192-token context or explicitly chunk the document. Second, numeric expressions that span multiple tokens are poorly represented; consider extracting numbers separately before embedding the text.

From tokens to document embeddings

Transformers produce one embedding per token. Many applications—such as factor construction, similarity search, and clustering—require a single vector per document. Four pooling strategies:

- **[CLS] pooling** uses the classification token’s embedding. Standard for classification tasks where the [CLS] token is trained to aggregate

sequence information.

- **Mean pooling** averages all token embeddings (weighted by the attention mask to exclude padding). Typically stronger for retrieval and similarity because it incorporates information from every token.
- **Max pooling** takes the element-wise maximum across tokens. Captures the most activated feature per dimension.
- **Attention pooling** learns a weighted combination of token embeddings. More expressive but requires task-specific training.

For classification, [CLS] is the default. For similarity and retrieval, Sentence-BERT (Reimers and Gurevych, 2019) with mean pooling consistently outperforms [CLS].

Embedding selection and long documents

Three trade-offs dominate the selection of embedding models in practice.

One trade-off involves **retrieval architecture**. Bi-encoders (Sentence-BERT, E5, BGE, GTE) embed documents independently—encode once, compare via cosine similarity. This scales linearly: encode n documents, then compare any pair in constant time. Cross-encoders (reranking models) jointly process document pairs, yielding higher accuracy for pairwise judgments but at $O(n^2)$ cost. Production systems use bi-encoders for candidate retrieval and cross-encoders for reranking small candidate sets (typically the top 50–100).

Another key issue is *context length*. Standard encoders handle 512 tokens—sufficient for headlines, abstracts, and short paragraphs. Earnings call transcripts (8,000–12,000 words) and 10-K sections require either chunking or long-context models: ModernBERT, GTE-large, and BGE-M3 support 8,192 tokens. Longer context increases compute cost per

document; justify the cost with a measurable improvement in quality on your evaluation set.

For high-volume pipelines processing millions of documents daily, *distilled models* (MiniLM, all-MiniLM-L6-v2) offer 5× faster inference with modest quality loss. Reserve larger encoders (E5-large, BGE-large) for high-value extraction tasks where accuracy is more important than latency. The MTEB benchmark provides standardized quality comparisons across model families, while retrieval- and finance-specific benchmarks, such as RTEB and FinMTEB, offer more targeted evaluation for financial retrieval use cases.

Modern embedding families such as E5, BGE, and GTE often use instruction-tuning, meaning they are trained to produce embeddings that depend on the task prompt, such as “represent this query for retrieval” or “represent this passage for search.” This helps the model place queries and documents in a vector space that better reflects their intended use, especially for retrieval. Many of these families also support multiple languages. For English financial text, appropriately sized models from these families will generally outperform older Sentence-BERT checkpoints. Pin model versions: when an embedding model changes, the meaning of its vectors can shift, potentially silently invalidating cached embeddings and downstream models trained on them.

Handling long documents

Financial documents routinely exceed model limits. A 10-K filing spans 50,000+ words; even an earnings call runs 8,000–12,000. Three strategies handle this:

- **Chunking and pooling** : Split documents into overlapping chunks (typically 256–512 tokens with 64-token overlap), encode each chunk independently, and aggregate via mean or attention-weighted

pooling. Simple, robust, and easy to parallelize. The overlap prevents information loss at chunk boundaries (see *Chapter 22* for more detail).

- **Hierarchical encoding:** Encode sentences or paragraphs individually, then feed the resulting sequence of embeddings to a lightweight document-level model (an LSTM, a small transformer, or learned attention pooling). This preserves intra-document structure that flat pooling discards.
- **Long-context models:** ModernBERT handles 8,192 tokens, a major improvement over classic 512-token encoders, enabling it to cover longer passages or shorter transcripts. Full 10-K filings and many complete earnings calls still require chunking, section selection, or hierarchical aggregation. For longer documents, retrieval-augmented approaches identify relevant passages before encoding; *Chapter 22* covers RAG pipelines in detail.

Verify that chunking does not introduce look-ahead bias by mixing sentences from different time periods within a single chunk. When processing a sequence of quarterly filings, chunk within each filing, not across them.

Text signal families and factor construction

Production text pipelines generate multiple signal families, each capturing different information from the same documents:

- **Sentiment** measures positive, negative, or neutral tone. Valuable but increasingly commoditized—most edge comes from speed and coverage breadth rather than model sophistication (baseline signal in `07_news_return_signals.py`).

- **Novelty and surprise** measure semantic distance from a recent baseline. A firm whose news embedding suddenly diverges from its 20-day average is experiencing a narrative shift—a topic change invisible to sentiment. This narrative-surprise factor is developed in the next subsection.
- **Attention** captures coverage intensity, source diversity, and headline prominence. Attention predicts volatility even when tone is neutral: a stock mentioned in 50 articles generates different risk dynamics than one mentioned in 2, regardless of whether coverage is positive.
- **Events** are structured extractions: guidance changes, buyback announcements, litigation disclosures, and executive departures. Schema-based extraction enables reliable aggregation (see *Extract-to-Schema Feature Engineering* below).
- **Entity relations** map who is mentioned with whom: supply chain links, competitive mentions, and regulatory exposure. These relational features feed *Chapter 23* 's knowledge graph pipelines.
- **Topic exposure** categorizes documents by subject (earnings, M&A, regulatory, litigation, macro). Topic attention series—how much coverage each topic receives over time—aggregate into tradable factors. Bybee et al. (2023) construct narrative factors from 180 news topics that explain approximately 25% of aggregate return variance.

Embedding-based factor construction

Embeddings are now the default substrate for systematic text features. An embedding becomes a tradable factor after you define two things:

- **Aggregation unit** : Compute one embedding per document. For a given firm on a given day, aggregate all document embeddings—mean pooling weighted by recency or source reliability. This produces a firm-day embedding vector.

- **Baseline** : The rolling-average embedding over the prior 20 news days serves as the firm's expected narrative. Any distance measure between the current firm-day embedding and this baseline produces a signal.

Three factor patterns cover most production use cases:

- **Semantic novelty (the news surprise metric)** : The cosine distance between a firm's daily embedding and its 20-day rolling baseline captures topic shifts that are invisible to sentiment. The signal:

$$surprise_t = 1 - cosine_similarity(e_t, mean(e_{t-20:t-1}))$$

When a company's news coverage shifts abruptly—sudden regulatory mentions after months of growth discussion—the surprise signal spikes regardless of whether the new coverage is positive or negative. Bhargava et al. (2023) find that unexpected narrative shifts predict returns at short horizons.

- **Peer-relative positioning** : Measure the distance between a firm's embedding and the centroid of its peer group (industry, supply chain, or similarity-based neighbors). A firm whose narrative differs from peers may be experiencing idiosyncratic developments that the market has not yet priced in. Cross-sectional dispersion in this distance serves as a second factor.
- **Narrative drift** : Treat the embedding series as a time series and compute standard features: period-over-period changes, rolling volatility of the embedding trajectory, and decay rate of similarity to older embeddings. Drift in narrative framing often matters more than levels—markets adapt to stable narratives but respond when the framing changes.

Take these two safeguards for research integrity. First, do not fit global transforms (PCA, clustering) on the full corpus without walk-forward

implementation—this leaks future distributional information into past features. Second, version the embedding model and preprocessing pipeline. Embeddings change meaning when models change. Pin versions and cache embeddings indexed by document hash and model identifier.



Practitioner warning: Coverage bias

The 20-day lookback counts news-days, not trading days. Tickers with fewer than 21 days of coverage in the lookback window are dropped, biasing the resulting factor toward high-attention stocks. Handle sparse coverage explicitly: impute with sector-level baselines, flag as missing, or model coverage itself as a separate signal.



Implementation :

- `07_news_return_signals.py` builds the deduplication pipeline, FinBERT scoring, and surprise factor at daily granularity—suitable for daily or longer holding periods.
- `09_filing_text_signals.py` applies sentiment and narrative-change signals to quarterly SEC 10-Q filings, showing how the same factor patterns work at lower frequency with point-in-time availability anchored to filing dates.

Intraday news-to-return alignment requires second-level timestamp precision, latency modeling, and venue-specific microstructure considerations, all of which are beyond the scope of this chapter.

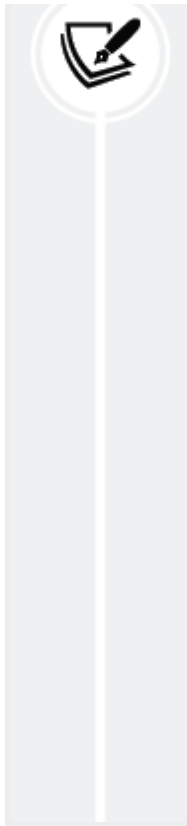
Evaluating text-derived signals

Standard diagnostics apply—IC, ICIR, t -statistics, quintile spreads—following the framework in *Chapter 7*. Text signals introduce three additional evaluation dimensions:

- **Decay analysis** : Compute IC at multiple horizons (1-day, 5-day, 20-day). Text signals typically decay faster than fundamental signals. Strong 1-day IC with negligible 5-day IC suggests the signal captures short-term attention rather than fundamental information—useful for short-horizon strategies but misleading for monthly rebalancing.
- **Coverage-conditional performance** : Evaluate IC separately for high-coverage and low-coverage stocks. If the signal works only for stocks with dense news flow, it is capturing attention dynamics rather than fundamental information—a distinction that affects portfolio construction.
- **Event-time alignment** : Verify that performance persists when you shift the availability timestamp by realistic amounts (for example, 30 minutes, 1 hour, or next-day open). A signal whose IC collapses with a 1-hour delay is pricing speed, not information.

Implementation : `08_text_feature_evaluation.py` provides IC/ICIR computation, decay curves across horizons, and coverage-conditional diagnostics. The notebook `09_filing_text_signals.py` illustrates decay analysis concretely on a 50-symbol 10-Q sample (cluster bootstrap by symbol, $B = 1,000$ iterations):

- Narrative change (embedding cosine distance between consecutive filings) shows IC = -0.151 (95% CI $[-0.226, -0.074]$, $p < 0.001$ cluster bootstrap, $n = 671$ across 47 symbols) at the 5-



day horizon, attenuating to $IC = +0.059$ (95% CI $[-0.021, +0.138]$, $p = 0.154$, $n = 670$) at 20 days — the significant short-horizon negative signal does not reverse into a significant medium-term positive on this universe.

- FinBERT sentiment on 10-Q MD&A sections shows no predictive power at the 5-day horizon ($IC = +0.055$, 95% CI $[-0.036, +0.160]$, $p = 0.264$, $n = 720$) and a directionally contrarian signal at 20 days ($IC = -0.082$, 95% CI $[-0.195, +0.034]$, $p = 0.172$, $n = 719$) that does not clear conventional significance under cluster bootstrap by symbol.

These profiles illustrate why horizon selection matters : the same document source produces a statistically significant short-horizon negative signal on narrative change and a directional but inconclusive medium-horizon signal on sentiment — suited to different research questions and trading directions, but only the narrative-change signal clears conventional significance on this 50-symbol universe.

Extract-to-schema feature engineering

Embeddings measure similarity and change. Many questions require explicit structure—not how similar today’s language is to yesterday’s, but whether the company announced a buyback and, if so, for how much.

Start with the output. A press release states: “Board authorizes \$10B share repurchase program through 2027.” A well-designed extraction pipeline produces:

```
{event: "buyback", magnitude: 10e9, currency: "USD", horizon:
```

The factor pipeline aggregates these records by firm: buyback intensity equals announced buyback divided by market capitalization over the trailing 90 days. This structured signal is testable with standard IC analysis, combinable with other factors, and fully auditable—properties that raw sentiment scores lack.

The schema that makes this possible contains four field groups:

- **Entity fields** : Issuer and counterparties mapped to stable identifiers (CIK, FIGI, ticker). Entity resolution must handle subsidiaries, M&A transitions, and ambiguous mentions.
- **Event fields** : Controlled vocabulary, such as guidance, buyback, issuance, litigation, regulatory action, executive change. Fixed vocabularies enable reliable aggregation across documents and time.
- **Attribute fields** : Direction, magnitude, units, and time horizon. These transform qualitative events into quantitative features.
- **Uncertainty fields** : Modality (hedging language), confidence score, and an abstention flag. When the model's confidence falls below the threshold, emit no record rather than a noisy one.

Implement extraction with fine-tuned encoder models for high-throughput tasks, prompt-constrained LLMs with JSON-mode decoding for complex or rare event types, or a cascade that uses cheap encoders for broad coverage and reserves LLM extraction for high-novelty or low-confidence regions. For LLM-based extraction, prefer constrained or schema-guided decoding when available. Modern inference stacks can restrict generation to JSON schemas, enumerated choices, regular expressions, or grammars, reducing the need for brittle post-processing. Schema guidance improves format reliability, but it does not guarantee

factual correctness: extracted entities, dates, magnitudes, and units still require validation against the source text and downstream sanity checks.

Two **safeguards** for systematic use:

- Enforce **schema validation** at the pipeline boundary—reject records with missing required fields, out-of-vocabulary event types, or magnitudes that fail sanity checks. Invalid extractions that slip through corrupt aggregated features silently.
- **Separate extraction from aggregation** . Extraction operates at the document level with a deterministic model version and an availability timestamp. Aggregation operates at the firm-day level using only records available at that point in time. This separation simplifies backtesting, failure auditing, and model upgrades—you can re-extract documents with a new model without rebuilding the entire feature history.



Implementation : See

`05_financial_ner_finetuning.py` for token-level entity extraction from financial text using fine-tuned encoders, including downstream feature engineering from extracted entities. *Chapters 22–24* extend these extraction patterns to agentic workflows with LLM-based structured extraction and schema validation.

Model interpretability

For risk-managed strategies, opacity is a liability. Regulators may require explainability, and production failures are harder to diagnose without inspecting the factors that drove a prediction.

SHAP (Lundberg and Lee, 2017) computes the marginal contribution of each input token to a model's output. Positive SHAP values push toward one class; negative values push toward another. The resulting attribution map shows which words mattered and in which direction.

Consider the sentence: “Net loss narrowed significantly from the prior year.” A dictionary-based model flags “loss” as negative. Fine-tuned FinBERT correctly classifies the sentence as positive with 69.7% confidence. SHAP reveals why: “narrowed” carries the highest positive attribution (+0.373), dominating the negative contribution of “loss.” The model learned that *improving* losses signals positive sentiment—a contextual judgment that no word list can replicate.

Token attributions serve three functions in a systematic workflow:

- **Risk management** : Before deploying a text signal, inspect SHAP attributions on a sample of high-confidence and low-confidence predictions. If the model relies on boilerplate phrases, ticker symbols, or formatting artifacts rather than substantive language, the signal is fragile.
- **Debugging** : When a text signal's IC degrades, SHAP attributions during the degraded period often reveal the cause—a new phrasing pattern the model misinterprets, a shift in boilerplate that the model overweights, or a data quality issue that introduces spurious tokens.
- **Alpha discovery** : Systematic analysis of high-attribution tokens across thousands of documents reveals which linguistic patterns consistently predict returns. Modal verbs (“will” versus “might”), hedging phrases, and specific financial terms may retain predictive power that is worth engineering into dedicated features.

A caveat: token-level attributions can be noisy, particularly for long documents where many tokens contribute small amounts. Treat SHAP as an audit and diagnostic tool, not as a source of causal explanations. For

full implementation details, including batched attribution computation and visualization, see *Chapter 12* .

This workflow delivers production pipelines with proper temporal alignment, fine-tuning recipes that adapt pre-trained models, embedding factors that capture semantic information beyond sentiment, structured extraction for reliable downstream aggregation, and interpretability diagnostics that make transformer-based signals auditable.

The cost is operational complexity. Text pipelines demand careful attention to deduplication, coverage bias, look-ahead leakage, and model versioning. Every shortcut in the pipeline contract—an approximate timestamp, an unversioned embedding model, a schema without validation—introduces a failure mode that may not surface until live trading.



Implementation : Notebooks `04_bert_finetuning.py` through `09_filing_text_signals.py` cover every major workflow step discussed in this section. Start with the pipeline contract checklist above; no notebook output is trustworthy until you can answer all seven questions for your specific data source.

The following summary consolidates the trade-offs across all four representation families to guide model selection in practice.

10.6 Summary

Text feature engineering has progressed primarily through improvements in how models represent *context* . Lexical methods (dictionaries, bag-of-words, TF-IDF) are simple and interpretable, but they discard word order and cannot resolve polysemy or negation. Static embeddings (Word2Vec,

GloVe) move from sparse counts to dense vectors that capture semantic similarity, yet still assign one representation per word regardless of usage. Sequential models (RNNs/LSTMs) incorporate preceding context, but their sequential computation and difficulty with long-range dependencies make them impractical for long financial documents.

Transformers address these limits with self-attention: each token can condition on all others, producing contextual embeddings that adapt to surrounding text. In practice, performance depends on matching the model to the domain and task: continue pre-training on financial corpora when needed, and fine-tune on labeled data to achieve measurable gains over lexical baselines. Because text arrives in real time, pipelines must also guard against look-ahead bias by respecting data availability and model training cutoffs.

For systematic trading, representation quality is only half the problem. The resulting embeddings must be turned into time-aligned, backtestable features (including factors based on narrative surprise or topic drift), and models must be auditable: interpretability tools such as SHAP help verify that predictions rely on sensible language cues rather than spurious artifacts. The next chapter shifts from representation to evaluation, situating these text features inside the broader ML pipeline that turns features into validated predictions; LLM-based generative workflows return in *Chapters 22–24* .

Q

Q

11

The ML Pipeline

Previous chapters constructed features across the case studies and evaluated their predictive power through information coefficients, factor spreads, and the triage framework of *Chapter 7*. Now we will take the next step: transforming those features into trading signals using machine learning.

This chapter introduces regularized linear models (Ridge, LASSO, and Elastic Net) as interpretable baselines that introduce the ML signal-generation workflow within the trading setup defined in *Chapter 6*. We will build the walk-forward validation pipeline to prevent look-ahead bias, the interpretability tools to verify economic sensibility, and a conformal prediction framework to quantify uncertainty, foundations that carry forward to every model in *Chapters 12–15* and translate into positions in *Chapters 16–19*.

By the end of this chapter, you will be able to:

- Choose between *regression and classification* formulations based on how predictions will be translated into trading decisions.
- Fit regularized linear models, including Ridge, LASSO, Elastic Net, and logistic regression, using point-in-time preprocessing and standardization.
- Tune and evaluate linear models with walk-forward validation, temporal buffers, and, when needed, nested cross-validation to reduce selection bias.

- Interpret model behavior with SHAP-based diagnostics to assess feature importance, economic plausibility, and stability across refits.
- Construct and evaluate conformal prediction intervals or prediction sets, and monitor where coverage degrades under non-stationary market conditions.
- Use cross-case-study evidence to judge when linear models provide a strong baseline and when a weak linear signal motivates more flexible models.

We will begin by motivating the use of regularization for prediction, then introduce linear regression and classification, show how to use SHAP values for explainability, and present conformal predictions to quantify uncertainty. We conclude by presenting insights from the case studies.

Chapters 12–14 extend the models introduced here. Every model must beat the regularized linear baseline from this chapter. A gradient-boosting model that underperforms Ridge has added complexity without improving predictive power. Each modeling chapter loads the linear baseline automatically and reports the improvement (or lack thereof) alongside the candidate model.

11.1 From inference to prediction

Classical **econometrics** (the application of statistics to economic data) and machine learning approach the same data with different objectives. Econometrics asks: “What is the relationship between these variables, and is it statistically reliable?” Machine learning asks: “Given inputs, what is the best forecast of the outcome?” This chapter develops linear models for this exact predictive task. The shift from inference to prediction changes which properties of an estimator matter and motivates the regularization methods that follow.

Two modeling cultures

Leo Breiman's 2001 paper "Statistical Modeling: The Two Cultures" formalized a distinction that had long been implicit:

- The *data modeling culture* , dominant in econometrics, posits a stochastic model for the data-generating process (DGP) and estimates its parameters
- The *algorithmic modeling culture* , dominant in machine learning, treats the DGP as unknown and evaluates models solely by their predictive accuracy on data not used to develop the model

The distinction is not about complexity. A linear regression can serve either culture: used for inference, it recovers coefficient estimates and tests hypotheses about them; used for prediction, it generates forecasts evaluated by out-of-sample error. The distinction is about the *success criterion* . In the data modeling culture, a model succeeds when it recovers interpretable, statistically significant parameters under a correctly specified model. In the algorithmic culture, a model succeeds when it predicts well. As Breiman argued, predictive accuracy should serve as the primary check on whether it has captured meaningful structure in the data.

For algorithmic trading, the more relevant criterion is predictive accuracy (generating stable, profitable forecasts), not unbiased parameter recovery (Simonian, 2024).

What inference requires

The inferential framework is powerful but demanding. **Ordinary Least Squares (OLS)** regression (the estimator behind models from the CAPM to Fama-French-Carhart) recovers coefficient estimates $\hat{\beta}_j$ that, under certain conditions, are unbiased estimates of the true *ceteris*

paribus effects of each feature x_j on the outcome. Those conditions are the **Gauss-Markov assumptions** that require:

- **Linearity** in the parameter vector β , implying the functional form $y = X\beta + \varepsilon$
- **Strict exogeneity** : $E[\varepsilon | X] = 0$, which implies that regressors contain no information about the error term (no omitted variables or lagged dependent variables)
- **No perfect multicollinearity**, meaning no regressor is an exact linear combination of the others, so β is identifiable
- **Spherical errors**, $\text{Var}(\varepsilon | X) = \sigma^2 I$, which assumes errors have constant variance and are uncorrelated across observations

When all four hold, the Gauss-Markov theorem guarantees that OLS is the **Best Linear Unbiased Estimator (BLUE)**: among all unbiased linear estimators, OLS has the smallest variance.

The linearity and exogeneity assumptions require that the model include the right variables in the right functional form. This is a strong requirement when the true DGP is unknown. In financial markets, the DGP almost certainly involves nonlinearities, regime changes, and time-varying parameters. When the model is misspecified, the unbiasedness guarantee fails, and the coefficient estimates become difficult to interpret as causal or structural parameters.

Given a correctly specified model, hypothesis testing evaluates whether a coefficient is distinguishable from zero. A p-value measures the probability of observing a test statistic as extreme as the one computed, assuming the null hypothesis ($\beta_j = 0$) is true. This is useful for inference (determining whether a variable has a statistically reliable relationship with the outcome), but it does not measure predictive power. A coefficient can be highly significant yet economically negligible (for example, a

large sample size with a tiny effect), or insignificant yet genuinely predictive when combined with other features.

The Gauss-Markov optimality of OLS is defined within the class of unbiased estimators. But for prediction, unbiasedness is not inherently desirable. What matters is total prediction error, which decomposes as:

$$\text{MSE} = \text{Bias}^2 + \text{Variance} + \sigma_{\varepsilon}^2$$

An estimator that introduces some bias but substantially reduces variance can achieve lower MSE than an unbiased estimator, particularly when the number of features is large relative to the number of observations. The Gauss-Markov theorem does not claim that OLS minimizes MSE; it claims that OLS minimizes variance **subject to the constraint of unbiasedness**. Relaxing that constraint opens a path to lower total error.

Why prediction demands a different approach

Consider a strategy research problem: predicting next-month returns using 100 features from momentum, valuation, and sentiment feature families across 240 monthly observations. OLS will fit this model. Under the Gauss-Markov assumptions, the resulting coefficient estimates are unbiased. But **unbiasedness alone does not ensure good predictions**. With 100 parameters estimated from 240 observations, estimation variance is large: small changes in the training sample produce large swings in $\hat{\beta}$, and therefore in predictions. The model fits training data well (it has enough free parameters to absorb idiosyncratic patterns), but those patterns do not generalize.

This is not a failure of OLS as an inferential tool. It is a consequence of using an estimator optimized for unbiased parameter recovery in a setting where predictive stability matters more. **OLS has no mechanism to**

trade bias for variance , because doing so would compromise the inferential guarantees that justify its use in econometrics.

Three aspects make the problem particularly acute in finance:

- **High dimensionality relative to observations** . As p/n grows, OLS estimates become unstable; when $p > n$, they are not uniquely defined.
- **Multicollinearity**. Financial features are pervasively correlated: momentum overlaps with trend, valuation ratios share accounting inputs. OLS assigns arbitrary weights to correlated features, producing offsetting coefficients that amplify noise.
- **Low signal-to-noise ratio** . Cross-sectional return regressions typically achieve R^2 of 1–3% in-sample and lower out-of-sample. Variance dominates MSE: an unconstrained model fits noise, while a constrained model sacrifices modest signal to suppress far more spurious patterns.

Regularization addresses this tradeoff directly. It adds a penalty to the OLS loss function that increases with the magnitude of the coefficients, shrinking estimates toward zero. This introduces bias (penalized estimates no longer center on the true β), but it reduces variance by preventing the model from assigning extreme weights to any feature or distributing weight erratically across correlated features.

The form of the penalty encodes a *prior about the structure of the signal* :

- **Ridge** (L_2) shrinks all coefficients proportionally, which is suited to diffuse signals with pervasive multicollinearity
- **LASSO** (L_1) drives coefficients to zero, performing automatic selection when the signal concentrates in a sparse subset
- **Elastic Net** combines both, retaining correlated feature groups while still shrinking toward sparsity, a common requirement when

momentum, trend, and mean-reversion indicators overlap

These are principled responses to the **bias-variance tradeoff**, not ad hoc corrections. The tuning parameter λ controls the tradeoff, and cross-validation selects the degree of shrinkage that minimizes out-of-sample error. *Section 11.2* develops each method formally.

The label families defined in *Chapter 7* (continuous returns, thresholded direction, quantile membership) map to regression (predicting magnitudes) and classification (predicting categories). The choice depends on alignment with the trading setup and the data. We first discuss regularized regression, then classification in more practical detail.



Implementation : `01_ols_inference` shows OLS summary tables, the Gauss-Markov diagnostic battery, variance inflation factors, and robust standard errors on the ETFs case study.

11.2 Regularized regression

Ridge, LASSO, and Elastic Net each encode a different assumption about how the signal is distributed across features. This section presents their formal objectives, the geometry that gives each method its distinctive behavior, and the practical machinery (standardization, hyperparameter optimization, and loss function choice) required to deploy them within the walk-forward protocol of *Chapter 6*.

Ridge regression for distributed signal

Ridge regression augments the OLS objective with an L_2 penalty on the coefficient vector:

$$\min_{\beta} \sum_{i=1}^n (y_i - X_i \beta)^2 + \lambda |\beta|_2^2$$

where $\lambda > 0$ controls regularization strength and $|\beta|_2^2 = \sum_j \beta_j^2$. The penalty increases with the sum of squared coefficients, discouraging large values but never forcing any coefficient exactly to zero.

The closed-form solution reveals the mechanism. OLS solves $\hat{\beta}_{\text{OLS}} = (X^T X)^{-1} X^T y$; Ridge solves $\hat{\beta}_{\text{Ridge}} = (X^T X + \lambda I)^{-1} X^T y$.

Adding λI to $X^T X$ stabilizes the inversion, shrinking coefficient estimates, and bounding their variance. When $\lambda = 0$, the solution reduces to OLS; as $\lambda \rightarrow \infty$, all coefficients shrink toward zero.

The geometry is instructive. In the **singular value decomposition (SVD)** of the feature matrix, each singular value corresponds to a direction in the feature space and measures the amount of variance the data exhibit along that direction:

- OLS estimates are least stable along directions of low variance: small denominators in the inversion produce large, noisy coefficients
- Ridge adds λ to every singular value before inverting, which disproportionately shrinks coefficients along low-variance directions while leaving well-identified directions largely intact (Hastie, Tibshirani, and Friedman, 2009)

The practical consequence: strong signals supported by substantial variation in the data survive with modest shrinkage; weak signals in poorly identified directions (precisely the ones most likely to reflect noise) are heavily attenuated.

This makes Ridge well-suited to settings where many features contribute modestly to the signal and multicollinearity is pervasive. If momentum, trend-following, and relative-strength indicators capture overlapping

information, OLS assigns weights in an erratic manner; Ridge retains all of them with reduced, stabilized weights. `02_regularization_paths` traces Ridge coefficient paths across regularization strengths and compares IC against the OLS baseline.

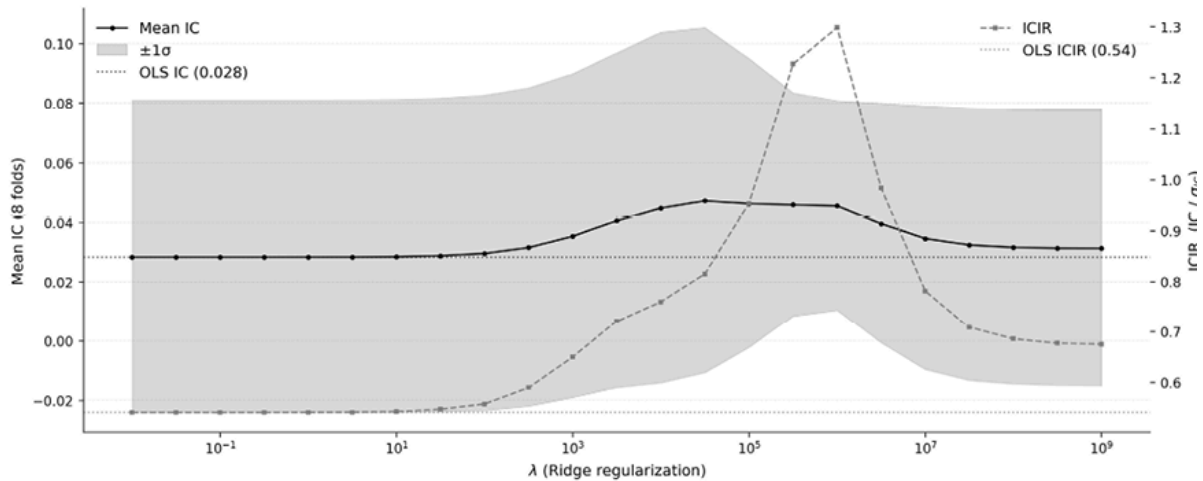


Figure 11.1: Ridge etfs performance for increasing regularization

Figure 11.1 traces Ridge prediction quality across ten orders of magnitude in penalty strength α , using the `etfs` case study with 21-day return labels ($n \approx 394,000$):

- Below $\alpha \approx 100$, the penalty is negligible, and Ridge is indistinguishable from OLS ($IC \approx 0.028$, $ICIR \approx 0.54$, right-hand scale).
- Between 10^3 and 10^6 , both IC and ICIR improve as shrinkage suppresses noisy coefficient directions; the peak ICIR of 1.30 at $\alpha \approx 10^6$ represents a $2.4\times$ improvement over the OLS baseline, driven by a 60% increase in mean IC (0.046 versus 0.028) combined with a 33% reduction in the IC standard deviation across folds.
- Mean IC peaks earlier at $\alpha \approx 3 \times 10^4$ ($IC = 0.047$) and declines slightly thereafter, but IC variance keeps shrinking faster, so ICIR continues improving until $\alpha \approx 10^6$, where it plateaus.

LASSO regression for sparse signal

LASSO replaces the L_2 penalty with an L_1 penalty:

$$\min_{\beta} \sum_{i=1}^n (y_i - X_i \beta)^2 + \lambda |\beta|_1$$

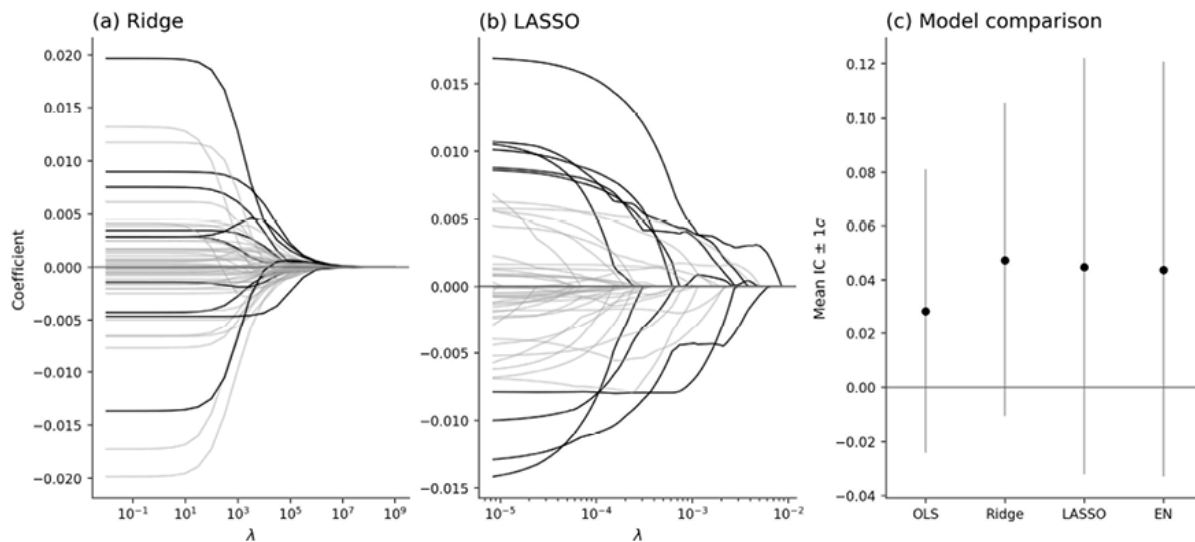
where $|\beta|_1 = \sum_j |\beta_j|$. Unlike Ridge, the L_1 penalty produces *sparse* solutions: some coefficients are exactly zeroed, performing automatic feature selection. This encodes a prior that the signal concentrates in a small subset of features: only a handful of the candidate indicators truly predict returns, and the rest are noise.

The **sparsity** arises from the geometry of the L_1 constraint set. The absolute-value penalty creates a diamond-shaped feasible region with corners on the coordinate axes. The OLS objective's contours typically intersect these corners, setting some coefficients exactly to zero (Tibshirani 1996). The L_2 penalty's circular constraint set has no corners, which is why Ridge shrinks but never zeros.

Sparsity has practical appeal: fewer active features mean fewer data pipelines to maintain and clearer PnL attribution. However, LASSO is **unstable when features are correlated**. Among a set of similar indicators, LASSO selects one at random and sets the others to zero. Which indicator survives can change with minor perturbations in training data, making the active feature set unstable across walk-forward windows. This instability does not necessarily harm predictive accuracy because the selected feature may serve as a reasonable proxy for the group. However, it complicates interpretation and can increase turnover if the trading signal depends on which specific features are active.

The LASSO coefficient path in `02_regularization_paths` shows which features enter the model first as λ decreases and how the active set shifts

across walk-forward folds (see *Figure 11.2*).



*Figure 11.2: Ridge (a) shrinks all 57 coefficients smoothly toward zero as λ increases, retaining every feature; LASSO (b) eliminates features sequentially, snapping coefficients to exactly zero at different λ thresholds. The 10 largest-magnitude features are highlighted, the remaining 47 in gray. Panel (c) compares mean out-of-sample IC ($\pm 1\sigma$) for OLS, Ridge, LASSO, and Elastic Net. All panels use standardized features from the *etfs* case study (21-day returns, single training fold).*

For the `etfs` case study, LASSO tells a different story than Ridge: even mild sparsity ($\alpha \approx 0.0001$) zeros out roughly a quarter of features, and stronger penalties *reduce* IC below OLS, while $\alpha \geq 0.01$ eliminates virtually all coefficients.

With 57 correlated momentum and volatility features, the ETF signal is diffusely distributed: Ridge's uniform shrinkage preserves it, while LASSO's variable selection discards informative features along with noise.

Combining both priors with elastic net

Elastic Net blends the L_1 and L_2 penalties:

$$\min_{\beta} \sum_{i=1}^n (y_i - X_i \beta)^2 + \lambda \left[\alpha |\beta|_1 + \frac{1-\alpha}{2} |\beta|_2^2 \right]$$

where $\lambda > 0$ controls overall regularization strength and $\alpha \in [0,1]$ controls the mixing ratio: $\alpha = 1$ recovers LASSO, $\alpha = 0$ recovers Ridge.

The combination produces a distinctive grouping behavior (Zou and Hastie, 2005). The L_2 component encourages correlated features to receive similar coefficients; the L_1 component then performs group-level selection, keeping or discarding entire clusters. For financial feature sets with many correlated indicators (momentum variants, overlapping sentiment measures, related valuation ratios), this grouping is more stable than LASSO's arbitrary within-cluster selection and more selective than Ridge's retention of everything.

Scikit-learn's `ElasticNet` parameterizes the objective as $\frac{1}{2n} |y - X\beta|^2 + \alpha \left[\ell_1_ratio \cdot |\beta|_1 + \frac{1-\ell_1_ratio}{2} |\beta|_2^2 \right]$, where `alpha` controls overall penalty strength, and `l1_ratio` controls the L1/L2 mix (`1.0` = pure LASSO, `0.0` = pure Ridge). Note the $1/2n$ normalization: `ElasticNet` averages the loss, unlike `Ridge`, which uses the unnormalized **sum-of-squares error** (**SSE**), so the `alpha` scales are not directly comparable between the two.

To begin, cross-validate over a grid of `l1_ratio` values (for example, `0.1,0.3,0.5,0.7,0.9`) and `alpha` values log-spaced over the range appropriate to n (see the scaling heuristic in the Optuna section below, adjusting for the $1/2n$ normalization). If the optimal `l1_ratio` consistently lands near `1.0` across walk-forward windows, the data favor sparsity; near `0`, the signal is diffuse across features. Unstable `l1_ratio` across windows reflects genuine variation in which feature relationships dominate. Consider averaging predictions across several mixing ratios

rather than selecting a single point estimate. The case study pipeline runs this grid search (see `06_linear` in each case study directory).

Standardization

Regularization penalties penalize all coefficients equally, whereas raw coefficients depend on the units of the features. A momentum signal, measured in percentage points, and a volume indicator, measured in millions of shares, have vastly different magnitudes. Without standardization, the penalty shrinks large-scale features more aggressively, a result driven by unit choices rather than predictive importance.

Standardize each feature to zero mean and unit variance:

$$x_j^{(\text{std})} = \frac{x_j - \mu_j}{\sigma_j}$$

After standardization, coefficient magnitudes are directly comparable: a coefficient of 0.3 on standardized momentum and 0.1 on standardized volume means the model assigns three times as much weight to momentum, regardless of the original scales.

Two implementation details are critical:

- **Fit on training data only** . Compute μ_j and σ_j from the training fold, then apply those parameters to transform validation and test data. Using full-sample moments introduces look-ahead bias. In a walk-forward protocol, recompute standardization parameters at every refit.
- **Winsorize before standardizing** . A single extreme observation can inflate σ_j , compressing all other observations toward zero. Clip at the 1st and 99th percentiles (computed from training data) before standardizing. Both errors (leaking moments and skipping

winsorization) are silent: no error is raised, but the model benefits from information unavailable at prediction time.

The walk-forward pipeline in `02_regularization_paths` applies `StandardScaler`, fitted only to the training data, at every fold, implementing the leakage-safe workflow described here.

Hyperparameter optimization with Optuna

Ridge, LASSO, and Elastic Net each introduce hyperparameters (regularization strength and, for Elastic Net, the mixing ratio) that control the bias-variance tradeoff. Grid search over a pre-specified schedule works for one or two parameters but scales poorly to joint optimization and does not adapt to the loss surface.

Optuna (Akiba et al. 2019) provides **sequential, model-based hyperparameter optimization** through a define-by-run API: the objective function samples parameters from distributions, evaluates walk-forward cross-validation performance, and Optuna's **Tree-structured Parzen Estimator** (TPE) sampler steers subsequent trials toward promising regions of the search space.

The critical risk is **validation overfitting**: running many trials on the same validation folds can select hyperparameters that exploit idiosyncrasies of the validation period rather than a durable signal. Nested cross-validation, with an inner loop for hyperparameter optimization and an outer loop for evaluation, is the cleanest correction, although it increases computation. For time series, the same principle requires nested walk-forward validation, with each outer walk-forward split evaluated only after tuning inside the corresponding training window (see *Chapter 6*).

A useful diagnostic is to examine the full distribution of validation ICs, the stability of trial rankings across folds, and the uncertainty around the selected trial's IC. A best trial that materially exceeds the rest of the search but owes its advantage to outliers in a single fold, regime, or random seed is a warning sign. Report the number of trials, the search space, the selected hyperparameters, and the fold-level dispersion of the selected configuration. *Chapter 12* develops Optuna's advanced features (pruning, multi-objective optimization, and time-series-aware tuning) for the larger hyperparameter space of gradient boosting.

Before launching an adaptive search, the search space must be calibrated to span substantively different model regimes.

This is especially important for the regularization strength because the penalty's numerical scale depends on the estimator's loss convention and feature scaling. With standardized features, `sklearn.Ridge` uses an unnormalized sum-of-squares objective, so values of alpha that materially affect the solution often scale with the sample size through the eigenvalues of $X^T X$. By contrast, `sklearn.ElasticNet` averages the loss over observations, absorbing a factor of n ; comparable penalties therefore differ by roughly this factor when translating between conventions.

In the ETF case study, with approximately 400,000 observations, a Ridge search that stopped at $\lambda = 10^2$ would mostly explore the near-OLS region and could falsely suggest that regularization has little effect. The notebook therefore maps a wider range, from 10^{-2} to 10^9 , to include both the under-regularized plateau and the strongly regularized regime.

Implementation : `04_nested_cv_hpo` sweeps λ from 10^{-2} to 10^9 across walk-forward folds. Mean IC stays mildly negative across most of the range, troughs near



$\lambda \approx 10^3$ at -0.039 , recovers monotonically as regularization tightens, and only crosses zero at $\lambda \gtrsim 10^7$ to reach a noisy peak of $\approx +0.007$ at the top of the search range. The fold-level standard deviation ranges from roughly 0.04 to 0.10 across the grid and dwarfs the cross- λ mean, flagging the landscape as noise-dominated. The notebook then implements both single-loop and nested cross-validation with Optuna, quantifying the selection-bias inflation (Cawley and Talbot, 2010).

When the underlying signal is weak, the protocol distinction is exactly when selection bias matters most. Here, it is large enough to flip the reported sign: single-loop CV reports a positive mean IC of $+0.028$; nested CV returns -0.032 on identical splits. We illustrate these principles by first optimizing a single hyperparameter (Ridge λ); *Chapter 12* extends to the joint tuning of multiple hyperparameters.

Box 11.1: Incremental updates

The walk-forward protocol above retrains from scratch at every refit. For high-frequency case studies, such as Crypto (8-hour) and NASDAQ-100 (minute bars), full retraining is computationally expensive.

Scikit-learn's `SGDRegressor` and `SGDClassifier` support `partial_fit` for incremental updates: present a single mini-batch of new observations and update the coefficients via one gradient step, rather than re-solving the full optimization. This is the fastest for online deployment but introduces sensitivity to the learning rate schedule and can drift without periodic full retraining.



Chapter 25 develops the full online learning framework for live trading systems.

Loss function choice

The regularizers above are usually paired with **mean squared error** (**MSE**) by default, especially in standard Ridge and Lasso regression. But the penalty does not require MSE: you can combine the same regularization idea with other loss functions when a different target (such as a median, a robust central tendency, or a tail quantile) better matches the trading objective.

MSE is a natural choice when the goal is to estimate the conditional mean of the return distribution, but the conditional mean may not be the right target:

- **MSE** penalizes errors quadratically, forcing the model to accommodate extreme observations that may reflect idiosyncratic events rather than systematic patterns.
- **Mean absolute error** (MAE) penalizes linearly and estimates the conditional *median* rather than the mean. Because outliers shift the median far less than the mean, MAE produces more stable coefficients when a few large moves dominate.
- **Huber loss** transitions from quadratic for small errors (like MSE) to linear for large errors (like MAE) at a threshold δ , chosen as a tuning parameter or scaled to a robust estimate of the residual dispersion, preserving MSE's efficiency for typical observations while limiting outlier influence (Huber, 1964).

All three losses estimate a location parameter of the conditional return distribution. Scikit-learn's `SGDRegressor` provides a unified interface: `loss='squared_error'` , `loss='epsilon_insensitive'` (MAE proxy),

and `loss='huber'` all accept the same regularization penalties and `sample_weight` parameter. `o2_regularization_paths` compares all three at the best Ridge alpha, isolating the loss function's effect on IC.

Quantile regression changes the estimand entirely, predicting specified percentiles rather than a central tendency. If a long-short strategy trades only the top and bottom deciles, the conditional mean is the wrong target: most of the model's capacity is spent distinguishing among assets that will never be traded. *Section 11.5* develops quantile regression as the foundation for conformalized prediction intervals.

Evaluating regression models

Training losses determine what the model optimizes; evaluation metrics determine how we judge the result. The appropriate metric depends on how predictions map to trades:

- **Information Coefficient (IC).** Spearman's rank correlation between predicted and realized returns is the primary metric for cross-sectional models that generate ranked signals. IC captures monotonic association without assuming linearity. In general, the coefficient of variation is the ratio of the mean to the standard deviation for a given metric; for the IC, this measure is also called the **IC Information Ratio (ICIR)** ; see *Chapter 7*).
- **RMSE and MAE.** RMSE, the square root of the MSE, measures the standard deviation of prediction errors; MAE measures the average absolute error and is less sensitive to outliers. Both are useful for relative model comparison within the same dataset and label definition, but neither has a standalone threshold for “good” performance: the relevant benchmark is always a naive model that, for example, predicts the cross-sectional mean.

- **Turnover.** No statistical metric measures tradability: a model that improves IC by 20% but triples turnover may degrade net performance after transaction costs. Report turnover alongside every metric. *Section 11.6* examines this tradeoff empirically across all nine case studies; *Chapter 17* formalizes the turnover-adjusted evaluation framework.

Since IC measures global ranking quality, strategies that trade only the extremes should also track quintile or decile spreads to verify that ranking accuracy translates into separation in the traded tails.



Implementation : `@2_regularization_paths` reports IC, RMSE, and R^2 across all four model types for the `etfs` case study. The stability of predicted rankings for the same test data, measured as the correlation between cross-sectional rankings from models trained on consecutive windows, diagnoses signal robustness to retraining; `@2_regularization_paths` computes this for Ridge across all fold pairs.

Sample weighting

Most scikit-learn estimators accept a `sample_weight` parameter, allowing observations to contribute unequally to the loss function. Two weighting schemes from *Section 7.2* apply directly:

- **Uniqueness weighting** ($w_{t,a}$ = average inverse concurrency) corrects for label overlap when using the $\bar{H}$ -bar labels defined in *Chapter 7*. Overlapping labels violate the independence assumption implicit in the sum-of-squared-errors objective; down-weighting observations with high concurrency partially mitigates this.

- **Recency weighting** ($w_t^{\text{recency}} = e^{-\lambda(T-t)}$) gives more influence to recent observations, adapting the model to regime evolution. The decay rate λ controls the effective lookback horizon.

The two compose multiplicatively: $w_{t,a}^{\text{combined}} = w_{t,a}^{\text{uniqueness}} \cdot w_t^{\text{recency}}$.

Report the effective sample size $N_{\text{eff}} = \sum w_{t,a}^{\text{combined}}$ alongside performance metrics. A small N_{eff} relative to the nominal sample size signals that a few observations dominate the fit, increasing the risk of overfitting to those observations.



Implementation : `@2_regularization_paths` shows recency weighting with exponential decay in the final walk-forward fold and reports the effective sample size.

Training loss versus trading objective

A model can reduce MSE while degrading PnL if the improvement comes from fitting the middle of the cross-section (assets that are never traded) or from increasing turnover. We therefore maintain a strict separation between *model selection* (validation loss and rank metrics within the leakage-safe walk-forward protocol) and *strategy evaluation* (turnover- and cost-adjusted returns, *Chapter 18*). Without this separation, in-sample statistical improvements masquerade as trading value. We now turn to a different prediction task that focuses on direction, or extreme moves.

11.3 Predicting direction with logistic regression

The previous section treated the label as a continuous return. Many trading setups, however, operate on discrete decisions (go long, go short, or stay flat), and the label families defined in *Chapter 7* include binary up/down flags, three-class directional labels, and quantile memberships. For these setups, classification models that directly predict discrete outcomes are the natural choice, and logistic regression is the regularized baseline.

Classification is not universally superior to regression. Direction prediction can misalign with PnL when payoffs are asymmetric: a model that correctly predicts direction 55% of the time but systematically misses large moves may underperform a regression model that captures magnitude in the tails.

The choice should follow from the *signal-to-trade mapping* defined in *Chapter 7*. If the mapping applies a threshold or quantile rule to convert predictions into positions, classification fits naturally because the model directly optimizes the decision that matters. If the mapping scales position size in proportion to the predicted return, regression preserves the magnitude information that classification discards. Ultimately, however, what works better in a given case is an empirical matter.

Binary logistic regression

Logistic regression models the probability that the outcome belongs to the positive class:

$$P(y = 1 \mid X) = \frac{1}{1 + e^{-X\beta}} = \sigma(X\beta)$$

where $\sigma(\cdot)$ is the sigmoid function, mapping any real-valued linear combination to the $(0,1)$ interval. The model learns β by maximizing the **log-likelihood** of the observed labels, or equivalently by minimizing the

cross-entropy (log-loss) between predicted probabilities and observed outcomes.

The key difference from linear regression is that logistic regression produces a *probability estimate* rather than a point forecast of the return. A prediction of **0.65** means that, under the fitted logistic specification, the model assigns a 65% probability to an up move. This interpretation is justified by the log-loss objective: in population, cross-entropy is minimized by the true conditional probability $P(y = 1 | X)$, provided the model class is rich enough and the data-generating process is stable.

In practice, however, the fitted probabilities *may not be well calibrated* due to misspecification, finite sample size, regularization, class weighting, resampling, noisy labels, or distribution shift. The output should therefore be treated as a probability estimate, not automatically as a calibrated empirical frequency. Calibration diagnostics or post-hoc calibration may be needed before using these probabilities for risk-adjusted position sizing (discussed later in this section).

Each coefficient β_j represents the change in log-odds per unit change in x_j . After standardization, a coefficient of **0.5** on momentum means that a one-standard-deviation increase in momentum raises the log-odds of an up move by **0.5**. At a baseline probability of **50**, this changes the predicted probability to $\sigma(0.5) \approx 62$. At a baseline probability of **90**, the same log-odds increase changes the predicted probability to approximately **94**. The probability-point effect is therefore not constant: **coefficients are linear on the log-odds scale**, not on the probability scale.

Classification labels collapse return magnitudes into categories: a 50% return and a 2% return both count as “up.” This is simultaneously a strength and a limitation. It prevents extreme observations from dominating the loss function, as they would with MSE in regression,

producing more stable coefficient estimates in the presence of fat tails. But it discards valuable magnitude information. The trade-off may favor classification when the trading decision is binary (long or short) and regression when position size varies with the predicted magnitude.



Implementation : `03_logistic_classification` traces the full walk-forward binary logistic regression pipeline on ETF data, including the label construction and the L2/L1 regularization sweeps described below.

Multi-class extension

The binary model extends to K classes via **multinomial logistic regression** (**softmax**). For the three-class label, with classes long (+1), neutral (0), and short (−1), the model estimates:

$$P(y = k | X) = \frac{e^{X\beta_k}}{\sum_{j=1}^K e^{X\beta_j}}$$

Each class receives its own coefficient vector β_k , and probabilities sum to one across classes (`multi_class='multinomial'` in scikit-learn). The neutral zone filters observations near the zero-return boundary, reducing label noise, but each additional class adds a full coefficient vector, increasing parameter count by roughly $K - 1$.

For quantile strategies trading only the extremes, a binary framing (top decile versus bottom decile, discarding the middle) is typically more efficient than a multi-class model that must also learn middle categories.

Regularization

The L_1 , L_2 , and Elastic Net penalties from *Section 11.2* apply identically in form, added to the negative log-likelihood rather than the sum of squared residuals:

$$\min_{\beta} - \sum_{i=1}^n [y_i \log \sigma(X_i \beta) + (1 - y_i) \log(1 - \sigma(X_i \beta))] + \lambda |\beta|_2^2$$

Scikit-learn's `LogisticRegression` uses $C = 1/\lambda$ (higher C means less regularization), with `penalty` and `l1_ratio` controlling the penalty type. All guidance from *Section 11.2* carries over.

Regularization is more critical here than in regression: the maximum likelihood objective diverges when classes are perfectly or nearly separable. In high-dimensional feature spaces where p approaches n , a separating hyperplane almost always exists, driving coefficients toward $\pm\infty$ and producing arbitrarily confident predictions that fail out of sample. Regularization bounds coefficients, ensuring finite solutions, a concern absent from OLS, which has a closed-form solution whenever $X^T X$ is invertible.

Probability calibration

Under correct specification, logistic regression is calibrated by construction: if the model predicts $P(y = 1 | X) = 0.7$ for a set of observations, approximately 70% of them will have $y = 1$. In practice, two forces distort this correspondence:

- **Regularization** systematically compresses predicted probabilities toward the base rate. Shrinking coefficients toward zero flattens the sigmoid, pulling extreme predictions toward 0.5. This is a predictable, directional distortion: the model becomes systematically underconfident in its predictions.

- **Misspecification** introduces less predictable distortions. If the true decision boundary is nonlinear but the model is linear, calibration breaks in ways that depend on the specific form of the misspecification.

Calibration matters when position sizes depend on predicted probability levels: if a probability of 0.7 triggers a larger position than 0.6, miscalibrated probabilities translate directly into misallocated capital:

- **Platt scaling** (fitting a logistic function to the model's raw outputs on a held-out set) corrects for the systematic compression caused by regularization.
- **Isotonic regression** (fitting a non-parametric monotone function) handles more complex miscalibration patterns but requires more data to estimate reliably (Niculescu-Mizil and Caruana, 2005).

Both must use data not used for training: the validation fold in the walk-forward protocol serves this purpose. Scikit-learn's `CalibratedClassifierCV` automates the procedure.

Bin predictions by predicted probability (for example, $[0.5, 0.55)$, $[0.55, 0.6)$, ...) and compute accuracy within each bin. Monotonically increasing hit rates as predicted probability moves away from 0.5 indicate well-calibrated confidence; non-monotonic patterns (high-confidence predictions performing worse than moderate-confidence ones) suggest overfitting or misspecification.

For threshold-based and rank-based signal generation, calibration is less critical because only the *ordering* of predictions matters. Assess whether the signal-to-trade mapping depends on probability levels or only on ranks before investing effort in calibration.



Implementation : `03_logistic_classification` produces calibration curves for the walk-forward



predictions and compares them against perfect calibration.

From probabilities to trading signals

The conversion from predicted probabilities to positions should match the trading setup defined in *Chapter 7*. Common approaches include:

- **Threshold-based** : Go long if $P(y = 1 | X) > \tau_{\text{upper}}$, short if $P(y = 1 | X) < \tau_{\text{lower}}$, flat otherwise. The thresholds control selectivity: $\tau_{\text{upper}} = 0.6$ and $\tau_{\text{lower}} = 0.4$ is a common starting point, but optimal values depend on transaction costs and the base rate. Asymmetric thresholds accommodate asymmetric costs of long versus short positions.
- **Probability-weighted** : Position size is proportional to the distance from 0.5, expressing graduated confidence. This requires calibrated probabilities: if the model systematically underestimates confidence (as regularization tends to do), positions will be systematically undersized relative to the true signal.
- **Rank-based** : Within a cross-section, rank assets by predicted probability and go long the top decile, short the bottom decile. This ensures a fixed number of positions regardless of the probability distribution and depends only on ordering, making it robust to miscalibration.

Thresholds and quantile cutoffs must use only information available at decision time: the walk-forward protocol (*Chapter 6*) enforces this by recomputing them within each training window.



Implementation : `03_logistic_classification`

compares all three conversion methods on the same fold, showing that identical predictions yield substantially



different portfolios. *Chapter 16* extends this comparison across full backtests.

Handling class imbalance

When predicting rare events (the 5% of stocks exceeding two standard deviations, or the minority class in an asymmetrically thresholded label), the minority class contributes little to the log-likelihood. The model minimizes loss by predicting the majority class for all observations, achieving high accuracy but generating no trading signal.

Class weighting scales each class's loss contribution inversely to its frequency, ensuring that minority-class errors carry proportionally more weight. Scikit-learn's `class_weight='balanced'` computes these weights automatically. This does not change the decision boundary's location in feature space but adjusts how aggressively the optimizer penalizes errors on each side. `03_logistic_classification` compares default and balanced class weighting on the last fold, confirming that the effect is modest when classes are roughly balanced.

When combining class weights with the uniqueness and recency weights from *Section 11.2*, monitor N_{eff} : a sharp drop below the nominal sample size signals that a few observations dominate the fit. This risk is acute when minority-class observations cluster in crisis periods. Verify that reweighted training sets span multiple market conditions rather than concentrating on a single regime.

Evaluating classification models

Classification evaluation requires metrics suited to the probabilistic and discrete nature of the outputs. As with regression (*Section 11.2*), the appropriate metric depends on how predictions map to trades:

- **AUC (area under the ROC curve)** measures the quality of ranking across all possible classification thresholds. An AUC of 0.5 indicates random ranking; values of 0.55–0.60 represent meaningful predictive power for financial applications, where signal-to-noise ratios are low. Values above 0.65 on out-of-sample financial data warrant scrutiny: they may indicate data leakage or an evaluation window that happens to align with a strong trend.
- **Precision and recall.** Precision (fraction of predicted positives that are true positives) guards against wasted trades; recall (fraction of true positives that are predicted) guards against missed opportunities. The trade-off depends on costs: high transaction costs favor precision (fewer, more accurate trades); strategies with capacity constraints favor recall (capturing all available opportunities). The F1 score balances both but assigns equal weight to each, which may not reflect the trade-offs in economics.
- **Log-loss.** If the signal-to-trade mapping uses probability levels rather than ranks, log-loss directly evaluates the quality of the probability estimates. It penalizes confident wrong predictions heavily, making it a useful complement to AUC in settings where calibration matters. The Brier score (the mean squared difference between predicted probabilities and binary outcomes) provides a complementary calibration-sensitive metric.



Turnover

As with regression, report turnover alongside every classification metric. A model that improves AUC by shifting the decision boundary (changing which assets are traded each period) may increase turnover beyond what the strategy can absorb.

Now that we have familiarized ourselves with the inner workings of the most popular linear models, let us take a closer look at model interpretability.



Implementation : `03_logistic_classification` shows the full workflow on the ETF case study. The per-case-study notebooks apply both regression and classification pipelines across all nine datasets.

11.4 Interpreting models with SHAP

A model that predicts returns but cannot explain why offers limited value. Understanding which features drive predictions (and whether those relationships make economic sense) separates robust signals from spurious curve-fitting. **SHapley Additive exPlanations** (SHAP) is the primary interpretability framework, embedded in the walk-forward pipeline as a continuous diagnostic.

The case for interpretability rests on three pillars:

- **Diagnosis** : When a model fails in production, you need to know why. A Ridge model that suddenly underperforms might be responding to a vendor data migration that changed how momentum is computed, or to genuine decay in the momentum premium. SHAP analysis of recent versus historical predictions can distinguish these scenarios.
- **Economic sanity-checking** : A momentum feature should positively contribute to high-momentum stocks; a volatility feature should reflect the expected risk-return relationship. When attributions

contradict established theory, the model is more likely to fit noise than capture signal.

- **Governance** : Model risk frameworks such as SR 11-7 (Federal Reserve) and SS1/23 (Bank of England) require that firms understand and justify algorithmic outputs. Even without regulatory obligations, “the model said so” does not explain a drawdown.

For standardized linear models, coefficient magnitudes offer a tempting shortcut, but shrinkage entangles importance with the penalty strength, correlated features split or steal credit depending on the penalty type, and coefficient interpretation does not generalize beyond linear models.

SHAP supplies a unified attribution language: it disentangles regularization distortion, allocates credit fairly among correlated features, and carries forward unchanged to trees (*Chapter 12*) and deep learning (*Chapter 13*).

Game-theoretic attribution with SHAP

SHAP builds on **Shapley values** from cooperative game theory, which determine each player’s contribution to the outcome. Here, the “players” are features, the “payoff” is the gap between a specific prediction and the model’s average prediction (the baseline), and the question is: how much did each feature contribute to moving this prediction away from baseline?

The *Shapley value* for feature j averages its marginal contribution across all orderings in which features could enter the model:

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(S \cup \{j\}) - f(S)]$$

where F is the full feature set, S ranges over subsets excluding j , and $f(S)$ is the expected prediction when only features in S are known.

Shapley (1953) proved that this is the **unique allocation satisfying four axioms** :

- Efficiency: attributions sum exactly to the prediction minus baseline
- Symmetry: identically contributing features receive equal credit
- Null player: irrelevant features receive zero credit
- Linearity: attributions for a sum of models equal the sum of attributions

Lundberg and Lee (2017) unified **LIME** , **DeepLIFT** , and classic Shapley regression within this framework, showing that SHAP is the unique additive attribution method that satisfies all four axioms.

Efficient computation

Exact computation requires evaluating $2^{|F|}$ subsets, which is intractable for realistic feature sets. Model-specific algorithms make it practical:

- **LinearSHAP** exploits the closed form for linear models:
$$\phi_j = \beta_j \cdot (x_j - E[x_j])$$
, computed over the training set. This is exact and $O(F)$ per observation, the right choice for this chapter.
- **TreeSHAP** (*Chapter 12*) and **KernelSHAP** (*Chapter 13*) extend the framework to nonlinear models.



Implementation : `05_shap_analysis` shows LinearSHAP end-to-end on the ETF case study, including verification that SHAP values match the closed-form coefficient attribution.

Practical application in global and local analysis

SHAP supports both global views (feature importance summarized across all predictions) and local views, decomposing the contribution of each feature to a single model score.

Global feature importance

SHAP summary plots rank features by mean $|\phi_j|$ across all test observations. Each point represents the SHAP value for one observation, colored by its feature value. The plot reveals which features matter most, the direction of effects, and whether importance is consistent or conditional.

In the `etfs` case study, volatility horizons dominate the top of the ranking: six-month volatility (`vol_126d`) contributes most, with a positive feature-to-SHAP relationship, while three-month volatility (`vol_63d`) is the second-most important and pulls in the opposite direction. Risk-adjusted return appears next (`sharpe_63d` , positive), followed by `vol_ratio_medium` and `yield_curve_slope` .

SHAP importance can differ from coefficient rankings: a feature with a moderate coefficient but high variance at informative observations can have a higher mean $|\phi_j|$ than a feature with a larger coefficient but lower variance. *Figure 11.3* shows the beeswarm summary for the ETF case study.

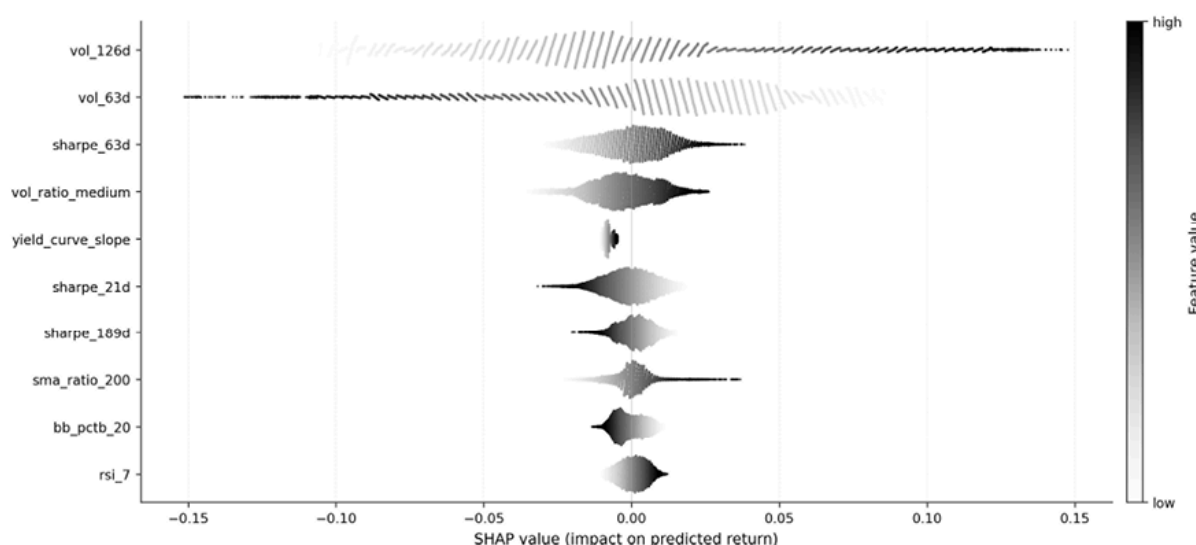


Figure 11.3: Importance of the top 20 etfs features

Local explanations

Waterfall plots decompose a single prediction from baseline to final value, feature by feature. For a high-conviction stock: baseline of 0.5% → momentum adds +0.8% → mean reversion subtracts −0.3% → volatility adds +0.2% → final prediction of 1.4%. This enables two checks:

1. An *attribution audit* : is outperformance driven by trusted features or suspicious ones (an illiquidity proxy reflecting data staleness)?
2. A *concentration-risk flag* : a prediction dominated by a single feature is fragile.

In practice, flagging predictions in which a single feature accounts for more than a certain threshold of the total $|\phi_j|$ could provide an automated alert for manual review or position-size reduction. *Figure 11.4* contrasts a correct high-conviction prediction whose attribution spreads across ten features with an incorrect one in which a single liquidity-proxy feature carries most of the model's conviction; the right-panel pattern is exactly what the concentration-risk flag is designed to catch.

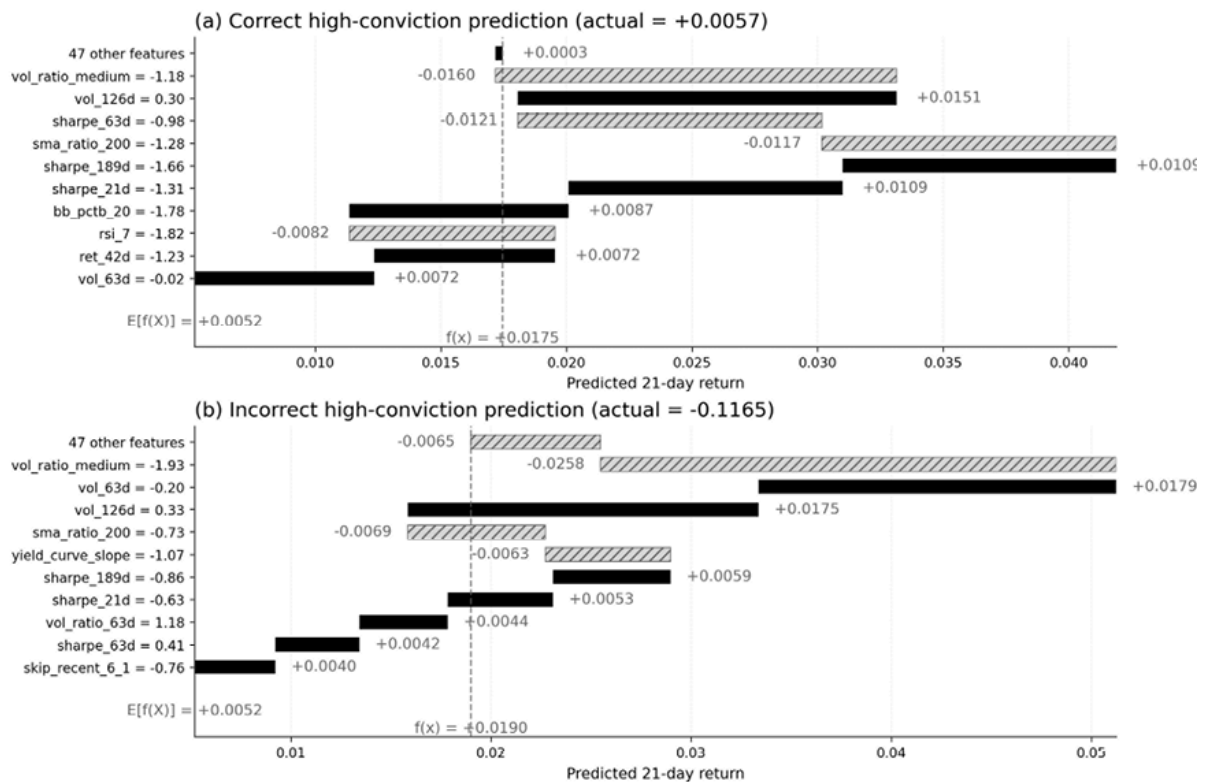


Figure 11.4: SHAP waterfall decomposition of two high-conviction predictions from the `etfs` case study (Ridge, 21-day forward returns). Both predictions sit in the top 5% by predicted magnitude. Panel (a): a correct call where ten features contribute roughly evenly, and the prediction reflects diversified evidence. Panel (b): an incorrect call where `vol_63d` (volume ratio) alone contributes +0.018, yet the realized return was -11.7%

Implementation : `05_shap_analysis` includes waterfall decompositions for both correct and incorrect high-conviction predictions.

Feature dependence

SHAP dependence plots show how a feature's SHAP value varies with its own value, colored by an interacting feature. For linear models, the relationship is linear by construction, but the plot still serves as a sanity check for sign consistency and outlier behavior.

For tree-based and neural network models (*Chapters 12 and 13*), dependence plots reveal threshold effects, saturation, and interaction

structures that are invisible in coefficient tables.

Building an economic narrative

SHAP's primary value for systematic investing is epistemic: it provides a structured protocol for distinguishing genuine signal from overfitting, organized in four layers:

- **Sign consistency** : does each feature's contribution have the expected sign? A model that has learned the opposite of every documented factor relationship is almost certainly fitting noise.
- **Magnitude plausibility** : Does a single feature imply a 5% monthly return contribution in a universe where median monthly returns are 0.8%? If so, the model is extrapolating.
- **Stability across walk-forward windows** : Compute SHAP values per fold and track whether the same features rank in the top five, maintain consistent signs, and show stable magnitudes. Plotting mean $|\phi_j|$ over time produces a "SHAP stability chart," one of the most informative validation diagnostics (*Figure 11.5*).
- **Regime-conditional analysis** : Partition test data by volatility tercile or regime label (*Chapter 7*) and compute SHAP summaries within each partition. A model that shifts from momentum to mean reversion across regimes may be economically sensible, but only if regime identification is itself robust.

A refinement: fold-to-fold variation in SHAP rankings conflates temporal regime shifts (interesting) with finite-sample estimation noise (uninteresting). Bootstrapping SHAP values within a single fold provides confidence intervals on feature importance, separating the two sources.

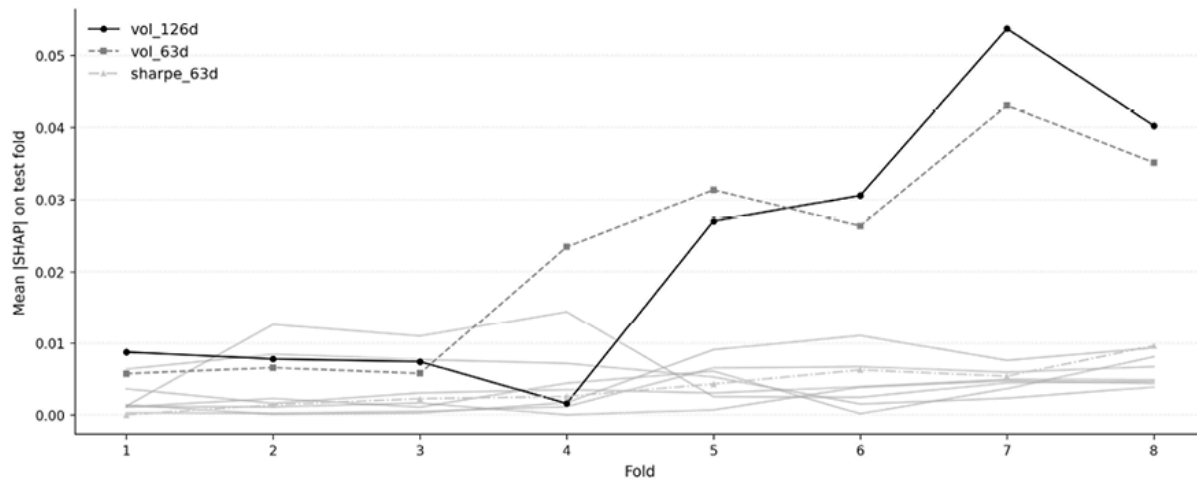


Figure 11.5: SHAP stability across folds

A complementary diagnostic compares SHAP **profiles for high-conviction predictions** that were directionally correct versus those that were incorrect. Features where the model relied heavily on wrong predictions (measured by the ratio of mean $|\phi_j|$ for incorrect to correct cases, or by large directional differences in mean signed SHAP) are candidates for re-engineering or nonlinear modeling. This right-versus-wrong decomposition targets the predictions that matter most for PnL.

Limitations and failure modes

Causal misinterpretation is a major risk. A high SHAP value for feature j means it contributed to *this model's prediction*, not that it caused the return. A feature such as `obv_zscore_63d` with high SHAP importance could reflect genuine predictive power, correlation with an omitted predictor, or a pipeline confound. Kumar et al. (2020) formalize how Shapley-based explanations can mislead when features are correlated or when they are interpreted causally. Use SHAP for descriptive attribution (“what is the model responding to?”), not causal claims.

Feature dependence and *impossible coalitions* are also significant issues. Standard SHAP marginalizes out absent features independently, creating

impossible feature combinations when features are correlated. For linear models, this is manageable; for nonlinear models, Aas et al. (2021) propose **conditional SHAP**, which requires estimating conditional distributions. **Permutation importance** and LIME lack SHAP's axiomatic guarantees: permutation importance inflates scores for correlated features; LIME attributions need not sum to the prediction.

Integration with the walk-forward pipeline

The SHAP stability chart across folds serves as a model-selection tool alongside IC and coverage metrics, providing qualitative evidence that the model has learned stable, economically sensible relationships.

The SHAP stability chart in `05_shap_analysis` tracks mean $|\phi_j|$ across all walk-forward folds for the top features. Models that pass quantitative hurdles yet exhibit unstable or economically implausible SHAP patterns warrant skepticism, regardless of their reported performance.



Implementation : `05_shap_analysis` implements SHAP computation within the walk-forward framework, including summary plots, waterfall decompositions, and right-versus-wrong diagnostic comparisons for high-conviction predictions. *Section 11.5* turns from explaining individual predictions to bounding them: how to attach calibrated uncertainty to each forecast.

11.5 Quantifying predictive uncertainty

A point forecast is incomplete because it suppresses information about the reliability of the estimate. In trading, this matters directly: the same predicted return can imply very different decisions depending on how uncertain the forecast is.

Consider a Ridge regression model that predicts a 2% return for two ETFs. For the first ETF, the current feature values lie near the center of the training distribution. For the second, volatility has spiked, liquidity has deteriorated, and several features sit near their historical extremes. The point forecast is the same, but the second prediction is much less reliable. Treating both forecasts as equivalent inputs to portfolio construction would overstate the precision of the second signal.

Classical **prediction intervals** often rely on assumptions that are fragile in financial data. Gaussian intervals of the form $\hat{y} \pm z_{\alpha/2} \hat{\sigma}$ assume a useful residual scale estimate and often rely, explicitly or implicitly, on approximately normal, homoskedastic errors. Asset returns instead exhibit heavy tails, volatility clustering, nonlinear residual dispersion, and regime dependence. The uncertainty around a forecast is rarely constant across assets, horizons, or market states.

Conformal prediction offers a model-agnostic alternative. It acts as a calibration layer placed on top of an existing forecasting model. After fitting the base model, we evaluate its forecast errors on a held-out calibration set and use the empirical distribution of those errors to construct prediction intervals or prediction sets for future observations. Because the procedure uses observed forecast errors rather than a parametric error model, it does not require normal residuals, homoskedasticity, or a correctly specified likelihood.

The key assumption behind the formal guarantee is **exchangeability**. A sequence of observations is exchangeable if its joint distribution is unchanged by reordering the observations. Independent and identically

distributed observations are exchangeable, but exchangeability is weaker than independence. This assumption matters because conformal prediction relies on the rank of the next observation's calibration score among the scores already observed. If the calibration scores and the next test score are exchangeable, the future score is equally likely to occupy any rank, which yields finite-sample marginal coverage.

Under exchangeability, conformal prediction delivers a finite-sample **marginal coverage guarantee**. For example, a 90% conformal interval will contain the realized outcome at least 90% of the time over repeated samples. This guarantee is distribution-free: it does not depend on the correctness of the base model, the shape of the return distribution, or the sample size. It is, however, a marginal guarantee. It does not imply that coverage is exactly 90% within every asset, fold, volatility regime, or predicted-return bucket.

Financial time series are not exchangeable in the strict sense because order matters. Autocorrelation, volatility clustering, changing liquidity, and structural breaks mean that calibration residuals from one period may not represent those in the next. Conformal prediction remains useful in this setting, but the theorem no longer mechanically justifies coverage. In financial applications, conformal intervals should therefore be treated as empirically calibrated uncertainty estimates whose coverage must be monitored, stress-tested, and adapted over time.

Split-conformal prediction

The simplest practical variant is split-conformal prediction (Papadopoulos et al., 2002; Lei et al., 2018). It separates model fitting from uncertainty calibration:

1. **Partition the data**. Divide the available training observations into a proper training set and a calibration set. The proper training set fits

the model and selects any hyperparameters. The calibration set must remain untouched until the fitted model is evaluated for uncertainty calibration.

2. **Compute calibration scores .** For each calibration observation i , compute a nonconformity score. For a symmetric regression interval, the usual score is the absolute residual: $s_i = |y_i - \hat{f}(x_i)|$.

These scores estimate the empirical distribution of the model's out-of-sample forecast errors.

3. **Compute the conformal quantile .** Let n_{cal} be the number of calibration observations, and let:

$$s_{(1)} \leq s_{(2)} \leq \dots \leq s_{(n_{\text{cal}})}$$

denote the sorted calibration scores. For target miscoverage rate α , define:

$$k = \lceil (n_{\text{cal}} + 1)(1 - \alpha) \rceil$$

The conformal correction is the conservative order statistic $\hat{q} = s_{(k)}$. This corresponds to using the higher empirical quantile rather than an interpolated quantile. If $k > n_{\text{cal}}$, the calibration set is too small to provide a finite conformal quantile at the desired coverage level.

4. **Construct prediction intervals .** For a new observation x , the split-conformal interval is:

$$\mathcal{C}(x) = [\hat{f}(x) - \hat{q}, \hat{f}(x) + \hat{q}]$$

This interval has fixed width $2\hat{q}$ for every observation. That simplicity is useful: split conformal is easy to implement, easy to audit, and robust to misspecification of the base model. Its limitation is equally clear: it cannot vary the interval width across market regimes, assets, or feature

configurations. A forecast made during a volatility spike receives the same residual correction as one made during a quiet regime.

Under exchangeability of the calibration observations and the next test observation, split-conformal prediction provides **finite-sample marginal coverage** :

$$P(Y_{n+1} \in C(X_{n+1})) \geq 1 - \alpha$$

This guarantee holds for any fitted base model and any outcome distribution, provided the calibration set was not used to fit or tune the model whose scores are being calibrated. The guarantee is marginal: it averages over the joint distribution of future observations. It does not ensure correct coverage conditional on a particular date, asset, volatility state, sector, or signal-strength bucket.



Implementation : `@6_conformal_prediction` implements split-conformal prediction with Ridge regression on the ETF case study.

The exchangeability problem

The exchangeability assumption is fragile in financial time series. The order of observations carries information: volatility clusters, correlations change, liquidity varies, and regimes shift. Randomly permuting daily returns or cross-sectional residuals would destroy part of the data-generating structure. As a result, the rank argument behind the conformal guarantee no longer applies exactly. Yesterday's calibration residuals may not be representative of tomorrow's forecast errors.

This does not make conformal prediction unusable. It changes how the method should be interpreted. In finance, conformal prediction is best viewed as a disciplined empirical calibration procedure whose formal

guarantee is exact under exchangeability and approximate under sufficiently stable nonexchangeable conditions. Coverage should therefore be measured out of sample, monitored by regime, and updated when the calibration distribution becomes stale.

Several adaptations are useful in practice:

- **Rolling calibration windows** . Instead of using all historical calibration scores, compute $\hat{q}$ from a recent window, such as the most recent 250 trading days. This makes the score distribution more responsive to current volatility and liquidity conditions. The trade-off is higher estimation noise because fewer calibration observations are used.
- **Weighted conformal calibration** . Assign larger weights to more recent or more relevant calibration observations. Exponential time decay is a practical heuristic for nonstationary time series. More formal weighted conformal methods are justified under specific distributional shift assumptions, such as covariate shift, when the relative likelihoods of calibration and test covariates can be estimated.
- **Regime-conditional calibration** . Partition the calibration scores by pre-specified market states, such as realized-volatility quantiles, VIX terciles, liquidity buckets, or macro regimes, and compute separate conformal corrections within each state. This can improve conditional coverage when residual dispersion differs sharply across regimes. The partition should be defined ex ante, and each regime must contain enough calibration observations to estimate a stable quantile.
- **Online coverage feedback** . Adaptive conformal methods monitor realized coverage as outcomes arrive and adjust the effective miscoverage rate or calibration threshold. Recent misses widen subsequent intervals; persistent overcoverage narrows them. This

provides a practical feedback mechanism when the historical calibration distribution no longer matches current conditions.

These adaptations do not fully restore the original exchangeability theorem. They make the calibration layer more responsive to the types of drift and dependence that matter in financial forecasting.

Adaptive conformal inference

Adaptive Conformal Inference (ACI) (Gibbs and Candès, 2021, 2023) modifies conformal calibration for online, nonstationary environments. Rather than fixing the miscoverage level α once, ACI updates an effective miscoverage parameter α_t after each realized outcome.

At time t , the model constructs a conformal set $C_t(X_t)$ using the current value α_t . After observing Y_t , the update is:

$$\alpha_{t+1} = \alpha_t + \gamma(\alpha - \mathbf{1}\{Y_t \notin C_t(X_t)\})$$

where α is the target miscoverage rate and $\gamma > 0$ is the learning rate.

The sign of the update is intuitive. If the interval misses, then $\mathbf{1}\{Y_t \notin C_t(X_t)\} = 1$, so α_{t+1} decreases. A lower miscoverage level produces a wider next interval. If the interval covers, then the indicator equals zero, so α_{t+1} increases and the next interval narrows.

Undercoverage triggers expansion; persistent overcoverage triggers contraction.

ACI provides a long-run coverage control mechanism for nonstationary sequences. Its guarantee is weaker than the finite-sample marginal guarantee of split conformal under exchangeability: it targets average empirical coverage over time rather than exact coverage for each time point, subgroup, or regime. This distinction is important in trading. ACI can help the interval sequence recover after drift, but it does not

eliminate the need to inspect coverage by fold, volatility regime, asset group, and signal bucket.

The *learning rate* γ controls the speed-stability trade-off. Larger values adapt faster after misses but can make interval widths oscillate. Smaller values produce smoother intervals but may respond too slowly to abrupt changes in volatility. TheT notebook tunes values such as $\gamma \in [0.005, 0.05]$ within the walk-forward framework. In implementation, the adaptive miscoverage level α_t should be clipped to a bounded range, such as $[0.001, 0.999]$, to avoid degenerate intervals.



Implementation : `06_conformal_prediction` shows the ACI online update, the resulting α_t trajectory, and the effect of adaptive feedback on coverage and interval width.

Conformalized quantile regression

Split-conformal intervals have constant width, which is often unrealistic in financial data. **Conformalized Quantile Regression** (CQR; Romano et al., 2019) addresses this limitation by combining conditional quantile models with conformal calibration.

For target coverage $1 - \alpha$, CQR proceeds as follows:

1. Train lower and upper quantile models. On the proper training set, estimate conditional quantiles:

$$\hat{q}_{\alpha/2}(x) \quad \text{and} \quad \hat{q}_{1-\alpha/2}(x)$$

For 90% coverage, these correspond to the 5th and 95th conditional quantiles.

2. Compute CQR calibration scores. On the calibration set, compute:

$$s_i = \max\{\hat{q}_{\alpha/2}(x_i) - y_i, y_i - \hat{q}_{1-\alpha/2}(x_i)\}$$

A positive score indicates that the realized outcome fell outside the estimated quantile band. A negative score indicates that the outcome fell inside the band, with room to spare.

3. Compute the conformal correction. Sort the calibration scores and compute the same conservative conformal order statistic $\hat{q}_{\text{CQR}}$ used in split conformal.
4. Construct calibrated intervals. For a new observation x , define:

$$C(x) = [\hat{q}_{\alpha/2}(x) - \hat{q}_{\text{CQR}}, \hat{q}_{1-\alpha/2}(x) + \hat{q}_{\text{CQR}}]$$

CQR retains the adaptive shape of the quantile model: intervals widen where the model predicts greater conditional dispersion and narrow where the model predicts lower uncertainty. The conformal correction then calibrates the empirical coverage of these intervals. Unlike split conformal, the interval width can vary across assets, dates, regimes, and feature configurations.

CQR is often preferable when residual dispersion is visibly heterogeneous, and enough calibration data is available. For cross-sectional financial models with many assets and regime-dependent uncertainty, it is usually more informative than a fixed-width split conformal. However, CQR depends on the quality of the underlying quantile model. If the quantile estimates are unstable, the calibration sample is small, or the regime shifts sharply, rolling split conformal or ACI may be more robust.

This uncertainty estimate also has a direct **interpretation in terms of allocation**. A simple preview is to scale the position size inversely with the interval width:

$$w_i \propto \frac{1}{\text{width}(C(x_i))}$$

so that forecasts with tighter calibrated intervals receive larger allocations, all else equal. A stock with a 50 basis point interval receives twice the uncertainty-adjusted weight of one with a 100 basis point interval before applying volatility, correlation, turnover, and portfolio-level constraints. *Chapter 17* develops this idea within a full-allocation framework.



Implementation : `@6_conformal_prediction` implements CQR with native quantile models and compares its interval-width distribution with split conformal. Ridge-based results should be interpreted as a baseline comparison unless the model explicitly estimates lower and upper conditional quantiles.

Conformal prediction sets for classification

Conformal prediction also applies to classification. Instead of producing a prediction interval for a continuous return, it produces a prediction set: a subset of labels that contains the true class with probability at least $1 - \alpha$ under exchangeability.

For a classifier with predicted class probabilities $\hat{p}(y \mid x)$, a simple nonconformity score is:

$$s_i = 1 - \hat{p}(y_i \mid x_i)$$

where y_i is the true class for calibration observation i . Large scores indicate that the model assigned low probability to the realized class.

After computing the conformal quantile $\hat{q}$ from the calibration scores, the prediction set for a new observation is:

$$C(x) = \{y: 1 - \hat{p}(y | x) \leq \hat{q}\} = \{y: \hat{p}(y | x) \geq 1 - \hat{q}\}$$

The result is a set of plausible labels rather than a single forced classification. For three-class direction labels such as long, neutral, and short, a singleton set such as `{long}` conveys greater directional confidence than a wider set such as `{long,neutral}`. A full set containing all classes indicates that the classifier cannot distinguish the alternatives at the desired coverage level.

This maps naturally to trading decisions. One conservative rule is to trade only singleton prediction sets and abstain when the conformal set contains multiple labels. This can reduce exposure when the model is uncertain about direction. However, the conformal guarantee applies to the prediction sets over all evaluated observations, not automatically to the subset of observations selected for trading. Coverage, hit rate, abstention rate, turnover, and realized performance should therefore be reported separately for singleton and non-singleton cases.

Conformal classification calibrates set coverage, not the probability estimates themselves. Poorly calibrated probabilities can still produce valid conformal sets under the required assumptions, but inefficient probability estimates may lead to unnecessarily large sets. Probability calibration and conformal set calibration are related diagnostics, but they are not the same object.

Evaluating calibration quality

A conformal method should be evaluated by whether its realized coverage matches its claimed coverage and whether the resulting intervals are useful for decision-making. A nominal 90% interval that covers only 70%

of realized outcomes is overconfident: it claims a 10% failure rate but delivers a 30% failure rate. Conversely, a method that covers 99% of outcomes at a 90% target may be too conservative, producing intervals too wide to support meaningful allocation decisions.

A calibration plot summarizes this relationship. For target coverage levels:

$$1 - \alpha \in \{0.10, 0.20, \dots, 0.90\}$$

plot nominal coverage against empirical coverage. Points below the diagonal indicate overconfidence; points above the diagonal indicate conservatism.

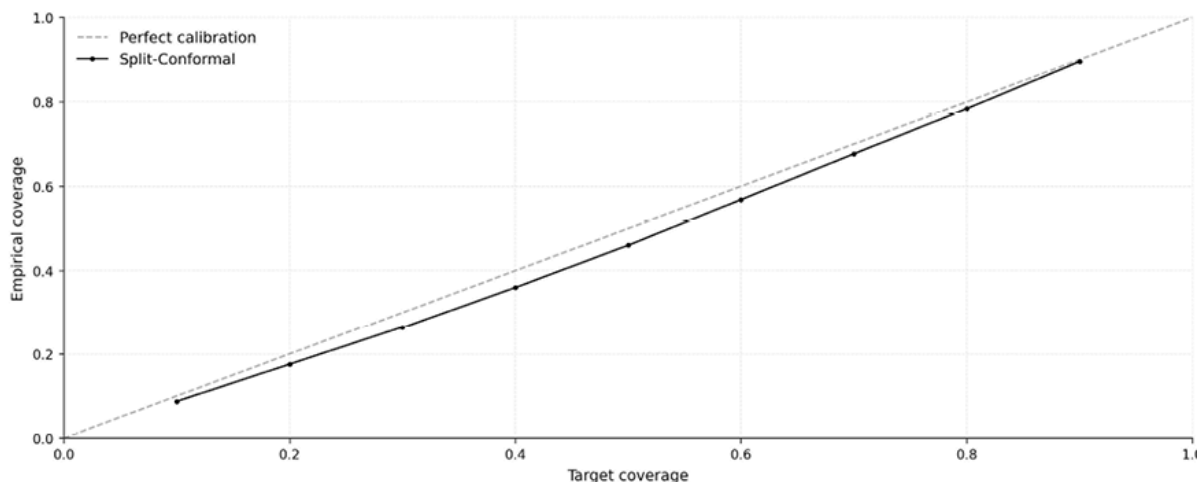


Figure 11.6: Conformal prediction calibration

Marginal coverage can hide failures that matter for trading. Conditional coverage should therefore be evaluated by stratifying observations by:

- **Walk-forward fold** , to test whether coverage is stable through time
- **Predicted return magnitude** , to test whether intervals fail in the signal tails
- **Market regime** , to test whether coverage deteriorates during high volatility or low liquidity

- **Asset characteristics** , to test whether coverage varies systematically by sector, liquidity, volatility, or instrument type

Coverage estimates should be reported with uncertainty. A realized coverage rate of 89.9% may be statistically indistinguishable from a 90% target, while 88.4% may or may not be material depending on the effective sample size, serial dependence, and cross-sectional clustering. Fold-level dispersion, block-bootstrap confidence intervals, or cluster-robust standard errors make the comparison more informative than a single pooled number.

On the `etfs` case study, conformal prediction is evaluated using daily data for 99 ETFs over 2006–2025, with out-of-sample evaluation across eight walk-forward folds spanning 2015–2023. Within each fold, the model uses an 80/20 proper-training/calibration split. The base model is Ridge regression.

Method	Target	Marginal Coverage	Mean Width (%)	Width σ (%)
Split-Conformal	90%	88.4%	16.9%	3.3
CQR	90%	89.9%	15.7%	7.5
ACI on SC base ($\gamma = 0.01$)	90%	89.5%	16.3%	6.8

Table 11.1: Conformal evaluation results

The results illustrate the trade-offs among the three methods:

- Split conformal is simple and auditable but produces fixed-width intervals and undercovers relative to the 90% target
- CQR comes closest to the target and produces the narrowest average interval, while allowing interval width to vary substantially across

observations

- ACI improves the static split-conformal baseline by adapting interval width over time, though its realized coverage remains slightly below target in this run

The width dispersion is also informative. Split conformal has relatively low width variation because all observations in a fold receive the same conformal correction. CQR has higher width dispersion because its quantile models adapt to the conditional uncertainty of each observation. ACI also produces more variable widths because recent coverage errors feed back into the effective miscoverage level.

These results are consistent with the challenges posed by nonexchangeable financial data, but they should not be mechanically attributed solely to exchangeability violations. Undercoverage can also arise from calibration leakage, overly small calibration windows, unstable quantile estimates, inappropriate score definitions, or implementation choices such as interpolated quantiles. The correct diagnostic response is to inspect coverage by fold, regime, and asset group before changing the method.

Several warning signs deserve attention. Coverage below 80% during volatility spikes, for a 90% target, indicates that the calibration distribution is stale or poorly stratified. Monotonically deteriorating coverage over the backtest suggests drift in the calibration scheme's tracking. Asymmetric violations, with many more outcomes above or below the interval, indicate a directional bias or residual skewness. Symmetric split conformal can widen intervals but cannot recenter a biased forecast. CQR can represent asymmetric conditional tails if the quantile models learn them, but the conformal correction itself calibrates coverage rather than correcting the predictive center.



Implementation : `06_conformal_prediction` shows the full calibration comparison, including marginal coverage, per-fold coverage, interval-width distributions, conditional coverage diagnostics, and ACI adaptation dynamics.

11.6 Case study insights

One regularized linear pipeline, run unchanged across all case studies, lets us ask a single question nine times: when does a linear model turn off-the-shelf features into a useful ranking of forward returns, and when does it not? Most of the time it does not. The linear model is the baseline the more flexible families in *Chapters 12* through *14* have to beat, so it matters as much where it fails as where it succeeds. Throughout, the measure of success is the average daily information coefficient (IC) between the predicted score and the realized return, read together with its 95% confidence interval (CI); all t -statistics use the Newey–West (HAC) correction. The full evidence is in `07_case_study_insights`.

The signal is real in three of the nine case studies and absent in the other six. *Table 11.4* sorts the case studies by whether the confidence interval around their IC clears zero. Three intervals exclude zero, three are positive but wide enough to contain it, and three sit below it. Reliable cross-sectional ranking from a linear model on engineered features is therefore the exception rather than the rule, even before any of the frictions of trading enter the picture.

Category	Count	Case studies
CI excludes zero	3	ETFs, US Equities, NASDAQ-100

Category	Count	Case studies
CI overlaps zero (positive)	3	Crypto, SP 500 Options, FX
CI overlaps zero (negative)	3	CME Futures, SP 500 Eq+Opt, US Firms

Table 11.4: Case studies by whether the regression IC confidence interval excludes zero

All nine case studies are scored this way on a regression label. Four of them also carry a direction label, which is a classification problem scored differently; those results are held back to the dedicated comparison near the end of the section, so that the regression picture stays clean first.

Where the signal is

ETFs, US equities, and intraday NASDAQ-100 are the three case studies whose linear models rank returns with an IC that is reliably positive.

Table 11.5 lists the best configuration, and its IC for each case study at its primary label, and *Figure 11.7* plots the same estimates so that the intervals clearing zero are visible at a glance. The prose below reads the pattern; the numbers live in the table.

Case study	Horizon	IC	t	95% CI	n
ETFs	21 days	+0.054	2.4	[+0.009, +0.098]	2,016
US Equities	1 day	+0.016	9.1	[+0.012, +0.019]	4,018
NASDAQ-100	15 min	+0.005	4.3	[+0.003, +0.007]	64,580

Case study	Horizon	IC	t	95% CI	n
Crypto	8 hours	+0.009	1.4	[−0.003, +0.020]	1,956
SP 500 Options	~30 days	+0.007	0.6	[−0.015, +0.029]	502
FX	1 day	+0.005	0.6	[−0.012, +0.022]	2,064
CME Futures	5 days	−0.000	−0.0	[−0.034, +0.034]	1,290
SP 500 Eq+Opt	5 days	−0.006	−0.4	[−0.033, +0.022]	502
US Firms	1 month	−0.005	−0.6	[−0.022, +0.011]	120

Table 11.5: Best linear configuration for the primary regression label measured on the validation set

The interval is the HAC 95% CI; n is the number of trading periods over which the daily IC is averaged. The labels correspond to the total forward return over the given horizon, except for S&P 500 options, where the label is the per-position profit and loss of an at-the-money short straddle held from formation to expiry (25–35 days), following the hold-to-maturity construction of O’Donovan and Yu (2024). See case study [README](#) for more details.

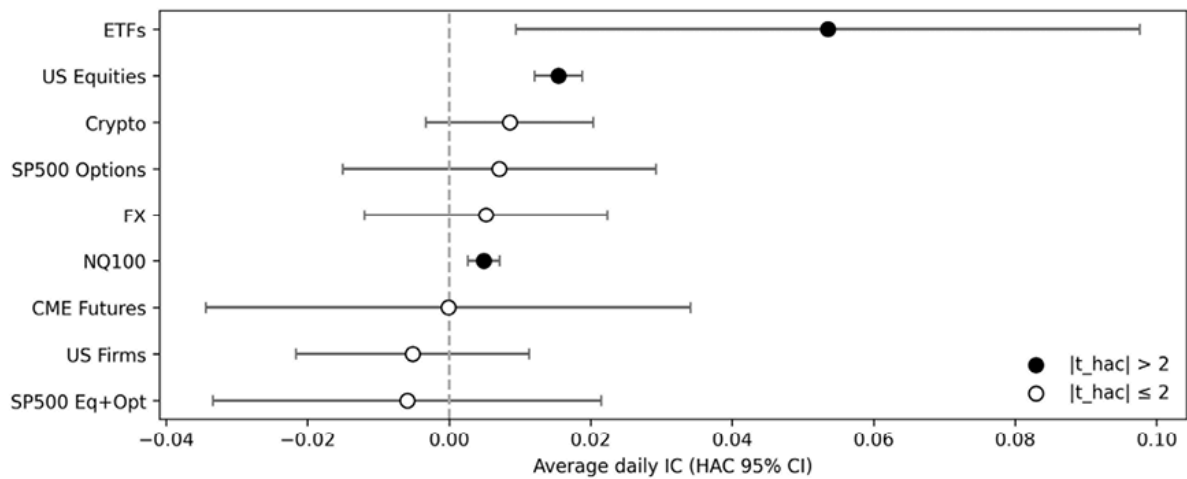


Figure 11.7: Best daily IC per case study at the primary regression label, with HAC 95% CI. Filled markers mark intervals that exclude zero; open markers mark intervals that overlap it

Two things separate these three from the rest. The first is that their features genuinely align with the cross-section of returns: momentum and value carry ranking information in equities and ETFs in ways that engineered features do not for currencies or futures. The second is statistical, not economic. Whether an IC clears zero depends as much on how many independent observations support it as on its size and variance. NASDAQ-100 reaches significance on the smallest IC of the three because its intraday history is large. Sample size, not effect size, is doing much of the work, which is worth remembering before reading a small but significant IC as a strong signal; both statistical and economic significance matter.

Choice of regularization

Each linear model is fit over a grid of regularization strengths, with the strength chosen by cross-validation, under three penalties: Ridge, which shrinks every coefficient toward zero; LASSO, which drives some coefficients to exactly zero; and Elastic Net, which blends the two. All three are fit on seven of the nine case studies.

Ridge gives the highest IC on every case study where the linear model has measurable signal. LASSO and Elastic Net match it elsewhere, and edge it on CME Futures, but the IC there is indistinguishable from zero under any penalty, so the gap is noise rather than a finding. Ridge holds up because of how the features are built: momentum at several horizons, volatility at several scales, and cross-sectional ranks are correlated by construction, so a penalty that keeps all of them and shrinks them together discards less information than one that zeroes some out. The gain from regularizing at all, relative to an unpenalized fit, is small, and it is largest exactly where the unpenalized fit lands far from the best strength. Regularization here buys numerical stability; it does not manufacture signal the features do not carry.



Ridge, LASSO, and Elastic Net are all fit on seven of the nine case studies. NASDAQ-100 and US Equities are represented by their best Ridge fit in *Table 11.5* ; LASSO and Elastic Net add nothing to the comparison on those two and are not reported.

Whether the signal persists

A positive IC averaged over the whole validation period can still come from a few good stretches, so the more demanding question is whether the signal holds up over time. The clearest way to see this is a rolling three-month average of the daily IC, which traces how ranking quality evolves rather than collapsing it to a single number. Because each case study covers a different validation window, the view is most informative for case studies with overlapping periods. *Figure 11.8* shows ETFs and FX over the same 2016–2023 span: the ETF signal spends most of the period above zero, with sharp but temporary drawdowns, while the FX

signal is split almost evenly across zero with no persistent sign. The ETF edge is a property of the whole period; the FX entry in *Table 11.5* averages a series that is positive and negative in roughly equal measure, which is why its interval includes zero.

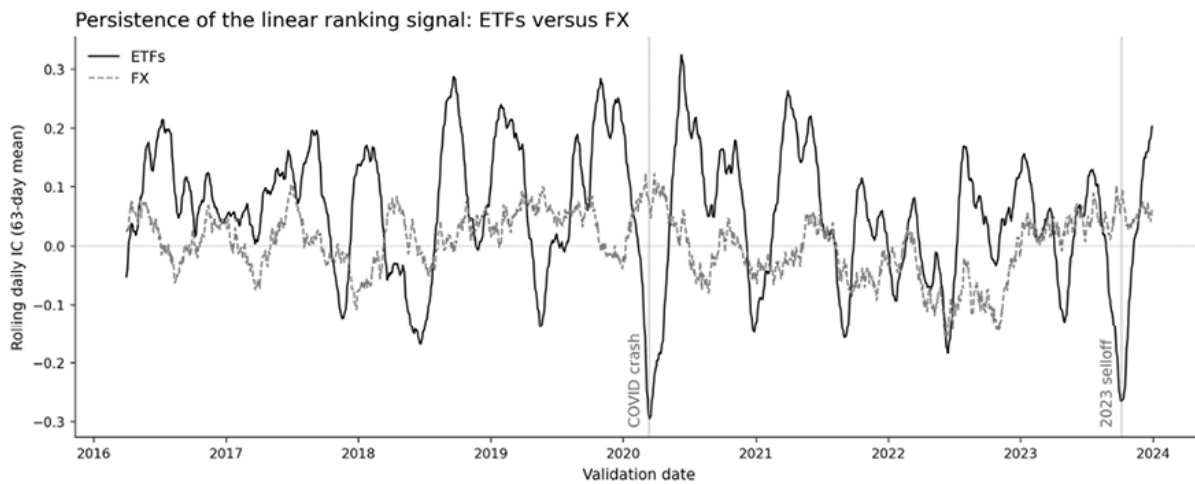


Figure 11.8: Three-month rolling average of the daily IC for the ETF and FX case studies over their common 2016–2023 validation window. The ETF series holds a positive level; the FX series crosses zero repeatedly

A second, simpler check guards against the opposite error: reading a near-zero IC as the symptom of an unstable fit rather than a genuine absence of signal. Across folds, the model keeps the sign of its coefficients in the vast majority of cases for each case study, even when the IC overlaps zero. The fits are stable; what the flat case studies lack is ranking content in the features, not a settled model.

Horizon effects

The horizon at which a label is defined affects how much signal a linear model can extract, and the effect varies by case study (*Figure 11.9*). For ETFs, US equities, and SP 500 Eq+Opt the IC rises with horizon, as the day-to-day noise averages out and a slower cross-sectional signal shows through. NASDAQ-100 is flat across its intraday horizons: a linear read of microstructure features carries the same ranking content at five, fifteen,

and sixty minutes. Crypto behaves the same way across its eight- and twenty-four-hour labels, consistent with a funding-rate signal that does not decay over a day.

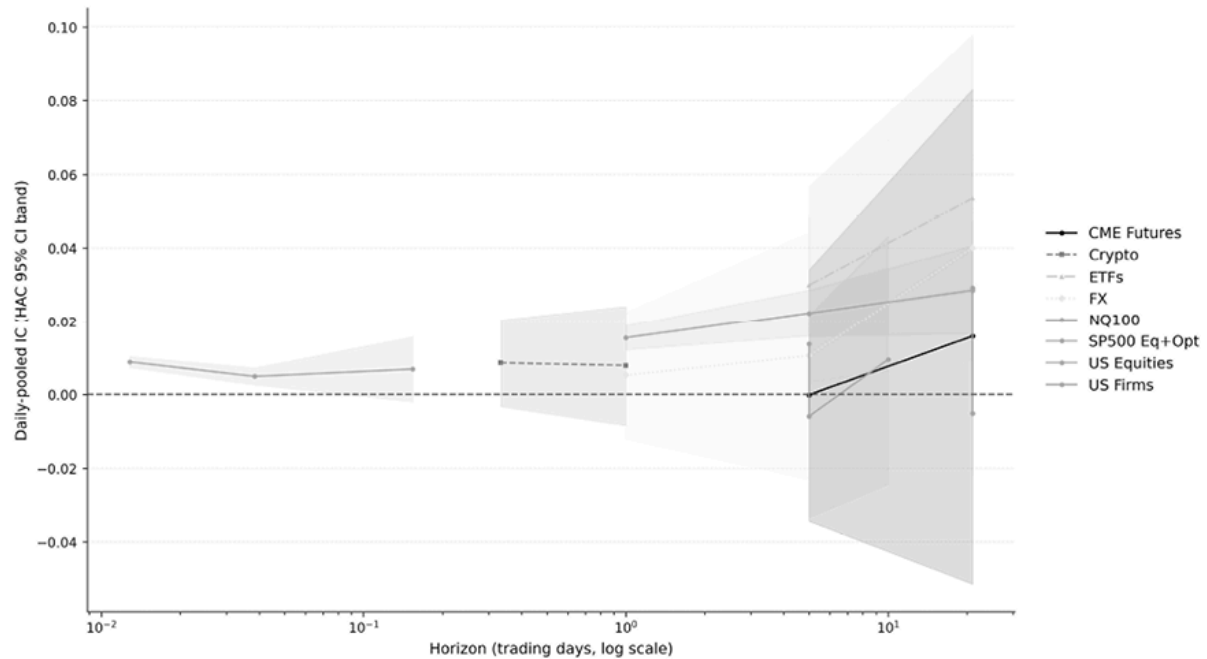


Figure 11.9: Best daily IC per case study and regression label, with HAC 95% CI bands, on a log-horizon axis. Only case studies with two or more regression labels appear

The practical reading is to match the horizon to the signal: where information accumulates slowly, a longer holding period yields a higher IC; where it lives inside the window, a shorter horizon captures as much. A single model trained jointly on several horizons can pool this structure, an idea *Chapter 14* develops in latent-factor form.

Direction versus magnitude

Predicting the sign of a return and ranking its size are different problems, and a model can be good at one while indifferent to the other. Three case studies (Crypto, SP 500 Eq+Opt, and US Firms) train both a regression model on continuous returns and a classifier on binary direction over the same horizon, allowing us to score each model on both targets. The

classifier is read by its AUC against the direction; the regression model by its IC against the return; and each is also cross-evaluated on the other's target (*Table 11.6*).

Case study	Horizon	Native AUC	Cross-IC (t)	Cross-AUC (regression)
Crypto	8 hours	0.509 [0.498, 0.521]	+0.033 (5.3)	0.498
US Firms	1 month	0.539 [0.528, 0.550]	+0.074 (6.8)	0.494
SP 500 Eq+Opt	5 days	0.510	overlaps zero	0.496
SP 500 Eq+Opt	10 days	0.507	overlaps zero	0.523

Table 11.6: Classifier and regression scores at matched horizons. Native AUC scores the classifier on direction; cross-IC scores the classifier on the continuous return; cross-AUC scores the regression model on direction

The two targets do not track each other. On Crypto and US Firms the classifier's score ranks continuous returns well, yet only on US Firms does its AUC separate direction from chance with any confidence. Running the comparison the other way, the regression score barely moves AUC away from one-half even where it ranks returns. The lesson is not that one target is better, but that direction and magnitude encode different information, so choosing which to predict is a modeling decision rather than a formality.

From IC to profitability

A higher IC does not guarantee a more profitable strategy because realized returns also depend on turnover, transaction costs, and position sizing. `08_ml_backtest_intro` makes the gap concrete on the ETF case study, where a simple 126-day momentum signal carries a lower IC than Ridge yet matches or beats it on net Sharpe ratio, because the Ridge portfolio trades far more to act on its scores. Turning a ranking into a return is the subject of *Chapters 16 through 19* : strategy simulation, portfolio construction under turnover constraints, transaction-cost modeling, and risk management.

11.7 Summary

This chapter established the end-to-end ML pipeline for translating alpha factors into trading signals. Regularized regression (Ridge, LASSO, and Elastic Net) provides the first rung of the baseline ladder: interpretable models whose coefficients reveal which features the model relies on and how strongly. Logistic regression extends the framework to direction prediction, offering an alternative when continuous return signals are weak. SHAP attributions verify that the model's reasoning aligns with economic intuition and remains stable across walk-forward folds. Conformal prediction wraps any base model in calibrated uncertainty bands, enabling position sizing that reflects prediction confidence rather than treating all forecasts as equally reliable.

The cross-dataset evaluation in *Section 11.6* reveals where linear models work and where they do not. These results define the baseline for *Chapter 12* (gradient boosting), *Chapter 13* (deep learning), and *Chapter 14* (latent factors), which test whether nonlinear models can extract signal on the datasets where the linear baseline runs out.

12

Advanced Models for Tabular Data

Gradient Boosting Machines (GBMs) dominate tabular prediction in both academic research and production trading systems, combining predictive power with computational efficiency. While *Chapter 11* 's linear models assume additive feature relationships, GBMs capture the interaction effects and threshold dynamics that financial data exhibit, without manual feature engineering.

This chapter covers the three industry-standard GBM libraries (XGBoost, LightGBM, and CatBoost) and the techniques that make them production-ready. By the end of this chapter, you will be able to:

- Explain how boosting differs from bagging and why sequential error correction makes GBMs effective for financial tabular prediction.
- Select among XGBoost, LightGBM, and CatBoost based on categorical structure, compute environment, latency needs, and leakage risk.
- Choose appropriate GBM objectives and constraints for financial tasks, including pointwise regression, learning to rank, and monotonic constraints.
- Tune GBMs efficiently with Optuna using pruning, multi-objective search, and time-series-aware validation.
- Use TreeSHAP to analyze feature effects, interactions, instability, and drift in deployed tree-based models.

- Evaluate when tabular deep learning alternatives such as TabPFN, TabM, and TabR are worth considering relative to GBMs.
- Interpret cross-case-study evidence to decide when nonlinear tree models earn their added complexity relative to linear baselines

We progress from the shared boosting framework through Bayesian hyperparameter optimization with Optuna, TreeSHAP-based interpretation and drift monitoring, and learning-to-rank objectives for quantile strategies. We then evaluate deep learning alternatives (TabPFN and TabM) using decision criteria for when added complexity pays off, and conclude with a cross-case study comparison that benchmarks GBMs against the linear baselines from *Chapter 11* .

12.1 From decision trees to ensembles

Gradient boosting rests on three building blocks: how decision trees partition feature space, how Random Forests reduce variance by averaging independent trees, and why averaging alone cannot correct systematic bias.

A **decision tree** makes predictions by recursively partitioning the feature space into regions. At each node, the algorithm selects the feature and threshold that best separate the data, typically using information gain for classification or variance reduction for regression.

Decision trees handle mixed feature types (continuous returns, categorical sectors) without encoding, capture non-linear relationships through hierarchical splits, and need no feature scaling. Their critical weakness is high variance: a small change in training data can produce a dramatically different tree structure, making individual predictions unstable. Yet individual trees can learn that momentum is predictive only when

volatility is below a threshold, an interaction that linear models require explicit engineering to capture. The question is how to harvest this flexibility without the instability.

Random forests - Reducing variance with bagging

Random forests (Breiman, 2001) apply the bagging principle to decision trees with an additional twist: at each split, only a random subset of features is considered. This feature randomization decorrelates the trees, making the variance reduction from averaging more effective.

For a regression task, a Random Forest prediction is simply the average of all individual tree predictions:

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M T_m(x)$$

where M is the number of trees and $T_m(x)$ is the prediction from tree m .

Random forests require minimal hyperparameter tuning and rarely overfit, even with many trees. In the model comparison notebooks for this chapter, Random Forests serve as a baseline whose accuracy GBMs must materially improve upon to justify their additional complexity. *Figure 12.1* illustrates the contrast between bagging (averaging independent trees) and boosting (sequential error correction).

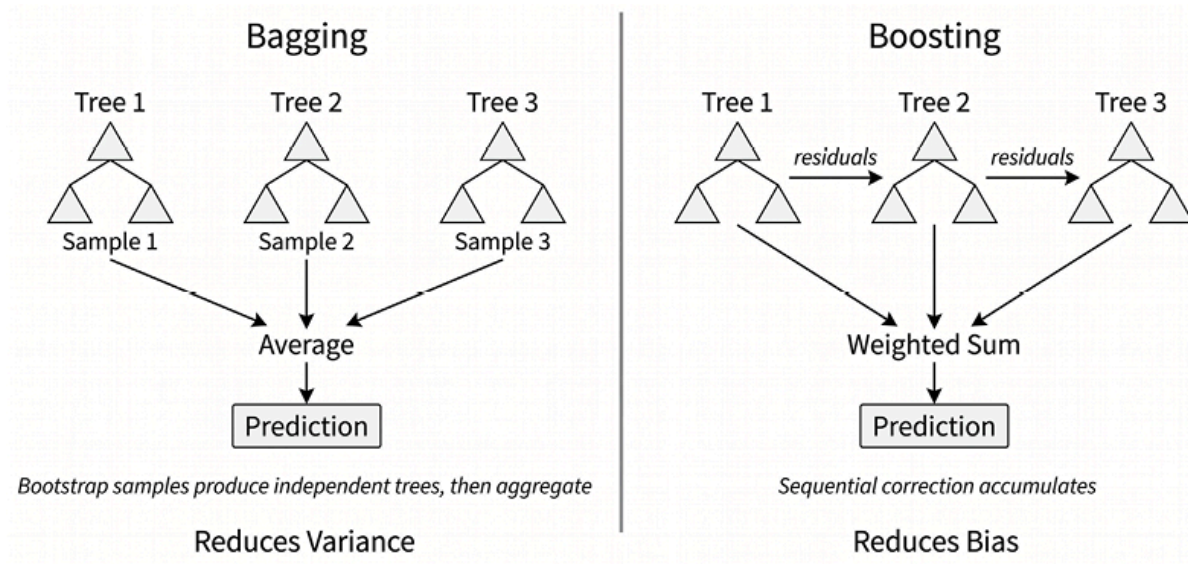


Figure 12.1: Bagging versus boosting

However, because trees are trained independently, they cannot correct each other's systematic errors. If all trees share a consistent bias (for example, underestimating extreme returns), averaging does not help. This limitation motivates the boosting paradigm covered next.

The path to boosting

Boosting builds models sequentially, so each new model targets the errors of the current ensemble. If the current prediction is 2% but the true return is 5%, the next tree learns to predict the 3% residual. Adding that correction to the ensemble gradually eliminates systematic under-prediction, the bias that averaging independent trees cannot address.

This sequential error correction, combined with shrinkage and regularization, forms the foundation of gradient boosting machines, which we detail in the next section. The notebook

`01_ensemble_foundations` benchmarks Random Forests against XGBoost, LightGBM, and CatBoost on the Chen–Pelger–Zhu firm characteristics dataset (414K train/307K valid/497K test observations across 1967–2016) and finds a consistent ordering on this benchmark:

each GBM achieves higher out-of-sample test IC than the Random Forest baseline (0.058–0.060 versus 0.056), with library-to-library differences (~0.002 IC) smaller than the boosting-versus-bagging gap.

12.2 Gradient boosting machines

The three dominant GBM libraries (XGBoost, LightGBM, and CatBoost) share the core boosting algorithm but differ in tree construction, categorical handling, and regularization. This section covers the shared framework, then examines each library’s distinctive innovations.

High performance with boosting

Gradient boosting, formalized by Friedman (2001), frames model training as iterative optimization in function space. Rather than fitting a single complex model, we construct an additive ensemble in which each new component explicitly targets the errors in the current prediction.

The algorithm proceeds as follows:

1. Initialize with a constant prediction (typically the target mean).
2. For each boosting iteration $m = 1, 2, \dots, M$:
 - a. Compute the negative gradient of the loss function with respect to the current predictions.
 - b. Fit a weak learner (typically a shallow tree) to these “pseudo-residuals.”
 - c. Add the new tree’s predictions to the ensemble, scaled by a learning rate η .

The update at each iteration is:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

where h_m is the tree fitted in iteration m . The final GBM prediction after M boosting rounds can be written as:

$$\hat{y} = F_M(x) = F_0(x) + \eta \sum_{m=1}^M h_m(x)$$

where $F_0(x)$ is the initial constant prediction, often the training-set mean for squared-error regression, and each $h_m(x)$ is a shallow tree fitted to the loss gradient implied by the current ensemble. This expression highlights the key difference from the Random Forest prediction above. A Random Forest averages independently trained trees, whereas a GBM starts with a baseline prediction and iteratively adds targeted corrections. Each tree is therefore not a standalone forecast of $\hat{y}$, but an incremental adjustment to the current prediction. For classification, the same additive structure applies to the model's raw score scale, such as log-odds for binary classification, before applying the appropriate link function to obtain probabilities.

The **learning rate** η (typically 0.01–0.3) controls how aggressively each tree's contribution is added. Smaller values require more trees but often yield better generalization: a GBM with 100 trees is not averaging 100 predictions but accumulating 100 incremental corrections.

The **choice of loss function** determines what “errors” the algorithm targets. In *Chapter 11*, we discussed mean squared error, mean absolute error, and Huber loss:

- **Quantile loss** predicts specific quantiles rather than the conditional mean, enabling direct estimation of prediction intervals.
- All three GBM libraries support **custom objectives**, enabling asymmetric losses (for example, penalizing under-prediction more than over-prediction) as long as you can provide stable first and

second derivatives with respect to the model's predictions (or use a smooth approximation when the loss is non-smooth).

The next section explains why this blend of flexibility and regularization gives GBMs such a strong track record on tabular data.

Why GBMs excel on tabular data

Gradient boosted decision trees have an **inductive bias** well-matched to financial tabular data:

- Financial datasets mix continuous variables (returns, volatility), categorical features (sector, exchange), and derived indicators. Tree-based methods accommodate **heterogeneous tabular inputs** well: continuous variables can be split by thresholds, while categorical variables are handled through library-specific encodings or native categorical split mechanisms. The details differ materially across libraries.
- Financial relationships often exhibit **discontinuities** (a P/E ratio above a threshold may trigger fundamentally different investor behavior than just below it), and trees capture these non-smooth boundaries directly. Unlike neural networks that impose smoothness through continuous activation functions, trees partition the feature space into discrete regions. Trees also discover feature interactions through hierarchical splits: a sufficiently deep tree learns that “high momentum is predictive only when volatility is moderate” without manual interaction engineering.

Several practical advantages compound this expressive power:

- Boosted trees rely on **threshold splits**, so they are typically more robust to extreme feature values than models that depend directly on magnitudes or distances

- Modern GBM implementations **handle missing values** by learning the optimal split direction for missing features, rather than requiring imputation
- Trees can also operate on raw feature values without normalization, eliminating a class of preprocessing bugs

The next section examines the most widely used libraries that implement these ideas: XGBoost, LightGBM, and CatBoost.

Inside the engines - XGBoost, LightGBM, and CatBoost

XGBoost, LightGBM, and CatBoost share the gradient boosting framework but differ in key algorithmic innovations that affect their performance characteristics.

XGBoost - Regularization and sparsity

XGBoost (eXtreme Gradient Boosting), introduced by Chen and Guestrin (2016), brought several innovations that shaped current gradient boosting practice.

Regularized objective : Unlike earlier implementations that relied primarily on early stopping and shrinkage, XGBoost explicitly adds regularization terms to the objective function:

$$\mathcal{L} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

where:

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

penalizes both tree complexity and the magnitude of the model's leaf scores. Here, T is the number of leaves in tree k , w_j is the prediction assigned to leaf j , γ controls the penalty for adding additional leaves, and λ controls the L2 penalty on leaf weights. Larger values of γ encourage simpler trees, while larger values of λ shrink leaf predictions toward zero. This regularization directly limits overfitting rather than relying only on heuristic stopping rules:

- **Sparsity-aware split finding** : Real-world data often contains extensive missing values or zero entries. XGBoost learns a default direction for each tree node: when a sample has a missing value for the split feature, it follows this learned default path. This eliminates the need for imputation while often improving predictive performance.
- **Second-order approximation** : XGBoost uses a second-order Taylor expansion of the loss function, incorporating both the gradients and the Hessian, which contains the second derivatives. This provides more accurate updates than first-order methods, particularly for loss functions with significant curvature.
- **Cache-aware access patterns** : XGBoost optimizes how data is laid out and accessed in memory to make repeated computations better use of the CPU cache. This improves training speed, especially on large tabular datasets where inefficient memory access can become a bottleneck.

LightGBM - Speed through sampling and bundling

LightGBM (Ke et al., 2017), developed by Microsoft, focuses on dramatically improving training speed while maintaining or improving accuracy. On large, sparse datasets, LightGBM used to train 5–10× faster

than pre-histogram XGBoost, but the CPU speed advantage has narrowed as XGBoost has adopted the histogram mode introduced by LightGBM, which added several innovations:

- **Histogram-based learning** : Instead of sorting feature values for exact split finding, LightGBM discretizes continuous features into histograms. Split finding then examines only histogram bin boundaries. Histogram construction still requires scanning the data, but once the histogram is built, split gain can be evaluated over bin boundaries rather than all distinct feature values. The split-evaluation step, therefore, scales with the number of bins, which is typically far smaller than the number of observations.
- **Leaf-wise tree growth** : LightGBM is best known for leaf-wise, best-first tree growth. At each step, it splits the leaf with the largest loss reduction. This can reduce training loss faster with a fixed number of leaves than depth-wise growth does, but it also increases the risk of overfitting on small or noisy datasets unless constrained by `num_leaves` , `max_depth` , `min_data_in_leaf` , or related regularization parameters.
- **Gradient-based One-Side Sampling (GOSS)** : The key insight is that samples with large gradients (those that the model is currently getting wrong) contribute more to information gain than samples with small gradients (those that are already well-predicted). GOSS keeps all large-gradient samples but randomly samples only a fraction of small-gradient samples (and reweights them to preserve a good estimate of split quality), significantly reducing computational cost without sacrificing split quality.
- **Exclusive Feature Bundling (EFB)** : In high-dimensional sparse data (common after one-hot encoding categorical variables), many features are mutually exclusive: they never take non-zero values simultaneously. EFB identifies and bundles such features into

combined features, reducing the effective number of features to evaluate when finding splits.

CatBoost - State-of-the-art categorical handling

CatBoost (Prokhorenkova et al., 2018), developed by Yandex, addresses a critical challenge: **how to handle categorical features without introducing target leakage** .

Standard target encoding (replacing a category with the mean target value of samples in that category) creates a subtle form of data leakage. When computing the encoded value for a training sample, that sample's target value influences its feature value, inflating the model's apparent performance.

CatBoost's solution is “ **ordered target statistics** .” For each training sample, target statistics are computed using only samples that appear earlier in a random permutation of the data. This ensures no sample's target value influences its own features. Multiple permutations are used and averaged to reduce variance.

Unlike XGBoost and LightGBM, where each node can split on a different feature, CatBoost uses symmetric, or “**oblivious**” **decision trees** , by default, where all nodes at the same depth use the same split condition. This simplicity enables extremely fast prediction (often the fastest inference among the three libraries) because the prediction path can be computed as a bitwise operation rather than a sequential tree traversal. This makes CatBoost particularly well-suited for latency-sensitive applications where inference speed matters.

While one-hot encoding becomes impractical for high-cardinality categoricals with thousands of unique values, CatBoost's ordered encoding naturally handles any cardinality. Features such as individual

stock identifiers, analyst IDs, and detailed sector codes can be used directly.

Practical comparison - Which GBM to choose?

Scikit-learn's `HistGradientBoostingRegressor` provides a no-dependency baseline. It uses the same histogram-based splitting as the specialized libraries and is often the fastest option on CPU for small-to-medium datasets. However, it lacks GPU support and the tuning ecosystem (Optuna callbacks and early stopping flexibility) that specialized libraries provide. We include it in the `02_gbm_comparison` benchmark alongside the three specialized libraries.

Table 12.1 summarizes the library selection decision points. The specialized libraries still differ in what they optimize for. XGBoost is the most general-purpose option of the three: it supports both CPU and CUDA-based GPU training, includes native categorical support, and remains a strong default when you want one library that covers a wide range of production and research settings. The choice between XGBoost, LightGBM, and CatBoost depends on your specific constraints and data characteristics:

Criterion	XGBoost	LightGBM	CatBoost
CPU training (4.9M rows, 8 threads)	~100s	~50s — fastest (~2× XGBoost)	~135s — slowest
GPU training (4.9M rows, CUDA)	~14s (~7× its CPU)	~14s (~3.5×; FP64-only, see below)	~10s (~14×) — fastest

Criterion	XGBoost	LightGBM	CatBoost
Prediction speed (heavy preset, 24K test rows)	0.5M rows/s CPU; 1.7M rows/s GPU	0.07M rows/s CPU (slowest)	6.5M rows/s CPU; symmetric trees → bitwise path, ~12× faster than XGBoost on CPU
High-cardinality categoricals	Native support	Native support	Ordered target statistics avoid leakage
Default performance	Good	Good	Often strong, especially with many categorical variables
Tuning sensitivity	Moderate	Moderate-high (leaf-wise growth requires depth/leaf constraints)	Often lower, but data-dependent
Memory efficiency	Moderate	Often best (histogram bins reduce memory)	Moderate

Table 12.1: GBM library characteristics

The `02_gbm_comparison` notebook benchmarks all four libraries on CPU and GPU with light, medium, and heavy presets, measuring accuracy, training time, and memory usage.

LightGBM remains especially attractive in CPU-first tabular workflows. Its leaf-wise tree growth can reduce training loss quickly, but it also makes the library somewhat more sensitive to parameters such as

`num_leaves` , `min_data_in_leaf` , and sometimes `max_depth`

All three libraries support **GPU-accelerated training** , but the speedup depends on dataset size and numerical precision:

- On the **US equities panel** (4.92M training rows, 72 features, 8 threads), CatBoost achieves the largest GPU speedup at $\sim 14\times$ with `task_type='GPU'` ($\sim 135\text{s CPU} \rightarrow \sim 10\text{s GPU}$), and XGBoost $\sim 7\times$ using `device='cuda'` ($\sim 100\text{s} \rightarrow \sim 14\text{s}$). LightGBM trains in $\sim 50\text{s}$ on CPU (the fastest CPU configuration in the scale benchmark) and $\sim 14\text{s}$ on GPU with `device='cuda'` , a more modest $\sim 3.5\times$ speedup; its GPU wall time matches XGBoost's but not CatBoost's, for the FP64 reason discussed below.
- On the smaller **ETF dataset** (227K rows), launch overhead compresses the speedups: for XGBoost and CatBoost the gain ranges from $\sim 2.4\times$ (XGBoost light) to $\sim 5.2\times$ (CatBoost heavy). LightGBM is the exception: its FP64 CUDA path is counterproductive at this scale, training *slower* on GPU than on CPU (about $\sim 9.5\text{s CPU}$ versus $\sim 48\text{s GPU}$ on the heavy preset), so LightGBM should stay on CPU for datasets this size. For XGBoost and CatBoost the GPU still helps, but the relative advantage is roughly an order of magnitude smaller than at the 4.9M-row scale.

LightGBM's CUDA speedup is modest even though its GPU wall time ($\sim 14\text{s}$ at the 4.9M-row scale) matches XGBoost's: its **CUDA backend uses double precision (FP64) exclusively** : the `gpu_use_dp` toggle that enables single-precision training “can be used only in OpenCL implementation (`device_type='gpu'`), in CUDA implementation only double precision is currently supported.” Note that:

- **Consumer GPUs** have an FP32:FP64 throughput ratio of roughly 64:1 (the RTX 3090 delivers 35.6 TFLOPS FP32 but only ~0.6 TFLOPS FP64), so LightGBM's GPU histogram computation runs at a fraction of the available throughput.
- **Data center GPUs** with better FP64 ratios (A100: 2:1) would show a different result.

LightGBM also offers a separate OpenCL backend (`device='gpu'`) that defaults to single precision, but it requires an OpenCL SDK and produces slightly different results due to the precision difference. The CUDA build requires compilation from source with `USE_CUDA=ON` , which is the build that produces the GPU times reported here; CUDA-enabled conda packages are now available on supported systems.

The practical picture at scale is therefore:

- **CatBoost GPU delivers the largest training speedup** (~14× at the 4.9M-row scale), making it the clear choice when GPU-accelerated training time matters. At that scale CatBoost finishes GPU training in ~10s, with XGBoost and LightGBM together at ~14s. On CPU, LightGBM is the fastest at ~50s, the fastest CPU configuration in the scale benchmark.
- CPU times at the 4.9M-row scale (8 threads) spread from LightGBM (~50s) to XGBoost (~100s) to CatBoost (~135s); LightGBM's histogram plus leaf-wise growth has the best parallel efficiency at this thread count and dataset size.

In practice, the accuracy gap between libraries is smaller than the gap between good and bad hyperparameter configurations of any single library. The four libraries fall within a narrow IC band after tuning, and their fold-level rankings vary over time.

The real selection criteria are therefore operational: CatBoost's GPU backend delivers the fastest training at scale, CatBoost's ordered

encoding eliminates subtle leakage bugs when working with sector codes or analyst identifiers, and XGBoost's mature ecosystem provides the best integration with SHAP, Optuna callbacks, and deployment tooling. On CPU at the 4.9M-row scale, LightGBM is the fastest of the three (roughly $2\times$ XGBoost and $2.7\times$ CatBoost at 8 threads). For readers with CUDA-capable GPUs and large datasets, CatBoost's GPU acceleration offers the most substantial training-time savings.

This book's nine **case study pipelines use LightGBM** as the default library because its CPU speed is particularly advantageous when running walk-forward cross-validation across multiple hyperparameter configurations. Readers with access to **CUDA-capable GPUs** should consider switching to **XGBoost** for the larger case studies (US equities, S&P 500 equity-option analytics) where GPU acceleration provides meaningful time savings.

Native feature importance and its limitations

Every GBM library reports feature importance after training: typically either the total gain from splits on each feature or the number of times each feature is used for splitting. These metrics are fast to compute and provide a quick first look at which features the model relies on. However, they have well-documented failure modes:

- **Gain-based importance** is biased toward high-cardinality features because more unique values create more candidate splits.
- **Split-count importance** is biased toward continuous features over categoricals for the same reason.

Both metrics are *unstable* : retraining with a different random seed can substantially change importance rankings, especially for correlated

features where the model can substitute one for another.

These limitations motivate the SHAP-based analysis in *Section 12.5* , which provides consistent, theoretically grounded attributions. Use native importance for quick screening during development and SHAP for any analysis that informs decisions: feature selection, model monitoring, or risk committee reporting.

Learning to rank

Standard GBM training minimizes pointwise losses (MSE or log-loss), treating each prediction independently. But a long-short portfolio that buys the top decile and sells the bottom profits from correct *ranking* , not accurate magnitude. When MSE is dominated by large absolute errors in the middle of the distribution, the model sacrifices ranking accuracy in the tails where trading actually occurs.

Learning-to-Rank (LTR) objectives directly optimize ranking quality. LightGBM supports the `lambdarank` objective, which implements LambdaMART, a gradient boosting algorithm where the loss function is defined implicitly through the gradients (the “lambdas”) that would improve pairwise ordering. Each gradient accounts for the change in a ranking metric when two samples swap positions, weighted by the metric’s sensitivity at that position. The result is a model trained to produce well-ordered predictions rather than accurate point estimates.

The standard target metric is **Normalized Discounted Cumulative Gain (NDCG)** , which assigns exponentially higher value to correctly ranking items near the top of the list. This aligns naturally with quantile strategies: correctly identifying the best and worst stocks matters far more than distinguishing between the 40th and 50th percentiles.

When LTR outperforms pointwise regression

LTR provides the largest gains in high-breadth cross-sections where only the tails are traded. Consider a universe of 500 stocks ranked monthly: a long-short decile strategy trades only the top and bottom 50 names. MSE-trained models optimize prediction accuracy across all 500 stocks equally, including the 400 in the middle that generate no trading signal.

LambdaMART concentrates its fitting capacity on correctly separating the extremes.

The advantage diminishes for strategies that trade the full cross-section (for example, rank-weighted portfolios) or for low-breadth universes where most assets enter positions. It also diminishes when the signal-to-noise ratio is so low that ranking and pointwise objectives converge to the same solution.

A practical consideration: LTR models produce scores that are well-ordered but not calibrated to return magnitudes. If downstream portfolio construction requires return forecasts (for example, mean-variance optimization in *Chapter 17*), either calibrate the LTR scores post-hoc or use pointwise predictions. For strategies that simply go long the top quintile and short the bottom quintile, uncalibrated ranking scores suffice.

A worked example - LambdaMART versus Pointwise MSE

In the notebook `02_gbm_comparison`, we compare LambdaMART against standard MSE regression on the ETF dataset using the same feature set and train/test fold. The key configuration difference is minimal:

- **Pointwise** : `objective='regression'`, optimizing MSE
- **LTR** : `objective='lamdarank'`, with `label_gain` mapping within-date return quintiles to relevance grades and `ndcg_eval_at=[10]` targeting top-of-rank quality

Both models produce ranked predictions, but they differ in where they spend fitting capacity. The notebook evaluates mean group-wise NDCG@10 (ranking quality by date) alongside IC against raw returns. This makes the objective–metric alignment explicit: LambdaMART is optimized for ordering, while pointwise regression is optimized for magnitude fit. For tail-trading strategies, ranking quality is usually the more relevant diagnostic.

LightGBM's `lambdarank` objective requires grouping samples by cross-section (date), specified via the `group` parameter. Each group defines the set of assets ranked against each other. XGBoost provides equivalent functionality through its `rank:ndcg` objective. CatBoost supports ranking via `YetiRank` and `YetiRankPairwise` objectives, which use their own gradient formulations optimized for CatBoost's symmetric tree architecture.

The worked example is implemented in `02_gbm_comparison`, including date-group construction, relevance labeling, LambdaMART training, and NDCG@10 comparison against a regression baseline.

Enforcing economic logic with monotonic constraints

While GBMs excel at discovering non-linearities, they are blind to economic theory. A purely data-driven model might learn that higher P/E ratios generally predict lower returns, but due to noise in specific training regions, it might predict a spike in returns for extremely high P/E values, almost certainly an artifact rather than a structural signal.

Monotonic constraints enforce directional relationships: a positive constraint (+1) requires that the prediction not decrease as the feature increases; a negative constraint (−1) requires the opposite.

Monotonic constraints serve as theory-driven regularization: in low-signal-to-noise environments, enforcing a sign (for example, higher-value scores predict higher returns) prevents the model from fitting non-monotonic artifacts in noisy regions. The constraint also improves deployability (risk committees accept strategies with directional logic they can audit) and robustness to regime shifts, because the model cannot learn complex, non-monotonic patterns that are unlikely to repeat.

All three libraries support monotonic constraints via a parameter-mapping feature that maps names to their intended directions.

SHAP dependence plots (*Section 11.4*) make the effect of constraints immediately visible. An unconstrained model's dependence plot for a value feature might show a generally downward slope with local reversals, regions where higher value scores paradoxically predict lower returns. The same model with a negative monotonic constraint produces a strictly monotone dependence curve, eliminating these artifacts while preserving the overall relationship. When the constrained model's IC is comparable to the unconstrained version (as it typically is for economically motivated constraints), the constraint is acting as regularization rather than information loss. This provides a concrete implementation of the economic sanity-checking framework in *Section 11.4* .

See `02_gbm_comparison` for constrained versus unconstrained SHAP dependence plots on the ETF case study.

We now turn to a more recent addition: deep learning models for tabular data.

12.3 Deep learning alternatives for tabular data

GBMs remain the defensible default for tabular financial data, but the landscape shifted materially between 2024 and 2026. Tabular foundation models, led by TabPFN’s Nature publication (Hollmann et al., 2025), moved from research curiosity to frontier baseline, matching or exceeding tuned GBMs on small-to-medium datasets without task-specific training. Parameter-efficient neural ensembles like TabM have shown that well-engineered MLPs, rather than exotic architectures, can compete with GBMs when given equivalent tuning budgets. Retrieval-augmented models like TabR showed that hybridizing deep learning with nearest-neighbor lookups improves predictions when local similarity matters.

The **TabArena** living benchmark (Erickson et al., 2025), evaluating 16 model families across 51 curated datasets, formalized a key finding: with sufficient tuning and ensembling, deep learning can match or exceed GBMs, but the practical winner depends on dataset size, feature composition, and compute budget.

For finance, a critical caveat applies: most major tabular benchmarks assume IID splits and exclude temporal dependence. The *TabReD* benchmark (Rubachev et al., 2024), which tested models under temporal distribution shift, found that attention-heavy architectures degraded faster than GBMs and simple MLPs, directly relevant to non-stationary financial data. Benchmark wins do not imply trading profitability without walk-forward validation.

Tabular foundation models

The most conceptually striking development is the rise of **tabular foundation models (TFMs)** that use **in-context learning (ICL)**. Unlike conventional models that train on a specific dataset via gradient descent, a TFM is pre-trained once on millions of synthetic datasets generated from structural causal models. At inference time, it uses the

training data as context to produce predictions for test samples in a single forward pass: no gradient updates, no hyperparameter tuning on the target dataset. The model has already learned *how to learn* from tabular structure during pre-training.

The interface is deceptively simple: given `(x_train, y_train, x_test)`, the model returns predictions immediately. Training is replaced by in-context computation, where the transformer's attention mechanism identifies patterns in the provided context that generalize to query samples.

TabPFN pioneered this paradigm for tabular data. The original version (Hollmann et al., *Nature*, 2025) validated the approach on curated benchmarks: datasets of up to 10,000 samples and 500 features, where the default TabPFN outperformed tuned baselines without any task-specific optimization. **TabPFN v2.5** (Grinsztajn et al., 2025) substantially expanded the practical envelope (to approximately 50,000 rows and 2,000 features) through sketching and feature selection. A separately released distillation tooling path converts the TFM's predictions into compact MLPs or tree ensembles for production deployment, addressing the inference latency bottleneck that limited earlier versions to batch-only workflows.

There are several known **limitations for financial applications**. TabPFN performs best on numeric-only data and degrades on purely categorical datasets (Ye et al., 2024). IID pretraining on synthetic data does not capture temporal dependence or regime shifts. Per-sample inference latency exceeds that of optimized tree libraries at high throughput, though TabPFN 2.5's distillation path mitigates this in batch workflows.

Calibration depends on similarity between the target data structure and the synthetic pretraining distribution; structural breaks absent from the synthetic priors may degrade zero-shot predictions.

There is a practical role that TabPFN can play. For financial ML, TabPFN’s strongest use case is **rapid prototyping** : test whether a feature set contains predictive signal before investing in GBM tuning. If TabPFN achieves non-trivial IC on a subsample, the features warrant deeper investigation with production-grade models. The distillation engine now makes deployment feasible by converting the TFM’s predictions into a tree ensemble that inherits the foundation model’s accuracy while achieving GBM-class inference speed. TabPFN 2.5 weights are released under a non-commercial license by default; readers working in production trading systems should treat them as a research prototyping tool unless a commercial license is secured.

The notebook `03_dl_vs_gbm` includes TabPFN as an optional model in the walk-forward comparison; if installed, it runs alongside GBMs and TabM on the ETF case study with per-fold IC reporting.

Modern neural baselines - TabM and strong defaults

A quieter but arguably more impactful development than foundation models is the rehabilitation of simple neural architectures through better training recipes and engineering.

Holzmüller et al. (2024) showed that **meta-tuned MLP defaults** (training recipes optimized across hundreds of datasets) can make neural networks competitive with GBMs without any architectural novelty. Their **RealMLP** achieves a strong time-accuracy trade-off on medium-to-large datasets by improving defaults rather than inventing new layers. The implication is that many older “DL loses to trees” conclusions reflected weak neural baselines rather than fundamental architectural limits.

TabM (Gorishniy et al., 2025) takes this further with parameter-efficient ensembling. Rather than training M independent MLPs (which is computationally prohibitive for large financial datasets), TabM maintains a shared weight backbone and applies rank-1 adapters (scaling parameters) to create M diverse ensemble members, all trained simultaneously in a single forward pass. Each individual member's predictions are noisy: the rank-1 perturbations introduce enough diversity that individual members overfit in different directions. Averaging cancels these uncorrelated errors while preserving the shared signal, achieving the generalization benefits of deep ensembles at the computational cost of a single network.

In evaluations across 46 benchmarks, TabM achieved the strongest performance among deep learning models, including on domain-aware temporal splits, which are historically challenging for neural networks. This temporal robustness is directly relevant to financial applications in which the training distribution shifts across walk-forward folds.

TabM represents the most practical case for “deep learning a practitioner can actually use on tabular financial data.” It is architecturally simple, well-documented, and competitive without transformer-scale complexity. Interpretability uses standard model-agnostic methods (permutation importance and KernelSHAP) at a moderate computational cost. Training is slower than GBMs but does not require the multi-GPU infrastructure of large transformer models.

The notebook `03_dl_vs_gbm` compares LightGBM, a vanilla MLP, and a TabM rank-1 adapter ensemble on the ETF case study across walk-forward folds, reporting per-fold IC and training time.

Retrieval-augmented models - TabR

TabR (Gorishniy et al., 2024) hybridizes a deep encoder with k -nearest-neighbor retrieval. At prediction time, the model retrieves similar training instances from a learned embedding space and uses an attention-like mechanism to extract signals from their features and labels. The retrieved neighbors effectively provide local context that the base model can exploit, a principled way to help neural networks with the kind of local pattern recognition that trees perform naturally through hierarchical splitting.

The conceptual appeal for finance is immediate: “these neighbors drove the prediction” aligns with intuitions about regime similarity and peer-relative valuation. If the current market environment resembles a historical period, retrieval surfaces that analogy explicitly rather than requiring the model to encode it implicitly in fixed weights.

Benchmark evaluations show that TabR consistently outperforms non-retrieval deep baselines, and it is included as a first-class method in modern tabular toolkits (Ye et al., 2024). The trade-off is operational complexity: retrieval adds memory overhead for the neighbor index and increases inference latency in proportion to the database size.

Box 12.1: Lookahead bias in retrieval-augmented models

Standard TabR implementations build a retrieval index over the entire training set without temporal constraints. In a walk-forward setting, the model can retrieve neighbors from time periods that overlap with or follow the prediction target, a form of lookahead bias that standard implementations do not prevent. Any deployment must enforce temporal isolation: neighbors must come strictly from periods before the prediction



date, with the same purging and embargo logic applied to the walk-forward splits. Implementing this remains an open engineering challenge; treat TabR results without temporal isolation as upper bounds, not production estimates.

When to look beyond GBMs - A decision framework

The choice between GBMs and neural alternatives for tabular (cross-sectional) prediction is regime-dependent. *Table 12.2* summarizes the decision framework for this class of problems, synthesizing the benchmark evidence with the operational constraints that matter in production financial ML. Sequential models (LSTMs, transformers, temporal convolutional networks) add a temporal dimension to this decision; *Chapter 13* covers that extension.

Data regime	Recommended	Rationale
Fewer than 10K effective samples, rapid signal testing	TabPFN (v2.5; distilled)	Strong “no-tuning” baseline for small data; fast iteration. Distillation engine enables deployment.
10K–100K samples, production tabular	LightGBM (or XGBoost)	Strong default accuracy/robustness; efficient training; TreeSHAP explanations are practical for tree ensembles.
10K–100K samples, DL	TabM (rank-1 adapter)	Strongest tabular DL baseline in recent

Data regime	Recommended	Rationale
warranted by complexity	ensemble)	evaluations; validate under walk-forward temporal splits.
More than 100K samples, heavy categoricals	CatBoost (or LightGBM)	CatBoost’s ordered scheme prevents leakage with categoricals; LightGBM supports categoricals via integer codes.
More than 1M samples, GPU available	LightGBM and TabM (or TabR)	No reliable crossover point. Trees often remain competitive; validate both families under your constraints.
Multi-modal inputs (text and tabular)	End-to-end DL (transformers)	Joint representation learning across modalities requires DL; GBMs don’t natively fuse modalities.
Local similarity/regime matching matters	TabR	Nearest-neighbor retrieval improves on some benchmarks; enforce temporal isolation to avoid leakage.

Table 12.2: Heuristics for model selection

For readers who want a performance ceiling rather than a single production model, **AutoGluon Tabular** (Erickson et al., 2020) provides a useful reference. Its “best quality” preset blends GBMs, neural networks, and TabPFN into a multi-layer stacking ensemble with automated

hyperparameter selection. These stacks frequently achieve the best benchmark results by exploiting complementary model strengths. The cost (multi-hour training budgets and operational overhead) makes direct deployment burdensome.

In our workflow, AutoGluon serves as a “ceiling sanity check”: if your tuned LightGBM or TabM is within striking distance, you have extracted most of the available signal. If AutoGluon dramatically outperforms your single-model baseline, inspect which model family in the stack contributes the lift. This diagnoses whether you are leaving signal on the table that a different architecture could capture.

For the medium-sized datasets typical of financial ML (where walk-forward splits reduce effective training size, signal-to-noise ratios are low, and features exhibit heavy tails), well-tuned GBMs remain a strong default. The gap is narrowing, and the regime map above identifies where neural alternatives are most likely to pay off.

The notebook `03_dl_vs_gbm` provides a systematic comparison of LightGBM, TabM, TabPFN, and a vanilla MLP on the ETF case study under walk-forward splits. LightGBM achieves competitive IC at a fraction of neural network training time, while TabM’s rank-1 adapter ensemble provides the strongest neural baseline, consistent with its showing across the broader case study comparison in *Section 12.6*, where TabM beats GBM on 3 of the 8 case studies with TabM coverage.

Box 12.2: Reading tabular benchmarks critically

The tabular ML literature now includes several large-scale benchmarks. Before transferring their conclusions to financial applications, four methodological issues deserve scrutiny:



- *IID assumptions.* TabArena (Erickson et al., 2025) and TALENT (Ye et al., 2024) focus on IID classification and regression. Both explicitly exclude temporal dependence from their current scope. Conclusions about model rankings under IID splits may not hold under walk-forward evaluation.
- *Tuning budget inequity.* Results depend heavily on whether models receive equal time for hyperparameter optimization. A tuned GBM versus a default neural network (or vice versa) proves nothing about the model families. Fair comparisons must specify and equalize the compute budget.
- *Post-hoc ensembling.* TabArena reports that cross-model ensembles (stacking GBMs with neural methods) often produce the best results. This changes the definition of “baseline” and complicates single-model comparisons. If your production constraint is a single model family, ensemble results are aspirational rather than actionable.
- *Dataset leakage.* Multiple benchmark studies have identified leakage in popular tabular datasets: features that encode the target, mislabeled tasks, or timestamp columns that enable lookahead. Curated subsets exist for a reason: TabArena retained 51 datasets from an initial pool of 1,053.

Regardless of the model family, performance depends heavily on hyperparameter configuration. The next section introduces Optuna’s Bayesian approach to efficiently navigating that search space.

12.4 Advanced hyperparameter tuning with Optuna

GBMs introduce tree depth, learning rate, regularization strength, sampling ratios, and leaf counts into the hyperparameter search space, making Bayesian optimization essential for cost-effective parameter optimization rather than just convenient.

The tree-structured Parzen Estimator

Optuna implements hyperparameter optimization (HPO) using sampling methods to learn the relationship between parameter configurations and objective values. The default **Tree-structured Parzen Estimator** (**TPE** , Bergstra et al., 2011) maintains two density estimators: $l(x)$ for hyperparameters that yielded good objective values, and $g(x)$ for those that yielded poor objective values. The acquisition function maximizes $l(x)/g(x)$, concentrating evaluations on promising regions. As trials accumulate, the density estimators sharpen, concentrating proposals on regions that have historically produced strong validation metrics.

TPE is particularly effective for the discrete and conditional parameter spaces common in GBM tuning. It handles integer parameters, categorical choices, and conditional dependencies naturally, for example, sampling `num_leaves` only when tree growth is leaf-wise. The notebook `04_optuna_tuning` shows TPE-based optimization on the ETF case study, with emphasis on search-space design and early-stopping integration.

The define-by-run API

Optuna's define-by-run API defines the search space dynamically within the objective function. During each trial, calls to `trial.suggest_int()` or

`trial.suggest_float()` request parameter values based on Optuna's current model of the objective landscape. This enables conditional parameters to sample regularization strength only for certain model types, hierarchical search to select a library, then tune its specific parameters, and dynamic stopping via pruning.

For iterative models like GBMs, we can evaluate intermediate performance after each boosting iteration. Optuna's **pruners** **automatically terminate trials** that appear unpromising:

- **MedianPruner** prunes trials worse than the median at each step
- **HyperbandPruner** uses adaptive resource allocation for more aggressive early pruning

Pruning can reduce total computation by 50% or more without sacrificing optimization quality.

For production-scale tuning, studies can be backed by a shared database (PostgreSQL, MySQL) so that multiple workers evaluate trials in parallel across machines.

GBM-specific tuning strategy

GBM hyperparameters fall into three families with distinct effects:

- **Tree structure** parameters (max depth, number of leaves, min samples per leaf) control model complexity: deeper trees capture more interactions but carry a higher risk of overfitting.
- **Boosting parameters** (learning rate, number of iterations, subsampling ratios) control training dynamics: lower learning rates with more iterations generally produce better results but increase training time.
- **Regularization parameters** (L1/L2 penalties, min child weight) directly penalize complexity.

A common pitfall is tuning the tree structure extensively while neglecting regularization. Regularization parameters (`reg_alpha` , `reg_lambda`) often have the greatest impact on out-of-sample performance because they directly address overfitting in low-signal-to-noise regimes. Set the learning rate at a low value (0.01–0.05) and let Optuna determine the optimal number of rounds via early stopping. This is generally more efficient than tuning both jointly. Start with 100–200 trials; beyond this range, marginal improvement tends to diminish while the risk of validation overfitting increases (*Box 12.3*).

Box 12.3: Validation overfitting in hyperparameter search



Validation overfitting is the primary risk of automated tuning. In low-signal-to-noise environments like daily return prediction, 500 TPE trials will find hyperparameters that exploit validation-set noise rather than genuinely better configurations: the 0.02 IC improvement vanishes on truly held-out data.

Defenses: limit trials to 50–100, use coarser parameter grids, and always evaluate final candidates on data untouched during optimization. This is not a theoretical concern; it is the most common source of backtest-to-live performance degradation.

Multi-objective optimization

Financial models often face competing objectives that cannot be collapsed into a single metric. Optuna supports multi-objective optimization through its `NSGAIISampler` (Deb et al., 2002), which finds

the **Pareto frontier** of non-dominated solutions, configurations where improving one objective necessarily harms another.

Optimizing IC versus turnover in practice

The notebook `06_optuna_multi_asset` tunes a LightGBM model on the ETF case study with two objectives: maximize rank IC and minimize portfolio turnover. Aggressive hyperparameters (deeper trees, lower regularization, more boosting rounds) tend to increase both IC and turnover simultaneously, because the model fits finer-grained patterns that change more frequently across rebalancing periods.

The Pareto frontier on this dataset is tight. NSGA-II finds two non-dominated solutions, one at $IC \approx 0.029$ with normalized turnover ≈ 0.004 and one at $IC \approx 0.030$ with turnover ≈ 0.005 , both at similarly conservative parameter ranges with modest depth and sample weights. Single-objective optimization on IC alone, run with TPE on the same search space, converges to $IC \approx 0.010$, substantially below either Pareto solution; the multi-objective NSGA-II search reached a region of the parameter space that the single-objective search did not.

The multi-objective approach also reveals when objectives are less conflicted than assumed. If the Pareto frontier is nearly flat in one region, for example, high IC with only modestly higher turnover, you can claim both objectives without significant compromise. Conversely, a steeply curved frontier signals that marginal IC improvements come at rapidly increasing turnover costs, warranting careful analysis of transaction-cost sensitivity.

Other common multi-objective pairs include return versus drawdown (aggressive models achieve higher returns while experiencing larger peak-to-trough declines) and Sharpe ratio versus capacity (strategies with higher Sharpe often trade in smaller, less liquid names).



Implementation :

- `04_optuna_tuning` demonstrates HPO on the ETF case study.
- `05_cross_library_hpo` compares the three GBM libraries XGBoost, LightGBM, and CatBoost with loss type as a tunable hyperparameter on the firm characteristics dataset.
- `06_optuna_multi_asset` compares single-objective and multi-objective approaches across asset classes.
- `07_hpo_comparison` compares the efficiency of grid search and Optuna on the same tuning problem, including empirical evidence of validation overfitting as trial budgets increase.

Well-tuned models need interpretation before deployment. The next section covers TreeSHAP as the primary tool for explaining GBM predictions and detecting feature drift.

12.5 Model explainability with SHAP

Chapter 11 introduced SHAP values (Lundberg and Lee, 2017) as sanity checks for economic models. TreeSHAP (Lundberg et al., 2020) makes this framework routine for GBMs by computing exact Shapley values in $O(TLD^2)$ time per prediction, where T , L , and D are the number of trees, leaves per tree, and average tree depth. For a 500-tree LightGBM model on 100K samples, computation takes seconds, fast enough to run on every walk-forward fold as standard diagnostic infrastructure. By contrast, neural networks require approximate methods (for example,

KernelSHAP, sampling) that are orders of magnitude slower and introduce approximation errors.

Dependence and interaction effects

Section 11.4 covered SHAP dependence plots and their use for identifying feature interactions. For GBMs, TreeSHAP additionally provides exact *interaction values* that decompose each prediction into main effects and pairwise interactions:

$$f(x) = E[f(X)] + \sum_j \phi_j^{\text{main}} + \sum_{j < k} \phi_{jk}^{\text{interaction}}$$

This decomposition quantifies how much of each prediction comes from features acting independently versus in combination, a capability unique to tree-based SHAP that approximate methods do not provide efficiently.

In our ETF case study, the strongest interaction ($H \approx 0.43$) is between the yield-curve z -score and the short-horizon volatility ratio, the top pair among 29 statistically strong interactions identified by the H-statistic. The SHAP interaction matrix shows that the yield-curve signal carries most of its predictive content when short-term volatility deviates meaningfully from its longer-horizon baseline; the interaction contribution can shift sign across volatility regimes. This conditional structure is invisible in standard feature-importance rankings, which rank the yield-curve z -score as a top feature regardless of regime, one reason the same model can look stable on average while its mechanism shifts beneath the surface.

`08_shap_analysis` shows TreeSHAP computation, dependence plots, and the full interaction decomposition on the ETF case study.

SHAP-based drift monitoring

The SHAP stability chart from *Section 11.4* (tracking feature importance across walk-forward folds) extends naturally to production drift monitoring, as illustrated in *Figure 12.2*, where the comparison shifts from training folds to rolling prediction windows.

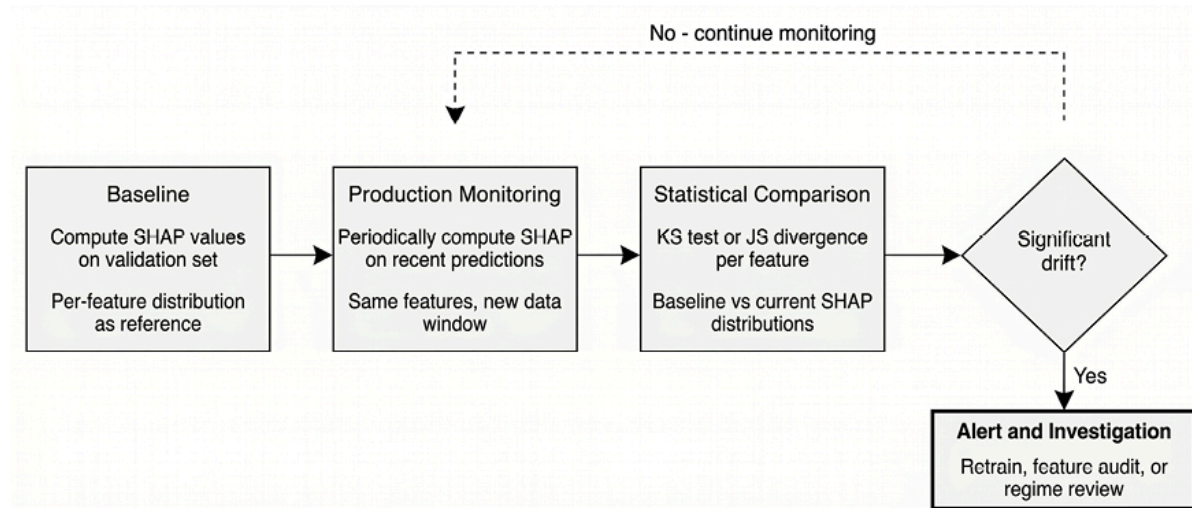


Figure 12.2: SHAP-based drift detection

Comparing SHAP distributions between a baseline period and recent predictions detects changes in mechanisms before they manifest in performance metrics. When features that were predictive lose their SHAP importance (or when previously irrelevant features gain prominence), investigate before performance deteriorates. This provides an earlier warning than outcome-based monitoring (rolling IC, Sharpe ratio) because it detects changes in the model's *reasoning*: by the time returns degrade, the damage is done. `08_shap_analysis` implements this comparison by tracking the mean $|\text{SHAP}|$ across walk-forward folds and flagging features whose importance changes exceed 50%.

Critically, SHAP drift is a *diagnostic signal* warranting investigation, not proof of concept drift. Feature distributions can shift without affecting predictions if the model does not rely heavily on those features. Validate with downstream metrics (rolling IC, residual patterns) before acting.

Chapter 19 covers risk monitoring infrastructure; *Chapter 26* covers production alert thresholds and automated response protocols.

The Rashomon Effect - When equally good models disagree

Models with similar predictive performance can produce strikingly different explanations, a phenomenon known as the Rashomon Effect, after *Rashomon*, the 1950 film directed by Akira Kurosawa, in which witnesses give contradictory accounts of the same event. If a Random Forest and a gradient boosting ensemble both achieve 0.05 IC but attribute predictions to different features, each explanation reveals model-specific logic, not ground truth about the data-generating process. This multiplicity is inherent and should temper confidence in any single model's explanation. When SHAP attributions matter for downstream decisions such as feature pruning, risk allocation, or regulatory reporting, validate findings across multiple model specifications.

In practice, when a GBM and a TabM disagree on which features drive predictions, this disagreement is a signal to investigate the underlying relationship in the data rather than declare one correct. Features that both model families rank as important are more likely to reflect genuine structure; features that only one family highlights may reflect architecture-specific fitting patterns. `09_xai_limitations` shows explanation instability: predictions that differ by roughly 1.3% can yield entirely different top-three SHAP contributors.

SHAP also applies to token-level attributions for NLP models. The notebook `10_shap_nlp_sentiment` shows this for FinBERT sentiment predictions; *Chapter 10* covers the text feature-engineering pipeline in detail.

Using SHAP for feature selection and pruning

TreeSHAP's efficiency enables SHAP-based feature selection as a practical workflow step. Compute mean absolute SHAP values across the training set, rank features by importance, and retrain on the top- k features. This approach can produce models with comparable or improved out-of-sample performance when the removed features primarily contribute noise, though the improvement depends on the original feature set's redundancy and the signal-to-noise ratio. Unlike wrapper methods that require retraining for each feature subset, SHAP importance from a single trained model provides a strong initialization for the feature selection search.

For financial models with dozens of features, SHAP-based pruning improves interpretability and reduces the risk of spurious correlations. Features with consistently near-zero SHAP values across walk-forward folds are strong candidates for removal. Features with high SHAP variance across folds (important in some periods but not others) may indicate regime-dependent signals worth investigating rather than discarding.

Comparing SHAP rankings against permutation importance (PFI) and mean decrease in impurity (MDI), as shown in `08_shap_analysis`, identifies features that are robustly important across methods, and helps filter out artifacts of any single technique.

From explanation to uncertainty and robustness

Conformal prediction (*Chapter 11*) applies directly to GBM residual workflows. The notebook `11_conformal_gbm` implements four variants

side by side: split conformal, quantile regression, **conformalized quantile regression (CQR)**, and an **adaptive conformal (ACI -style)** online update loop. Split conformal provides the finite-sample coverage guarantee under exchangeability; CQR preserves asymmetric intervals while restoring calibration; ACI targets stable coverage under non-stationary residual dynamics through online alpha updates. When drift monitoring flags a regime change, these interval diagnostics typically widen, creating a feedback loop between explainability and uncertainty quantification. *Chapter 19* covers the position-sizing implications.

With the model-building and explanation toolkit in place, we now benchmark GBMs against the linear baselines from *Chapter 11* across all nine case studies to assess where nonlinear models earn their additional complexity.

12.6 Case study insights

Chapter 11 fit a linear baseline to nine case studies and asked each one question: does the model turn engineered features into a cross-sectional ranking that survives out-of-sample? The question and the case studies stay fixed; the two more flexible families introduced in this chapter (gradient-boosted trees and the TabM tabular network) now sit beside that baseline. The `12_case_study_insights` notebook fits all three families to every case study at its primary regression label and reads the results from the locked registry. Each model is summarized by its average daily information coefficient (IC), the mean of the daily cross-sectional rank correlations between prediction and realized return. The uncertainty around it is the 95% confidence interval computed with a HAC correction for the autocorrelation in that daily series. An interval that excludes zero is the bar for distinguishable cross-sectional signal.

Added flexibility does not win by default. Gradient boosting's interval clears zero on five of the nine case studies, against three for the linear baseline; TabM clears it on three of the eight it covers. The flexible families widen the set of case studies with measurable signal, but only modestly, and on no case study do they manufacture a ranking where the linear model found none.

Where flexibility adds ranking content

Figure 12.3 places the three families side by side: for each case study, the highest-IC configuration of each family at the primary label, with its HAC interval. *Table 12.3* holds the underlying numbers. Read together, they separate three situations, beginning with the numbers in the table.

Case study	Horizon	Target	Linear	GBM	TabM
US Firms	1 month	forward return	−0.005	+0.080	+0.031
ETFs	21 days	forward return	+0.054	+0.037	+0.041
CME Futures	5 days	forward return	−0.000	+0.032	+0.004
US Equities	1 day	forward return	+0.016	+0.032	+0.017
SP500 Options	to expiry	straddle return	+0.007	+0.018	+0.002
Crypto	8 hours	forward return	+0.009	+0.011	+0.003

Case study	Horizon	Target	Linear	GBM	TabM
NASDAQ-100	15 minutes	forward return	+0.005	+0.006	—
SP500 Eq+Opt	5 days	forward return	−0.006	+0.006	+0.011
FX	1 day	forward return	+0.005	+0.003	+0.007

Table 12.3: Average daily IC at each case study’s primary regression label, for the highest-IC configuration of each family. Bold marks a HAC 95% confidence interval that excludes zero.

TabM has no NASDAQ-100 coverage. The SP500 Options target is the return of a hold-to-maturity at-the-money straddle; shorter-horizon and delta-hedged alternatives post higher in-sample IC but, as O’Donovan and Yu (2024) document, do not survive realistic option transaction costs, so each case study is held at its primary label

Figure 12.3 then presents the same comparison visually:

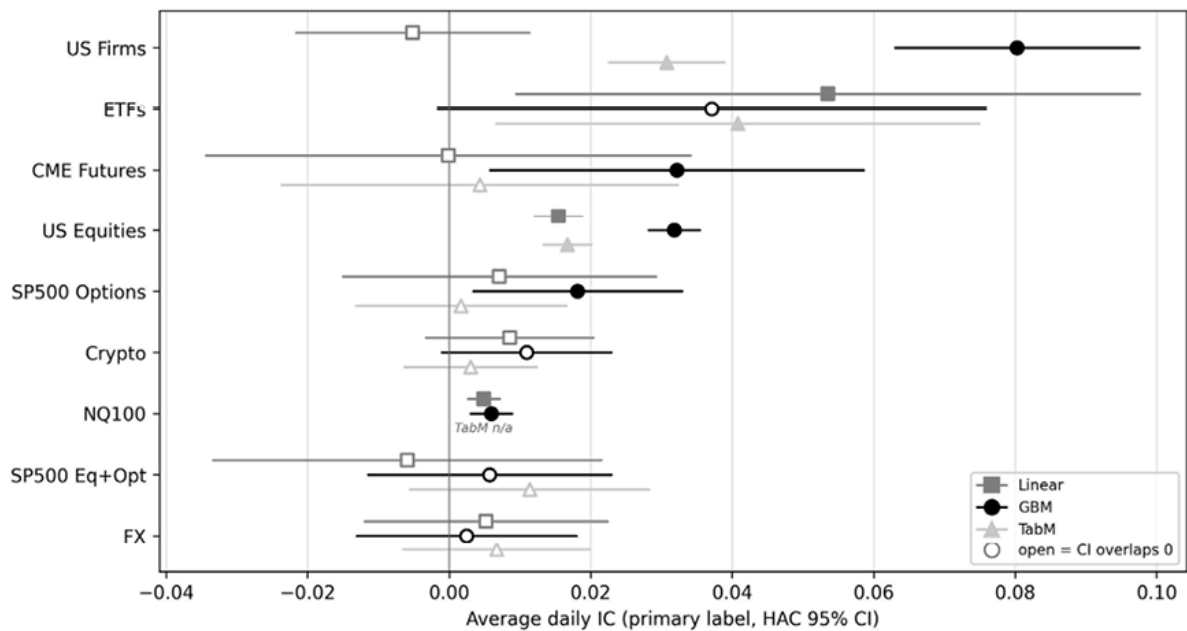


Figure 12.3: Highest-IC configuration of each family per case study at the primary regression label, with HAC 95% confidence intervals. Filled markers indicate an interval that excludes zero; open markers indicate one that overlaps zero

On the case studies where the linear model already found signal, the flexible families match or extend it. US Equities and NASDAQ-100 microstructure carry signal under all three families; on US Equities gradient boosting raises the daily IC from +0.016 to +0.032 with a tight interval, and on the NASDAQ-100 all three agree at a small but well-resolved value near +0.005. Gradient boosting then crosses the significance line on three case studies whose linear interval overlapped zero: US Firms (+0.080), CME Futures (+0.032), and the SP500 Options straddle (+0.018). This is the clearest sign that nonlinear structure carries ranking content the linear projection cannot reach.

The reverse also happens. On ETFs the linear model posts the highest IC of the three (+0.054) and is the only family whose interval clears zero; gradient boosting (+0.037) and TabM (+0.041) order the cross-section slightly worse and less certainly. Cross-asset momentum is close to linear in these features, and tree splits spend capacity on interactions that do not survive out of sample. Where the linear baseline was flat (FX and the combined SP500 equity-and-options case study), all three families stay within noise of zero. Added capacity moves the point estimate without moving the interval off zero.

TabM follows gradient boosting in sign more than in magnitude: when boosting adds content, TabM usually does too, but the two disagree on which case study each serves best. TabM edges boosting on ETFs and FX; boosting is far ahead on US Firms and the SP500 Options straddle. The two encode nonlinearity through different inductive biases (axis-aligned tree splits against learned dense embeddings), and neither dominates across the case studies.

What the lift rests on

The interval comparison in *Figure 12.3* already marks where boosting genuinely improves: on US Equities its interval clears the linear one outright, and on US Firms, CME Futures, and the SP500 Options straddle it crosses zero over a linear baseline whose own interval overlapped it. Where the two families' intervals overlap (the NASDAQ-100, where both sit near +0.005), there is nothing to separate. Where boosting does add content, the mechanism is nonlinear conditioning: splits encode regime indicators, volatility-rank crossings, and cross-asset correlations whose predictive value depends on conditional structure a linear model cannot express. On the case studies with both saved linear coefficients and a stored booster, the most important features reorder substantially between the linear coefficient ranking and the boosting gain ranking, and the per-fold feature ranking is far more stable on US Firms than on US Equities even though US Equities carries the tighter IC. Credible cross-sectional signal does not require a stable top-feature set.

Choosing the configuration

The registry grid crosses three loss functions (squared error, absolute error, and Huber) with leaf budgets from 7 to 127. This is a deliberately small slice of the hyperparameter space: it varies the loss and the leaf budget but leaves at their defaults the regularization controls that usually accompany tree complexity, namely maximum depth, the L1 and L2 leaf penalties, and feature and row subsampling (LightGBM documentation, *Parameters Tuning*). A production model would search those jointly; the goal here is to compare the families on equal footing, not to wring out every last basis point, and the book returns at the end to how lightly these demonstrations are tuned relative to a deployment workflow. On most case studies the loss changes the IC by less than one interval half-width: absolute error (MAE) posts the highest IC on eight of the nine primary labels and squared error (MSE) on the remaining one, ETFs, with Huber

competitive but never highest. US Firms is the exception that shows why the default matters. *Figure 12.4* contrasts it with US Equities on shared axes. On US Firms the MAE curve sits near +0.080 across every leaf budget while Huber and MSE bunch around +0.030 to +0.035, a gap of two to three interval half-widths that absolute error opens by refusing to chase the heavy-tailed extreme returns squared error spends capacity on. On US Equities the same three losses fall within a single half-width at every depth; the configuration barely resolves and the operative choice lies elsewhere.

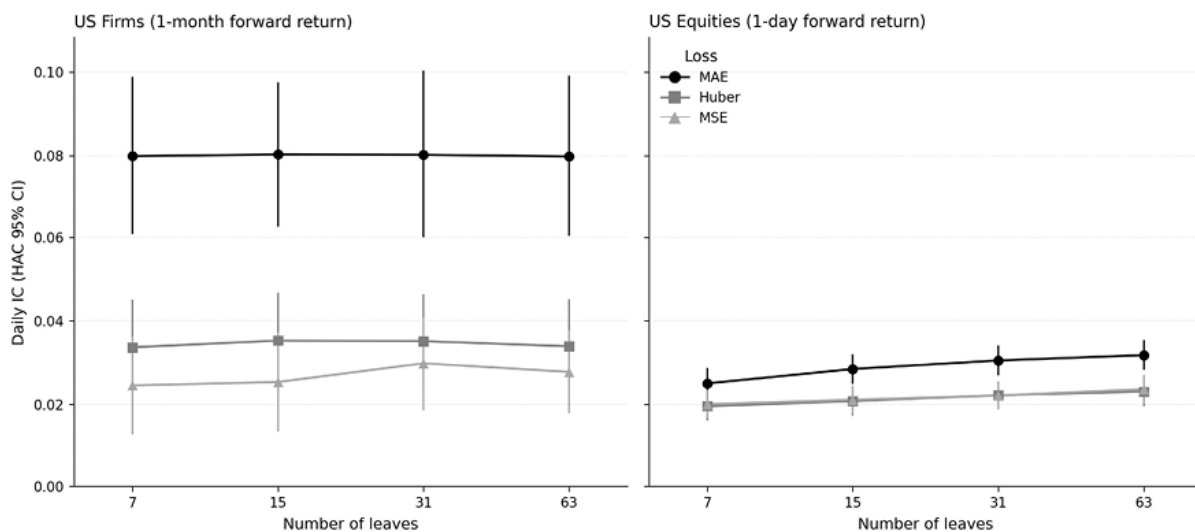


Figure 12.4: Daily IC by loss function and leaf budget for the GBM grid, on US Firms and US Equities, with HAC 95% confidence intervals and shared axes. The configurations spread widely on US Firms but fall within a single interval half-width on US Equities

Tree depth is the least consequential of these hyperparameters. The highest-IC leaf budget scatters across the interior of the grid (seven leaves on three case studies, 63 on three others), and within any case study the depth profiles differ by less than one interval half-width, so depth rarely changes whether the interval clears zero. The hyperparameter that pays operationally is early stopping. The boosting trajectories peak across a wide range: SP500 Options and ETFs by 50 trees, CME Futures and Crypto by 100, FX by 150, while US Firms peaks near 350 and US Equities, the NASDAQ-100, and the SP500 equity-and-options case

study are still climbing at the 500-tree budget cap. Early stopping should be a tuning outcome rather than a fixed prior: a coarse checkpoint grid retires a large fraction of the trees on the early-peaking case studies and is simply inactive where IC keeps rising.

Ranking each case study's grid by HAC-corrected significance (t_{HAC}) rather than by IC selects the same configuration on seven of the nine case studies, and on the two exceptions (US Equities and the NASDAQ-100) both choices sit far inside the region where the interval excludes zero ($t_{HAC} \approx 17$ and ≈ 4), so the substitution never changes whether a case study's interval clears zero. The two rankings nearly coincide because the HAC standard error varies little across configurations within a case study and family; scoring by IC and by significance reorders only configurations whose ICs are already statistically indistinguishable.

Whether the lift persists

A single-period IC can hide a signal that lived in a few windows. *Figure 12.5* tracks the rolling three-month daily IC of gradient boosting relative to the linear baseline across two contrasting case studies, with walk-forward fold boundaries marked. On US Equities, across 2000–2015, the boosting curve sits above the linear curve through most of the fifteen-year span, including the 2008–2009 dislocation. The improvement is a standing feature, not a one-window artifact. On ETFs, across 2015–2023, the two curves cross repeatedly and neither holds a durable margin, consistent with the linear model being the better-resolved choice there. Persistence, not the point estimate, is what separates a deployable edge from a lucky window.

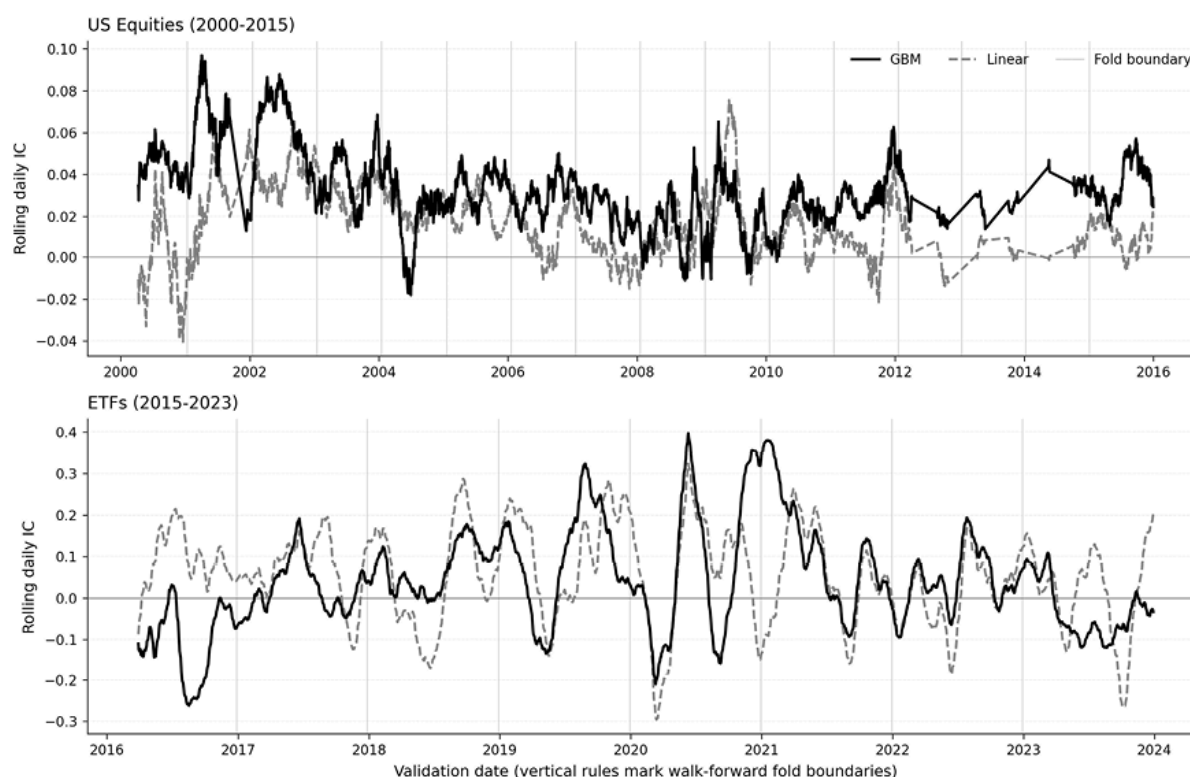


Figure 12.5: Rolling three-month daily IC for gradient boosting and the linear baseline on US Equities (2000–2015) and ETFs (2015–2023). Vertical rules mark walk-forward fold boundaries

Regression versus classification

Three case studies (Crypto, the SP500 equity-and-options case study, and US Firms) train gradient boosting on both the continuous return and the binary direction over the same horizon, allowing the two targets to be compared directly. Each model emits a continuous score, but they are trained and evaluated against different targets: the regression score is read by IC against the continuous return, and the classifier score by AUC against the realized direction (*Chapter 7*). *Table 12.4* cross-evaluates them, asking whether the scores are interchangeable: whether the classifier also ranks magnitudes, and whether the regression score also separates signs.

Case study	Horizon	Classifier score $\rightarrow$ IC	Classifier $\rightarrow$ AUC	Regression score $\rightarrow$ AUC
Crypto	8 hours	+0.020	0.513	0.502
US Firms	1 month	+0.084	0.543	0.540
SP500 Eq+Opt	5 days	+0.004	0.507	0.489
SP500 Eq+Opt	10 days	-0.010	0.505	0.498

Table 12.4: GBM regression-versus-classification cross-evaluation. “Classifier score $\rightarrow$ IC” reads the binary-direction model’s score by IC against the continuous return; “Classifier $\rightarrow$ AUC” scores that model against its own direction label; “Regression score $\rightarrow$ AUC” reads the continuous-return model’s score against direction. Bold marks an interval that excludes the null (zero for IC, one-half for AUC)

They are not interchangeable. On Crypto the classifier’s eight-hour score reaches an IC of +0.020 against the continuous return, with an interval excluding zero, and its direction AUC of 0.513 clears one-half: the gradient-boosted classifier recovers directional content on Crypto that the linear classifier in *Chapter 11* did not. On US Firms the classifier score reaches IC +0.084 with a direction AUC of 0.543. On the SP500 equity-and-options case study neither target separates at either horizon: the classifier IC overlaps zero and its AUC sits at one-half. Read the other way, the regression score’s AUC against direction stays within ± 0.04 of one-half on every pairing, so a model tuned to rank magnitudes is close to uninformative about sign. The same lesson held in *Chapter 11*; the loss function, logistic against squared, decides which discriminative content the score carries. Both views feed downstream: *Chapter 16*’s signal stage ranks on the IC view, and the direction-aware classifiers continue to drive long–short construction in the case-study chapters.

Reading the model

Native gain and split-count importances rank features differently (gain rewards large conditional effects, split count rewards features used for fine partitioning), so neither alone explains a booster. TreeSHAP resolves the disagreement by decomposing each prediction into exact per-feature contributions. *Figure 12.6* shows the SHAP beeswarm for the ETFs and SP500 Options boosters: on ETFs the yield-curve slope and its z-score dominate the spread of contributions, while on the SP500 straddle the GARCH variance-risk-premium and the realized-volatility ratios lead. The fold-level view in the notebook shows some features contributing consistently and others only in particular regimes; monitoring the regime-dependent features for drift, as *Section 12.5* develops, is the operational implication.

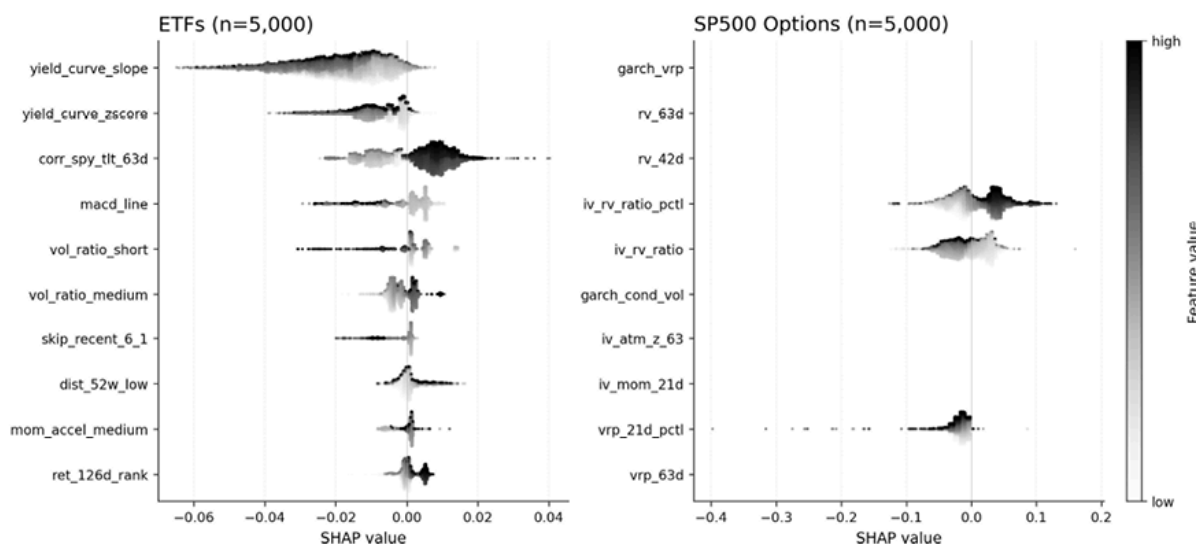


Figure 12.6: SHAP beeswarm of per-feature contributions for the GBM boosters on ETFs and SP500 Options. Each dot is one prediction; shade encodes the feature value; horizontal position is the SHAP value

From IC to profitability

An interval that clears zero certifies a ranking, not a return. On five case studies (US Firms, US Equities, CME Futures, the SP500 Options

straddle, and the NASDAQ-100) gradient boosting supplies the cross-sectional ordering that the portfolio-construction and simulation chapters turn into positions. Where boosting does not clear zero, another family may still carry the ranking (the linear model does so on ETFs), so a null for boosting is not a null for the case study. *Chapters 16 through 19* take these ranked predictions through execution, cost, and risk, to ask which of the statistically distinguishable signals survives as economic profit.

12.7 Summary

Gradient boosting machines are the defensible default for tabular financial prediction. XGBoost, LightGBM, and CatBoost achieve similar accuracy after tuning. The real selection criteria are operational: training speed (LightGBM), categorical integrity (CatBoost), or ecosystem maturity (XGBoost). The cross-case study evaluation confirms that shallow trees with MAE loss and early stopping constitute a strong starting configuration, while monotonic constraints encode economic logic as regularization without sacrificing predictive power. TreeSHAP provides the interpretability infrastructure (dependence analysis, interaction decomposition, and drift monitoring) that production deployment demands.

Deep learning alternatives are narrowing the gap. TabPFN enables rapid signal testing without tuning; TabM provides a practical neural baseline competitive with GBMs on medium datasets. TabPFN degrades faster than GBMs under temporal distribution shift, and all benchmark claims must be validated under walk-forward protocols with purging and embargo before they translate to trading decisions. The regime-dependent decision framework in *Section 12.3* identifies where neural alternatives are most likely to pay off.

Chapter 13 extends to sequential data, covering deep learning architectures (transformers, state-space models, and temporal convolutional networks) for time-series prediction, in which the temporal structure that GBMs treat as features becomes the model's native input format.

13

Deep Learning for Time Series

Deep learning enters time-series prediction when the ordering of observations may carry information beyond the most recent datapoint. *Chapters 11 and 12* treated prediction mainly as a cross-sectional problem: given today's features, forecast a future label. This chapter asks when the sequence history itself adds incremental signal, which architectures can extract it, and how the formulation of the forecast target changes the answer.

That formulation choice matters. Some models generate a future path one step at a time; others predict the entire future path in a single forward pass; many financial ML pipelines skip the path and predict only a horizon label, such as a 5-day return, 21-day return, direction, or cross-sectional rank. This chapter focuses on the architectures and evaluates them in terms of predictive performance, in line with the other modeling chapters. *Chapter 17* then returns to the trading objective itself, showing how deep learning can learn positions or portfolio weights directly under allocation-level losses, such as the Sharpe ratio, drawdown, and cost-aware objectives.

The answer to the architecture choice is not “use Transformers.” Modern time-series modeling is better understood as a competition among inductive biases: recurrence, explicit decomposition, attention, convolutions, state-space models, and strong linear baselines. Each makes different assumptions about temporal structure, covariates, and forecast horizon. The practical task is to decide when sequence-model complexity is warranted, when a direct horizon-label model is the cleaner formulation, and when the problem should be moved downstream into an allocation objective.

By the end of this chapter, you will be able to:

- Explain why recurrent networks became a bottleneck for long-context forecasting tasks.
- Compare the main temporal modeling philosophies and explain when each is most appropriate.
- Use strong baselines and diagnostics to judge whether sequence-model complexity is warranted.
- Distinguish the design logic of modern time-series Transformer variants and relate those choices to multivariate structure, covariates, and forecast horizon.
- Distinguish recursive one-step forecasting, direct multi-horizon path forecasting, and direct horizon-label prediction, and explain why the distinction matters.
- Evaluate time-series foundation model adaptation modes for financial applications, including the implications of transfer mismatch and pretraining contamination.
- Apply practical uncertainty estimation methods, including MC Dropout and deep ensembles, to support risk-aware trading decisions.

Figure 13.1 lays out the timeline of the architecture evolution. We begin with Long Short-Term Memory (LSTM) networks and the limitations that created space for both N-BEATS and Transformer-based architectures. A pivotal 2022 critique by Zeng et al. showed that simple linear models often outperform complex Transformers, challenging assumptions about the need for architectural sophistication. Modern architectures like PatchTST and iTransformer respond through stronger temporal inductive biases. The chapter concludes with a practitioner's framework for selecting architectures based on problem characteristics rather than research hype.

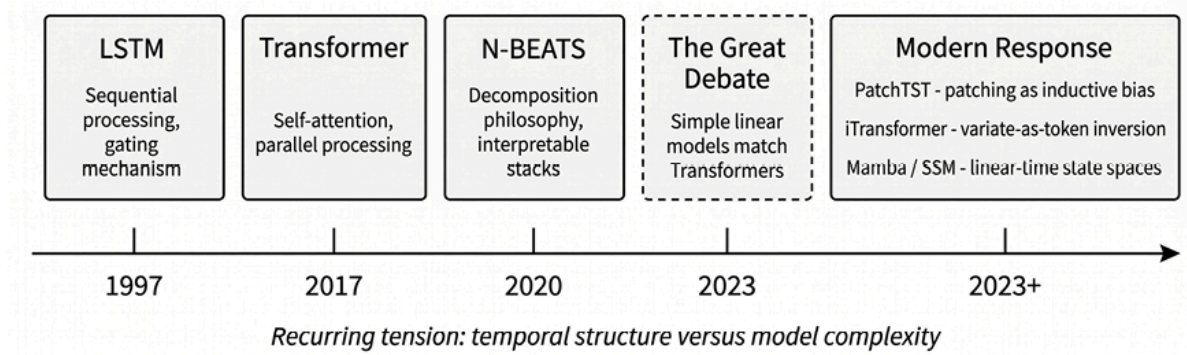


Figure 13.1: Timeline of time-series architecture evolution

Recurrent networks come first; their limitations motivated every architecture that follows.

13.1 Recurrent networks and their limits

Recurrent neural networks (RNNs) maintain a **hidden state** that evolves as the network processes each element of a sequence. This design made them the default architecture for time series forecasting through the mid-2010s.

Long Short-Term Memory (**LSTM**) networks (Hochreiter and Schmidhuber, 1997) addressed the problem that limited vanilla RNNs: **vanishing gradients** prevented the model from learning long-range dependencies (Hochreiter et al., 2001). The LSTM introduced a cell state that acts as a conveyor belt, allowing information to flow across many time steps with minimal degradation.

The LSTM gating mechanism

The LSTM's power derives from three gates (forget, input, and output) that regulate information flow (*Figure 13.2*). The cell state update combines their decisions:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

where f_t and i_t are sigmoid-gated functions of the previous hidden state and current input. This gating mechanism allows LSTMs to selectively remember or forget information, theoretically capturing dependencies spanning hundreds of time steps. In financial applications, LSTMs have been used for volatility forecasting, return prediction, and limit order book modeling (Zhang, Zohren, and Roberts, 2019).

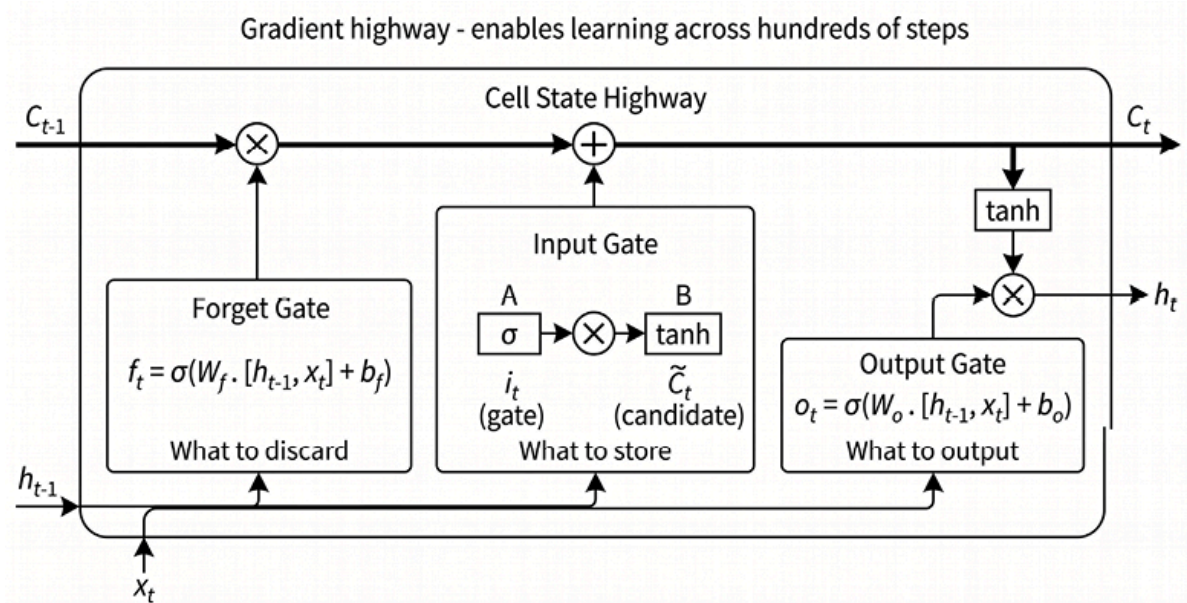


Figure 13.2: LSTM cell with forget, input, and output gates

Two limitations

LSTMs have two limitations that motivated the search for alternatives.

- **The sequential computation bottleneck .** The hidden state at time t depends on the hidden state at time $t - 1$, creating an inherent computational dependence on the sequence's length. LSTMs still benefit from GPU acceleration, but their recurrence limits *temporal* parallelism compared with attention-based models discussed in *Section 13.3*. For high-frequency data with tens of thousands of observations, this bottleneck can become prohibitive. Transformers process all time steps simultaneously through matrix operations, often yielding substantially faster training at similar sequence lengths.

- **Gradient flow challenges** . While LSTMs were designed to address vanishing gradients, they do not eliminate the problem entirely. For very long sequences, gradients must still flow through many gating operations during backpropagation through time (Werbos, 1990). In practice, long nominal lookbacks do not guarantee that the model effectively uses distant information; they often increase noise, training instability, and sensitivity to hyperparameters. For financial series, where the signal is weak and non-stationary, that distinction matters more than the architecture's theoretical capacity.

`01_core_architectures` illustrates the **computation-versus-signal trade-off** on a deliberately simple ETF task: pooled univariate 60-day return windows predict the same ETF's next-day return. On this noisy objective the LSTM trains about twice as slowly as a comparable MLP (32 versus 17 seconds) yet ranks next-day returns no better: both models post a cross-sectional Spearman IC near zero.

The **Gated Recurrent Unit (GRU)** simplifies the LSTM by merging the forget and input gates into a single update gate and eliminating the separate cell state, reducing parameters by roughly one-quarter in the

`01_core_architectures` implementation. This streamlining can reduce model size and sometimes improve generalization on smaller datasets. However, the GRU inherits both fundamental limitations: sequential computation remains $O(T)$, and gradient flow through the merged gating mechanism still degrades over long sequences. The GRU trains slightly faster, and its IC is also near zero: a lighter recurrent cell does not solve the return-prediction problem.

The comparison shows architecture, not strategy: recurrence adds sequential computation without improving next-day return ranking, which reflects the weak signal in return sequences.

Some financial processes exhibit **long-memory** properties, such as volatility clustering (*Chapter 9*), persistent order flow, and calendar effects, in which dependencies span multiple trading days or longer. These long-range

dependencies can exceed what LSTMs reliably capture through gradient-based learning, especially when the useful signal is weak and non-stationary.

These limits motivated parallel architectures along two lines: embedding classical decomposition into neural networks (*Section 13.2*) and importing attention from NLP (*Section 13.3*).

13.2 N-BEATS and explicit decomposition

N-BEATS (Oreshkin et al., 2019) encodes a specific inductive bias: time series are compositions of interpretable components (trend and seasonality) that neural networks should decompose explicitly rather than discover implicitly.

Blocks, stacks, and doubly residual learning

The N-BEATS architecture builds complexity from simple components through a hierarchical structure.

- **The basic block** . The fundamental unit is a fully-connected block that takes a lookback window of historical data as input. The input passes through several dense layers with ReLU activations. The block produces two outputs: a *forecast* for the future horizon and a *backcast* that reconstructs the historical input. The backcast represents what the block has “understood” about the input signal.
- **Residual stacking** . Multiple blocks connect sequentially within a stack. The first block receives the original input. Each subsequent block receives the *residual* from its predecessor: the original input minus the previous block’s backcast. This forces each block to focus on what earlier blocks failed to capture, progressively extracting finer patterns from the signal.

- **Doubly residual architecture** . The full model chains multiple stacks together, applying the same residual principle at a higher level. The second stack receives the residual from the entire first stack. This creates a deep, hierarchical decomposition in which early stacks capture dominant patterns, and later stacks refine the details. The final forecast is the sum of all block-level forecasts across all stacks. This progressive refinement acts as a strong regularizer, paralleling boosting in tree-based methods (*Chapter 12*), since each block must explain only what its predecessors missed.

Figure 13.3 illustrates the complete block-stack-doubly-residual structure.

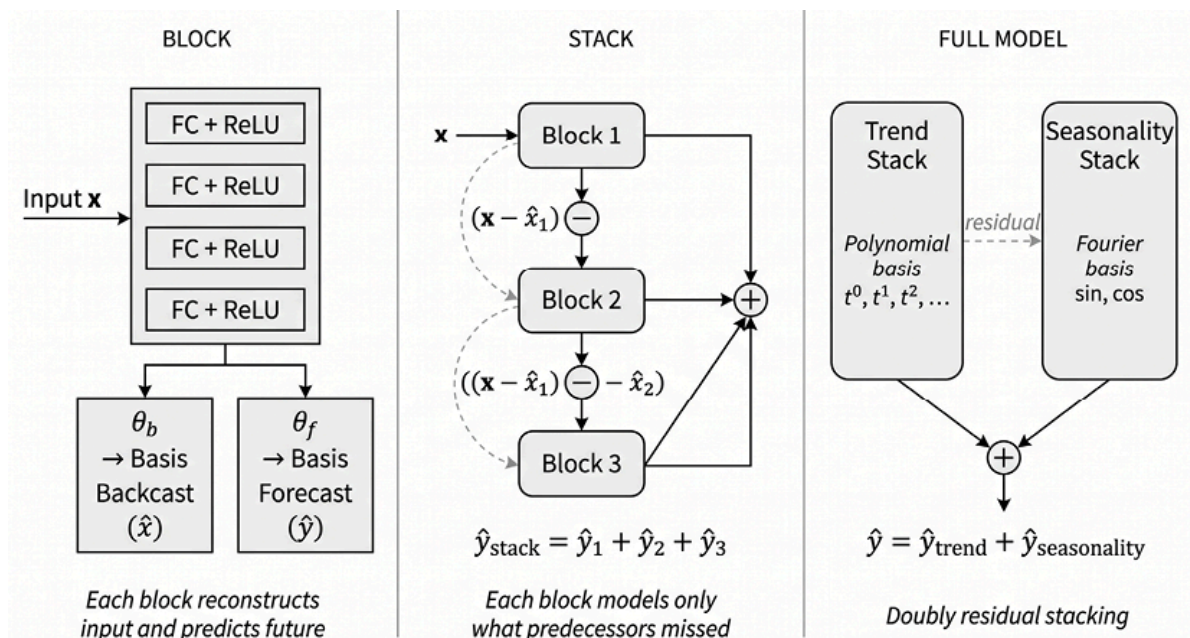


Figure 13.3: N-BEATS block-stack doubly-residual architecture

Basis expansion

The mechanism by which N-BEATS generates its outputs distinguishes it from generic neural networks. Rather than directly outputting forecast values, each block's final layers produce *expansion coefficients* that weight the *basis functions* .

For the forecast, the block outputs coefficients θ_f that combine with basis vectors g_f :

$$\hat{y} = \sum_{i=1}^{|\theta_f|} \theta_{f,i} \cdot g_{f,i}$$

This approach generalizes classical decomposition. Fitting a trend is equivalent to finding coefficients for a polynomial basis; modeling seasonality is equivalent to finding coefficients for a Fourier basis. In the generic N-BEATS configuration, the model learns suitable basis functions from data. In the interpretable configuration, these bases are explicitly constrained.

Making N-BEATS interpretable

The interpretable configuration, **N-BEATS-I** , constrains each stack to model a specific component.

The *Trend Stack* uses a polynomial basis. For a polynomial of degree p , the trend forecast becomes:

$$\hat{y}_{trend}(t) = \sum_{i=0}^p \theta_{trend,i} \cdot t^i$$

The neural network learns the optimal polynomial coefficients that capture the underlying trend component.

The *Seasonality Stack* uses a Fourier basis of sine and cosine terms, with the network learning the coefficients that best represent periodic patterns.

The final forecast decomposes additively: Forecast = Trend + Seasonality. You can plot each component separately, providing direct insight into what drives predictions. This transparency is rare among deep learning models and valuable for building trust in production systems.

Extensions - N-BEATSx and N-HiTS

N-BEATS gained prominence by matching the M4 competition winner's accuracy using a purely deep-learning approach, showing that appropriate inductive biases could close the gap between neural and traditional methods across diverse forecasting domains.

The architecture has inspired two notable extensions:

- **N-BEATSx** incorporates exogenous variables, which are crucial in applications where external factors, such as weather or macroeconomic indicators, drive forecasts. For financial applications, this enables conditioning on macroeconomic releases, calendar effects, or cross-asset signals.
- **N-HiTS** (Challu et al., 2022) introduces hierarchical interpolation and multi-rate signal processing, improving long-horizon forecasting accuracy by capturing patterns at multiple temporal resolutions simultaneously. Its MaxPool downsampling reduces input dimensionality for long-range stacks while preserving detail in short-range ones.

Both extensions preserve the core philosophy of structured decomposition while expanding the model's applicability.

Practical caveats

N-BEATS is sensitive to the **lookback-to-horizon ratio** : too short a lookback starves the decomposition of context, while very long horizons can destabilize training. The interpretable configuration can underperform the generic variant when the true generating process does not decompose cleanly into polynomial trend and Fourier seasonality, a common situation in financial data where regime changes violate both assumptions. N-BEATS-I should therefore be benchmarked against N-BEATS-G on the target data before committing to the interpretable variant.

Decomposition is one escape from sequential processing; importing the attention mechanism from language models is another, and it changes how models relate distant time steps.



Implementation : `02_nbeats_interpretable` implements this architecture from scratch in PyTorch, building blocks, stacks, and the doubly-residual connections step by step. It also trains and evaluates both configurations on SPY data.

13.3 Attention for time series

While N-BEATS evolved classical decomposition principles, a more radical approach came from natural language processing. The Transformer architecture (Vaswani et al., 2017) dispensed with recurrence entirely in favor of self-attention, a mechanism that models relationships between all pairs of sequence elements simultaneously. Its adaptation for time series represents a bet that a sufficiently general mechanism can learn all temporal structure from data alone, without decomposition priors.

Adapting Transformers for temporal data

Applying Transformers to time series requires addressing the architecture's origin in discrete token sequences. Three key adaptations bridge this gap:

- **Tokenization via patching** . Unlike words, which are natural discrete units, time series values are continuous and densely sampled. A naive approach treats each time step as a token, but this creates prohibitively long sequences for the attention mechanism. The modern solution is *patching* : segmenting the input window into fixed-length subsequences that become input tokens. A lookback window of 512 time steps might

yield 32 patches, each of length 16. Patching reduces computational cost while preserving local temporal patterns within each patch.



Caution: Data leakage in patching

Patching can introduce data leakage if normalization statistics are computed across the full window rather than causally. Practitioners must ensure that all preprocessing respects temporal boundaries: either by normalizing each patch independently or by using only backward-looking statistics.

- **Positional encoding** . Self-attention is *permutation-invariant* : without additional information, it cannot distinguish the sequence (x_1, x_2, x_3) from (x_3, x_1, x_2) . For time series, where temporal order is essential, this is a critical deficiency. Positional encodings inject order information by adding position-dependent vectors to each token's embedding. The original Transformer used fixed sinusoidal encodings; many time series variants learn these encodings during training.
- **Encoder-decoder structure** . The standard Transformer comprises an encoder that processes the historical lookback window to create contextual representations, and a decoder that generates forecasts. Time-series adaptations use several variants of that template, including encoder-only and direct multi-horizon heads.

Forecast formulation is a separate design choice from architecture. A model can be recurrent, attention-based, decomposition-based, or linear, but it still needs a target definition. Three formulations matter in this chapter (see *Figure 13.4*):

1. Recursive one-step forecasting learns a one-period-ahead model and rolls it forward. The model predicts y_{t+1} from the past window, appends that prediction to the input, predicts y_{t+2} , and repeats until y_{t+H} . This is the natural extension of one-step statistical forecasting, including the

ARIMA-style models used in *Chapter 9* for model-based features, but forecast errors can compound across the horizon.

2. Direct multi-horizon path forecasting predicts the entire future path in a single forward pass $(y_{t+1}, y_{t+2}, \dots, y_{t+H})$. Many deep forecasting architectures, including sequence-to-sequence models and multi-horizon heads, are designed for this task. They avoid recursive error propagation, but they still model the future path and must learn the joint structure of all intermediate steps.
3. Direct horizon-label prediction skips the path and predicts only the final label used by the investment problem: a 5-day return, 21-day return, direction, volatility premium, or cross-sectional rank. This is the dominant formulation in *Chapters 11, 12, 13, and 14*. The model may use sequential inputs, but the target is a supervised horizon-label problem rather than a time-series path forecast.

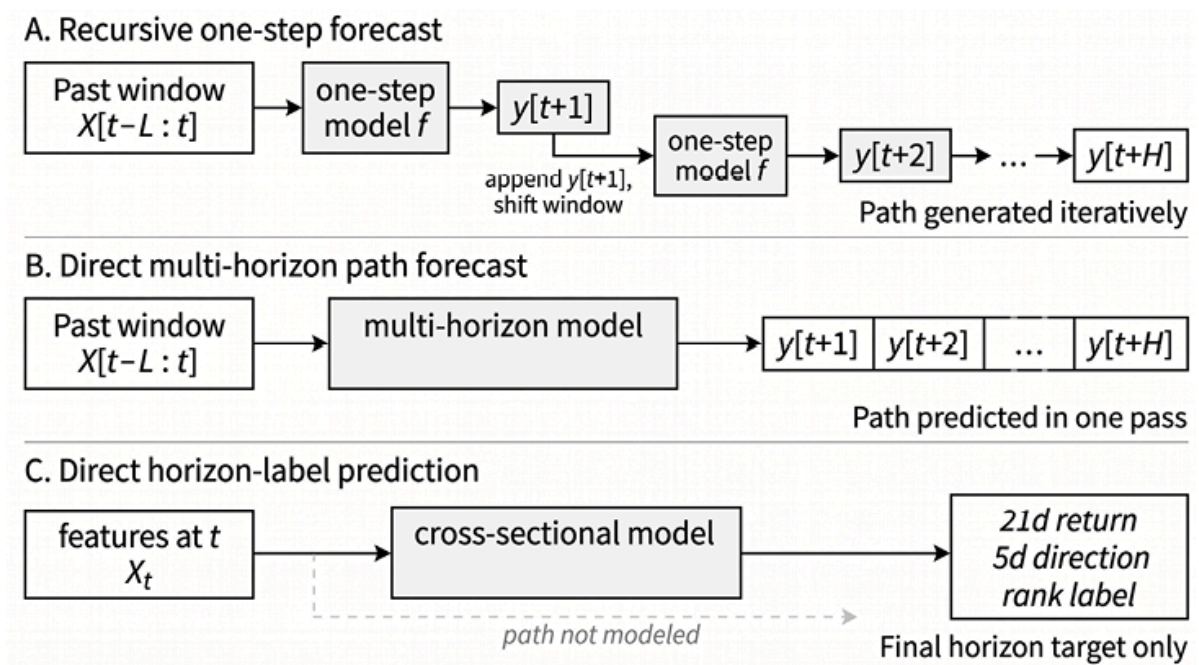


Figure 13.4: Three forecast formulations for financial time series

Recursive forecasting generates the path one step at a time, direct multi-horizon forecasting predicts the path in one pass, and direct horizon-label prediction skips the path and predicts only the final target.

All three formulations recur in this chapter, but the case-study evaluation focuses on how sequence architectures perform under direct horizon-label prediction.

The self-attention mechanism

The **self-attention mechanism**, introduced in *Chapter 10* for language applications, operates identically when applied to time series tokens. Each position attends to all others via Query, Key, and Value projections (see *Figure 13.5*):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Unlike RNNs, which must pass information sequentially through hidden states, self-attention creates direct connections between any two positions, regardless of distance. Multiple **attention heads** learn different relationship types in parallel (local trends, seasonality, regime shifts), and their outputs combine through a learned projection.

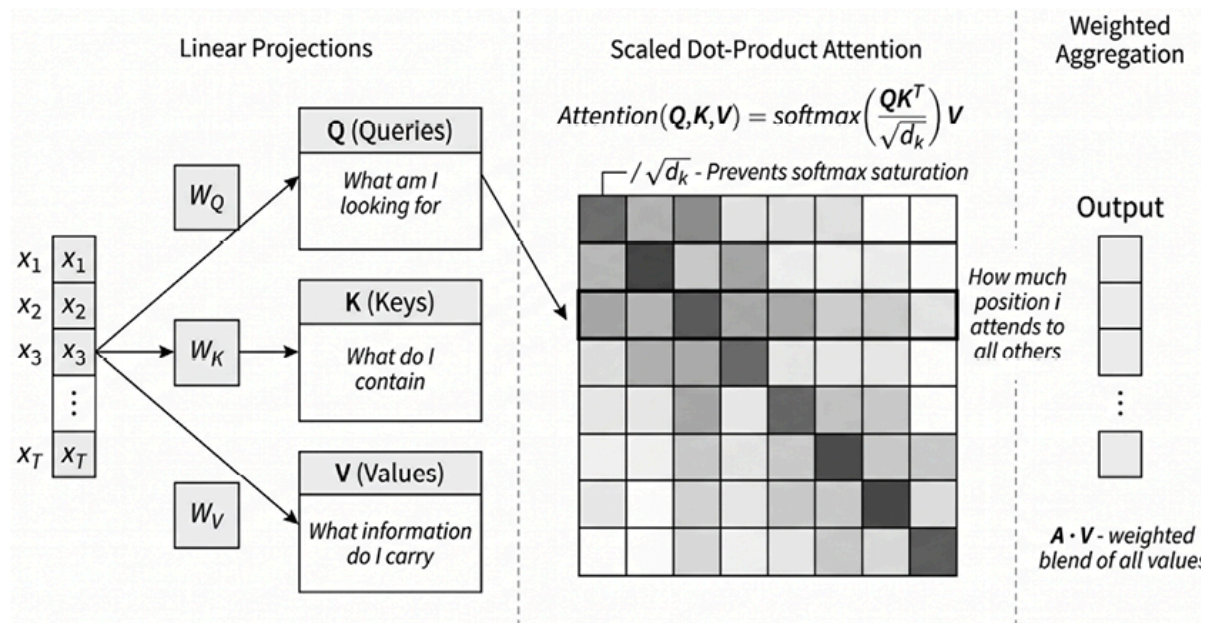


Figure 13.5: Self-attention via query, key, and value projections

The vanilla Transformer's $O(L^2)$ (quadratic) attention complexity motivated several *efficiency-focused variants*, including **Informer**, **Autoformer**, **FEDformer**, and **Pyraformer**, each reducing cost through sparse or frequency-domain attention. All shared a *common assumption*: time series should be tokenized like language, with attention computing relationships between temporal positions. Whether attention genuinely extracted temporal structure (or merely overfitted to patterns that simpler models could match) remained an open question until the empirical challenge the next section takes up.

13.4 Linear baselines versus Transformers

Zeng et al. (2022) showed that simple linear models can outperform several Transformer-based architectures on standard long-term forecasting benchmarks. The result changed the field's baseline expectations: a new architecture should not be taken seriously unless it improves on strong linear benchmarks.

At the same time, these results do not establish that attention is ineffective for time-series modeling in general. They show, rather, that on widely used benchmark tasks, additional architectural sophistication often failed to improve predictive accuracy.

The practical implication is clear: begin with strong, simple baselines, then add complexity only when it delivers robust out-of-sample gains.

Permutation-invariance and temporal order

The core argument against using Transformers for time-series targets the mathematical properties of self-attention. The mechanism is **permutation-invariant**: without positional encodings, attention treats input sequences as

unordered sets. The dot product between a query at position i and keys at all other positions produces the same values regardless of how those positions are arranged.

Positional encodings inject order information, but Zeng et al. argue this is a patch on a mechanism that may underutilize temporal structure. The issue is not that Transformers *cannot* represent order (they can) but that order must be explicitly injected. It may be underweighted in practice depending on architecture and training dynamics.

There is a deeper point. Transformers excel in natural language processing because semantic relationships often transcend position: the words “king” and “queen” relate conceptually regardless of where they appear in a sentence. Time series forecasting is about positional relationships: the value at time t depends causally on values at $t - 1$, $t - 2$, and so on, so the temporal ordering itself may carry the predictive signal. Applying a permutation-invariant architecture to a task where order is paramount is a mismatch of tool and problem.

The LTSF-Linear benchmark

To empirically test their critique, Zeng et al. (2022) introduced a family of “embarrassingly simple” one-layer linear models as new baselines for **Long-Term Series Forecasting** (LTSF), the task of predicting many steps rather than just the next value:

- **Linear** maps the lookback window directly to the forecast horizon via a single matrix multiplication, with no hidden layers, nonlinearities, or attention.
- **D-Linear** first decomposes the series into trend and seasonal components using a simple moving average, then applies separate linear layers to each component, and finally sums them. This adds minimal complexity while incorporating the decomposition insight that N-BEATS had validated.

- **N-Linear** normalizes for distribution shift: subtract the last input value before the linear layer, add it back to the output. This simple trick stabilizes predictions when test-time statistics differ from those used during training. `@3_great_debate` implements all three LTSF-Linear variants, along with a vanilla Transformer encoder, for direct comparison.

The models require seconds to train on standard hardware. Their total parameter counts are orders of magnitude smaller than those of Transformer variants.

Results and diagnostic evidence

In the Zeng et al. (2022) experiments, these linear models outperformed Transformer architectures across all nine LTSF benchmark datasets, achieving 20-50% improvements over Informer, Autoformer, and FEDformer. Perceived progress had been illusory, driven by comparisons against weak baselines rather than genuine architectural advances.

Diagnostic experiments revealed specific failure modes. Forecasting error *increased* for most Transformers as the lookback window grew: they were confused by additional noise rather than extracting useful signals. When input sequences were randomly shuffled (destroying all temporal structure), D-Linear's MSE more than quadrupled (0.345 to 1.406), while the Transformer's barely changed (0.379 to 0.391; *Figure 13.6*). D-Linear exploits temporal structure and collapses without it; the Transformer relies primarily on channel correlations, so removing the time axis incurs little cost. Attention was not learning temporal patterns.



Implementation : `@3_great_debate` replicates these diagnostics for ETF return data.

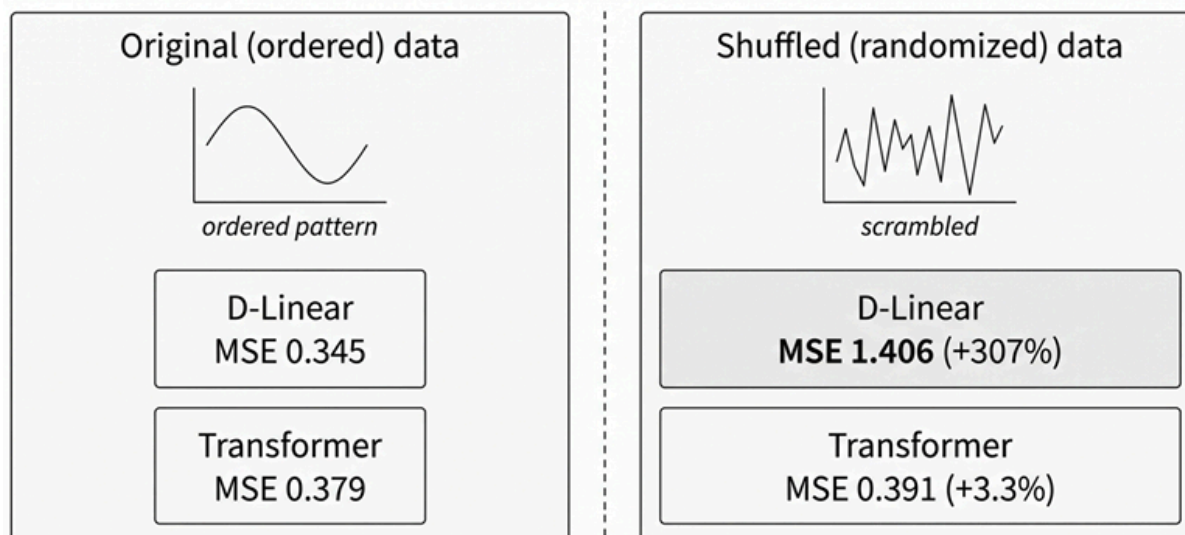


Figure 13.6: The Shuffle experiment: D-Linear versus Transformer under shuffled inputs

Counterpoints and caveats

The paper by Zeng et al. (2022) is not the final word. The original study focused on univariate forecasting; Transformers may do better in *multivariate settings* with rich covariates, where cross-series attention can extract meaningful relationships (see the next section). The comparison may also understate the Transformer’s potential: with careful *hyperparameter tuning*, attention-based models can match or exceed linear baselines on some of the same datasets, indicating sensitivity to the training budget. Finally, **the benchmarks are small** by language model standards. Time-series foundation models trained on millions of diverse series might unlock scaling benefits. However, *Section 13.6* indicates that the evidence regarding financial data is mixed.

Subsequent large-scale evaluations reinforce this message while generalizing it. Small experimental changes (hyperparameter budgets, train/test splits, metric choice) can flip model rankings on standard LTSF suites; across multiple cross-domain forecasting benchmarks, statistical and linear baselines frequently match or beat deep models deemed state-of-the-art on individual datasets. The problem extends beyond “Transformers versus Linear” to the **fragility of benchmark conclusions** themselves.

A closer look at *why* some Transformers succeed reveals that **tokenization and normalization choices matter more than the attention mechanism itself**. Performance on these benchmarks is largely driven by intra-variate dependencies and dataset stationarity, partially explaining why shuffle critiques hold on standard suites yet prove less predictive on harder tasks. The practical comparison is not “Transformers versus Linear” but “strong linear baselines versus time-series-aware tokenization with careful tuning,” and the latter’s advantage is conditional on problem characteristics.

The paper served as a reset rather than an indictment. Any new architecture must now show clear improvement over LTSF-Linear, not just over previous Transformers. The goal shifted from making Transformers bigger to making them *smarter*, and the next generation of architectures, from **PatchTST**’s temporal patching to **iTransformer**’s variate inversion, responded directly to this challenge.

13.5 Modern Transformer variants

The Zeng et al. (2022) critique catalyzed a new wave of innovation rather than ending Transformer-based forecasting. Modern architectures address the identified weaknesses by incorporating stronger temporal inductive biases while retaining the attention mechanism’s ability to model complex dependencies. Three models exemplify this evolved approach: PatchTST, iTransformer, and the Temporal Fusion Transformer (TFT).

Patching as inductive bias with PatchTST

PatchTST, introduced by Nie et al. (2023), offers a simple, effective response to the critique: rather than fight the Transformer’s permutation invariance, design the input representation to work with it.

PatchTST formalizes patching as the central inductive bias . The input time series is segmented into non-overlapping patches of fixed length P . A lookback window of L time steps becomes roughly L/P patches, each treated as a single token. This design has several implications:

- **Computational efficiency** : Attention complexity is $O(N_{patches}^2)$ rather than $O(L^2)$, a large reduction that enables much longer lookback windows
- **Local pattern preservation** : Each patch encapsulates local temporal structure, information that raw pointwise tokenization would scatter across the attention mechanism
- **Semantic coherence** : Patches represent meaningful units of temporal evolution, more analogous to words in language than individual characters

For *multivariate time series* , PatchTST ignores cross-channel dependencies during training. Each channel (each time series in a multivariate set) is processed independently through shared Transformer weights. Forecasts are generated per-channel without attempting to model correlations between variables.

This seems counterintuitive, since multivariate data presumably contains cross-series information worth exploiting. But channel-independence acts as a regularizer. On many benchmarks, cross-channel attention overfits to spurious correlations in the training data that do not generalize. By forcing the model to predict each series solely from its own history, PatchTST avoids this trap while still benefiting from shared representations learned across all channels.

PatchTST often employs **Reversible Instance Normalization** (RevIN) to handle distribution shifts, neutralizing local distributional shifts before the attention mechanism processes the data. This normalization trick (subtracting the instance mean and dividing by the instance standard deviation, then

reversing the transformation on the outputs) is crucial for time series that wander across different statistical regimes over time.

The combination of patching, channel-independence, and RevIN allowed PatchTST to achieve state-of-the-art results on LTSF benchmarks, outperforming both earlier Transformer variants and the linear models that had previously matched them.

Inverting the dimensions with iTransformer

Where PatchTST preserves the standard Transformer structure while changing its inputs, iTransformer (Liu et al., 2024) takes a more radical approach: it **inverts the dimensions on which attention operates**.

In a standard multivariate Transformer, each token represents a time step containing values from all variables. Attention operates across time, relating different moments to each other. iTransformer inverts this: **each token represents an entire univariate series** (or a substantial portion of it), and attention operates across variables.

Concretely, given a multivariate series with N variables, iTransformer creates N tokens, each embedding the full temporal history of one variable. Attention then learns relationships between variables: how one asset's behavior relates to another's, how a leading indicator connects to a lagging one.

Why does inversion work? This design sidesteps the temporal order critique entirely. The model does not apply permutation-invariant attention to time steps; temporal patterns within each variable are captured by the initial embedding process, which can use any time-aware architecture (such as an MLP or convolution applied along the time axis). Attention instead **models cross-variable dependencies**, a domain where permutation invariance is less problematic since there is often no intrinsic ordering among variables.

For *financial applications* involving portfolios of correlated assets, iTransformer offers a natural fit. The model can learn that technology stocks move together, that gold inversely correlates with the dollar, or that volatility spikes in one market presage spikes in others. These cross-asset dynamics represent the type of complex, non-local relationships that attention mechanisms excel at capturing.

Covariate-rich forecasting with Temporal Fusion Transformers

While PatchTST and iTransformer optimize for pure time-series forecasting, the **Temporal Fusion Transformer** (TFT) addresses a different challenge: forecasting when rich metadata and known future inputs are available (Lim et al., 2021). Financial applications often include:

- Static covariates (sector, exchange, asset class)
- Time-varying observed inputs (volume, volatility, macroeconomic indicators)
- Known future inputs (calendar features, scheduled events, options expiry dates)

TFT provides a principled architecture for integrating all three: its distinguishing features are *learned variable selection* (a gating mechanism that assigns time-varying importance weights to inputs, providing interpretability absent from PatchTST and iTransformer) and multi-horizon quantile output, producing *calibrated prediction intervals* rather than point forecasts. For risk management and position sizing, this native uncertainty quantification is more principled than post-hoc methods applied to point-forecast architectures.

When is TFT the right choice? It is strongest in covariate-rich settings with heterogeneous input types, such as portfolio forecasting that incorporates sector metadata, macro indicators, and calendar effects. For pure time series

without covariates, PatchTST’s simpler design typically matches or exceeds TFT.

Covariate-aware forecasting is a design axis. TFT’s covariate handling is no longer unique. Recent benchmarks formalize covariate categories (static, past-only, dynamic, known-future) as a primary axis of evaluation. An emerging result shows that reframing forecasting as a tabular regression with temporal features achieves competitive covariate-aware performance without a time-series-specific architecture, which connects directly to the tabular methods of *Chapters 11–12* .

Table 13.1 summarizes the comparison.

Model	Key Innovation	Attention Domain	Best For
Vanilla Transformer	Self-attention	Time steps	Short sequences, simple baselines
TFT	Variable selection + covariates	Time steps	Covariate-rich, multi-horizon with uncertainty
PatchTST	Patching + Channel-independence	Time patches	Long-horizon univariate/multivariate LTSF
iTransformer	Variate-as-token inversion	Variables	Cross-asset dynamics, portfolio forecasting

Table 13.1: Transformer summary

There is no benchmark substitute for validation on the target data: ETTh1 electricity demand has little in common with cryptocurrency returns.

Transformers are far from the only response to the recurrent bottleneck: temporal convolutions, state-space models, and pretrained foundation models each offer distinct trade-offs that the next section surveys.



Implementation : `04_transformers` benchmarks PatchTST, iTransformer, and Ridge on ETF return prediction. On a 60/20/20 temporal split with cross-sectional IC, Ridge sits at +0.010, a stable anchor across reruns at the same random seed. Both Transformers clear that anchor by a small margin, but their relative ordering (PatchTST versus iTransformer) drifts across reruns of the same code on the same data. Single-split point estimates at this signal-to-noise ratio are not sufficiently stable to rank the architectures; the cuDNN non-determinism in the attention path is large relative to the ETF-panel IC scale. The walk-forward case studies in *Section 13.9* are the authoritative ranking. There, the marginal benefit of temporal modeling depends on how much signal the feature pipeline has already extracted.

13.6 Alternative architectures and foundation models

Beyond decomposition and attention, **several architectures address specific forecasting requirements :**

- **Temporal Convolutional Networks (TCNs ;** Bai, Kolter, and Koltun, 2018) enforce causality through dilated causal convolutions, offering a middle ground between RNNs and Transformers with efficient parallelism and explicit temporal ordering.
- **TSMixer** (Chen et al., 2023) shows that well-designed MLPs that alternate time-mixing and feature-mixing layers can compete with attention on standard benchmarks.
- **CNN-based approaches** (Jiang et al., 2020) convert price histories into images using Gramian Angular Fields (GAFs), which transform a time series into a matrix capturing pairwise temporal relationships. This lets

CNNs detect recurring market patterns by treating price dynamics as visual texture.

Hybrid statistical-neural models represent a design philosophy rather than a single architecture: let a classical model handle known structure such as level, seasonality, and trend, while a neural network learns residuals or time-varying parameters. The ES-RNN (Smyl, 2020), winner of the M4 forecasting competition, combines Holt-Winters exponential smoothing with an RNN. Holt-Winters is a classical forecasting method that recursively updates estimates of level, trend, and seasonality; ES-RNN uses this decomposition to remove non-stationary structure before the RNN learns multiplicative corrections.

Deep state space models (Rangapuram et al., 2018) take the probabilistic variant: neural networks generate time-varying parameters for a linear state space model, with inference performed via Kalman filtering (see *Chapter 9* for details), embedding well-understood Bayesian filtering structure while allowing non-stationary parameterization.

This hybrid philosophy lives on in N-BEATS’s decomposition priors (*Section 13.2*), TFT’s structured covariate handling (*Section 13.5*), and the modern SSMs below, but the explicit approach remains relevant in low-data regimes where pure neural networks overfit, and classical structure provides essential regularization (Lim and Zohren, 2021).

Three developments warrant deeper treatment: state space models for their different approach to sequence processing, foundation models for their implications about transfer learning in finance, and the emerging evaluation infrastructure that determines whether any claimed advance can be trusted.

State space models

State Space Models (SSMs) have become a primary alternative to Transformers for sequence modeling. SSMs are inspired by **continuous-time**

dynamical systems and are discretized for sequential processing. A latent state evolves according to learned dynamics:

$$h_t = Ah_{t-1} + Bx_t$$

$$y_t = Ch_t + Dx_t$$

In a conventional SSM, the matrices A , B , C , and D are learned during training but then applied uniformly across sequence positions.

The **Mamba** architecture, introduced by Gu and Dao (2023), achieves linear $O(L)$ complexity while maintaining the ability to model long-range dependencies. Mamba's innovation is **selective state spaces**: the model learns functions that generate input-dependent SSM coefficients for each time step. In particular, B_t , C_t , and the discretization step Δ_t depend on the current input x_t ; the base transition matrix A remains learned and shared, while the effective discretized transition varies through Δ_t .

This makes the recurrence content-aware, allowing the model to retain relevant information and filter out contextually noisy inputs. Mamba therefore combines the temporally structured memory of recurrent models with hardware-aware parallelization, making it efficient on very long sequences where even patched Transformers can become costly.

A limitation of base SSMs is their reliance on a single temporal resolution. Financial data contains signals across multiple timescales: microstructure dynamics within minutes, momentum over days, regime shifts over months. The **Multi-scale Mamba** architecture (**ms-Mamba**) addresses this by deploying parallel Mamba blocks operating at different sampling rates, capturing high-frequency and low-frequency patterns simultaneously without the quadratic cost of multi-scale attention (Karadag et al., 2025). This directly targets the long-memory properties discussed in *Section 13.1*: volatility clustering and persistent order flow produce dependencies spanning dozens or more trading days across multiple temporal granularities, precisely the setting where a single-resolution model loses information.

SSMs are best understood as part of a **sequence-mixing primitives toolkit** rather than an outright attention replacement. The practical question is: what is my context length, latency budget, and maintenance burden? For sequences exceeding 10,000 time steps, such as tick-level or minute-bar data spanning minutes to weeks, SSMs offer clear efficiency advantages because their linear complexity avoids the memory exhaustion that quadratic attention methods encounter. For shorter sequences with rich cross-variate structure, attention-based models retain their edge because cross-variable attention captures portfolio dynamics that SSMs do not natively model.

Hybrid architectures that interleave attention and SSM layers are an active research direction, but lack broad independent validation. The most promising finance-specific work, **FinMamba** (Hu et al., 2025), combines market-aware graph construction with multi-level Mamba blocks for stock movement prediction, but remains a single-paper result awaiting replication.

We evaluate each architecture for ETF return prediction against a Ridge baseline. On the single 60/20/20 split, Ridge anchors the comparison with a cross-sectional Spearman IC of +0.016 (MSE 0.0035), stable across reruns with the same seed. TCN and TSMixer point estimates sit within the rerun noise of Ridge: TCN has flipped from +0.051 to -0.013 across reruns of identical code, TSMixer from +0.013 to +0.001, with MSEs (TCN 0.0043, TSMixer 0.0039) close to Ridge's. Single-split IC at this signal-to-noise ratio is an architectural sanity check, not a ranking signal; *Section 13.9*'s case studies offer a more thorough walk-forward comparison, and TSMixer reaches IC +0.029 at the 21-day horizon on the ETF panel under multi-step Darts retraining.

Implementation :

- `@5_tcn` implements dilated causal convolutions with a receptive field spanning the full lookback window
- `@6_tsmixer` alternates time-mixing and feature-mixing MLP layers





- `07_mamba_ssm` for a Mamba/SSM implementation on ETF data
- `08_cnn_image_encoding` converts price histories into Gramian Angular Field images and applies convolutional feature extraction

Time series foundation models

Foundation models (FMs) for time series follow the LLM playbook: pretrain on massive corpora to learn universal temporal representations, then adapt to downstream tasks.

The first generation, comprising **Chronos** (Amazon), **TimesFM** (Google), **Lag-Llama** , and **MOMENT** , shared a common limitation: univariate-only inference with no native support for exogenous covariates or multivariate structure, making them poorly suited to quantitative finance where assets coevolve, and external signals are central.

Time-series foundation models import the *pretrain-then-adapt* logic of large language models into forecasting. Early systems showed that large pretrained models could produce useful zero-shot forecasts. However, many were evaluated primarily in univariate settings and were not designed for the multivariate, covariate-rich structure common in real applications.

The second generation addresses this directly. **Chronos-2** (Ansari et al., 2025) introduces a group attention mechanism that processes multivariate series and exogenous covariates through a unified pipeline, enabling covariate-informed zero-shot forecasting via in-context learning. By assigning group identifiers to targets and their associated covariates, the model shares dynamic information across related series without requiring task-specific fine-tuning. **Moirai-MoE** (Liu et al., 2024) replaces static frequency-based input projections with sparse **Mixture-of-Experts** routing, in which token-level gating automatically specializes to the data's heterogeneous characteristics.

The result is **competitive performance with a fraction of the active parameters** of dense models: 11 million active parameters, achieving accuracy that previously required billion-parameter architectures. These advances do not invalidate the first-generation negative results discussed in this chapter, but they do substantially change the adaptation calculus.

Adaptation modes

How a foundation model is deployed matters as much as which model is chosen. The literature has converged on four adaptation modes, each with distinct compute, data, and maintenance requirements:

- **Zero-Shot Inference** : Deploy the pretrained model directly without weight updates. Viable when the target domain resembles the pretraining corpus: energy demand, retail sales, and weather. Empirical evidence suggests that TSFMs help most in data-scarce regimes but do not reliably dominate specialized models when sufficient training data is available, and that scaling laws are not strictly monotonic. For financial returns, zero-shot performance is unreliable (see below).
- **In-Context Learning (ICL)** : Feed historical reference series and covariates alongside the target within the context window; the model infers the current distribution without weight updates. Chronos-2's group attention natively enables this. Recent work demonstrates that training TSFMs to use multiple related series in context at inference time can sometimes match the performance of explicit fine-tuning on target domains. ICL is most valuable for cold-start scenarios and rapid prototyping where gradient-based adaptation is impractical.
- **Parameter-Efficient Fine-Tuning (PEFT)** : Adapt a subset of model weights (LoRA adapters, low-rank projections) on domain-specific data. Standard PEFT can lock adapters into suboptimal minima near initialization, particularly on noisy financial data. Two directions show promise: progressive training that stochastically deactivates adapters to encourage broader exploration, and prune-then-finetune, where

removing up to 90% of parameters before adaptation yields superior accuracy. Both exploit the same insight: foundation models are overparameterized for any single task, and constraining the adaptation surface improves generalization.

- **Test-Time Model Selection and Ensembling** : Rather than fine-tuning one large model, build a portfolio of smaller pretrained models and combine them via selection or ensembling at inference time. Recent work demonstrates that this approach can match single-model accuracy while reducing inference cost, and can be more compute-efficient than test-time fine-tuning. This pathway suits production environments where latency, maintenance burden, and reproducibility matter more than marginal gains in accuracy.

The practical implication is to treat foundation models as *initializations* rather than solutions. The adaptation mode should match the compute budget, data availability, and deployment constraints, not the model's marketing materials.

The limits of scaling

The NLP-imported assumption that performance scales monotonically with parameters does not hold for time series. Evidence from the **GIFT-Eval benchmark** (Aksu et al., 2024), which enforces strict pretrain/test separation across 28 datasets and 177 million data points, bears this out: hybrid architectures interleaving Fast-Fourier Transform (FFT) based long convolutions with linear RNNs achieve competitive zero-shot performance with under 3 million parameters, matching the accuracy of dense models with 1.5 billion parameters.

Independent evaluations confirm that a strict scaling law does not consistently hold for forecasting and that smaller models can occupy favorable positions on the accuracy-efficiency Pareto frontier. In practice, this means high-performance zero-shot forecasting is feasible on standard hardware without debilitating inference latency. The right evaluation metric is the **accuracy-efficiency frontier** , not the raw parameter count. When

comparing TSFMs, report active parameter counts, wall-clock inference times, and memory footprints alongside predictive metrics.

The finance transfer gap

Despite these advances, foundation models face persistent challenges in financial applications. The gap between benchmark performance on standard forecasting tasks and actual utility for trading signals remains the central tension in applying TSFMs to finance.

Rahimikia et al. (2025) provide a *large-scale evaluation of TSFMs* in financial forecasting, covering daily excess returns across 94 countries over 34 years. Off-the-shelf TSFMs underperform tree-based ensembles on both statistical and economic metrics:

- In the U.S. zero-shot setting with a 512-day input window, Chronos Large achieves an out-of-sample R^2 of -1.37%, while TimesFM 500M reaches -2.80%; by comparison, CatBoost achieves -0.03% for the same window size, and -0.10% when averaged across all window sizes.
- **Directional accuracy remains close to random** . Chronos large is just above 51%, TimesFM is just below 50%, and CatBoost reaches 51.16% in the 512-day setting. Even the tree ensembles therefore operate close to the noise floor, reinforcing the weak-signal-plus-non-stationarity premise that runs throughout this book.
- Pretraining TSFMs on financial data substantially improves economic performance: the paper reports, for example, annualized returns and Sharpe ratios of 36.84% and 5.42 for Chronos small, and 30.36% and 3.66 for TimesFM 20M, both with a 512-day window, supporting the view that **domain-specific adaptation is required** rather than optional.

This poor transfer stems from a *distributional mismatch* : general time-series corpora emphasize seasonality, trend, and clear patterns that are largely absent in financial returns. The heavy tails, low signal-to-noise ratio, and non-stationarity of asset prices differ from the electricity demand, weather, and traffic data that dominate pretraining corpora.

TSFMs show promise in risk forecasting. The return-prediction results do not generalize to all financial tasks. For volatility and Value-at-Risk forecasting, where the target exhibits stronger and more transferable structure (volatility clustering, persistence, and mean reversion), TSFMs show credible evidence. Studies evaluating TimesFM against econometric baselines such as GARCH report that fine-tuned foundation models rank among the top performers across multiple VaR quantiles. Statistical forecast-comparison tests support these gains:

- The **Diebold-Mariano** test evaluates whether two models have equal average predictive accuracy under a chosen loss function
- The **Giacomini-White** test evaluates conditional predictive ability, asking whether performance differences remain significant given the information available at the time of forecasting

Zero-shot forecasting is insufficient even for these more structured targets; incremental fine-tuning is essential to learn volatility dynamics specific to the asset class and evaluation period.

Leakage risks specific to pretrained models are a concern. When evaluating TSFMs on financial data, standard temporal train/test splits are necessary but not sufficient. The pretraining corpus becomes a new leakage channel: if the TSFM was pretrained on data that overlaps with or is correlated with the evaluation period, zero-shot claims are compromised.

The **TIME** benchmark (Zou et al., 2025) formalizes this using “fresh datasets” and a human-in-the-loop construction pipeline designed to ensure strict zero-shot integrity. The growing training corpora used by TSFMs make test-set integrity increasingly difficult, analogous to the contamination challenges facing LLM evaluation. Practitioners must verify that evaluation data postdate the pretraining corpus or are excluded from it, and that no proxy leakage occurs via correlated series.

For return prediction, tree-based ensembles with engineered features (*Chapter 12*) remain a hard baseline; for risk forecasting (volatility, VaR),

fine-tuned TSFMs are a credible addition to the toolkit. Prefer models with Student-t or mixture output distributions for financial data.

With this many architectures available, the question shifts from which model is best to which model fits the problem. The next section gives a decision framework for matching architecture to data characteristics, computational budget, and deployment constraints.



Implementation : `09_foundation_models` compares zero-shot Chronos and TTM against trained baselines on ETF return prediction. Chronos achieves IC of -0.015 , and TTM reaches -0.017 , both worse than random, while a task-specific LSTM (50K parameters) achieves IC $+0.025$, and even Ridge regression (60 parameters) reaches $+0.022$. Chronos carries roughly 20 million parameters, and TTM 1–5 million, yet both produce negative zero-shot signals, confirming the distributional mismatch between general pretraining corpora and financial returns.

13.7 A practical framework

The chapter's evidence and trade-offs translate into a practical model-selection process.

Step 1: Establish strong baselines

Before evaluating any deep learning model, establish a baseline performance that any complex architecture must beat. The following models form a baseline ladder: each rung represents a stronger comparison that must be surpassed before adding complexity is justified:

1. **Seasonal naive** : For data with a seasonal pattern (not the case for, for example, return series), predict using the value at the same point in the

previous cycle.

2. **LTSF-linear** : The D-Linear variant decomposes before applying linear layers; N-Linear normalizes by the last input value to handle distribution shift. If a sophisticated model cannot outperform D-Linear, its complexity is unjustified.
3. **Statistical models** : ARIMA and ETS remain strong baselines for univariate series with clear trend and seasonality (*Chapter 9*). For multivariate settings, VAR provides a comparable baseline.
4. **Gradient boosting models** : LightGBM or XGBoost with lagged features often match neural approaches, particularly with limited data. Applying GBMs to time series requires transforming the sequential problem into a tabular regression problem that includes lag features ($t - 1, t - 2, \dots, t - k$), rolling statistics, and calendar features. GBMs excel at learning complex nonlinear relationships in these engineered features but cannot extrapolate beyond the training range.

When does DL outperform GBMs? Chapters 11 and 12 showed that gradient boosting tops many tabular cross-sectional problems. In the case study comparison, the paired daily IC delta between the DL and best-tabular models, with a HAC interval on the common-date difference, excludes zero on three of eight DL-covered case studies: Crypto perpetuals funding, where NLinear at IC +0.029 sits credibly above GBM at +0.011; US Equities, where LSTM at +0.007 sits credibly below GBM at +0.032; and CME Futures, where the LSTM near zero sits credibly below GBM at +0.032. The DL point estimate also exceeds the best tabular baseline on ETFs and FX, but the paired interval overlaps zero on both. On the remaining three panels, the baselines are comparable or stronger. In *Section 13.9*, *Table 13.4* reports the per-case-study evidence. The practical decision rule is unchanged: test whether temporal ordering adds incremental signal beyond lag-feature engineering, and keep the simpler model unless gains are robust out-of-sample.

TabM (*Chapter 12*) extends tabular deep learning with batch-level feature interactions, offering a middle ground between GBMs and sequence architectures. On cross-sectional panels without a strong sequential structure, it is a useful additional baseline alongside LightGBM and Ridge.

Sample size thresholds are calibrated on the book's daily-frequency, moderate-SNR case studies and should be adjusted for the target data's signal strength and feature dimensionality:

- **Fewer than 5,000 samples** : Ridge regression or LightGBM. Deep learning risks overfitting with insufficient data.
- **5,000–20,000 samples** : LightGBM typically offers the best accuracy-to-compute ratio. Deep learning may match but rarely exceeds.
- **More than 20,000 samples** : LSTM and other deep architectures can be *tested at scale* , but superiority is not guaranteed. A large sample size should trigger model comparison, not model replacement.

Once established baselines are beaten, **foundation models offer a progressive adaptation ladder** :

1. Zero-shot inference (free but unreliable for financial returns).
2. In-context learning with domain examples.
3. PEFT/LoRA adaptation on target data.
4. Full domain-specific training.

Each step adds compute and data requirements and improves domain fit (*Section 13.6*). For covariate-rich problems, the chosen model should be verified to handle the relevant covariate types natively: TFT and Chronos-2 support static, past-only, and known-future covariates, while sequence-only models (PatchTST, Mamba) require feature engineering to incorporate external signals.

Step 2: Diagnose the problem

The problem is likely to belong to one of the following categories:

- **Univariate versus multivariate** . For univariate forecasting, N-BEATS and TCNs are natural choices. For multivariate forecasting, the choice depends on whether cross-series dependencies are known or must be learned.
- **Known versus unknown relationships** . If the relationship structure is known (for example, a physical network or supply chain), GNNs (*Chapter 23*) exploit it directly. If relationships must be learned, iTransformer’s cross-variate attention or PatchTST’s channel independence is appropriate.
- **Forecast horizon** . For long-horizon (weeks to months), PatchTST and modern Transformers are appropriate. For short-horizon (minutes to days), performance gaps narrow, favoring simpler approaches.
- **Interpretability** . N-BEATS-I provides trend/seasonal decomposition; TFT offers learned variable importance; GBMs offer feature importance. Standard Transformers are black boxes. Note that post hoc methods such as SHAP and LIME (*Chapter 11*) typically ignore sequential dependencies among inputs, making them unreliable for attributing importance in recurrent or attention-based architectures (Lim and Zohren, 2021).
- **Covariate richness** . If the problem includes static metadata, known future inputs, or heterogeneous feature types, TFT’s variable selection architecture is purpose-built for this setting.
- **Sequence length** . For extremely long sequences (more than 10K steps), Mamba’s linear complexity is necessary. PatchTST handles moderately long sequences via patching.

Step 3: Apply the selection matrix

With the problem diagnosed, match characteristics to recommended architectures (*Table 13.2*):

Scenario	Primary Choice	Alternative	Baseline
Univariate, interpretability required	N-BEATS-I	Autoformer	Seasonal ARIMA
Univariate, black-box acceptable	PatchTST	N-BEATS (generic)	D-Linear
Multivariate, known graph structure	GNN (Ch. 23)	-	VAR
Multivariate, unknown correlations	PatchTST	iTransformer	D-Linear
High dimensionality (>50 series)	iTransformer	PatchTST	Linear
Long horizon LTSF task	PatchTST	D-Linear	Naive Repeat
Covariate-rich, multi-horizon	TFT	PatchTST	LightGBM
Extremely long sequences (more than 10K steps)	Mamba	TCN	Rolling statistics and GBM
Limited data, need robustness	ARIMA/LightGBM	N-Linear	Seasonal Naive
Zero adaptation budget	D-Linear	Zero-shot TSFM	Seasonal Naive
Moderate adaptation budget	PatchTST and LoRA	TSFM ensemble	LightGBM

Scenario	Primary Choice	Alternative	Baseline
(PEFT)			

Table 13.2: Architecture selection matrix. Match problem characteristics to recommended architectures

Apply the walk-forward protocol (*Chapter 6*) for validation and evaluation. Always include non-deep baselines: omitting them inflates perceived gains. For TSFM evaluation, verify pretrain/test data separation to rule out leakage through pretraining corpora (*Section 13.6*). Report bootstrapped confidence intervals alongside economic metrics (IC, Sharpe, maximum drawdown).

Forecasting paths versus predicting horizon labels

The architectures in this chapter originate in time-series forecasting, predicting future values of a series from its history. Our case study pipelines solve a related but distinct problem: given a lookback window of time series values and features for each asset, predict the forward return as a scalar. This is direct panel regression with sequential inputs, not classical forecasting. In other words, the model sees a sequence, but the supervised target is a scalar label at horizon H , often multiple time steps ahead, not the full path from $t + 1$ to $t + H$.

The distinction matters in practice:

- True forecasting predicts a path (daily returns over a multi-day horizon), then maps that path to a label
- Direct regression skips the path and predicts the label itself

For the cross-sectional ranking task central to this book, direct regression has two advantages: it matches the evaluation metric (Spearman IC on the predicted labels), and it avoids compounding prediction errors over the horizon.

This differs from both recursive and direct multi-horizon forecasting (see *Figure 13.4*):

- Recursive forecasting generates the path one step at a time
- Direct multi-horizon forecasting predicts the path in one pass
- Direct horizon-label prediction skips the intermediate path entirely

That last formulation is often the right choice for cross-sectional ranking because the evaluation metric is defined on the final label, not on the intermediate daily path. If we care about the path, say from a risk perspective, the choice might change.

We tested the forecasting formulation directly. For N-BEATS and TSMixer, we ran the faithful library implementations (via Darts), which predict daily return paths and compound them to the horizon label, the task these architectures were designed for. For ETFs (21-day horizon) and CME futures (5-day horizon), the forecasting formulations produced substantially weaker signals than the direct regression models (LSTM, NLinear) on the same data. The long horizons amplified per-step errors, and the forecasting objective was misaligned with the ranking metric.

This finding carries a practical lesson: architecture selection is necessary but not sufficient. The formulation of the prediction problem itself (direct regression versus multi-step forecasting, cross-sectional ranking versus per-series prediction) determines whether temporal modeling adds value. N-BEATS and TSMixer are faithful forecasting models for forecasting a future path. LSTM and NLinear can be better matched to the book's direct horizon-label prediction task because they learn the scalar ranking target directly. The sophisticated forecasting components of N-BEATS, TSMixer, and PatchTST are valuable when the path matters; they can add unnecessary complexity when only the horizon label is evaluated.

There is a fourth formulation that we cover in *Chapter 17 : direct learning of portfolio allocation* . Instead of predicting a path or horizon label, the model outputs positions or portfolio weights and is trained on an economic objective such as Sharpe ratio, drawdown, turnover, or transaction-cost-adjusted return.

Forecasting libraries

This chapter implements architectures directly in PyTorch for demonstration. For standard forecasting tasks, several mature libraries reduce implementation effort substantially:

- **Darts** (Unit8) provides the broadest model coverage with a unified `fit()` / `predict()` API spanning classical and deep learning methods. Darts is the appropriate choice for rapidly benchmarking diverse approaches on standard forecasting tasks.
- **NeuralForecast** (Nixtla) offers native panel-data support and optimized training loops. Choosing NeuralForecast is the primary requirement when training a shared model across multiple related time series.
- **PyTorch Forecasting** (maintained under the sktime umbrella) is built around the Temporal Fusion Transformer. It is the appropriate choice when the problem involves rich covariates: static metadata, known future inputs, and multiple target variables.
- **GluonTS** (Amazon) anchors the foundation model ecosystem around Chronos and Chronos-2 with native probabilistic outputs. Choose GluonTS when zero-shot or fine-tuned foundation models are the deployment target.
- **sktime** provides a scikit-learn-compatible meta-framework that wraps Darts, NeuralForecast, HuggingFace, and classical methods under a single `fit` / `predict` / `score` API. sktime is the appropriate choice for swapping backends without rewriting data pipelines.

For standard per-series forecasting, Darts or NeuralForecast will save significant implementation time. For cross-sectional panel prediction (ranking N assets by predicted return and evaluating via Spearman IC, as our case studies require), raw PyTorch with custom training loops remains the most natural fit. Our case study experiments illustrate this directly: running N-BEATS and TSMixer through Darts's forecasting formulation produced a substantially weaker signal than the direct regression adaptations (see

Forecasting paths versus predicting horizon labels above), confirming that library choice implies a modeling choice.

Library	Strengths	Panel Support	Foundation Models
Raw PyTorch	Full control, custom losses, cross-sectional	Manual	Manual
Darts	Broad model coverage, unified API	Limited	TimesFM
NeuralForecast	Speed, native multi-series	Native (unique_id)	No
PyTorch Forecasting	TFT, rich covariates	Via TimeSeriesDataSet	No
GluonTS / AutoGluon	Chronos-2, probabilistic	Via DeepAR / Chronos-2	Yes
sktime	Composable pipelines, wraps backends	Via reduction	Via wrappers

Table 13.3: DL for time series library overview

Selecting the right architecture and library is only the first step: a point forecast without a confidence measure gives no guidance on how aggressively to trade the signal. The next section addresses uncertainty quantification, the bridge between prediction and position sizing.



Implementation : `11_library_landscape` provides a hands-on comparison of raw PyTorch, sktime, and Darts on the same



forecasting task, with per-library API details and code-effort metrics.

13.8 Quantifying prediction uncertainty

Point forecasts are only part of the signal (see *Chapter 11*). For risk management, portfolio construction, and position sizing, the uncertainty surrounding a prediction is often as important as the prediction itself. A forecast of +2% warrants a different response when accompanied by a narrow predictive interval than when surrounded by wide uncertainty bands. In practice, useful uncertainty estimates can often be obtained from standard deep learning models without moving to fully Bayesian neural networks or other specialized probabilistic architectures.

Estimating uncertainty with MC dropout and deep ensembles

Deep learning models can provide useful uncertainty estimates without requiring fully Bayesian neural networks. Two practical approaches are **Monte Carlo (MC) dropout** (Gal and Ghahramani, 2016) and **deep ensembles** (Lakshminarayanan et al., 2017). Both start from the same idea: instead of relying on a single deterministic prediction, generate several plausible predictions from closely related models and use both their average and their spread.

The average prediction often improves accuracy because idiosyncratic errors partially cancel out. The spread across predictions then serves as an uncertainty signal. When many plausible versions of the model produce similar forecasts, confidence is higher. When they disagree substantially, the prediction is less reliable. For trading applications, the main practical value is that uncertainty estimates help distinguish forecasts that warrant larger

positions from those that call for smaller positions, tighter risk limits, or no trade at all.

MC dropout obtains this distribution from a single trained model. Dropout remains active during inference, and the model is evaluated repeatedly, typically over 50-100 forward passes. Each pass samples a different subnetwork, producing a distribution of predictions rather than a single output. The sample mean becomes the point forecast, and the sample standard deviation provides a simple measure of predictive uncertainty. This uncertainty is commonly interpreted as reflecting uncertainty in the fitted model itself: the model's forecast changes because different plausible subnetworks produce different answers.

Deep ensembles generate the same kind of distribution in different ways. Instead of sampling many subnetworks from one trained model, they train several independent models, typically 3 to 10, with different random initializations and sometimes different data orderings or bootstrap samples. Each model converges to a somewhat different solution, so the ensemble captures a range of plausible predictive functions. Averaging across ensemble members often improves predictive performance, while disagreement among members provides a natural estimate of uncertainty.

In practice, **deep ensembles are often more robust than MC dropout**, but they also cost more because both training and inference scale approximately linearly with the number of ensemble members. The tradeoff is straightforward:

- MC dropout is cheaper and easier to add to an existing model because it requires little architectural change
- Deep ensembles usually demand more compute, but they often provide stronger predictive performance and better-calibrated uncertainty estimates

In many applied settings, an ensemble of about five models offers a reasonable compromise between performance and computational cost.

A more elaborate variant extends deep ensembles by having each model predict not only a mean forecast but also a variance through a **heteroscedastic output head**. This aims to capture the idea that some observations are intrinsically noisier than others: even a well-specified model may be less certain for some assets, horizons, or market regimes than for others.

This extension can produce richer predictive intervals, but it also increases modeling complexity and is not necessary to obtain the main practical benefit of ensembles. The essential point is simpler: model averaging can improve forecasts, and model disagreement can identify less trustworthy predictions.

Whatever method is used, uncertainty estimates should be evaluated empirically rather than accepted at face value. Useful diagnostics include empirical coverage, interval width, sharpness, and reliability plots for predictive intervals. In trading applications, the most important test is whether forecasts flagged as more uncertain actually produce larger out-of-sample errors, and whether any chosen interval target achieves its intended coverage (see *Section 11.5*). Where nominal coverage is poor, conformal calibration on a held-out window can improve interval validity.



Implementation : `10_uncertainty` illustrates both MC dropout and a five-member LSTM ensemble on ETF data, together with **split-conformal calibration** on a held-out window. Without calibration, Gaussian intervals from either method are under-covered by roughly an order of magnitude at the 95% nominal level. After split-conformal post-processing, empirical coverage lands within about three percentage points of nominal at the 50, 80, and 95% levels for both methods. At the 95% target, MC dropout reaches 0.97 and the ensemble 0.97. The calibration step recovers interval validity without retraining the base model. Across reruns at the same random seed, the ensemble's uncertainty-error rank

correlation exceeds MC dropout's, but the precise magnitudes drift; the *qualitative ordering* (ensemble disagreement tracks error better than dropout-subnetwork variance on this LSTM) is the result that survives rerunning.

Foundation model calibration

Uncertainty estimation becomes more challenging for foundation models, especially under the regime shifts, heavy tails, and volatility clustering that characterize financial data. A model pretrained on broad time-series or textual corpora may produce useful point forecasts in finance, yet still be poorly calibrated in the tails or during market stress. In these settings, nominal predictive intervals may be too narrow exactly when risk control matters most.

Conformal calibration (see *Chapter 11*) offers a practical post hoc correction. By calibrating on a held-out window, conformal methods can improve empirical coverage without retraining the base model. This is particularly valuable when the pretraining distribution differs materially from the target financial environment.

For risk-sensitive tasks such as **Value-at-Risk** or **Expected Shortfall** , calibration is not secondary to point accuracy. A time-series foundation model may predict the conditional mean reasonably well while still understating tail risk. In such cases, *domain-specific fine-tuning* and *post-hoc calibration* are often essential not only for improving accuracy but also for obtaining usable uncertainty estimates.

With architectures selected and uncertainty estimates evaluated, the question becomes empirical: which approaches actually deliver useful signals across asset classes, horizons, and data regimes? The next section synthesizes the evidence from the case studies to answer that question.

13.9 Case study insights

Chapters 11 and 12 measured how far regularized linear models, gradient boosting, and the TabM tabular network turn engineered features into a cross-sectional ranking. The deep sequence models of this chapter join that running comparison under the same test: reading the return sequence directly, do they rank assets better than the tabular families already do? The deep architectures cover eight of the nine case studies (all but US Firms) at each case study's primary regression label, with NLinear fit on every one, the LSTM on all but the SP500 Options straddle, and TCN, TSMixer, and PatchTST contributing on a further subset. Each model is summarized by its average daily Spearman information coefficient (IC) and the 95% confidence interval around it, computed with a HAC correction for the autocorrelation in the daily series. An interval that excludes zero is the bar for distinguishable signal.

Where the sequence models add ranking content

Two questions separate here. The first is whether a deep model clears zero; the second, sharper one is whether it clears the tabular baseline. *Table 13.4* reports, for each case study, the highest-IC deep architecture for the primary label, alongside the strongest tabular model from *Chapters 11 and 12*. The deep family's interval excludes zero on four case studies (ETFs, Crypto, US Equities, and the NASDAQ-100). It overlaps zero on the SP500 Options straddle, CME Futures, the SP500 equity-and-options case study, and FX.

Case study	Horizon	Best deep model	Deep IC (t)	Best tabular	Tabular IC	Δ IC
ETFs	21 days	NLinear	+0.062 (3.0)	Ridge	+0.054	+0.009

Case study	Horizon	Best deep model	Deep IC (t)	Best tabular	Tabular IC	Δ IC
Crypto	8 hours	NLinear	+0.029 (4.6)	GBM	+0.011	+0.018
SP500 Options	to expiry	PatchTST	+0.013 (1.8)	GBM	+0.018	−0.005
FX	1 day	NLinear	+0.011 (1.3)	TabM	+0.007	+0.004
SP500 Eq+Opt	5 days	PatchTST	+0.011 (1.1)	TabM	+0.011	≈ 0
US Equities	1 day	LSTM	+0.007 (5.4)	GBM	+0.032	−0.025
NASDAQ-100	15 minutes	NLinear	+0.005 (2.3)	GBM	+0.006	−0.001
CME Futures	5 days	LSTM	−0.001 (−0.1)	GBM	+0.032	−0.033

Table 13.4: Highest-IC deep architecture per case study at the primary regression label, with its HAC t -statistic, beside the strongest tabular baseline (Ridge, GBM, or TabM) from Chapters 11 and 12. Δ IC is the deep-model point estimate minus the best tabular point estimate

Clearing zero is not the same as improving on the baseline, and the two come apart sharply. On US Equities, the LSTM’s IC is credibly above zero yet, at +0.007, sits far below gradient boosting’s +0.032 on the same data. To compare the families, we compute the average difference between the two daily IC series and an HAC interval around this mean difference, pairing the models day by day. On this measure, the deep-minus-tabular delta excludes zero on three case studies (*Figure 13.7*): Crypto, where the deep model adds +0.018 in daily IC over a baseline near zero; US Equities, where it trails gradient boosting by 0.025; and CME Futures, where it trails by 0.033. On

the other five case studies, the paired interval overlaps zero, so the deep model is statistically indistinguishable from the strongest tabular alternative.

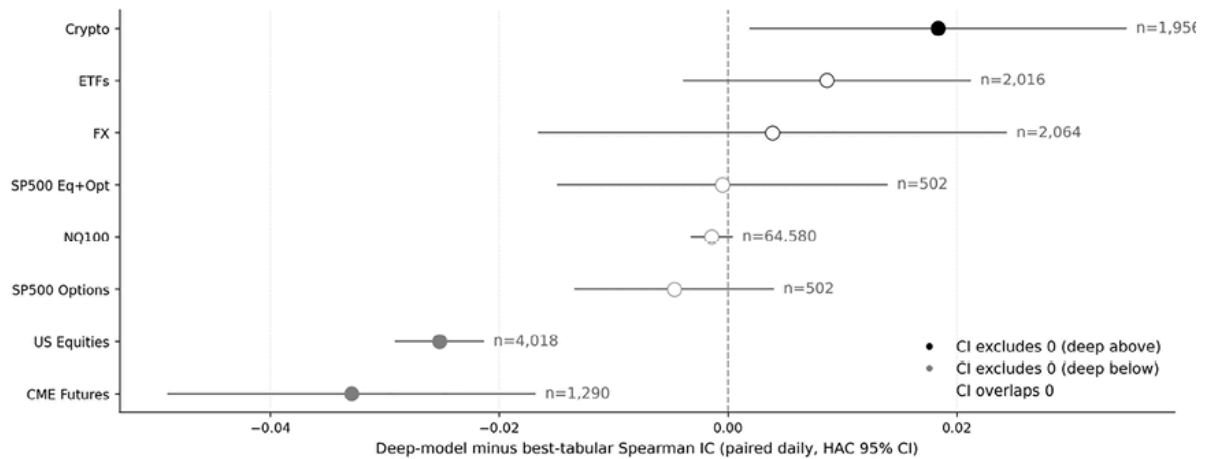


Figure 13.7: Deep-model-minus-best-tabular daily IC delta per case study, with paired HAC 95% confidence intervals on the common-date difference. The three intervals that exclude zero are Crypto (deep model above) and CME Futures and US Equities (deep model below)

The result, stated plainly: across eight case studies, direct sequence modeling credibly improves the ranking on one, Crypto; credibly trails on two, US Equities and CME Futures; and matches the tabular families on the other five, at substantially higher training cost.

Which architectures carry the signal

NLinear reaches the highest deep-model IC on four case studies (ETFs, Crypto, the NASDAQ-100, and FX), the LSTM on two (CME Futures and US Equities), and PatchTST on two (the SP500 Options straddle and the SP500 equity-and-options case study); the temporal convolutional network leads on none. That ordering follows from how the architectures are used here. The case studies frame each model as a direct predictor of the horizon-label return (see *Section 13.7*), rather than as a classical multi-step path forecaster. In that role, NLinear's normalize-and-project structure and the LSTM's gating retain enough temporal information to rank the cross-section without incurring the error-compounding cost of a multi-step objective. N-BEATS, TSMixer, PatchTST, and iTransformer carry inductive biases

designed for path forecasting or cross-variate modeling, and those biases do not align with a cross-sectional rank label.

Training-budget trajectories fall into two regimes. On the high-frequency case studies (the NASDAQ-100, SP500 Options, and Crypto), daily IC peaks early and then decays. On the daily and longer-horizon case studies (FX, CME Futures, and US Equities), it drifts upward as training progresses. The late-trajectory spread is tightest where coverage is widest, so early stopping is best treated as a per-case-study tuning outcome rather than a fixed rule.

Whether the predictive uncertainty is reliable

A credible ranking and a reliable uncertainty estimate are separate properties, and the deep models earn them unevenly. Calibrating a single split-conformal layer on one fold's residuals and reading empirical coverage against the 90% nominal level splits the eight case studies three ways. On CME Futures, ETFs, and US Equities coverage lands near nominal, close to 0.90.

The two SP500 case studies over-cover, returning intervals wider than the data warrant. The rest under-cover by varying margins: Crypto returns intervals well short of nominal, FX modestly short of it, and the NASDAQ-100 falls to near-zero empirical coverage. The spread reflects the fragility of a single calibration split in finite samples, whose validity rests on the calibration and evaluation residuals remaining exchangeable, something a stationary label does not guarantee.

Coverage refines the IC reading rather than restating it. A case study with a credibly nonzero IC and near-nominal coverage, such as ETFs or US Equities, sits in a different operational position from one with a credibly nonzero IC but unreliable coverage, such as Crypto or the NASDAQ-100, where the model may still rank assets while its uncertainty layer cannot yet be trusted. Multi-fold conformal calibration is the natural response where split noise dominates, and Mondrian, group-conditional calibration where

coverage fails along known groupings (asset class, horizon, or volatility regime) at the cost of fewer calibration points per group.

Direct prediction versus forecasting

The case studies treat sequence models as direct horizon-label predictors, and a controlled comparison shows why. Trained instead as a multi-step forecaster through Darts, TSMixer reaches an IC of +0.029 on ETFs at the 21-day horizon—real signal, but below both the direct-regression NLinear and Ridge on the same data (*Table 13.4*), because errors compound over the forecast path and degrade the eventual ranking. To remove compounding from the comparison, the US Equities case study was resampled to weekly, non-overlapping returns, in which predicting the five-day-ahead return becomes a single step.

Model	Approach	Mean IC (4 folds)
NLinear	Direct regression	+0.018
LSTM	Direct regression	+0.006
N-BEATS (Darts)	One-step forecasting	−0.013

Table 13.5: Weekly-frequency US Equities experiment on non-overlapping Friday-to-Friday returns, where the five-day horizon reduces to a single forecast step

Matching the data frequency to the horizon removes one source of error. However, it does not salvage the forecasting objective: direct regression still ranks better than one-step forecasting (*Table 13.5*), and both trail the daily tabular baselines, with gradient boosting reaching +0.032 on the primary one-day label. The finding restates *Section 13.7* : cross-sectional ranking is a direct prediction problem, not a path-forecasting one. It is also specific to this objective. Evaluating deep models for risk-adjusted position sizing instead and scoring on Sharpe ratio, downside risk, and cost robustness, Saly-Kaufmann et al. (2026) find hybrid recurrent and feature-selection architectures more competitive, a reminder that the target and the metric, not

the architecture alone, determine what is being tested. *Chapter 17* returns to that risk-aware framing.



Implementation :

`case_studies/us_equities_panel/12_dl_weekly` produces the results in this section.

From IC to profitability

The case studies where the deep and tabular models part ways carry that result into the portfolio-construction and simulation chapters: Crypto, where the deep model adds content the tabular families miss, and US Equities and CME Futures, where gradient boosting holds the edge. On the rest, the choice between a deep and a tabular model is a choice between roughly equal ICs at very different training and inference costs. *Chapters 16* through *19* weigh that trade-off against turnover, capacity, and transaction costs to determine which ranking yields economic profit.

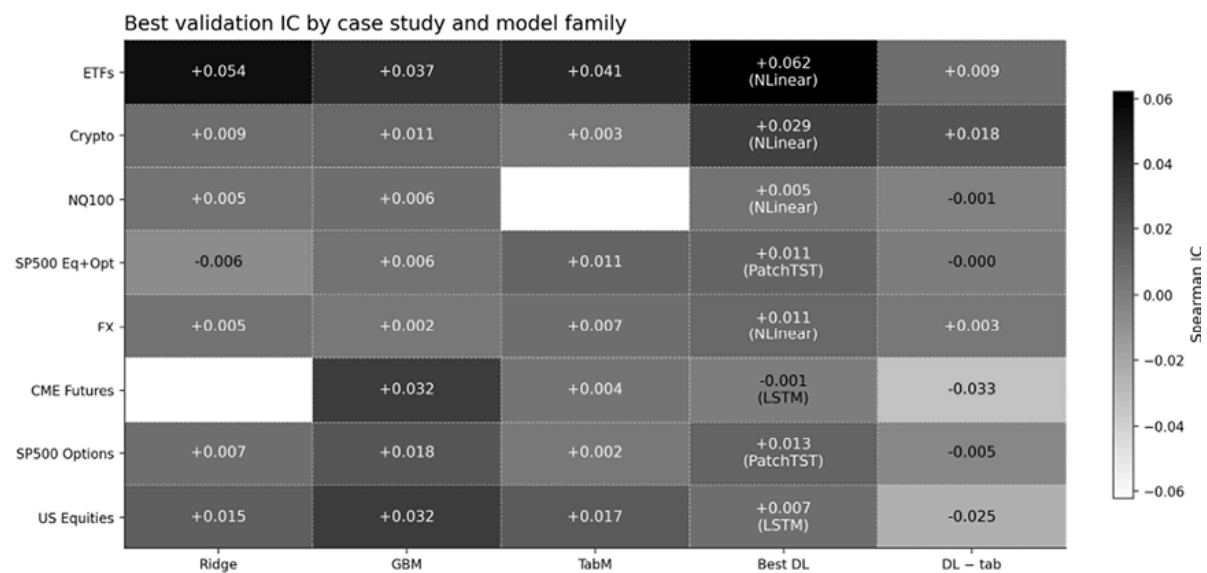


Figure 13.8: Best validation IC by family (Ridge, GBM, TabM, and the best deep model) for each case study, with the deep-minus-best-tabular delta in the final column. Shade encodes IC magnitude

13.10 Summary

Deep learning for time series has evolved from sequential RNNs through two competing philosophies, namely decomposition (N-BEATS) and attention (Transformers), into a mature toolkit where architecture choice depends on data characteristics, not benchmark rankings. The Zeng et al. critique remains a central lesson: any architecture must justify its complexity against a one-layer linear baseline. Modern responses like PatchTST and iTransformer succeed precisely because they embed stronger inductive biases rather than relying on raw model capacity.

Foundation models extend this toolkit with pretrained representations. However, in finance, they generally require domain adaptation: zero-shot inference on returns is limited, whereas parameter-efficient fine-tuning appears more promising for tasks such as volatility and risk forecasting. Evaluation also remains fragile because leakage, tuning budgets, and benchmark design can easily distort rankings. As a result, rolling-origin validation, strong non-deep baselines, and economic performance metrics remain essential.

The cross-dataset case study experiments reinforce a practical point: in the book's horizon-label, prediction-first setting, deep learning clearly beat the best tabular baseline in only one of eight case studies, while several others were statistically indistinguishable. This does not show that deep learning fails in finance; it shows that performance depends heavily on how the problem is formulated.

That leads to the chapter's central conclusion: **architecture matters only after the target is defined correctly**. Recursive one-step forecasting, direct multi-horizon forecasting, horizon-label prediction, and direct allocation learning solve different problems.

In practice, begin with Ridge and LightGBM, test whether temporal structure is genuinely informative, and only then choose a more complex architecture that matches the data and objective. Uncertainty methods such as MC

Dropout and Deep Ensembles can improve calibration for sizing and risk control. At the same time, generative and graph-based models extend the framework to scenario generation and relational data. *Chapter 14* next turns to latent factor models, which use dimensionality reduction to extract the underlying drivers of returns.

14

Latent Factor Models

Hundreds of return predictors have been proposed in the empirical asset-pricing literature. The question is not how many exist but which of them reflect a stable common structure, which are priced, and which are artifacts of specification search or portfolio construction.

This chapter approaches that problem from the perspective of latent-factor estimation. It begins with Principal Component Analysis (PCA), then moves to Instrumented PCA (IPCA) and Risk-Premium PCA (RP-PCA), and finally to conditional autoencoders (CAE) and related stochastic-discount-factor (SDF) approaches. The unifying theme: use the data to summarize high-dimensional return variation with fewer factors, while being explicit about what each method can and cannot identify.

These methods differ because they optimize different objects. PCA maximizes explained covariance variation. IPCA allows factor loadings to vary with observable characteristics. RP-PCA trades off variance fit against cross-sectional pricing performance. Conditional autoencoders generalize the loading function nonlinearly, and adversarial SDF methods target pricing restrictions more directly. Keeping these objectives distinct is essential: a factor that explains covariation is not necessarily a priced factor, and a factor that helps price assets need not be the one that explains the most variance.

By the end of this chapter, you will be able to:

- Distinguish covariance-explaining attribution factors from priced factors, and explain why that distinction matters for prediction, risk decomposition, and trading applications.

- Implement PCA on asset returns, interpret principal components as latent risk dimensions or eigenportfolios, and diagnose key practical issues including covariance noise, component selection, and loading instability.
- Explain how IPCA and RP-PCA extend PCA by introducing time-varying characteristic-based betas and pricing-error penalties, and evaluate when these extensions are preferable to plain variance maximization.
- Implement and evaluate Conditional Autoencoders using walk-forward validation, ensemble averaging, and interpretability diagnostics such as SHAP, while recognizing their main failure modes.
- Explain how adversarial SDF estimation enforces no-arbitrage restrictions, how its objective differs from CAE reconstruction, and when direct pricing-error minimization is likely to add value.
- Compare latent factor methods across datasets and modeling objectives, and choose among PCA, IPCA, RP-PCA, CAE, and SDF approaches based on dimensionality, economic goal, and evaluation design.
- Move from the from-scratch teaching implementations to the `ml4t-models` library's `LatentFactorForecastPipeline` for production use, and recognize when a model (SDF, SAE) does not fit that pipeline.

A **stochastic discount factor (SDF)**, also called a **pricing kernel**, is the object that links future payoffs to today's asset prices. An asset's price equals the expected value of its future payoff weighted by the SDF, so states of the world in which wealth is especially valuable receive a higher weight.

Formally, for a gross return $r_{i,t+1}$, the no-arbitrage condition is

$E_t[M_{t+1}r_{i,t+1}] = 1$, or equivalently $E_t[M_{t+1}r_{i,t+1}^e] = 0$, for excess returns.

The methods in this chapter pursue three different objectives, and the predictive workflow differs accordingly. PCA is a covariance-decomposition tool; eigenportfolios and yield-curve PCA inherit that role. IPCA and CAE are conditional latent-factor models that estimate factor returns together with characteristic-conditioned loadings. RP-PCA also estimates latent factors but tilts the criterion toward priced variation through a pricing-error penalty. The adversarial SDF estimates a pricing kernel directly from no-arbitrage moment

restrictions, and the supervised autoencoder learns a return forecast end-to-end without an intermediate factor representation.

Several of these models share a predictive workflow that *Section 14.5* develops as the **three-stage latent-factor forecasting adapter** :

1. **Estimate a factor representation** from realized returns and, where applicable, conditioning information.
2. **Forecast the factor premia** from the estimated factor-return history.
3. **Map factor-premium forecasts to asset-level expected returns** using current exposures.

If stages 2 and 3 are grouped as the forecasting layer, the same workflow can also be described as a two-block structure: factor estimation followed by forecasting and mapping. IPCA, RP-PCA, and CAE pass through the adapter as a matter of routine; PCA can be wrapped in it when factor-timing is the goal, but its main use in this chapter is risk decomposition. The adversarial SDF and the supervised autoencoder sit entirely outside the adapter: the SDF learns a discounting object directly, and the SAE predicts returns end-to-end. The `ml4t-models` library packages the adapter as a single composable object; the teaching notebooks derive each stage from scratch.

Section 14.1 frames latent factor models as a response to the factor zoo.

Sections 14.2–14.4 develop the linear foundation (PCA on equities, eigenportfolios, yield curves). *Section 14.5* introduces IPCA and RP-PCA and operationalizes the three-stage adapter. *Section 14.6* extends it to the nonlinear CAE and clarifies why the SDF requires a different design. *Section 14.7* walks through a full from-scratch CAE implementation and contrasts it with the supervised autoencoder. *Section 14.8* compares results across five case studies, and *Section 14.9* distills the lessons.

14.1 Making the case for latent factors

Empirical asset pricing started with a single ‘market’ factor (CAPM), grew to three (Fama and French, 1993) and then five (Fama and French, 2015), and eventually became what John Cochrane called the “factor zoo” in his 2011 AFA Presidential Address: hundreds of published factors, each claiming to explain the cross-section of expected returns (Cochrane, 2011). By the mid-2010s, the count exceeded 400 (Harvey and Liu, 2019). The question facing every practitioner is how much of this proliferation reflects genuine economic content and how much is a statistical artifact (*Figure 14.1*).

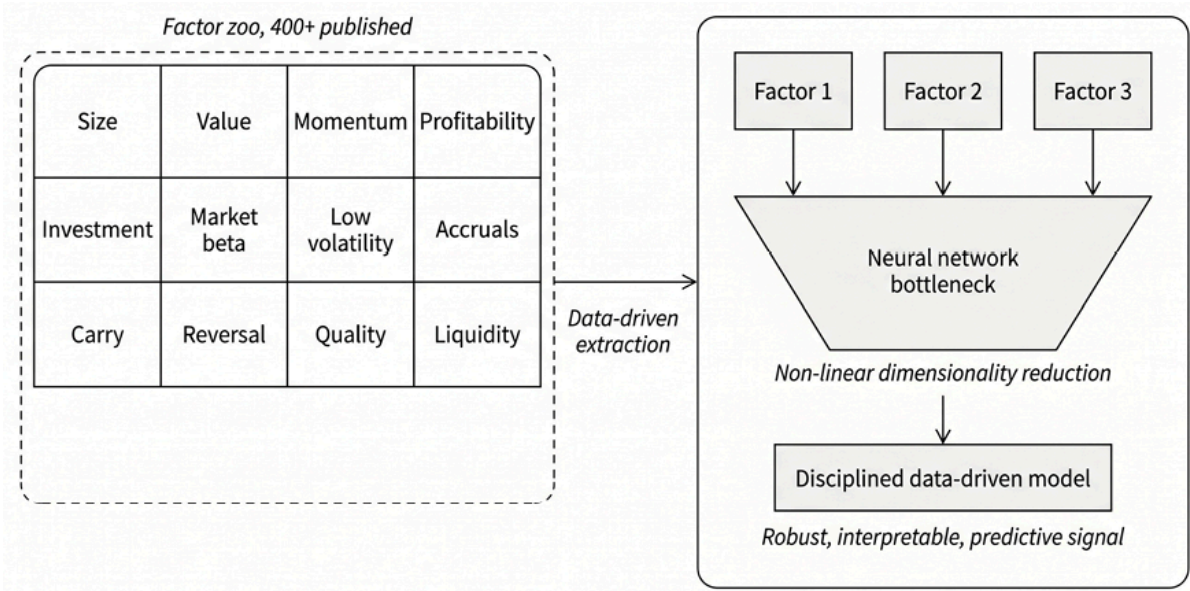


Figure 14.1: From Factor Zoo to Latent Factor Models

Academic research does not support a single, settled verdict on the factor zoo. One strand emphasizes multiple testing, publication bias, and the proliferation of published factors. Another emphasizes that replication rates can remain high once factors are evaluated within consistent frameworks and across broader datasets.

A balanced summary would be: the field has genuine false-positive and specification-risk problems, but the strongest conclusion is not that the entire factor zoo is illusory. Rather, the evidence points to a smaller, more robust set of recurring factor themes embedded in a much larger set of fragile or implementation-sensitive results.

Three axes of the factor zoo debate

The controversy runs along three axes (statistical validity, identification, and economic interpretation), drawing on work that includes Harvey (2017), Hou, Xue, and Zhang (2020), and Cochrane (2011).

Axis 1 - Statistical validity

The primary culprit is **data mining**. Harvey, Liu, and Zhu (2016) quantified the damage. When hundreds of factors have been tested across the literature, the conventional t-statistic threshold $t > 2.0$ is far too lenient. This threshold implies an estimated effect at least two standard errors away from zero, which, in turn, for large samples, corresponds to a (two-sided) p-value below 0.05. Equivalently, if the true effect were zero, the probability of observing an effect this or more extreme would be less than 5%. Instead, the authors suggest a multiple-testing adjustment (see *Chapter 7*) of around $t > 3.0$ for newly proposed factors, a rule of thumb that, if applied retroactively, eliminates a large share of published results. Harvey later stresses that even this threshold may be insufficient when effects are rare, treating the problem as one of research culture rather than a simple threshold correction.

Publication bias compounds the problem: the publication process creates strong incentives toward significant results and selective reporting. The resulting distribution of published t-statistics shows truncation below 2.0 and unusual clustering in the ranges just above it, consistent with selective reporting of marginally significant results.

A growing body of evidence, however, suggests the data-mining problem may be less severe than these headline numbers imply:

- McLean and Pontiff (2016) find that predictor returns decline by about 26% beyond the original sample period (consistent with modest overfitting), while a further 58% post-publication decline suggests that arbitrage erodes returns after discovery, a separate mechanism from statistical bias.

- Jensen et al. (2022) mount the most comprehensive challenge, developing a hierarchical Bayesian replication framework across 153 factors in 93 countries. Testing CAPM alpha rather than raw returns (the theoretically correct benchmark), they find an 82% replication rate, show that the factors cluster into roughly 13 themes that hold up globally out-of-sample, and that the evidence is *strengthened*, not weakened, by the large number of observed factors.
- Chen (2024) arrives at a consistent conclusion from a different direction, developing false-discovery bounds showing that at least 75–91% of published predictors are statistically valid and demonstrating that Harvey, Liu, and Zhu’s high false-discovery estimates stem from misinterpreting statistical insignificance as falsity.

The emerging picture is that most published factors reflect genuine statistical regularities, but whether they survive trading costs and market learning is a distinct and often more demanding question.

Axis 2 - Identification and measurement

Hou, Xue, and Zhang (2020) show why construction choices matter so much. They define **microcaps** using the 20th percentile of NYSE market equity, and when that breakpoint is applied to the full stock universe, these firms represent a majority of listed names but only a very small share of total market capitalization. Therefore, equal-weighted anomaly portfolios give microcaps disproportionate influence. Under NYSE breakpoints and value-weighted returns, they report that 65% of 452 anomalies fail even the conventional $|t| > 1.96$ hurdle, with especially severe attrition in trading-frictions variables.

Anomaly evidence is therefore highly sensitive to the definition of the universe and portfolio construction, so any comparison between named and latent factors must keep those choices fixed.

Axis 3 - Risk versus mispricing

Even among factors that survive statistical and methodological filters, the field remains divided on *why* they work: whether the return premium compensates for systematic risk or reflects persistent behavioral mispricing sustained by limits to arbitrage. Cochrane (2011) frames this as modern finance’s “dark matter” problem: we observe the gravitational pull of risk premia but cannot directly identify the specific macroeconomic shocks that cause them. This risk-versus-mispricing distinction has direct consequences for model design and evaluation, a theme that recurs throughout *Sections 14.5–14.6*.

The recurring core

Recent replication and model-selection work repeatedly returns to a compact core of factors (Hou, Xue, and Zhang, 2020; Fama and French, 2015; Barillas and Shanken, 2018):

- **Market** : The equity risk premium
- **Value** : Buying cheap assets, potentially proxying for distress or duration risk
- **Momentum** : Trend-following, reflecting behavioral underreaction or liquidity cascades
- **Profitability** : Buying productive assets, central to both the Fama-French five-factor model and the q-factor model
- **Investment** : Buying conservative assets, consistent with low discount rates in production models

This compact set is not a definitive list, but it is striking that independent lines of research converge on it. Fama and French’s five-factor model derives its factors from the dividend discount model; Hou, Xue, and Zhang’s (2015) q-factor model derives its factors from the firm’s investment optimization (the Investment CAPM), later augmented with an expected-growth factor (Hou et al., 2021). Both arrive at overlapping empirical predictions centered on profitability, investment, and valuation.

Model-comparison work by Barillas and Shanken (2018) finds that both factor families are dominated by specifications that include momentum, further

reinforcing the recurrence of this core. Jensen et al. (2022) find that their 153-factor universe clusters into roughly 13 themes, and Swade et al. (2023) show that about 15 factors suffice to span the alpha of the full set, consistent with a true dimensionality much smaller than the zoo but somewhat larger than the five highlighted above.

Machine learning reinforces this picture. Gu, Kelly, and Xiu (2020) found that neural networks' dominant predictive inputs are familiar (price trends, liquidity, accounting quality), and that gains come from nonlinear interactions among core factors rather than esoteric new signals. Feng et al. (2020) use a double-selection LASSO (*Chapter 15*) to show that most proposed new factors are redundant relative to the existing menu, with profitability and investment retaining the clearest incremental content, a finding about disciplined testing rather than a definitive reduction to a handful of labels.

From the zoo to latent factors

A key conceptual distinction clarifies what latent factor models can and cannot resolve. Following Idzorek, Kaplan, and Ibbotson (2024):

- **Attribution factors** explain co-movement among assets but may carry zero expected return
- **Priced factors** emanate from asset pricing models and carry genuine risk premia

PCA is designed to extract common variation in returns, which may or may not line up with priced expected-return factors. The three objectives introduced previously organize methods that progressively tighten this link: variance maximization (*Sections 14.2–14.4*), variance plus pricing errors (*Section 14.5*), and direct pricing-error minimization (*Section 14.6*).

Latent factor models do not make the factor-zoo debate disappear. They change the object of selection. Instead of proposing and testing a long list of named characteristics one at a time, they infer a low-dimensional structure directly from returns, sometimes augmented by pricing restrictions or

observable characteristics. This reduces one source of researcher discretion, but it does not, by itself, guarantee that the extracted factors are priced, economically interpretable, or stable out-of-sample.

The methods that follow are complementary responses to factor proliferation: powerful tools for discovering structure, not clean escape hatches from the data-mining problem.

14.2 Extracting latent factors with PCA

PCA solves the first of the three latent factor objectives introduced in *Section 14.1* : maximizing explained variance. It transforms a set of correlated asset returns into uncorrelated components that capture maximum variance (Goyal, 2012).

The transformation relies on eigendecomposition of the return covariance matrix, a standard linear algebra operation with direct financial interpretation. As a pure variance-maximization method, PCA finds the dominant directions of co-movement but is agnostic to whether those dimensions carry risk premia, a limitation that *Section 14.5* addresses.

The mechanics of eigendecomposition

The PCA procedure begins with a matrix of asset returns (time periods as rows, assets as columns). Returns are demeaned; standardizing to unit variance is optional but standard for cross-sectional work (see *Assumptions and design choices* , the following subsection).

From the standardized return matrix, we compute an $N \times N$ covariance matrix Σ . When inputs are standardized to unit variance, this yields the correlation matrix, a distinction that matters for interpretation, taken up in the next subsection. PCA then solves the eigenvalue problem: finding all pairs $(\lambda, \mathbf{v})$ such that $\Sigma \mathbf{v} = \lambda \mathbf{v}$.

The solution yields two key outputs. **Eigenvalues** (λ_i) are scalars representing the magnitude of variance explained by each corresponding principal component. They are ranked in descending order, so $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. The proportion of total variance explained by the i -th component equals $\lambda_i / \sum \lambda$. A “scree plot” shows eigenvalues in descending order; the “elbow,” where explained variance levels off, suggests how many components to retain (*Figure 14.2*).

`01_pca_equity_sectors` constructs this scree plot for sector ETFs and adds bootstrap confidence intervals on the loadings to assess which sector exposures are statistically reliable.

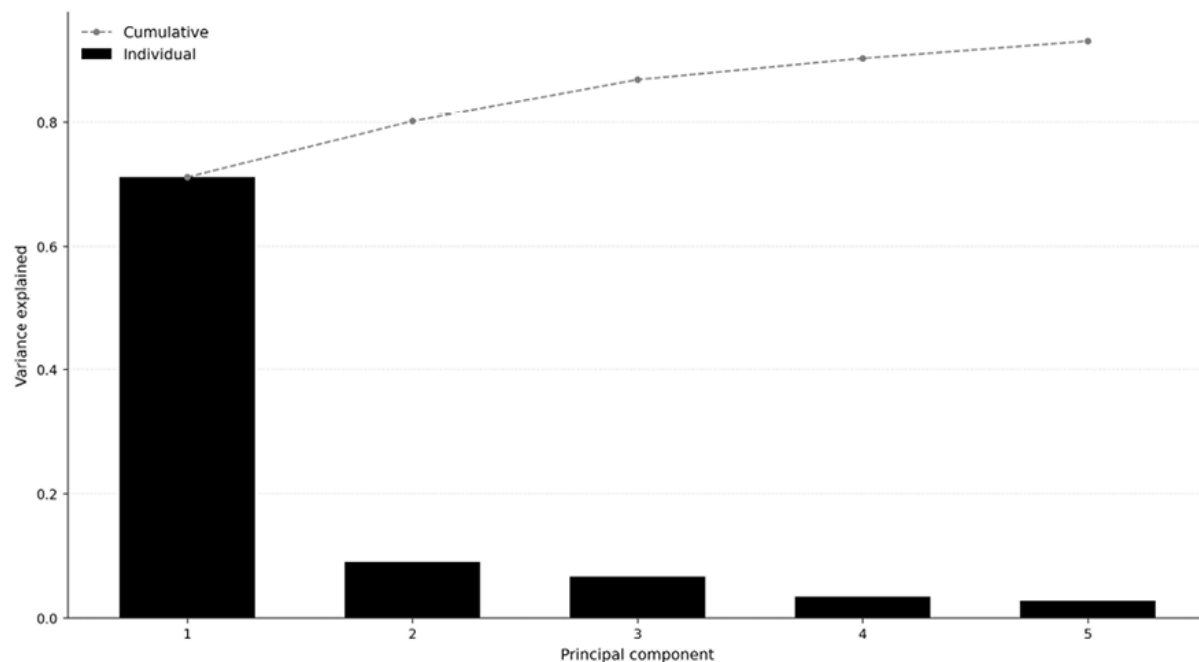


Figure 14.2: Variance explained by the top 5 principal components

Eigenvectors (v_i) are vectors of “loadings” that define each principal component as a linear combination of the original asset returns. These vectors are mutually orthogonal, meaning the resulting principal components are uncorrelated with one another. In finance, each eigenvector can be interpreted as portfolio weights, a concept we develop fully in the next section on eigenportfolios. `02_eigenportfolios` demonstrates this interpretation on a broad US equity universe.

Assumptions and design choices

PCA is a useful tool, but its interpretation in finance depends on several important assumptions.

- **Linearity** . PCA captures only linear patterns of co-movement. In financial markets, however, relationships often change across regimes, especially during stress, and many interactions among characteristics are non-linear. For example, the effect of momentum may depend on volatility or liquidity. Rolling-window PCA can partially adapt to temporal variation, but it does not address the underlying linearity constraint. *Section 14.6* returns to this limitation when we introduce autoencoders. The notebook `01_pca_equity_sectors` illustrates rolling PCA for sector ETFs.
- **Variance is not the same as pricing relevance** . PCA is designed to explain as much return variation as possible, not to identify factors with high expected returns. A factor can explain a large share of covariance yet earn little or no risk premium; conversely, a factor with modest variance can be economically important if it commands a large premium. This gap between covariance structure and pricing relevance motivates RP-PCA in *Section 14.5* .
- **Focus on second moments** . PCA works entirely through the covariance matrix of centered returns, so it uses only second-moment information. Under Gaussian assumptions, this aligns with the maximum-likelihood estimation of a linear factor model. Financial returns, however, are rarely Gaussian: they are often skewed, heavy-tailed, and prone to jumps. In those settings, PCA still finds directions of maximum variance, but it may miss structure that appears in higher moments and may still matter economically.

Two key design choices deserve a closer look.

Covariance or correlation?

A basic implementation choice is whether to apply PCA to the covariance matrix or the correlation matrix.

- Using the **covariance matrix** preserves the natural scale of returns, so more volatile assets receive more weight in the decomposition
- Using the **correlation matrix** first standardizes each asset to unit variance, preventing high-volatility assets from dominating the results

For cross-sectional equity analysis, correlation-based PCA is usually preferred because it emphasizes common structure rather than differences in raw volatility. For multi-asset portfolios with markedly different volatilities, the choice should be deliberate rather than by default. The notebook

`01_pca_equity_sectors` uses correlation-PCA for the sector ETF example.

Idiosyncratic volatility normalization

A useful third option goes beyond the simple covariance-versus-correlation choice: normalize each asset's returns by its lagged idiosyncratic volatility before running PCA.

This adjustment targets a specific problem. In raw covariance PCA, high-volatility assets can dominate the decomposition. Correlation PCA solves that problem by forcing all assets to have equal total variance, but that is often too strong: it removes not only idiosyncratic volatility differences but also meaningful differences in common-factor exposure. Idiosyncratic volatility normalization is more selective. It scales returns only by the diversifiable component of volatility, leaving common variation more intact.

In practice, this often leads to cleaner separation between signal and noise, more stable eigenvectors over time, and more parsimonious factor structures, meaning that fewer components explain the same share of variation. To implement this correctly, idiosyncratic volatility must be estimated from residuals in a prior estimation window, and current returns must be normalized using only that lagged information. This preserves the point-in-time structure and avoids lookahead bias.

Covariance estimation quality

A practical warning: PCA is only as reliable as the covariance matrix it decomposes. When the number of assets N , approaches or exceeds the number of time periods, T , the sample covariance matrix becomes difficult to estimate. In that regime, sample eigenvalues are distorted upward, weak factors become hard to distinguish from noise, and estimated eigenvectors can become unstable across samples.

Random matrix theory provides a useful benchmark for separating signal from noise in large covariance matrices. In particular, the **Marchenko-Pastur distribution** characterizes the range of eigenvalues that can arise from noise alone, given the N/T ratio. Eigenvalues that fall within this range should be treated cautiously, since they may not reflect genuine common structure.

A common remedy is to use **shrinkage estimators**, with the **Ledoit-Wolf** family being the best-known example. These methods pull extreme sample eigenvalues toward a more structured target, reducing estimation error and improving stability. For large equity cross-sections, applying shrinkage before PCA is often advisable; otherwise, the leading eigenportfolios may reflect sampling noise rather than persistent market structure. See the Primer for a fuller discussion of the Marchenko-Pastur benchmark and shrinkage-based covariance estimation.

How many PCA factors are real?

When the number of assets, N , is large relative to the number of time periods, T , PCA starts to pick up noise as well as signal. In this high-dimensional setting, sample eigenvalues are biased upward, and weak factors can become indistinguishable from random variation (noise).

The **Baik-Ben Arous-Péché (BBP)** phase transition (Baik et al., 2005; see Paleologo, 2025, Ch. 7) quantifies this: a factor is recoverable only if its population eigenvalue exceeds $1 + \sqrt{\gamma}$, where $\gamma = N/T$ and eigenvalues are

normalized to unit idiosyncratic variance. Below this threshold, no estimation technique can separate signal from noise.

The practical message is simple: the larger n/T becomes, the harder it is to recover anything except the strongest factors. In our case studies, this works out quite differently across datasets. For ETFs, where $\gamma \approx 0.2$, the threshold is relatively low, so several components can be retained. For a 500-stock equity panel with only one year of daily data, $\gamma \approx 2.0$, and only the market plus a few dominant sector factors are likely to survive. For CME futures, where $\gamma \approx 0.06$, most factors remain identifiable.

A good rule of thumb is therefore to keep n modest relative to T whenever possible. If that is not feasible, expect PCA to recover only the dominant part of the factor structure. When γ becomes moderately large, eigenvalue shrinkage is usually more reliable than a visual scree-plot rule for deciding how many components to keep. See the Primer for details on the BBP threshold, the Marchenko-Pastur benchmark, and shrinkage-based alternatives.



Implementation : See `@1_pca_equity_sectors` for rolling PCA and bootstrap stability analysis.

Next, we discuss how to use the eigendecomposition techniques to construct equity portfolios.

14.3 Eigenportfolios for equity strategies

Reading eigenvectors as portfolio weights yields uncorrelated “eigenportfolios,” building blocks for risk management and systematic trading (Avellaneda and Lee, 2010). See `@2_eigenportfolios` for the full PCA pipeline, including universe selection, variance decomposition, and market factor validation.

Constructing and interpreting eigenportfolios

The first eigenportfolio, corresponding to the largest eigenvalue, captures the dominant source of variance. Empirically, nearly all weights are positive: it acts as a data-driven proxy for the broad market portfolio.

In our analysis of the 500 most liquid stocks by dollar volume from the US Equities Panel dataset, the first principal component (PC1) correlates with the equal-weighted market return by 0.99, confirming that it serves as a data-driven market proxy. Each stock's loading on PC1 measures its exposure to this statistically extracted market mode. This is analogous to the **CAPM beta** in the sense that both quantify sensitivity to a common factor. Unlike CAPM beta, however, the PC1 loading is not estimated relative to a prespecified market portfolio and does not carry an asset-pricing interpretation; it follows mechanically from the sample covariance matrix and the PCA normalization.

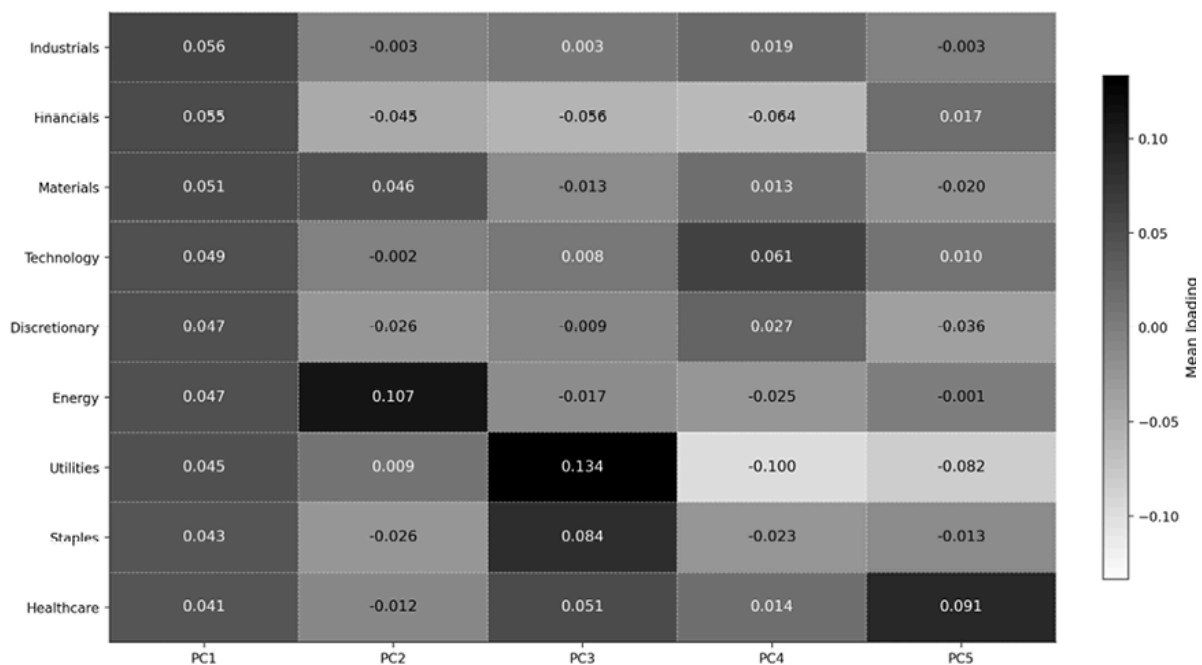


Figure 14.3: Eigenportfolio loadings across sectors

Whereas the first eigenportfolio *PC1* loads positively on all sectors, PC2 reveals a growth-versus-value rotation, loading positively on Technology and

Communication Services while negatively on Financials and Energy. PC3 captures defensive-versus-cyclical dynamics (see *Figure 14.3*). These **interpretable patterns** emerge purely from the return covariance structure, without any imposed economic labels.

Higher-order eigenportfolios are, by mathematical construction, orthogonal to each other. **Orthogonality** ensures that the extracted component returns are uncorrelated in the sample. Higher-order eigenportfolios are often approximately market-neutral in practice, but they are not automatically dollar- or beta-neutral. These portfolios are typically long-short in nature (positive weights in some stocks, negative in others) and often capture intuitive economic exposures such as sector rotations or style tilts. For example, the second eigenportfolio might represent a long position in growth stocks against a short in value stocks, or a long in technology against a short in utilities.

Stationarity is one caveat to consider. Eigenportfolio loadings can rotate significantly across estimation windows. The second component might capture growth-versus-value in one decade and defensive-versus-cyclical in the next, as sector correlations shift with the macroeconomic regime. This instability limits the use of eigenportfolios in live trading without periodic re-estimation, and it means that economic interpretations of higher-order components should be treated as descriptive rather than structural.

Statistical factors can also be interpreted via regression. Statistical PCA factors are abstract by construction: they maximize variance explained but carry no economic label. A practical interpretation method: regress each eigenvector's loadings cross-sectionally on observable firm characteristics such as sector indicators, market capitalization, momentum, and book-to-market ratio (Connor and Korajczyk, 2009). The regression R^2 indicates how much of the variation in each statistical factor is explained by known fundamentals. If PC1 loadings correlate strongly with sector dummies, it is effectively a sector factor under a different name. If PC4 shows low R^2 against all available characteristics, it may capture a genuinely novel latent risk dimension,

precisely the type of structure that justifies data-driven factor discovery over pre-specified models.

Applications in quantitative finance

Annualized return, volatility, and Sharpe ratio statistics for each eigenportfolio quantify the risk-return profile of each latent factor: the first component typically shows the highest volatility while higher-order components may offer modest risk-adjusted returns.

Any portfolio's risk decomposes into exposures to these orthogonal factors, revealing concentrations that pre-specified factor models miss: a manager might discover that apparent sector diversification masks exposure to a single statistical factor. This decomposition feeds directly into *Chapter 17*'s portfolio construction: eigenportfolio betas serve as the risk dimensions over which position sizing is optimized.

Regressing stock returns on the top K eigenportfolios isolates residuals that are hypothesized to mean-revert; the eigenportfolio return series can also serve as inputs for factor-timing models. Both applications face practical headwinds: mean-reversion parameters are unstable out-of-sample, and evidence for reliable short-horizon factor timing remains mixed.

Improving interpretability with hierarchical PCA

A common critique of standard PCA is that higher-order factors lack a clear economic interpretation and can be statistically unstable across different time periods. Marco Avellaneda's **Hierarchical PCA (HPCA)** addresses this by injecting known economic structure into the analysis (Avellaneda, 2019).

HPCA proceeds in two steps. First, PCA is performed independently *within* each known economic grouping, such as GICS sectors. This identifies the dominant factor for technology stocks, another for financials, and so forth. Second, another PCA is applied to the correlation matrix of these sector-level

factors to capture cross-sector dynamics. The resulting factors are much easier to interpret: they correspond to clear inter-sector bets (long financials versus short industrials) or intra-sector momentum effects, resolving much of the ambiguity of standard PCA. The notebook `02_eigenportfolios` demonstrates both standard eigenportfolio construction and the HPCA two-step procedure on GICS sectors.

Fixing eigenvector instability

A persistent practical challenge with rolling PCA is that factor loadings can change abruptly between adjacent estimation windows, not because the underlying factor structure has shifted, but because of a mathematical pathology. Understanding and correcting this instability is essential before using PCA loadings for portfolio allocation or performance attribution.

Consider the problem of near-degenerate eigenvalues. When two eigenvalues are close in magnitude, the corresponding eigenvectors become poorly identified. A small perturbation (adding or removing a single day of returns, or changing one stock in the estimation universe) can cause the eigenvectors to rotate within their shared subspace or swap entirely. In rolling PCA on equity returns, this manifests as factor loadings that “flip sign” between adjacent estimation windows: sector exposures suddenly invert, creating phantom turnover in downstream portfolio weights. The first principal component is typically immune (its eigenvalue is well-separated), but components 3 and beyond frequently exhibit this instability (Paleologo, 2025).

Measuring instability is vital. Cosine similarity between consecutive eigenvectors provides a simple diagnostic. For eigenvectors $v(t)$ and $v(t + 1)$, cosine similarity equals $|v(t)^T v(t + 1)|$ (taking the absolute value because eigenvectors are identified only up to sign). A value of 1 indicates stable loadings; values near 0 indicate rotation into a different subspace. Track this metric for each principal component across estimation windows. Components with frequent drops below 0.8 are unreliable for attribution or allocation without correction.

A practical fix is to align loading matrices across adjacent windows by an **orthogonal Procrustes rotation** :

- Let B_t and B_{t+1} denote the $N \times K$ loading matrices from two consecutive estimations
- Compute the singular value decomposition of $B_t^\top B_{t+1} = USV^\top$
- The orthogonal matrix that best aligns the two loading spaces is $R^* = VU^\top$, and the aligned next-window loadings are $\tilde{B}_{t+1} = B_{t+1}R^*$

This transformation preserves orthogonality and factor span, but it removes arbitrary rotations within nearly degenerate eigenspaces. The result is not a different covariance model; it is the same model written in a temporally smoother coordinate system.

Always check eigenvector stability before using PCA loadings for allocation or attribution:

- If the top 3–5 principal components have well-separated eigenvalues (ratios exceeding 2:1 between consecutive eigenvalues), Procrustes rotation is unnecessary: the loadings are naturally stable
- If eigenvalues are clustered, which is common for equity returns beyond the third component, always apply Procrustes

The stability diagnostic connects directly to *Chapter 17*’s portfolio construction: unstable loadings produce an unstable covariance matrix, which generates unnecessary turnover, and that turnover incurs real transaction costs.

Practitioner recipe for two-stage production PCA

The notebooks in this chapter demonstrate PCA on a single estimation window with uniform time weighting. Risk models used by institutional investors apply a more sophisticated procedure that separates two empirical facts about financial markets: volatility changes quickly (days to weeks), while the correlation structure changes slowly (months to quarters). Mixing both

dynamics in a single estimation window either overreacts to volatility spikes or underreacts to shifts in correlation.

The following two-stage procedure, formalized by Paleologo (2025), addresses this separation:

1. **Capture volatility dynamics** . Apply exponentially-weighted time weighting (half-life ~ 20 days) to emphasize recent volatility. Run a preliminary PCA ($p \approx 5$ components), compute residuals, and estimate per-asset idiosyncratic volatility.
2. **Estimate the correlation factor structure** . Normalize returns by Stage 1's idiosyncratic volatility, apply slow exponential weighting (half-life ~ 120 days), and perform the production PCA. Apply BBP-informed eigenvalue shrinkage (*Section 14.2*) and Procrustes rotation to stabilize loadings. Reconstruct the full covariance matrix:

$$\Omega = D_{\sigma}(B\Omega_f B^T + \Omega_{\varepsilon})D_{\sigma}, \text{ where } D_{\sigma} \text{ restores the original volatility scale.}$$

This procedure matters for the ML4T pipeline because it produces the covariance matrix that feeds directly into the portfolio construction methods of *Chapter 17* . Mean-variance optimization, hierarchical risk parity, and risk budgeting all consume a covariance estimate as their primary input. The separation of volatility and correlation estimation also connects to *Chapter 18* 's short-term volatility updating for transaction cost models. The two-stage structure ensures that a sudden volatility spike (for example, a VIX event) immediately updates risk estimates without destabilizing the slower-moving factor structure that determines diversification benefits. Next, we apply Principal Component Analysis to the yield curve to identify fixed income factors.



Implementation : See `02_eigenportfolios` for eigenportfolio construction, including sector loading heatmaps, HPCA, and cumulative return visualization.

14.4 Decoding the yield curve

PCA's clearest success is in fixed income. Litterman and Scheinkman (1991) showed that three factors explain 95–99% of variation in the Treasury yield curve, a result that has been replicated across decades and markets.

Level, slope, and curvature

When PCA is applied to daily or monthly changes in Treasury yields across maturities (from 1-month bills through 30-year bonds), studies consistently find that 95–99% of variation can be attributed to three orthogonal principal components, each with a widely-accepted economic interpretation (*Figure 14.4*):

- **Level** : Explains over 90% of the total variance. Loadings are nearly uniform across maturities, so a level shock corresponds to a parallel shift, all rates moving up or down together. Economically, this reflects broad changes in inflation expectations and real growth prospects.
- **Slope** : Explains 5–8% of variance. Loadings have opposite signs at short and long maturities, so a slope shock steepens or flattens the curve. This factor is closely tied to monetary policy: rate hikes compress the short-long spread, flattening the curve.
- **Curvature** : Captures 1–2% of variance. Mid-curve maturities move opposite to both ends, creating a “butterfly” twist. This factor is linked to rate volatility and path uncertainty. Observing it clearly requires at least 5–7 maturities spanning the curve; with fewer, the third component captures residual variation rather than a clean butterfly.

Three principal components are shown in *Figure 14.4* :

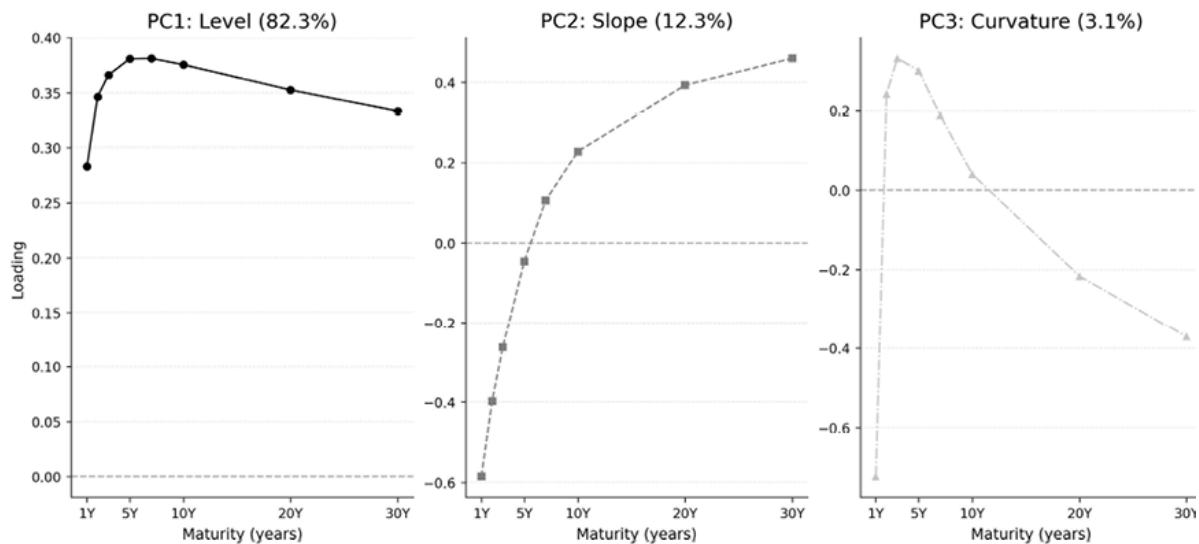


Figure 14.4: Principal component analysis on yield curve components



Implementation : `@3_yield_curve_decomposition` applies PCA to daily changes in eight Treasury constant-maturity yields (1Y, 2Y, 3Y, 5Y, 7Y, 10Y, 20Y, 30Y), which recovers the classical Level/Slope/Curvature decomposition with variance shares of 82.3%, 12.3%, and 3.1% (97.8% cumulative). Each maturity is reconstructed from the three factors, and the generalized-duration hedging framework is illustrated on the resulting exposures.

Why PCA works for yield curves

The yield curve is the rare case where the variance objective and the pricing objective largely coincide: no-arbitrage constraints in fixed income aim to ensure that variance-explaining factors are also priced. The contrast with equities highlights a structural difference:

- Interest rate drivers are low-dimensional and persistent: inflation expectations, real growth, and monetary policy affect all bonds simultaneously, with sensitivity varying by maturity. At the same time, PCA on yield changes is a descriptive decomposition of curve movements, not a proof that the same components are the uniquely priced

term-premium factors. No-arbitrage term-structure models and return-predictive term-premium models add further structure that PCA alone does not impose.

- Equities are the opposite: thousands of stocks driven by idiosyncratic news, sector effects, macro shocks, and behavioral factors produce a covariance structure that is complex, time-varying, and regime-dependent, explaining why PCA on equities yields less interpretable and less stable factors.

Practical application for efficient hedging

This low-dimensional structure enables efficient hedging. Rather than managing exposure to dozens of individual bonds, a manager neutralizes three factor exposures, namely level, slope, and curvature, using liquid instruments (Treasury futures, interest rate swaps). This **generalized duration** approach is more precise, cheaper in transaction terms, and grounded in the yield curve's empirical low-rank structure.

Factor	Variance Explained	Movement Pattern	Standard Interpretation
Level (PC1)	>90%	Parallel shift	Inflation, growth expectations
Slope (PC2)	~5-8%	Steepening/flattening	Monetary policy stance
Curvature (PC3)	~1-2%	Butterfly twist	Rate volatility, uncertainty

Table 14.1: Principal components of the yield curve

Economic interpretations are widely used heuristics, not definitive causal mappings, as shown in *Table 14.1*. Exact percentages depend on the maturity

set, sample period, and whether PCA is applied to the correlation or covariance matrix of yield changes.

Next, we explore how to enrich the statistical techniques introduced so far with economic data to make them more useful in practice.

14.5 Bridging economics and statistics with advanced models

The yield curve's low-dimensional success highlights PCA's limitations for equities: high dimensionality, weak structural constraints, and time-varying relationships produce factors that explain variance but not necessarily returns. Two innovations address this:

- **IPCA** (Kelly, Pruitt, and Su, 2019) makes factor loadings dynamic through observable characteristics
- **RP-PCA** (Lettau and Pelger, 2020) explicitly targets pricing-relevant factors

Before deriving either model, we explain the predictive structure they share.

The three-stage latent-factor forecasting adapter

Several factor-representation models in this chapter can be used for prediction by adding a forecasting layer. PCA, RP-PCA, IPCA, and CAE all estimate latent factors, but they do so for different purposes: PCA explains covariance, RP-PCA tilts factor extraction toward priced variation, IPCA estimates linear characteristic-conditioned loadings, and CAE generalizes that loading map nonlinearly. To turn these representations into ex-ante asset-level forecasts, the **three-stage latent-factor forecasting adapter** wraps each estimator in the same workflow (*Figure 14.5*):

1. **Stage 1: Factor estimation** . Compress the $T \times N$ excess-return panel R^e into a $T \times K$ factor history F and a per-asset loading map. PCA solves this by SVD on R^e ; RP-PCA modifies the criterion by adding a pricing-error penalty; IPCA and CAE parameterize the loading map $\beta_{i,t}$ as a function of lagged characteristics. Stage 1’s training objective (variance, variance-plus-pricing-error, or cross-sectional reconstruction) uses contemporaneous returns; this is what the structural model is for.
2. **Stage 2: Factor-premium forecaster** . Predict $\hat{\lambda}_{t+1} \in \mathbb{R}^K$ from the training-window factor history $F_{1:t}$. The simplest choice, used implicitly by KPS (2019) and GKX (2020) and labeled “IID-BS” by Engel et al. (2025), is the training-sample mean. The library’s `ExpandingMeanFactorForecaster` implements this baseline. AR(1), EWMA, gradient-boosted regression (LightGBM, an analog to Engel et al.’s Q-Boost), and pretrained foundation models (their ZS-Chronos) are drop-in alternatives that target the same object with progressively richer representations of factor dynamics. *Figure 14.6* catalogs the forecasters used in this chapter.
3. **Stage 3: Asset map** . Combine today’s exposures with the forecast premium: $\hat{\mu}_{i,t+1} = \beta_{i,t} \cdot \hat{\lambda}_{t+1}$

If Stages 2 and 3 are grouped as a single forecasting layer, the same workflow reads as a two-block structure: factor estimation followed by forecasting and mapping. The numbered three-stage formulation is the one used consistently throughout this chapter.

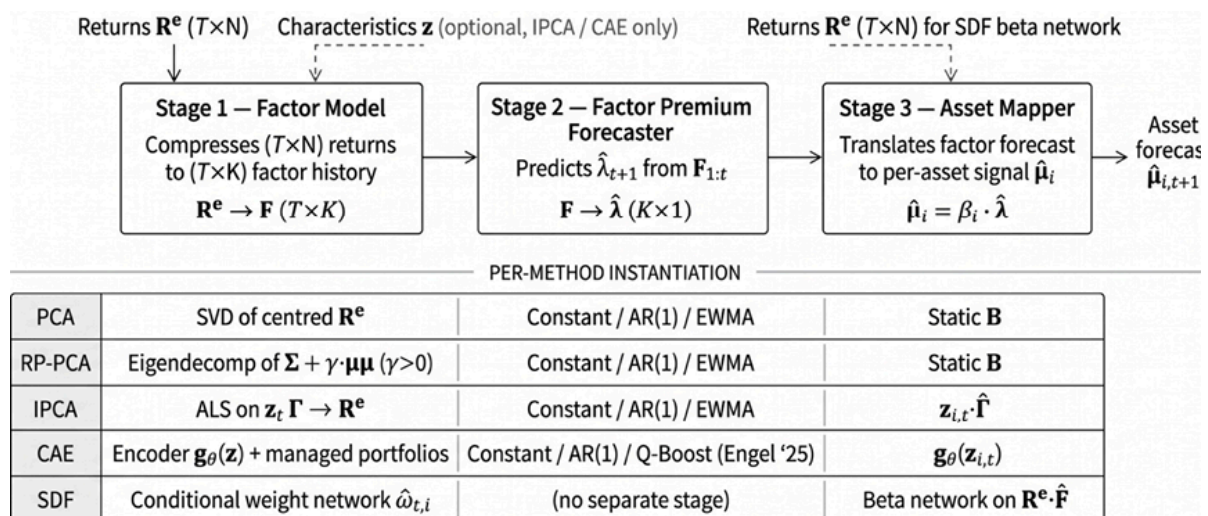


Figure 14.5: The three-stage latent-factor forecasting adapter. Stage 1 compresses the excess-return panel into a factor history and a per-asset loading map; Stage 2 forecasts the next-period factor premium from that history; Stage 3 combines current loadings with the forecast premium to produce asset-level expected returns. RP-PCA, IPCA, and the CAE differ only in Stage 1

Two design properties follow. First, Stage 1 and Stage 2 are *separable*: the same structural model can pair with any forecaster, and the same forecaster works across structural models. The chapter notebooks exploit this by sharing the Stage 2 catalog across IPCA, RP-PCA, and CAE. Second, forecast quality enters at Stage 2: better forecasts of the factor premium translate directly into better asset-level signals through Stage 3.

The adversarial SDF and the supervised autoencoder do not pass through this adapter. The SDF estimates a pricing kernel directly from no-arbitrage moment restrictions and produces pricing-error diagnostics rather than a factor-return history; *Section 14.7* shows why a separate Stage 2 forecast is not the natural output. The supervised autoencoder predicts returns end-to-end and has no factor-return intermediate to forecast; *Section 14.7* presents it as a direct-prediction contrast.

Engel et al. (2025) extend the adapter by ranking factors by forecast *uncertainty* and using only the most predictable ones in tangency portfolios; that extension lives at Stage 2 and is also a natural place to plug in modern probabilistic forecasters.

The `ml4t-models` library packages the adapter as a single composable object, `LatentFactorForecastPipeline(model, forecaster, mapper)`. The teaching notebooks are built from scratch at each stage; the library is the production format used by the case-study runners.

Dynamic betas with instrumented PCA

Standard PCA estimates a single, static beta per asset, an unrealistic assumption when risk profiles evolve. Instrumented PCA (IPCA; Kelly, Pruitt, and Su, 2019) makes betas time-varying by modeling them as functions of observable characteristics: size, book-to-market, momentum, and others.

The IPCA model specifies excess returns as:

$$r_{i,t+1}^e = \mathbf{z}_{i,t}^\top \Gamma \mathbf{f}_{t+1} + \varepsilon_{i,t+1}$$

where $\mathbf{z}_{i,t}$ is a $P \times 1$ vector of characteristics observed at time t , Γ is a $P \times K$ matrix, and $\mathbf{f}_{t+1}$ is the $K \times 1$ vector of latent factor returns. Equivalently, $\beta_{i,t} = \Gamma^\top \mathbf{z}_{i,t}$, so characteristics govern time-varying loadings rather than expected returns directly. This timing matters. Characteristics must be lagged relative to returns, and the interpretation is conditional risk exposure rather than a reduced-form anomaly regression.

The central insight is that *characteristics are covariances*: firm characteristics predict returns not because they represent standalone anomalies but because they proxy for time-varying exposures to latent risk factors. A small-cap value stock loads on different factors than a large-cap growth stock, and loadings update as characteristics change.

This interpretation enables a direct empirical test of risk versus mispricing:

- If characteristics primarily explain returns through their role in shaping risk exposures, IPCA identifies strong latent factors and insignificant pricing errors (alphas)
- If characteristics represent pure mispricing independent of risk, the model attributes their effect to a significant alpha term

Kelly, Pruitt, and Su (2019) report that a five-factor IPCA model explains cross-sectional returns far more accurately than existing models, with characteristic effects almost entirely absorbed by their role as instruments for risk exposures.

The original Kelly, Pruitt, and Su (KPS) study uses 94 characteristics; our `us_firm_characteristics` dataset provides 57 after filtering. Missing characteristics should be imputed (using the cross-sectional median is standard) rather than dropping observations, since missingness is informative: smaller firms tend to have fewer available characteristics.

IPCA estimation uses **Alternating Least Squares** (**ALS**), iterating between updating Γ (the mapping from characteristics to loadings) and updating f_t (the latent factors). The number of characteristics P matters: too few limit the model's ability to capture time-varying loadings; too many introduce estimation noise.

Three sensitivities regarding identification recur across post-2019 replications:

- Results depend on the number of latent factors K ; report a range (typically 3–8) rather than selecting a single optimum, since the standard out-of-sample pricing-error rule is sensitive to the test-asset set (Didisheim et al., 2023, show that more complex factor models can outperform out-of-sample).
- Missing characteristics are absorbed into residuals rather than systematic risk. The 57 characteristics in our `us_firm_characteristics` implementation (down from 94 in the original KPS study) are a floor, not a ceiling.
- The linear map $\beta = \Gamma^\top z$ cannot capture interactions (momentum's behavior conditional on volatility, size effects conditional on liquidity); *Section 14.6* 's non-linear extensions trade identification clarity for expressive power.



Implementation : `04_ipca` includes a parameter-recovery exercise on synthetic data that confirms ALS



estimation recovers the true Γ before applying the model to real data, and directly demonstrates K -sensitivity.

Finding priced factors with risk-premium PCA

The second PCA flaw (focusing on variance rather than expected returns) is addressed by RP-PCA (Lettau and Pelger, 2020). The defining feature of RP-PCA is not a different loading map but a different objective: standard PCA is augmented with a pricing-error penalty so that factors that explain small amounts of variance yet carry meaningful risk premia are not overlooked.

RP-PCA minimizes a weighted combination of unexplained variance and cross-sectional pricing errors:

$$\min_{\Lambda, F} \frac{1}{NT} \sum_{i,t} (\tilde{X}_{i,t} - \tilde{F}_t^\top \Lambda_i)^2 + \kappa \frac{1}{N} \sum_i (\bar{X}_i - \bar{F}^\top \Lambda_i)^2$$

where the first term is the usual reconstruction loss on time-demeaned returns and the second term penalizes pricing errors in mean returns.

The parameter κ controls the trade-off. At $\kappa = 0$, RP-PCA reduces to standard PCA, which maximizes pure variance with no regard for pricing. As κ increases, the objective shifts toward factors that explain cross-sectional return differences, even if they explain little of the total variance. In the limit $\kappa \rightarrow \infty$, the method focuses entirely on pricing and ignores variance altogether.

The objective is solved by applying standard PCA to a modified matrix, $((1/T)X^\top X - \bar{X}\bar{X}^\top) + \kappa \bar{X}\bar{X}^\top$, the sample covariance plus a penalty that overweights the mean-return information (so $\kappa = 0$ recovers ordinary covariance PCA). The result is factors that are both statistically significant (explaining co-movement) and economically significant (carrying high Sharpe ratios).

A factor capturing credit risk might explain only 2% of the return variance yet have a Sharpe ratio of 0.8, remaining invisible to standard PCA, which ranks it below the 15th component. RP-PCA with $\kappa = 5$ elevates it to the 3rd. Lettau and Pelger (2020) report out-of-sample Sharpe ratios more than double those of standard PCA on US equities, 1963–2017; results are sample-dependent and require out-of-sample validation.

Note that **double-selection LASSO** (Feng, Giglio, and Xiu, 2020), the modern standard for testing new candidate factors against an existing zoo, is a special case of the debiased ML framework developed in *Chapter 15*, applied to the factor returns from this chapter's case studies.



Implementation : `05_rp_pca` for a comparison of standard PCA against RP-PCA across multiple penalty values. The notebook's `gamma` parameter is this κ , and `gamma=0` reproduces standard PCA. Lettau and Pelger write the penalty weight as $1 + \gamma$ with PCA recovered at $\gamma = -1$; the chapter's $\kappa = 1 + \gamma$ shifts this so that $\kappa = 0$ is the no-penalty case.

Test assets as a design choice

Test assets are a design choice, not a neutral backdrop. Recent work shows that factor strength depends on the span of the test assets used in estimation and evaluation:

- Giglio, Xiu, and Zhang (2021) emphasize this point and propose **supervised PCA** to select informative assets when weak factors are present
- Bryzgalova, Pelger, and Zhu (2025) take a complementary route, building test assets endogenously with trees so that the resulting basis assets better span the stochastic discount factor

The practical implication is straightforward: compare methods on a common benchmark set of test assets, but also report how results change when the test-

asset span is enriched or redesigned.

All methods in this section share a limitation: factor loadings are linear functions of characteristics. Momentum behaves differently at high versus low volatility, size effects vary with liquidity, and linear models cannot capture these interactions without manual feature engineering. *Section 14.6* addresses this with deep learning models that learn non-linear mappings while retaining the economic structure developed here.

Model	Estimation objective	Loading structure	Forecast adapter	Best use
PCA	Maximize covariance explained	Static loadings	Optional	Risk decomposition
RP-PCA	Variance plus pricing-error penalty	Static latent loadings	Yes, as factor extractor	Priced-factor discovery
IPCA	Conditional latent factor model	Linear characteristic-conditioned betas	Yes	Interpretable conditional betas
CAE	Nonlinear conditional latent factor model	Neural characteristic-conditioned betas	Yes	Nonlinear conditional betas
Adversarial SDF	No-arbitrage moment restrictions	Pricing-kernel weights	No	Direct pricing-error minimization
SAE	Supervised return prediction plus	Bottleneck representation	No	Predictive benchmark

Model	Estimation objective	Loading structure	Forecast adapter	Best use
	reconstruction regularizer			

Table 14.2: Latent-factor and related models: estimation objective, loading structure, and relationship to the three-stage forecasting adapter

RP-PCA, IPCA, and CAE pass through the adapter as a matter of routine; PCA can be wrapped in it for factor-timing, but is used here for risk decomposition. The adversarial SDF and the SAE sit outside the adapter. The CAE is introduced in *Section 14.6* ; the SDF and the SAE in *Section 14.7* .

The shared adapter, therefore, covers PCA (optionally), RP-PCA, IPCA, and CAE; the differences between these four models live in Stage 1. *Figure 14.6* catalogs the three-tier Stage 2 forecaster ladder (Tier 1: Constant, AR(1), EWMA; Tier 2: Ridge, LightGBM; Tier 3: LSTM, TCN) that the teaching notebooks make swappable; *Section 14.8* reports how the choice interacts with the structural estimator. The next section introduces non-linear techniques for dimensionality reduction.

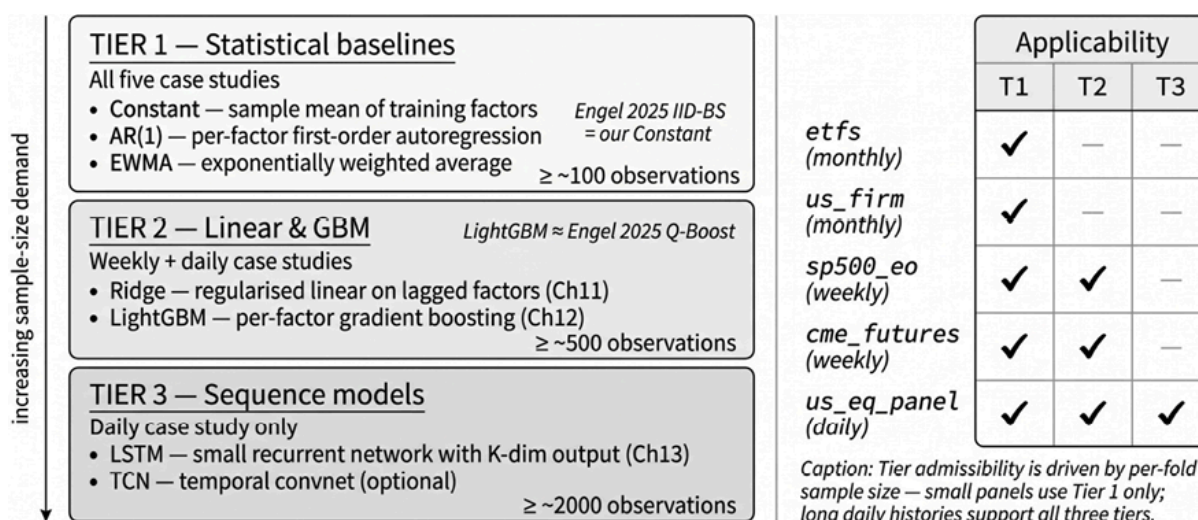


Figure 14.6: The Stage 2 factor-premium forecaster catalog. Tier 1 holds the constant, AR(1), and EWMA baselines; Tier 2 adds ridge and gradient-boosted regressions; Tier 3 adds LSTM and TCN sequence models. Any forecaster pairs with any Stage 1 estimator without retraining the factor model



Implementation : `04_ipca` demonstrates ALS estimation with recovery on synthetic data plus the swappable Stage 2 forecaster catalog; `05_rp_pca` compares standard PCA against RP-PCA across κ values within the same framework. For real-data applications with walk-forward validation, see applicable case study notebooks.

14.6 The conditional autoencoder

Section 14.5 operationalized the three-stage forecasting adapter with linear loading maps: IPCA ties conditional betas to characteristics through a single matrix, and RP-PCA tilts the factor-extraction criterion toward priced variation. Both keep the map between characteristics and loadings linear. The **conditional autoencoder** (CAE; Gu, Kelly, and Xiu, 2019) is the nonlinear member of the same family: it keeps the conditional-factor structure and replaces only the linear loading map with a neural network.

The CAE is a strict nonlinear generalization of IPCA. Both start from the same conditional factor representation:

$$r_{i,t+1} = \beta_{i,t}^\top f_{t+1} + \varepsilon_{i,t+1}$$

and differ only in the loading map. IPCA restricts conditional betas to be linear in lagged characteristics, $\beta_{i,t} = \Gamma^\top z_{i,t}$, whereas the CAE replaces this map with a neural network, $\beta_{i,t} = g(z_{i,t}; W)$. With a single linear layer and no nonlinear activation, the CAE collapses to IPCA, so it is best understood as the same conditional-factor idea with a more expressive loading map, rather than a separate direct-prediction architecture.

That placement is what keeps the CAE inside the adapter. Its training objective reconstructs realized returns from conditional betas and latent factor returns; it does not directly optimize next-period expected returns. The fitted model is nevertheless a conditional asset-pricing representation: covariates guide the dimension reduction, and nonlinear interactions among characteristics shape

the factor exposures. Once estimated, the CAE supplies the same two objects as IPCA, namely current conditional betas and a history of latent-factor returns, so the Stage 2 forecaster catalog from *Section 14.5* carries over unchanged. The training-sample mean of factor returns gives the baseline forecast, while AR(1), EWMA, Q-Boost, and ZS-Chronos replace it without retraining the CAE. The CAE therefore changes only Stage 1: the beta network learns nonlinear characteristic-to-loading relationships, and the factor side extracts latent factor returns from characteristic-managed portfolios.

This also locates the CAE relative to the time-series models of *Chapter 13*. Those models capture temporal patterns in realized return histories; the CAE captures cross-sectional structure at each point in time. Temporal features derived from *Chapter 13* models can enter the CAE as additional characteristics, but the CAE itself is not primarily a sequence model.

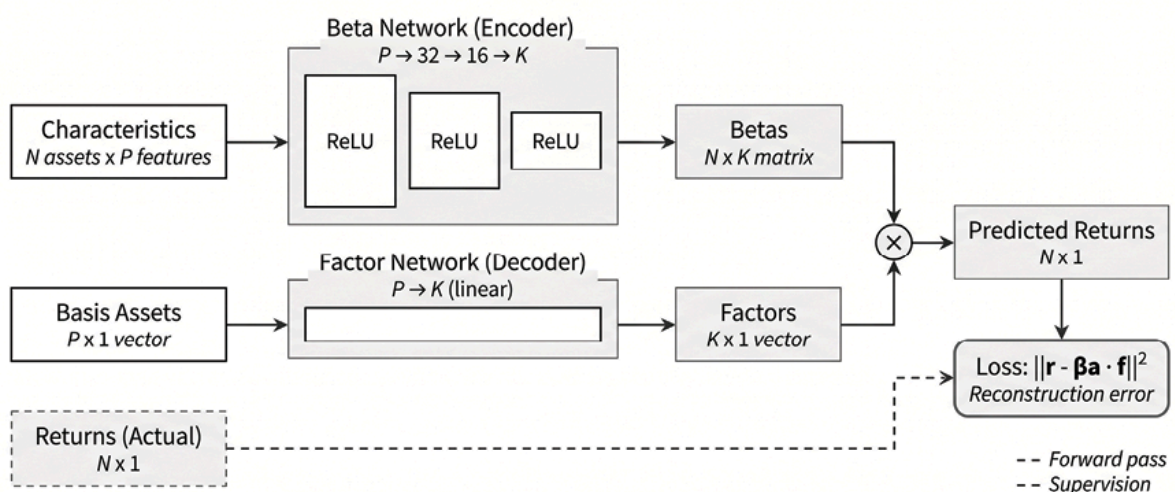


Figure 14.7: The conditional autoencoder. A beta network maps each asset's lagged characteristics to conditional factor loadings; a factor side extracts latent factor returns from characteristic-managed portfolios; their inner product reconstructs the contemporaneous cross-section

Preparing the characteristic panel

The `us_firm_characteristics` case study (*Section 14.1*) provides the main dataset. Custom panels should apply the same survivorship-bias-free, point-in-time requirements introduced in *Chapter 4* : characteristics must be observable before the return they explain or forecast, and universe membership must be

defined without future information. The investable universe should be fixed before estimation, because microcap and illiquidity filters determine which firms contribute to the estimated factor structure and which premia the model can learn. Since characteristic-based predictability is often concentrated among smaller and less liquid firms, report results both with and without the smallest 20% of firms by market capitalization to separate tradable signal from microcap and liquidity artifacts.

The dataset used here contains 63 precomputed features: the 57 fundamental characteristics from *Section 14.5* plus six temporal features. The original CAE study uses a larger set, but the implementation logic is unchanged. Neural networks are sensitive to input scale, and firm characteristics often carry extreme outliers, so we rank stocks by each characteristic each month and rescale the ranks to $[-1,1]$. This cross-sectional transformation reduces the influence of outliers, makes characteristics comparable across units, and preserves the relative information available at each date.

Specifying the dual-network architecture

The CAE contains two linked components: a beta network and a factor side.

The **beta network** maps each stock's P lagged characteristics into K conditional factor loadings. In the PyTorch implementation, it is a feed-forward network with ReLU activations and batch normalization, typically pyramidal (for example, $64 \rightarrow 32 \rightarrow 16 \rightarrow 8$). For each date t , it produces an $N_t \times K$ matrix of conditional exposures:

$$B_t = g(Z_{t-1}; W)$$

where $Z_{t-1} \in \mathbb{R}^{N_t \times P}$ holds lagged characteristics for the N_t firms observed at date t .

The **factor side** estimates the latent factor returns used to reconstruct the cross-section. Let $r_t \in \mathbb{R}^{N_t}$ collect contemporaneous excess returns. The first

step forms characteristic-managed portfolio returns by projecting realized returns on the lagged characteristic matrix:

$$x_t = (Z_{t-1}^\top Z_{t-1})^{-1} Z_{t-1}^\top r_t$$

where $x_t \in \mathbb{R}^P$ summarizes how the cross-section of returns loads on each characteristic at date t ; a pseudo-inverse or regularized inverse replaces the inverse when $Z_{t-1}^\top Z_{t-1}$ is ill-conditioned. The resulting vector reads as returns on characteristic-managed basis portfolios. This is a cross-sectional projection rather than a univariate sort: the sorting interpretation is useful intuition, but the formal object is the projection coefficient vector x_t . A simple factor network, often a single linear layer, maps these managed-portfolio returns into K latent factor returns:

$$f_t = h(x_t; \theta)$$

The beta network and factor side are estimated jointly so that the model reconstructs realized returns as:

$$\hat{r}_{i,t} = \beta_i(z_{i,t-1}; W)^\top f_t$$

This is the central difference between the CAE and a direct prediction network. The CAE does not map characteristics to next-period returns; it estimates a conditional factor representation and then uses it as input to a separate forecasting step.

Estimating the CAE

For each training window, the CAE minimizes the cross-sectional reconstruction loss:

$$\sum_{t,i} \left(r_{i,t} - \beta_i(z_{i,t-1}; W)^\top f_t \right)^2$$

with regularization and early stopping. This objective estimates the latent factor structure and should not be confused with the final forecasting objective. The implementation uses L1 regularization to encourage sparse

weights and early stopping to limit overfitting. Deep networks are sensitive to random initialization, so the notebook trains an ensemble across seeds and averages the predictions; ensemble averaging reduces the variance of the fitted loading map and yields more stable downstream forecasts than a single initialization.

From the `us_firm_characteristics` case study, effective beta-network ranges are two to three hidden layers, widths from 32 to 128 in a pyramidal structure, learning rates from 10^{-4} to 10^{-2} , L1 regularization from 10^{-5} to 10^{-3} , and batch sizes of 1,000–5,000 observations, typically close to a full monthly cross-section. Dropout above 0.3 tends to underfit; below 0.05, overfitting becomes more likely. The factor network is simpler, usually a single linear layer. The identification caveats from *Section 14.5* apply here too: the number of latent factors K should be large enough to capture persistent cross-sectional structure but small enough to avoid redundant or unstable factors. Start with $K = 3 - 10$ for equity models; after applying the model's normalization conventions, highly correlated learned factors signal that K is too large or that regularization is too weak.

Turning latent factors into forecasts

After Stage 1 estimation, the CAE provides current conditional betas and a historical series of latent factor returns. Forward prediction requires the Stage 2 factor-premium forecast. The simplest rule uses the training-sample mean of each latent factor return, recovering the baseline GKX-style setup; AR(1), EWMA, Q-Boost, or ZS-Chronos replace only this factor-premium forecaster. Given a forecast $\hat{\lambda}_{t+1}$, the asset-level expected return is:

$$\hat{r}_{i,t+1} = \beta_i(z_{i,t}; W)^\top \hat{\lambda}_{t+1}$$

The decomposition is useful because the CAE estimates the cross-sectional structure, while the forecasting rule specifies how much compensation each latent factor is expected to earn in the next period. A CAE can reconstruct realized returns well and still forecast weakly if the extracted factors carry

little persistent premium, which is why model selection rests on downstream forecast quality rather than reconstruction loss alone.

Tuning, validation, and failure modes

Reconstruction loss is an optimization diagnostic: it shows whether the model can fit the training cross-section, not that the learned factors earn stable out-of-sample premia. The validation protocol therefore evaluates the complete forecasting workflow. At each walk-forward origin, fit the characteristic transformation, CAE parameters, regularization choices, and factor-premium forecaster on the training window only; then freeze those components, generate forecasts for the next validation period, and evaluate the asset-level predictions. Compare configurations on validation IC, IC information ratio, quintile spreads, turnover, and portfolio-level performance, reserving reconstruction loss for diagnosing optimization failure or underfitting. Optuna's Bayesian optimization (*Section 12.4*) can explore the architecture and regularization space, using downstream validation IC or IC information ratio as the objective, with at least 50 trials to ensure a stable search.

The common failure modes are visible in the estimated factors, betas, and validation behavior. Diverging reconstruction loss usually indicates either too high a learning rate or insufficient gradient control; strong seed sensitivity argues for a larger ensemble; highly correlated latent factors point to an oversized K model or weak regularization; near-identical betas across assets indicate beta-network collapse, diagnosed by cross-sectional beta dispersion. Statistical performance must also withstand economic checks: a high-IC signal with high turnover may be untradeable after costs are deducted. Quintile spreads, turnover, and factor decay across horizons indicate whether predictability is economically meaningful, and gross and net performance should both be stated. A simple approximation is net return $\approx$ gross return $- 2 \times$ turnover $\times$ one-way cost, with cost assumptions matched to the universe: 5–10 basis points one-way for large-cap equities, 50 or more for microcaps.

Empirical evidence and replication caveats

In the original GKX study of US equities, long-short portfolios built on CAE predictions outperformed linear IPCA benchmarks, with the gains driven by nonlinear interactions among firm characteristics. The economics are intuitive: the effect of momentum, value, size, volatility, or liquidity need not be additive or constant across the cross-section, and a neural loading map can represent interactions such as momentum behaving differently between high- and low-volatility stocks without the researcher pre-specifying those terms.

The gains are not mechanical, and our `us_firm_characteristics` implementation illustrates the difficulty of replication. On the primary one-month label, the CAE flips negative at -0.030 in IC, well below the supervised autoencoder treated in *Section 14.7*, a result not directly comparable to the predictive R^2 metrics reported by GKX. It still points to the same conclusion as the literature: latent-factor signals are modest in absolute terms and sensitive to sample period, universe definition, characteristic coverage, and validation design. The case study uses a shorter and more recent sample, a smaller universe, and fewer characteristics than the original, which likely accounts for much of the gap.

Neural factor loadings lack the eigenvector interpretation available for PCA. SHAP (*Section 11.4*) provides a partial diagnostic by attributing each asset's learned loadings to its input characteristics: a high loading on a learned factor may be associated with momentum and low volatility, while size contributes negatively; aggregating SHAP values across assets yields factor-level importance rankings. Two caveats are essential. SHAP explains how the fitted beta network uses characteristics; it does not prove those characteristics causally drive returns, and with correlated inputs the attribution can be unstable across samples. It is an interpretability diagnostic, not an independent validation of the economic mechanism.

Recent extensions modify Stage 1 while preserving the broader adapter. Self-attention lets the beta network condition each asset's loading on the broader cross-section, capturing sector, peer, and supply-chain relationships that per-asset feed-forward networks miss (Kelly et al., 2025), and text embeddings from earnings calls, filings, and news (*Chapter 10*) can enter as additional characteristics. In each case, the Stage 2 forecaster and Stage 3 asset map stay conceptually unchanged. Engel et al. (2025) push the idea further, using a CAE to extract latent factor portfolios and then training separate time-series models on each factor, with predictive uncertainty used to select the most forecastable factors for tangency-portfolio construction.



Implementation : `06_conditional_autoencoder` builds the full workflow in PyTorch: characteristic preprocessing, dual-network estimation, ensemble training, three Stage 2 forecasters (Constant, AR(1), EWMA), the Stage 3 asset map, SHAP interpretability, and beta-dispersion diagnostics.

14.7 The stochastic discount factor and the supervised autoencoder models

The adapter of *Section 14.5* separates factor estimation from factor-premium forecasting, and PCA, RP-PCA, IPCA, and the CAE all fit it: each estimates a factor representation that a Stage 2 model then forecasts. Two deep models break that pattern for opposite reasons. The **stochastic discount factor** (SDF; Chen, Pelger, and Zhu, 2021) collapses the stages by learning the pricing object directly from no-arbitrage conditions, leaving no factor-return history to hand to a separate forecaster. The **supervised autoencoder** (SAE) skips the structure altogether, mapping characteristics to forward returns end-to-end with no factor intermediate. They are grouped here by that shared trait: each

bypasses the three-stage adapter, one because it prices directly and one because it predicts directly. This architecture is shown in *Figure 14.8* :

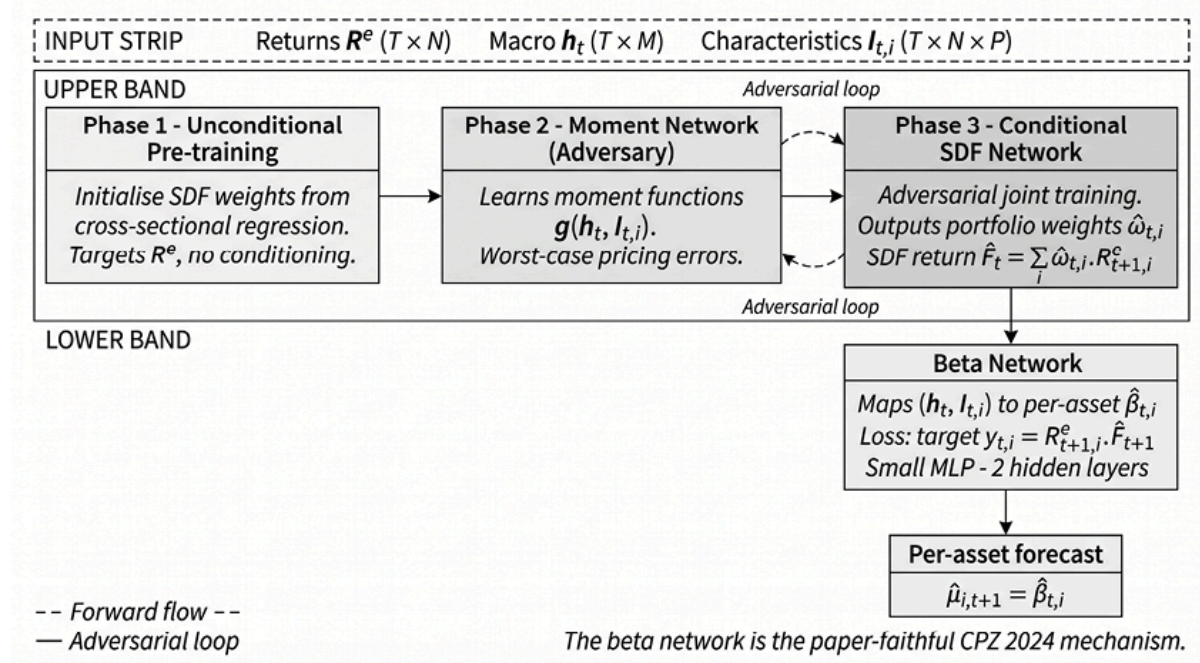


Figure 14.8: Where the SDF and the supervised autoencoder sit relative to the three-stage adapter

The SDF learns the pricing kernel directly from no-arbitrage moment conditions and emits pricing-error diagnostics rather than a factor-return history; the supervised autoencoder maps characteristics to forward returns end-to-end. Neither exposes a factor premium for a separate Stage 2 forecaster.

The stochastic discount factor

The Chen-Pelger-Zhu (CPZ, 2021) framework operationalizes the third objective introduced in *Section 14.1* : minimizing pricing errors directly through conditional moment restrictions. No-arbitrage is naturally written one step ahead. For gross returns $r_{i,t+1}$ the stochastic discount factor M_{t+1} satisfies $E_t[M_{t+1}r_{i,t+1}] = 1$, or equivalently $E_t[M_{t+1}r_{i,t+1}^e] = 0$ for excess returns. CPZ parameterize M_{t+1} with a flexible network of firm characteristics and macro-state variables, and estimate it by minimizing violations of these

conditional moment restrictions. The output is a learned pricing kernel $\hat{M}_t$ and the associated pricing-error diagnostics, not a factor-return history designed for a downstream premium forecaster, which is why the model has no natural Stage 2 hook.

Building the SDF with adversarial deep learning

The conditional moment set is effectively infinite, so evaluating every restriction is infeasible; the adversarial mechanism is how CPZ make the problem tractable. A second network learns to construct the worst-case test portfolio, the linear combination of assets that the current SDF estimate would most misprice. Training then alternates between two objectives. The SDF network updates to minimize pricing errors on the adversary's portfolio, and the adversary updates to find new portfolios the improved SDF still misprices. This minimax structure disciplines the SDF against the hardest cases rather than the average ones, the crucial difference from the CAE's average reconstruction loss. The `07_stochastic_discount_factor` notebook implements this as a three-phase training loop with configurable adversarial rounds.

The state variables enter through a recurrent macro encoder. A MacroLSTM learns a low-dimensional state from FRED indicators (yield spreads, inflation, volatility) so the kernel adapts to the regime without the researcher pre-specifying which macro variables matter, the time-series analog of letting the beta network discover characteristic interactions in the CAE.

Evaluation differs from the CAE because the object being estimated is different. Report the SDF-implied maximum Sharpe ratio, which the Hansen-Jagannathan bound ties to the magnitude of pricing errors; the pricing errors against benchmark factor models; and the worst-case adversarial portfolio at each step. One caveat warrants close attention: cross-sectional GMM can yield spuriously high fit through weighting-matrix choices when the model is misspecified (Gospodinov, Kan, and Robotti, 2014), so rerun under alternative

weightings and across both fixed and adaptive test-asset sets. The teaching and case-study notebooks run these diagnostics by default. For comparability with the other model families, the case study selects the reported SDF checkpoint on the validation IC rather than the validation-Sharpe criterion used in the original CPZ protocol.

The supervised autoencoder

The supervised autoencoder is the other model outside the adapter, but it sits there as a practical benchmark rather than a structural asset-pricing object. It is the useful direct-prediction alternative when the goal is a strong tabular signal rather than a factor decomposition. An encoder compresses characteristics to a bottleneck, a decoder reconstructs the inputs as a regularizer, and a supervised prediction head maps the bottleneck to forward returns. The architecture is best known as the winning entry in the Jane Street-sponsored Kaggle market-prediction competition (2020–2021), which motivates its inclusion as a strong baseline; the chapter README links the original write-up.

There is no Stage 1/Stage 2/Stage 3 separation. The supervised loss shapes the bottleneck, and the prediction head is the asset mapper, so evaluation follows direct-prediction conventions, cross-sectional IC and AUC for directional labels, rather than the pricing-error or factor-premium diagnostics that suit the SDF. The contrast with the CAE is the point worth carrying forward: both compress characteristics through a network, but the CAE's bottleneck is a conditional factor structure disciplined by reconstruction, while the SAE's bottleneck is whatever representation best predicts the label. The first is interpretable as risk; the second is not, and is not meant to be.

Implementation :

- `07_stochastic_discount_factor` builds the adversarial SDF with MacroLSTM and three-phase minimax training on a top-200 US equities universe with eight characteristics





- `@8_supervised_autoencoder` implements the supervised variant with purged cross-validation, performing direction classification (AUC scoring) with characteristic-reconstruction regularization; `ml4t.models.SAEModel` is the production form

14.8 Case study insights

Chapters 11 through 13 trained models directly on the forward-return label, from regularized linear regressions to gradient boosting and deep sequence networks. The latent-factor estimators reach the same cross-sectional ranking from the other side: they recover structure from the joint behavior of returns and characteristics with little or no return supervision, then score assets on it. The question here is whether that structure ranks assets as well as the supervised families already do.

Five of the nine case studies have cross-sections sufficiently balanced to fit the full latent menu: ETFs, the S&P 500 equity-and-options study, US Equities, US Firm Characteristics, and CME Futures. The five estimators differ in what they optimize: PCA takes the leading variance directions, IPCA fits conditional linear betas from characteristics, the conditional autoencoder (CAE) minimizes reconstruction error, the stochastic discount factor (SDF) targets a no-arbitrage pricing error, and the supervised autoencoder (SAE) adds a return-prediction task to the reconstruction.

Coverage is uneven: ETFs and the equity-and-options study run all five estimators; US Firm Characteristics drops PCA, because anonymized firm identifiers are not stable from month to month and break the balanced panel PCA requires; CME Futures runs PCA and the SDF, and US Equities runs PCA and IPCA. Each estimator is summarized by its average daily information coefficient (IC) and a 95% confidence interval computed with a HAC correction for the autocorrelation in the daily series; an interval that excludes zero is the bar for significant signal.

Where latent factors add ranking content

The strongest latent estimator clears zero on four of the five case studies (ETFs, US Firm Characteristics, CME Futures, and US Equities) and overlaps zero on the equity-and-options study. *Table 14.3* places each one beside the strongest model from *Chapters 11* through *13*. Neural objectives carry the latent result almost everywhere: the SDF leads on ETFs and CME Futures and the SAE on US Firm Characteristics, while IPCA, the conditional linear estimator, carries US Equities at a small but credibly positive value. PCA, the unconditioned baseline, reaches the highest latent IC on none of the five.

Case study	Horizon	Best latent model	Latent IC (t)	Strongest prior model	Prior IC	Δ IC
ETFs	21 days	SDF	+0.085 (4.9)	NLinear	+0.062	+0.023
US Firm Characteristics	1 month	SAE	+0.062 (5.5)	GBM	+0.080	−0.018
CME Futures	5 days	SDF	+0.037 (2.8)	GBM	+0.032	+0.005
S&P 500 Eq+Opt	5 days	SDF	+0.012 (0.7)	TabM	+0.011	+0.001
US Equities	1 day	IPCA	+0.005 (2.2)	GBM	+0.032	−0.027

Table 14.3: Highest-IC latent estimator at each case study's primary forward-return label, with its HAC t-statistic, beside the strongest supervised model from Chapters 11 through 13 (a regularized linear model, gradient boosting, TabM, or a deep sequence network). Δ IC is the latent point estimate minus that of the strongest prior model

Clearing zero is not the same as improving on the best model already in hand, and the Δ column shows the two apart. To compare each latent estimator

directly against the strongest prior model, we compute the daily IC series difference on the dates they share and place a HAC interval around the mean difference, the paired counterpart of the single-model intervals in *Table 14.3*.

On this measure, the latent estimator credibly leads the strongest prior model on ETFs, where the SDF sits 0.023 above NLinear, and credibly trails on two case studies: US Equities, where IPCA sits 0.027 below gradient boosting, and US Firm Characteristics, where the SAE sits 0.019 below it. On CME Futures and the equity-and-options study, the latent point leads in expectation, but the interval spans zero. Latent factors therefore match or beat the strongest supervised family on three of the five case studies and credibly trail on two, a substantial showing for estimators that lean on structure rather than direct return supervision.

The equity-and-options study is the one case where the primary label hides the signal rather than lacking it. Every estimator overlaps zero at the 5-day return, but IPCA on the risk-adjusted 5-day label reaches +0.041 with an interval clear of zero, the only credibly nonzero latent point on that study.

Which objective extracts the signal

The estimator that does best is the one whose objective matches what the cross-section rewards, and the case studies do not agree on which that is.

Figure 14.9 sets the five estimators against the five case studies; conditioning and supervision help where unconditioned variance does not. PCA recovers the directions of largest return variance, which need not price the cross-section, and it ranks no case study best. The SDF and SAE, which add a pricing constraint or a prediction task on top of the factor structure, reach the highest latent IC on four of the five; IPCA's characteristic-conditioned betas carry the fifth.

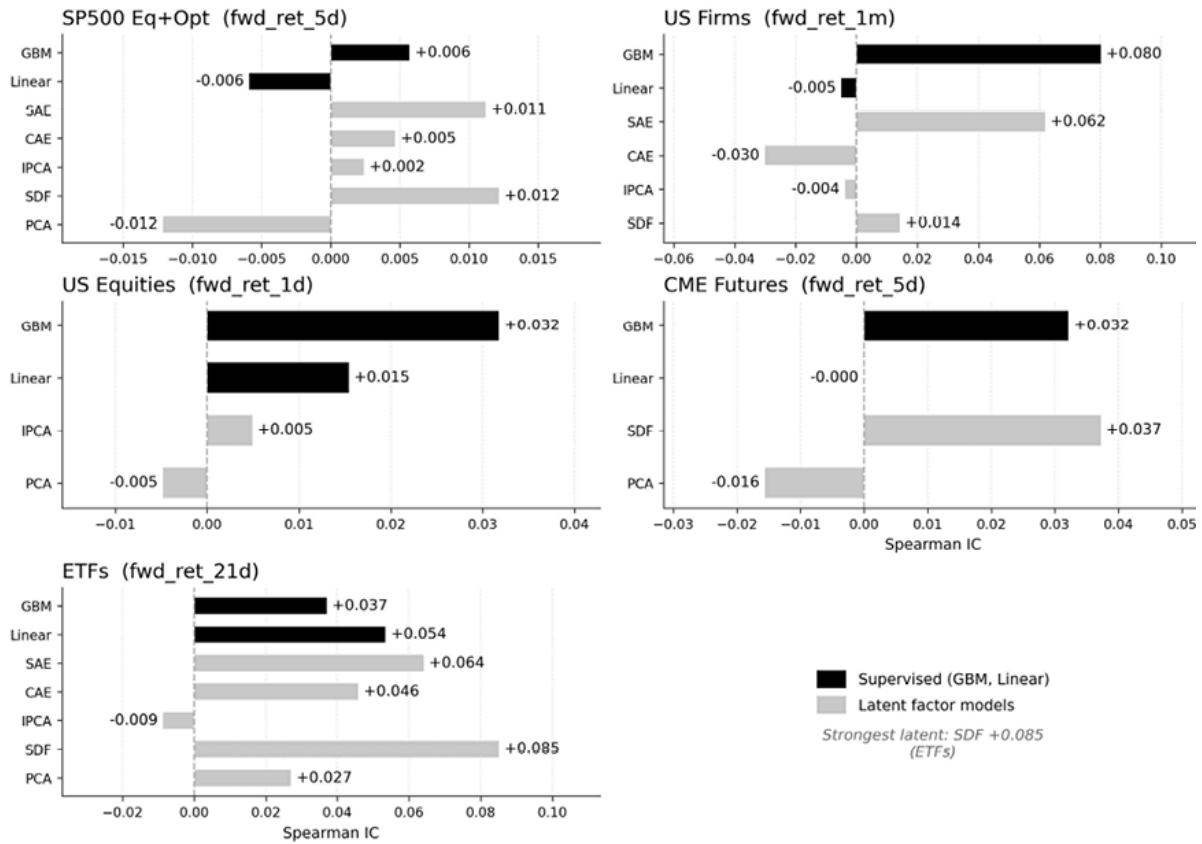


Figure 14.9: IC by latent estimator across the five case studies at the primary label. Shade encodes IC magnitude

US Firm Characteristics shows the spread most cleanly because it runs four objectives on a single monthly cross-section. At the one-month label, the SAE reaches +0.062 while the CAE lands at -0.030 , with the SDF and IPCA bunched near zero between them (*Figure 14.10*). Both extremes clear their intervals and carry opposite signs: a multi-task objective that uses the return label yields a positive ranking, while a pure reconstruction objective, which fits contemporaneous factors with no forward target, points in the wrong direction. This is the prediction-protocol caveat from *Section 14.5* : contemporaneous-factor reconstruction is a poor proxy for one-step-ahead forecasting on monthly characteristics.

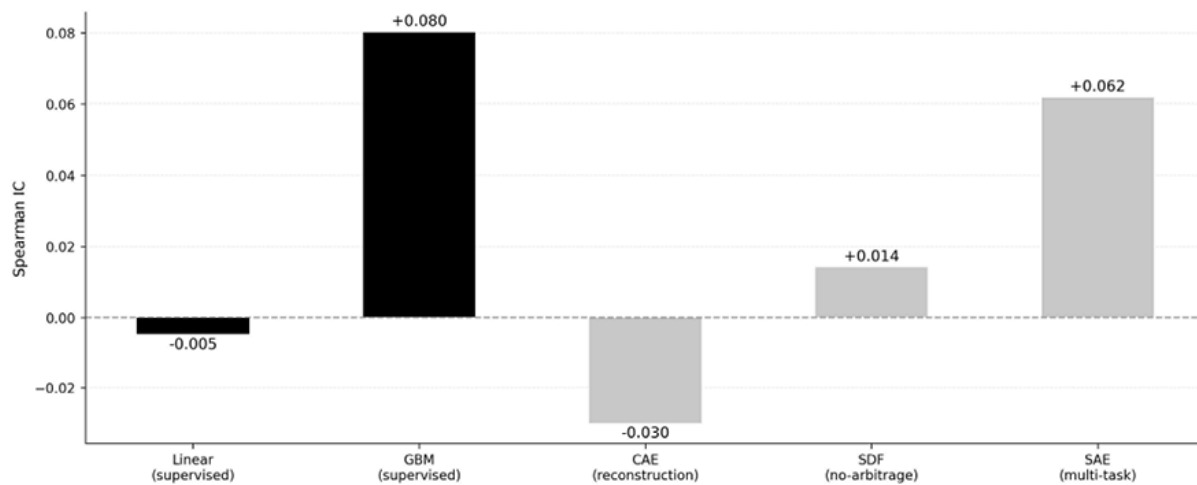


Figure 14.10: Objective ladder on US Firm Characteristics: SAE highest, CAE negative, SDF and IPCA near zero between them

Because the objectives capture different structure, their predictions disagree. Across the three neural estimators for US Firm Characteristics, the pairwise rank correlations have different signs, so the models are not near-duplicates of a single latent signal. That disagreement is the empirical opening for the multi-objective averaging *Chapter 20* builds on.

Dimensionality and stability

The latent estimators face markedly different estimation problems across the five case studies, and panel dimensionality accounts for much of the spread. CME Futures and ETFs sit in a favorable regime, with far more time periods than assets, so the signal eigenvalues cleanly separate from the noise floor predicted by random-matrix theory. US Equities and US Firm Characteristics are severely high-dimensional (US Firm Characteristics carries roughly nine times as many assets as time periods), where the sample return covariance is too noisy to invert directly. The estimators that still resolve a credible IC there, IPCA on US Equities and the SAE on US Firm Characteristics, both sidestep that covariance by mapping characteristics to factor loadings rather than estimating loadings from returns.

Stability over the walk-forward folds tracks the resolution of the label. *Figure 14.11* plots the per-fold ICs against their means; even the case studies whose

intervals exclude zero with margin show wide fold-to-fold variation, which is why the HAC interval, not the point estimate, is the threshold throughout. The monthly US Firm Characteristics result, built on a structural cross-sectional edge, is positive on every fold; the daily US Equities result, built on a thin per-day edge, is positive on about three folds in five. The first is a small effect measured cleanly; the second a smaller effect measured against more noise.

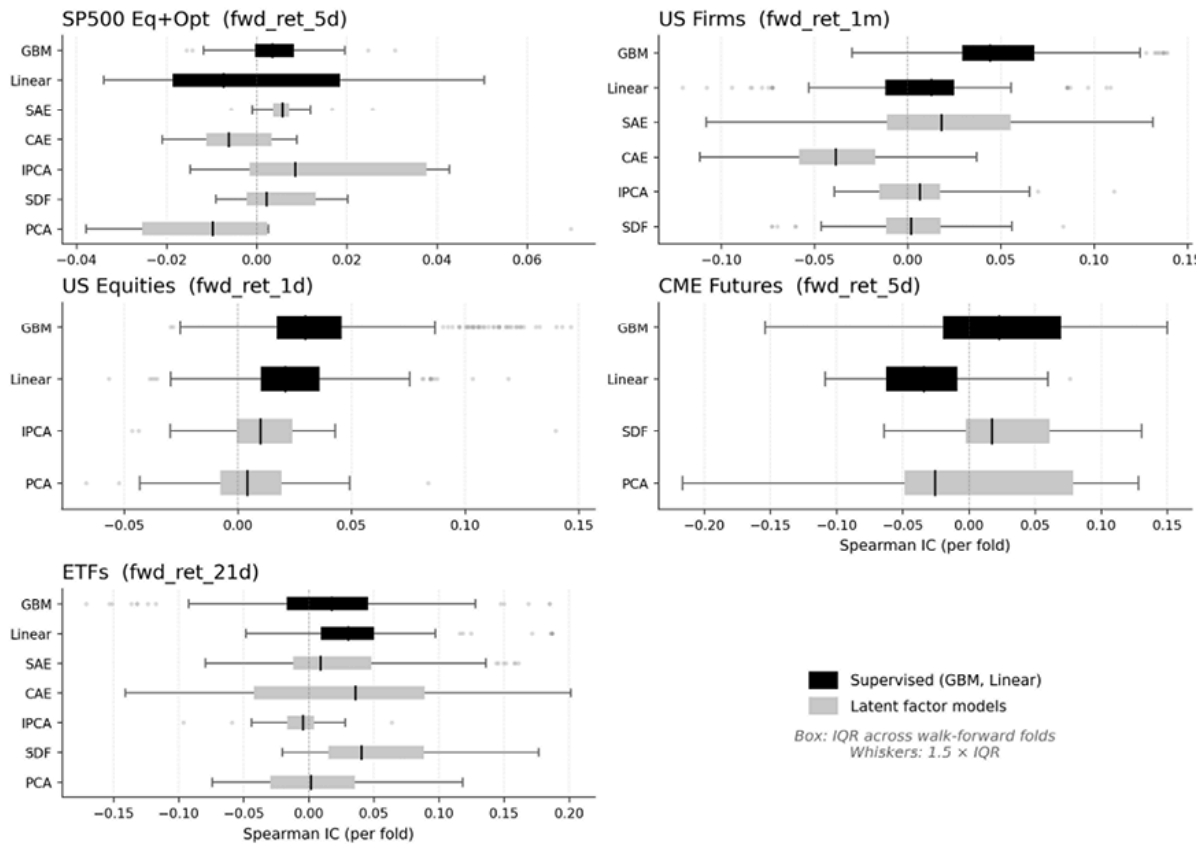


Figure 14.11: Per-fold IC distribution against fold means across the five case studies

Regression versus classification

Only US Firm Characteristics carries a classification label through the latent stack, the one-month direction label. Under the symmetric metric introduced in *Section 11.6* (IC for a regression score against the continuous return, AUC for a classifier against the binary direction), IPCA's continuous score reaches an IC of +0.010 against the realized return, a low-margin result whose interval spans zero. It neither confirms nor contradicts the ordering established by the

regression labels, which remains the more informative reading for this monthly cross-section.

Choosing a method

These estimators are complements rather than substitutes. PCA focuses on risk decomposition, with its eigenportfolios supporting position sizing in *Chapter 17* and risk management in *Chapter 19* ; IPCA, the CAE, the SDF, and the SAE add conditioning, non-linearity, a no-arbitrage discipline, and a supervised task. Estimating several under one walk-forward protocol, then reading agreement as a likely signal and divergence as model risk, is what carries forward.

The case studies where a latent estimator clears zero (ETFs, US Firm Characteristics, and CME Futures most clearly) take that ranking into *Chapters 16* through *19* , which weigh it against turnover and cost, while the disagreement measured here is what *Chapter 20* turns into a multi-objective ensemble.

14.9 Summary

The four latent-factor methods of this chapter share a common predictive workflow, the three-stage forecasting adapter. Stage 1 estimates a factor representation, Stage 2 forecasts the factor premium, and Stage 3 combines today's exposures with that forecast. RP-PCA, IPCA, and the CAE pass through the adapter as a matter of routine, and PCA can be wrapped in it for factor-timing. The four differ in Stage 1 alone: PCA maximizes covariance explained, RP-PCA adds a pricing-error penalty, IPCA estimates linear characteristic-conditioned betas, and the CAE generalizes that map nonlinearly.

The adversarial SDF and the supervised autoencoder sit outside the adapter. The SDF estimates a pricing kernel directly under no-arbitrage moment restrictions; the SAE predicts returns end-to-end and earns its place as a

neural-network benchmark rather than a structural factor model. The `ml4t-models` library packages the adapter as a composable pipeline, while the chapter notebooks derive every stage from scratch.

Across the five case studies, the objective that matches what the cross-section rewards reaches the highest information coefficient, and the case studies disagree on which objective that is. The SDF reaches the highest latent IC on three of them, the supervised autoencoder on US Firm Characteristics, and IPCA on US Equities; unconditioned PCA leads on none. The clearest case is ETFs, where the SDF clears the HAC threshold with an IC of +0.085. The strongest latent estimator meets that threshold in four of the five case studies, overlapping zero only in the equity-and-options study.

Clearing zero is not the same as improving on the supervised families of *Chapters 11 through 13*. Compared with the strongest prior model on shared dates, the latent estimator matches or beats it in three case studies (credibly leading on ETFs and statistically indistinguishable on CME Futures and the equity-and-options study) and credibly trails in two, US Equities and US Firm Characteristics, where gradient boosting ranks clearly higher. That is a substantial showing for estimators that lean on structure rather than direct return supervision, though not a wholesale improvement over models that learn the label directly.

The practical lesson is that latent-factor models compete where the panel carries more periods than assets and genuine cross-sectional structure, fall behind where richly featured supervised models dominate, and stay useful as risk-decomposition tools regardless of predictive ranking (*Chapters 17 and 19*). *Chapter 15* moves from prediction to causal estimation; *Chapter 16* turns the predictions from *Chapters 11 through 14* into trading signals.

15

Causal Machine Learning

We introduced causal thinking as a falsification filter in *Chapter 7*: directed acyclic graphs (DAGs) to encode mechanism assumptions, the three structural roles of confounders, mediators, and colliders to guide conditioning decisions, and four plausibility checks to triage features before committing to heavier modeling. Those diagnostics are bivariate – they test one feature at a time and cannot detect multivariate confounding. This chapter provides the multivariate estimation machinery that the survivors require.

Feature-level triage asked whether a single feature's association with the label is consistent with a proposed mechanism. The methods here ask harder questions:

- What is the magnitude of a treatment effect after orthogonalizing against high-dimensional confounders?
- Did a discrete event actually move prices relative to a data-driven counterfactual?
- Which variables in a multivariate system cause which, and at what lags?

Each question demands a different estimator, and each estimator rests on assumptions that must be stated and tested. After completing this chapter, you will be able to:

- Define a **causal research question** in terms of treatment, outcome, estimand, and counterfactual, and use DAGs to justify an admissible adjustment set.
- Apply validation and refutation tools, including placebo tests, sensitivity analysis, and subset-stability checks, to assess whether a causal claim is robust enough to inform trading research.
- Use Double Machine Learning (DML) to **estimate causal effects of continuous treatments** in the presence of high-dimensional confounders, while respecting temporal cross-fitting and pre-treatment timing discipline.
- Use Bayesian Structural Time-Series (BSTS) to **estimate the impact of discrete events** by constructing data-driven counterfactuals and evaluating spillover risk in control series.
- Use **causal discovery methods** such as PCMCI, NOTEARS, and VAR-LiNGAM to generate candidate structures and interpret them as hypotheses requiring further

validation rather than as definitive causal truth.

- Distinguish predictive signal from causal effect, and interpret cross-dataset evidence with attention to confounding bias, multiple testing, and the gap between statistical significance and refutation survival.

The chapter begins with the conceptual bridge from predictive features to causal estimands, then develops the identification framework in *Section 15.2* and the validation and refutation toolkit in *Section 15.3* that every causal claim must survive. *Section 15.4* applies Double Machine Learning to continuous treatment-effect estimation across the case studies, while *Section 15.5* shifts to discrete-event impact measurement using BSTS. *Section 15.6* reverses the direction of inquiry, using causal discovery algorithms to infer structure from data rather than imposing it. *Section 15.7* synthesizes the cross-dataset evidence, and *Section 15.8* distills the practical guidelines.

15.1 From theory to estimation

The conceptual toolkit from *Section 7.5* (DAGs, structural roles, mechanism plausibility checks) establishes *whether* a causal story is worth pursuing. This section bridges from that qualitative assessment to quantitative estimation: given a DAG and an identification strategy, how do we measure causal effects with valid uncertainty quantification?

The causal estimation challenge

Classical causal designs begin with a research design, not with an algorithm. They ask **what source of variation could identify the effect of interest** :

- Backdoor adjustment with observed controls
- An instrument that shifts treatment without directly affecting the outcome
- A before-and-after comparison with a credible control group
- A threshold that approximates random assignment
- A counterfactual path constructed from an untreated series

In each case, the central question is the same: *why should the observed data isolate the causal effect rather than reflect confounding, selection, anticipation, or spillover* ?

In finance, these designs remain essential, but they are often difficult to apply cleanly. Confounders are numerous, nuisance relationships are nonlinear, effects vary across regimes, events propagate across assets, and treated units are often few. Machine learning helps estimate these **nuisance components** more flexibly, but it does not create

identification where none exists. The methods in this chapter, therefore, combine causal design with modern predictive tools:

- **Double Machine Learning** (DML) for continuous treatments under backdoor-style adjustment
- **Bayesian Structural Time-Series** (BSTS) for event-level counterfactual construction
- **Causal discovery** methods for generating structural hypotheses when the graph is not known in advance

Before we make these design families concrete, we need to distinguish identification from estimation, and clarify where the chapter's three main methods fit.

Three estimation problems

The chapter is organized around **three distinct causal questions** within that broader design landscape, each requiring a different estimator:

1. Does momentum *cause* forward returns after controlling for volatility and regime? **DML** orthogonalizes the treatment against confounders using flexible ML models, yielding debiased effect estimates with valid confidence intervals.
2. How much did a Fed rate decision move bond ETF prices relative to what would have happened absent the announcement? **BSTs** constructs a synthetic counterfactual from the control series and measures the gap.
3. Which cross-asset return series actually drive others, and which merely co-move? **PCMCI** tests conditional independence at multiple lags, recovering a time-lagged causal graph; neural methods (NOTEARS, VAR-LiNGAM) scale discovery to higher dimensions.

Table 15.1 maps research questions to estimators and Python libraries.

DAG known?	Treatment type	Method	Library
Yes	Binary/Cont.	Backdoor adj.	DoWhy
Yes	Continuous	DML	EconML
Yes	Binary (event)	BSTs	tfp-causalimpact
No	N/A (discovery)	PCMCI	Tigramite
No	N/A (discovery)	NOTEARS	causal-learn

DAG known?	Treatment type	Method	Library
No	N/A (discovery)	VAR-LiNGAM	causal-learn

Table 15.1: Method selection guide, from research question to Python library

A sophisticated estimator cannot rescue an identification failure. The pipeline runs: DAG specification → adjustment-set identification → estimand derivation → ML-based estimation → refutation testing. *Section 15.2* covers the shared identification and validation framework that precedes all three estimation strategies; the method-specific sections that follow assume valid identification is in place.

The goal is not to prove causation from observational data but to distinguish robust claims from those that collapse under scrutiny. A key step in this regard is identifying the causal mechanism, to which we turn next.



Implementation : `01_library_overview` compares the libraries in *Table 15.1* .

15.2 Identification and validation

Causal estimation is only as credible as the design that supports it. Estimation asks how to compute an effect once the causal question has been defined. Identification asks the prior question: whether the effect can be learned from the available data under explicit assumptions.

This distinction matters in financial markets because prediction, correlation, and causation often point in different directions. A variable may predict returns because it captures risk, proxies for a hidden market state, is downstream of the outcome, or reflects a genuine causal mechanism. Causal analysis begins by separating these possibilities before fitting a model.

From outcome and intervention to estimand

Every causal question requires specifying what is being varied, what is being measured, and what comparison defines the effect. These choices should be made before fitting a model.

The **intervention** , or treatment, is the exposure whose effect we want to estimate. In markets, treatments are rarely interventions that a researcher can assign. They are usually

events, rules, or shocks that approximate interventions: a change in index membership, a regulatory change, an earnings surprise, or a revision to a portfolio allocation rule. The *treatment definition* should specify the treated units, the timing of treatment, and the relevant treatment horizon.

The **outcome** is the variable whose response defines the question. It should match both the proposed mechanism and the decision problem. Depending on the application, the outcome may be a forward return, volatility, drawdown, liquidity measure, or risk exposure. Outcome choice is part of the research design because it defines the estimand and affects the plausibility of the identifying assumptions. A mechanism closely related to the outcome may support a more credible causal claim than a broad trading outcome that is influenced by many other forces. This does not make the broader outcome irrelevant; it means the causal claim is likely harder to defend.

The **estimand** is the causal quantity to be estimated. Common examples include:

- **Average Treatment Effect (ATE)** : the average effect across the target population
- **Average Treatment Effect on the Treated (ATT)** : the average effect for the units that actually received treatment, often the relevant estimand in event studies
- **Conditional Average Treatment Effect (CATE)**: the effect conditional on observed state variables, such as volatility regime, liquidity, or valuation conditions

The estimand also fixes the target population, treatment timing, outcome horizon, and comparison being made. For example, the effect of a funding-rate shock on next-period premium reversion is not the same estimand as the effect of the same shock on multi-day forward returns. The first asks whether the funding premium itself mean-reverts after the shock. The second asks whether that shock translates into a broader trading outcome after other return drivers have also acted.

Several closely related terms recur throughout the chapter and are easy to conflate. The following definitions are used consistently in the sections and notebooks that follow.

Term	Definition
ATE	Average marginal treatment effect across the target population, under the specified controls
Subgroup ATE	The ATE re-estimated within a stratum, such as a volatility regime
CATE	A covariate-conditional effect from a single heterogeneity model (causal forest, X-learner)

Term	Definition
Effect modifier	A variable along which the treatment effect is allowed to vary
Confounder (control)	A pre-treatment variable used for adjustment to block a backdoor path
Refutation	A diagnostic that can weaken a causal claim but cannot prove it

Table 15.2: Definitions for key terms

The distinction between a subgroup ATE and a CATE matters in practice. Splitting the sample by regime and re-estimating the ATE within each stratum yields subgroup ATEs; it does not fit a model of how the effect varies with covariates. A CATE requires a heterogeneity estimator that directly learns the effect surface. Likewise, a confounder and an effect modifier play different roles: a confounder is adjusted for to remove bias, whereas an effect modifier defines how the effect changes. Many estimators, including the double machine learning interface used later in this chapter, expose these two roles through separate arguments, and using the wrong one changes the quantity being estimated.

Several assumptions are required before an estimand can be interpreted causally:

- **Consistency** means that the observed outcome under the observed treatment corresponds to the relevant potential outcome
- **Overlap** means that the data support the comparison, so both treated and untreated observations are available for the relevant covariates
- **No interference** , or **Stable Unit Treatment Value Assumption (SUTVA)** , means that one unit's treatment does not change another unit's outcome
- **Conditional exchangeability** means that, after conditioning on an admissible set of pre-treatment variables, treatment assignment is as good as random with respect to the potential outcomes

These assumptions are **demanding in financial markets** . Prices aggregate information quickly, assets interact through common factors and portfolio flows, and treatment timing is often ambiguous. The point is not to pretend these assumptions are automatically satisfied. The point is to state the causal question precisely enough that the assumptions can be inspected.

Define the treatment, outcome, estimand, target population, and timing before fitting any model. Without this discipline, causal analysis degenerates into post-hoc rationalization,

which is another form of the selection bias it aims to eliminate. We will use T to denote the treatment, Y the outcome, and W a set of pre-treatment covariates.

Directed acyclic graphs and adjustment sets

A directed acyclic graph, or DAG, makes the identification assumptions explicit. *Section 7.5* introduced confounders, mediators, and colliders as structural roles in a causal graph. Here, those roles become rules for deciding which variables belong in the adjustment set, a specific group of covariates conditioned on to estimate the causal effect of an intervention.

The central problem is confounding. A **confounder** is a common cause of treatment and outcome. If market stress affects both funding rates and future returns, then a raw association between funding rates and returns may partly reflect a common driver rather than a causal effect of funding rates. **Adjustment** aims to compare treated and untreated observations that are similar with respect to such common causes.

The **backdoor criterion** formalizes the conditions under which this comparison is valid. A set of variables W is an admissible adjustment set for the effect of T on Y if it blocks every noncausal path from T to Y that begins with an arrow into T , and if it contains no descendants of T . Under consistency, overlap, no interference, and conditional exchangeability given W , the interventional distribution for a discrete treatment can be written as:

$$P(Y = y \mid do(T = t)) = \sum_w P(Y = y \mid T = t, W = w) P(W = w)$$

For a continuous outcome, the same idea is often more useful in expectation form:

$$\mathbb{E}[Y \mid do(T = t)] = \sum_w \mathbb{E}[Y \mid T = t, W = w] P(W = w)$$

When W is continuous, the corresponding expression is:

$$\mathbb{E}[Y \mid do(T = t)] = \int \mathbb{E}[Y \mid T = t, W = w] dF_W(w)$$

Adjustment compares treated and untreated observations within strata of admissible covariates W , where the relevant backdoor paths have been blocked, and then averages those conditional comparisons over the population distribution of W . In words, it asks what the outcome would look like if everyone were set to treatment level t , while the distribution of pre-treatment covariates remained unchanged.

This expression defines the estimand implied by the DAG assumptions. Estimation methods differ in how they approximate the conditional expectation $\mathbb{E}[Y \mid T = t, W = w]$. Linear regression imposes a parametric form. Matching and weighting more directly approximate the comparison. Double machine learning uses flexible models for the nuisance functions. The identification question comes first in all cases: is W an admissible adjustment set?

The answer depends on **causal structure**, not predictive power. True confounders are good controls because conditioning on them blocks spurious association. Strong pre-treatment predictors of Y may also improve precision, provided they are not colliders, mediators, treatment descendants, or selection variables. By contrast, poor controls can worsen a causal estimate. Conditioning on a mediator removes part of the pathway through which treatment affects the outcome, thereby changing the estimand. Conditioning on a collider opens a noncausal path that was previously closed. Conditioning on a treatment descendant introduces post-treatment bias.

This is why **kitchen-sink regression is dangerous** in causal work. Adding more variables does not automatically make an estimate safer. It may do the opposite if some of those variables sit downstream of treatment or are common effects of treatment and outcome. The DAG determines which variables belong in the adjustment set. Statistical significance, feature importance, and predictive accuracy do not.

Timing discipline is the most practical screen in trading applications. Every control in W should be known before treatment is realized. Event studies should avoid variables that may react contemporaneously to the event, especially when macro announcements move broad markets immediately. Factor studies should avoid realized performance, portfolio outcomes, or variables mechanically constructed from future returns. This timing rule implements the requirement that the adjustment set contain no treatment descendants.

Timing discipline is necessary, but not sufficient. A variable observed before treatment can still be a collider, a selection variable, or a proxy for a conditioning event that changes the population being analyzed. For example, filtering on liquidity, analyst coverage, or fund flows may appear harmless because these variables are observed before the outcome horizon. Yet they can still create bias if they are common effects of prior performance, investor attention, and treatment exposure. The DAG remains the governing object.

This also clarifies the relationship to the simpler checks from *Section 7.5*. The timing placebo and shared-driver checks introduced there were informal, single-feature versions of the same logic. A feature that survives those checks has passed an initial plausibility

screen. The adjustment-set problem asks the harder multivariate question: whether the apparent effect survives conditioning on a causally admissible set of pre-treatment confounders.

In practice, the DAG should be written down before the estimator is chosen. Libraries such as `dowhy` make this discipline operational by requiring the researcher to specify the graph, declare the treatment and outcome, and identify the estimand implied by the assumptions. The software does not remove judgment; it exposes where judgment enters. If identification fails, the problem is not that the estimator is weak. The problem is that the research design has not justified a causal interpretation.

Once the DAG has been translated into an admissible adjustment set and an explicit estimand, the next question is whether adjustment on observed controls is credible. When important confounders may be unobserved, poorly measured, or impossible to condition on without introducing new bias, identification must come from a different source of variation.

Alternative identification designs

Backdoor adjustment identifies a causal effect by conditioning on observed confounders. When that strategy is not credible, the design problem changes. The question is no longer only which controls to include, but which source of variation can plausibly separate treatment from the factors that would otherwise confound the comparison.

Each alternative design uses a different comparison. Instrumental variables use variation in treatment induced by an external instrument. Difference-in-differences uses changes over time between treated and untreated groups. Regression discontinuity uses assignment rules near a threshold. Event-study counterfactual methods use pre-event dynamics and control series to estimate a no-event path. These designs differ in implementation, but they serve the same purpose: they replace broad unconfoundedness with a narrower assumption about the source of identifying variation.

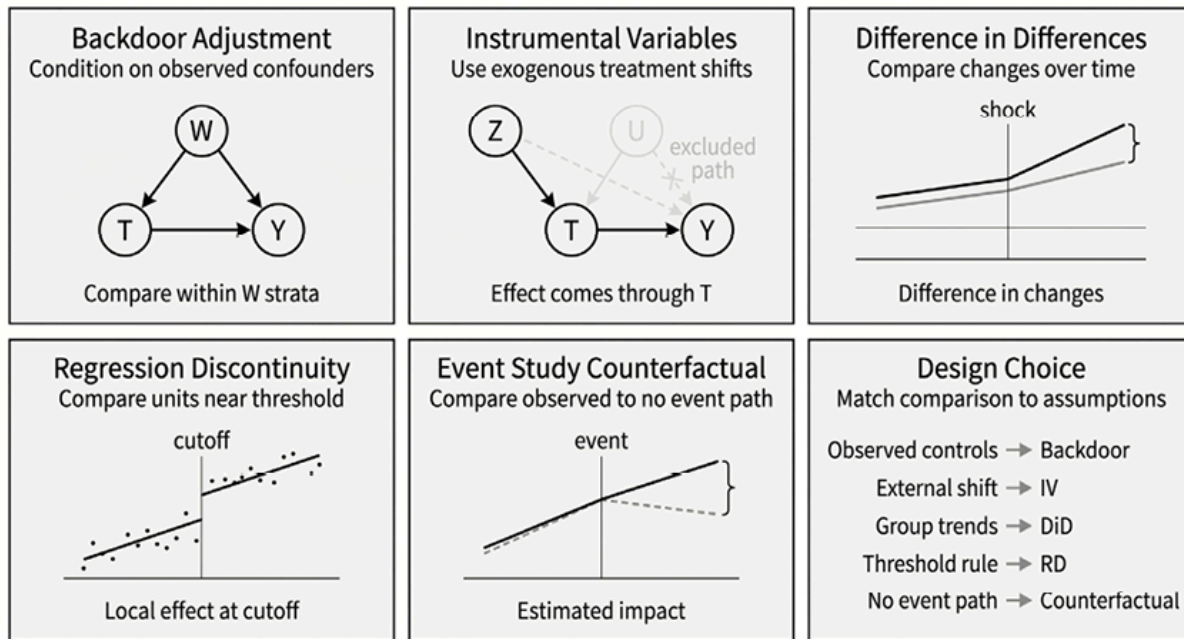


Figure 15.1: Alternative identification designs isolate causal effects using different comparisons

Instrumental variables

Instrumental variables use a variable Z that shifts treatment T but affects the outcome Y only through that treatment. The instrument must satisfy three conditions:

1. **Relevance** requires that Z meaningfully moves T .
2. **Independence** requires that Z is unrelated to the unobserved causes of Y .
3. **Exclusion** requires that Z has no direct path to Y except through T .

The exclusion restriction is usually the hardest condition to defend. A policy announcement, an index assignment rule, or a change in market structure may shift the treatment of interest, but it may also affect attention, liquidity, risk appetite, funding conditions, or other channels that directly influence the outcome. If those channels cannot be ruled out, the IV estimate should not be interpreted as the causal effect of T alone.

IV estimates also require careful interpretation. When treatment effects vary across units, the design often identifies a local effect for the units whose treatment status is shifted by the instrument, rather than an average effect for the entire population. This may still be the relevant estimand, but it should be stated explicitly. We do not implement IV estimation in this chapter; for applied work, the `linearmodels` library provides standard estimators.

Difference in differences

Difference in differences (DiD) identifies effects from differential changes over time. It compares treated and control units before and after an event, asking whether treated units changed more than comparable untreated units. The identifying assumption is that, absent treatment, the treated and control groups would have followed parallel trends.

A typical finance example is a short-selling ban that applies to one group of stocks but not to another. The relevant comparison is not simply whether treated stocks moved after the ban. The question is whether their returns, spreads, or volatility differed from those of a comparable control group exposed to the same broad market conditions.

The main threat is that treated and control assets may not share the same counterfactual trend. Sector shocks, time-varying betas, liquidity regimes, or macro news can all generate differential movements unrelated to the treatment. Pre-trend checks, narrow event windows, and economically justified controls can make the design more credible, but they do not prove parallel trends. In panel settings with staggered treatment timing, simple fixed-effects implementations also require care, as heterogeneous treatment effects can distort the resulting estimate.

Regression discontinuity

Regression discontinuity exploits treatment assignment rules with sharp thresholds. If units just above and just below a cutoff are otherwise comparable, the discontinuous change in treatment at the threshold can approximate local randomization.

In finance, thresholds arise in index inclusion rules, credit ratings, eligibility constraints, regulatory categories, and portfolio mandates. The identifying comparison is local: units close to the cutoff are compared with nearby units on the other side of the cutoff. The resulting estimate applies to the margin around the threshold, not necessarily to the full population.

The key assumption is continuity. In the absence of treatment, potential outcomes should evolve smoothly through the cutoff. Credibility is weakened if units can manipulate their position around the threshold, if other rules change at the same cutoff, or if too few observations lie near the boundary. RD can be highly persuasive when the assignment rule is transparent and difficult to manipulate, but its strength comes from its locality.

Event study counterfactuals

Event-study counterfactual methods identify effects by comparing the observed path after an event with an estimated no-event path. The source of the identified variation is event

timing, combined with the assumption that pre-event dynamics and unaffected control series can predict what would have happened without the event.

BSTS, discussed in *Section 15.5*, implements this logic with a Bayesian time-series model. It estimates a counterfactual path from pre-event behavior and contemporaneous controls, then attributes the gap between the observed and counterfactual paths to the event. The design is synthetic-control-like because it uses control series to model the untreated trajectory, but it remains a model-based counterfactual rather than a randomized experiment.

The main threat is contamination. Controls must help predict the treated series while remaining unaffected by the event. This is difficult for macro events because the same announcement may move nearly all plausible control assets. For example, an unexpected Fed decision may affect a bond ETF, related rates instruments, credit spreads, equity indices, and risk sentiment at the same time. In such settings, the estimate should be interpreted as model-dependent and supported with placebo dates, alternative control sets, and sensitivity checks.

Choosing among designs

The right design is the one whose identifying variation is most credible for the question. Backdoor adjustment is appropriate when the main confounders are observed and can be conditioned on without introducing post-treatment or selection bias. IV is useful when an external source shifts treatment but does not otherwise affect the outcome. Difference-in-differences is appropriate when treated and control units plausibly share a counterfactual trend. Regression discontinuity is strongest when a threshold creates locally comparable treated and untreated units. Event-study counterfactuals are useful when the no-event path can be modeled from stable pre-event relationships and uncontaminated controls.

No design is automatically superior. Each exchanges one set of assumptions for another. The research task is to choose the comparison whose assumptions are most defensible, state the corresponding estimand, and then test whether the resulting claim is fragile under plausible perturbations.

15.3 Validation and refutation

Identification follows from assumptions. Validation asks whether the resulting claim is fragile, contradicted by placebo evidence, or dependent on implausibly strong relationships with omitted confounders. These checks do not prove causality. They test whether the design behaves as it should if the identifying assumptions are credible.

A useful validation workflow separates three questions:

- First, does the method detect effects where none should exist? This is the role of placebo and negative-control tests.
- Second, how much omitted confounding would be required to overturn the result? This is the role of sensitivity analysis.
- Third, does the estimated effect persist across reasonable samples, specifications, and outcome definitions? This is the role of stability checks.

Placebo and negative control tests

Placebo tests ask whether the pipeline produces an effect when the treatment, timing, or outcome should not generate one. A credible design should estimate near-zero effects in these settings. If placebo effects are large or frequent, the model is likely capturing confounding, leakage, overfitting, or contaminated controls.

A **placebo treatment** replaces the actual treatment with a randomized, shifted, or otherwise irrelevant treatment assignment. For example, an event-study pipeline can be rerun by moving event dates to periods when no event occurred. A backdoor adjustment design can be tested using a treatment variable that should not affect the outcome once the same controls are applied. The goal is not to obtain exactly zero in finite samples, but to verify that the method does not systematically discover effects where the design says none should exist.

A **placebo date** is especially useful for event studies. The same counterfactual model is applied to dates without the event. If the model repeatedly finds large effects on non-event dates, the estimated event effect may reflect model instability or contaminated controls rather than the event itself.

A **negative control** is an outcome that should not respond to the treatment. If the treatment appears to affect such an outcome, the result points to residual confounding, timing leakage, or misspecified adjustment. Negative-control outcomes are especially useful when the proposed mechanism is specific. They test whether the estimated effect appears only where the mechanism predicts it should appear.

Sensitivity to omitted confounding

Adjustment only addresses confounders that are measured and included in an admissible way. Even after careful DAG design, unobserved confounding may remain. Sensitivity

analysis asks how strong an omitted confounder would need to be to materially change the conclusion.

A practical implementation perturbs the design by adding synthetic confounding, varying assumed confounder strength, or measuring how the estimate changes under alternative assumptions about hidden common causes. `Dowhy` supports refutation tests and sensitivity checks that probe this kind of robustness. These diagnostics do not prove that omitted confounding is absent. They quantify how dependent the conclusion is on the assumption that remaining confounding is limited.

The interpretation should be conservative. If a small omitted-confounder perturbation changes the sign, magnitude, or significance of the estimate, the result is not separable from the assumed confounder structure at the strengths tested. It should be treated as a hypothesis rather than as evidence for a causal effect. If the estimate remains stable under large and economically plausible perturbations, the claim becomes more credible, but still conditional on the design assumptions.

Stability across samples, specifications, and outcomes

Stability checks assess whether the estimated effect is specific to a single sample, model, or outcome definition. They are not substitutes for identification, but they reveal whether the claim depends on narrow modeling choices:

- **Subsample stability** checks whether the effect persists across time periods, regimes, assets, or market conditions. A funding-rate effect that appears only in one short volatility regime, for example, may still be real, but the estimand should then be described as regime-specific rather than general.
- **Specification stability** checks whether the effect survives reasonable alternative adjustment sets, nuisance-function models, or estimator choices. In DML, this includes varying the models used for the treatment and outcome nuisance functions. In event-study counterfactuals, it includes using alternative control series and model windows. A claim that depends on one narrow specification is weaker than a claim that survives a defensible range of specifications.
- **Outcome triangulation** assesses whether the pattern of effects aligns with the proposed mechanism. The same treatment need not affect every outcome in the same direction or with the same strength. A mechanism-near outcome may show a clearer effect than a broad trading outcome influenced by many other forces. The relevant question is whether the estimated effects appear where the mechanism

predicts they should appear, weaken where the mechanism is indirect, and disappear for outcomes that should not respond.

Taken together, placebo tests, sensitivity analysis, and stability checks define the evidentiary status of the result. A claim that passes these checks is not proven causal. It is credible under the stated assumptions and tested perturbations. A claim that fails them should not be discarded automatically; rather, it should be downgraded from a causal conclusion to a research hypothesis that requires a stronger design.



Implementation : `02_dowhy_causal_graph` walks through the full validation workflow (DAG specification, identification, estimation with temporal splits, placebo and refutation checks) for the crypto funding-rate two-outcome contrast (premium reversion against forward returns). We develop the quantitative diagnostics in the next section.

15.4 Isolating factor effects with DML

Double/Debiased Machine Learning (DML) bridges modern machine learning and causal inference (Chernozhukov et al., 2018). It estimates the causal effect of a factor while controlling for many potential confounders, using flexible ML models for the nuisance components and a debiased score for the target parameter.

The *identification checklist* for DML, based on *Section 15.2* , reads as follows:

- Treatment T timing is fixed; covariates X are strictly pre-treatment
- Estimand declared (ATE, ATT, or CATE)
- Unconfoundedness argued: X blocks all backdoor paths from T to Y
- Cross-fitting respects time ordering (expanding or rolling windows, not random folds)
- Sensitivity analysis probes robustness to omitted confounders

The intuition behind double machine learning

DML estimates a *low-dimensional set of causal parameters* while using flexible machine learning models to capture the high-dimensional nuisance relationships among covariates, treatment, and outcome (Chernozhukov et al., 2018).

In this chapter, the relevant setting is a *backdoor-style design* : after specifying treatment T , outcome Y , and an admissible pre-treatment adjustment set X (the covariate set W

from *Section 15.2*), DML estimates the effect of T on Y while allowing the relationships involving X to be nonlinear and high-dimensional.

In its most intuitive form, DML proceeds in three steps (see *Figure 15.2*):

1. **Predict the outcome from the controls.** Estimate the nuisance function $g(X) = \mathbb{E}[Y | X]$. This captures the part of the outcome predictable from the controls. The residual $\tilde{Y} = Y - \hat{g}(X)$ is the part of the outcome not explained by X .
2. **Predict the treatment from the controls.** Estimate the nuisance function $m(X) = \mathbb{E}[T | X]$. The residual $\tilde{T} = T - \hat{m}(X)$ is the part of the treatment not explained by X .
3. **Estimate the causal effect from the residualized variables.** Regress $\tilde{Y}$ on $\tilde{T}$. In the partially linear case, the slope from this final-stage regression is the DML estimate of the causal effect of T on Y .

This three-step workflow is the machine-learning analog of the **Frisch-Waugh-Lovell** logic from econometrics: remove the variation explained by admissible controls, then estimate the relationship in the remaining variation. It also generalizes the partial-IC idea from *Section 7.5* , in which known drivers were partialled out before examining the residual signal.

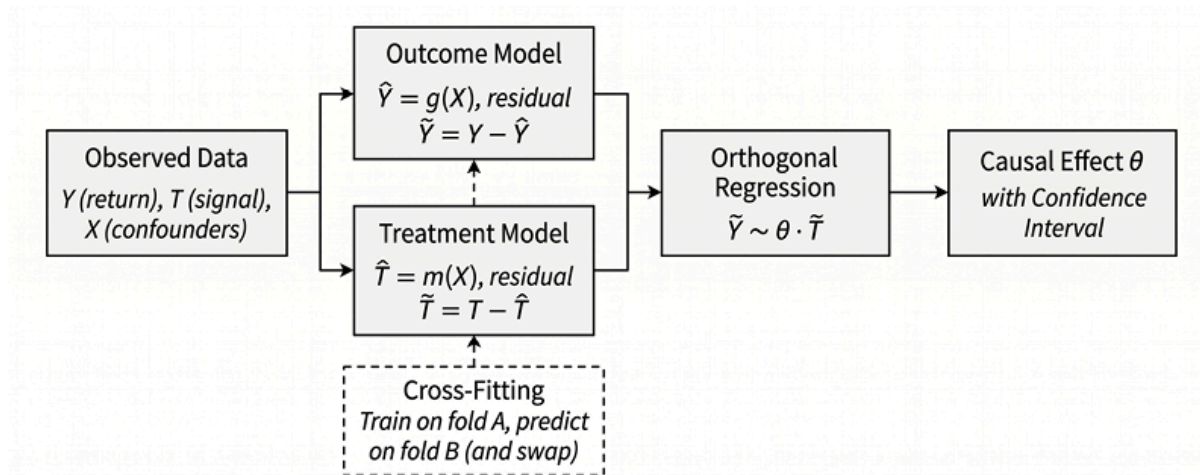


Figure 15.2: The Double ML workflow: orthogonalize both treatment and outcome against confounders, then regress the residuals

What makes DML statistically useful, however, is not residualization alone. The key idea is that DML uses an **orthogonal score** , meaning an estimating equation constructed so that small errors in the nuisance models have only a second-order effect on the target causal parameter. This makes it possible to use flexible machine learning models for the

nuisance functions without letting regularization bias dominate inference on the treatment effect (Chernozhukov et al., 2018).

DML also uses **cross-fitting**. Nuisance models are trained on one fold and evaluated on another, so the same observations are not used for both learning the nuisance structure and scoring the causal effect. This reduces overfitting bias in the final estimate. For time-series data, cross-fitting must respect temporal ordering: use expanding or rolling windows rather than random folds, and include a temporal gap when needed to avoid leakage from overlapping forward returns.

This point is important for interpretation. The three-step residualization picture is the clearest intuition for the *partially linear ATE-style setup* used in this section. More general DML-style estimators extend the same logic to treatment effects on the treated and to heterogeneous treatment effects, but they do so using different orthogonal scores and specialized learners rather than a single generic regression recipe.

DML, therefore, depends on two ingredients:

1. **Credible identification** : the covariates (X) must form an admissible adjustment set under the causal design described in *Section 15.2*.
2. **Adequate nuisance estimation** : if the nuisance models (the ML models used in steps 1 and 2) are too weak, unstable, or poorly tuned, the resulting causal estimate may be noisy or fragile even when the design itself is sound.

For this reason, nuisance-model sensitivity is part of the empirical diagnosis: poorly specified nuisance models produce biased causal estimates. Mitigate nuisance-model dependence by using flexible learners (gradient boosting, random forests), tuning via cross-validation, and comparing estimates across at least two different model classes.

Large changes in the estimated effect across reasonable nuisance learners should be treated as evidence that the causal conclusion is unstable (Chernozhukov et al., 2018). The ETF momentum analysis in `03_econml_dml` illustrates this: the DML ATE ranges from -0.055 (linear nuisance) to -0.060 (shallow gradient boosting) to -0.088 (deep gradient boosting), a roughly $1.6 \times$ spread that reflects genuine estimation uncertainty.

Beyond average effects, frameworks such as `EconML` support **heterogeneous treatment effects** : estimating how causal impacts vary across conditions (for example, whether momentum works differently in high- and low-volatility regimes). `EconML` provides metalearners and causal forests for this purpose.

A closely related application is *post-double-selection LASSO* for factor-zoo validation. Adapting Belloni, Chernozhukov, and Hansen (2014) following Feng, Giglio, and Xiu

(2020), one tests whether candidate factors retain marginal pricing power after controlling for the existing factor zoo. Among the four managed-portfolio factors examined in `11_factor_zoo_validation`, only MeanRev clears the naive significance bar at $t \approx +2.6$; that signal collapses to insignificance once the first ten principal components of the asset universe are included as controls: the BCH/FGX correction substantively changes the inferential conclusion.



Implementation :

- `03_econml_dml` : DML for ETF momentum, manual and EconML implementations; nuisance-model sensitivity sweep across linear and gradient-boosting specifications; block-permutation refutation.
- `11_factor_zoo_validation` : post-double-selection LASSO applied to four managed-portfolio factors, with principal-component controls.

Measuring the causal impact of momentum

We apply DML to the ETF Rotational Momentum strategy to estimate the causal effect of momentum on forward returns, controlling for volatility and market regime, two confounders that affect both the treatment and the outcome.

The causal framework is:

- **Treatment (T)** : Momentum score (risk-adjusted 6-month return)
- **Outcome (Y)** : 21-day forward return
- **Confounders (X)** : Realized volatility, yield curve regime

The naive approach (regressing returns on momentum) produces a biased estimate because volatility and regime affect both treatment and outcome.

Applied to ETF momentum data in `03_econml_dml` under a strict temporal train/test split, the naive OLS slope is -0.0385 with HAC-corrected 95% confidence interval $[-0.0484, -0.0285]$, a negative relationship consistent with reversal at the 21-day horizon. The HAC bandwidth matches the 21-day forward outcome, since overlapping returns induce autocorrelation that a shorter bandwidth would understate. After orthogonalization, the EconML DML estimate is -0.0537 with 95% confidence interval $[-0.0613, -0.0462]$, which excludes zero; a manual DML implementation with walk-forward cross-fitting agrees at -0.0494 . The causal effect of momentum on forward

returns becomes *more* negative once volatility and regime confounders are controlled for. The naive slope understates the DML magnitude by roughly 28% , so naive factor research misses part of the effect rather than overstating it. The confounders enter the estimator as controls used for residualization, not as effect modifiers, the distinction that separates a single average effect from a model of how the effect varies.

On crypto perpetual funding rates, the picture is reversed: in `04_dml_crypto_regime` , the naive and DML estimates keep the same sign, but confounders suppress roughly 90 percent of the effect magnitude, so the naive estimate captures only a fraction of the DML-estimated signal. A block-permutation test that shuffles treatment in seven-day blocks passes ($z = -4.1$), confirming the effect is distinguishable from placebo; shortening the blocks to one day inflates the statistic to $z = -6.2$, an illustration that too-short blocks understate the autocorrelation null. Neither estimate is statistically significant, however (naive $t = -0.44$, DML $t = -1.23$): even DML cannot guarantee robustness when the underlying signal is weak.

Confounders distort the naive estimate through two channels: high volatility weakens momentum signals and increases return dispersion, and regime transitions affect both the treatment and the outcome. In the crypto configuration, regime-stratified estimates remain directionally similar, and a single-model interaction test finds the high-vol versus low-vol difference statistically insignificant ($t = -0.63$, $p = 0.53$), so the dominant lesson is that the effect magnitude is confounded rather than that regime heterogeneity is strong.

When evaluating any factor, the workflow proceeds as follows:

1. Identify potential confounders based on domain knowledge.
2. Apply DML to estimate the causal effect after controlling for confounders.
3. Compare to the naive estimate to quantify confounding bias.
4. Examine confidence intervals to assess statistical certainty.
5. Test robustness using refutation tests (placebo treatments, random common causes, data subsets) and block-permutation tests.

A narrow confidence interval that excludes zero, combined with stable estimates in sensitivity analyses, provides stronger evidence of factor robustness than correlation-based approaches. Robust estimates change minimally when synthetic confounders are added or when the estimator is run on data subsets; fragile estimates flip sign under small perturbations. This qualitative difference determines which causal claims can support trading decisions.

Outcome choice shapes causal credibility

A key lesson from the crypto funding-rate analysis is that causal credibility depends not only on the treatment and estimator, but also on the outcome being analyzed. The same treatment can support a more or less plausible causal interpretation depending on whether the outcome is tightly connected to the proposed mechanism or exposed to many broader market forces.

We define the treatment as episodes of extremely high funding-rate premiums: observations where the funding-rate z-score exceeds 2. We then compare two outcomes: subsequent forward returns and subsequent premium reversion. These are related but distinct causal questions. Forward returns are a broad market outcome. Extreme funding rates may contain information about future returns, but the relationship is exposed to common drivers such as speculative demand, leverage, volatility, liquidity conditions, and market-wide risk appetite. These forces may influence both the treatment and the outcome, making the causal interpretation fragile.

Premium reversion is more closely tied to the institutional mechanism of perpetual futures. Funding is designed to help anchor the perpetual contract price to the spot price. When the perpetual trades at a large premium, positive funding makes long positions more expensive and short positions more attractive. This creates an incentive, subject to trading costs, liquidity, and limits to arbitrage, to sell the perpetual contract and buy the spot asset. The proposed treatment-outcome link is therefore narrower and easier to connect to an explicit market mechanism.

This does not mean that premium reversion is automatically causal. Because the treatment is defined by an extreme value of the same funding-rate process, subsequent reversion may partly reflect ordinary mean reversion after an extreme observation. The stronger claim is therefore not that premium reversion proves an arbitrage effect, but that it is a more mechanism-consistent outcome than broad forward returns.

Dowhy's *refutation checks* help summarize this distinction. Here, we report two diagnostics derived from the refutation results:

1. **Out-of-sample drift** (OOS) measures how much the estimated effect changes when the model is refitted or evaluated across alternative sample splits; larger drift indicates lower stability.
2. **Placebo ratio** reports the share of placebo refutations in which the placebo estimate is large enough to challenge the original effect; a higher ratio suggests that comparable effects are easier to reproduce under falsified conditions.

These diagnostics do not establish identification and do not rule out omitted confounding. They provide narrower evidence: whether the estimate remains stable across resamples and whether similar effects appear under placebo treatments or outcomes.

Outcome	Mechanism	OOS Drift	Placebo Ratio
Forward returns	Broad market outcome exposed to sentiment, positioning, and risk appetite	85%	12.7%
Premium reversion	Mechanism-adjacent outcome linked to funding incentives and arbitrage pressure	51%	5.7%

Table 15.3: Refutation diagnostics for two outcome choices

The results are consistent with this interpretation. The forward-return estimate is substantially more fragile: its estimated effect changes more across out-of-sample configurations, and placebo tests more often produce effects resembling the original estimate. This is unsurprising for a broad market outcome. Extreme funding rates may predict returns, but the same latent forces that produce extreme funding may also drive subsequent returns. Unless these forces are adequately measured and adjusted for, the estimated effect remains vulnerable to confounding.

The premium-reversion estimate is less fragile, but not conclusive. Its lower OOS drift and placebo ratio suggest that the result is more stable and less easily reproduced under falsified conditions. This is consistent with the proposed mechanism: high funding premiums alter the relative attractiveness of long and short positions in the perpetual market and can create pressure for the premium to compress. At the same time, the remaining 51% drift is not small, and the outcome is mechanically close to the treatment definition. The evidence, therefore, supports a *cautious conclusion*: premium reversion is a more credible outcome for this causal question than forward returns, but the refutation checks do not by themselves prove a causal arbitrage effect.

The broader methodological lesson is that causal estimation should begin with the treatment-outcome pair, not the estimator. Before applying DML or any other causal method, ask whether the outcome is connected to a plausible, testable mechanism. A narrow, mechanism-consistent outcome is not automatically valid, and a broad market outcome is not automatically invalid. But broad outcomes usually require stronger assumptions, richer controls, and more careful sensitivity analysis. A simple estimator applied to a well-chosen outcome can support a more credible claim than a sophisticated estimator applied to an outcome only weakly connected to the treatment.



Implementation : `@2_dowhy_causal_graph` shows the full two-outcome comparison, including sensitivity and placebo-refutation tests.

Regime-conditional position sizing - A case study

The preceding sections establish DML methodology. The case study tests whether the estimated causal effects translate into improved position sizing on out-of-sample data; see `@5_momentum_causal_trading` for implementation details.

ETF rotational momentum (buying recent winners and selling losers) has historically generated positive returns. Standard implementations apply uniform position sizing regardless of regime. The CATE estimates from DML offer a candidate alternative: scale exposure by the regime-conditional causal strength of the signal.

To avoid in-sample overfitting, we implement a strict *temporal split* :

- Training period (2007-01 to 2022-11): Estimate all causal effects, learn regime scaling
- Test period (2022-12 to 2025-12): Apply learned scaling to held-out data
- No leakage: Regime thresholds, CATE estimates, and position rules are fixed before the test period begins

`CausalForestDML` from `EconML` is fit on training data with market volatility and the regime indicators as effect modifiers (EconML's `x` argument) and the volatility-horizon panel plus yield-curve slope as controls (EconML's `w` argument, which together with the effect modifiers form the adjustment set), using the same walk-forward folds as the `LinearDML` stage. The CATE varies in *sign* across regimes: $+0.019$ in low volatility, $+0.043$ in mid volatility, and -0.066 in high volatility. The ATE is -0.0097 with 95% CI $[-0.0138, -0.0055]$, against a naive OLS coefficient of -0.0037 , a 61% bias once the controls are absorbed.

The sizing rule applies a regime multiplier to a gross-normalized portfolio weight. Let

$w_{i,t}^{\text{base}}$ denote the long-short quintile weight for asset i at date t , rescaled so

$\sum_i |w_{i,t}^{\text{base}}| = 1$. The regime-scaled weight is:

$$w_{i,t} = w_{i,t}^{\text{base}} s(r_t)$$

where $s(r_t)$ uses the signal-to-noise ratio CATE/std within each regime, clipped to $[0.5, 1.5]$ so that imprecise estimates shrink toward neutral exposure. The resulting factors are **0.50** in high volatility, **1.09** in low volatility, and **1.28** in mid volatility: exposure is cut by half where the effect reverses sign and rises where the signal is both strong and precisely estimated.

The companion notebook compares three strategies during the test period:

1. **Naive** : Uniform momentum, no regime adjustment.
2. **Causal-Informed** : Scale by training-period CATE estimates, shrunk by their estimation uncertainty.
3. **Simple Heuristic** : Reduce exposure in high-vol (no causal machinery).

Results include transaction costs (10 basis points), with a sweep over the 5–20 basis-point range. Sharpe ratios are positive in training (Naive **+0.12** , Causal **+0.23** , Heuristic **+0.20**) and negative in holdout (Naive **−0.18** , Causal **−0.05** , Heuristic **−0.11**). Causal is the best of the three in holdout, but does not deliver positive risk-adjusted returns:

- Causal scaling reduces exposure in high volatility, where the training-period CATE was negative, and increases it in mid volatility, where the effect was largest and most precisely estimated. That heterogeneity partly persists, which is why the causal strategy degrades least.
- Momentum IC drops from **+0.023** in training to **−0.053** in holdout: the base signal inverts post-2022. No sizing rule rescues a treatment whose relationship to forward returns flips sign; the negative holdout Sharpes reflect that signal failure rather than a failure of the causal machinery.

Causal analysis reduces out-of-sample degradation when training-period heterogeneity carries information; it does not guarantee alpha when the underlying signal breaks down.

All estimates use only training data; the test period remains untouched. Cross-fitting uses walk-forward folds within training (no random folds). Regime classification uses pre-treatment volatility. Transaction costs are included. Results are sample-dependent and may not generalize.

The workflow differs from correlation-based analysis in two ways. First, CATE estimates measure the causal strength of momentum after controlling for the volatility and regime confounders, rather than the unconditional statistical correlation. Second, the test period is held out before any estimation choice is made, so the reported Sharpe ratios reflect post-decision behavior rather than in-sample fit.

DML handles continuous treatment effects: how a *dose* of factor exposure causally affects returns. Many financial questions instead involve discrete events: a specific Fed announcement, a regulatory change, a CEO departure. These require explicit counterfactual construction rather than averaging across treatment intensities, which we will discuss next.

15.5 Measuring event impact with Bayesian structural time-series

While DML addresses continuous treatment effects, many financial questions involve discrete events: Did the Fed's surprise rate hike affect bond prices? What was the impact of a CEO's departure on the stock? Bayesian Structural Time-Series (BSTS) models provide a rigorous framework for answering such questions by constructing data-driven counterfactuals (Brodersen et al., 2015).

The identification checklist for BSTS, applying *Section 15.2* , is:

- Estimand is the post-period counterfactual difference (ATT on the treated series)
- Control series selection rule defined; spillover risk assessed
- Pre-period relationship stability verified via fit diagnostics
- Placebo dates tested to validate model specification
- Robustness checked across alternative control sets

The intuition behind BSTS

BSTS constructs a counterfactual: a model of what the target series would have done in the absence of the event. The difference between the observed series and this “synthetic twin” is the estimated causal impact, illustrated by *Figure 15.3* .

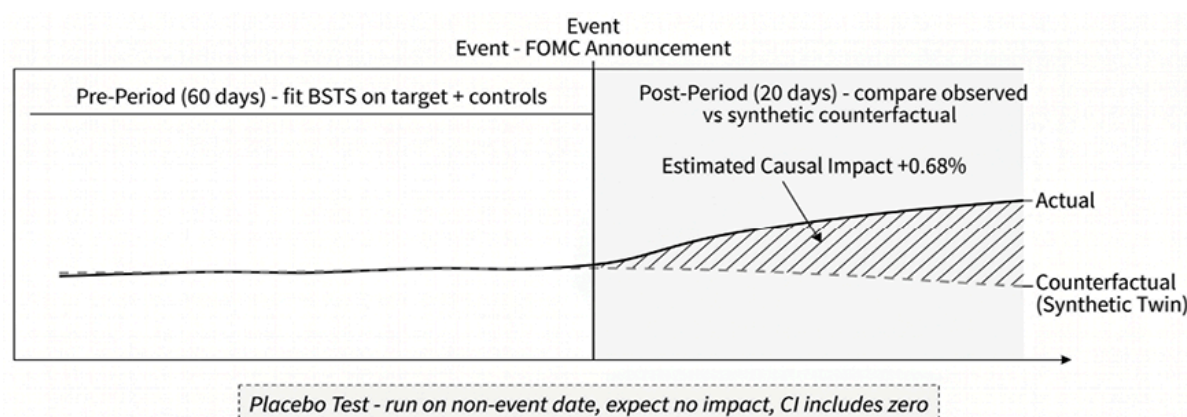


Figure 15.3: BSTS constructs a counterfactual from pre-event control relationships, then measures the gap between observed and projected series as the causal impact

The workflow involves three steps:

1. Learn target-control relationships in the pre-event period.
2. Project the counterfactual “business as usual” path using post-event control movements.
3. Take the difference between actual and counterfactual as the causal impact.

Bayesian methods provide posterior distributions with credible intervals. Unlike traditional event studies, which assume constant expected returns, BSTS adapts to changing market conditions through control series.

Two assumptions underlie BSTS event studies:

- **Stable relationships** : The target-control relationship learned in the pre-period must hold in the post-period, absent the event.
- **No spillover** : The event must not affect the control series. If a Federal Reserve Open Market Committee (FOMC) monetary policy announcement moves both IEF and SPY, using SPY as a control corrupts the counterfactual.

Spillover is the most common failure mode in financial event studies. Mitigate it by choosing controls from asset classes with weak fundamental linkages to the event (for example, agricultural commodities for banking regulation). A useful diagnostic is to run BSTS on an asset the analyst believes is unaffected: if it shows a “significant” effect, the control group is contaminated.

`06_fed_announcement_bsts` automates this by running BSTS with each control as the target; any control showing significant impact is flagged for exclusion. For macro events where truly unaffected controls may not exist, rely on placebo-date validation and robustness across alternative control sets rather than assuming the problem away.

These assumptions cannot be tested directly: they must be justified by domain knowledge and validated indirectly through placebo tests.

The original Google R package (Brodersen et al., 2015) uses Bayesian structural time-series. We use `tfcausalimpact`, the actively maintained TensorFlow-Probability port that computes posterior credible intervals for the cumulative event effect.

The impact of a Fed announcement on bond ETFs

We apply BSTS to measure the impact of FOMC announcements on the IEF Treasury bond ETF.

The event-study setup is:

- **Target** : IEF (7-10 Year Treasury Bond ETF)
- **Controls** : VEA (Developed Markets), EFA (EAFE), DBC (Commodities)
- **Event** : March 2023 FOMC announcement (25bp hike amid banking stress)
- **Pre-period** : 60 trading days before announcement
- **Post-period** : 20 trading days after announcement

The controls are correlated with bond prices through macro factors, but should not be directly affected by the FOMC decision.

Illustrative results

BSTS analysis of four FOMC announcements (March 2023, July 2023, November 2023, March 2024) in `06_fed_announcement_bsts` runs on log-price indexes anchored to the start of each window. Hence, the cumulative effect reads as a cumulative abnormal log return rather than a dollar quantity that depends on the ETF's price level. The estimated effects on IEF carry signs consistent with the macro context:

- The March 2023 announcement (a 25bp hike amid banking stress) produced a positive cumulative effect of $+0.59$ over the post-window, consistent with flight-to-quality demand for Treasuries.
- The July 2023 announcement (a 25bp hike in calmer conditions) produced a cumulative effect of -0.28 : the same policy action, an opposite market response.
- The November 2023 hold and the March 2024 hold-with-revised-projections both produced negative cumulative effects (-0.09 and -0.29).

All four events have 95% credible intervals on the cumulative effect that exclude zero. Taken at face value, that would mean every announcement had a significant impact on IEF. The validation tests below show why the model in its current form does not support that reading.

Placebo test validation

We re-run the same BSTS specification on twelve trading days with no Fed announcement. A well-specified model should flag very few of these as “significant.” In the companion notebook, seven of twelve placebo dates yield credible intervals that exclude zero, corresponding to a *placebo false-positive rate of 58%* . With only twelve

placebo dates, this is a coarse estimate, and these dates are not screened against other macro releases such as CPI prints, payrolls, or Treasury refunding announcements, so the rate is illustrative rather than precise. Even so, the magnitude is large enough to indicate that the model finds “significant” effects on arbitrary non-event dates at over half the rate it finds them on FOMC dates. Under that condition, the apparent significance of the four FOMC events cannot be attributed to the announcements themselves.

The control-as-target spillover check uses BSTS to ask whether each control series shows a significant response on each FOMC date. Running the check per event, rather than assuming the verdict from a single date transfer, shows the contamination is pervasive: all three control ETFs (VEA, EFA, DBC) respond on the March 2023, November 2023, and March 2024 dates, and DBC alone responds in July 2023. A control is flagged solely when its own post-period cumulative-effect credible interval excludes zero. For macro events such as FOMC announcements, this is not surprising (the same policy shock propagates across global equities, developed-market bonds, and commodities through the dollar and global risk-appetite channels). Still, it means the counterfactual constructed from these controls absorbs the announcement effect rather than isolating it. The control set is the plausible choice, not an unaffected one; a more conservative design would use duration-matched non-US sovereign bonds or a local-level model with no cross-asset controls.

Taken together, the validation tests indicate that BSTS on log-price indexes with global cross-asset controls is not a credible event-study design for FOMC announcements. The four reported event impacts should not be read as each Fed announcement moving IEF by a known cumulative amount; the placebo and spillover checks reject the design before any individual finding can be taken seriously.

Posterior probability versus credible intervals

A common source of confusion is how different libraries report uncertainty. A true **posterior probability of a causal effect** is the integral of the posterior distribution above or below zero, a Bayesian tail probability over the parameter, not over time. The fraction of point-effect rows whose mean is positive is not such a probability; it is a sample frequency over the post-period that can be high or low for reasons unrelated to whether the effect is real. The reliable diagnostic is whether the **credible interval for the cumulative effect** excludes zero, supported by placebo tests with a low false-positive rate. When the placebo rate is high, neither the credible interval nor any derived sign frequency is trustworthy.